

Security Middleware (SMW) (release 5.5)

Rev. 1 — 8 September 2026

User manual

Document information

Information	Content
Keywords	UM12513 SMW
Abstract	User Manual of the Security Middleware library interfacing with NXP [®] Secure Subsystems (OPTEE, ELE, SECO)



1 Introduction

The Security Middleware library provides an application's generic API to perform Security operations supported by the Secure Subsystems present in the system.

The SMW Library is designed to be OS-agnostic through the use of the [OSAL](#) module, which abstracts OS-specific operations.

The library configuration and Secure Subsystems configuration are detailed in the [Configuration](#) chapter.

The [Secure Subsystems Features and Capabilities](#) chapter describes the operations supported by each Secure Subsystem and their specific features, capabilities and limitations.

The [APIs Reference](#) chapter provides the complete APIs documentation.

2 Library Configuration

The SMW Library configuration is split into two parts:

- [Library Configuration](#): Defines how library is initialized to run in the OS environment.
- [Subsystems Configuration](#): Defines the mapping between Secure Subsystems and their supported operations.

2.1 Library Configuration

The SMW Library allows user to configure the Secure Subsystems storage. This includes the storage identification for the Secure Enclave (SECO or ELE) and the Trusted Application (TA) identification for the TEE Secure Subsystem. These settings are OS-dependent.

Hence it's handled by the [OSAL](#) module.

2.2 Subsystems Configuration

The SMW Library allows user to configure the Secure Subsystems and their supported operations through a subsystem configuration file. The configuration is a structured text format buffer that defines the mapping between subsystems and their supported operations. This configuration is loaded by the [smw_config_load\(\)](#) function. This configuration part is detailed in the following sections.

Note:

In the context of the SMW Library running on the Linux OS, the configuration is a text file loaded by the OSAL module and passed to the [smw_config_load\(\)](#) function (see [OSAL](#) chapter).

The subsystem configuration definition must respect [Syntactic rules](#) and writing structure to properly define the capabilities and constraints of each operation across different Secure Subsystems.

The subsystem configuration definition is divided in sections:

- [Global configuration](#): Defines the parser version and other settings.
- [Secure Subsystems definition](#): Defines each Secure Subsystem name and their loading/unloading method.
- [Security Operations definition](#): Defines for each Secure Subsystem the supported operations and their capabilities and constraints.

2.2.1 Syntactic rules

- Characters must be encoded in ASCII.
- Semicolon ';' specifies end of line.
- Colon ':' is a separator of multiples entries.
- Spaces and line separators are ignored.
- Decimal numbers are written using the US/UK format (i.e. separator is '.').
- Negative numbers are preceded by '-'.
- Range value is defined using the format '**min:max**'.
- Range value defined using the format '**min:**', means a range from min to maximum unsigned integer value.
- Range value defined using the format '**val:val**', means a single value (min equals max).
- String must not be quoted.
- Commented sections start with '/' and finish with '/'. Comments are ignored during parsing.

2.2.2 Global configuration

```
VERSION=<integer: parser version must be 1>;
PSA_DEFAULT=<string: subsystem name>[:ALT];
[SECURE_SUBSYSTEM]
...
```

The following rules apply to the global configuration:

- The first tag to define is the **VERSION** tag specifying the parser version compatibility. Supported version is **1**.
- The tag **PSA_DEFAULT**, if present, must be after the tag **VERSION**. This tag defines which Secure Subsystem is targeted to execute the PSA operation. If present after the first occurrence of **[SECURE_SUBSYSTEM]**, it is ignored. The possible values are the Secure Subsystems names listed in [Secure Subsystems](#) table. Adding option **ALT** (":ALT") after the Secure Subsystem name allows the selection of another Secure Subsystem if the default one doesn't support the requested Security Operation.

2.2.3 Secure Subsystems definition

```
[SECURE_SUBSYSTEM] /* First part of the 1st Subsystem */
  <string: name of subsystem>;
  <string: load/unload method>;
  [SECURITY_OPERATION]
  ...
  [SECURITY_OPERATION]
  ...
[SECURE_SUBSYSTEM] /* 2nd Subsystem definition */
  <string: name of subsystem>;
  <string: load/unload method>;
  [SECURITY_OPERATION]
  ...
  [SECURITY_OPERATION]
  ...
[SECURE_SUBSYSTEM] /* Second and last part of the 1st Subsystem */
  <string: name of subsystem>;
  [SECURITY_OPERATION]
  ...
```

The following rules apply to the Secure Subsystem definition:

- The tag **[SECURE_SUBSYSTEM]** defines a Secure Subsystem configuration block.
- Each **[SECURE_SUBSYSTEM]** block must start with the subsystem name identifier matching the names defined in the Secure Subsystems name [Secure Subsystems](#) table.
- An optional load/unload method can be specified within the block after the subsystem name using the format **<string: load/unload method>**. The supported methods are listed in [Secure Subsystem - load/unload methods](#) table. This method must be defined only once per **[SECURE_SUBSYSTEM]** block.
- A **[SECURE_SUBSYSTEM]** configuration can be split into multiple parts. Each part must start with the same subsystem name identifier.
- A **[SECURE_SUBSYSTEM]** block can contain one or more secure operation definition block **[SECURITY_OPERATION]**.

Note:

- *The configuration definition order is important. It defines the priority of the Secure Subsystems versus Security Operation when the library must select by itself which Secure Subsystems to be used to execute the Security Operation (when Subsystem Name is not specified in the API argument). The first **[SECURE_SUBSYSTEM]** block has the highest priority and the last one has the lowest priority.*

- *Secure Subsystem is implicit when an operation uses a key identifier as key is owned by the Secure Subsystem.*

2.2.4 Security Operations definition

```
[SECURITY_OPERATION]
<string: name of operation>;
/* A combination of the lines below describes */
/* the Secure Subsystem capabilities for this Security Operation. */
USE_ELA; /* Optional: Enable ELA */
<param1>_VALUES=<value1>:<value2>:<value3>;
<param2>_SIZE_RANGE=<integer: min>:<integer: max>;
<param3>_SIZE_RANGE=<integer: max>; /* threshold lower than */
<param4>_SIZE_RANGE=<integer: min>; /* threshold greater than */
```

The following rules apply to the Security Operation definition:

- The tag **[SECURITY_OPERATION]** defines a Security Operation configuration block within a **[SECURE_SUBSYSTEM]** block.
- Each **[SECURITY_OPERATION]** block must start with the operation name as matching the names defined in the *Security Operations*.
- Each **[SECURITY_OPERATION]** block must contain one operation name identifier.
- A Security Operation name must not be duplicated within the same Secure Subsystem configuration, even if Subsystem configuration is split across multiple **[SECURE_SUBSYSTEM]** blocks.
- The **USE_ELA** flag is optional and enables hardware EdgeLock Accelerator (ELA) for cryptographic operations (cipher, AEAD, hash and HMAC) when using plaintext keys. See [USE_ELA](#) for details.
- The Security Operation can define its capabilities values (e.g. key types, hash algorithms...) using tags **<param#>_VALUES** (as describes in [Capabilities tags](#)). Each value is a non-quoted string separated by a colon ':'.
 • The Security Operation can define its capabilities range using tags **<param#>_SIZE_RANGE**. Range values are integer defining minimum and/or maximum capability value.

2.2.5 Naming Convention

2.2.5.1 Secure Subsystems

The [Secure Subsystems](#) table below lists all Secure Subsystems supported by the Security Middleware library.

Table 1. Secure Subsystems

Name	Description
SECO	Use the SECO Secure Enclave on device type i.MX8qxp.
TEE	Use the Secure OS called OPTEE and running in ARM Trustzone Secure world.
ELE	Use the ELE Secure Enclave (EdgeLock Enclave) on devices type: • i.MX8ULP • i.MX9x

2.2.5.2 Subsystem load/unload methods

The following [Secure Subsystem - load/unload methods](#) table defines the possible string value of the load/unload method.

Table 2. Secure Subsystem - load/unload methods

Name	Description
AT_FIRST_CALL_LOAD	At first Secure Subsystem call, the Secure Subsystem is loaded. The Secure Subsystem is unloaded when the configuration is unloaded. It's the default method if no specified.
AT_CONTEXT_CREATION_DESTRUCTION	At Secure Subsystem context creation, the Secure Subsystem is loaded. It is unloaded when the Secure Subsystem operation is finish. In case multi-part operation, when the final step is performed or when the cancel context operation is called.

2.2.5.3 Security Operations

This section defines the Security Operations organized by functional category:

- Key Management Operations, [Key management](#).
- Cryptographic Operations, [Cryptographic operations](#)
- Data Management Operations, [Data management](#).
- Device Management Operations, [Device management](#).
- Operation's capabilities tags, [Capabilities tags](#).

2.2.5.3.1 Key management

Following [Security Operations - key management](#) table below lists all key management security operations.

Table 3. Security Operations - key management

Name	Description	Capabilities Tags
GENERATE_KEY	Generate a cryptographic key (private, keypair). Public key can be exported.	KEY_TYPE_VALUES
DERIVE_KEY	Derive a key from an existing cryptographic key.	KEY_TYPE_VALUES OP_TYPE_VALUES
IMPORT_KEY	Import cryptographic key (public, private, keypair).	KEY_TYPE_VALUES
EXPORT_KEY	Export cryptographic key. Private key exportation is function of the Secure Subsystem capabilities. Public key are always exportable.	KEY_TYPE_VALUES
DELETE_KEY	Delete a key.	KEY_TYPE_VALUES

2.2.5.3.2 Cryptographic operations

Following [Security Operations - Cryptographic operations](#) table below lists all cryptographic security operations.

Table 4. Security Operations - Cryptographic operations

Name	Description	Capabilities Tags
HASH	Oneshot message digest.	USE_ELA HASH_ALGO_VALUES
HASH_MULTI_PART	Multipart message digest.	HASH_ALGO_VALUES
MAC	Message Authentication Code.	USE_ELA

Table 4. Security Operations - Cryptographic operations...continued

Name	Description	Capabilities Tags
		MAC_ALGO_VALUES MAC_ALGO_VALUES HASH_ALGO_VALUES
SIGN	Oneshot asymmetric signature generation.	SIGN_ALGO_VALUES SIGN_TYPE_VALUES HASH_ALGO_VALUES
VERIFY	Oneshot asymmetric signature verification.	SIGN_ALGO_VALUES SIGN_TYPE_VALUES HASH_ALGO_VALUES
SIGN_MULTI_PART	Multipart asymmetric signature generation.	SIGN_ALGO_VALUES SIGN_TYPE_VALUES HASH_ALGO_VALUES
VERIFY_MULTI_PART	Multipart asymmetric signature verification.	SIGN_ALGO_VALUES SIGN_TYPE_VALUES HASH_ALGO_VALUES
CIPHER	Oneshot cipher encryption and decryption.	USE_ELA KEY_TYPE_VALUES MODE_VALUES OP_TYPE_VALUES
CIPHER_MULTI_PART	Multipart cipher encryption and decryption.	KEY_TYPE_VALUES MODE_VALUES OP_TYPE_VALUES
AEAD	Oneshot authentication encryption.	USE_ELA KEY_TYPE_VALUES MODE_VALUES OP_TYPE_VALUES
AEAD_MULTI_PART	Multipart authentication encryption.	KEY_TYPE_VALUES MODE_VALUES OP_TYPE_VALUES
ASYMM_ENCRYPT	Oneshot asymmetric encryption.	ENC_ALGO_VALUES MODE_VALUES HASH_ALGO_VALUES
ASYMM_DECRYPT	Oneshot asymmetric decryption.	ENC_ALGO_VALUES MODE_VALUES HASH_ALGO_VALUES
RNG	Generate a Random data number.	RNG_SIZE_RANGE

2.2.5.3.3 Data management

Following [Security Operations - Data management](#) table below lists all data management security operations.

Table 5. Security Operations - Data management

Name	Description	Capabilities Tags
STORAGE_STORE	Store data in secure storage.	KEY_TYPE_VALUES MODE_VALUES

Table 5. Security Operations - Data management...continued

Name	Description	Capabilities Tags
		HASH_ALGO_VALUES MAC_ALGO_VALUES
STORAGE_RETRIEVE	Retrieve data from secure storage.	N/A
STORAGE_DELETE	Delete data from secure storage.	N/A

2.2.5.3.4 Device management

Following [Security Operations - Device management](#) table below lists all device management security operations.

Table 6. Security Operations - Device management

Name	Description	Capabilities Tags
DEVICE_ATTESTATION	Device attestation certificate.	N/A
DEVICE_LIFECYCLE	Set and Get device lifecycle.	N/A
DEVICE_REPROVISION	Device reprovisioning	N/A

2.2.5.3.5 Capabilities tags

2.2.5.3.5.1 KEY_TYPE_VALUES

Following [Security Operation - Key type values](#) table below lists all supported key type values.

To each key type value, a range tag is associated as shown in the table below. The key range tag is used to define the minimum and/or maximum key security size in bits. The key range tag is optional and when not specified, the key size is not restricted.

The key range definition must respect the range format explained in the [Syntactic rules](#) section above.

Table 7. Security Operation - Key type values

Name	Description	Range Tag
SECP_R1	SECP R1 Elliptic Curve key.	SECP_R1_SIZE_RANGE
BRAINPOOL_R1	Brainpool R1 Elliptic Curve key.	BRAINPOOL_R1_SIZE_RANGE
BRAINPOOL_T1	Brainpool T1 Elliptic Curve key.	BRAINPOOL_T1_SIZE_RANGE
ED25519 (1)	Edwards Curve25519 key.	N/A
X25519 (1)	ECDH key exchange based on Montgomery Curve25519.	N/A
ED448 (1)	Edwards Curve448 key.	N/A
X448 (1)	ECDH key exchange based on Montgomery Curve448.	N/A
AES	AES symmetric encryption	AES_SIZE_RANGE
DES	DES symmetric encryption	DES_SIZE_RANGE
DES3	Triple-DES symmetric encryption	DES3_SIZE_RANGE
SM4	SM4 symmetric encryption	N/A
HMAC	HMAC message authentication	HMAC_SIZE_RANGE

Table 7. Security Operation - Key type values...continued

Name	Description	Range Tag
RSA	RSA asymmetric encryption	RSA_SIZE_RANGE
DH	Diffie-Hellman key exchange	DH_SIZE_RANGE
TLS_MASTER (1)	TLS Master secret derivation	N/A
RAW (1)	Raw public key format	N/A
DERIVE (1)	Derive key material	N/A

1. These key types do not support configurable key size ranges. If a key size range is specified, the operation will fail with `SMW_STATUS_KEY_RANGE_NOT_CONFIGURABLE`.

2.2.5.3.5.2 OP_TYPE_VALUES

The capabilities definition apply to key derivation and some cryptographic operations. The following section lists all supported operation type values for key derivation and cryptographic operations.

Key derivation operation type

Following [Security Operation - Key derivation operation type values](#) table below lists all key derivation operation type values.

Table 8. Security Operation - Key derivation operation type values

OP_TYPE_VALUES	Description
HKDF	HMAC-based Key Derivation Function.
HKDF_EXTRACT	HMAC-based Key Derivation Function Extract step.
HKDF_EXPAND	HMAC-based Key Derivation Function Expand step.
ECDH	Elliptic Curve Diffie-Hellman key derivation.
TLS12_KEY_EXCHANGE	SECO subsystem TLS 1.2 Key Exchange. Deprecated prefer to use the operation <code>TLS12_OP_KEY_EXCHANGE</code> . Refer to SMW API - Key Derivation .
TLS12_OP_KEY_EXCHANGE	TLS 1.2 "Operation-based" Key Exchange.
TLS13_KEY_EXCHANGE	TLS 1.3 Key Exchange.
OEM_MASTER_KEY	OEM Master key derivation.

Note:

Starting with SMW Library version 5.5, the context which is part of `TLS12_OP_KEY_EXCHANGE` is freed by the library upon completion of the key expansion operation, so the application doesn't need to call the [smw_cancel_operation\(\)](#) function.

Cryptographic operation type

Following [Security Operation - Cryptographic operation type values](#) table below lists all cryptographic operation type values.

Table 9. Security Operation - Cryptographic operation type values

OP_TYPE_VALUES	Description
ENCRYPT	Encryption operation.
DECRYPT	Decryption operation.

2.2.5.3.5.3 HASH_ALGO_VALUES

Following [Security Operation - Hash algorithm values](#) table below lists all hash algorithm values.

Table 10. Security Operation - Hash algorithm values

HASH_ALGO_VALUES	Description
MD5	Message Digest 5.
SHA1	Secure Hash Algorithm 1.
SHA224	Secure Hash Algorithm 2, 224 bits
SHA256	Secure Hash Algorithm 2, 256 bits
SHA384	Secure Hash Algorithm 2, 384 bits
SHA512	Secure Hash Algorithm 2, 512 bits
SHA3_224	Secure Hash Algorithm 3, 224 bits
SHA3_256	Secure Hash Algorithm 3, 256 bits
SHA3_384	Secure Hash Algorithm 3, 384 bits
SHA3_512	Secure Hash Algorithm 3, 512 bits
SM3	ShangMi 3
SHAKE256	Secure Hash Algorithm KECCAK, 256 bits

2.2.5.3.5.4 MAC_ALGO_VALUES

Following [Security Operation - MAC algorithm values](#) table below lists all MAC algorithm values.

Table 11. Security Operation - MAC algorithm values

MAC_ALGO_VALUES	Description
CMAC	Cipher-based Message Authentication Code.
CMAC_TRUNCATED	Cipher-based Message Authentication Code truncated.
HMAC	Hash-Based Message Authentication Code.
HMAC_TRUNCATED	Hash-Based Message Authentication Code truncated.

2.2.5.3.5.5 SIGN_ALGO_VALUES

Following [Security Operation - Asymmetric signature algorithm values](#) table below lists all asymmetric signature algorithm values.

The column **SIGN_TYPE_VALUES** in the following table [Security Operation - Asymmetric signature algorithm values](#) table defines the possible signature type value that could be defined if the signature algorithm is used. If the type of signature is not defined, all signature type values are supported.

Table 12. Security Operation - Asymmetric signature algorithm values

SIGN_ALGO_VALUES	SIGN_TYPE_VALUES	Description
ECDSA	N/A	Elliptic Curve Digital Signature.
EDDSA		Edwards Curve Digital Signature.
	PURE_EDDSA	Pure EdDSA signature type. The message input is full message.

Table 12. Security Operation - Asymmetric signature algorithm values...continued

SIGN_ALGO_VALUES	SIGN_TYPE_VALUES	Description
	EDDSA_PH	Pre-hashed EdDSA signature type. The message input is pre-hashed.
	EDDSA_CTX	EdDSA signature additional context.
RSA		Rivest-Shamir-Adleman signature.
	PKCS1_1_5	PKCS#1 v1.5 signature type.
	PSS	Probabilistic Signature Scheme type.
DSA		Digital Signature Algorithm.
TLS_1_2		Transport Layer Security 1.2 signature.
	CLIENT	TLS 1.2 client signature type.
	SERVER	TLS 1.2 server signature type.

2.2.5.3.5.6 ENC_ALGO_VALUES

The following [Security Operation - Asymmetric encryption algorithm values](#) table below lists all asymmetric encryption algorithm values for the asymmetric encryption algorithm operations.

Table 13. Security Operation - Asymmetric encryption algorithm values

ENC_ALGO_VALUES	Description
RSA	Rivest-Shamir-Adleman asymmetric.

2.2.5.3.5.7 MODE_VALUES

The capabilities definition apply to the symmetric encryption, authentication encryption and asymmetric encryption cryptographic operations. The following section lists all support mode values per cryptographic operation type.

Symmetric encryption algorithm mode values

Following [Security Operation - Symmetric encryption algorithm mode values](#) table below lists all mode values for the symmetric algorithm operations.

Table 14. Security Operation - Symmetric encryption algorithm mode values

MODE_VALUES	Description
ECB	Electronic Codebook mode.
CBC	Cipher Block Chaining mode.
CTR	Counter mode.
CFB	Cipher Feedback mode.
OFB	Output Feedback mode.
CTS	Cipher Text Stealing mode.
XTS	XTS mode.

Authentication Encryption algorithm mode values

Following [Security Operation - AEAD algorithm mode values](#) table below lists all mode values for the authentication encryption algorithm operations.

Table 15. Security Operation - AEAD algorithm mode values

MODE_VALUES	Description
GCM	Galois/Counter Mode.
CCM	Counter with CBC-MAC mode.
CHACHA20_POLY1305	ChaCha20-Poly1305 AEAD mode.

Asymmetric Encryption algorithm mode values

Following [Security Operation - Asymmetric algorithm mode values](#) table below lists all mode values for the asymmetric encryption algorithm operations.

Table 16. Security Operation - Asymmetric algorithm mode values

MODE_VALUES	Description
OAEP	Optimal Asymmetric Encryption Padding.
PKCS1_1_5	PKCS#1 v1.5 encryption mode.
NO_PAD	No padding mode.

2.2.5.3.5.8 USE_ELA

The EdgeLock Accelerator (ELA) is a Cryptographic Accelerator accessible in the ELE allowing to perform cryptographic operations with a hardware accelerator.

The tag **USE_ELA** enables use of ELA hardware for cryptographic operations in an ELE subsystem when all the below mentioned conditions are met:

- Platform supports ELA hardware (e.g. i.MX943)
- Key is plaintext
- Cryptography operation (algorithm/mode) is supported by ELA

If all conditions are not met or if **USE_ELA** is not specified, operations use the standard ELE subsystem implementation.

Syntax:

```
[SECURITY_OPERATION]
  <operation_name>;
  USE_ELA;
  <other_capabilities>;
```

Example Usage:

```
[SECURE_SUBSYSTEM]
  ELE;
  [SECURITY_OPERATION]
    CIPHER;
    USE_ELA;
    KEY_TYPE_VALUES=AES;
    MODE_VALUES=CBC:CTR:ECB;
```

2.2.6 Example

Here is a possible Secure Subsystems configuration example. It's define ELE (like i.MX93) and TEE secure subsystems operation and capabilities.

The configuration is written to give the priority on the ELE subsystem for the secure operations if the secure operation argument does not specify a particular secure subsystem.

Secure Subsystem(s) configuration example

```
/* Configuration file */
VERSION=1;
PSA_DEFAULT=ELE:ALT;
[SECURE_SUBSYSTEM]
ELE;
/* No Start/Stop method specified. Default is 1. */
[SECURITY_OPERATION]
GENERATE_KEY;

KEY_TYPE_VALUES=AES:SECP_R1:BRAINPOOL_R1:HMAC:RSA:ED25519:X25519:ED448:X448;
SECP_R1_SIZE_RANGE=224:521;
BRAINPOOL_R1_SIZE_RANGE=224:384;
HMAC_SIZE_RANGE=224:512;
RSA_SIZE_RANGE=2048:4096;
[SECURITY_OPERATION]
DERIVE_KEY;

OP_TYPE_VALUES=TLS12_OP_KEY_EXCHANGE:TLS13_KEY_EXCHANGE:OEM_MASTER_KEY:CKDF;
[SECURITY_OPERATION]
EXPORT_KEY;
[SECURITY_OPERATION]
DELETE_KEY;
[SECURITY_OPERATION]
HASH;
[SECURITY_OPERATION]
HASH_MULTI_PART;
[SECURITY_OPERATION]
SIGN;
SIGN_ALGO_VALUES=ECDSA:RSA:EDDSA:TLS_1_2;
SIGN_TYPE_VALUES=PKCS1_1_5:PSS:CLIENT:SERVER:PURE_EDDSA:EDDSA_PH;
[SECURITY_OPERATION]
VERIFY;
SIGN_ALGO_VALUES=ECDSA:RSA:EDDSA:TLS_1_2;
SIGN_TYPE_VALUES=PKCS1_1_5:PSS:CLIENT:SERVER:PURE_EDDSA:EDDSA_PH;
[SECURITY_OPERATION]
SIGN_MULTI_PART;
SIGN_ALGO_VALUES=ECDSA:RSA:EDDSA;
SIGN_TYPE_VALUES=PKCS1_1_5:PSS:CLIENT:SERVER:PURE_EDDSA:EDDSA_PH;
[SECURITY_OPERATION]
VERIFY_MULTI_PART;
SIGN_ALGO_VALUES=ECDSA:RSA:EDDSA;
SIGN_TYPE_VALUES=PKCS1_1_5:PSS:CLIENT:SERVER:PURE_EDDSA:EDDSA_PH;
[SECURITY_OPERATION]
RNG;
[SECURITY_OPERATION]
CIPHER;
KEY_TYPE_VALUES=AES;
MODE_VALUES=CBC:CFB:CTR:ECB:OFB;
[SECURITY_OPERATION]
MAC;
[SECURITY_OPERATION]
DEVICE_ATTESTATION;
[SECURITY_OPERATION]
STORAGE_STORE;
```

```

    KEY_TYPE_VALUES=AES;
    MODE_VALUES=CBC:CFB:CTR:ECB:OFB;
    MAC_ALGO_VALUES=CMAC;
[SECURITY_OPERATION]
    STORAGE_RETRIEVE;
[SECURITY_OPERATION]
    STORAGE_DELETE;
[SECURITY_OPERATION]
    DEVICE_LIFECYCLE;
[SECURITY_OPERATION]
    AEAD;
    KEY_TYPE_VALUES=AES;
    MODE_VALUES=CCM:GCM:CHACHA20_POLY1305;
[SECURITY_OPERATION]
    DEVICE_REPROVISION;
[SECURITY_OPERATION]
    ASYMM_ENCRYPT;
    ENC_ALGO_VALUES=RSA;
    MODE_VALUES=PKCS1_1_5:OAEP;
[SECURITY_OPERATION]
    ASYMM_DECRYPT;
    ENC_ALGO_VALUES=RSA;
    MODE_VALUES=PKCS1_1_5:OAEP;
[SECURE_SUBSYSTEM]
    TEE;
    /* No Start/Stop method specified. Default is 1. */
[SECURITY_OPERATION]
    GENERATE_KEY;

KEY_TYPE_VALUES=AES:DES:DES3:SM4:BRAINPOOL_R1:SECP_R1:HMAC:RSA:ED25519:SM2;
[SECURITY_OPERATION]
    DERIVE_KEY;
    OP_TYPE_VALUES=HKDF:ECDH;
[SECURITY_OPERATION]
    IMPORT_KEY;
[SECURITY_OPERATION]
    EXPORT_KEY;
[SECURITY_OPERATION]
    DELETE_KEY;
[SECURITY_OPERATION]
    HASH;
[SECURITY_OPERATION]
    HASH_MULTI_PART;
[SECURITY_OPERATION]
    SIGN;
    SIGN_ALGO_VALUES=ECDSA:RSA:EDDSA:SM2;
    SIGN_TYPE_VALUES=PKCS1_1_5:PSS:PURE_EDDSA:EDDSA_PH:EDDSA_CTX;
[SECURITY_OPERATION]
    VERIFY;
    SIGN_ALGO_VALUES=ECDSA:RSA:EDDSA:SM2;
    SIGN_TYPE_VALUES=PKCS1_1_5:PSS:PURE_EDDSA:EDDSA_PH:EDDSA_CTX;
[SECURITY_OPERATION]
    SIGN_MULTI_PART;
    SIGN_ALGO_VALUES=ECDSA:RSA:EDDSA:SM2;
    SIGN_TYPE_VALUES=PKCS1_1_5:PSS:PURE_EDDSA:EDDSA_PH:EDDSA_CTX;
[SECURITY_OPERATION]
    VERIFY_MULTI_PART;
    SIGN_ALGO_VALUES=ECDSA:RSA:EDDSA:SM2;
    SIGN_TYPE_VALUES=PKCS1_1_5:PSS:PURE_EDDSA:EDDSA_PH:EDDSA_CTX;

```

```
[SECURITY_OPERATION]
    RNG;
[SECURITY_OPERATION]
    CIPHER;
    KEY_TYPE_VALUES=AES:DES:DES3:SM4;
    MODE_VALUES=CBC:CTR:CTS:ECB:XTS;
[SECURITY_OPERATION]
    CIPHER_MULTI_PART;
    KEY_TYPE_VALUES=AES:DES:DES3:SM4;
    MODE_VALUES=CBC:CTR:CTS:ECB:XTS;
[SECURITY_OPERATION]
    MAC;
[SECURITY_OPERATION]
    STORAGE_STORE;
[SECURITY_OPERATION]
    STORAGE_RETRIEVE;
[SECURITY_OPERATION]
    STORAGE_DELETE;
[SECURITY_OPERATION]
    AEAD;
    KEY_TYPE_VALUES=AES;
    MODE_VALUES=CCM:GCM;
[SECURITY_OPERATION]
    AEAD_MULTI_PART;
    KEY_TYPE_VALUES=AES;
    MODE_VALUES=CCM:GCM;
[SECURITY_OPERATION]
    ASYMM_ENCRYPT;
    ENC_ALGO_VALUES=RSA;
    MODE_VALUES=PKCS1_1_5:OAEP:NO_PAD;
[SECURITY_OPERATION]
    ASYMM_DECRYPT;
    ENC_ALGO_VALUES=RSA;
    MODE_VALUES=PKCS1_1_5:OAEP:NO_PAD;
[SECURE_SUBSYSTEM]
    ELE;
[SECURITY_OPERATION]
    /* Importing plain text key in ELE is not possible, must be a blob */
    IMPORT_KEY;
[SECURITY_OPERATION]
    /* TLS 1.3 Specific derivation operation */
    DERIVE_KEY;
    OP_TYPE_VALUES=ECDH:HKDF;
```

3 OSAL

In order to work on any Operating System (OS), the SMW library requires a module called OSAL. The module is split in two parts:

- The [Core library interface](#) that defines the contract between the SMW core library and the OSAL module.
- The OSAL user interface that allows applications to use the SMW library. The project delivery include a [OS-specific implementation](#) of the OSAL module.

The OSAL module includes a concept of object database used to store object's information like metadata. The [Core library interface](#) gives more details.

The integrator should adapt the implementation to their specific OS requirements and constraints.

Provided Linux implementation can be used as a reference but can also be replaced by a custom implementation that fits the target OS architecture and constraints.

3.1 Core library interface

The SMW-OSAL interface defines the contract between the SMW core library and the OSAL module. It specifies the structures and functions that the OSAL implementation must provide to enable proper communication and integration with the SMW core library.

This interface ensures that any OSAL implementation can be seamlessly integrated without modifying the core library itself, allowing for OS-specific customizations while maintaining a consistent contract.

Note:

This implementation can be customized by the integrator.

3.1.1 Object database

The OSAL module is in charge of creating/maintaining a database for the object (key/data) present in the Secure Subsystem(s) storage(s).

This database objectives are:

- Storing object metadata like identifier, attributes, subsystem owner, user label.
- Generating unique user object identifier if no provided by the user.
- Providing search operation by object attributes, user label, ...
- Identifying Secure Subsystem owning the object.
- Classifying object per persistency. Knowing that transient objects are not resisting to a system reset, the OSAL can destroy the transient object after system reset.

Note:

The database can't be used to store asymmetric private key or symmetric key.

Note:

The database is not mandatory but integrator must be aware that some features will not be supported like database search operation, user metadata like label and identifier.

Note:

The Linux OSAL reference supports importing public keys in the database as a helper for subsystems that cannot import public keys. During cryptographic operations (e.g. verifying a signature), subsystems may retrieve the public key from the database. To enable such scenario, the database must support this capability, and code in the SMW Library checks for the `SMW_OSAL_DB_CAPABILITY_PUBLIC_KEY_IMPORT` flag at runtime.

3.1.2 OSAL APIs

3.1.2.1 Initialization

3.1.2.1.1 smw_init

enum [smw_status_code](#) **smw_init**(const struct [smw_ops](#) *ops)

Initialize the SMW library.

Parameters

- **ops** (const struct [smw_ops](#)*) - pointer to the structure SMW OSAL operations interface.

3.1.2.1.1.1 Description

This function initializes the Security Middleware core library and must be called before using the library. The function registers the OSAL operations interface.

It verifies that all mandatory operations are implemented and all optional operations group are either all NULL or valid.

3.1.2.1.1.2 Return

- **SMW_STATUS_OK**
Initialization is successful
- **SMW_STATUS_OPS_INVALID**
ops is invalid
- **SMW_STATUS_MUTEX_INIT_FAILURE**
Mutex initialization has failed
- Other error code from [enum smw_status_code](#)

3.1.2.2 Deinitialization

3.1.2.2.1 smw_deinit

enum [smw_status_code](#) **smw_deinit**(void)

Deinitialize the SMW library.

Parameters

- **void** - no arguments

3.1.2.2.1.1 Description

This function deinitializes the Security Middleware core library. It frees all memory dynamically allocated by SMW.

3.1.2.2.1.2 Return

- **SMW_STATUS_OK**

Deinitialization is successful

- **SMW_STATUS_INVALID_LIBRARY_CONTEXT**

Library context is not valid

- **SMW_STATUS_MUTEX_DESTROY_FAILURE**

Mutex destruction has failed

- Other error code from [enum smw_status_code](#)

3.1.3 OSAL operations

3.1.3.1 Interface

Following structure describes the OSAL operations interface optional and mandatory function pointers.

3.1.3.1.1 struct smw_ops

struct **smw_ops**

SMW OSAL operations interface

3.1.3.1.1.1 Definition

```
struct smw_ops {
    smw_osal_critical_section_start_t critical_section_start;
    smw_osal_critical_section_stop_t critical_section_stop;
    smw_osal_mutex_init_t mutex_init;
    smw_osal_mutex_destroy_t mutex_destroy;
    smw_osal_mutex_lock_t mutex_lock;
    smw_osal_mutex_unlock_t mutex_unlock;
    smw_osal_thread_create_t thread_create;
    smw_osal_thread_cancel_t thread_cancel;
    smw_osal_vprint_t vprint;
    smw_osal_hex_dump_t hex_dump;
    smw_osal_register_active_subsystem_t register_active_subsystem;
    smw_osal_get_subsystem_info_t get_subsystem_info;
    smw_osal_is_lib_initialized_t is_lib_initialized;
    smw_osal_db_get_obj_t get_obj_info;
    smw_osal_db_add_obj_t add_obj_info;
    smw_osal_db_update_obj_t update_obj_info;
    smw_osal_db_delete_obj_t delete_obj_info;
    smw_osal_db_find_init_t find_obj_init;
    smw_osal_db_find_next_t find_obj_next;
    smw_osal_db_find_final_t find_obj_final;
    smw_osal_wait_t wait;
    smw_osal_file_initialize_t file_initialize;
    smw_osal_file_write_t file_write;
    smw_osal_file_read_t file_read;
    smw_osal_dcache_invalidate_t dcache_invalidate;
    smw_osal_dcache_clean_t dcache_clean;
    smw_osal_shared_memory_init_t shared_memory_init;
    smw_osal_shared_memory_deinit_t shared_memory_deinit;
    smw_osal_shared_memory_alloc_t shared_memory_alloc;
    smw_osal_shared_memory_free_t shared_memory_free;
    smw_osal_get_mu_base_t get_mu_base;
    smw_osal_db_has_capability_t db_has_capability;
}
```

```
}
```

3.1.3.1.1.2 Members

critical_section_start

(optional) Start critical section, see [smw_osal_critical_section_start_t\(\)](#).

critical_section_stop

(optional) Stop critical section, see [smw_osal_critical_section_stop_t\(\)](#).

mutex_init

(mandatory) Initialize a mutex, see [smw_osal_mutex_init_t\(\)](#).

mutex_destroy

(mandatory) Destroy a mutex, see [smw_osal_mutex_destroy_t\(\)](#).

mutex_lock

(mandatory) Lock a mutex, see [smw_osal_mutex_lock_t\(\)](#).

mutex_unlock

(mandatory) Unlock a mutex, see [smw_osal_mutex_unlock_t\(\)](#).

thread_create

(mandatory) Create a thread, see [smw_osal_thread_create_t\(\)](#).

thread_cancel

(mandatory) Cancel a thread, see [smw_osal_thread_cancel_t\(\)](#).

vprint

(optional) Print debug trace, see [smw_osal_vprint_t\(\)](#).

hex_dump

(optional) Print buffer content in hexadecimal format, see [smw_osal_hex_dump_t\(\)](#).

register_active_subsystem

(optional) Register the active Secure Subsystem, see [smw_osal_register_active_subsystem_t\(\)](#).

get_subsystem_info

(mandatory) Get Subsystem configuration information, see [smw_osal_get_subsystem_info_t\(\)](#).

is_lib_initialized

(mandatory) Check if the library was successfully initialized by OSAL, see [smw_osal_is_lib_initialized_t\(\)](#).

get_obj_info

(mandatory) Get an object information from database, see [smw_osal_db_get_obj_t\(\)](#).

add_obj_info

(mandatory) Add an object information into database, see [smw_osal_db_add_obj_t\(\)](#).

update_obj_info

(mandatory) Update an object information into database, see [smw_osal_db_update_obj_t\(\)](#).

delete_obj_info

(mandatory) Delete an object information from database, see [smw_osal_db_delete_obj_t\(\)](#).

find_obj_init

(mandatory) Initialize the find object query, see [smw_osal_db_find_init_t\(\)](#).

find_obj_next

(mandatory) Find the next object matching criteria, see [smw_osal_db_find_next_t\(\)](#).

find_obj_final

(mandatory) Close the find object query, see [smw_osal_db_find_final_t\(\)](#).

wait

(optional) Wait for a specified amount of time, see [smw_osal_wait_t\(\)](#).

file_initialize

(optional) Initialize NVM storage file system, see [smw_osal_file_initialize_t\(\)](#).

file_write

(optional) Write data chunk to NVM storage, see [smw_osal_file_write_t\(\)](#).

file_read

(optional) Read data chunk from NVM storage, see [smw_osal_file_read_t\(\)](#).

dcache_invalidate

(optional) Invalidate data cache for a memory range, see [smw_osal_dcache_invalidate_t\(\)](#).

dcache_clean

(optional) Clean data cache for a memory range, see [smw_osal_dcache_clean_t\(\)](#).

shared_memory_init

(optional) Initialize shared memory for ELE, see [smw_osal_shared_memory_init_t\(\)](#).

shared_memory_deinit

(optional) Deinitialize shared memory for ELE, see [smw_osal_shared_memory_deinit_t\(\)](#).

shared_memory_alloc

(optional) Allocate shared buffer for ELE, see [smw_osal_shared_memory_alloc_t\(\)](#).

shared_memory_free

(optional) Free shared buffer allocated for ELE, see [smw_osal_shared_memory_free_t\(\)](#).

get_mu_base

(optional) Get ELE MU base address, see [smw_osal_get_mu_base_t\(\)](#).

db_has_capability

(mandatory) Check if the object database supports the specified capability.

3.1.3.1.1.3 Description

This structure defines the SMW OSAL operations interface using function pointers that are implemented in the OSAL module.

Function pointers:

- `mutex_*` functions pointers are optional together.
- `critical_*` functions pointers are optional together.

3.1.3.2 Critical section

3.1.3.2.1 typedef smw_osal_critical_section_start_t

void **smw_osal_critical_section_start_t**(void)

Start a critical section.

Parameters

- **void** - no arguments

3.1.3.2.1.1 Description

This function starts a critical section barrier to prevent operation executing after this call to be interrupted.

The critical section is stopped with [smw_osal_critical_section_stop_t\(\)](#).

3.1.3.2.1.2 Return

None

3.1.3.2.2 typedef smw_osal_critical_section_stop_t

void **smw_osal_critical_section_stop_t**(void)

Stop a critical section.

Parameters

- **void** - no arguments

3.1.3.2.2.1 Description

This function stops the critical section barrier after this call execution can be interrupted.

The critical section is started with [smw_osal_critical_section_start_t\(\)](#).

3.1.3.2.2.2 Return

None

3.1.3.3 Mutex protection

3.1.3.3.1 typedef smw_osal_mutex_init_t

int **smw_osal_mutex_init_t**(void **mutex)

Initialize a mutex.

Parameters

- **mutex** (void**) - [out] Pointer to mutex object created.

3.1.3.3.1.1 Description

The function allocates and initialize a mutex object. The mutex must be released with

[smw_osal_mutex_destroy_t\(\)](#).

3.1.3.3.1.2 Return

- 0 on success.
- negative value on failure.

3.1.3.3.2 typedef smw_osal_mutex_destroy_t

int **smw_osal_mutex_destroy_t**(void **mutex)

Destroy a mutex.

Parameters

- **mutex** (void**) - [in/out] Pointer to mutex object to destroy.

3.1.3.3.2.1 Description

The function stops and releases a mutex object previously created with [smw_osal_mutex_init_t\(\)](#).

The `mutex` pointer is set to NULL after destruction.

3.1.3.3.2.2 Return

- 0 on success.
- negative value on failure.

3.1.3.3.3 typedef smw_osal_mutex_lock_t

int **smw_osal_mutex_lock_t**(void *mutex)

Lock a mutex.

Parameters

- **mutex** (void*) - [in] Pointer to mutex object to lock.

3.1.3.3.3.1 Description

The functions locks a mutex object previously created with [smw_osal_mutex_init_t\(\)](#).

3.1.3.3.3.2 Return

- 0 on success.
- negative value on failure.

3.1.3.3.4 typedef smw_osal_mutex_unlock_t

int **smw_osal_mutex_unlock_t**(void *mutex)

Unlock a mutex.

Parameters

- **mutex** (void*) - [in] Pointer to mutex object to unlock.

3.1.3.3.4.1 Description

The functions unlocks a mutex object previously locked with [smw_osal_mutex_lock_t\(\)](#).

3.1.3.3.4.2 Return

- 0 on success.
- negative value on failure.

3.1.3.4 Thread management

3.1.3.4.1 typedef smw_osal_thread_create_t

int **smw_osal_thread_create_t**(*unsigned long *thread, void *(*start_routine)(void*), void *args*)

Create a thread.

Parameters

- **thread** (unsigned long*) - [out] Pointer to thread object created.
- **start_routine** (void *(*)(void*)) - [in] Pointer to the thread start function.
- **args** (void*) - [in] Pointer to arguments passed to the thread start function.

3.1.3.4.1.1 Description

The function creates a new thread that executes the function pointed to by `start_routine()` with the argument `arg`. The new thread is identified by the pointer `thread` which must be released with [smw_osal_thread_cancel_t\(\)](#).

3.1.3.4.1.2 Return

- 0 on success.
- negative value on failure.

3.1.3.4.2 typedef smw_osal_thread_cancel_t

int **smw_osal_thread_cancel_t**(*unsigned long thread*)

Cancel a thread.

Parameters

- **thread** (unsigned long) - [in] Thread object to cancel.

3.1.3.4.2.1 Description

The function cancels a thread object previously created with [smw_osal_thread_create_t\(\)](#). On success, the thread is terminated and the thread data is released.

3.1.3.4.2.2 Return

- 0 on success.
- negative value on failure.

3.1.3.5 Debug trace

3.1.3.5.1 typedef smw_osal_vprint_t

void **smw_osal_vprint_t**(*unsigned int level, const char *fmt, va_list args*)

Print a message with variable arguments.

Parameters

- **level** (unsigned int) - [in] Log level for the message.
- **fmt** (const char*) - [in] Pointer to the format string.
- **args** (va_list) - [in] Variable argument list.

3.1.3.5.1.1 Description

This function allows to print on the console or into a log file the library debug traces function of the input level value and the VERBOSE built option. It's implementation define.

The debug trace is enabled if the project is built with VERBOSE set to the maximum trace level desired. If the project is built in Debug mode the maximum trace level is up to level 5, else it's limited to level 2.

3.1.3.5.1.2 The level value definition

- 1. No trace.
- 1. Failures of which the user must be aware.
- 1. Traces which could interest the user.
- 1. First level of debugging information.
- 1. Second level of debugging information.
- 1. Maximum level of debugging information.

3.1.3.5.2 typedef smw_osal_hex_dump_t

void **smw_osal_hex_dump_t**(*unsigned int level, const unsigned char *addr, unsigned int size, unsigned int align*)

Print the hexadecimal content of a buffer.

Parameters

- **level** (unsigned int) - [in] Log level for the message.
- **addr** (const unsigned char*) - [in] Pointer to the buffer to dump.
- **size** (unsigned int) - [in] Size in bytes of the buffer to dump.
- **align** (unsigned int) - [in] Alignment in bytes for line wrapping of the dump.

3.1.3.5.2.1 Description

This function allows to print on the console or into a log file some buffer content in hexadecimal. It's implementation define.

Like the [smw_osal_vprint_t\(\)](#) function, the hexadecimal dump is function of the level of trace. The buffer dump is enabled if the project is built in Debug mode with VERBOSE option set to level 3 or 4.

3.1.3.6 Active subsystem

3.1.3.6.1 typedef smw_osal_register_active_subsystem_t

void **smw_osal_register_active_subsystem_t**([smw_subsystem_t](#) subsystem)

Register an active subsystem.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Subsystem identifier to register as active.

3.1.3.6.1.1 Description

This function is called to register the latest subsystem used. The purpose of this feature is purely for debug purposes.

Usage of this functionality is implementation define.

This information is used by the application to know which subsystem was executing the last operation by using a specific OSAL API function like for example the [smw_osal_latest_subsystem_name\(\)](#).

3.1.3.7 Get the subsystem information

3.1.3.7.1 typedef smw_osal_get_subsystem_info_t

int **smw_osal_get_subsystem_info_t**([smw_subsystem_t](#) subsystem, void *info)

Get subsystem information.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Subsystem identifier.
- **info** (void*) - [out] Pointer to subsystem information structure.

3.1.3.7.1.1 Description

This function returns the `subsystem` information required to load and configure the Secure Subsystem. The subsystem information is set when the library is loaded.

The way the subsystem information are defined is implementation define.

The OSAL API [smw_osal_set_subsystem_info\(\)](#) allows to setup the subsystem information.

In proposed Linux OSAL implementation, the subsystem information is defined by the system configuration file *smw.conf* and parsed during library initialization, the [smw_osal_set_subsystem_info\(\)](#) overwrites the system configuration file settings.

The `info` parameter points to a subsystem information structure that must be [struct_se_info](#) for NXP i.MX Secure Enclave subsystem (ELE, SECO) or [struct_tee_info](#) for TEE subsystem.

3.1.3.7.1.2 Return

- 0 on success.
- -1 on failure.

3.1.3.8 Library initialization

3.1.3.8.1 typedef smw_osal_is_lib_initialized_t

bool **smw_osal_is_lib_initialized_t**(void)

Check if library is initialized.

Parameters

- **void** - no arguments

3.1.3.8.1.1 Description

This function returns if the library is initialized or not.

3.1.3.8.1.2 Return

- True if library is initialized.
- False otherwise.

3.1.3.9 Object database management

3.1.3.9.1 typedef smw_osal_db_get_obj_t

int **smw_osal_db_get_obj_t**(struct [smw_osal_object](#) *descriptor)

Get object information from database.

Parameters

- **descriptor** (struct [smw_osal_object](#)*) - [in/out] Pointer to OSAL object descriptor structure.

3.1.3.9.1.1 Description

This function retrieves object information from the database based on the `descriptor->obj_desc->id` identifier. This identifier is the identifier returned to the user API during the object creation.

The function populates the `descriptor` structure with the object information stored in the database.

3.1.3.9.1.2 Return

- 0 on success.
- negative value on failure.

3.1.3.9.2 typedef smw_osal_db_add_obj_t

int **smw_osal_db_add_obj_t**(struct [smw_osal_object](#) *descriptor)

Add object information to database.

Parameters

- **descriptor** (struct [smw_osal_object](#)*) - [in/out] Pointer to OSAL object descriptor structure.

3.1.3.9.2.1 Description

This function adds object information to the database based on the input `descriptor` content.

How the object information is stored in the database is implementation define.

The function must return the `descriptor->obj_desc->id` identifier which is used for object identifier at user application level. This identifier can be given by the user during the object creation if object is persistent.

3.1.3.9.2.2 Return

- 0 on success.
- negative value on failure.

3.1.3.9.3 typedef smw_osal_db_update_obj_t

int **smw_osal_db_update_obj_t**(struct [smw_osal_object](#) *descriptor)

Update object information in database.

Parameters

- **descriptor** (struct [smw_osal_object](#)*) - [in] Pointer to OSAL object descriptor structure.

3.1.3.9.3.1 Description

This function updates object information in the database with the content of the `descriptor` structure.

The object is identified by the `descriptor->obj_desc->id` identifier returned by the [smw_osal_db_add_obj_t\(\)](#) function during object creation.

3.1.3.9.3.2 Return

- 0 on success.
- negative value on failure.

3.1.3.9.4 typedef smw_osal_db_delete_obj_t

int **smw_osal_db_delete_obj_t**(struct [smw_osal_object](#) *descriptor)

Delete object information from database.

Parameters

- **descriptor** (struct [smw_osal_object](#)*) - [in] Pointer to OSAL object descriptor structure.

3.1.3.9.4.1 Description

This function deletes the object information from the database based on the content of the `descriptor` structure.

The object is identified by the `descriptor->obj_desc->id` identifier returned by the [smw_osal_db_add_obj_t\(\)](#) function during object creation.

3.1.3.9.4.2 Return

- 0 on success.
- negative value on failure.

3.1.3.9.5 typedef smw_osal_db_find_init_t

int **smw_osal_db_find_init_t**(void **context, struct [smw_osal_object](#) *descriptor)

Initialize object search in database.

Parameters

- **context** (void**) - [out] Pointer to search context.
- **descriptor** (struct [smw_osal_object](#)*) - [in] Pointer to OSAL object descriptor structure used as search criteria.

3.1.3.9.5.1 Description

This function initializes a search context for finding objects in the database that match the criteria specified in the `descriptor` structure.

The function allocates the `context` that must be used each `smw_osal_db_find_next_t()` call.

The `context` must be freed by calling `smw_osal_db_find_final_t()`.

3.1.3.9.5.2 Return

- 0 on success.
- negative value on failure.

3.1.3.9.6 typedef smw_osal_db_find_next_t

```
int smw_osal_db_find_next_t(void *context, struct smw_osal_object *descriptor)
```

Get next object from database search.

Parameters

- **context** (void*) - [in/out] Pointer to search context.
- **descriptor** (struct [smw_osal_object](#)*) - [out] Pointer to OSAL object descriptor structure.

3.1.3.9.6.1 Description

The function retrieves the next object from the database search results corresponding to the search context initialized by `smw_osal_db_find_init_t()`.

The function may or not update the `context` depending on the implementation.

The `descriptor` structure is populated with the next object information found.

The `descriptor->obj_desc->id` field must be set to 0 when no more objects are found in the database search results.

3.1.3.9.6.2 Return

- 0 on success.
- negative value on failure.

3.1.3.9.7 typedef smw_osal_db_find_final_t

```
int smw_osal_db_find_final_t(void *context)
```

Finalize object search in database.

Parameters

- **context** (void*) - [in] Pointer to search context.

3.1.3.9.7.1 Description

This function finalizes a search context previously initialized by the `smw_osal_db_find_init_t()` function and updated by the `smw_osal_db_find_next_t()` function.

The function release the resources allocated by `smw_osal_db_find_init_t()`.

3.1.3.9.7.2 Return

- 0 on success.
- negative value on failure.

3.1.3.9.8 struct smw_osal_object

struct **smw_osal_object**

OSAL object descriptor

3.1.3.9.8.1 Definition

```
struct smw_osal_object {  
    struct smw_object_descriptor *obj_desc;  
    unsigned int obj_id_subsystem;  
}
```

3.1.3.9.8.2 Members

obj_desc

Object descriptor (user information), see [struct smw_object_descriptor](#).

obj_id_subsystem

Object identifier in Secure Subsystem.

3.1.3.9.8.3 Description

The user API identifier is transported by the `obj_desc->id`. This identifier value is set when creating the object in the database. This value could be different from the object identifier assigned by the Secure Subsystem.

The `obj_id_subsystem` is the object identifier assigned by the Secure Subsystem and used internally.

3.1.3.10 NVM storage file operations

3.1.3.10.1 typedef smw_osal_file_initialize_t

int **smw_osal_file_initialize_t**(void)

Initialize NVM storage file system

Parameters

- **void** - no arguments

3.1.3.10.1.1 Return

- 0 on success.
- negative value on failure.

3.1.3.10.2 typedef smw_osal_file_write_t

int **smw_osal_file_write_t**(uint32_t blob_id_msb, uint32_t blob_id_lsb, uint32_t blob_ext, uint32_t *chunk, size_t chunk_sz)

Write data chunk to NVM storage

Parameters

- **blob_id_msb** (uint32_t) - Most significant bits of blob identifier
- **blob_id_lsb** (uint32_t) - Least significant bits of blob identifier
- **blob_ext** (uint32_t) - Blob extension/type identifier
- **chunk** (uint32_t*) - Pointer to data chunk to write
- **chunk_sz** (size_t) - Size of data chunk in bytes

3.1.3.10.2.1 Return

- 0 on success.
- negative value on failure.

3.1.3.10.3 typedef smw_osal_file_read_t

int **smw_osal_file_read_t**(uint32_t blob_id_msb, uint32_t blob_id_lsb, uint32_t blob_ext, uint32_t *chunk, size_t *chunk_sz)

Read data chunk from NVM storage

Parameters

- **blob_id_msb** (uint32_t) - Most significant bits of blob identifier
- **blob_id_lsb** (uint32_t) - Least significant bits of blob identifier
- **blob_ext** (uint32_t) - Blob extension/type identifier
- **chunk** (uint32_t*) - Pointer to data chunk to read
- **chunk_sz** (size_t*) - Size of data chunk in bytes

3.1.3.10.3.1 Return

- 0 on success.
- negative value on failure.

3.1.3.11 Data cache operations

3.1.3.11.1 typedef smw_osal_dcache_invalidate_t

void **smw_osal_dcache_invalidate_t**(void *addr, size_t size)

Invalidate data cache for a memory range

Parameters

- **addr** (void*) - Pointer to the start of the memory range
- **size** (size_t) - Size of the memory range in bytes

3.1.3.11.2 typedef smw_osal_dcache_clean_t

void **smw_osal_dcache_clean_t**(void *addr, size_t size)

Clean data cache for a memory range

Parameters

- **addr** (void*) - Pointer to the start of the memory range
- **size** (size_t) - Size of the memory range in bytes

3.1.3.12 Shared memory operations

3.1.3.12.1 typedef smw_osal_shared_memory_init_t

void **smw_osal_shared_memory_init_t**(void)

Initialize shared memory for ELE

Parameters

- **void** - no arguments

3.1.3.12.1.1 Description

This function initializes the shared memory for communication with EdgeLock Enclave. It should be called once during system initialization before any communication with ELE.

3.1.3.12.2 typedef smw_osal_shared_memory_deinit_t

void **smw_osal_shared_memory_deinit_t**(void)

Deinitialize shared memory for ELE

Parameters

- **void** - no arguments

3.1.3.12.2.1 Description

This function deinitializes the shared memory for communication with EdgeLock Enclave. It should be called during system deinitialization.

3.1.3.12.3 typedef smw_osal_shared_memory_alloc_t

[void_ptr_t](#) **smw_osal_shared_memory_alloc_t**(void *buf, size_t size, uintptr_t *phys_addr)

Allocate shared buffer for ELE

Parameters

- **buf** (void*) - pointer to the buffer to be shared with ELE
- **size** (size_t) - size of the buffer in bytes
- **phys_addr** (uintptr_t*) - pointer where to output the physical address of the allocated buffer, which will be used by ELE.

3.1.3.12.3.1 Description

This function allocates a shared buffer for ELE. If MMU is enabled, the function allocates an aligned buffer in a memory region with appropriate attributes and copies the content of the input buffer to the allocated buffer. If MMU is not enabled, the function returns the input buffer as is.

3.1.3.12.3.2 Return

Pointer to the allocated shared buffer, or NULL if allocation fails.

3.1.3.12.4 typedef smw_osal_shared_memory_free_t

void **smw_osal_shared_memory_free_t**(void *buf, size_t size, void *original_buf)

Free shared buffer allocated for ELE

Parameters

- **buf** (void*) - pointer to the buffer to be freed
- **size** (size_t) - size of the buffer in bytes
- **original_buf** (void*) - pointer to the original buffer provided to [smw_osal_shared_memory_alloc_t\(\)](#)

3.1.3.12.4.1 Description

This function frees a shared buffer allocated for ELE. If MMU is enabled, the function unmaps the memory region allocated for the buffer. If MMU is not enabled, the function does nothing.

3.1.3.13 MU base address

3.1.3.13.1 typedef smw_osal_get_mu_base_t

[void_ptr_t](#) **smw_osal_get_mu_base_t**(void)

Get MU base address

Parameters

- **void** - no arguments

3.1.3.13.1.1 Return

MU Base pointer, or NULL if no MU found.

3.1.3.14 Additional data type

3.1.3.14.1 void_ptr_t

type **void_ptr_t**

Generic pointer type.

3.1.3.14.1.1 Definition

```
typedef void *void_ptr_t;
```

3.1.3.14.1.2 Description

void_ptr_t is a generic pointer type used to reference any type of data.

3.2 OS-specific implementation

The OSAL module must be implemented for the target Operating System. The SMW library project delivery includes a reference implementation for Linux.

The OS-specific OSAL implementation is responsible for:

- Load and unload library in the OS environment.
- File system operations for object database management.
- Thread and synchronization primitives.
- Logging.

Note:

This implementation can be customized by the integrator.

3.2.1 Library initialization methods

In Linux, the SMW library initialization is performed by calling the [smw_osal_lib_init\(\)](#) function.

Two methods are proposed to configure the SMW library:

- Using the linux system configuration file approach (recommended).
- Using the programmatic approach (runtime configuration).

3.2.1.1 Linux system configuration file approach

The system configuration file path is `/etc/opt/smw/smw.conf`.

The file is divided into sections as shown in the template below:

SMW configuration file template

```
#
# This file contains the configuration of the SMW library.
# It must be installed in the system folder `/etc/opt/smw/smw.conf`
# All sections are not mandatory.
#
[setup]
# SMW configuration file describing the subsystem(s) operations
# smw_config_file=[path/to/file]

# SMW object database containing all known objects
# database=[path/to/file]
database=/usr/share/smw/smw_objects_database.dat

[TEE]
# OPTEE TA UUID to be loaded if subsystem used.
# ta_uuid must be defined.

[SECO]
# SECO subsystem NVM Secure Storage configuration if subsystem used.
# Following 3 items must be defined
# id=
# nonce=
# replay=

[ELE]
# ELE subsystem NVM Secure Storage configuration if subsystem used.
```

```
# Following 3 items must be defined
# id=
# nonce=
# shared=
```

The `smw_system_conf.sh` could be used to change the content of the `smw.conf` file. It's installed on in the `/etc/opt/smw/system` folder.

```
$ /etc/opt/smw/smw_system_conf.sh --help
```

To help fast integration of the SMW library, a default configuration file is provided and installed in the `/etc/opt/smw/system` folder. This file contains reference to the default subsystem(s) configurations, a default database file. This file contains the configuration of the subsystem(s) included during the build of the SMW library. It can be modified to match specific integration requirements.

The default configuration is:

- database file: `/usr/share/smw/smw_objects_database.dat`
- subsystem configuration files used are in the `/usr/share/smw/config/` and depends on the subsystems included during the build.
- If TEE subsystem enabled, the TA UUID is the one provided as reference “11b5c4aa-6d20-11ea-bc55-0242ac130003”.
- If ELE subsystem enabled, the NVM Secure Storage; identifier is 0x454C4500, nonce is 0x534D57 and the storage is created to be shared between applications (i.e., shared is yes).
- If SECO subsystem enabled, the NVM Secure Storage; identifier is 0x454C4500, nonce is 0x534D57 and the replay attack counter is 3000.

Note:

ELE and SECO subsystems are platforms specific and can not be available together at the same time. TEE is present on all platforms.

3.2.1.2 Programmatic approach

The programmatic approach allows direct configuration of the SMW library through OSAL APIs without relying on the configuration file. This method provides runtime flexibility and allows applications to configure the SMW library dynamically based on their specific requirements.

The configuration must be completed before calling the SMW library initialization function [smw_osal_lib_init\(\)](#).

The following steps outline the programmatic configuration process:

1. Set the `SMW_CONFIG_FILE` linux environment variable to the desired file path describing the subsystem(s)/operations mapping.
2. Call [smw_osal_set_subsystem_info\(\)](#) to setup the subsystem(s) with the desired parameters (e.g., TEE or Secure Element information).
3. Call [smw_osal_open_obj_db\(\)](#) to open and configure the object database file path.
4. Call [smw_osal_lib_init\(\)](#) to initialize the SMW library.
5. The SMW library is now configured and ready to use.

You can refer the following code [Example](#).

3.2.2 OSAL APIs

3.2.2.1 Initialization

3.2.2.1.1 smw_osal_lib_init

enum [smw_status_code](#) **smw_osal_lib_init**(void)

Initialize the SMW library

Parameters

- **void** - no arguments

3.2.2.1.1.1 Description

This function must be called before any key management, cryptographic operations to initialize the SMW library. It loads the subsystem configuration either defined in the system *smw.conf* file or by set in the linux environment variable *SMW_CONFIG_FILE*.

CAUTION:

The environment variable takes precedence over the configuration file.

Note:

*This function is not thread-safe. Other initialization functions from the PSA or PKCS#11 APIs can call this function directly or indirectly. When using those APIs, prefer using only those methods of initialization, e.g. *psa_crypto_init()* or *C_Initialize()* respectively. This function should only be called when using the SMW APIs.*

3.2.2.1.1.2 Return

- **SMW_STATUS_OK**
Library initialization success.
- **SMW_STATUS_LIBRARY_ALREADY_INIT**
Library already initialized.
- Other error code from [enum smw_status_code](#)

3.2.2.2 Configuration

3.2.2.2.1 smw_osal_set_subsystem_info

enum [smw_status_code](#) **smw_osal_set_subsystem_info**([smw_subsystem_t](#) subsystem, void *info, size_t info_size)

Set the Subsystem configuration information.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Subsystem name.
- **info** (void*) - [in] Subsystem information.
- **info_size** (size_t) - [in] Size in bytes of *info* parameter.

3.2.2.2.1.1 Description

This function sets the subsystem configuration information used when the subsystem is loaded.

CAUTION:

This method overwrites the configuration set in file smw.conf.

3.2.2.2.1.2 Return

- **SMW_STATUS_OK**
Success.
- **SMW_STATUS_SUBSYSTEM_LOADED**
Subsystem is already loaded.
- **SMW_STATUS_INVALID_PARAM**
Function parameter error.
- **SMW_STATUS_ALLOC_FAILURE**
Allocation failure.
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME**
Unknown subsystem name.
- Other error code from [enum smw_status_code](#)

3.2.2.2.2 smw_osal_open_obj_db

enum [smw_status_code](#) **smw_osal_open_obj_db**(const char *file, size_t len)

Setup the object database file to open.

Parameters

- **file** (const char*) - [in] Fullname of the object database, null-terminated string.
- **len** (size_t) - [in] Length is bytes of the filename string.

3.2.2.2.2.1 Description

If the library is already initialized, database may be already in use by another application. If application must be separated, this function allows to define database file per application overwritten the file set in the system configuration file 'smw.conf'. This function must be called before [smw_osal_lib_init\(\)](#).

CAUTION:

This method overwrites the configuration set in file smw.conf.

3.2.2.2.2.2 Return

- **SMW_STATUS_OK**
Success.
- **SMW_STATUS_LIBRARY_ALREADY_INIT**
Library already initialized.
- **SMW_STATUS_CONFIGURATION_FAILURE**
Error of configuration.
- **SMW_STATUS_ALLOC_FAILURE**

Out of memory.

- Other error code from [enum smw_status_code](#)

3.2.2.2.3 struct tee_info

struct **tee_info**

TEE Subsystem information

3.2.2.2.3.1 Definition

```
struct tee_info {  
    char ta_uuid[TEE_TA_UUID_SIZE_MAX];  
}
```

3.2.2.2.3.2 Members

ta_uuid

TEE TA UUID value as a null-terminated string.

3.2.2.2.4 struct se_info

struct **se_info**

Secure Enclave information about the NVM Secure Storage

3.2.2.2.4.1 Definition

```
struct se_info {  
    unsigned int storage_id;  
    unsigned int storage_nonce;  
    unsigned short storage_replay;  
    bool storage_shared;  
}
```

3.2.2.2.4.2 Members

storage_id

User defined identifier.

storage_nonce

User defined nonce to authenticate the storage.

storage_replay

Replay attack counter (Only for SECO Secure Subsystem).

storage_shared

Allows to share storage between multiple applications (Only for ELE Secure Subsystem).

3.2.2.2.5 macro TEE_TA_UUID_SIZE_MAX

TEE_TA_UUID_SIZE_MAX()

Size in bytes of the TEE TA UUID.

3.2.2.2.5.1 Description

Size in bytes of the TEE TA UUID string size including the null terminator.

3.2.2.3 Debug purpose

3.2.2.3.1 smw_osal_latest_subsystem_name

[smw_subsystem_t](#) [smw_osal_latest_subsystem_name](#)(void)

Return the latest Secure Subsystem name.

Parameters

- **void** - no arguments

3.2.2.3.1.1 Description

In DEBUG mode only, function returns the name of the latest Secure Subsystem invoked by SMW. This Secure Subsystem have been either explicitly requested by the caller or selected by SMW given the operation arguments and the configuration file. In other modes, function always returns NULL.

3.2.2.3.1.2 Return

- The latest active Secure Subsystem name, in DEBUG mode only,
- SMW_SUBSYSTEM_NAME_NONE otherwise.

3.2.2.4 Check capability flags

3.2.2.4.1 smw_osal_obj_db_has_capability

enum [smw_status_code](#) [smw_osal_obj_db_has_capability](#)([smw_osal_db_capability_t](#) flag)

Check the object database capability flag.

Parameters

- **flag** ([smw_osal_db_capability_t](#)) - [in] Capability flag to check.

3.2.2.4.1.1 Description

Check the object database capability flag. The database must already be opened.

3.2.2.4.1.2 Return

- **SMW_STATUS_OK**
Success and flag is supported.
- **SMW_STATUS_INVALID_LIBRARY_CONTEXT**

Library is not initialized.

- **SMW_STATUS_OBJ_DB_CAPABILITY_NOT_SUPPORTED**

Capability flag is not supported.

3.2.3 Linux object database implementation

The SMW Linux OSAL reference distributed with the SMW Library manages the object database by using the SQLite3 library. Thus, the database is actually an SQLite3 table where each known object occupies a row. An object could be a key or data object that is associated with a subsystem.

Since the database was introduced in SMW Library version 5.0, the database has been versioned, and version 1 was introduced. The metadata corresponding to each object are as follows:

- the identifier of the object in the database
- the identifier of the object as assigned by the subsystem
- the identifier as defined by the user
- the subsystem associated with the object
- object type (e.g. key or data)
- object size
- object attributes
- key type (if object is a key)
- storage identifier (if assigned by the subsystem)
- key group (if assigned by the subsystem)
- label (if assigned by the PKCS#11 API)
- key permitted algorithm (if object is a key)
- key usage (if object is a key)

SMW Library version 5.5 has modified the schema so the database version has been increased to 2. In this version, a new column was added:

- key public data (only if object is a public key)

This column has the purpose of storing public keys for subsystems that cannot import them, but could use them for cryptographic operations if they are provided as plain bytes.

Note:

The reference code automatically updates the database when it detects that the existing database version is behind the version supported by the SMW Library. The SMW Library distribution also includes a script that can be used to perform database upgrades manually. The SMW User Guide, section 4.3, contains examples of how the script can be used.

3.2.4 Example

CAUTION:

This method overwrites the configuration set in file `smw.conf`.

```
/*
 * Example of database, here database is lost at reboot because
 * it's in /var/tmp
 */
#define DEFAULT_OBJ_DB "/var/tmp/obj_db_smw_test.dat"

/*
```

```

/* TEE TA UUID, it's a unique identifier for the TA, here it's the
/* default TA UUID provided as a reference example.
*/
static const struct tee_info tee_default_info = {
    { "11b5c4aa-6d20-11ea-bc55-0242ac130003" }
};

/*
/* Configure the Secure Element Subsystem storage parameters for ELE
/* Subsystem.
/* The storage_shared flag indicates if the storage is shared between
/* multiple applications.
*/
static const struct se_info se_default_info = { .storage_id = 0x534d5754,
                                                .storage_nonce = 0x444546,
                                                .storage_shared = true };

int main(int argc, char *argv[])
{
    int res = ERR_CODE(FAILED);

    // Configure the TEE Subsystem: TA UUID (and so key storage)
    res = smw_osal_set_subsystem_info(SMW_SUBSYSTEM_NAME_TEE, &tee_default_info,
                                     sizeof(tee_default_info));

    if (res != SMW_STATUS_OK)
        goto exit;

    // Configure the ELE Subsystem: Key storage identifier and shared flag
    res = smw_osal_set_subsystem_info(SMW_SUBSYSTEM_NAME_ELE, &se_default_info,
                                     sizeof(se_default_info));

    if (res != SMW_STATUS_OK)
        goto exit;

    // Open/Create the application object database
    res = smw_osal_open_obj_db(DEFAULT_OBJ_DB, strlen(DEFAULT_OBJ_DB) + 1);
    if (res != SMW_STATUS_OK)
        goto exit;

    // Load and initialize the library. OSAL is loading the application
    // SMW configuration file defined by the system environment variable
    // 'SMW_CONFIG_FILE'
    res = smw_osal_lib_init();
    if (res != SMW_STATUS_OK)
        goto exit;

    // Execute the application
    ...

    exit:

    return res;
}

```


4 Subsystems Overview

4.1 TEE Subsystem

The TEE subsystem provides a secure execution environment based on ARM TrustZone technology. It uses OPTEE OS (Open Portable Trusted Execution Environment) to isolate security-critical operations from the normal world operating system.

The TEE subsystem interfaces with the SMW Library through the OP-TEE Client library and communicates with a Trusted Application (TA) running in the TrustZone secure world.

Supported Platforms: All i.MX platforms with TrustZone support

4.2 ELE Subsystem

The ELE subsystem is NXP's dedicated security subsystem that provides hardware-based cryptographic operations and secure key management. It offers enhanced security features including secure boot, secure storage, and cryptographic acceleration.

The ELE subsystem supports both standard cryptographic operations and specialized features such as EdgeLock 2GO provisioning and NVM Secure Storage.

Supported Platforms: i.MX8ULP, i.MX9x series

Warning:

ELE FW 2.0.7 - EL2GO Key Service Limitation

*ELE FW 2.0.7 introduces a limitation on devices where the EL2GO provisioned keys are missing their Key Check Values (KCV), causing certain crypto services to be disabled. This affects **only** i.MX952, i.MX937, i.MX94x and i.MX95 B1 devices that were not delivered with the KCV fix.*

The following EL2GO keys are affected:

Table 17. Affected EL2GO Keys

EL2GO Key Name	Key ID	Key Type	Description
NXP_DIE_ID_AUTH_PRK	0x7FFF816C	ECDSA NIST P-384	Key used to identify the device when establishing the EdgeLock 2GO connection. Key is used to sign the EdgeLock 2GO TLS self-signed Certificate.
NXP_DIE_ATTEST_AUTH_PRK	0x7FFF8173	ECDSA NIST P-384	Key used to attest the authenticity of asset exchange between Client and Server.
IOT_DIE_ATTEST_AUTH_PRK	0x7FFF8174	ECDSA NIST P-384	Key used to sign the public key attestation certificate.

Affected devices can only be identified at runtime by calling [smw_device_attestation\(\)](#). If it returns `SMW_STATUS_OPERATION_DISABLED`, the device lacks KCV.

Table 18. Disabled operations per EL2GO key

EL2GO Key	Affected Operations
NXP_DIE_ID_AUTH_PRK	smw_sign() , smw_verify() , smw_export_key()
NXP_DIE_ATTEST_AUTH_PRK	smw_verify() , smw_export_key()
IOT_DIE_ATTEST_AUTH_PRK	smw_export_key() , smw_key_attestation()

In addition, the device attestation service ([smw_device_attestation\(\)](#)) is disabled on affected devices and returns `SMW_STATUS_OPERATION_DISABLED`.

4.2.1 EdgeLock Accelerator (ELA)

The ELA is a cryptographic accelerator accessible in the ELE. It provides hardware-accelerated cryptographic operations for improved performance on supported platforms.

ELA acceleration is enabled when all of the below mentioned conditions are met:

- The `USE_ELA` tag is defined in the config file (see [USE_ELA](#))
- The cryptographic operation (algorithm/mode) is supported by ELA
- Plaintext key buffers are provided (opaque keys are not supported)

Supported Platforms: i.MX94x (i.MX943, i.MX942, i.MX941), i.MX952, i.MX937

4.3 SECO Subsystem

SECO is NXP's first-generation secure enclave for i.MX8 platforms. It provides hardware-based security services including cryptographic operations and secure key storage.

Supported Platforms: i.MX8QXP

5 Secure Subsystems Features and Capabilities

The Secure Subsystems Features and Capabilities section provides detailed documentation of the cryptographic and security operations supported by various subsystems including TEE (Trusted Execution Environment), ELE (EdgeLock Enclave) and SECO (Security Controller - First MPU Secure Enclave) subsystems.

5.1 Key and Data Storage

5.1.1 Overview

The SMW library provides storage capabilities for managing keys and data objects across different subsystems. Each subsystem may support different storage types and persistence models.

5.1.2 Storage Types

The library supports two main storage types:

5.1.2.1 Persistent Storage

Persistent storage maintains keys and data objects across system reboots and power cycles. Objects stored persistently are saved to non-volatile memory.

5.1.2.2 Transient Storage

Transient storage maintains keys and data objects only during the current session. Objects are lost when the system is powered down or the library context is destroyed.

5.1.3 Subsystem Storage Support

The different subsystems provide varying levels of storage support as listed in following table:

Table 19. Storage Capabilities by Subsystem

Subsystem	Persistent Storage				Transient Storage Location
	Location	Shareable	Anti-Rollback	Re-Provision	
ELE	NVM	Yes	Yes	Yes	Secure Enclave Volatile Memory
SECO	NVM	No	Yes	Yes but not support by SMW	Secure Enclave Volatile Memory
TEE	REE	No	No	No	TA Volatile Memory
	RPMB	No	Yes	No	TA Volatile Memory

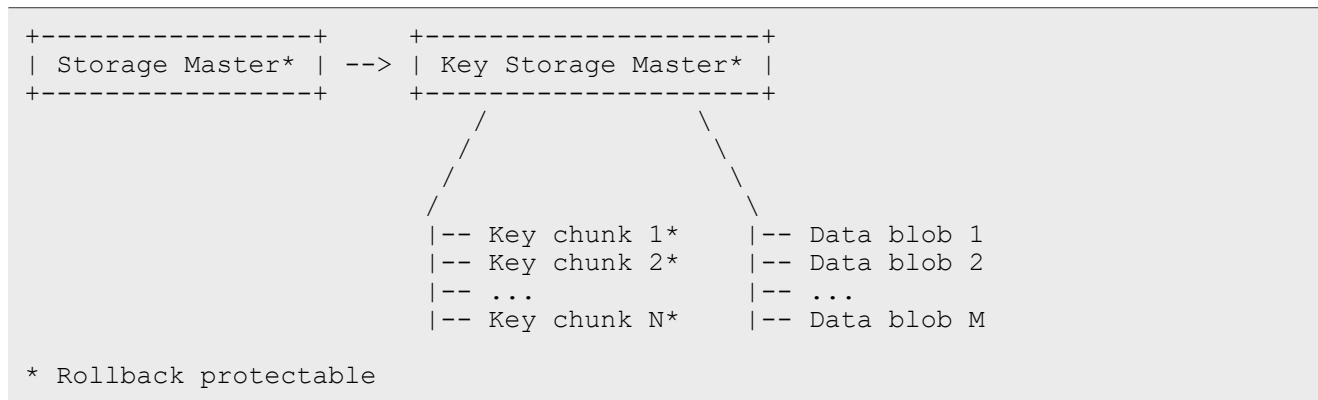
5.1.4 Configuration

Storage configuration is managed through (see [OS-specific implementation](#))

5.1.4.1 ELE Subsystem

The EdgeLock Enclave (ELE) subsystem supports both persistent and transient storage types. Persistent storage is backed by non-volatile memory (NVM) managed by the ELE firmware, while transient storage is maintained in ELE volatile memory during the active session.

The NVM Storage is organized in a storage master to which is linked a key storage master. The key storage master manages the storage of both encrypted key chunks (group of keys) and data blobs (one data per blob).



5.1.4.1.1 Rollback protection

The anti-rollback feature provides protection against downgrade attacks by maintaining version counters for stored objects. When enabled, the ELE Firmware prevents loading of objects with lower version numbers than previously stored versions. The anti-rollback counter is stored in secure fuses and duplicated in the Storage Master blob of the NVM Secure Storage. By propagation, the rollback protection extends to the Key Storage Master and all key chunks.

The data blobs even if linked to the Key Storage Master are not protected.

The rollback protection is increased when the commit operation is performed.

Table 20. Storage commit APIs Comparison

API	Function
SMW	smw_commit_key_storage()
PSA	Not supported
PKCS11	Not supported

5.1.4.1.2 Storage Reprovisioning

The Storage reprovisioning APIs allow users to create a new secure enclave NVM Secure Storage to replace the storage that is rollback protected.

The storage reprovisioning operation is performed through the following APIs:

Table 21. Storage Reprovisioning APIs Comparison

API	Function
SMW	smw_device_reprovision()
PSA	Not supported
PKCS11	Not supported

The reprovisioning process consists of two steps in the **same power cycle**:

- Step 1: If SMW database exists, it must be cleaned to ensure all ELE persistent objects are destroyed. The simple way is:
 - Unloading the SMW library (close all applications using NVM Secure Enclave storage to replace).

- Removing the SMW's database file.
- Reloading the SMW library (restart the application) and perform key provisioning (e.g. key generation, key import)
- Step 2: Send re-provisioning request to EdgeLock Secure Enclave:
 - Generate the ELE re-provisioning signed message. The function `smw_device_reprovision_prepare()` can be used to prepare the message to be signed with signing tool like NXP [SPSDK](#) tool. The documentation is available [here](#). The command used to sign is [nxpimage](#).
 - Send the request to ELE Secure Enclave to allow re-provisioning by calling the `smw_device_reprovision()` function. Upon success, the SMW Library requests the creation of the fresh EdgeLock Secure Storage.

Warning:

All keys and data previously stored in the NVM Secure Storage will be erased during the reprovisioning process. The SMW database for persistent object will be un-synchronized if not erased or cleaned up.

Important:

The new created NVM Secure Storage is empty but must not be removed as the anti-rollback counters are stored in the secure fuses and can't be reset. The new create NVM Secure Storage contains specific information that rollback protection is not enabled with the storage until next storage commitment.

5.1.4.1.3 Storage Capacities**Table 22. ELE Key Storage Capacities**

Nb Storage	Type	Key storage	
		max keys/storage	nb chunk
2	Persistent	100	50 (SMW defined)
	Transient		50 (SMW defined)

Table 23. ELE Data Storage Capacities

max size/blob	nb blobs
2 Kbytes	Limited by the available OS storage

5.1.4.2 SECO Subsystem

The Security Controller (SECO) subsystem supports the same Secure Storage concept as ELE. SECO provides both persistent and transient storage types with similar organizational structure and anti-rollback protection capabilities. The SECO NVM Storage follows the same master-to-key storage hierarchy as ELE, with a Storage Master linked to a Key Storage Master that manages encrypted key chunks and data blobs.

5.1.4.2.1 Rollback protection

The anti-rollback protection mechanism in SECO operates identically to ELE, maintaining version counters in secure fuses and duplicating them in the Storage Master blob. This ensures that key chunks and the Key Storage Master are protected against rollback attacks, while data blobs remain unprotected by the anti-rollback feature.

The difference between ELE and SECO subsystem is that on SECO each persistent key creation increment the rollback protection counter up to the limit of the max counter update define during the creation of the key storage. When the limit of the counter is reached, rollback protection is disabled for subsequent key creations.

5.1.4.2.2 Storage Reprovisioning

The Storage reprovisioning is possible for SECO subsystem but not supported by the SMW library for this subsystem.

5.1.4.2.3 Storage Capacities

Table 24. SECO Key Storage Capacities

Nb Storage	Type	Key storage	
		max bytes/chunk	nb chunk
1	Persistent	4 Kbytes	512 (SMW defined)
	Transient		512 (SMW defined)

Table 25. SECO Data Storage Capacities

Data storage	
max size/blob	nb blobs
2 Kbytes	Limited by the available OS storage

5.1.4.3 TEE Subsystem

The Trusted Execution Environment (TEE) subsystem, based on TrustZone OPTEE OS, supports persistent storage through the REE (Rich Execution Environment) filesystem or RPMB (Replay Protected Memory Block) partition of MMC/SD card. The REE storage is the default option, use of the RPMB is selectable at compilation time. More details about TEE storage can be found in the [OP-TEE documentation - Secure storage](#).

The persistent storage is isolated and encrypted by OPTEE OS, ensuring data confidentiality and integrity for each Trusted Application instance.

The transient storage is maintained in volatile memory during the active loaded Trusted Application (TA) session and is lost when the TA is unloaded or the system is powered down.

The persistent storage rollback protection is applicable only in case of RPMB usage, while REE filesystem storage does not provide anti-rollback.

Key or data are stored as objects in the persistent storage. The maximum size of a single object is limited by the available storage space.

5.1.4.3.1 Storage Capacities

Table 26. TEE Storage Capacities

Nb Storage	Key/data storage	
	max keys/storage	max size/object
1 per TA	Limited by the available TEE Storage and performance to load/unload objects	

5.1.5 Bad Practice

In OS environment where subsystem storages are files stored in the filesystem, manually deleting these files should be avoided as it will cause inconsistency between SMW's database and secure storage content.

The best practice is to call the [Delete Key](#) and the [Delete Data](#) APIs to delete keys and data objects respectively, ensuring database consistency and proper cleanup of secure storage resources.

5.2 Key Management

This section documents the key management operations supported across different security subsystems (ELE, TEE, SECO) with their respective key types and sizes.

5.2.1 Supported Operations versus Subsystems

Table 27. Key Management Operations vs. subsystem

Operations		Subsystem			Notes
		ELE	TEE	SECO	
Generate Key		Y	Y	Y	
Delete Key		Y	Y	Y	
Import Key	Plaintext	N	Y	N	
	EdgeLock 2GO blob	Y	N	N	
	EdgeLock Enclave blob	Y	N	N	
Export Public Key		Y	Y	Y	
Key Derivation	HKDF	Y*	Y	N	ELE HKDF is only supported on i.MX937, i.MX943 and i.MX952.
Key Agreement	ECDH	N	Y	N	
	OEM Master key	Y	N	N	
TLS 1.2 (PRF)		Y	N	Y*	SECO Require specific Firmware.
TLS 1.3 (TLS13-KDF)		Y	N	N	
Get key attributes		Y	Y	Y*	SECO managed limited attributes.
Public key attestation		Y	N	N	

5.2.2 Asymmetric Keys

This section documents the asymmetric keys supported across different security subsystems (ELE, TEE, SECO) with their respective key sizes in bits.

5.2.2.1 Secp R1

Table 28. Secp R1 vs. subsystem

Subsystem	Security size in bits
ELE	192 / 224 / 256 / 384 / 521
TEE	192 / 224 / 256 / 384 / 521
SECO	256 / 384

5.2.2.2 Brainpool R1

Table 29. Brainpool R1 vs. subsystem

Subsystem	Security size in bits
ELE	224 / 256 / 384
TEE	Not supported
SECO	256 / 384

5.2.2.3 Twisted Edwards

Aka. ED25519 and ED448

Table 30. Twisted Edwards vs. subsystem

Subsystem	Security size in bits
ELE	255 / 448
TEE	255
SECO	Not supported

5.2.2.4 Montgomery

Aka. X25519 and X448

Table 31. Montgomery vs. subsystem

Subsystem	Security size in bits
ELE	255 / 448
TEE	Not supported
SECO	Not supported

5.2.2.5 ShāngMì 2

Table 32. SM2 vs. subsystem

Subsystem	Security size in bits
ELE	256
TEE	256
SECO	Not supported

5.2.2.6 RSA Keys

Table 33. RSA vs. subsystem

Subsystem	Security size in bits	Comment
ELE	2048 / 3072 / 4096	Not supported on i.MX8ULP.
TEE	256 to 4096 (multiple of 2 bits)	
SECO	Not supported	

5.2.3 Symmetric Keys

This section documents the symmetric keys supported across different security subsystems (ELE, TEE, SECO) with their respective key sizes in bits.

5.2.3.1 AES

Table 34. AES vs. subsystem

Subsystem	Security size in bits
ELE	128 / 192 / 256
TEE	128 / 192 / 256
SECO	128 / 192 / 256

5.2.3.2 DES

Table 35. DES vs. subsystem

Subsystem	Security size in bits
ELE	Not supported
TEE	56
SECO	Not supported

5.2.3.3 Triple-DES

Aka. DES3

Table 36. Triple-DES vs. subsystem

Subsystem	Security size in bits
ELE	Not supported
TEE	112 / 168
SECO	Not supported

5.2.3.4 SM4

Table 37. SM4 vs. subsystem

Subsystem	Security size in bits
ELE	Not supported
TEE	128
SECO	Not supported

5.2.3.5 HMAC

Table 38. HMAC vs. subsystem

Subsystem	Security size in bits	Comment
ELE	224 / 256 / 384 / 512	
TEE	64 to 1024 bits (multiple of 8 bits)	
SECO	224 / 256 / 384 / 512	Require specific Firmware.

Table 38. HMAC vs. subsystem...continued

Subsystem	Security size in bits	Comment
		Build option ENABLE_TLS must be set ON.

5.2.4 Generate Key

The **generate key** operation is used to generate cryptographic persistent or transient keys within a secure subsystem storage.

Table 39. Generate Key APIs Comparison

Key Type	API	Function / Key Type or Mechanism
Asymmetric Keys		
Secp R1	SMW	smw_generate_key() SMW_KEY_TYPE_NAME_SECP_R1
	PSA	psa_generate_key() PSA_ECC_FAMILY_SECP_R1
	PKCS11	C_GenerateKeyPair() CKM_EC_KEY_PAIR_GEN
Brainpool R1	SMW	smw_generate_key() SMW_KEY_TYPE_NAME_BRAINPOOL_R1
	PSA	psa_generate_key() PSA_ECC_FAMILY_BRAINPOOL_P_R1
	PKCS11	Not defined
Twisted Edwards (ED25519/ED448)	SMW	smw_generate_key() SMW_KEY_TYPE_NAME_ED25519 SMW_KEY_TYPE_NAME_ED448
	PSA	psa_generate_key() PSA_ECC_FAMILY_TWISTED_EDWARDS
	PKCS11	C_GenerateKeyPair() CKM_EC_EDWARDS_KEY_PAIR_GEN
Montgomery (X25519/X448)	SMW	smw_generate_key() SMW_KEY_TYPE_NAME_X25519 SMW_KEY_TYPE_NAME_X448
	PSA	psa_generate_key() PSA_ECC_FAMILY_MONTGOMERY
	PKCS11	C_GenerateKeyPair() CKM_EC_MONTGOMERY_KEY_PAIR_GEN
RSA	SMW	smw_generate_key() SMW_KEY_TYPE_NAME_RSA
	PSA	psa_generate_key() PSA_KEY_TYPE_RSA_KEY_PAIR
	PKCS11	C_GenerateKeyPair() CKM_RSA_PKCS_KEY_PAIR_GEN CKM_RSA_X9_31_KEY_PAIR_GEN
SM2	SMW	smw_generate_key()

Table 39. Generate Key APIs Comparison...continued

Key Type	API	Function / Key Type or Mechanism
		SMW_KEY_TYPE_NAME_SM2
	PSA	Not defined
	PKCS11	Not defined
Symmetric Keys		
AES	SMW	smw_generate_key() SMW_KEY_TYPE_NAME_AES
	PSA	psa_generate_key() PSA_KEY_TYPE_AES
	PKCS11	C_GenerateKey() CKM_AES_KEY_GEN
DES	SMW	smw_generate_key() SMW_KEY_TYPE_NAME_DES
	PSA	psa_generate_key() PSA_KEY_TYPE_DES
	PKCS11	C_GenerateKey() CKM_DES_KEY_GEN
Triple-DES	SMW	smw_generate_key() SMW_KEY_TYPE_NAME_DES3
	PSA	psa_generate_key() PSA_KEY_TYPE_DES3
	PKCS11	C_GenerateKey() CKM_DES3_KEY_GEN
SM4	SMW	smw_generate_key() SMW_KEY_TYPE_NAME_SM4
	PSA	psa_generate_key() PSA_KEY_TYPE_SM4
	PKCS11	C_GenerateKey() CKM_SM4_KEY_GEN (NXP vendor define)
HMAC	SMW	smw_generate_key() SMW_KEY_TYPE_NAME_HMAC
	PSA	psa_generate_key() PSA_KEY_TYPE_HMAC
	PKCS11	C_GenerateKey() CKM_GENERIC_SECRET_KEY_GEN
DERIVE	SMW	smw_generate_key() SMW_KEY_TYPE_NAME_DERIVE
	PSA	psa_generate_key() PSA_KEY_TYPE_DERIVE
	PKCS11	Not defined

5.2.5 Delete Key

The **delete key** operation allows removing a previously generated or imported persistent or transient key from the subsystem system storage.

Table 40. Delete Key APIs Comparison

API	Function
SMW	smw_delete_key()
PSA	psa_destroy_key()
PKCS11	C_DestroyObject()

5.2.6 Import Key

The **import key** operation allows importing a key material (plaintext or encrypted) in the subsystem system storage. The key can be persistent or transient.

5.2.6.1 Plaintext

Table 41. Import Plaintext Key APIs Comparison

Key Type	API	Function / Key Type
Asymmetric Keys		
Secp R1	SMW	smw_import_key() SMW_KEY_TYPE_NAME_SECP_R1
	PSA	psa_import_key() PSA_ECC_FAMILY_SECP_R1
	PKCS11	C_CreateObject() CKK_EC
Brainpool R1	SMW	smw_import_key() SMW_KEY_TYPE_NAME_BRAINPOOL_R1
	PSA	psa_import_key() PSA_ECC_FAMILY_BRAINPOOL_P_R1
	PKCS11	Not defined
Twisted Edwards (ED25519/ED448)	SMW	smw_import_key() SMW_KEY_TYPE_NAME_ED25519 SMW_KEY_TYPE_NAME_ED448
	PSA	psa_import_key() PSA_ECC_FAMILY_TWISTED_EDWARDS
	PKCS11	C_CreateObject() CKK_EC_EDWARDS
Montgomery (X25519/X448)	SMW	smw_import_key() SMW_KEY_TYPE_NAME_X25519 SMW_KEY_TYPE_NAME_X448
	PSA	psa_import_key() PSA_ECC_FAMILY_MONTGOMERY
	PKCS11	C_CreateObject() CKK_EC_MONTGOMERY

Table 41. Import Plaintext Key APIs Comparison...continued

Key Type	API	Function / Key Type
RSA	SMW	smw_import_key() SMW_KEY_TYPE_NAME_RSA
	PSA	psa_import_key() PSA_KEY_TYPE_RSA_KEY_PAIR
	PKCS11	C_CreateObject() CKK_RSA
Symmetric Keys		
AES	SMW	smw_import_key() SMW_KEY_TYPE_NAME_AES
	PSA	psa_import_key() PSA_KEY_TYPE_AES
	PKCS11	C_CreateObject() CKK_AES
DES	SMW	smw_import_key() SMW_KEY_TYPE_NAME_DES
	PSA	psa_import_key() PSA_KEY_TYPE_DES
	PKCS11	C_CreateObject() CKK_DES
Triple-DES	SMW	smw_import_key() SMW_KEY_TYPE_NAME_DES3
	PSA	psa_import_key() PSA_KEY_TYPE_DES3
	PKCS11	C_CreateObject() CKK_DES3
SM4	SMW	smw_import_key() SMW_KEY_TYPE_NAME_SM4
	PSA	psa_import_key() PSA_KEY_TYPE_SM4
	PKCS11	C_CreateObject() CKK_SM4 (NXP vendor define)
HMAC	SMW	smw_import_key() SMW_KEY_TYPE_NAME_HMAC
	PSA	psa_import_key() PSA_KEY_TYPE_HMAC
	PKCS11	C_CreateObject() CKK_GENERIC_SECRET CKK_MD5_HMAC CKK_SHA_1_HMAC CKK_SHA224_HMAC CKK_SHA256_HMAC CKK_SHA384_HMAC

Table 41. Import Plaintext Key APIs Comparison...continued

Key Type	API	Function / Key Type
		CKK_SHA512_HMAC
		CKK_SHA3_224_HMAC
		CKK_SHA3_256_HMAC
		CKK_SHA3_384_HMAC
		CKK_SHA3_512_HMAC

5.2.6.2 EdgeLock 2GO blob

This service is a NXP provisioning service involving the NXP EdgeLock 2GO server. It's only supported by the EdgeLock Enclave (ELE).

The server allows to provision key and data embedded in blob. Visit [EdgeLock 2GO](#).

The type of key blob to import is identified by the key location attribute:

- SMW API, the key location is defined by the 'storage_id' field of the key attribute in the key descriptor (refer to the [smw_key_descriptor](#)).
- PSA API, the key location is defined by the 'lifetime' field of the key attribute (refer to the [psa_key_attributes_t](#)). The location is defined by the bits[31:8] of the field.

Table 42. EdgeLock 2GO - Location

	SMW 'storage_id'	PSA 'lifetime' bits[31:8]
EdgeLock 2GO Key	0x00E00000	0xE00000
EdgeLock 2GO Data	0x00E08000	0xE08000

The SMW or PSA key import API allows to import EdgeLock 2GO blob of type key or data.

5.2.6.3 EdgeLock Enclave blob

This service is a NXP service provided by the EdgeLock Secure Enclave Firmware. A EdgeLock Enclave blob is used to import secure keys. The blob contains the key attributes, the wrapped key (AES CBC padded or RFC 3394) and a blob signature (CMAC).

The wrapping key and the signing key are both derived from the [OEM Master key](#). On device both keys are derived and kept internal, on host side, the OEM Master key, the wrapping key and signing key must be derived using the HMAC two steps standard [RFC5869](#).

OEM import WRAP and CMAC keys derivation parameters:

- The salt used in the first step: extract can be either:
 - A zeros string. Its length, in bytes, is equal to the hash algorithm (used in the key derivation process) length byte size.
 - Device SRKH. As salt length is the hash algorithm (used in the key derivation process) length byte size, device SRKH value could be truncated or concatenated with 0s to match the required salt length.
- The IKM is the [OEM Master key](#) shared secret key
- The fixed info in a byte string imposed by EdgeLock Secure Enclave Firmware. It differs switch derived key:
 - For wrapping key, fixed info is: "oemelefwkeyimportwrap256" string (quotation marks not included).
 - For signing key, fixed info is: "oemelefwkeyimportcmac256" string (quotation marks not included).
- The key derivation algorithm is imposed by EdgeLock Secure Enclave Firmware as HKDF_SHA256.

The EdgeLock Enclave blob is encoded as described in [EdgeLock Enclave blob](#) table.

Table 43. EdgeLock Enclave blob

Field		Tag	Length (bytes)	Description / Value
Magic		0x40	21	EdgeLock Secure Enclave identification blob. Value is the hexadecimal string "edgelockenclaveimport". [65 64 67 65 6c 6f 63 6b 65 6e 63 6c 61 76 65 69 6d 70 6f 72 74]
Key ID		0x41	4	Key identifier in the subsystem: • 0x0 if the key is transient. • between 0x1 and 0x3FFFFFFF if the key is persistent.
Key properties	Algorithm	0x42	4	ELE key permitted algorithm.
	Usage	0x43	4	ELE key usage flags.
	Type	0x44	2	ELE Type of key.
	Bits	0x45	4	Key security size in bits.
	Lifetime	0x46	4	ELE Lifetime: • 0xC0020000, transient key • 0xC0020001, persistent key • 0xC00200FF, permanent key
Key lifecycle		0x47	4	ELE device lifecycle flags when key is usable: • OEM OPEN: 0x01 • OEM CLOSED: 0x02 • OEM CLOSED_LOCKED: 0x04
OEM Master key identifier		0x50	4	OEM Master key identifier resulting OEM Master key .
Wrapping algorithm		0x51	4	Wrapping algorithm of the key blob. This field is required to distinguish between different flavors of wrapping algorithms. Possible values are: • 0x01: RFC 3394 wrapping. • 0x02: AES-CBC wrapping (padding cipher, ISO7816-4 Appendix C).
IV		0x52	16	IV to use for CBC wrapping. Not used if wrapping algorithm not equal 0x02.
Signing algorithm		0x54	4	Algorithm used to sign the blob itself. Field Signature of this blob. It must be 0x01 (CMAC).
Wrapped private key		0x55	Variable	Private key data in encrypted format as defined by the Wrapping Algorithm. Key used to do the encryption must be wrapping key derived from the OEM Master key.
Signature		0x5E	16	Signature of all previous fields of this blob including the signature tag (0x5E) and signature length fields. Key used to do the signature must be signing key derived from the OEM Master key.

5.2.6.3.1 Usage Example (with OEM Master key derivation)

Here's an example of host source code C to derive the OEM Master key

In this example:

- [MbedTLS Library](#) is used as third party cryptographic service.
- ECDSA NIST 256 bits keypair are used on device and host side as based key of the OEM Master key agreement.

- ECDH + HKDF-SHA256 algorithm is used for the key agreement operation.

Note:

The following code is not complete, failure must be handled correctly and resource freed. The following examples aim to give the guidelines to write application on host and device.

1. **On device**, generates and exports an ECDSA NIST 256 bits keypair:

The SMW API [smw_generate_key\(\)](#) is used to generate the key. The key is stored as a transient key. The public key is exported if the operation success. The public key is the device peer key that will be used to derive the OEM Master key on the host side.

This key identifier will be identified in this example by `ELE_base_id`.

The public key of this key will be identified in this example by `ELE_peer_key`.

```
int res = -1;
enum smw_status_code status = SMW_STATUS_OK;
struct smw_generate_key_args args = { 0 };
struct smw_key_descriptor key = { 0 };
struct smw_key_attributes *attributes = &key.attributes;
struct smw_keypair_buffer buffer = { 0 };

/* Allocate the public key buffer of the key */
buffer.gen.public_length = 64;
buffer.gen.public_data = malloc(buffer.gen.public_length);
if (!buffer.gen.public_data)
    return res;

args.subsystem_name = SMW_SUBSYSTEM_NAME_ELE;
args.key_descriptor = &key;

key.type_name = SMW_KEY_TYPE_NAME_SECP_R1;
key.security_size = 256;
key.buffer = &buffer;

attributes.permitted_algo = SMW_ATTR_ALGO_KEY_AGREEMENT(SMW_ATTR_ALGO_ECDH,
                                                         SMW_ATTR_ALGO_HKDF,
                                                         SMW_ATTR_HASH_SHA256);
attributes.usage_flags = SMW_ATTR_USAGE_DERIVE;
attributes.attributes = SMW_ATTR_SET_PERSISTENCE(attributes.attributes,
                                                  SMW_ATTR_PERSISTENCE_TRANSIENT);

status = smw_generate_key(&args);
if (status != SMW_STATUS_OK) {
    free(buffer.gen.public_data);
    return -1;
}

/*
 * At this stage:
 * - ``ELE_peer_key`` is the buffer.gen.public_data
 * - ``ELE_base_id`` is the identifier returned in key.id
 */
...
```

2. **On host**, generates and exports an ECDSA NIST 256 bits keypair:

The public key is exported if the operation success. The public key is the host peer key that will be used to derive the OEM Master key on the device side.

In this example, the Host peer key is named `Host_peer_key`.

```
int res = -1;
mbedtls_pk_context key;
mbedtls_entropy_context entropy;
mbedtls_ctr_drbg_context ctr_drbg;
mbedtls_ecp_group grp;
mbedtls_ecp_point Host_base_key;
unsigned char Host_peer_key[MBEDTLS_ECP_MAX_PT_LEN];
mbedtls_pk_type_t pk_alg = MBEDTLS_PK_ECKEY;
size_t len = 0;

mbedtls_pk_init(&key);
mbedtls_entropy_init(&entropy);
mbedtls_ctr_drbg_init(&ctr_drbg);

res = mbedtls_pk_setup(&key, mbedtls_pk_info_from_type(pk_alg));
if (res)
    /* Failure */

res = mbedtls_ctr_drbg_seed(&ctr_drbg, mbedtls_entropy_func,
                           &entropy,
                           (const unsigned char *) "ecdsa",
                           strlen(pers));

if (res)
    /* Failure */

res = mbedtls_ecp_gen_key(MBEDTLS_ECP_DP_SECP256R1,
                           mbedtls_pk_ec(key),
                           mbedtls_ctr_drbg_random,
                           &ctr_drbg);

if (res)
    /* Failure */

mbedtls_ecp_group_init(&grp);
mbedtls_ecp_point_init(&pt);

res = mbedtls_ecp_export(ecp, &grp, NULL, &Host_base_key);
if (res)
    /* Failure */

res = mbedtls_ecp_point_write_binary(&grp, &Host_base_key,
                                     MBEDTLS_ECP_PF_UNCOMPRESSED,
                                     &len, Host_peer_key,
                                     sizeof(Host_peer_key));

if (res)
    /* Failure */

/*
 * At this stage the ``Host_peer_key`` is the public key value starting with
 * the uncompress tag 0x4.
 */

...
```

3. On device, derive the OEM Master key:

- `Host_peer_key` is the host peer public key buffer.
- `ELE_base_id` is the ELE base key identifier.

Fixed info is optional. If set, its digest must be present in the signed content payload.

Signed content payload must be configured as explained in [OEM Master key](#) and where the peer public key digest must be the result of `SHA256(Host_peer_key)`. This signed content payload key will be identified in this example by `OEM_Payload`.

In this example, the OEM Master key is a persistent key with a key identifier named `ELE_OEM_MK_ID`. It's assumed that the signed payload is valid with slat flags set to 0 (no salt used to derived OEM Master key, wrapping and signing keys).

In this example, the fixed info is not set.

```
int res = -1;
enum smw_status_code status = SMW_STATUS_OK;
struct smw_derive_key_args args = { 0 };
struct smw_kdf_oem_master_key_args oem_mk_args = { 0 };
struct smw_key_descriptor key_base = { 0 };
struct smw_derived_key_descriptor key_derived = { 0 };

args.subsystem_name = SMW_SUBSYSTEM_NAME_ELE;
args.key_descriptor_base = &key_base;
args.kdf_name = SMW_KDF_NAME_OEM_MASTER_KEY;
args.kdf_arguments = &oem_mk_args;
args.store_derived_key = true; /* Request to store the key in Secure Storage
*/
args.key_descriptor_base = &key_base;
args.key_descriptor_derived = &key_derived;

oem_mk_args.op = SMW_OEM_MK_OP_NAME_DERIVE;
oem_mk_args.peer_public_buffer = Host_peer_key;
oem_mk_args.peer_public_buffer_length = sizeof(Host_peer_key);
oem_mk_args.payload = OEM_Payload;
oem_mk_args.payload_length = sizeof(OEM_Payload);

key_base.id = ELE_base_id;

key_derived.type_name = SMW_KEY_TYPE_NAME_DERIVE;
key_derived.security_size = 256;
key_derived.id = ELE_OEM_MK_ID;
key_derived.attributes.permitted_algo =
    SMW_ATTR_ALGO_KEY_DERIVATION_HKDF(SHA256);
key_derived.attributes.usage_flags = SMW_ATTR_USAGE_DERIVE;
key_derived.attributes.attributes = SMW_ATTR_PERSISTENCE_PERSISTENT;

status = smw_derive_key(&args);
if (status != SMW_STATUS_OK)
    return -1;

/*
 * At this stage, the OEM Master key is created in the EdgeLock Enclave,
 * as persistent key identifier ``ELE_OEM_MK_ID``
 */
```

Note:

The wrapping and signing keys of the key import blob are not derived at this step but each time the Key Import API is executed.

4. **On host**, generates same OEM Master key:

This operation is done in two steps: key agreement and key derivation.

In this example, the host OEM Master key identifier will be named `Host_OEM_MK_ID`.

a. Key agreement:

This step generates a shared secret that will be used as input of key derivation step.

The algorithm used is ECDH.

In this example, the base key is the key generated in previous step and named `Host_base_key`. The peer key is the device public key generated in previous step and named `ELE_peer_key`.

Here's a sample of code that can be used to generate the shared secret:

```
int res = -1;
uint32_t peer_key_size = (256 / 8) * 2;
uint32_t mbed_peer_key_size = peer_key_size + 1;
uint8_t peer_key[mbed_peer_key_size];
mbedtls_ecp_group grp;
mbedtls_ecp_point peer_pt;
mbedtls_mpi ecdh;

/* Prepare public key */
memset(&grp, 0, sizeof(grp));
res = mbedtls_ecp_group_load(&grp, MBEDTLS_ECP_DP_SECP256R1);
if (res != 0)
    return failure;

/* Prepare public key, a 0x04 leading byte must be added */
peer_key[0] = 0x04;
memcpy(peer_key + 1, ELE_peer_key, peer_key_size);

/* Load public key from buffer */
mbedtls_ecp_point_init(&peer_pt);
res = mbedtls_ecp_point_read_binary(&grp, &peer_pt, peer_key,
                                     mbed_peer_key_size);

if (res != 0)
    /* Failure */

/* Compute ECDH */
mbedtls_mpi_init(&ecdh);

/*
 * The ``Host_base_key`` is the result of the key generated in
 * previous step.
 */
res = mbedtls_ecdh_compute_shared(&grp, &ecdh, &peer_pt,
                                  &Host_base_key.private_d,
                                  my_mbedtls_rand, NULL);

if (res != 0)
    /* Failure */

/*
 * At this point, the shared secret is the buffer located at
 * ecdh.private_p. Its size in bytes is 8 * ecdh.private_n
 * (ecdh.private_n is the size in limbs, i.e number of 64 bits blocks).
 *
 * CAUTION: Switch your environment configuration, MbedTLS may store
 * the shared secret ecdh.private_p in little endian BUT we need it
 * in big endian for the next step (key derivation).
 * Then a little endian to big endian conversion is needed.
 */
...
```

Note:

*my_mbedtls_rand must be a user function with the following definition: int my_mbedtls_rand(void *rng_state, unsigned char *output, size_t len). The function fills output buffer with a random value of len bytes. It must return 0 on success. Implementation depends on your environment.*

b. Key derivation:

In this step, the shared secret buffer generated in previous step is used as input. Let's identify it as `ecdh` and `ecdh_size` (size in bytes).

Note:

ECDH buffer must be in big endian.

Here's a sample of code that can be used to derive the host OEM Master key named `Host_OEM_MK` key:

```
int res = -1;
const mbedtls_md_info_t *md = NULL;
uint32_t salt_size = 0;
uint8_t *salt = NULL;
uint32_t info_size = 0;
uint8_t *info = NULL;
uint32_t Host_OEM_MK_size = 256 / 8;
uint8_t *Host_OEM_MK[Host_OEM_MK_size];

/* Set md info */
md = mbedtls_md_info_from_type(MBEDTLS_MD_SHA256);
if (md == NULL)
    /* Failure */

/*
 * In this example:
 * Salt is NULL and salt_size is 0: a string of zeros of SHA256 bytes
 * length will be used.
 *
 * Info is NULL and info_size is 0: no fixed info are used in the
 * derivation process.
 *
 * Both Salt and Fixed info could be enabled/set but previous steps
 * must defined the same.
 */
res = mbedtls_hkdf(md, salt, salt_size, ecdh, ecdh_size, info,
                  info_size, Host_OEM_MK, Host_OEM_MK_size);
if (res != 0)
    /* Failure */

...
```

5. On host, derives the wrapping key:

Use third party crypto service (MbedTLS library).

In this step, the OEM Master key is the `Host_OEM_MK` key buffer generated in previous step is used as input.

Here's a sample of code that can be used to derive the wrapping key identified by `Host_OEM_WRAP_key` key:

```
int res = -1;
const mbedtls_md_info_t *md = NULL;
uint32_t salt_size = 0;
uint8_t *salt = NULL;
uint32_t info_size = 24;
uint8_t *info = "oemelefwkeyimportwrap256";
uint32_t Host_OEM_WRAP_key_size = 256 / 8;
```

```

uint8_t *Host_OEM_WRAP_key[Host_OEM_WRAP_key_size];

/* Set md info */
md = mbedtls_md_info_from_type(MBEDTLS_MD_SHA256);
if (md == NULL)
    /* Failure */

/*
 * In this example:
 * Salt is NULL and salt_size is 0: a string of zeros of SHA256 bytes
 * length will be used.
 *
 * Salt could be enabled/set but previous steps must defined the same.
 *
 * Info is imposed by ELE FW.
 */
res = mbedtls_hkdf(md, salt, salt_size, Host_OEM_MK, Host_OEM_MK_size,
                  info, info_size, Host_OEM_WRAP_key,
                  Host_OEM_WRAP_key_size);

if (res != 0)
    /* Failure */

...

```

6. On host, derives the signing key:

Use third party crypto service (MbedTLS library).

In this step, the OEM Master key is the `Host_OEM_MK` key buffer generated in previous step is used as input.

Here's a sample of code that can be used to derive the wrapping key identified by `Host_OEM_CMAC_key` key:

```

int res = -1;
const mbedtls_md_info_t *md = NULL;
uint32_t salt_size = 0;
uint8_t *salt = NULL;
uint32_t info_size = 24;
uint8_t *info = "oemelefwkeyimportcmac256";
uint32_t Host_OEM_CMAC_key_size = 256 / 8;
uint8_t *Host_OEM_CMAC_key[Host_OEM_CMAC_key_size];

/* Set md info */
md = mbedtls_md_info_from_type(MBEDTLS_MD_SHA256);
if (md == NULL)
    /* Failure */

/*
 * In this example:
 * Salt is NULL and salt_size is 0: a string of zeros of SHA256 bytes
 * length will be used.
 *
 * Salt could be enabled/set but previous steps must defined the same.
 *
 * Info is imposed by ELE FW.
 */
res = mbedtls_hkdf(md, salt, salt_size, Host_OEM_MK, Host_OEM_MK_size,
                  info, info_size, Host_OEM_CMAC_key,
                  Host_OEM_CMAC_key_size);

if (res != 0)
    /* Failure */

```

...

7. **On host**, build the EdgeLock Enclave blob to import the key:

The blob must be built by yourself and encoded as described in [EdgeLock Enclave blob](#) table.

Third party crypto service (MbedTLS library) is used to wrap the imported key and signed the blob.

Here's a sample of code that can be used to wrap the imported key with [RFC3394](#) algorithm:

```
uint32_t imported_key_size = 256 / 8;
uint8_t imported_key[imported_key_size];
uint32_t wrap_key_size = imported_key_size + 8; /* RFC3394 algorithm adds 8
bytes */
uint8_t wrap_key[wrap_key_size];
uint32_t out_len = 0;
mbedtls_nist_kw_context kw_ctx;

/* MbedTLS KW context init and setup */
mbedtls_nist_kw_init(&kw_ctx);

res = mbedtls_nist_kw_setkey(&kw_ctx, MBEDTLS_CIPHER_ID_AES,
                            Host_OEM_WRAP_key, 256, 1);
if (res != 0)
    /* Failure */

/* Execute key wrapping operation */
res = mbedtls_nist_kw_wrap(&kw_ctx, MBEDTLS_KW_MODE_KW, imported_key,
                           imported_key_size, wrap_key, &out_len,
                           wrap_key_size);
if (res != 0)
    /* Failure */

...
```

Here's a sample of code that can be used to sign the blob with CMAC algorithm:

```
int res = -1;
const mbedtls_cipher_info_t *cipher_info = NULL;
uint32_t signature_size = 16; /* Default CMAC output size */
uint32_t message_size = tlv_size - signature_size; /* Full TLV buffer size */
uint8_t tlv[tlv_size]; /* The TLV buffer you built previously */
uint8_t *signature_ptr = tlv + message_size;

cipher_info = mbedtls_cipher_info_from_type(MBEDTLS_CIPHER_AES_256_ECB);
if (cipher_info == NULL)
    return failure;

res = mbedtls_cipher_cmac(cipher_info, Host_OEM_CMAC_key, 256, tlv,
                           message_size, signature_ptr);
if (res != 0)
    /* Failure */

...
```

Here's an example of EdgeLock Enclave blob to import an AES 256 bits key:

- identifier = 0x66
- permitted algorithm is defined to permit all ciphers mode
- usage encrypt/decrypt
- key is persistent

- lifecycle is current device lifecycle
- OEM Master key identifier = 0x2

```
uint8_t aes_key_ex[] = {
    0x40, 0x15, 0x65, 0x64, 0x67, 0x65, 0x6c, 0x6f,
    0x63, 0x6b, 0x65, 0x6e, 0x63, 0x6c, 0x61, 0x76,
    0x65, 0x69, 0x6d, 0x70, 0x6f, 0x72, 0x74, 0x41,
    0x04, 0x00, 0x00, 0x00, 0x66, 0x42, 0x04, 0x84,
    0xc0, 0xff, 0x00, 0x43, 0x04, 0x00, 0x00, 0x03,
    0x00, 0x44, 0x02, 0x24, 0x00, 0x45, 0x04, 0x00,
    0x00, 0x01, 0x00, 0x46, 0x04, 0xc0, 0x02, 0x00,
    0x01, 0x47, 0x04, 0x00, 0x00, 0x00, 0x00, 0x50,
    0x04, 0x00, 0x00, 0x00, 0x02, 0x51, 0x04, 0x00,
    0x00, 0x00, 0x01, 0x54, 0x04, 0x00, 0x00, 0x00,
    0x01, 0x55, 0x28, 0x23, 0xb4, 0xa9, 0xf5, 0x9a,
    0x91, 0x9c, 0xd6, 0xfc, 0x6d, 0x27, 0xa7, 0xa6,
    0x27, 0x4e, 0xed, 0xf5, 0x6e, 0x92, 0x9f, 0x04,
    0xeb, 0xed, 0xef, 0xf0, 0x31, 0xe4, 0xc9, 0x51,
    0x9d, 0x12, 0x4d, 0x42, 0x50, 0xe8, 0x98, 0x82,
    0x95, 0x42, 0x96, 0x5e, 0x10, 0x77, 0x17, 0xd2,
    0x6c, 0x47, 0x55, 0x67, 0xf4, 0x0b, 0xeb, 0x7f,
    0x39, 0x31, 0xea, 0x8b, 0x86
};
```

8. On device, import the key:

The SMW API [smw_import_key\(\)](#) is used to import the key.

In the following example, the key imported is the blob example of the AES 256 bits key above.

```
int res = -1;
enum smw_status_code status = SMW_STATUS_OK;
struct smw_import_key_args args = { 0 };
struct smw_key_descriptor key = { 0 };
struct smw_key_attributes *attributes = &key.attributes;
struct smw_keypair_buffer buffer = { 0 };

/* Set the private key buffer to use the blob example */
buffer.gen.public_length = sizeof(aes_key_ex);
buffer.gen.public_data = aes_key_ex;

args.subsystem_name = SMW_SUBSYSTEM_NAME_ELE;
args.key_descriptor = &key;

key.id = 0x66;
key.type_name = SMW_KEY_TYPE_NAME_AES;
key.security_size = 256;
key.buffer = &buffer;

attributes.permitted_algo =
    SMW_ATTR_ALGO_SYMMETRIC_ENCRYPTION(SMW_ATTR_ALGO_AES,
    SMW_ATTR_MODE_ANY);
attributes.usage_flags = SMW_ATTR_USAGE_ENCRYPT | SMW_ATTR_USAGE_DECRYPT;
attributes.attributes = SMW_ATTR_SET_PERSISTENCE(attributes.attributes,
    SMW_ATTR_PERSISTENCE_PERSISTENT);

status = smw_import_key(&args);
```

...

5.2.7 Export Public Key

The **export public key** operation is used to extract the public key from a key pair stored in the secure subsystem storage.

The PKCS11 interface to use is the `C_GetAttributeValue()` function to retrieve the public key attribute containing the public key plaintext value.

Table 44. Export Public Key APIs Comparison

Key Type	API	Function / Key Type or Key Attribute
Secp R1	SMW	smw_export_key() SMW_KEY_TYPE_NAME_SECP_R1
	PSA	psa_export_key() / psa_export_public_key() PSA_ECC_FAMILY_SECP_R1
	PKCS11	C_GetAttributeValue() CKA_EC_POINT
Brainpool R1	SMW	smw_export_key() SMW_KEY_TYPE_NAME_BRAINPOOL_R1
	PSA	psa_export_key() / psa_export_public_key() PSA_ECC_FAMILY_BRAINPOOL_P_R1
	PKCS11	Not defined
Twisted Edwards (ED25519/ED448)	SMW	smw_export_key() SMW_KEY_TYPE_NAME_ED25519 SMW_KEY_TYPE_NAME_ED448
	PSA	psa_export_key() / psa_export_public_key() PSA_ECC_FAMILY_TWISTED_EDWARDS
	PKCS11	C_GetAttributeValue() CKA_EC_POINT
Montgomery (X25519/X448)	SMW	smw_export_key() SMW_KEY_TYPE_NAME_X25519 SMW_KEY_TYPE_NAME_X448
	PSA	psa_export_key() / psa_export_public_key() PSA_ECC_FAMILY_MONTGOMERY
	PKCS11	C_GetAttributeValue() CKA_EC_POINT
RSA	SMW	smw_export_key() SMW_KEY_TYPE_NAME_RSA
	PSA	psa_export_key() / psa_export_public_key() PSA_KEY_TYPE_RSA_KEY_PAIR
	PKCS11	C_GetAttributeValue() CKA_MODULUS and CKA_PUBLIC_EXPONENT
SM2	SMW	smw_export_key() SMW_KEY_TYPE_NAME_SM2

Table 44. Export Public Key APIs Comparison...continued

Key Type	API	Function / Key Type or Key Attribute
	PSA	Not defined
	PKCS11	Not defined

5.2.8 Key Derivation

The **key derivation** operation is used to derive one or more keys from a secret value such as a shared secret or a password.

Table 45. Key Derivation APIs Comparison

KDF Type	API	Function / Algorithm
HKDF	SMW	smw_derive_key() SMW_KDF_NAME_HKDF SMW_KDF_NAME_HKDF_EXTRACT SMW_KDF_NAME_HKDF_EXPAND
	PSA	psa_key_derivation_output_key() / psa_key_derivation_output_bytes() PSA_ALG_HKDF(hash) PSA_ALG_HKDF_EXTRACT(hash) PSA_ALG_HKDF_EXPAND(hash)
	PKCS11	C_DeriveKey() CKM_HKDF_DERIVE

5.2.9 Key Agreement

The **key agreement** operation is used to derive a shared secret from two key pairs. The shared secret can then be used as input to a key derivation function.

The Secure Subsystems able to do key agreement are limited to ELE and SECO (if SECO specific FW is loaded).

The key agreement supported are ECDH and [OEM Master key](#). The OEM Master key is used to import secure key in the ELE Secure Subsystem using a blob where key is encrypted.

Table 46. Key Derivation & Key Agreement APIs Comparison

KDF Type	API	Function / Algorithm
ECDH	SMW	smw_derive_key() SMW_KDF_NAME_ECDH
	PSA	psa_key_agreement() / psa_key_derivation_key_agreement() PSA_ALG_ECDH
	PKCS11	C_DeriveKey() CKM_ECDH1_DERIVE
OEM Master Key	SMW	smw_derive_key() SMW_KDF_NAME_OEM_MASTER_KEY
	PSA	Not supported
	PKCS11	Not supported

5.2.9.1 OEM Master key

The OEM Master key is a key agreement between a secure host ECDSA key and a device ECDSA key. The resulting key is a symmetric key used to derive two keys:

- A wrap key to wrap secure key to import
- A sign key to sign EdgeLock Enclave blob including the wrapped key and its attributes.

The OEM Master key requests a EdgeLock Enclave signed message to authenticate the key attributes and derivation parameters.

The SMW library offers the possibility to pre-fill the message payload to be signed (see :ref:smw_kdf_oem_master_key_args).

The NXP SPSDK tool can be used to sign the message and can be installed from [SPSDK releases](#). The documentation is available [here](#). The command used to sign is [nxpimage](#).

The OEM Master key attributes are almost pre-defined, the following [OEM Master key attributes](#) table lists the value to set in the signed payload and the SMW key attributes.

The signed payload defined also a `derived key group` field where the key will be stored. Value must be in the range [0:99] SMW defined 2 groups range function of the key persistency:

- 0:49 Groups of the persistent keys
- 50:99 Groups of the transient keys

It's impossible during runtime to predict the group number where the key could be stored if other keys are already present in the ELE NVM Secure Storage. If the key group can't be known in advance, it's advised to use the SMW operation pre-filling the payload content (see [struct smw_kdf_oem_master_key_args](#)) that will try to find a key group number where the key could be stored. Note, even that, it's not 100% guaranty that the key group will have enough place to store the key at the time the key derivation operation will be requested.

It's advised to store this key as persistent key with a known key identifier. In case of transient key the ELE FW will assign the key identifier after key derivation operation success.

Table 47. OEM Master key attributes

Key attributes	ELE value	SMW value
Key type	0x9200	SMW_KEY_TYPE_NAME_DERIVE
Security size (bits)	256	256
Usage	0x00004000	SMW_ATTR_USAGE_DERIVE
Permitted algorithm	0x0800109	SMW_ATTR_ALGO_KEY_DERIVATION_HKDF(SHA256)
Lifetime	0x00000000	SMW_ATTR_PERSISTENCE_TRANSIENT flag
	0x00000001	SMW_ATTR_PERSISTENCE_PERSISTENT flag
Lifecycle flags	0x00	SMW_ATTR_LIFECYCLE_CURRENT
	0x01	SMW_ATTR_LIFECYCLE_OPEN
	0x02	SMW_ATTR_LIFECYCLE_CLOSED
	0x03	SMW_ATTR_LIFECYCLE_CLOSED_LOCKED

Important:

The OEM SRKH must be fused.

5.2.9.1.1 Usage Example

See the [Example \(with OEM Master key derivation\)](#)

5.2.10 TLS 1.2 (PRF)

The TLS 1.2 Pseudo-Random Function (PRF) is used to derive key material during the TLS handshake process. The subsystem supports the following TLS 1.2 operations:

- Master secret generation from pre-master secret
- Key expansion to derive encryption/decryption keys and IVs
- Finished message verification data generation

Warning:

TLS 1.2 is considered legacy. For new implementations, consider using TLS 1.3 which provides improved security and performance.

5.2.10.1 Key Exchange and Ciphersuites

Only Elliptic Curve Diffie-Hellman Ephemeral (ECDHE) key exchange is supported, using SECP_R1 key type.

The base key size used for ECDH(E) determines which ciphersuites can be used:

- **256-bit keys:** Support SHA256-based ciphersuites
- **384-bit keys:** Support SHA384-based ciphersuites

Note:

ELE does not allow some operations on data with length less than the key size. Therefore, the key size of the base key used for ECDH(E) dictates which ciphersuites are available. For example, if the key size is 384 bits, you may only use ciphersuites that use SHA384.

5.2.10.2 Supported Ciphersuites

The following ciphersuites are supported:

Table 48. TLS 1.2 Ciphersuites

Cipher Suite	OpenSSL equivalent
TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256	ECDHE-ECDSA-AES128-SHA256
TLS_ECDHE_ECDSA_WITH_AES_128_CCM	ECDHE-ECDSA-AES128-CCM
TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256	ECDHE-ECDSA-AES128-GCM-SHA256
TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384	ECDHE-ECDSA-AES256-SHA384
TLS_ECDHE_ECDSA_WITH_AES_256_CCM	ECDHE-ECDSA-AES256-CCM
TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384	ECDHE-ECDSA-AES256-GCM-SHA384
TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256	ECDHE-ECDSA-CHACHA20-POLY1305

5.2.10.3 Encryption Types

The supported cipher suites use the following encryption types:

- **AES CBC and GCM:** Defined in [RFC 5289](#)
- **AES CCM:** Defined in [RFC 7251](#)
- **CHACHA20_POLY1305:** Defined in [RFC 7905](#)

5.2.10.4 TLS 1.2 PRF Algorithm

The TLS 1.2 PRF is based on HMAC and uses the following construction:

```
PRF(secret, label, seed) = P_<hash>(secret, label + seed)
```

Where:

- **secret:** The pre-master secret or master secret
- **label:** An ASCII string identifying the purpose of the key derivation
- **seed:** Random data (typically client_random + server_random)
- **hash:** SHA256 or SHA384 depending on the ciphersuite

5.2.10.5 Key Derivation Process

The typical TLS 1.2 key derivation process involves:

1. Master Secret Derivation:

```
master_secret = PRF(pre_master_secret, "master secret",
                    client_random + server_random)[0..47]
```

2. Key Material Expansion:

```
key_block = PRF(master_secret, "key expansion",
                 server_random + client_random)
```

3. Key Material Partitioning:

The key_block is partitioned into:

- Client write MAC key
- Server write MAC key
- Client write encryption key
- Server write encryption key
- Client write IV
- Server write IV

5.2.10.6 Subsystem Support

Table 49. TLS 1.2 Subsystem Support

Subsystem	Support	Notes
ELE	Yes	Full TLS 1.2 PRF support.
TEE	No	Not supported.
SECO	Yes*	Requires specific Firmware. Following encryption types are not supported: • AES CCM • CHACHA20_POLY1305_SHA256

5.2.10.7 Usage Example with SMW API

Here's a basic example of doing TLS 1.2 key derivation with SMW API to use a TLS_ECDHE_ECDSA_WITH_AES_128_CCM_SHA256 cipher suite.

```
#include <smw_keymgr.h>
#include <smw_crypto.h>

static int derive_tls12_master_key(struct smw_op_context *ctx,
                                   unsigned int pre_master_secret_id,
                                   unsigned char client_random[32],
                                   unsigned char server_random[32],
                                   unsigned char *peer_key,
                                   unsigned int peer_key_size,
                                   unsigned int *master_key_id)
{
    enum smw_status_code status;
    struct smw_op_context ctx = { 0 };
    struct smw_derive_key_args args = { 0 };
    struct smw_key_descriptor base_key = { 0 };
    struct smw_derived_key_descriptor derived_key = { 0 };
    struct smw_kdf_tls12_op_args tls12_args = { 0 };
    struct smw_kdf_tls12_random_data random_data = { 0 };

    /* Configure TLS 1.2 KDF arguments */
    args.subsystem_name = SMW_SUBSYSTEM_NAME_ELE;
    args.key_descriptor_base = &base_key;
    args.key_descriptor_derived = &derived_key;
    args.kdf_name = SMW_KDF_NAME_TLS12_OP_KEY_EXCHANGE;
    args.kdf_arguments = &tls12_args;

    /* Set base key (pre-master secret) */
    base_key.id = pre_master_secret_id;

    /* Configure TLS 1.2 specific parameters */
    tls12_args.op_name = SMW_TLS12_OP_NAME_MASTER_SECRET;
    tls12_args.prf_name = SMW_HASH_ALGO_NAME_SHA256;
    tls12_args.client_random = client_random;
    tls12_args.client_random_length = sizeof(client_random);
    tls12_args.server_random = server_random;
    tls12_args.server_random_length = sizeof(server_random);
    tls12_args.context = &ctx;

    tls12_args.master_secret.key_exchange_name = SMW_TLS12_KEA_NAME_ECDH_ECDSA;
    tls12_args.master_secret.ext_master_key = false;
    tls12_args.master_secret.peer_public_buffer = peer_key;
    tls12_args.master_secret.peer_public_buffer_length = peer_key_size;

    tls12_args.master_secret.random_data = &random_data;
    random_data.client_random = client_random;
    random_data.client_random_length = sizeof(client_random);
    random_data.server_random = server_random;
    random_data.server_random_length = sizeof(server_random);

    /* Derive master secret */
    status = smw_derive_key(&args);
    if (status != SMW_STATUS_OK) {
        /* Handle error */
        return -1;
    }
}
```

```

    }

    /* Store derived master key ID */
    *master_key_id = derived_key.id;

    return 0;
}

static int derive_tls12_key_material(struct smw_op_context *ctx,
                                     unsigned int master_key_id,
                                     unsigned char client_random[32],
                                     unsigned char server_random[32],
                                     unsigned char client_iv[12],
                                     unsigned char server_iv[12],
                                     unsigned int *client_wr_enc_key_id,
                                     unsigned int *server_wr_enc_key_id,
                                     unsigned int *client_wr_mac_key_id,
                                     unsigned int *server_wr_mac_key_id)
{
    enum smw_status_code status;
    struct smw_derive_key_args args = { 0 };
    struct smw_key_descriptor base_key = { 0 };
    struct smw_derived_key_descriptor derived_key = { 0 };
    struct smw_kdf_tls12_op_args tls12_args = { 0 };
    struct smw_kdf_tls12_random_data random_data = { 0 };

    /* Configure TLS 1.2 KDF arguments */
    args.subsystem_name = SMW_SUBSYSTEM_NAME_ELE;
    args.key_descriptor_base = &base_key;
    args.key_descriptor_derived = &derived_key;
    args.kdf_name = SMW_KDF_NAME_TLS12_OP_KEY_EXCHANGE;
    args.kdf_arguments = &tls12_args;

    /* Set base key (master key derived before) */
    base_key.id = master_key_id;

    /* Configure TLS 1.2 specific parameters */
    tls12_args.op_name = SMW_TLS12_OP_NAME_KEY_EXPANSION;
    tls12_args.prf_name = SMW_HASH_ALGO_NAME_SHA256;
    tls12_args.client_random = client_random;
    tls12_args.client_random_length = sizeof(client_random);
    tls12_args.server_random = server_random;
    tls12_args.server_random_length = sizeof(server_random);
    tls12_args.context = &ctx;

    tls12_args.key_expansion.encryption_name = SMW_TLS12_ENC_NAME_AES_128_CCM;
    tls12_args.key_expansion.random_data = &random_data;
    random_data.client_random = client_random;
    random_data.client_random_length = sizeof(client_random);
    random_data.server_random = server_random;
    random_data.server_random_length = sizeof(server_random);

    tls12_args.key_expansion.client_w_iv = client_iv;
    tls12_args.key_expansion.client_w_iv_length = sizeof(client_iv);
    tls12_args.key_expansion.server_w_iv = server_iv;
    tls12_args.key_expansion.server_w_iv_length = sizeof(server_iv);

    /* Derive master secret */
    status = smw_derive_key(&args);
}

```

```

    if (status != SMW_STATUS_OK) {
        /* Handle error */
        return -1;
    }

    /* Store derived key material IDs */
    *client_wr_enc_key_id = tls12_args.key_expansion.client_w_enc_key_id;
    *server_wr_enc_key_id = tls12_args.key_expansion.server_w_enc_key_id;
    *client_wr_mac_key_id = tls12_args.key_expansion.client_w_mac_key_id;
    *server_wr_mac_key_id = tls12_args.key_expansion.server_w_mac_key_id;

    return 0;
}

int derive_tls12_keys(...)
{
    int res = 0;
    enum smw_status_code status;
    struct smw_op_context ctx = { 0 };
    unsigned int master_key_id = 0;
    unsigned int client_wr_enc_key_id = 0;
    unsigned int server_wr_enc_key_id = 0;
    unsigned int client_wr_mac_key_id = 0;
    unsigned int server_wr_mac_key_id = 0;
    unsigned char client_iv[12] = { 0 };
    unsigned char server_iv[12] = { 0 };

    status = smw_allocate_context(&ctx);
    if (status != SMW_STATUS_OK)
        return -1;

    /*
     * Pre-master key id, client_random, server_random, peer_key,
     * peer_key_size are expected to be provided when calling this function.
     */

    res = derive_tls12_master_secret(&ctx, pre_master_secret_id,
                                     client_random, server_random,
                                     peer_key, peer_key_size, &master_key_id);

    if (res != 0) {
        smw_cancel_context(&ctx);
        return res;
    }

    res = derive_tls12_key_material(&ctx, master_key_id, client_random,
                                     server_random, client_iv, server_iv,
                                     &client_wr_enc_key_id,
                                     &client_wr_mac_key_id,
                                     &server_wr_enc_key_id,
                                     &server_wr_mac_key_id);

    if (res != 0)
        return res;

    /* Now you can use keys and ivs for encryption/decryption */
    /* ... */
}

```

5.2.10.8 Usage Example with PSA API

Here's a basic example of doing TLS 1.2 key derivation with PSA API to use a TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 cipher suite.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "psa/crypto.h"

/* Step 1: Derive master secret from pre-master secret */
static int derive_master_secret(psa_key_id_t ecdh_key_id,
                                const uint8_t *peer_public_key,
                                size_t peer_public_key_length,
                                const uint8_t *client_random,
                                const uint8_t *server_random,
                                psa_key_id_t *master_secret_id)
{
    psa_status_t status;
    psa_key_derivation_operation_t operation =
        PSA_KEY_DERIVATION_OPERATION_INIT;
    psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
    const uint8_t label[] = "master secret";
    uint8_t *seed = NULL;

    /* Prepare seed: label || client_random || server_random */
    seed = malloc(13 + 32 + 32); /* 13 bytes for label + 32 + 32 for randoms */
    if (!seed) {
        printf("Memory allocation failed\n");
        return -1;
    }

    memcpy(seed, label, 13);
    memcpy(seed + 13, client_random, 32);
    memcpy(seed + 13 + 32, server_random, 32);

    /* Setup key derivation operation for TLS 1.2 PRF */
    status = psa_key_derivation_setup(&operation,
                                       PSA_ALG_TLS12_PRF(PSA_ALG_SHA_256));
    if (status != PSA_SUCCESS) {
        printf("psa_key_derivation_setup failed: %d\n", status);
        return -1;
    }

    /* Set the input secret for the pre-master key internal generation */
    status = psa_key_derivation_key_agreement(&operation,
                                              PSA_KEY_DERIVATION_INPUT_SECRET,
                                              ecdh_key_id,
                                              peer_public_key,
                                              peer_public_key_length);
    if (status != PSA_SUCCESS) {
        printf("psa_key_derivation_key_agreement failed: %d\n", status);
        goto cleanup;
    }

    /* Set the seed (label + client_random + server_random) */
    status = psa_key_derivation_input_bytes(&operation,
                                             PSA_KEY_DERIVATION_INPUT_SEED,
                                             seed, sizeof(seed));
```



```

    if (status != PSA_SUCCESS) {
        printf("psa_key_derivation_input_bytes (seed) failed: %d\n", status);
        goto cleanup;
    }

    /* Output master secret key id for further derivation */
    psa_set_key_usage_flags(&attributes, PSA_KEY_USAGE_DERIVE);
    psa_set_key_type(&attributes, PSA_KEY_TYPE_DERIVE);

    /* Output master secret key id */
    status = psa_key_derivation_output_key(&attributes, &operation,
    master_secret_id);
    if (status != PSA_SUCCESS) {
        printf("psa_key_derivation_output_key failed: %d\n", status);
        goto cleanup;
    }

    printf("Derived master secret with ID: %u\n", *master_secret_id);

cleanup:
    if (seed)
        free(seed);

    psa_key_derivation_abort(&operation);
    return (status == PSA_SUCCESS) ? 0 : -1;
}

/* Step 2: Derive key material (encryption keys, MAC keys, IVs) */
static int derive_key_material(psa_key_id_t master_secret_id,
                               const uint8_t *client_random,
                               const uint8_t *server_random,
                               psa_key_id_t *client_mac_key_id,
                               psa_key_id_t *server_mac_key_id,
                               psa_key_id_t *client_enc_key_id,
                               psa_key_id_t *server_enc_key_id,
                               uint8_t *client_iv,
                               uint8_t *server_iv)
{
    psa_status_t status;
    psa_key_derivation_operation_t operation =
    PSA_KEY_DERIVATION_OPERATION_INIT;
    psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
    const uint8_t label[] = "key expansion";
    uint8_t *seed = NULL;

    /* Prepare seed: label || server_random || client_random (note the order) */
    seed = malloc(13 + 32 + 32); /* 13 bytes for label + 32 + 32 for randoms */
    if (!seed) {
        printf("Memory allocation failed\n");
        return -1;
    }

    memcpy(seed, label, 13);
    memcpy(seed + 13, server_random, 32);
    memcpy(seed + 13 + 32, client_random, 32);

    /* Setup key derivation operation for TLS 1.2 PRF */
    status = psa_key_derivation_setup(&operation,
    PSA_ALG_TLS12_PRF(PSA_ALG_SHA_256));

```

```

    if (status != PSA_SUCCESS) {
        printf("psa_key_derivation_setup failed: %d\n", status);
        return -1;
    }

    /* Set the master secret as input */
    status = psa_key_derivation_input_key(&operation,
                                          PSA_KEY_DERIVATION_INPUT_SECRET,
                                          master_secret_id);

    if (status != PSA_SUCCESS) {
        printf("psa_key_derivation_input_key failed: %d\n", status);
        goto cleanup;
    }

    /* Set the seed (label + server_random + client_random) */
    status = psa_key_derivation_input_bytes(&operation,
                                          PSA_KEY_DERIVATION_INPUT_SEED,
                                          seed, sizeof(seed));

    if (status != PSA_SUCCESS) {
        printf("psa_key_derivation_input_bytes (seed) failed: %d\n", status);
        goto cleanup;
    }

    /* Output client AES encryption key (AES-128-GCM) */
    psa_set_key_usage_flags(&attributes, PSA_KEY_USAGE_ENCRYPT |
PSA_KEY_USAGE_DECRYPT);
    psa_set_key_algorithm(&attributes, PSA_ALG_GCM);
    psa_set_key_type(&attributes, PSA_KEY_TYPE_AES);
    psa_set_key_bits(&attributes, 128);
    psa_set_key_lifetime(&attributes, PSA_KEY_LIFETIME_VOLATILE);

    status = psa_key_derivation_output_key(&attributes, &operation,
client_enc_key_id);
    if (status != PSA_SUCCESS) {
        printf("psa_key_derivation_output_key failed: %d\n", status);
        goto cleanup;
    }

    /* Output server AES encryption key (AES-128-GCM) */
    status = psa_key_derivation_output_key(&attributes, &operation,
server_enc_key_id);
    if (status != PSA_SUCCESS) {
        printf("psa_key_derivation_output_key failed: %d\n", status);
        goto cleanup;
    }

    /* Output IVs (for GCM, these are the implicit parts) */
    status = psa_key_derivation_output_bytes(&operation, client_iv, 4);
    if (status != PSA_SUCCESS) {
        printf("psa_key_derivation_output_bytes (client_iv) failed: %d\n",
status);
        goto cleanup;
    }

    status = psa_key_derivation_output_bytes(&operation, server_iv, 4);
    if (status != PSA_SUCCESS) {
        printf("psa_key_derivation_output_bytes (server_iv) failed: %d\n",
status);
        goto cleanup;
    }

```

```

    }

    printf("Derived all key material successfully\n");
    printf("Client encryption key handle: %lu\n", client_enc_key);
    printf("Server encryption key handle: %lu\n", server_enc_key);

cleanup:
    if (seed)
        free(seed);

    psa_key_derivation_abort(&operation);
    return (status == PSA_SUCCESS) ? 0 : -1;
}

/* Complete TLS 1.2 key derivation workflow */
int tls12_derive_keys_psa(...)
{
    psa_status_t status;
    psa_key_id_t ecdh_key_id = 0;
    psa_key_id_t master_secret_id = 0;
    psa_key_id_t client_enc_key_id = 0;
    psa_key_id_t server_enc_key_id = 0;
    uint8_t client_iv[12] = {0}; /* Full IV for GCM */
    uint8_t server_iv[12] = {0}; /* Full IV for GCM */
    int ret = 0;

    /*
     * Pre-master key id, client_random, server_random, peer_key,
     * peer_key_size are expected to be provided whe calling this function.
     */

    printf("\n=== TLS 1.2 Key Derivation with PSA API ===\n\n");

    /* Step 3: Derive master secret */
    printf("\nStep 1: Deriving master secret...\n");
    if (derive_master_secret(ecdh_key_id, peer_public_key, 64,
                           client_random, server_random,
                           &master_secret_id) != 0) {
        ret = -1;
        goto cleanup;
    }

    /* Step 4: Derive key material */
    printf("\nStep 2: Deriving key material...\n");
    if (derive_key_material(master_secret_id,
                           client_random, server_random,
                           &client_enc_key_id, &server_enc_key_id,
                           client_iv, server_iv) != 0) {
        ret = -1;
        goto cleanup;
    }

    printf("\n=== TLS 1.2 key derivation completed successfully! ===\n");

    /* Now you can use keys and ivs for encryption/decryption */
    /* ... */
}

```

5.2.10.9 Usage Example with PKCS11 API

Here's a basic example of using TLS 1.2 key derivation with PKCS11 API to use a TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 cipher suite.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "pkcs11.h"

/* Step 1: Perform ECDH to derive pre-master secret */
static int derive_pre_master_secret(CK_SESSION_HANDLE session,
                                   CK_OBJECT_HANDLE priv_key,
                                   CK_BYTE *peer_public_key,
                                   CK_ULONG peer_key_len,
                                   CK_OBJECT_HANDLE *premaster_secret)
{
    CK_RV rv;
    CK_ECDH1_DERIVE_PARAMS ecdh_params = {
        .kdf = CKD_NULL,
        .pSharedData = NULL,
        .ulSharedDataLen = 0,
        .pPublicData = peer_public_key,
        .ulPublicDataLen = peer_key_len
    };

    CK_MECHANISM mechanism = {
        .mechanism = CKM_ECDH1_DERIVE,
        .pParameter = &ecdh_params,
        .ulParameterLen = sizeof(ecdh_params)
    };

    /* Template for derived pre-master secret */
    CK_OBJECT_CLASS secret_class = CKO_SECRET_KEY;
    CK_KEY_TYPE key_type = CKK_GENERIC_SECRET;
    CK_ULONG key_len = 32; /* 256 bits for SECP256R1 */
    CK_MECHANISM_TYPE key_allowed_mech = { CKM_TLS12_MASTER_KEY_DERIVE_DH };

    CK_ATTRIBUTE template[] = {
        { CKA_CLASS, &secret_class, sizeof(secret_class) },
        { CKA_KEY_TYPE, &key_type, sizeof(key_type) },
        { CKA_VALUE_LEN, &key_len, sizeof(key_len) },
        { CKA_ALLOWED_MECHANISMS, &key_allowed_mech, sizeof(key_allowed_mech) },
        { CKA_DERIVE, &(CK_BBOOL){CK_TRUE}, sizeof(CK_BBOOL) },
        { CKA_EXTRACTABLE, &(CK_BBOOL){CK_FALSE}, sizeof(CK_BBOOL) }
    };

    rv = p11_func->C_DeriveKey(session, &mechanism, priv_key,
                               template, 6, premaster_secret);

    if (rv != CKR_OK) {
        printf("C_DeriveKey (ECDH) failed: 0x%lx\n", rv);
        return -1;
    }

    return 0;
}

/* Step 2: Derive master secret from pre-master secret */
static int derive_master_secret(CK_SESSION_HANDLE session,
```

```

        CK_OBJECT_HANDLE premaster_secret,
        CK_BYTE *client_random,
        CK_BYTE *server_random,
        CK_OBJECT_HANDLE *master_secret)
{
    CK_RV rv;
    CK_SSL3_RANDOM_DATA random_data = {
        .pClientRandom = client_random,
        .ulClientRandomLen = 32,
        .pServerRandom = server_random,
        .ulServerRandomLen = 32
    };

    CK_TLS12_MASTER_KEY_DERIVE_PARAMS params = {
        .prfHashMechanism = CKM_SHA256,
        .RandomInfo = random_data,
        .pVersion = NULL /* Use default TLS 1.2 */
    };

    CK_MECHANISM mechanism = {
        .mechanism = CKM_TLS12_MASTER_KEY_DERIVE,
        .pParameter = &params,
        .ulParameterLen = sizeof(params)
    };

    /* Template for master secret */
    CK_OBJECT_CLASS secret_class = CKO_SECRET_KEY;
    CK_KEY_TYPE key_type = CKK_GENERIC_SECRET;
    CK_ULONG key_len = 48; /* TLS master secret is always 48 bytes */
    CK_MECHANISM_TYPE key_allowed_mech = { CKM_TLS12_KEY_AND_MAC_DERIVE };

    CK_ATTRIBUTE template[] = {
        { CKA_CLASS, &secret_class, sizeof(secret_class) },
        { CKA_KEY_TYPE, &key_type, sizeof(key_type) },
        { CKA_VALUE_LEN, &key_len, sizeof(key_len) },
        { CKA_ALLOWED_MECHANISMS, &key_allowed_mech, sizeof(key_allowed_mech) },
        { CKA_DERIVE, &(CK_BBOOL){CK_TRUE}, sizeof(CK_BBOOL) }
    };

    rv = p11_func->C_DeriveKey(session, &mechanism, premaster_secret,
                               template, 5, master_secret);
    if (rv != CKR_OK) {
        printf("C_DeriveKey (Master Secret) failed: 0x%lx\n", rv);
        return -1;
    }

    return 0;
}

/* Step 3: Derive key material (encryption keys, MAC keys, IVs) */
static int derive_key_material(CK_SESSION_HANDLE session,
                               CK_OBJECT_HANDLE master_secret,
                               CK_BYTE *client_random,
                               CK_BYTE *server_random,
                               CK_OBJECT_HANDLE *client_mac_key,
                               CK_OBJECT_HANDLE *server_mac_key,
                               CK_OBJECT_HANDLE *client_enc_key,
                               CK_OBJECT_HANDLE *server_enc_key,
                               CK_ULONG iv_size_bits,
                               CK_BYTE *client_iv,

```

```

                                CK_BYTE *server_iv)
{
    CK_RV rv;
    CK_SSL3_RANDOM_DATA random_data = {
        .pClientRandom = client_random,
        .ulClientRandomLen = 32,
        .pServerRandom = server_random,
        .ulServerRandomLen = 32
    };

    /* For AES-128-GCM-SHA256 cipher suite */
    CK_SSL3_KEY_MAT_OUT key_material = {
        .pIVClient = client_iv,
        .pIVServer = server_iv,
    };

    CK_TLS12_KEY_MAT_PARAMS params = {
        .prfHashMechanism = CKM_SHA256,
        .bIsExport = CK_FALSE,
        .RandomInfo = random_data,
        .ulIV_SizeInBits = iv_size_bits,
        .ulKeySizeInBits = 160,
        .ulMacSizeInBits = 256,
        .pReturnedKeyMaterial = &key_material,
    };

    CK_MECHANISM mechanism = {
        .mechanism = CKM_TLS12_KEY_AND_MAC_DERIVE,
        .pParameter = &params,
        .ulParameterLen = sizeof(params)
    };

    /* Template for key encryption derived key */
    CK_OBJECT_CLASS secret_class = CKO_SECRET_KEY;
    CK_KEY_TYPE key_type = CKK_AES;
    CK_ULONG key_len = 32;
    CK_MECHANISM_TYPE key_allowed_mech = { CKM_AES_CBC };

    CK_ATTRIBUTE template[] = {
        { CKA_CLASS, &secret_class, sizeof(secret_class) },
        { CKA_KEY_TYPE, &key_type, sizeof(key_type) },
        { CKA_VALUE_LEN, &key_len, sizeof(key_len) },
        { CKA_ALLOWED_MECHANISMS, &key_allowed_mech, sizeof(key_allowed_mech) },
        { CKA_ENCRYPT, &(CK_BBOOL){CK_TRUE}, sizeof(CK_BBOOL) }
    };

    rv = p11_func->C_DeriveKey(session, &mechanism, master_secret,
                                template, 5, NULL);
    if (rv != CKR_OK) {
        printf("C_DeriveKey (Key Material) failed: 0x%lx\n", rv);
        return -1;
    }

    /* Extract derived keys and IVs */
    *client_mac_key = key_material.hClientMacSecret;
    *server_mac_key = key_material.hServerMacSecret;
    *client_enc_key = key_material.hClientKey;
    *server_enc_key = key_material.hServerKey;

    return 0;
}

```

```

/* Complete TLS 1.2 key derivation workflow */
int tls12_derive_keys_pkcs11(...)
{
    int ret = 0;
    CK_RV rv;
    CK_SESSION_HANDLE session;
    CK_SLOT_ID slot_id = 0;
    CK_OBJECT_HANDLE priv_key, pub_key;
    CK_OBJECT_HANDLE premaster_secret, master_secret;
    CK_OBJECT_HANDLE client_mac_key, server_mac_key;
    CK_OBJECT_HANDLE client_enc_key, server_enc_key;
    CK_BYTE client_iv[12], server_iv[12];

    /*
     * Pre-master key id, client_random, server_random, peer_key,
     * peer_key_size are expected to be provided whe calling this function.
     */

    /* Step 1: Derive pre-master secret using ECDH */
    if (derive_premaster_secret(session, priv_key, peer_public_key,
                                sizeof(peer_public_key),
                                &premaster_secret) != 0) {
        ret = -1;
        goto cleanup;
    }

    /* Step 2: Derive master secret */
    if (derive_master_secret(session, premaster_secret,
                             client_random, server_random,
                             &master_secret) != 0) {
        ret = -1;
        goto cleanup;
    }

    /* Step 3: Derive key material */
    if (derive_key_material(session, master_secret,
                             client_random, server_random,
                             &client_mac_key, &server_mac_key,
                             &client_enc_key, &server_enc_key,
                             sizeof(client_iv) * 8,
                             client_iv, server_iv) != 0) {
        ret = -1;
        goto cleanup;
    }

    printf("TLS 1.2 key derivation completed successfully!\n");
    printf("Client MAC key handle: %lu\n", client_mac_key);
    printf("Server MAC key handle: %lu\n", server_mac_key);
    printf("Client encryption key handle: %lu\n", client_enc_key);
    printf("Server encryption key handle: %lu\n", server_enc_key);

    /* Now you can use keys and ivs for encryption/decryption */
    /* ... */
}

```

5.2.10.10 API Comparison

Table 50. TLS 1.2 Key Derivation APIs

API	Functions & Algorithm
SMW	smw_derive_key() SMW_KDF_NAME_TLS12_PRF
PSA	psa_key_derivation_output_key() / psa_key_derivation_output_bytes() PSA_ALG_TLS12_PRF(hash_alg) PSA_ALG_TLS12_PSK_TO_MS(hash_alg)
PKCS11	C_DeriveKey() CKM_ECDH1_DERIVE CKM_TLS12_MASTER_KEY_DERIVE CKM_TLS12_KEY_AND_MAC_DERIVE CKM_TLS12_KEY_SAFE_DERIVE

5.2.10.11 References

- [RFC 5246](#) - The Transport Layer Security (TLS) Protocol Version 1.2
- [RFC 5289](#) - TLS Elliptic Curve Cipher Suites with SHA-256/384 and AES Galois Counter Mode (GCM)
- [RFC 7251](#) - AES-CCM Elliptic Curve Cryptography (ECC) Cipher Suites for TLS
- [RFC 7905](#) - ChaCha20-Poly1305 Cipher Suites for Transport Layer Security (TLS)

5.2.11 TLS 1.3 (TLS13-KDF)

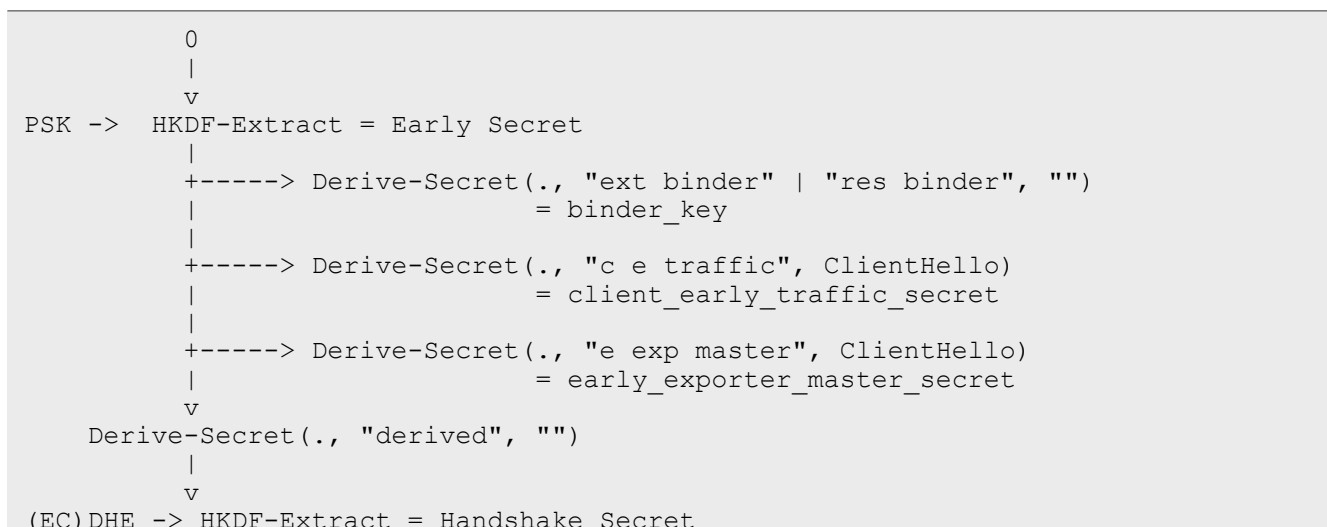
TLS 1.3 introduces a redesigned key derivation function based on HKDF (HMAC-based Key Derivation Function) as defined in [RFC 8446](#). The TLS 1.3 key schedule provides improved security through a more structured key derivation process and forward secrecy.

Note:

Not supported on i.MX8ULP.

5.2.11.1 Key Schedule Overview

The TLS 1.3 key schedule derives multiple secrets in a hierarchical manner:




```

|
+-----> Derive-Secret(., "c hs traffic",
|                               ClientHello...ServerHello)
|                               = client_handshake_traffic_secret
|
+-----> Derive-Secret(., "s hs traffic",
|                               ClientHello...ServerHello)
|                               = server_handshake_traffic_secret
|
v
Derive-Secret(., "derived", "")
|
v
0 -> HKDF-Extract = Master Secret
|
+-----> Derive-Secret(., "c ap traffic",
|                               ClientHello...server Finished)
|                               = client_application_traffic_secret_0
|
+-----> Derive-Secret(., "s ap traffic",
|                               ClientHello...server Finished)
|                               = server_application_traffic_secret_0
|
+-----> Derive-Secret(., "exp master",
|                               ClientHello...server Finished)
|                               = exporter_master_secret
|
+-----> Derive-Secret(., "res master",
|                               ClientHello...client Finished)
|                               = resumption_master_secret

```

5.2.11.2 Internal Secret Management

The following secrets are computed internally by the subsystem and are **not exported**:

- Early secret
- ECDH shared secret
- Handshake secret
- Master secret

This design ensures that sensitive key material never leaves the secure subsystem, providing enhanced security.

5.2.11.3 Supported Key Types

The subsystem supports the following elliptic curve key types for TLS 1.3:

Table 51. TLS 1.3 Supported Key Types

Key Type	Security size (bits)	Notes
SECP_R1	256 / 384 / 521	NIST P-256, P-384, P-521 curves
X25519	255	Curve25519 for ECDH
X448	448	Curve448 for ECDH

5.2.11.4 Derivable Secrets

The subsystem supports derivation of the following TLS 1.3 secrets:

Table 52. TLS 1.3 Secrets

TLS 1.3 Secret Name	Label (without null term)	Usage
Binder key	"ext binder" or "res binder"	PSK binder computation
Client early traffic secret	"c e traffic"	0-RTT data encryption (client)
Early exporter master secret	"e exp master"	Early data exporters
Client handshake traffic secret	"c hs traffic"	Handshake encryption (client)
Server handshake traffic secret	"s hs traffic"	Handshake encryption (server)
Client application traffic secret 0	"c ap traffic"	Application data (client)
Server application traffic secret 0	"s ap traffic"	Application data (server)
Exporter master secret	"exp master"	Exporters for external protocols
Resumption master secret	"res master"	Session resumption PSK

5.2.11.5 Key and IV Derivation

After deriving the required traffic secret, the TLS 1.3 API can be used to further derive encryption keys and IVs as specified in [RFC 8446](#):

```
[sender]_write_key = HKDF-Expand-Label(Secret, "key", "", key_length)
[sender]_write_iv  = HKDF-Expand-Label(Secret, "iv", "", iv_length)
```

Where:

- **Secret:** The traffic secret (e.g., `client_handshake_traffic_secret`)
- **key_length:** Length of the encryption key (depends on cipher suite)
- **iv_length:** Length of the IV (typically 12 bytes for AEAD ciphers)

5.2.11.6 Key Attributes for Derived Keys

When deriving keys from traffic secrets, the derived keys must have appropriate attributes set based on their intended use:

Example 1: Deriving AES-128-GCM key from server handshake traffic secret

- **Key type:** AES
- **Security size:** 128 bits
- **Algorithm:** AES-GCM
- **Usage:** Encrypt and/or Decrypt

Example 2: Deriving HMAC-256 key for Finished message

- **Key type:** HMAC
- **Security size:** 256 bits
- **Algorithm:** HMAC-SHA256
- **Usage:** Sign and/or Verify

Important:

All derived keys need to have the proper attributes (type, size, algorithm, and usage) set before performing the derivation operation.

5.2.11.7 Supported Cipher Suites

TLS 1.3 supports the following cipher suites:

Table 53. TLS 1.3 Cipher Suites

Cipher Suite	AEAD	Hash
TLS_AES_128_GCM_SHA256	AES-128-GCM	SHA-256
TLS_AES_256_GCM_SHA384	AES-256-GCM	SHA-384
TLS_CHACHA20_POLY1305_SHA256	ChaCha20-Poly1305	SHA-256
TLS_AES_128_CCM_SHA256	AES-128-CCM	SHA-256

5.2.11.8 Subsystem Support

Table 54. TLS 1.3 Subsystem Support

Subsystem	Support	Notes
ELE	Yes	Not supported on i.MX8ULP
TEE	No	Not supported
SECO	No	Not supported

5.2.11.9 Usage Example with SMW API

Here's a basic example of using TLS 1.3 key derivation with SMW API to use a TLS_AES_128_GCM_SHA256 cipher suite.

```
#include <smw_keymgr.h>
#include <smw_crypto.h>
#include <string.h>

/* Step 1: Derive handshake traffic secret */
int derive_handshake_secret(unsigned int ecdh_key_id,
                           unsigned char *label,
                           unsigned char transcript_hash[32],
                           unsigned char *peer_key,
                           unsigned int peer_key_len,
                           unsigned int *secret_id)
{
    int res = 0;
    enum smw_status_code status;
    struct smw_derive_key_args args = { 0 };
    struct smw_key_descriptor base_key = { 0 };
    struct smw_derived_key_descriptor derived_key = { 0 };
    struct smw_tls13_kdf_args tls13_args = { 0 };
    struct smw_tls13_expand_label_args exp_args = { 0 };

    /* Configure TLS 1.3 KDF arguments */
    args.subsystem_name = SMW_SUBSYSTEM_NAME_ELE;
    args.key_descriptor_base = &base_key;
    args.key_descriptor_derived = &derived_key;
    args.kdf_name = SMW_KDF_NAME_TLS13_KEY_EXCHANGE;
    args.kdf_arguments = &tls13_args;
    args.store_derived_key = false;

    /* Base key is the ECDH shared secret (managed internally) */
```

```

    base_key.id = ecdh_key_id;

    exp_args.length = 32; /* SHA-256 output length */
    exp_args.label = label;
    exp_args.label_length = strlen(label);
    exp_args.context = transcript_hash;
    exp_args.context_length = sizeof(transcript_hash);

    tls13_args->expanded_label_length =
SMW_TLS13_EXPANDED_LABEL_LENGTH(&exp_args);
    tls13_args->expanded_label = malloc(tls13_args->expanded_label_length);
    if (!tls13_args->expanded_label)
        return -1;

    exp_args.expanded_label_length = tls13_args->expanded_label_length;
    exp_args.expanded_label = tls13_args->expanded_label;

    res = smw_tls13_expand_label(&exp_args);
    if (res != SMW_STATUS_OK) {
        res = -1;
        goto end;
    }

    /* Configure TLS 1.3 specific parameters */
    tls13_args.prf_name = SMW_HASH_ALGO_NAME_SHA256;
    tls13_args.peer_public_buffer = peer_key;
    tls13_args.peer_public_buffer_length = peer_key_length;

    /* Derive server handshake traffic secret */
    status = smw_derive_key(&args);
    if (status != SMW_STATUS_OK)
        res = -1;
    else
        *secret_id = derived_key.id;

end:
    free(tls13_args->expanded_label);

    return res;
}

/* Step 2: Derive AES-128-GCM key from traffic secret */
int derive_aes_key_from_traffic_secret(unsigned int traffic_secret_id,
                                       unsigned int *aes_key_id)
{
    int res = 0;
    enum smw_status_code status;
    struct smw_derive_key_args args = { 0 };
    struct smw_key_descriptor base_key = { 0 };
    struct smw_derived_key_descriptor derived_key = { 0 };
    struct smw_tls13_kdf_args tls13_args = { 0 };
    struct smw_tls13_expand_label_args exp_args = { 0 };

    /* Configure derivation arguments */
    args.subsystem_name = SMW_SUBSYSTEM_NAME_ELE;
    args.key_descriptor_base = &base_key;
    args.key_descriptor_derived = &derived_key;
    args.kdf_name = SMW_KDF_NAME_TLS13_EXPAND_LABEL;
    args.kdf_arguments = &tls13_args;
    args.store_derived_key = false;

```

```

    /* Base key is the traffic secret */
    base_key.id = traffic_secret_id;

    /* Configure HKDF-Expand-Label parameters */
    exp_args.label = "key";
    exp_args.label_length = strlen(exp_args.label);
    exp_args.context = NULL;
    exp_args.context_length = 0;

    tls13_args->expanded_label_length =
SMW_TLS13_EXPANDED_LABEL_LENGTH(&exp_args);
    tls13_args->expanded_label = malloc(tls13_args->expanded_label_length);
    if (!tls13_args->expanded_label)
        return -1;

    exp_args.expanded_label_length = tls13_args->expanded_label_length;
    exp_args.expanded_label = tls13_args->expanded_label;

    res = smw_tls13_expand_label(&exp_args);
    if (res != SMW_STATUS_OK) {
        res = -1;
        goto end;
    }

    /* Configure derived AES key attributes */
    derived_key.type_name = SMW_KEY_TYPE_NAME_AES;
    derived_key.security_size = 128;
    derived_key.attributes.permitted_algo =
        SMW_ATTR_ALGO_SYMMETRIC_ENCRYPTION(SMW_ATTR_ALGO_AES,
                                           SMW_ATTR_MODE_GCM);
    derived_key.attributes.usage_flags =
        SMW_ATTR_USAGE_ENCRYPT | SMW_ATTR_USAGE_DECRYPT;
    derived_key.attributes.attributes =
        SMW_ATTR_SET_PERSISTENCE(derived_key.attributes.attributes,
                                SMW_ATTR_PERSISTENCE_TRANSIENT);

    /* Configure TLS 1.3 specific parameters */
    tls13_args.prf_name = SMW_HASH_ALGO_NAME_SHA256;

    /* Derive AES key */
    status = smw_derive_key(&args);
    if (status != SMW_STATUS_OK)
        res = -1;
    else
        *aes_key_id = derived_key.id;

end:
    free(tls13_args->expanded_label);

    return res;
}

/* Step 3: Derive IV from traffic secret */
int derive_iv_from_traffic_secret(unsigned int traffic_secret_id,
                                unsigned char *iv, size_t iv_len)
{
    int res = 0;
    enum smw_status_code status;
    struct smw_derive_key_args args = { 0 };

```

```

struct smw_key_descriptor base_key = { 0 };
struct smw_derived_key_descriptor derived_key = { 0 };
struct smw_tls13_kdf_args tls13_args = { 0 };
struct smw_tls13_expand_label_args exp_args = { 0 };

/* Configure derivation arguments */
args.subsystem_name = SMW_SUBSYSTEM_NAME_ELE;
args.key_descriptor_base = &base_key;
args.key_descriptor_derived = &derived_key;
args.kdf_name = SMW_KDF_NAME_TLS13_EXPAND_LABEL;
args.kdf_arguments = &tls13_args;
args.store_derived_key = false; /* Output to buffer */

/* Base key is the traffic secret */
base_key.id = traffic_secret_id;

/* Configure HKDF-Expand-Label parameters */
exp_args.label = "iv";
exp_args.label_length = strlen(exp_args.label);
exp_args.context = NULL;
exp_args.context_length = 0;

tls13_args->expanded_label_length =
SMW_TLS13_EXPANDED_LABEL_LENGTH(&exp_args);
tls13_args->expanded_label = malloc(tls13_args->expanded_label_length);
if (!tls13_args->expanded_label)
    return -1;

exp_args.expanded_label_length = tls13_args->expanded_label_length;
exp_args.expanded_label = tls13_args->expanded_label;

res = smw_tls13_expand_label(&exp_args);
if (res != SMW_STATUS_OK) {
    res = -1;
    goto end;
}

/* Configure derived key result as shared buffer */
derived_key.shared_secret = iv;
derived_key.shared_secret_len = iv_len;

/* Configure TLS 1.3 specific parameters */
tls13_args.prf_name = SMW_HASH_ALGO_NAME_SHA256;

/* Derive IV */
status = smw_derive_key(&args);
if (status != SMW_STATUS_OK)
    res = -1;

end:
free(tls13_args->expanded_label);
return res;
}

/* Complete workflow example */
int tls13_derive_encryption_keys(...)
{
    int ret;
    unsigned int hs_secret_id;
    unsigned int aes_key_id;

```

```

unsigned char iv[12]; /* Standard IV length for AEAD */
unsigned char transcript_hash[32]; /* SHA-256 hash */

/* Compute transcript hash: Hash(ClientHello...ServerHello) */
/* ... */

/* Peer key and ecdh_key_id obtained from key exchange */

/* Step 1: Derive client handshake traffic secret */
ret = derive_handshake_secret(ecdh_key_id, "c hs traffic",
                             peer_key, peer_key_len,
                             transcript_hash, &hs_secret_id);

if (ret != 0)
    return -1;

/* Step 2: Derive AES-128-GCM encryption key */
ret = derive_aes_key_from_traffic_secret(hs_secret_id, &aes_key_id);
if (ret != 0)
    return -1;

/* Step 3: Derive IV */
ret = derive_iv_from_traffic_secret(hs_secret_id, iv, sizeof(iv));
if (ret != 0)
    return -1;

/* Now you can use keys and ivs for encryption/decryption */
/* ... */
}

```

5.2.11.10 Usage Example with PSA API

Here's a basic example of doing TLS 1.3 key derivation with PSA API to use a TLS_AES_128_GCM_SHA256 cipher suite.

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "psa/crypto.h"

/* Step 1: Derive client handshake traffic secret */
static int derive_handshake_traffic_secret(psa_key_id_t ecdh_key_id,
                                           const char *label,
                                           const uint8_t *peer_public_key,
                                           size_t peer_public_key_length,
                                           const uint8_t *transcript_hash,
                                           psa_key_id_t *hs_traffic_id)
{
    psa_status_t status;
    psa_key_derivation_operation_t operation =
        PSA_KEY_DERIVATION_OPERATION_INIT;
    psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
    uint8_t info[256] = {0};
    size_t info_len;
    uint16_t length_be;
    size_t label_len = strlen(label);

    /* Build HKDF-Expand-Label info structure:

```

```

    * struct {
    *     uint16 length = 32 (SHA-256 output length);
    *     opaque label<7..255> = "tls13 " + Label;
    *     opaque context<0..255> = transcript_hash;
    * } HkdfLabel;
    */
size_t pos = 0;

/* Length (2 bytes, big-endian) */
length_be = 32; /* SHA-256 output length */
info[pos++] = (length_be >> 8) & 0xFF;
info[pos++] = length_be & 0xFF;

/* Label length (1 byte) */
info[pos++] = 6 + label_len; /* "tls13 " + label */

/* Label ("tls13 " + label) */
memcpy(&info[pos], "tls13 ", 6);
pos += 6;
memcpy(&info[pos], label, label_len);
pos += label_len;

/* Context length (1 byte) */
info[pos++] = 32; /* SHA-256 hash length */

/* Context (transcript hash) */
memcpy(&info[pos], transcript_hash, 32);
pos += 32;

info_len = pos;

/* Setup key derivation operation for TLS 1.3 PRF */
status = psa_key_derivation_setup(&operation,
                                   PSA_ALG_VENDOR_TLS13(PSA_ALG_SHA_256));
if (status != PSA_SUCCESS) {
    printf("psa_key_derivation_setup failed: %d\n", status);
    return -1;
}

/* Set the input secret for the pre-master key internal generation */
status = psa_key_derivation_key_agreement(&operation,
                                           PSA_KEY_DERIVATION_INPUT_SECRET,
                                           ecdh_key_id,
                                           peer_public_key,
                                           peer_public_key_length);
if (status != PSA_SUCCESS) {
    printf("psa_key_derivation_key_agreement failed: %d\n", status);
    goto cleanup;
}

/* Set the seed (label + client_random + server_random) */
status = psa_key_derivation_input_bytes(&operation,
                                         PSA_KEY_DERIVATION_INPUT_INFO,
                                         info, sizeof(info));
if (status != PSA_SUCCESS) {
    printf("psa_key_derivation_input_bytes (info) failed: %d\n", status);
    goto cleanup;
}

```



```

    /* Output master secret key id for further derivation */
    psa_set_key_usage_flags(&attributes, PSA_KEY_USAGE_DERIVE);
    psa_set_key_type(&attributes, PSA_KEY_TYPE_DERIVE);

    /* Output handshake traffic key id */
    status = psa_key_derivation_output_key(&attributes, &operation,
hs_traffic_id);
    if (status != PSA_SUCCESS) {
        printf("psa_key_derivation_output_key failed: %d\n", status);
        goto cleanup;
    }

    printf("Derived HS traffic with ID: %u\n", *hs_traffic_id);

cleanup:
    if (seed)
        free(seed);

    psa_key_derivation_abort(&operation);
    return (status == PSA_SUCCESS) ? 0 : -1;
}

/* Step 2: Derive AES-128-GCM encryption key */
static int derive_aes_key_from_traffic_secret(psa_key_id_t traffic_secret_id,
                                              psa_key_id_t *aes_key_id)
{
    psa_status_t status;
    psa_key_derivation_operation_t operation =
        PSA_KEY_DERIVATION_OPERATION_INIT;
    psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
    uint8_t info[256] = {0};
    size_t info_len;
    uint16_t length_be;
    const char *label = "key";
    size_t label_len = strlen(label);

    /* Build HKDF-Expand-Label info for "key" derivation */
    size_t pos = 0;

    /* Length (2 bytes, big-endian) - 16 bytes for AES-128 */
    length_be = 16;
    info[pos++] = (length_be >> 8) & 0xFF;
    info[pos++] = length_be & 0xFF;

    /* Label length */
    info[pos++] = 6 + label_len;

    /* Label */
    memcpy(&info[pos], "tls13 ", 6);
    pos += 6;
    memcpy(&info[pos], label, label_len);
    pos += label_len;

    /* Context length (empty context) */
    info[pos++] = 0;

    info_len = pos;

    /* Setup HKDF derivation */
    status = psa_key_derivation_setup(&operation,

```

```

                                PSA_ALG_VENDOR_TLS13(PSA_ALG_SHA_256));
if (status != PSA_SUCCESS) {
    printf("psa_key_derivation_setup failed: %d\n", status);
    return -1;
}

/* Input the traffic secret */
status = psa_key_derivation_input_key(&operation,
                                      PSA_KEY_DERIVATION_INPUT_SECRET,
                                      traffic_secret_id);

if (status != PSA_SUCCESS) {
    printf("psa_key_derivation_input_key failed: %d\n", status);
    goto cleanup;
}

/* Input the info */
status = psa_key_derivation_input_bytes(&operation,
                                       PSA_KEY_DERIVATION_INPUT_INFO,
                                       info, info_len);

if (status != PSA_SUCCESS) {
    printf("psa_key_derivation_input_bytes failed: %d\n", status);
    goto cleanup;
}

/* Configure AES-128-GCM key attributes */
psa_set_key_usage_flags(&attributes,
                       PSA_KEY_USAGE_ENCRYPT | PSA_KEY_USAGE_DECRYPT);
psa_set_key_algorithm(&attributes, PSA_ALG_GCM);
psa_set_key_type(&attributes, PSA_KEY_TYPE_AES);
psa_set_key_bits(&attributes, 128);
psa_set_key_lifetime(&attributes, PSA_KEY_LIFETIME_VOLATILE);

/* Derive the AES key */
status = psa_key_derivation_output_key(&attributes, &operation, aes_key_id);
if (status != PSA_SUCCESS) {
    printf("psa_key_derivation_output_key (AES) failed: %d\n", status);
    goto cleanup;
}

printf("Derived AES-128-GCM encryption key with ID: %u\n", *aes_key_id);

cleanup:
    psa_reset_key_attributes(&attributes);
    psa_key_derivation_abort(&operation);
    return (status == PSA_SUCCESS) ? 0 : -1;
}

/* Step 3: Derive IV from traffic secret */
static int derive_iv_from_traffic_secret(psa_key_id_t traffic_secret_id,
                                         uint8_t *iv, size_t iv_len)
{
    psa_status_t status;
    psa_key_derivation_operation_t operation =
        PSA_KEY_DERIVATION_OPERATION_INIT;
    uint8_t info[256] = {0};
    size_t info_len;
    uint16_t length_be;
    const char *label = "iv";
    size_t label_len = strlen(label);

```

```

/* Build HKDF-Expand-Label info for "iv" derivation */
size_t pos = 0;

/* Length (2 bytes, big-endian) - 12 bytes for GCM IV */
length_be = iv_len;
info[pos++] = (length_be >> 8) & 0xFF;
info[pos++] = length_be & 0xFF;

/* Label length */
info[pos++] = 6 + label_len;

/* Label */
memcpy(&info[pos], "tls13 ", 6);
pos += 6;
memcpy(&info[pos], label, label_len);
pos += label_len;

/* Context length (empty context) */
info[pos++] = 0;

info_len = pos;

/* Setup HKDF derivation */
status = psa_key_derivation_setup(&operation,
                                   PSA_ALG_VENDOR_TLS13(PSA_ALG_SHA_256));
if (status != PSA_SUCCESS) {
    printf("psa_key_derivation_setup failed: %d\n", status);
    return -1;
}

/* Input the traffic secret */
status = psa_key_derivation_input_key(&operation,
                                       PSA_KEY_DERIVATION_INPUT_SECRET,
                                       traffic_secret_id);
if (status != PSA_SUCCESS) {
    printf("psa_key_derivation_input_key failed: %d\n", status);
    goto cleanup;
}

/* Input the info */
status = psa_key_derivation_input_bytes(&operation,
                                       PSA_KEY_DERIVATION_INPUT_INFO,
                                       info, info_len);
if (status != PSA_SUCCESS) {
    printf("psa_key_derivation_input_bytes failed: %d\n", status);
    goto cleanup;
}

/* Derive the IV bytes */
status = psa_key_derivation_output_bytes(&operation, iv, iv_len);
if (status != PSA_SUCCESS) {
    printf("psa_key_derivation_output_bytes (IV) failed: %d\n", status);
    goto cleanup;
}

print_hex("Derived IV", iv, iv_len);

cleanup:
psa_key_derivation_abort(&operation);
return (status == PSA_SUCCESS) ? 0 : -1;

```

```

}

/* Complete TLS 1.3 key derivation workflow */
int tls13_derive_keys_psa(...)
{
    psa_key_id_t hs_traffic_id = 0;
    psa_key_id_t aes_key_id = 0;
    uint8_t client_iv[12] = {0}; /* Full IV for GCM */
    unsigned char transcript_hash[32]; /* SHA-256 hash */
    int ret = 0;

    /* Compute transcript hash: Hash(ClientHello...ServerHello) */
    /* ... */

    /* Peer key and ecdh_key_id obtained from key exchange */

    printf("\n=== TLS 1.3 Key Derivation with PSA API ===\n\n");

    /* Step 1: Derive client handshake traffic secret */
    ret = derive_handshake_secret(ecdh_key_id, "c hs traffic",
                                peer_key, peer_key_len,
                                transcript_hash, &hs_secret_id);

    if (ret != 0)
        return -1;

    /* Step 2: Derive AES-128-GCM encryption key */
    ret = derive_aes_key_from_traffic_secret(hs_secret_id, &aes_key_id);
    if (ret != 0)
        return -1;

    /* Step 3: Derive IV */
    ret = derive_iv_from_traffic_secret(hs_secret_id, iv, sizeof(iv));
    if (ret != 0)
        return -1;

    printf("\n=== TLS 1.3 key derivation completed successfully! ===\n");

    /* Now you can use keys and ivs for encryption/decryption */
    /* ... */
}

```

5.2.11.11 Usage Example with PKCS11 API

Here's a basic example of using TLS 1.3 key derivation with PKCS11 API to use a TLS_AES_128_GCM_SHA256 cipher suite.

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "pkcs11.h"

/* Helper: Build HKDF-Expand-Label info structure for TLS 1.3 */
static CK_ULONG build_hkdf_label(CK_BYTE *output,
                                CK_ULONG length,
                                const char *label,
                                const CK_BYTE *context,
                                CK_ULONG context_len)

```

```

{
    CK_ULONG pos = 0;
    CK_ULONG label_len = strlen(label);

    /* Length (2 bytes, big-endian) */
    output[pos++] = (length >> 8) & 0xFF;
    output[pos++] = length & 0xFF;

    /* Label length (1 byte) - "tls13 " + label */
    output[pos++] = 6 + label_len;

    /* Label - "tls13 " prefix */
    memcpy(&output[pos], "tls13 ", 6);
    pos += 6;

    /* Label - actual label */
    memcpy(&output[pos], label, label_len);
    pos += label_len;

    /* Context length (1 byte) */
    output[pos++] = context_len;

    /* Context data */
    if (context_len > 0) {
        memcpy(&output[pos], context, context_len);
        pos += context_len;
    }

    return pos;
}

/* Step 1: Perform ECDH to derive pre-master secret */
static int derive_ecdh_secret(CK_SESSION_HANDLE session,
                              CK_OBJECT_HANDLE priv_key,
                              CK_BYTE *peer_public_key,
                              CK_ULONG peer_key_len,
                              CK_OBJECT_HANDLE *shared_secret)
{
    CK_RV rv;
    CK_ECDH1_DERIVE_PARAMS ecdh_params = {
        .kdf = CKD_NULL,
        .pSharedData = NULL,
        .ulSharedDataLen = 0,
        .pPublicData = peer_public_key,
        .ulPublicDataLen = peer_key_len
    };

    CK_MECHANISM mechanism = {
        .mechanism = CKM_ECDH1_DERIVE,
        .pParameter = &ecdh_params,
        .ulParameterLen = sizeof(ecdh_params)
    };

    /* Template for derived pre-master secret */
    CK_OBJECT_CLASS secret_class = CKO_SECRET_KEY;
    CK_KEY_TYPE key_type = CKK_GENERIC_SECRET;
    CK_ULONG key_len = 32; /* 256 bits for SECP256R1 */
    CK_MECHANISM_TYPE key_allowed_mech = { CKM_HKDF_DERIVE };
    CK_BBOOL true_val = CK_TRUE;

```

```

CK_BBOOL false_val = CK_TRUE;

CK_ATTRIBUTE template[] = {
    { CKA_CLASS, &secret_class, sizeof(secret_class) },
    { CKA_KEY_TYPE, &key_type, sizeof(key_type) },
    { CKA_VALUE_LEN, &key_len, sizeof(key_len) },
    { CKA_ALLOWED_MECHANISMS, &key_allowed_mech, sizeof(key_allowed_mech) },
    { CKA_DERIVE, &true_val, sizeof(CK_BBOOL) },
    { CKA_EXTRACTABLE, &>false_val, sizeof(CK_BBOOL) }
};

rv = p11_func->C_DeriveKey(session, &mechanism, priv_key,
                           template, 6, shared_secret);

if (rv != CKR_OK) {
    printf("C_DeriveKey (ECDH) failed: 0x%lx\n", rv);
    return -1;
}

return 0;
}

/* Step 2: Derive handshake traffic secret */
static int derive_handshake_traffic_secret(CK_SESSION_HANDLE session,
                                           CK_OBJECT_HANDLE shared_secret,
                                           const CK_BYTE *transcript_hash,
                                           const char *labal,
                                           CK_OBJECT_HANDLE *traffic_secret)
{
    CK_RV rv;
    CK_BYTE info[256];
    CK_ULONG info_len;

    /* HKDF parameters for TLS 1.3 */
    CK_HKDF_PARAMS hkdf_params = {
        .bExtract = CK_FALSE, /* We're doing expand only */
        .bExpand = CK_TRUE,
        .prfHashMechanism = CKM_SHA256,
        .ulSaltType = CKF_HKDF_SALT_NULL,
        .pSalt = NULL,
        .ulSaltLen = 0,
        .pInfo = info,
        .ulInfoLen = info_len
    };

    CK_MECHANISM mechanism = {
        .mechanism = CKM_HKDF_DERIVE,
        .pParameter = &hkdf_params,
        .ulParameterLen = sizeof(hkdf_params)
    };

    CK_OBJECT_CLASS secret_class = CKO_SECRET_KEY;
    CK_KEY_TYPE key_type = CKK_GENERIC_SECRET;
    CK_ULONG key_len = 32; /* SHA-256 output length */
    CK_MECHANISM_TYPE key_allowed_mech = { CKM_HKDF_DERIVE };
    CK_BBOOL true_val = CK_TRUE;

    CK_ATTRIBUTE template[] = {
        { CKA_CLASS, &secret_class, sizeof(secret_class) },
        { CKA_KEY_TYPE, &key_type, sizeof(key_type) },
        { CKA_VALUE_LEN, &key_len, sizeof(key_len) },
    };

```

```

        { CKA_DERIVE, &true_val, sizeof(CK_BBOOL) },
        { CKA_ALLOWED_MECHANISMS, &key_allowed_mech,
sizeof(CK_MECHANISM_TYPE) },
    };

    /* Build HKDF-Expand-Label info structure */
    info_len = build_hkdf_label(info, 32, label, transcript_hash, 32);

    rv = p11_func->C_DeriveKey(session, &mechanism, shared_secret,
                                template, 5, traffic_secret);
    if (rv != CKR_OK) {
        printf("C_DeriveKey (HKDF - %s) failed: 0x%lx\n", label, rv);
        return -1;
    }

    printf("Derived %s (handle: %lu)\n", label, *traffic_secret);
    return 0;
}

/* Step 3: Derive key encryption */
static int derive_encryption_key(CK_SESSION_HANDLE session,
                                  CK_OBJECT_HANDLE traffic_secret,
                                  CK_OBJECT_HANDLE *aes_key)
{
    CK_RV rv;
    CK_BYTE info[256];
    CK_ULONG info_len;

    CK_HKDF_PARAMS hkdf_params = {
        .bExtract = CK_FALSE,
        .bExpand = CK_TRUE,
        .prfHashMechanism = CKM_SHA256,
        .ulSaltType = CKF_HKDF_SALT_NULL,
        .pSalt = NULL,
        .ulSaltLen = 0,
        .pInfo = info,
        .ulInfoLen = info_len
    };

    CK_MECHANISM mechanism = {
        .mechanism = CKM_HKDF_DERIVE,
        .pParameter = &hkdf_params,
        .ulParameterLen = sizeof(hkdf_params)
    };

    CK_OBJECT_CLASS secret_class = CKO_SECRET_KEY;
    CK_KEY_TYPE key_type = CKK_AES;
    CK_ULONG key_len = 16; /* AES-128 */
    CK_BBOOL true_val = CK_TRUE;
    CK_MECHANISM_TYPE key_allowed_mech = { CKM_AES_GCM };

    CK_ATTRIBUTE template[] = {
        { CKA_CLASS, &secret_class, sizeof(secret_class) },
        { CKA_KEY_TYPE, &key_type, sizeof(key_type) },
        { CKA_VALUE_LEN, &key_len, sizeof(key_len) },
        { CKA_ENCRYPT, &true_val, sizeof(CK_BBOOL) },
        { CKA_DECRYPT, &true_val, sizeof(CK_BBOOL) },
        { CKA_ALLOWED_MECHANISMS, &key_allowed_mech,
sizeof(CK_MECHANISM_TYPE) },
    };

```

```

};

/* Build HKDF-Expand-Label info for "key" derivation */
info_len = build_hkdf_label(info, 16, "key", NULL, 0);

rv = p11_func->C_DeriveKey(session, &mechanism, traffic_secret,
                           template, 6, aes_key);
if (rv != CKR_OK) {
    printf("C_DeriveKey (AES key) failed: 0x%lx\n", rv);
    return -1;
}

printf("Derived AES-128-GCM encryption key (handle: %lu)\n", *aes_key);
return 0;
}

/* Step 4: Derive IV from traffic secret */
static int derive_iv(CK_SESSION_HANDLE session,
                    CK_OBJECT_HANDLE traffic_secret,
                    CK_BYTE *iv,
                    CK_ULONG iv_len)
{
    CK_RV rv;
    CK_BYTE info[256];
    CK_ULONG info_len;
    CK_OBJECT_HANDLE temp_key;

    CK_HKDF_PARAMS hkdf_params = {
        .bExtract = CK_FALSE,
        .bExpand = CK_TRUE,
        .prfHashMechanism = CKM_SHA256,
        .ulSaltType = CKF_HKDF_SALT_NULL,
        .pSalt = NULL,
        .ulSaltLen = 0,
        .pInfo = info,
        .ulInfoLen = info_len
    };

    CK_MECHANISM mechanism = {
        .mechanism = CKM_HKDF_DERIVE,
        .pParameter = &hkdf_params,
        .ulParameterLen = sizeof(hkdf_params)
    };

    CK_OBJECT_CLASS secret_class = CKO_SECRET_KEY;
    CK_KEY_TYPE key_type = CKK_GENERIC_SECRET;
    CK_BBOOL true_val = CK_TRUE;
    CK_BBOOL false_val = CK_FALSE;

    CK_ATTRIBUTE template[] = {
        { CKA_CLASS, &secret_class, sizeof(secret_class) },
        { CKA_KEY_TYPE, &key_type, sizeof(key_type) },
        { CKA_VALUE_LEN, &iv_len, sizeof(iv_len) },
        { CKA_EXTRACTABLE, &true_val, sizeof(CK_BBOOL) },
        { CKA_TOKEN, &false_val, sizeof(CK_BBOOL) }
    };

    /* Build HKDF-Expand-Label info for "iv" derivation */
    info_len = build_hkdf_label(info, iv_len, "iv", NULL, 0);

```



```

/* Derive IV as extractable generic secret */
rv = pll_func->C_DeriveKey(session, &mechanism, traffic_secret,
                           template, 5, &temp_key);

if (rv != CKR_OK) {
    printf("C_DeriveKey (IV) failed: 0x%lx\n", rv);
    return -1;
}

/* Extract the IV value */
CK_ATTRIBUTE extract_template[] = {
    { CKA_VALUE, iv, iv_len }
};

rv = pll_func->C_GetAttributeValue(session, temp_key,
                                   extract_template, 1);

if (rv != CKR_OK) {
    printf("C_GetAttributeValue (IV) failed: 0x%lx\n", rv);
    pll_func->C_DestroyObject(session, temp_key);
    return -1;
}

/* Clean up temporary key */
pll_func->C_DestroyObject(session, temp_key);

return 0;
}

/* Complete TLS 1.3 key derivation workflow */
int tls13_derive_keys_pkcs11(...)
{
    CK_OBJECT_HANDLE ecdhe_key = CK_INVALID_HANDLE;
    CK_OBJECT_HANDLE client_hs_secret = CK_INVALID_HANDLE;
    CK_OBJECT_HANDLE server_hs_secret = CK_INVALID_HANDLE;
    CK_OBJECT_HANDLE client_aes_key = CK_INVALID_HANDLE;
    CK_OBJECT_HANDLE server_aes_key = CK_INVALID_HANDLE;
    CK_BYTE client_iv[12];
    CK_BYTE server_iv[12];
    int ret = 0;
    CK_RV rv;

    /* Compute transcript hash: Hash(ClientHello...ServerHello) */
    /* ... */

    /* Peer key and ecdsa_key_id obtained from key exchange */

    printf("\n=== TLS 1.3 Key Derivation for TLS_AES_128_GCM_SHA256 ===\n");

    /* Step 1: Derive ECDH shared secret */
    if (derive_ecdh_shared_secret(session, ecdsa_key_id, peer_public_key,
                                  sizeof(peer_public_key),
                                  &ecdhe_key) != 0) {
        ret = -1;
        goto cleanup;
    }

    /* Step 2: Derive client handshake traffic secret */
    if (derive_handshake_traffic_secret(session, ecdhe_key,
                                         transcript_hash,
                                         "c hs traffic",
                                         &client_hs_secret) != 0) {

```

```

        ret = -1;
        goto cleanup;
    }

    /* Step 3: Derive server handshake traffic secret */
    if (derive_handshake_traffic_secret(session, ecdhe_key,
                                        transcript_hash,
                                        "s hs traffic",
                                        &server_hs_secret) != 0) {

        ret = -1;
        goto cleanup;
    }

    /* Step 4: Derive client AES-128-GCM key */
    if (derive_encryption_key(session, client_hs_secret,
                              &client_aes_key) != 0) {

        ret = -1;
        goto cleanup;
    }

    /* Step 5: Derive server AES-128-GCM key */
    if (derive_encryption_key(session, server_hs_secret,
                              &server_aes_key) != 0) {

        ret = -1;
        goto cleanup;
    }

    /* Step 6: Derive client IV */
    if (derive_iv(session, client_hs_secret, client_iv,
                  sizeof(client_iv)) != 0) {

        ret = -1;
        goto cleanup;
    }

    /* Step 7: Derive server IV */
    if (derive_iv(session, server_hs_secret, server_iv,
                  sizeof(server_iv)) != 0) {

        ret = -1;
        goto cleanup;
    }

    printf("\n=== TLS 1.3 key derivation completed successfully! ===\n");
    printf("\nDerived keys summary:\n");
    printf("- ECDH shared secret handle: %lu\n", ecdhe_key);
    printf("- Client handshake traffic secret handle: %lu\n",
           client_hs_secret);
    printf("- Server handshake traffic secret handle: %lu\n",
           server_hs_secret);
    printf("- Client AES-128-GCM key handle: %lu\n", client_aes_key);
    printf("- Server AES-128-GCM key handle: %lu\n", server_aes_key);
    printf("\nThese keys can now be used for TLS 1.3 handshake encryption.\n");

    /* Now you can use keys and ivs for encryption/decryption */
    /* ... */
}

```

5.2.11.12 Security Considerations

TLS 1.3 provides several security improvements over TLS 1.2:

1. **Forward Secrecy:** All handshakes use ephemeral key exchange (ECDHE)
2. **Simplified Key Schedule:** Clearer separation of key derivation stages
3. **AEAD-only Ciphers:** All cipher suites use authenticated encryption
4. **Reduced Attack Surface:** Removed legacy features and weak algorithms
5. **0-RTT Support:** Optional early data with replay protection considerations

Warning:

When using 0-RTT (early data), be aware of replay attack risks. Only use 0-RTT for idempotent operations or implement additional replay protection.

5.2.11.13 API Comparison

Table 55. TLS 1.3 Key Derivation APIs

API	Functions / Algorithm
SMW	smw_derive_key() SMW_KDF_NAME_TLS13 SMW_KDF_NAME_TLS13_EXPAND_LABEL
PSA	psa_key_derivation_output_key() / psa_key_derivation_output_bytes() PSA_ALG_VENDOR_TLS13(hash_alg)
PKCS11	C_DeriveKey() CKM_ECDH1_DERIVE CKM_HKDF_DERIVE

5.2.11.14 References

- [RFC 8446](#) - The Transport Layer Security (TLS) Protocol Version 1.3
- [RFC 5869](#) - HMAC-based Extract-and-Expand Key Derivation Function (HKDF)
- [RFC 7748](#) - Elliptic Curves for Security (X25519 and X448)
- [RFC 8439](#) - ChaCha20 and Poly1305 for IETF Protocols

5.2.12 Get key attributes

The **get key attributes** operation is used to retrieve metadata about stored key from the secure subsystem without retrieving the actual key material.

Table 56. Get key attributes APIs Comparison

API	Function
SMW	smw_get_key_attributes()
PSA	psa_get_key_attributes()
PKCS11	C_GetAttributeValue()

The metadata available for the key are described in the [Objects Attributes definitions](#) chapter and are:

- **Type:** Type of the key (e.g. RSA, ECC, AES, HMAC, etc.).
- **Security size:** The security size of the data in bits.
- **Permitted algorithm:** Algorithms that are allowed to be used with this key.

- **Key usage:** Indicates the operations permitted on the key (e.g., sign, verify, encrypt, decrypt, derive, etc.).
- **Persistence:** Indicates whether the key is persistent or volatile.
- **Lifecycle:** Indicates the device lifecycle stage where the key is valid.
- **Storage location:** Key storage location.

Note:

The key attributes are not all managed by the different secure subsystem like SECO, TEE. In this case, the SMW library will provide the key attributes stored in its database when data was created by SMW and if the key object is still referenced.

Note:

If the key is not referenced in the SMW database, each secure subsystem enabled in the SMW configuration is queried to retrieve the missing key attributes in the limit of information managed by the secure subsystem, some may not be retrieval.

5.2.13 Public Key Attestation

The **public key attestation** operation provides cryptographic proof that a asymmetric public key is owned by a secure subsystem. This operation produces a signed attestation certificate that can be verified by a third party to establish trust in the key's provenance and security properties.

Table 57. Key Attestation Subsystem Support

Subsystem	Support	Attestation Certificate
ELE	Yes	Custom TLV certificate structure
TEE	No	Not supported
SECO	No	Not supported

Table 58. Key Attestation APIs Comparison

API	Function
SMW	smw_key_attestation()
PSA	<code>psa_attest_key()</code>
PKCS11	Not supported

5.2.13.1 Custom TLV Certificate Structure

The following table describes the custom TLV (Tag-Length-Value) certificate structure used for key attestation in the ELE subsystem:

Table 59. Custom TLV Key attestation certificate

Field		Tag	Length (bytes)	Description / Value
Key ID		0x41	4	Identifier, in the subsystem, of the key attest.
Key properties	Algorithm	0x42	4	ELE key permitted algorithm.
	Usage	0x43	4	ELE key usage flags.
	Type	0x44	2	ELE Type of key.
	Bits	0x45	4	Key security size in bits.

Table 59. Custom TLV Key attestation certificate...continued

Field		Tag	Length (bytes)	Description / Value
	Lifetime	0x46	4	ELE Lifetime: • Key generated by the device: – 0x00000000: Transient key – 0x00000001: Persistent key – 0x000000FF: Permanent key • Key imported with EdgeLock Enclave import: – 0xC0020000: Transient key – 0xC0020001: Persistent key – 0xC00200FF: Permanent key • Key imported with EdgeLock 2GO import: – 0xE0000200: Transient key – 0xE0000201: Persistent key – 0xE00002FF: Permanent key
Key lifecycle		0x47	4	Key lifecycle usage flags: • OEM OPEN: 0x01. • OEM CLOSED: 0x02. • OEM CLOSED_LOCKED: 0x04.
Attestation challenge		0x50	Variable	Input value chosen by the user, could be a random or the current data/time.
Key attestation ID		0x51	4	Key identifier in the subsystem of the key used to attest the public key by signing this TLV.
Public key buffer		0x52	Variable	Public key buffer (big-endian).
Key origin		0x53	1	Information about the key origin: • 0x01: Key has been generated by the device. • 0x02: Key has been imported into the device. • 0x03: Key has been injected into the device. • 0x04: Key has been imported with EdgeLock 2GO provisioning. • 0x05: Key has been derived.
Device UUID		0x54	16	Device UUID (big-endian).
Signing algorithm		0x56	4	Algorithm used to sign the blob itself. Field Signature of this blob: Possible values are: • 0x01: CMAC. • 0x02: ECDSA-NIST.
Signature		0x5E	16	Signature of all previous fields of this blob including the signature tag (0x5E) and signature length fields.

5.3 Data Management

This section documents the data management operations supported across different security subsystems (ELE, TEE, SECO).

5.3.1 Supported Operations versus Subsystems

Table 60. Data Management Operations vs. subsystem

Operations		Subsystem			Notes
		ELE	TEE	SECO	
Store Data	Plaintext	Y	Y	Y	
	EdgeLock 2GO blob	Y*	N	N	The psa_its_set() can't be used to provision a EdgeLock 2GO data, the psa_import_key() function must be used.
	Encrypt and Sign	Y	N	N	
Retrieve Data		Y	Y	Y	
Delete Data		Y	Y	N	
Get Information		Y	Y	Y*	SECO managed limited information.

5.3.2 Data Size Limits

Table 61. Data Size Limits vs. subsystem

Subsystem	Maximum Size (bytes)	Notes
ELE	2048	Limited by secure storage capacity
TEE	N/A	Limited by available secure filesystem
SECO	2048	Limited by secure storage capacity

5.3.3 Store Data

The store data operation is used to securely store arbitrary data in the secure subsystem's persistent storage.

Table 62. Store Data APIs Comparison

API	Function
SMW	smw_store_data()
PSA	psa_its_set()
PKCS11	C_CreateObject() CKO_DATA

5.3.3.1 Plaintext

The data stored in plaintext are retrieved also in plaintext.

5.3.3.2 EdgeLock 2GO blob

This service is a NXP provisioning service involving the NXP EdgeLock 2GO server. It's only supported by the ELE subsystem.

The server allows to provision key and data embedded in blob. Visit [EdgeLock 2GO](#) for more information.

The type of key blob to import is identified by the key location attribute:

- SMW API, the key location is defined by the 'storage_id' field of the key attribute in the key descriptor (refer to the [smw_key_descriptor](#)).

- PSA API, the key location is defined by the 'lifetime' field of the key attribute (refer to the [psa_key_attributes_t](#)). The location is defined by the bits[31:8] of the field.

Table 63. EdgeLock 2GO - Location

	SMW 'storage_id'	PSA 'lifetime' bits[31:8]
EdgeLock 2GO Key	0x00E00000	0xE00000
EdgeLock 2GO Data	0x00E08000	0xE08000

Note:

The SMW [smw_store_data\(\)](#) allows to store EdgeLock 2GO blob of type data, but the [psa_key_set\(\)](#) can't be used to store EdgeLock 2GO blob. The [psa_import_key\(\)](#) must be used to provision a EdgeLock 2GO data blob identified by the PSA's lifetime attribute. More information in the [Import Key](#) chapter.

5.3.3.3 Encrypt and Sign

The plaintext data can be encrypted and signed before being stored in the secure subsystem's persistent storage. This feature is available by ELE secure subsystem and using the SMW API, only.

The encryption and signature algorithms are defined in the argument structure of the [smw_store_data\(\)](#).

5.3.3.3.1 Encryption of the data

The encryption key is a symmetric key present in the secure subsystem storage and identified by its key identifier. The encryption algorithm supported is a symmetric encryption mode as defined in the [Symmetric Encryption](#).

Note that an Initialization Vector (IV) might be required depending on the encryption mode selected. The IV can be given as input parameter or generated by the secure subsystem. The value of the IV is returned in the resulting data blob.

5.3.3.3.2 Signature of the data

The signature key is symmetric key present in the secure subsystem storage and identified by its key identifier. The signature algorithm supported is CMAC (Cipher-based Message Authentication Code) only as defined in the [Message Authentication Code \(MAC\)](#).

5.3.3.3.3 Data blob format

The encrypted and signed data is retrieved through a TLV (Tag-Length-Value) blob format. The following [Encrypted and Signed Data - TLV Format](#) table describes the structure of the encrypted and signed data blob including metadata tags for device identification, initialization vectors, encrypted payload and blob signature.

The encrypted and signed data is stored in a TLV blob format as detailed in the [Encrypted and Signed Data - TLV Format](#) table below.

Table 64. Encrypted and Signed Data - TLV Format

Tag	Length (bytes)	Value/Description
0x41	16	Device UUID in big endian format.
0x45	16	Value of the IV used to encrypt data in case encryption algorithm use an IV. The IV can be either: • User input value.

Table 64. Encrypted and Signed Data - TLV Format...continued

Tag	Length (bytes)	Value/Description
		Randomly generated by the secure. In this case, the user input IV buffer address must be NULL and its length equal 0.
0x46	Variable	Encrypted data. Maximum length is 2048 bytes.
0x5E	16	Signature of all previous fields of this blob including the signature tag (0x5E) and signature length fields.

5.3.4 Retrieve Data

The retrieve data operation is used to read previously stored data from the secure subsystem's persistent storage.

Table 65. Retrieve Data APIs Comparison

API	Function
SMW	smw_retrieve_data()
PSA	psa_its_get()
PKCS11	C_GetAttributeValue() CKA_VALUE

5.3.5 Delete Data

The delete data operation is used to permanently remove data from the secure subsystem's persistent storage.

Table 66. Delete Data APIs Comparison

API	Function
SMW	smw_delete_data()
PSA	psa_its_remove()
PKCS11	C_DestroyObject()

Note:

Even if the [psa_its_set\(\)](#) function can't be used to store A EdgeLock 2GO data blob, the [psa_its_remove\(\)](#) function must be used to delete it.

5.3.6 Get Information

The get information operation is used to retrieve metadata about stored data from the secure subsystem's persistent storage without retrieving the actual data content.

Table 67. Get Information APIs Comparison

API	Function
SMW	smw_get_data_info()
PSA	psa_its_get_info()
PKCS11	C_GetAttributeValue()

The metadata available for the data are described in the [Objects Attributes definitions](#) chapter and are:

- **Size:** The size of the stored data in bytes.

- **Persistence:** Indicates whether the data is persistent or volatile.
- **Lifecycle:** Indicates the device lifecycle stage where the key is valid.
- **Read/Write protection:** Access control policies applied to the data object.
- **Storage location:** Data storage location.

Note:

If the data is not referenced in the SMW database, each secure subsystem enabled in the SMW configuration is queried to retrieve the missing data information within the limits of information managed by the secure subsystem. Some information may not be retrievable depending on subsystem capabilities.

5.4 Digest (Hash)

This section documents the hash/digest operations supported across different security subsystems (ELE, TEE, SECO) with their respective algorithms.

5.4.1 Supported Operations versus Subsystems

Table 68. Hash Operations vs. subsystem

Algorithm	Digest Size (bytes)	Processing	Subsystem			Notes
			ELE	TEE	SECO	
MD5	16	Single-Part	Y*	Y	N	ELE not supported on: • i.MX8ULP
		Multi-Part	Y*	Y	N	
SHA-1	20	Single-Part	Y*	Y	Y	ELE not supported on: • i.MX8ULP
		Multi-Part	Y*	Y	Y	
SHA-224	28	Single-Part	Y	Y	Y	
		Multi-Part	Y	Y	Y	
SHA-256	32	Single-Part	Y	Y	Y	
		Multi-Part	Y	Y	Y	
SHA-384	48	Single-Part	Y	Y	Y	
		Multi-Part	Y	Y	Y	
SHA-512	64	Single-Part	Y	Y	Y	
		Multi-Part	Y	Y	Y	
SHA3-224	28	Single-Part	Y*	Y	N	ELE not supported on: • i.MX8ULP
		Multi-Part	Y*	Y	N	
SHA3-256	32	Single-Part	Y*	Y	N	ELE not supported on: • i.MX8ULP
		Multi-Part	Y*	Y	N	
SHA3-384	48	Single-Part	Y*	Y	N	ELE not supported on: • i.MX8ULP
		Multi-Part	Y*	Y	N	
SHA3-512	64	Single-Part	Y*	Y	N	ELE not supported on: • i.MX8ULP
		Multi-Part	Y*	Y	N	
SHAKE256	any length (default 64)	Single-Part	Y*	Y	N	ELE not supported on: • i.MX8ULP
		Multi-Part	Y*	Y	N	
SM3	32	Single-Part	Y*	Y	N	

Table 68. Hash Operations vs. subsystem...continued

Algorithm	Digest Size (bytes)	Processing	Subsystem			Notes
			ELE	TEE	SECO	
		Multi-Part	Y*	Y	N	

Note:

The digest multi-part support on **SECO** subsystem is calculated by SW and not by the Secure Enclave.

Note:

SHAKE256 is an extendable-output function (XOF). The digest size can be any size greater than 0. The digest size shown is the default output length (64 bytes).

Note:

- i.MX8ULP, i.MX91, i.MX93 and i.MX95 don't support the SM3 algorithm with ELE.

Table 69. Hash APIs

Operations	Processing	API	Functions
Hash	Single-Part	SMW	smw_hash()
		PSA	psa_hash_compute()
		PKCS11	C_DigestInit() C_Digest()
	Multi-Part	SMW	smw_hash_init() smw_hash_update() smw_hash_final()
		PSA	psa_hash_setup() psa_hash_update() psa_hash_finish()
		PKCS11	C_DigestInit() C_DigestUpdate() C_DigestFinal()

5.4.2 API Algorithm Identifiers

The following table maps hash algorithm identifiers across different APIs.

Table 70. Hash Algorithm Identifiers vs. API

Algorithm	API	Define/Mechanism
MD5	SMW	SMW_HASH_ALGO_NAME_MD5
	PSA	PSA_ALG_MD5
	PKCS11	CKM_MD5
SHA-1	SMW	SMW_HASH_ALGO_NAME_SHA1
	PSA	PSA_ALG_SHA_1
	PKCS11	CKM_SHA_1
SHA-224	SMW	SMW_HASH_ALGO_NAME_SHA224
	PSA	PSA_ALG_SHA_224

Table 70. Hash Algorithm Identifiers vs. API...continued

Algorithm	API	Define/Mechanism
	PKCS11	CKM_SHA224
SHA-256	SMW	SMW_HASH_ALGO_NAME_SHA256
	PSA	PSA_ALG_SHA_256
	PKCS11	CKM_SHA256
SHA-384	SMW	SMW_HASH_ALGO_NAME_SHA384
	PSA	PSA_ALG_SHA_384
	PKCS11	CKM_SHA384
SHA-512	SMW	SMW_HASH_ALGO_NAME_SHA512
	PSA	PSA_ALG_SHA_512
	PKCS11	CKM_SHA512
SHA3-224	SMW	SMW_HASH_ALGO_NAME_SHA3_224
	PSA	PSA_ALG_SHA3_224
	PKCS11	CKM_SHA3_224
SHA3-256	SMW	SMW_HASH_ALGO_NAME_SHA3_256
	PSA	PSA_ALG_SHA3_256
	PKCS11	CKM_SHA3_256
SHA3-384	SMW	SMW_HASH_ALGO_NAME_SHA3_384
	PSA	PSA_ALG_SHA3_384
	PKCS11	CKM_SHA3_384
SHA3-512	SMW	SMW_HASH_ALGO_NAME_SHA3_512
	PSA	PSA_ALG_SHA3_512
	PKCS11	CKM_SHA3_512
SHAKE256	SMW	SMW_HASH_ALGO_NAME_SHAKE256
	PSA	PSA_ALG_SHAKE256_512
	PKCS11	Not Defined
SM3	SMW	SMW_HASH_ALGO_NAME_SM3
	PSA	PSA_ALG_SM3
	PKCS11	Not Defined

5.5 Message Authentication Code (MAC)

This section documents the Message Authentication Code (MAC) operations supported across different security subsystems (ELE, TEE, SECO) with their respective algorithms and key types.

5.5.1 Supported Operations versus Subsystems

Table 71. MAC Operations vs. subsystem

MAC Algorithm	Processing	Subsystem			Notes
		ELE	TEE	SECO	
HMAC	Single-Part	Y	Y	Y*	SECO Require specific Firmware.
Truncated HMAC	Single-Part	Y	N	N	Generated HMAC Length is truncated. MAC Length must be 8 bytes up to Full MAC Length.
CMAC	Single-Part	Y	Y	Y	
Truncated CMAC	Single-Part	Y	N	N	Generated CMAC Length is truncated. MAC Length must be 8 bytes up to Full MAC Length.

Table 72. MAC APIs

Operations	Processing	API	Functions
MAC Generation	Single-Part	SMW	smw_mac()
		PSA	psa_mac_compute()
		PKCS11	C_SignInit() C_Sign()
			C_MessageSignInit() C_SignMessage()
MAC Verification	Single-Part	SMW	smw_mac()
		PSA	psa_mac_verify()
		PKCS11	C_VerifyInit() C_Verify()
			C_MessageVerifyInit() C_VerifyMessage()

5.5.2 HMAC

Table 73. HMAC Support Details

Hash	MAC Length (bytes)	Subsystem	Key	
			Opaque	Plaintext
MD5	16	ELE	N	N
		TEE	Y	Y
		SECO	N	N
SHA-1	20	ELE	N	N
		TEE	Y	Y
		SECO	N	N
SHA-224	28	ELE	N	N
		TEE	Y	Y

Table 73. HMAC Support Details...continued

Hash	MAC Length (bytes)	Subsystem	Key	
			Opaque	Plaintext
		SECO	Y	N
SHA-256	32	ELE	Y	Y (1)
		TEE	Y	Y
		SECO	Y	N
SHA-384	48	ELE	Y	Y (1)
		TEE	Y	Y
		SECO	Y	N
SHA-512	64	ELE	N	N (2)
		TEE	Y	Y
		SECO	Y	N
SHA3-224	28	ELE	N	N
		TEE	Y	Y
		SECO	N	N
SHA3-256	32	ELE	N	N
		TEE	Y	Y
		SECO	N	N
SHA3-384	48	ELE	N	N
		TEE	Y	Y
		SECO	N	N
SHA3-512	64	ELE	N	N
		TEE	Y	Y
		SECO	N	N
SM3	32	ELE	N	N
		TEE	Y	Y
		SECO	N	N

1. ELA hardware acceleration supports HMAC SHA-256, HMAC SHA-384 and HMAC SHA-512 on all platforms except i.MX8ULP, i.MX91, i.MX93 and i.MX95 with plaintext key buffer only. ELA requires a non-empty input message.
2. ELE subsystem does not support HMAC SHA-512. However, ELA does support HMAC SHA-512 with plaintext key buffer.

5.5.2.1 Key attributes

5.5.2.1.1 Key Usage

For MAC operations, keys must be configured with appropriate key usage flags:

Table 74. HMAC Key Usage Flags

Operation	API	Required Key Usage
MAC Generation	SMW	SMW_ATTR_USAGE_SIGN_MESSAGE
	PSA	PSA_KEY_USAGE_SIGN_MESSAGE
	PKCS11	CKA_SIGN
MAC Verification	SMW	SMW_ATTR_USAGE_VERIFY_MESSAGE
	PSA	PSA_KEY_USAGE_VERIFY_MESSAGE
	PKCS11	CKA_VERIFY

5.5.2.1.2 Permitted Algorithm

The following table outlines the HMAC permitted algorithms.

Depending of the subsystem, the HMAC generated can be truncated (refer to the [MAC Operations vs. subsystem](#) table). In this case, the key permitted algorithm can be set to limit key usage when HMAC generated is truncated.

The [Permitted Algorithms for Full HMAC](#) table details the key permitted algorithms for full HMAC operations across different APIs and hash. The Truncated HMAC is supported with this permitted algorithm.

The [Permitted Algorithms for Truncated HMAC](#) table details the key permitted algorithms for Truncated HMAC operations.

Table 75. Permitted Algorithms for Full HMAC

MAC	API	Permitted Algorithm
HMAC	SMW	SMW_ATTR_ALGO_MAC_HMAC(hash, mac) Where: - hash is one of SMW Attribute Hash algorithm define in the Algorithm Hash value table. - mac is 0
	PSA	PSA_ALG_HMAC(hash) Where hash is one of PSA Hash algorithm (PSA_ALG_XXX) such as PSA_ALG_IS_HASH is true.
	PKCS11	Key's CKA_ALLOWED_MECHANISMS must be one or more CKM_SHAxxx_HMAC value. If multiple CKM_SHAxxx_HMAC values are set, the key permitted algorithm is HMAC any hash type.

Table 76. Permitted Algorithms for Truncated HMAC

MAC	API	Permitted Algorithm
HMAC fix truncated length	SMW	SMW_ATTR_ALGO_MAC_HMAC(hash, mac) Where: - hash is one of SMW Attribute Hash algorithm define in the Algorithm Hash value table. - mac is output MAC length that can't exceed the full MAC length as detailed in the HMAC Support Details table.
	PSA	PSA_ALG_TRUNCATED_MAC(PSA_ALG_HMAC(hash), mac_length) Where: - hash is one of PSA Hash algorithm (PSA_ALG_XXX) such as PSA_ALG_IS_HASH is true. - mac_length is output MAC length that can't exceed the full MAC length as detailed in the HMAC Support Details table.

Table 76. Permitted Algorithms for Truncated HMAC...continued

MAC	API	Permitted Algorithm
HMAC minimum truncated length	SMW	SMW_ATTR_SET_MIN_MAC_LENGTH(SMW_ATTR_ALGO_MAC_HMAC(hash, 0), length) Where: - hash is one of SMW Attribute Hash algorithm define in the Algorithm Hash value table. - length is minimum output MAC length, can't exceed the full MAC length as detailed in the HMAC Support Details table.
	PSA	PSA_ALG_AT_LEAST_THIS_LENGTH_MAC(PSA_ALG_HMAC(hash), min_mac_length) Where: - hash is one of PSA Hash algorithm (PSA_ALG_XXX) such as PSA_ALG_IS_HASH is true. - min_mac_length is minimum output MAC length that can't exceed the full MAC length as detailed in the HMAC Support Details table.

5.5.3 CMAC

Table 77. CMAC Support Details

Cipher	MAC Size (bytes)	Subsystem	Key	
			Opaque	Plaintext
AES	16	ELE	Y	Y
		TEE	Y	Y
		SECO	Y	N

5.5.3.1 Key attributes

5.5.3.1.1 Key Usage

For MAC operations, keys must be configured with appropriate key usage flags:

Table 78. CMAC Key Usage Flags

Operation	API	Required Key Usage
MAC Generation	SMW	SMW_ATTR_USAGE_SIGN_MESSAGE
	PSA	PSA_KEY_USAGE_SIGN_MESSAGE
	PKCS11	CKA_SIGN
MAC Verification	SMW	SMW_ATTR_USAGE_VERIFY_MESSAGE
	PSA	PSA_KEY_USAGE_VERIFY_MESSAGE
	PKCS11	CKA_VERIFY

5.5.3.1.2 Permitted Algorithm

The following table outlines the CMAC permitted algorithms.

Table 79. Permitted Algorithms for CMAC

MAC	API	Permitted Algorithm
CMAC	SMW	SMW_ATTR_ALGO_MAC_CMAC(cipher) Where cipher is SMW_ATTR_CIPHER_AES.
	PSA	PSA_ALG_CMAC

Table 79. Permitted Algorithms for CMAC...continued

MAC	API	Permitted Algorithm
		Refer to <code>PSA_ALG_CMALC()</code> .
	PKCS11	Key's <code>CKA_ALLOWED_MECHANISMS</code> must be <code>CKM_AES_CMALC</code> value.

Depending of the subsystem, the CMAC generated can be truncated (refer to the [MAC Operations vs. subsystem](#) table). In this case, the key permitted algorithm can be set to limit key usage when CMAC generated is truncated.

The [Permitted Algorithms for Full CMAC](#) table details the key permitted algorithms for full CMAC operations across different APIs and hash. The Truncated HMAC is supported with this permitted algorithm.

The [Permitted Algorithms for Truncated CMAC](#) table details the key permitted algorithms for Truncated CMAC operations.

Table 80. Permitted Algorithms for Full CMAC

MAC	API	Permitted Algorithm
HMAC	SMW	SMW_ATTR_ALGO_MAC(algo, mode, mac) Where: • algo is SMW_ATTR_ALGO_AES. • mode is SMW_ATTR_MODE_CMALC. • mac is 0
	PSA	PSA_ALG_CMALC Refer to <code>PSA_ALG_CMALC()</code> .
	PKCS11	Key's <code>CKA_ALLOWED_MECHANISMS</code> must be <code>CKM_AES_CMALC</code> value.

Table 81. Permitted Algorithms for Truncated CMAC

MAC	API	Permitted Algorithm
CMAC fix truncated length	SMW	SMW_ATTR_ALGO_MAC(algo, mode, mac) Where: • algo is SMW_ATTR_ALGO_AES. • mode is SMW_ATTR_MODE_CMALC. • mac is output MAC length that can't exceed the full MAC length as detailed in the CMAC Support Details table.
	PSA	PSA_ALG_TRUNCATED_MAC(PSA_ALG_CMALC, mac_length) Where: • mac_length is output MAC length that can't exceed the full MAC length as detailed in the CMAC Support Details table.
CMAC minimum truncated length	SMW	SMW_ATTR_SET_MIN_MAC_LENGTH(SMW_ATTR_ALGO_MAC(algo, mode, 0), length) Where: • algo is SMW_ATTR_ALGO_AES. • mode is SMW_ATTR_MODE_CMALC. • length is minimum output MAC length, can't exceed the full MAC length as detailed in the CMAC Support Details table.
	PSA	PSA_ALG_AT_LEAST_THIS_LENGTH_MAC(PSA_ALG_CMALC, min_mac_length) Where: • min_mac_length is minimum output MAC length that can't exceed the full MAC length as detailed in the CMAC Support Details table.

5.6 Symmetric Encryption

This section documents the symmetric encryption operations supported across different security subsystems (ELE, TEE, SECO) with their respective modes and algorithms.

5.6.1 Supported Operations versus Subsystems

Table 82. Symmetric Encryption Operations vs. subsystem

Key Type	Modes	Processing	Subsystem		
			ELE	TEE	SECO
AES	Electronic Codebook (ECB) with no padding	Single-Part	Y (2)	Y	Y
		Multi-Part	Y (1)	Y	N
	Cipher Block Chaining (CBC) with no padding	Single-Part	Y (2)	Y	Y
		Multi-Part	Y (1)	Y	N
	Counter (CTR)	Single-Part	Y (2)	Y	N
		Multi-Part	Y (1)	Y	N
	Cipher Feedback (CFB)	Single-Part	Y	N	N
		Multi-Part	Y (1)	N	N
	Output Feedback (OFB)	Single-Part	Y	N	N
		Multi-Part	Y (1)	N	N
	Ciphertext Stealing (CTS)	Single-Part	N	Y	N
		Multi-Part	N	Y	N
	XEX with Ciphertext Stealing (XTS)	Single-Part	N	Y	N
		Multi-Part	N	Y	N
DES	Electronic Codebook (ECB) with no padding	Single-Part	N	Y	N
		Multi-Part	N	Y	N
	Cipher Block Chaining (CBC) with no padding	Single-Part	N	Y	N
		Multi-Part	N	Y	N
Triple DES (DES3)	Electronic Codebook (ECB) with no padding	Single-Part	N	Y	N
		Multi-Part	N	Y	N
	Cipher Block Chaining (CBC) with no padding	Single-Part	N	Y	N
		Multi-Part	N	Y	N
SM4	Electronic Codebook (ECB) with no padding	Single-Part	N	Y	N
		Multi-Part	N	Y	N
	Cipher Block Chaining (CBC) with no padding	Single-Part	N	Y	N
		Multi-Part	N	Y	N
	Counter (CTR)	Single-Part	N	Y	N
		Multi-Part	N	Y	N

1. ELE multi-part encryption and decryption operations not supported on the i.MX8ULP, i.MX93 and i.MX91 platforms.

2. ELA hardware acceleration is supported on i.MX94x (i.MX943, i.MX942, i.MX941), i.MX952 and i.MX937 platforms when using plaintext key buffers.

Table 83. Symmetric Encryption APIs

Operations	Processing	API	SMW	
Encryption	Single-Part	SMW	smw_cipher()	
		PSA	psa_cipher_encrypt()	
		PKCS11	C_EncryptInit() C_Encrypt()	
			C_MessageEncryptInit() C_EncryptMessage()	
	Multi-Part	SMW	smw_allocate_context() smw_cipher_init() smw_cipher_update() smw_cipher_final() smw_cancel_operation()	
		PSA	psa_cipher_encrypt_setup() psa_cipher_generate_iv() psa_cipher_set_iv() psa_cipher_update() psa_cipher_finish() psa_cipher_abort()	
		PKCS11	C_EncryptInit() C_EncryptUpdate() C_EncryptFinal()	
			C_MessageEncryptInit() C_EncryptMessageBegin() C_EncryptMessageNext() C_EncryptMessageFinal()	
Decryption	Single-Part	SMW	smw_cipher()	
		PSA	psa_cipher_decrypt()	
		PKCS11	C_DecryptInit() C_Decrypt()	
			C_MessageDecryptInit() C_DecryptMessage()	
	Multi-Part	SMW	smw_allocate_context() smw_cipher_init() smw_cipher_update() smw_cipher_final() smw_cancel_operation()	
		PSA	psa_cipher_decrypt_setup() psa_cipher_set_iv() psa_cipher_update() psa_cipher_finish() psa_cipher_abort()	
		PKCS11	C_DecryptInit()	

Table 83. Symmetric Encryption APIs...continued

Operations	Processing	API	SMW
			C_DecryptUpdate()
			C_DecryptFinal()
			C_MessageDecryptInit()
			C_DecryptMessageBegin()
			C_DecryptMessageNext()
			C_DecryptMessageFinal()

5.6.2 AES Symmetric Encryption

Table 84. AES Support Details

Key Security Size (bits)	Subsystem	Key	
		Opaque	Plaintext
128	ELE	Y	Y
	TEE	Y	Y
	SECO	Y	N
192	ELE	Y	Y
	TEE	Y	Y
	SECO	Y	N
256	ELE	Y	Y
	TEE	Y	Y
	SECO	Y	N

5.6.2.1 Key attributes

5.6.2.1.1 Key Usage

For AES Symmetric Encryption operations, keys must be configured with appropriate key usage flags:

Table 85. AES Symmetric encryption Key Usage Flags

Operation	API	Required Key Usage
Encryption	SMW	SMW_ATTR_USAGE_ENCRYPT
	PSA	PSA_KEY_USAGE_ENCRYPT
	PKCS11	CKA_ENCRYPT
Decryption	SMW	SMW_ATTR_USAGE_DECRYPT
	PSA	PSA_KEY_USAGE_DECRYPT
	PKCS11	CKA_DECRYPT

5.6.2.1.2 Permitted Algorithm

The following table outlines the permitted AES Symmetric Encryption algorithms.

Table 86. Permitted Algorithms for AES Symmetric Encryption

Key Type	API	Permitted Algorithm																		
AES	SMW	SMW_ATTR_ALGO_SYMMETRIC_ENCRYPTION(algo, mode) Where: • algo is SMW_ATTR_ALGO_AES. • mode is one of the value of the following table.																		
		<table><tr><th>Mode</th><th>Value</th></tr><tr><td>Electronic Codebook (ECB) with no padding</td><td>SMW_ATTR_MODE_ECB_NO_PAD</td></tr><tr><td>Cipher Block Chaining (CBC) with no padding</td><td>SMW_ATTR_MODE_CBC_NO_PAD</td></tr><tr><td>Counter (CTR)</td><td>SMW_ATTR_MODE_CTR</td></tr><tr><td>Cipher Feedback (CFB)</td><td>SMW_ATTR_MODE_CFB</td></tr><tr><td>Ciphertext Stealing (CTS)</td><td>SMW_ATTR_MODE_CTS</td></tr><tr><td>Output Feedback (OFB)</td><td>SMW_ATTR_MODE_OFB</td></tr><tr><td>XEX with Ciphertext Stealing (XTS)</td><td>SMW_ATTR_MODE_XTS</td></tr><tr><td>Any mode</td><td>SMW_ATTR_MODE_ANY</td></tr></table>	Mode	Value	Electronic Codebook (ECB) with no padding	SMW_ATTR_MODE_ECB_NO_PAD	Cipher Block Chaining (CBC) with no padding	SMW_ATTR_MODE_CBC_NO_PAD	Counter (CTR)	SMW_ATTR_MODE_CTR	Cipher Feedback (CFB)	SMW_ATTR_MODE_CFB	Ciphertext Stealing (CTS)	SMW_ATTR_MODE_CTS	Output Feedback (OFB)	SMW_ATTR_MODE_OFB	XEX with Ciphertext Stealing (XTS)	SMW_ATTR_MODE_XTS	Any mode	SMW_ATTR_MODE_ANY
		Mode	Value																	
		Electronic Codebook (ECB) with no padding	SMW_ATTR_MODE_ECB_NO_PAD																	
		Cipher Block Chaining (CBC) with no padding	SMW_ATTR_MODE_CBC_NO_PAD																	
		Counter (CTR)	SMW_ATTR_MODE_CTR																	
		Cipher Feedback (CFB)	SMW_ATTR_MODE_CFB																	
		Ciphertext Stealing (CTS)	SMW_ATTR_MODE_CTS																	
		Output Feedback (OFB)	SMW_ATTR_MODE_OFB																	
		XEX with Ciphertext Stealing (XTS)	SMW_ATTR_MODE_XTS																	
	Any mode	SMW_ATTR_MODE_ANY																		
	PSA	Permitted algorithm is one of the value of the following table.																		
		<table><tr><th>Mode</th><th>Value</th></tr><tr><td>Electronic Codebook (ECB) with no padding</td><td>PSA_ALG_ECB_NO_PADDING</td></tr><tr><td>Cipher Block Chaining (CBC) with no padding</td><td>PSA_ALG_CBC_NO_PADDING</td></tr><tr><td>Counter (CTR)</td><td>PSA_ALG_CTR</td></tr><tr><td>Cipher Feedback (CFB)</td><td>PSA_ALG_CFB</td></tr><tr><td>Output Feedback (OFB)</td><td>PSA_ALG_OFB</td></tr><tr><td>XEX with Ciphertext Stealing (XTS)</td><td>PSA_ALG_XTS</td></tr></table>	Mode	Value	Electronic Codebook (ECB) with no padding	PSA_ALG_ECB_NO_PADDING	Cipher Block Chaining (CBC) with no padding	PSA_ALG_CBC_NO_PADDING	Counter (CTR)	PSA_ALG_CTR	Cipher Feedback (CFB)	PSA_ALG_CFB	Output Feedback (OFB)	PSA_ALG_OFB	XEX with Ciphertext Stealing (XTS)	PSA_ALG_XTS				
		Mode	Value																	
		Electronic Codebook (ECB) with no padding	PSA_ALG_ECB_NO_PADDING																	
		Cipher Block Chaining (CBC) with no padding	PSA_ALG_CBC_NO_PADDING																	
		Counter (CTR)	PSA_ALG_CTR																	
		Cipher Feedback (CFB)	PSA_ALG_CFB																	
		Output Feedback (OFB)	PSA_ALG_OFB																	
XEX with Ciphertext Stealing (XTS)	PSA_ALG_XTS																			
PKCS11	Key's CKA_ALLOWED_MECHANISMS must be one of the mechanism of the following table. If multiple symmetric encryption mechanisms are allowed, the key can be used with any of symmetric encryption mechanisms listed below.																			
	<table><tr><th>Mode</th><th>Value</th></tr><tr><td>Electronic Codebook (ECB) with no padding</td><td>CKM_AES_ECB</td></tr><tr><td>Cipher Block Chaining (CBC) with no padding</td><td>CKM_AES_CBC</td></tr><tr><td>Counter (CTR)</td><td>CKM_AES_CTR</td></tr><tr><td>Cipher Feedback (CFB)</td><td>CKM_AES_CFB128</td></tr><tr><td>Output Feedback (OFB)</td><td>CKM_AES_OFB</td></tr></table>	Mode	Value	Electronic Codebook (ECB) with no padding	CKM_AES_ECB	Cipher Block Chaining (CBC) with no padding	CKM_AES_CBC	Counter (CTR)	CKM_AES_CTR	Cipher Feedback (CFB)	CKM_AES_CFB128	Output Feedback (OFB)	CKM_AES_OFB							
	Mode	Value																		
	Electronic Codebook (ECB) with no padding	CKM_AES_ECB																		
	Cipher Block Chaining (CBC) with no padding	CKM_AES_CBC																		
	Counter (CTR)	CKM_AES_CTR																		
Cipher Feedback (CFB)	CKM_AES_CFB128																			
Output Feedback (OFB)	CKM_AES_OFB																			

Table 86. Permitted Algorithms for AES Symmetric Encryption...continued

Key Type	API	Permitted Algorithm	
		Mode	Value
		Ciphertext Stealing (CTS)	CKM_AES_CTS
		XEX with Ciphertext Stealing (XTS)	CKM_AES_XTS

5.6.3 DES Symmetric Encryption

Table 87. DES Support Details

Key Security Size (bits)	Subsystem	Key	
		Opaque	Plaintext
56	ELE	N	N
	TEE	Y	Y
	SECO	N	N

5.6.3.1 Key attributes

5.6.3.1.1 Key Usage

For DES Symmetric Encryption operations, keys must be configured with appropriate key usage flags:

Table 88. DES Symmetric encryption Key Usage Flags

Operation	API	Required Key Usage
Encryption	SMW	SMW_ATTR_USAGE_ENCRYPT
	PSA	PSA_KEY_USAGE_ENCRYPT
	PKCS11	CKA_ENCRYPT
Decryption	SMW	SMW_ATTR_USAGE_DECRYPT
	PSA	PSA_KEY_USAGE_DECRYPT
	PKCS11	CKA_DECRYPT

5.6.3.1.2 Permitted Algorithm

The following table outlines the permitted DES Symmetric Encryption algorithms.

Table 89. Permitted Algorithms for DES Symmetric Encryption

Key Type	API	Permitted Algorithm	
DES	SMW	SMW_ATTR_ALGO_SYMMETRIC_ENCRYPTION(algo, mode) Where: • <code>algo</code> is SMW_ATTR_ALGO_DES. • <code>mode</code> is one of the value of the following table.	
		Mode	Value
		Electronic Codebook (ECB) with no padding	SMW_ATTR_MODE_ECB_NO_PAD

Table 89. Permitted Algorithms for DES Symmetric Encryption...continued

Key Type	API	Permitted Algorithm	
		Mode	Value
		Cipher Block Chaining (CBC) with no padding	SMW_ATTR_MODE_CBC_NO_PAD
		Any mode	SMW_ATTR_MODE_ANY
	PSA	Permitted algorithm is one of the value of the following table.	
		Mode	Value
		Electronic Codebook (ECB) with no padding	PSA_ALG_ECB_NO_PADDING
		Cipher Block Chaining (CBC) with no padding	PSA_ALG_CBC_NO_PADDING
	PKCS11	Key's CKA_ALLOWED_MECHANISMS must be one of the mechanism of the following table. If multiple symmetric encryption mechanisms are allowed, the key can be used with any of symmetric encryption mechanisms listed below.	
		Mode	Value
		Electronic Codebook (ECB) with no padding	CKM_DES_ECB
		Cipher Block Chaining (CBC) with no padding	CKM_DES_CBC

5.6.4 Triple-DES (DES3) Symmetric Encryption

Table 90. Triple-DES Support Details

Key Security Size (bits)	Subsystem	Key	
		Opaque	Plaintext
112	ELE	N	N
	TEE	Y	Y
	SECO	N	N
168	ELE	N	N
	TEE	Y	Y
	SECO	N	N

5.6.4.1 Key attributes

5.6.4.1.1 Key Usage

For Triple-DES Symmetric Encryption operations, keys must be configured with appropriate key usage flags:

Table 91. Triple-DES Symmetric encryption Key Usage Flags

Operation	API	Required Key Usage
Encryption	SMW	SMW_ATTR_USAGE_ENCRYPT
	PSA	PSA_KEY_USAGE_ENCRYPT
	PKCS11	CKA_ENCRYPT
Decryption	SMW	SMW_ATTR_USAGE_DECRYPT
	PSA	PSA_KEY_USAGE_DECRYPT
	PKCS11	CKA_DECRYPT

5.6.4.1.2 Permitted Algorithm

The following table outlines the permitted Triple-DES Symmetric Encryption algorithms.

Table 92. Permitted Algorithms for Triple-DES Symmetric Encryption

Key Type	API	Permitted Algorithm								
Triple-DES (DES3)	SMW	SMW_ATTR_ALGO_SYMMETRIC_ENCRYPTION(algo, mode) Where: • algo is SMW_ATTR_ALGO_DES3. • mode is one of the value of the following table.								
		<table><tr><th>Mode</th><th>Value</th></tr><tr><td>Electronic Codebook (ECB) with no padding</td><td>SMW_ATTR_MODE_ECB_NO_PAD</td></tr><tr><td>Cipher Block Chaining (CBC) with no padding</td><td>SMW_ATTR_MODE_CBC_NO_PAD</td></tr><tr><td>Any mode</td><td>SMW_ATTR_MODE_ANY</td></tr></table>	Mode	Value	Electronic Codebook (ECB) with no padding	SMW_ATTR_MODE_ECB_NO_PAD	Cipher Block Chaining (CBC) with no padding	SMW_ATTR_MODE_CBC_NO_PAD	Any mode	SMW_ATTR_MODE_ANY
		Mode	Value							
		Electronic Codebook (ECB) with no padding	SMW_ATTR_MODE_ECB_NO_PAD							
		Cipher Block Chaining (CBC) with no padding	SMW_ATTR_MODE_CBC_NO_PAD							
	Any mode	SMW_ATTR_MODE_ANY								
	PSA	Permitted algorithm is one of the value of the following table.								
		<table><tr><th>Mode</th><th>Value</th></tr><tr><td>Electronic Codebook (ECB) with no padding</td><td>PSA_ALG_ECB_NO_PADDING</td></tr><tr><td>Cipher Block Chaining (CBC) with no padding</td><td>PSA_ALG_CBC_NO_PADDING</td></tr></table>	Mode	Value	Electronic Codebook (ECB) with no padding	PSA_ALG_ECB_NO_PADDING	Cipher Block Chaining (CBC) with no padding	PSA_ALG_CBC_NO_PADDING		
		Mode	Value							
	Electronic Codebook (ECB) with no padding	PSA_ALG_ECB_NO_PADDING								
Cipher Block Chaining (CBC) with no padding	PSA_ALG_CBC_NO_PADDING									
PKCS11	Key's CKA_ALLOWED_MECHANISMS must be one of the mechanism of the following table. If multiple symmetric encryption mechanisms are allowed, the key can be used with any of symmetric encryption mechanisms listed below.									
	<table><tr><th>Mode</th><th>Value</th></tr><tr><td>Electronic Codebook (ECB) with no padding</td><td>CKM_DES3_ECB</td></tr><tr><td>Cipher Block Chaining (CBC) with no padding</td><td>CKM_DES3_CBC</td></tr></table>	Mode	Value	Electronic Codebook (ECB) with no padding	CKM_DES3_ECB	Cipher Block Chaining (CBC) with no padding	CKM_DES3_CBC			
	Mode	Value								
Electronic Codebook (ECB) with no padding	CKM_DES3_ECB									
Cipher Block Chaining (CBC) with no padding	CKM_DES3_CBC									

5.6.5 SM4 Symmetric Encryption

Table 93. SM4 Support Details

Key Security Size (bits)	Subsystem	Key	
		Opaque	Plaintext
128	ELE	N	N
	TEE	Y	Y
	SECO	N	N

5.6.5.1 Key attributes

5.6.5.1.1 Key Usage

For SM4 Symmetric Encryption operations, keys must be configured with appropriate key usage flags:

Table 94. SM4 Symmetric encryption Key Usage Flags

Operation	API	Required Key Usage
Encryption	SMW	SMW_ATTR_USAGE_ENCRYPT
	PSA	PSA_KEY_USAGE_ENCRYPT
	PKCS11	CKA_ENCRYPT
Decryption	SMW	SMW_ATTR_USAGE_DECRYPT
	PSA	PSA_KEY_USAGE_DECRYPT
	PKCS11	CKA_DECRYPT

5.6.5.1.2 Permitted Algorithm

The following table outlines the permitted SM4 Symmetric Encryption algorithms.

Table 95. Permitted Algorithms for SM4 Symmetric Encryption

Table 60: Permitted Algorithms for SM4 Symmetric Encryption												
Key Type	API	Permitted Algorithm										
SM4	SMW	SMW_ATTR_ALGO_SYMMETRIC_ENCRYPTION(algo, mode) Where: • <code>algo</code> is SMW_ATTR_ALGO_SM4. • <code>mode</code> is one of the value of the following table.										
		<table><tr><th>Mode</th><th>Value</th></tr><tr><td>Electronic Codebook (ECB) with no padding</td><td>SMW_ATTR_MODE_ECB_NO_PAD</td></tr><tr><td>Cipher Block Chaining (CBC) with no padding</td><td>SMW_ATTR_MODE_CBC_NO_PAD</td></tr><tr><td>Counter (CTR)</td><td>SMW_ATTR_MODE_CTR</td></tr><tr><td>Any mode</td><td>SMW_ATTR_MODE_ANY</td></tr></table>	Mode	Value	Electronic Codebook (ECB) with no padding	SMW_ATTR_MODE_ECB_NO_PAD	Cipher Block Chaining (CBC) with no padding	SMW_ATTR_MODE_CBC_NO_PAD	Counter (CTR)	SMW_ATTR_MODE_CTR	Any mode	SMW_ATTR_MODE_ANY
		Mode	Value									
		Electronic Codebook (ECB) with no padding	SMW_ATTR_MODE_ECB_NO_PAD									
		Cipher Block Chaining (CBC) with no padding	SMW_ATTR_MODE_CBC_NO_PAD									
		Counter (CTR)	SMW_ATTR_MODE_CTR									
	Any mode	SMW_ATTR_MODE_ANY										
PSA	Permitted algorithm is one of the value of the following table.											

Table 95. Permitted Algorithms for SM4 Symmetric Encryption...continued

Key Type	API	Permitted Algorithm	
		Mode	Value
		Electronic Codebook (ECB) with no padding	PSA_ALG_ECB_NO_PADDING
		Cipher Block Chaining (CBC) with no padding	PSA_ALG_CBC_NO_PADDING
		Counter (CTR)	PSA_ALG_CTR
	PKCS11	Key's CKA_ALLOWED_MECHANISMS must be one of the mechanism of the following table. If multiple symmetric encryption mechanisms are allowed, the key can be used with any of symmetric encryption mechanisms listed below.	
		Mode	Value
		Electronic Codebook (ECB) with no padding	CKM_SM4_ECB (Vendor define)
		Cipher Block Chaining (CBC) with no padding	CKM_SM4_CBC (Vendor define)
		Counter (CTR)	CKM_SM4_CTR (Vendor define)

5.7 Asymmetric Encryption

This section documents the asymmetric encryption operations supported across different security subsystems (ELE, TEE, SECO) with their respective algorithms and key types.

5.7.1 Supported Operations versus Subsystems

Table 96. Asymmetric Encryption Operations vs. subsystem

Key Type	Algorithm	Processing	Subsystem			Notes
			ELE	TEE	SECO	
RSA	PKCS#1 v1.5	Single-Part	Y*	Y	N	ELE not supported on: • i.MX8ULP
	OAEP	Single-Part	Y	Y	N	
	No Padding	Single-Part	N	Y	N	

Table 97. Asymmetric Encryption APIs

Operations	Processing	API	Functions
Encryption	Single-Part	SMW	smw_asymmetric_encrypt()
		PSA	psa_asymmetric_encrypt()
		PKCS11	C_EncryptInit() C_Encrypt()
			C_MessageEncryptInit() C_EncryptMessage()
Decryption	Single-Part	SMW	smw_asymmetric_decrypt()

Table 97. Asymmetric Encryption APIs...continued

Operations	Processing	API	Functions
		PSA	psa_asymmetric_decrypt()
		PKCS11	C_DecryptInit()
			C_Decrypt()
			C_MessageDecryptInit()
			C_DecryptMessage()

5.7.2 RSA PKCS#1 v1.5 Asymmetric Encryption

Table 98. RSA PKCS#1 v1.5 Support Details

Key Size	Subsystem	Key	
		Opaque	Plaintext
2048 / 3072 / 4096	ELE	Y	Y
256 to 4096 multiple of 2 bits	TEE	Y	Y

5.7.2.1 Key attributes

5.7.2.1.1 Key Usage

For RSA PKCS#1 v1.5 Asymmetric Encryption operations, keys must be configured with appropriate key usage flags:

Table 99. RSA PKCS#1 v1.5 Asymmetric encryption Key Usage Flags

Operation	API	Required Key Usage
Encryption	SMW	SMW_ATTR_USAGE_ENCRYPT
	PSA	PSA_KEY_USAGE_ENCRYPT
	PKCS11	CKA_ENCRYPT
Decryption	SMW	SMW_ATTR_USAGE_DECRYPT
	PSA	PSA_KEY_USAGE_DECRYPT
	PKCS11	CKA_DECRYPT

5.7.2.1.2 Permitted Algorithm

The following table outlines the permitted RSA PKCS#1 v1.5 Asymmetric Encryption algorithms.

Table 100. Permitted Algorithms for RSA PKCS#1 v1.5 Asymmetric Encryption

Key Type	API	Permitted Algorithm
RSA	SMW	SMW_ATTR_ALGO_ASYMMETRIC_ENCRYPTION(mode, hash) Where: - mode is SMW_ATTR_ALGO_RSA_PKCS1V15. - hash is the SMW_ATTR_HASH_NONE.
	PSA	Permitted algorithm is PSA_ALG_RSA_PKCS1V15_CRYPT.
	PKCS11	Key's CKA_ALLOWED_MECHANISMS must be CKM_RSA_PKCS.

5.7.3 RSA OAEP Asymmetric Encryption

Table 101. RSA OAEP Support Details

Key Size	Hash	Subsystem	Key	
			Opaque	Plaintext
2048 / 3072 / 4096	SHA1 SHA224 SHA256 SHA384 SHA512	ELE	Y	Y
256 to 4096 multiple of 2 bits	SHA1 SHA224 SHA256 SHA384 SHA512	TEE	Y	Y

5.7.3.1 Key attributes

5.7.3.1.1 Key Usage

For RSA OAEP Asymmetric Encryption operations, keys must be configured with appropriate key usage flags:

Table 102. RSA OAEP Asymmetric encryption Key Usage Flags

Operation	API	Required Key Usage
Encryption	SMW	SMW_ATTR_USAGE_ENCRYPT
	PSA	PSA_KEY_USAGE_ENCRYPT
	PKCS11	CKA_ENCRYPT
Decryption	SMW	SMW_ATTR_USAGE_DECRYPT
	PSA	PSA_KEY_USAGE_DECRYPT
	PKCS11	CKA_DECRYPT

5.7.3.1.2 Permitted Algorithm

The following table outlines the permitted RSA OAEP Asymmetric Encryption algorithms.

Table 103. Permitted Algorithms for RSA OAEP Asymmetric Encryption

Key Type	API	Permitted Algorithm
RSA	SMW	SMW_ATTR_ALGO_ASYMMETRIC_ENCRYPTION(mode, hash) Where: - mode is SMW_ATTR_ALGO_RSA_OAEP. - hash is one of SMW Attribute Hash algorithm define in the Algorithm Hash value table. Supported hash are listed in the RSA OAEP Support Details table. Defining a hash algorithm as SMW_ATTR_HASH_ANY, allow to use the key for any hash algorithm.
	PSA	PSA_ALG_RSA_OAEP(hash) Where hash is one of PSA Hash algorithm (PSA_ALG_XXX) such as PSA_ALG_IS_HASH is true.

Table 103. Permitted Algorithms for RSA OAEP Asymmetric Encryption...continued

Key Type	API	Permitted Algorithm
		Supported hash are listed in the RSA OAEP Support Details table. Defining a <code>hash</code> algorithm as <code>PSA_ALG_ANY_HASH</code> , allow to use the key for any hash algorithm.
	PKCS11	Key's <code>CKA_ALLOWED_MECHANISMS</code> must be <code>CKM_RSA_PKCS_OAEP</code> .

5.7.4 RSA No Padding Asymmetric Encryption

Table 104. RSA No Padding Support Details

Key Size	Subsystem	Key	
		Opaque	Plaintext
256 to 4096 multiple of 2 bits	TEE	Y	Y

5.7.4.1 Key attributes

5.7.4.1.1 Key Usage

For RSA No Padding Asymmetric Encryption operations, keys must be configured with appropriate key usage flags:

Table 105. RSA No Padding Asymmetric encryption Key Usage Flags

Operation	API	Required Key Usage
Encryption	SMW	SMW_ATTR_USAGE_ENCRYPT
	PSA	PSA_KEY_USAGE_ENCRYPT
	PKCS11	CKA_ENCRYPT
Decryption	SMW	SMW_ATTR_USAGE_DECRYPT
	PSA	PSA_KEY_USAGE_DECRYPT
	PKCS11	CKA_DECRYPT

5.7.4.1.2 Permitted Algorithm

The following table outlines the permitted RSA No Padding Asymmetric Encryption algorithms.

Table 106. Permitted Algorithms for RSA No Padding Asymmetric Encryption

Key Type	API	Permitted Algorithm
RSA	SMW	SMW_ATTR_ALGO_ASYMMETRIC_ENCRYPTION(mode, hash) Where: <code>mode</code> is <code>SMW_ATTR_MODE_NO_PAD</code>. <code>hash</code> is the <code>SMW_ATTR_HASH_NONE</code>.
	PSA	No defined.
	PKCS11	Key's <code>CKA_ALLOWED_MECHANISMS</code> must be <code>CKM_RSA_X_509</code> .

5.8 Asymmetric Signature

This section documents the asymmetric signature operations supported across different security subsystems (ELE, TEE, SECO) with their respective algorithms and key types.

5.8.1 Supported Operations versus Subsystems

Table 107. Asymmetric Signature Operations vs. subsystem

Signature Scheme	Processing	Subsystem		
		ELE	TEE	SECO
ECDSA	Single-Part	Y	Y	Y
	Multi-Part	Y	Y	N
EdDSA	Single-Part	Y	Y	N
	Multi-Part	Y	Y	N
RSA PKCS#1 v 1.5	Single-Part	Y	Y	N
	Multi-Part	Y	Y	N
RSA PSS	Single-Part	Y	Y	N
	Multi-Part	Y	Y	N
SM2	Single-Part	Y	Y	N
	Multi-Part	Y	Y	N

Table 108. Asymmetric Signature APIs

Operations	Processing	API	Functions
Signature Generation	Single-Part	SMW	smw_sign()
		PSA	psa_sign_hash() psa_sign_message()
		PKCS11	C_SignInit() C_Sign() C_MessageSignInit() C_SignMessage()
	Multi-Part	SMW	smw_sign_init() smw_sign_update() smw_sign_final()
		PKCS11	C_SignInit() C_SignUpdate() C_SignFinal() C_MessageSignInit() C_SignMessageBegin() C_SignMessageNext() C_SignMessageFinal()
Signature Verification	Single-Part	SMW	smw_verify()
		PSA	psa_verify_hash()

Table 108. Asymmetric Signature APIs...continued

Operations	Processing	API	Functions
		PKCS11	psa_verify_message()
			C_VerifyInit()
			C_Verify()
			C_MessageVerifyInit()
	Multi-Part	SMW	C_VerifyMessage()
			smw_verify_init()
			smw_verify_update()
			smw_verify_final()
		PKCS11	C_VerifyInit()
			C_VerifyUpdate()
			C_VerifyFinal()
			C_MessageVerifyInit()
			C_VerifyMessageBegin()
			C_VerifyMessageNext()
			C_VerifyMessageFinal()

5.8.2 ECDSA Signatures

Table 109. ECDSA Support Details

Curve	Size (bits)	Subsystem	Message		Key	
			Pre-hashed	Full	Opaque	Plaintext
Secp R1	192	ELE	N	N	N	N
		TEE	Y	Y	Y	Y
		SECO	N	N	N	N
	224	ELE	Y	Y	Y	Y
		TEE	Y	Y	Y	Y
		SECO	N	N	N	N
	256	ELE	Y	Y	Y	Y
		TEE	Y	Y	Y	Y
		SECO	Y	Y	Y	Y (verify only)
	384	ELE	Y	Y	Y	Y
		TEE	Y	Y	Y	Y
		SECO	Y	Y	Y	Y (verify only)
Brainpool R1	224	ELE	Y	Y	Y	Y
		TEE	N	N	N	N
		SECO	N	N	N	N
	256	ELE	Y	Y	Y	Y
		TEE	Y	Y	Y	Y
		SECO	N	N	N	N
		SECO	N	N	N	N

Table 109. ECDSA Support Details...continued

Curve	Size (bits)	Subsystem	Message		Key	
			Pre-hashed	Full	Opaque	Plaintext
		TEE	N	N	N	N
		SECO	Y	Y	Y	Y (verify only)
	384	ELE	Y	Y	Y	Y
		TEE	N	N	N	N
		SECO	Y	Y	Y	Y (verify only)

5.8.2.1 Key attributes

5.8.2.1.1 Key Usage

For signature operations, keys must be configured with appropriate key usage flags:

Table 110. ECDSA Key Usage Flags

Operation	API	Required Key Usage
Sign full message	SMW	SMW_ATTR_USAGE_SIGN_MESSAGE
	PSA	PSA_KEY_USAGE_SIGN_MESSAGE
	PKCS11	CKA_SIGN
Sign message hashed	SMW	SMW_ATTR_USAGE_SIGN_HASH
	PSA	PSA_KEY_USAGE_SIGN_HASH
	PKCS11	CKA_SIGN
Verify full message	SMW	SMW_ATTR_USAGE_VERIFY_MESSAGE
	PSA	PSA_KEY_USAGE_VERIFY_MESSAGE
	PKCS11	CKA_VERIFY
Verify message hashed	SMW	SMW_ATTR_USAGE_VERIFY_HASH
	PSA	PSA_KEY_USAGE_VERIFY_HASH
	PKCS11	CKA_VERIFY

5.8.2.1.2 Permitted Algorithm

The following table outlines the ECDSA permitted signature algorithms.

Table 111. Permitted Algorithms for ECDSA Asymmetric Signature

Signature	API	Permitted Algorithm
ECDSA	SMW	SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_ECDSA(curve, hash) Where: • curve is SMW_ATTR_CURVE_ANY. • hash is one of SMW Attribute Hash algorithm define in the Algorithm Hash value table. Defining a hash algorithm as SMW_ATTR_HASH_ANY, allow to use the key for any hash algorithm.
	PSA	PSA_ALG_ECDSA(hash)

Table 111. Permitted Algorithms for ECDSA Asymmetric Signature...continued

Signature	API	Permitted Algorithm
		Where <code>hash</code> is one of PSA Hash algorithm (<code>PSA_ALG_XXX</code>) such as PSA_ALG_IS_HASH is true. Defining a <code>hash</code> algorithm as <code>PSA_ALG_ANY_HASH</code> , allow to use the key for any hash algorithm.
	PKCS11	Key's <code>CKA_ALLOWED_MECHANISMS</code> must be one or more <code>CKM_ECDSA_SHAxxx</code> value. If multiple <code>CKM_ECDSA_SHAxx</code> values are set, the key permitted algorithm is ECDSA Signature any hash type.

5.8.3 EdDSA Signatures

Table 112. EdDSA Support Details

Curve	Subsystem	Message			Key	
		Pre-hashed	Context	Full	Opaque	Plaintext
Ed448	ELE	Y	N	Y	Y	Y
	TEE	N	N	N	N	N
Ed25519	ELE	Y	N	Y	Y	Y
	TEE	Y	Y	Y	Y	Y

Note:

- *i.MX8ULP doesn't support the EdDSA signature.*
- *i.MX94x (i.MX943, i.MX942, i.MX941), i.MX95, i.MX952, i.MX937 Pre-Hashed signature is not supported.*

5.8.3.1 Key attributes

5.8.3.1.1 Key Usage

For signature operations, keys must be configured with appropriate key usage flags:

Table 113. EdDSA Key Usage Flags

Operation	API	Required Key Usage
Sign full message	SMW	SMW_ATTR_USAGE_SIGN_MESSAGE
	PSA	PSA_KEY_USAGE_SIGN_MESSAGE
	PKCS11	CKA_SIGN
Sign message hashed	SMW	SMW_ATTR_USAGE_SIGN_HASH
	PSA	PSA_KEY_USAGE_SIGN_HASH
	PKCS11	CKA_SIGN
Verify full message	SMW	SMW_ATTR_USAGE_VERIFY_MESSAGE
	PSA	PSA_KEY_USAGE_VERIFY_MESSAGE
	PKCS11	CKA_VERIFY
Verify message hashed	SMW	SMW_ATTR_USAGE_VERIFY_HASH
	PSA	PSA_KEY_USAGE_VERIFY_HASH

Table 113. EdDSA Key Usage Flags...continued

Operation	API	Required Key Usage
	PKCS11	CKA_VERIFY

5.8.3.1.2 Permitted Algorithm

The following table outlines the EdDSA permitted signature algorithms.

Table 114. Permitted Algorithms for EdDSA Asymmetric Signature

Signature	API	Permitted Algorithm
EdDSA	SMW	SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_EDDSA(curve, hash, param) Where: - For ED25519 Pre-hashed signature scheme: - curve is SMW_ATTR_CURVE_ED25519 - hash is SMW_ATTR_HASH_NONE - param is SMW_ATTR_SIGN_PARAM_EDDSA_PREHASHED - For ED25519 with context signature scheme: - curve is SMW_ATTR_CURVE_ED25519 - hash is SMW_ATTR_HASH_NONE - param is SMW_ATTR_SIGN_PARAM_EDDSA_CONTEXT - For ED448 Pre-hashed signature scheme: - curve is SMW_ATTR_CURVE_ED448 - hash is SMW_ATTR_HASH_NONE - param is SMW_ATTR_SIGN_PARAM_EDDSA_PREHASHED - For Pure EDDSA with or without context signature scheme: - curve is SMW_ATTR_CURVE_ANY - hash is SMW_ATTR_HASH_NONE - param is SMW_ATTR_SIGN_PARAM_EDDSA_NONE - For any signature scheme: - curve is SMW_ATTR_CURVE_ANY - hash is SMW_ATTR_HASH_ANY - param is SMW_ATTR_SIGN_PARAM_EDDSA_NONE
	PSA	- For ED25519 Pre-hashed signature scheme use PSA_ALG_ED25519PH - For ED448 Pre-hashed signature scheme use PSA_ALG_ED448PH - For Pure EDDSA signature scheme use PSA_ALG_PURE_EDDSA
	PKCS11	Key's CKA_ALLOWED_MECHANISMS must be CKM_EDDSA value. It will allow to support any type of Edwards signature scheme.

5.8.4 RSA PKCS#1 v1.5 Signatures

Table 115. RSA PKCS#1 v1.5 Support Details

Key Size	Hash	Subsystem	Message	Key	
			Full	Opaque	Plaintext
2048 / 3072 / 4096	SHA-224 SHA-256 SHA-384 SHA-512	ELE	Y	Y	Y
256 to 4096 multiple of 2 bits	MD5 SHA-1	TEE	Y	Y	Y

Table 115. RSA PKCS#1 v1.5 Support Details...continued

Key Size	Hash	Subsystem	Message	Key	
			Full	Opaque	Plaintext
	SHA-224 SHA-256 SHA-384 SHA-512 SHA3-224 SHA3-256 SHA3-384 SHA3-512				

5.8.4.1 Key attributes

5.8.4.1.1 Key Usage

For signature operations, keys must be configured with appropriate key usage flags:

Table 116. RSA PKCS#1 v1.5 Key Usage Flags

Operation	API	Required Key Usage
Sign full message	SMW	SMW_ATTR_USAGE_SIGN_MESSAGE
	PSA	PSA_KEY_USAGE_SIGN_MESSAGE
	PKCS11	CKA_SIGN
Verify full message	SMW	SMW_ATTR_USAGE_VERIFY_MESSAGE
	PSA	PSA_KEY_USAGE_VERIFY_MESSAGE
	PKCS11	CKA_VERIFY

5.8.4.1.2 Permitted Algorithm

The following table outlines the RSA PKCS#1 v1.5 permitted signature algorithm.

Table 117. Permitted Algorithms for RSA PKCS1 v1.5 Asymmetric Signature

Signature	API	Permitted Algorithm
RSA PKCS#1 v1.5	SMW	SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_RSA(mode, hash, salt) Where: - mode is SMW_ATTR_MODE_PKCS1_1_5 - hash is one of SMW Attribute Hash algorithm define in the Algorithm Hash value table. - salt is 0 Defining a hash algorithm as SMW_ATTR_HASH_ANY, allow to use the key for any hash algorithm.
	PSA	PSA_ALG_RSA_PKCS1V15_SIGN(hash) Where hash is one of PSA Hash algorithm (PSA_ALG_XXX) such as PSA_ALG_IS_HASH is true. Defining a hash algorithm as PSA_ALG_ANY_HASH, allow to use the key for any hash algorithm.
	PKCS11	Key's CKA_ALLOWED_MECHANISMS must be CKM_RSA_PKCS or CKM_SHAxxx_RSA_PKCS value.

Table 117. Permitted Algorithms for RSA PKCS1 v1.5 Asymmetric Signature...continued

Signature	API	Permitted Algorithm
		If CKM_RSA_PKCS is set or if multiple CKM_SHAxxx_RSA_PKCS are set, the key permitted algorithm is RSA PKCS1V15 Signature any hash type.

5.8.5 RSA PSS Signatures

Table 118. RSA PSS Support Details

Key Size	Hash	Subsystem	Message	Salt (bytes)	Key	
			Full		Opaque	Plaintext
2048 / 3072 / 4096	SHA-224 SHA-256 SHA-384 SHA-512	ELE	Y	$0 \leq \text{len} \leq \text{HashLen}$	Y	Y
256 to 4096 multiple of 2 bits	SHA-1 SHA-224 SHA-256 SHA-384 SHA-512 SHA3-224 SHA3-256 SHA3-384 SHA3-512	TEE	Y	$0 \leq \text{len} \leq \text{HashLen}$	Y	Y

5.8.5.1 Key attributes

5.8.5.1.1 Key Usage

For signature operations, keys must be configured with appropriate key usage flags:

Table 119. RSA PSS Key Usage Flags

Operation	API	Required Key Usage
Sign full message	SMW	SMW_ATTR_USAGE_SIGN_MESSAGE
	PSA	PSA_KEY_USAGE_SIGN_MESSAGE
	PKCS11	CKA_SIGN
Verify full message	SMW	SMW_ATTR_USAGE_VERIFY_MESSAGE
	PSA	PSA_KEY_USAGE_VERIFY_MESSAGE
	PKCS11	CKA_VERIFY

5.8.5.1.2 Permitted Algorithm

The following table outlines the RSA PSS permitted signature algorithms.

Table 120. Permitted Algorithms for RSA PSS Asymmetric Signature

Signature	API	Permitted Algorithm
RSA PSS	SMW	SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_RSA(mode, hash, salt) Where:

Table 120. Permitted Algorithms for RSA PSS Asymmetric Signature...continued

Signature	API	Permitted Algorithm
		- mode is SMW_ATTR_MODE_PSS - hash is one of SMW Attribute Hash algorithm define in the Algorithm Hash value table. - salt is 0 Defining a hash algorithm as SMW_ATTR_HASH_ANY, allow to use the key for any hash algorithm.
	PSA	PSA_ALG_RSA_PSS(hash) Where hash is one of PSA Hash algorithm (PSA_ALG_XXX) such as PSA_ALG_IS_HASH is true. Defining a hash algorithm as PSA_ALG_ANY_HASH, allow to use the key for any hash algorithm.
	PKCS11	Key's CKA_ALLOWED_MECHANISMS must be CKM_RSA_PKCS_PSS or CKM_SHAxxx_RSA_PKCS_PSS value. If CKM_RSA_PKCS_PSS is set or if multiple CKM_SHAxxx_RSA_PKCS_PSS are set, the key permitted algorithm is RSA PSS Signature any hash type.

5.8.6 SM2 Signatures

Table 121. SM2 Support Details

Subsystem	Message		Key	
	Pre-hashed	Full	Opaque	Plaintext
ELE	Y	Y	Y	Y
TEE	Y	Y	Y	Y

Note:

- i.MX8ULP, i.MX91, i.MX93 and i.MX95 don't support the SM2 signature with ELE.*

5.8.6.1 Key attributes

5.8.6.1.1 Key Usage

For signature operations, keys must be configured with appropriate key usage flags:

Table 122. SM2 Key Usage Flags

Operation	API	Required Key Usage
Sign full message	SMW	SMW_ATTR_USAGE_SIGN_MESSAGE
Sign message hashed	SMW	SMW_ATTR_USAGE_SIGN_HASH
Verify full message	SMW	SMW_ATTR_USAGE_VERIFY_MESSAGE
Verify message hashed	SMW	SMW_ATTR_USAGE_VERIFY_HASH

Note:

- SM2 key type is not supported with PSA and PKCS11 APIs.*
- If an SM2 key is present, it cannot be used with PSA and PKCS11 APIs and its attributes are not retrievable as well with PSA and PKCS11 APIs.*

5.8.6.1.2 Permitted Algorithm

The following table outlines the SM2 permitted signature algorithms.

Table 123. Permitted Algorithms for SM2 Asymmetric Signature

Signature	API	Permitted Algorithm
SM2	SMW	SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_SM2(hash) Where: hash is one of SMW Attribute Hash algorithm define in the Table 144. Only SM3 is supported now.
	PSA	Not defined
	PKCS11	Not defined

5.9 Authentication Encryption with Associated Data (AEAD)

This section documents the AEAD (Authentication Encryption with Associated Data) operations supported across different security subsystems (ELE, TEE, SECO) with their respective algorithms and modes.

5.9.1 Supported Operations versus Subsystems

Table 124. AEAD Operations vs. subsystem

Key Type	Mode	Processing	Subsystem		
			ELE	TEE	SECO
AES	Counter with CBC-MAC (CCM)	Single-Part	Y	Y	Y
		Multi-Part	Y (2)	Y	N
	Galois/Counter Mode (GCM)	Single-Part	Y (1)(3)	Y	Y
		Multi-Part	Y (2)(4)	Y	N
	ChaCha20-Poly1305	Single-Part	Y (1)	N	N
		Multi-Part	N	N	N

1. ELE single-part encryption and decryption operations using GCM and ChaCha20-Poly1305 are not supported on the i.MX8ULP platform.
2. ELE Multi-part encryption and decryption operations not supported on the i.MX8ULP, i.MX93 and i.MX91 platforms.
3. ELA hardware acceleration for single-part AEAD GCM mode is supported on all platforms except i.MX8ULP, i.MX91, i.MX93 and i.MX95. ELA for GCM requires plaintext key buffer, a 12-byte IV and a non-NULL AAD whose length is a multiple of 16 bytes. If these conditions are not met, the operation is executed by ELE.
4. On i.MX95 B0, ELE multi-part AES-GCM encryption with a non-NULL AAD and an opaque key produces an incorrect authentication tag. This is a known ELE firmware limitation. The operation succeeds and produces the correct tag when AAD is NULL or when a plaintext key is used.

Table 125. AEAD APIs

Operations	Processing	API	SMW
Encryption	Single-Part	SMW	smw_aead()
		PSA	psa_aead_encrypt()
		PKCS11	C_EncryptInit() C_Encrypt()

Table 125. AEAD APIs...continued

Operations	Processing	API	SMW
	Multi-Part		C_MessageEncryptInit() C_EncryptMessage()
			smw_allocate_context() smw_aead_init() smw_aead_update_aad() smw_aead_update() smw_aead_final() smw_cancel_operation()
		PSA	psa_aead_encrypt_setup() psa_aead_generate_nonce() psa_aead_set_nonce() psa_aead_set_lengths() psa_aead_update_ad() psa_aead_update() psa_aead_finish() psa_aead_abort()
			C_EncryptInit() C_EncryptUpdate() C_EncryptFinal()
		PKCS11	C_MessageEncryptInit() C_EncryptMessageBegin() C_EncryptMessageNext() C_MessageEncryptFinal()
Decryption	Single-Part	SMW	smw_aead()
		PSA	psa_aead_decrypt()
		PKCS11	C_DecryptInit() C_Decrypt()
			C_MessageDecryptInit() C_DecryptMessage()
	Multi-Part	SMW	smw_allocate_context() smw_aead_init() smw_aead_update_aad() smw_aead_update() smw_aead_final() smw_cancel_operation()
		PSA	psa_aead_decrypt_setup() psa_aead_set_nonce() psa_aead_set_lengths() psa_aead_update_ad() psa_aead_update() psa_aead_verify() psa_aead_abort()
		PKCS11	C_DecryptInit() C_DecryptUpdate()

Table 125. AEAD APIs...continued

Operations	Processing	API	SMW
			C_DecryptFinal()
			C_MessageDecryptInit()
			C_DecryptMessageBegin()
			C_DecryptMessageNext()
			C_MessageDecryptFinal()

5.9.2 AES AEAD

Table 126. AES AEAD Support Details

Key Security Size (bits)	Subsystem	Key	
		Opaque	Plaintext
128	ELE	Y	Y
	TEE	Y	Y
	SECO	Y	N
192	ELE	Y	Y
	TEE	Y	Y
	SECO	Y	N
256	ELE	Y	Y
	TEE	Y	Y
	SECO	Y	N

Table 127. AES AEAD IV length Support

Mode	Subsystem	Nonce or IV length (bytes)	Tag length (bytes)
Counter with CBC-MAC (CCM)	ELE SECO	12	16
	TEE	From 1 to 13	4 / 6 / 8 / 10 / 12 / 14 / 16
Galois/Counter Mode (GCM)	ELE	Encryption: • 0 (subsystem generates full IV) • 4 (subsystem generates 8 bytes of IV) • 12 (user supplied full IV)	16
		Decryption: • 12 (user supplied full IV)	
	SECO	Encryption: • 0 (subsystem generates full IV) • 4 (subsystem generates 8 bytes of IV)	16
		Decryption: • 12 (user supplied full IV)	
	TEE	From 1 to 16 (Recommended is 12)	12 / 13 / 14 / 15 / 16

Table 127. AES AEAD IV length Support...continued

Mode	Subsystem	Nonce or IV length (bytes)	Tag length (bytes)
ChaCha20-Poly1305	ELE	12	16

5.9.2.1 Key attributes

5.9.2.1.1 Key Usage

For AES AEAD operations, keys must be configured with appropriate key usage flags:

Table 128. AES AEAD Key Usage Flags

Operation	API	Required Key Usage
Encryption	SMW	SMW_ATTR_USAGE_ENCRYPT
	PSA	PSA_KEY_USAGE_ENCRYPT
	PKCS11	CKA_ENCRYPT
Decryption	SMW	SMW_ATTR_USAGE_DECRYPT
	PSA	PSA_KEY_USAGE_DECRYPT
	PKCS11	CKA_DECRYPT

5.9.2.1.2 Permitted Algorithm

The following table outlines the permitted AES AEAD algorithms.

Table 129. Permitted Algorithms for AES AEAD

Key Type	API	Permitted Algorithm										
AES	SMW	SMW_ATTR_ALGO_AEAD(algo, mode, tag) Where: • <code>algo</code> is SMW_ATTR_ALGO_AES. • <code>mode</code> is one of the values of the following table. • <code>tag</code> is 0. Not used for the key permitted algorithm.										
		<table><tr><th>Mode</th><th>Value</th></tr><tr><td>Counter with CBC-MAC (CCM)</td><td>SMW_ATTR_MODE_CCM</td></tr><tr><td>Galois/Counter Mode (GCM)</td><td>SMW_ATTR_MODE_GCM</td></tr><tr><td>ChaCha20-Poly1305</td><td>SMW_ATTR_MODE_POLY1305</td></tr><tr><td>Any mode</td><td>SMW_ATTR_MODE_ANY</td></tr></table>	Mode	Value	Counter with CBC-MAC (CCM)	SMW_ATTR_MODE_CCM	Galois/Counter Mode (GCM)	SMW_ATTR_MODE_GCM	ChaCha20-Poly1305	SMW_ATTR_MODE_POLY1305	Any mode	SMW_ATTR_MODE_ANY
		Mode	Value									
		Counter with CBC-MAC (CCM)	SMW_ATTR_MODE_CCM									
		Galois/Counter Mode (GCM)	SMW_ATTR_MODE_GCM									
		ChaCha20-Poly1305	SMW_ATTR_MODE_POLY1305									
	Any mode	SMW_ATTR_MODE_ANY										
PSA	Permitted algorithm is one of the value of the following table.											
	<table><tr><th>Mode</th><th>Value</th></tr><tr><td>Counter with CBC-MAC (CCM)</td><td>PSA_ALG_CCM</td></tr><tr><td>Galois/Counter Mode (GCM)</td><td>PSA_ALG_GCM</td></tr><tr><td>ChaCha20-Poly1305</td><td>PSA_ALG_CHACHA20_POLY1305</td></tr></table>	Mode	Value	Counter with CBC-MAC (CCM)	PSA_ALG_CCM	Galois/Counter Mode (GCM)	PSA_ALG_GCM	ChaCha20-Poly1305	PSA_ALG_CHACHA20_POLY1305			
Mode	Value											
Counter with CBC-MAC (CCM)	PSA_ALG_CCM											
Galois/Counter Mode (GCM)	PSA_ALG_GCM											
ChaCha20-Poly1305	PSA_ALG_CHACHA20_POLY1305											

Table 129. Permitted Algorithms for AES AEAD...continued

Key Type	API	Permitted Algorithm								
	PKCS11	Key's CKA_ALLOWED_MECHANISMS must be one of the mechanism of the following table. If multiple AEAD mechanisms are allowed, the key can be used with any of AEAD mechanisms listed below.								
		<table><tr><th>Mode</th><th>Value</th></tr><tr><td>Counter with CBC-MAC (CCM)</td><td>CKM_AES_CCM</td></tr><tr><td>Galois/Counter Mode (GCM)</td><td>CKM_AES_GCM</td></tr><tr><td>ChaCha20-Poly1305</td><td>CKM_CHACHA20_POLY1305</td></tr></table>	Mode	Value	Counter with CBC-MAC (CCM)	CKM_AES_CCM	Galois/Counter Mode (GCM)	CKM_AES_GCM	ChaCha20-Poly1305	CKM_CHACHA20_POLY1305
		Mode	Value							
		Counter with CBC-MAC (CCM)	CKM_AES_CCM							
		Galois/Counter Mode (GCM)	CKM_AES_GCM							
ChaCha20-Poly1305	CKM_CHACHA20_POLY1305									

5.10 Device Management

Device management operations allow interaction with device-specific features such as attestation, UUID, lifecycle management and device information.

5.10.1 Supported Operations versus Subsystems

Table 130. Device Management Operations vs. subsystem

Operations	Subsystem		
	ELE	TEE	SECO
Device Attestation	Y	N	N
Get Device UUID	Y	N	N
Device Lifecycle	Y	N	N
Device information	Y	N	N

5.10.2 Device Attestation

Device attestation provides cryptographic proof of device identity and integrity.

Table 131. Device Attestation APIs Comparison

API	Function
SMW	smw_device_attestation()
PSA	psa_initial_attest_get_token()
PKCS11	Not supported

The device attestation requires a challenge value to guarantee the certificate request. The challenge value maximum length depends of the device as listed in the following table.

Table 132. ELE Attestation Challenge

Device	Challenge in bytes	Version	Certificate	Comment
i.MX8ULP	4	0x1	Device Attestation Certificate Contents - version 1	

Table 132. ELE Attestation Challenge...continued

Device	Challenge in bytes	Version	Certificate	Comment
i.MX91 i.MX93	16	0x2	Device Attestation Certificate Contents - version 2	OEM SRKH up to 512 bits
i.MX95 i.MX943 i.MX942 i.MX941 i.MX952 i.MX937	16	0x3	Device Attestation Certificate Contents - version 3	Hybrid OEM SRKH up to 512 bits and PQC 512 bits

The device attestation returned certificate is signed by the device manufacturer and contains the device public key and identity information.

Table 133. Device Attestation Certificate Contents - version 1

Word (32 bits)	Certificate Field	Size (bytes)	Description
0	Command	1	0xDB: Device Attestation
	Version	1	Version of the certificate (0x1)
	Length	2	Length of certificate data in bytes
1	SOC ID	2	System-on-Chip identifier
	SOC Revision	2	Revision of the System-on-Chip
2	Lifecycle State	2	Current device lifecycle state: • 0x0010: OEM Open • 0x0040: OEM Closed • 0x0080: OEM Field Return • 0x0100: NXP Field Return • 0x0200: OEM Closed and Locked
	SSSM State	1	Security Subsystem State machine (internal device value)
	Reserved	1	Reserved
3	UID	16	Unique device identifier
7	ROM Patch (SHA256)	32	SHA256 of ELE ROM patch fuses
15	FW (SHA256)	32	SHA256 of FW installed
23	Nonce	4	Attestation challenge
24	Signature	96	Signature of previous data. ECC P-384 format

Table 134. Device Attestation Certificate Contents - version 2

Word (32 bits)	Certificate Field	Size (bytes)	Description
0	Command	1	0xDB: Device Attestation
	Version	1	Version of the certificate (0x2)

Table 134. Device Attestation Certificate Contents - version 2...continued

Word (32 bits)	Certificate Field	Size (bytes)	Description
	Length	2	Length of certificate data in bytes
1	SOC ID	2	System-on-Chip identifier
	SOC Revision	2	Revision of the System-on-Chip
2	Lifecycle State	2	Current device lifecycle state: • 0x0010: OEM Open • 0x0040: OEM Closed • 0x0080: OEM Field Return • 0x0100: NXP Field Return • 0x0200: OEM Closed and Locked
	SSSM State	1	Security Subsystem State machine (internal device value)
	Reserved	1	Reserved
3	UID	16	Unique device identifier
7	ROM Patch (SHA256)	32	SHA256 of ELE ROM patch fuses
15	FW (SHA256)	32	SHA256 of FW installed
23	OEM SRKH	64	OEM SRKH fuses
39	TRNG State	1	Current state of the TRNG: • 0x1: Programming mode • 0x2: Generating entropy on-going • 0x3: Valid and ready • 0x4: Generating entropy error
	CSAL State	1	Cryptographic library random context initialization state: • 0x0: Initialization not done • 0x1: Initialization on-going • 0x2: Initialization succeed • 0x3: Initialization Failed • 0x4: Initialization in "pause" mode
	IMEM State	1	ELE Internal RAM State after a Real Time Domain power down: • 0xCA: IMEM not lost • 0xFE: IMEM lost and must be restored
	Reserved	1	Reserved
40	Nonce	16	Attestation challenge
56	Signature	96	Signature of previous data. ECC P-384 format

Table 135. Device Attestation Certificate Contents - version 3

Word (32 bits)	Certificate Field	Size (bytes)	Description
0	Command	1	0xDB: Device Attestation
	Version	1	Version of the certificate (0x3)
	Length	2	Length of certificate data in bytes

Table 135. Device Attestation Certificate Contents - version 3...continued

Word (32 bits)	Certificate Field	Size (bytes)	Description
1	SOC ID	2	System-on-Chip identifier
	SOC Revision	2	Revision of the System-on-Chip
2	Lifecycle State	2	Current device lifecycle state: • 0x0010: OEM Open • 0x0040: OEM Closed • 0x0080: OEM Field Return • 0x0100: NXP Field Return • 0x0200: OEM Closed and Locked
	SSSM State	1	Security Subsystem State machine (internal device value)
	Reserved	1	Reserved
3	UID	16	Unique device identifier
7	ROM Patch (SHA256)	32	SHA256 of ELE ROM patch fuses
15	FW (SHA256)	32	SHA256 of FW installed
23	OEM SRKH	64	OEM SRKH fuses
39	TRNG State	1	Current state of the TRNG: • 0x1: Programming mode • 0x2: Generating entropy on-going • 0x3: Valid and ready • 0x4: Generating entropy error
	CSAL State	1	Cryptographic library random context initialization state: • 0x0: Initialization not done • 0x1: Initialization on-going • 0x2: Initialization succeed • 0x3: Initialization Failed • 0x4: Initialization in "pause" mode
	Reserved	2	Reserved
40	OEM PQC SRKH	64	OEM PQC SRKH fuses
56	Reserved	32	Reserved
64	Nonce	16	Attestation challenge
68	Signature	96	Signature of previous data. ECC P-384 format

Warning:

If EL2GO keys used during the provisioning process are missing their Key Check Values (KCV) on i.MX952, i.MX937, i.MX94x, and i.MX95 B1 devices, the device attestation service is disabled. [smw_device_attestation\(\)](#) returns `SMW_STATUS_OPERATION_DISABLED` on such devices. See [ELE FW 2.0.7 - EL2GO Key Service Limitation](#) for the full list of affected EL2GO keys and disabled operations.

5.10.3 Get Device UUID

Retrieve the unique identifier (UUID) of the device.

Note:

The device UUID is a unique identifier that remains constant throughout the device's lifetime. The value returned by the API is a big-endian value.

Table 136. Get Device UUID APIs Comparison

API	Function
SMW	smw_device_get_uuid()
PSA	Not supported
PKCS11	Not supported

5.10.4 Device Lifecycle

Query and manage the device lifecycle state.

Warning:

Changing the device lifecycle (set operation) is irreversible. Refer to the device documentation to get more details about the lifecycle.

Table 137. Device Lifecycle APIs Comparison

API	Function
SMW	smw_device_set_lifecycle()
	smw_device_get_lifecycle()
PSA	Not supported
PKCS11	Not supported

5.10.5 Device information

Retrieve device information such as SoC identifier and revision, if the OEM SRKH is fused, the device lifecycle.

Table 138. Get Device information APIs Comparison

API	Function
SMW	smw_device_get_info()
PSA	Not supported
PKCS11	Not supported

6 APIs reference

The SMW Library provides a unified interface to security services available on NXP platforms.

Two sets of APIs are exposed:

- The NXP's Security Middleware APIs called [SMW APIs](#).
- The ARM's Platform Security Architecture APIs called [PSA APIs](#).
 - The key manager and cryptographic APIs are compliant with the [ARM PSA Certified Cryptography API v1.3.2](#).
 - The attestation APIs are compliant with the [ARM PSA Certified Attestation API v1.0.4](#).
 - The storage APIs are compliant with the [ARM PSA Certified Secure Storage API v1.0.4](#).
- The PKCS#11 Oasis standard APIs called [PKCS11 APIs](#), compliant with the following versions:
 - [PKCS#11 v2.40](#).
 - [PKCS#11 v3.2](#).

Both APIs set are working independently without concurrence.

6.1 SMW APIs

6.1.1 Library information APIs

The Library information APIs allow users of the library to query information about the library such as version.

6.1.1.1 Library version

6.1.1.1.1 Function

6.1.1.1.1.1 smw_get_version

enum [smw_status_code](#) **smw_get_version**(unsigned int *major, unsigned int *minor)

Get the library version.

Parameters

- **major** (unsigned int*) - [out] Library major version.
- **minor** (unsigned int*) - [out] Library minor version.

Description

This function returns the library major and minor version numbers.

Return

- **SMW_STATUS_OK:**
Success.
- **SMW_STATUS_INVALID_PARAM:**
Either `major` or `minor` parameter is NULL.
- Other error codes from [enum smw_status_code](#)

6.1.2 Configuration APIs

The configuration APIs allow user of the library to:

- Get information about the state of the Secure Subsystems.
- Get the capabilities of the library operations.
- Load/Unload library configuration.

6.1.2.1 Subsystems Information

6.1.2.1.1 Functions

6.1.2.1.1.1 smw_config_subsystem_present

enum [smw_status_code](#) smw_config_subsystem_present([smw_subsystem_t](#) subsystem)

Check if the subsystem is present or not.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Name of the subsystem.

Description

This function checks if a subsystem is present in the configuration.

Return

- **SMW_STATUS_OK:**
subsystem is present.
- **SMW_STATUS_INVALID_PARAM:**
subsystem is SMW_SUBSYSTEM_NAME_NONE.
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
subsystem is not valid.
- Other error code from [enum smw_status_code](#)

6.1.2.1.1.2 smw_config_subsystem_loaded

enum [smw_status_code](#) smw_config_subsystem_loaded([smw_subsystem_t](#) subsystem)

Return if the subsystem is loaded or not.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Name of the subsystem.

Description

This functions returns if a subsystem is loaded and operational.

Return

- **SMW_STATUS_SUBSYSTEM_LOADED:**

subsystem is loaded.

- **SMW_STATUS_SUBSYSTEM_NOT_LOADED:**

subsystem is not loaded.

- **SMW_STATUS_INVALID_PARAM:**

subsystem is SMW_SUBSYSTEM_NAME_NONE.

- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**

subsystem is not valid.

- **SMW_STATUS_INVALID_LIBRARY_CONTEXT:**

Library context is not valid.

- Other error code from [enum smw_status_code](#)

6.1.2.2 Key capabilities

6.1.2.2.1 Functions

6.1.2.2.1.1 smw_config_check_generate_key

enum [smw_status_code](#) smw_config_check_generate_key([smw_subsystem_t](#) subsystem, struct [smw_key_info](#) *info)

Check key type generation support.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Name of the subsystem.
- **info** (struct [smw_key_info](#)*) - [in/out] Key information.

Description

Function checks if the key type provided in the `info` structure can be generated on the given subsystem or default subsystem if `subsystem` is not specified ([SMW_SUBSYSTEM_NAME_NONE](#)).

If `info`'s security size field is equal 0, returns the security key range size in bits supported by the subsystem for the key type. Else checks if the security size is supported.

Return

- **SMW_STATUS_OK:**

Key type is supported.

- **SMW_STATUS_INVALID_PARAM:**

- `info` is NULL.
- `info->key_type_name` is SMW_KEY_TYPE_NAME_NONE.

- **SMW_STATUS_UNKNOWN_KEY_TYPE_NAME:**

`info->key_type_name` is not valid.

- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**

Key type is not supported.

- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**

`subsystem` is not valid.

- Other error code from [enum smw_status_code](#)

6.1.2.2.1.2 smw_config_check_derive_key

enum [smw_status_code](#) **smw_config_check_derive_key**([smw_subsystem_t](#) subsystem, [smw_kdf_t](#) kdf)

Check key derivation support.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Name of the subsystem.
- **kdf** ([smw_kdf_t](#)) - [in] Key derivation name.

Description

Function checks if the `kdf` is supported on the given subsystem or default subsystem if `subsystem` is not specified ([SMW_SUBSYSTEM_NAME_NONE](#)).

Return

- **SMW_STATUS_OK:**
Key derivation is supported.
- **SMW_STATUS_INVALID_PARAM:**
`kdf` is `SMW_KDF_NAME_NONE`.
- **SMW_STATUS_UNKNOWN_KDF_NAME:**
`kdf` is not valid key derivation name.
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
`kdf` is not supported.
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
`subsystem` is not valid.
- Other error code from [enum smw_status_code](#)

6.1.2.2.2 Structures

6.1.2.2.2.1 struct smw_key_info

struct **smw_key_info**

Key information

Definition

```
struct smw_key_info {
    smw_key_type_t key_type_name;
    unsigned int security_size;
    unsigned int security_size_min;
    unsigned int security_size_max;
}
```

Members

key_type_name

[in] Key type name. See [struct smw_key_type_t](#).

security_size

[in] Key security size in bits.

security_size_min

[out] Key security size minimum in bits.

security_size_max

[out] Key security size maximum in bits.

6.1.2.3 Cryptography capabilities

6.1.2.3.1 Functions

6.1.2.3.1.1 smw_config_check_digest

enum [smw_status_code](#) **smw_config_check_digest**([smw_subsystem_t](#) subsystem, [smw_hash_algo_t](#) algo)

Check digest support.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Name of the subsystem.
- **algo** ([smw_hash_algo_t](#)) - [in] Digest algorithm name.

Description

Function checks if the digest `algo` is supported on the given subsystem or default subsystem if `subsystem` is not specified ([SMW_SUBSYSTEM_NAME_NONE](#)).

Return

- **SMW_STATUS_OK:**
`algo` is supported.
- **SMW_STATUS_INVALID_PARAM:**
`algo` is not specified.
- **SMW_STATUS_UNKNOWN_ALGO_NAME:**
`algo` is not valid.
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
`algo` is not supported.
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
`subsystem` is not valid.
- Other error code from [enum smw_status_code](#)

6.1.2.3.1.2 smw_config_check_sign

enum [smw_status_code](#) **smw_config_check_sign**([smw_subsystem_t](#) subsystem, struct [smw_signature_info](#) *info)

Check signature generation support.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Name of the subsystem.
- **info** (struct [smw_signature_info](#)*) - [in] Signature information.

Description

Function checks if the signature generation supports the signature information `info` on the given subsystem or default subsystem if `subsystem` is not specified ([SMW_SUBSYSTEM_NAME_NONE](#)).

The hash algorithm name field `info->hash_algo_name` is optional. If set, the function checks if given subsystem or default subsystem supports the hash algorithm for the signature generation.

The signature type name field `info->type_name` is optional. If set, the function checks if given subsystem or default subsystem supports the signature type for the signature generation.

Return

- **SMW_STATUS_OK:**
Signature operation is supported.
- **SMW_STATUS_INVALID_PARAM:**
 - `info` is NULL.
 - `info->algo_name` is [SMW_SIGNATURE_ALGO_NAME_NONE](#).
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
Signature operation is not supported.
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
`subsystem` is not valid.
- Other error code from [enum smw_status_code](#)

6.1.2.3.1.3 smw_config_check_verify

enum [smw_status_code](#) **smw_config_check_verify**([smw_subsystem_t](#) subsystem, struct [smw_signature_info](#) *info)

Check signature verification support.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Name of the subsystem.
- **info** (struct [smw_signature_info](#)*) - [in] Signature information.

Description

Function checks if the signature verification supports the signature information `info` on the given subsystem or default subsystem if `subsystem` is not specified ([SMW_SUBSYSTEM_NAME_NONE](#)).

The hash algorithm name field `info->hash_algo_name` is optional. If set, the function checks if given subsystem or default subsystem supports the hash algorithm for the signature verification.

The signature type name field `info->type_name` is optional. If set, the function checks if given subsystem or default subsystem supports the signature type for the signature verification.

Return

- **SMW_STATUS_OK:**
Verify operation is supported.
- **SMW_STATUS_INVALID_PARAM:**
 - `info` is NULL.
 - `info->algo_name` is `SMW_SIGNATURE_ALGO_NAME_NONE`.
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
Verify operation is not supported.
- Other error code from [enum smw_status_code](#)

6.1.2.3.1.4 smw_config_check_cipher

enum [smw_status_code](#) `smw_config_check_cipher(smw\_subsystem\_t subsystem, struct smw\_cipher\_info *info)`

Check cipher support.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Name of the subsystem.
- **info** (struct [smw_cipher_info](#)*) - [in] Cipher information.

Description

Function checks if the cipher information `info` is supported on the given subsystem or default subsystem if `subsystem` is not specified ([SMW_SUBSYSTEM_NAME_NONE](#)).

Return

- **SMW_STATUS_OK:**
Cipher operation is supported.
- **SMW_STATUS_INVALID_PARAM:**
 - `info` is NULL.
 - `info->key_type_name` is `SMW_KEY_TYPE_NAME_NONE`.
 - `info->mode_name` is `SMW_CIPHER_MODE_NAME_NONE`.
 - `info->op_type_name` is `SMW_CIPHER_OP_TYPE_NAME_NONE`.
- **SMW_STATUS_UNKNOWN_KEY_TYPE_NAME:**
`info->key_type_name` is not valid.
- **SMW_STATUS_UNKNOWN_MODE_NAME:**
`info->mode_name` is not valid.
- **SMW_STATUS_UNKNOWN_OP_TYPE_NAME:**
`info->op_type_name` is not valid.
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
Cipher operation is not supported.

- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**

`subsystem` is not valid.

- Other error code from [enum smw_status_code](#)

6.1.2.3.1.5 smw_config_check_aead

enum [smw_status_code](#) **smw_config_check_aead**([smw_subsystem_t](#) subsystem, struct [smw_aead_info](#) *info)

Check authentication encryption support.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Name of the subsystem.
- **info** (struct [smw_aead_info](#)*) - [in] AEAD information.

Description

Function checks if the authentication encryption information `info` is supported on the given subsystem or default subsystem if `subsystem` is not specified ([SMW_SUBSYSTEM_NAME_NONE](#)).

Return

- **SMW_STATUS_OK:**

AEAD operation is supported.

- **SMW_STATUS_INVALID_PARAM:**

- `info` is NULL.
- `info->key_type_name` is [SMW_KEY_TYPE_NAME_NONE](#).
- `info->mode_name` is [SMW_AEAD_MODE_NAME_NONE](#).
- `info->op_type_name` is [SMW_AEAD_OP_TYPE_NAME_NONE](#).

- **SMW_STATUS_UNKNOWN_KEY_TYPE_NAME:**

`info->key_type_name` is not valid.

- **SMW_STATUS_UNKNOWN_MODE_NAME:**

`info->mode_name` is not valid.

- **SMW_STATUS_UNKNOWN_OP_TYPE_NAME:**

`info->op_type_name` is not valid.

- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**

AEAD operation is not supported.

- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**

`subsystem` is not valid.

- Other error code from [enum smw_status_code](#)

6.1.2.3.1.6 smw_config_check_mac

enum [smw_status_code](#) **smw_config_check_mac**([smw_subsystem_t](#) subsystem, struct [smw_mac_info](#) *info)

Check MAC support.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Name of the subsystem.
- **info** (struct [smw_mac_info*](#)) - [in] MAC information.

Description

Function checks if the MAC information `info` is supported on the given subsystem or default subsystem if `subsystem` is not specified ([SMW_SUBSYSTEM_NAME_NONE](#)).

The hash algorithm name field `info->hash_algo_name` is optional. If set, the function checks if given subsystem or default subsystem supports the hash algorithm for the MAC operation.

The MAC algorithm name field `info->mac_algo_name` is optional. If set, the function checks if given subsystem or default subsystem supports the MAC algorithm for the MAC operation.

Return

- **SMW_STATUS_OK:**
MAC operation is supported.
- **SMW_STATUS_INVALID_PARAM:**
 - `info` is NULL.
 - `info->key_type_name` is `SMW_KEY_TYPE_NAME_NONE`.
- **SMW_STATUS_UNKNOWN_KEY_TYPE_NAME:**
`info->key_type_name` is not valid.
- **SMW_STATUS_UNKNOWN_ALGO_NAME:**
`info->mac_algo_name` or `info->hash_algo_name` is not valid.
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
MAC operation is not supported.
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
`subsystem` is not valid.
- Other error code from [enum smw_status_code](#)

6.1.2.3.1.7 smw_config_check_asymmetric_encrypt

enum [smw_status_code](#) `smw_config_check_asymmetric_encrypt(smw\_subsystem\_t subsystem, struct smw\_asymmetric\_encrypt\_info *info)`

Check asymmetric encryption support.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Name of the subsystem.
- **info** (struct [smw_asymmetric_encrypt_info*](#)) - [in] Asymmetric encryption operation information.

Description

Function checks if the asymmetric encryption information `info` is supported on the given subsystem or default subsystem if `subsystem` is not specified ([SMW_SUBSYSTEM_NAME_NONE](#)).

The hash algorithm name field `info->hash_algo_name` is optional. If set, the function checks if given subsystem or default subsystem supports the hash algorithm for the asymmetric encryption operation.

The encryption mode name field `info->mode_name` is optional. If set, the function checks if given subsystem or default subsystem supports the encryption mode for the asymmetric encryption operation.

Return

- **SMW_STATUS_OK:**
Encryption operation is supported.
- **SMW_STATUS_INVALID_PARAM:**
 - `info` is NULL.
 - `info->algo_name` is `SMW_ASYMMETRIC_ENCRYPTION_ALGO_NAME_NONE`.
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
Encryption operation is not supported.
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
`subsystem` is not valid.
- Other error code from [enum smw_status_code](#)

6.1.2.3.1.8 smw_config_check_asymmetric_decrypt

enum [smw_status_code](#) `smw_config_check_asymmetric_decrypt(smw\_subsystem\_t subsystem, struct smw\_asymmetric\_encrypt\_info *info)`

Check asymmetric decryption support.

Parameters

- **subsystem** ([smw_subsystem_t](#)) - [in] Name of the subsystem.
- **info** (struct [smw_asymmetric_encrypt_info](#)*) - [in] Asymmetric encryption operation information.

Description

Function checks if the asymmetric encryption information `info` is supported on the given `subsystem` or default subsystem if `subsystem` is not specified ([SMW_SUBSYSTEM_NAME_NONE](#)).

The hash algorithm name field `info->hash_algo_name` is optional. If set, the function checks if given subsystem or default subsystem supports the hash algorithm for the asymmetric decryption operation.

The encryption mode name field `info->mode_name` is optional. If set, the function checks if given subsystem or default subsystem supports the encryption mode for the asymmetric decryption operation.

Return

- **SMW_STATUS_OK:**
Decryption operation is supported.
- **SMW_STATUS_INVALID_PARAM:**
 - `info` is NULL.
 - `info->algo_name` is `SMW_ASYMMETRIC_ENCRYPTION_ALGO_NAME_NONE`.
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**

Decryption operation is not supported.

- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
 `subsystem` is not valid.
- Other error code from [enum smw_status_code](#)

6.1.2.3.2 Structures

6.1.2.3.2.1 struct smw_signature_info

struct **smw_signature_info**

Signature operation information

Definition

```
struct smw_signature_info {  
    smw_signature_algo_t algo_name;  
    smw_signature_type_t type_name;  
    smw_hash_algo_t hash_algo_name;  
}
```

Members

algo_name

[in] Signature algo name. See [typedef smw_signature_algo_t](#).

type_name

[in] (**optional**) Signature type name. See [typedef smw_signature_type_t](#).

hash_algo_name

[in] (**optional**) Hash algorithm name. See [typedef smw_hash_algo_t](#).

6.1.2.3.2.2 struct smw_cipher_info

struct **smw_cipher_info**

Cipher operation information

Definition

```
struct smw_cipher_info {  
    bool multipart;  
    smw_key_type_t key_type_name;  
    smw_cipher_mode_t mode_name;  
    smw_cipher_op_type_t op_type_name;  
}
```

Members

multipart

[in] True if it's a cipher multi-part operation

key_type_name

[in] Key type name. See [typedef smw_key_type_t](#)

mode_name

[in] Operation mode name. See [typedef smw_cipher_mode_t](#)

op_type_name

[in] Operation type name. See [typedef smw_cipher_op_type_t](#)

6.1.2.3.2.3 struct smw_aead_info

struct smw_aead_info

Authentication encryption operation information

Definition

```
struct smw_aead_info {  
    bool multipart;  
    smw_key_type_t key_type_name;  
    smw_aead_mode_t mode_name;  
    smw_aead_op_type_t op_type_name;  
}
```

Members

multipart

[in] True if it's a AEAD multi-part operation

key_type_name

[in] Key type name. See [typedef smw_key_type_t](#)

mode_name

[in] Operation mode name. See [typedef smw_aead_mode_t](#)

op_type_name

[in] Operation type name. See [typedef smw_aead_op_type_t](#)

6.1.2.3.2.4 struct smw_mac_info

struct smw_mac_info

MAC operation information

Definition

```
struct smw_mac_info {  
    smw_key_type_t key_type_name;  
    smw_mac_algo_t mac_algo_name;  
    smw_hash_algo_t hash_algo_name;  
}
```

Members

key_type_name

[in] Key type name. See [typedef smw_key_type_t](#)

mac_algo_name

[in] (optional) MAC algorithm name. See [typedef smw_mac_algo_t](#)

hash_algo_name

[in] (optional) Hash algorithm name. See [typedef smw_hash_algo_t](#)

6.1.2.3.2.5 struct smw_asymmetric_encrypt_info

struct smw_asymmetric_encrypt_info

Asymmetric encryption/decryption operation information

Definition

```
struct smw_asymmetric_encrypt_info {
    smw_asymmetric_encryption_algo_t algo_name;
    smw_asymmetric_encryption_mode_t mode_name;
    smw_hash_algo_t hash_algo_name;
}
```

Members

algo_name

[in] Encryption algorithm name. See [typedef smw_asymmetric_encryption_algo_t](#)

mode_name

[in] (optional) Encryption mode (padding scheme) name. See [typedef smw_asymmetric_encryption_mode_t](#)

hash_algo_name

[in] (optional) Hash algorithm name. See [typedef smw_hash_algo_t](#)

6.1.2.4 Library configuration

6.1.2.4.1 Functions

6.1.2.4.1.1 smw_config_load

enum [smw_status_code](#) **smw_config_load**(char *buffer, unsigned int size, unsigned int *offset)

Load a configuration.

Parameters

- **buffer** (char*) - [in] pointer to the plaintext configuration.
- **size** (unsigned int) - [in] size in bytes of the plaintext configuration.
- **offset** (unsigned int*) - [out] offset in plaintext configuration where parsing failed.

Description

This function loads a configuration.

The plaintext configuration is parsed and the content is stored in the library configuration database.

If the parsing of plaintext configuration fails, `offset` points to the number of characters that have been correctly parsed. The beginning of the remaining plaintext which cannot be parsed is printed out.

Return

- **SMW_STATUS_OK:**
Configuration load is successful.
- **SMW_STATUS_INVALID_LIBRARY_CONTEXT:**
Library context is not valid.
- **SMW_STATUS_INVALID_BUFFER:**
 - `buffer` is NULL.
 - `size` is 0.
- **SMW_STATUS_CONFIG_ALREADY_LOADED:**
A configuration is already loaded.
- Other error code from [enum smw_status_code](#)

6.1.2.4.1.2 smw_config_unload

enum [smw_status_code](#) **smw_config_unload**(void)

Unload the current configuration.

Parameters

- **void** - no arguments

Description

This function unloads the current configuration.

It frees all memory dynamically allocated by SMW.

Return

- **SMW_STATUS_OK:**
Configuration unload is successful.
- **SMW_STATUS_INVALID_LIBRARY_CONTEXT**
Library context is not valid.
- **SMW_STATUS_NO_CONFIG_LOADED**
No configuration is loaded.
- Other error code from [enum smw_status_code](#)

6.1.3 Key Manager APIs

The Key Manager APIs allow users of the library to manage cryptographic keys. such as generation, derivation, import, export, and deletion.

6.1.3.1 Key Creation

6.1.3.1.1 Key Generate

6.1.3.1.1.1 smw_generate_key

enum [smw_status_code](#) smw_generate_key(struct [smw_generate_key_args](#) *args)

Generate a Key.

Parameters

- **args** (struct [smw_generate_key_args](#)*) - Pointer to the structure that contains the Key generation arguments.

Description

This function generates a Key.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - args is NULL.
 - args->key_descriptor is NULL.
 - args->key_descriptor->type_name is SMW_KEY_TYPE_NAME_NONE.
 - args->key_descriptor->security_size is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.3.1.1.2 struct smw_generate_key_args

struct **smw_generate_key_args**

Key generation arguments

Definition

```
struct smw_generate_key_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_key_descriptor *key_descriptor;  
}
```

Members

version

[in] Version of this structure

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#)

key_descriptor

Pointer to a Key descriptor object. See [struct smw_key_descriptor](#)

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the default configured Secure Subsystem is used.

The `key_descriptor` fields:

- `id`: [in] key identifier:
 - if set to 0, the API will return a new key identifier.
 - if set by the caller (other than 0) subsystem will create a key with user defined key identifier. If key identifier already exists, operation fails. By principle, the user defined key identifier must not be set when transient key is created. Some subsystems may not support user defined key identifiers for transient keys.
- `type_name`: [in] Key type name. See [typedef smw_key_type_t](#)
- `security_size`: [in] Security size in bits.
- `attributes`: Key attributes. See [struct smw_key_attributes](#).
 - [in] key attributes to set.
 - [out] key attributes effectively set by the subsystem.
- `buffer`: [in/out] **Optional**, used to export the asymmetric public key generated. The buffer array and size must be set to contain the public key value. If buffer is too small, operation returns with `SMW_STATUS_OUTPUT_TOO_SHORT` updating the expected buffer size. Key is not created.

6.1.3.1.2 Key import

6.1.3.1.2.1 smw_import_key

enum [smw_status_code](#) `smw_import_key(struct smw_import_key_args *args)`

Import a key.

Parameters

- `args` (struct [smw_import_key_args*](#)) - Pointer to the structure that contains the key import arguments.

Description

This function imports a key into the storage managed by the Secure Subsystem. The key must be in Secure Subsystem import key supported format, that could be plaintext or specific format conveying key value in encrypted mode. Refer to the [Secure Subsystems Features and Capabilities](#) for more details.

Return

- `SMW_STATUS_OK`:

Operation succeeded.

- **SMW_STATUS_INVALID_PARAM:**

- args is NULL.
- args->key_descriptor is NULL.
- args->key_descriptor->type_name is SMW_KEY_TYPE_NAME_NONE.
- args->key_descriptor->security_size is 0.
- Invalid definition of the key buffer(s).
- Additional invalid parameters returned by the subsystem.

- Other error code from [enum smw_status_code](#)

6.1.3.1.2.2 struct smw_import_key_args

struct **smw_import_key_args**

Key import arguments

Definition

```
struct smw_import_key_args {
    unsigned char version;
    smw_subsystem_t subsystem_name;
    struct smw_key_descriptor *key_descriptor;
}
```

Members

version

[in] Version of this structure

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#)

key_descriptor

[in/out] Pointer to a Key descriptor object. See [struct smw_key_descriptor](#)

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the default configured Secure Subsystem is used.

The `key_descriptor` fields:

- `id`: [in] key identifier:
 - if set to 0, the API will return a new key identifier.
 - if set by the caller (other than 0) subsystem will create a key with user defined key identifier. If key identifier already exists, operation fails. By principle, the user defined key identifier must not be set when transient key is created. Some subsystems may not support user defined key identifiers for transient keys.
- `type_name`: [in] Key type name. See [typedef smw_key_type_t](#)
- `security_size`: [in] Security size in bits.
- `attributes`: Key attributes. See [struct smw_key_attributes](#).
 - [in] key attributes to set.

- [out] key attributes effectively set by the subsystem.
- `buffer`: [in/out] define the key value to import. Function of the subsystem, importable key can be:
 - Symmetric key private key buffer contains the key value.
 - ECC or Edwards asymmetric public key: public key buffer contains the key value.
 - ECC or Edwards asymmetric keypair: public and private key buffer contains the key value.
 - RSA asymmetric public key: public and modulus key buffers contains the key value.
 - RSA asymmetric keypair: public, private and modulus key buffers contains the key value.

The `buffer` field `format_name` is optional. The default value is "HEX".

Secure subsystem may accept or not to import a private or secure key buffer plaintext value.

Secure subsystem may only accept to import a private or secure key buffer encoded in a specific format conveying the encrypted value of the key. The `key_descriptor->attributes.storage_id` must be set according to the key encoding format.

6.1.3.2 Key Derivation

6.1.3.2.1 Function

6.1.3.2.1.1 `smw_derive_key`

enum [smw_status_code](#) `smw_derive_key`(struct [smw_derive_key_args](#) *args)

Derive a Key.

Parameters

- `args` (struct [smw_derive_key_args](#)*) - Pointer to the structure that contains the Key derivation arguments.

Description

This function derives a Key.

On operation completion, the `args->key_descriptor_derived->shared_secret_len` is updated to the correct value when:

- Shared secret buffer length is bigger than expected. In this case, operation succeeds.
- Shared secret buffer length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->key_descriptor` is NULL.
 - `args->key_descriptor->type_name` is `SMW_KEY_TYPE_NAME_NONE`.
 - `args->key_descriptor->security_size` is 0.
 - If expected to return a shared buffer, `args->key_descriptor_derived` Shared Secret buffer not correctly defined.
 - Additional invalid parameters returned by the subsystem.

- Other error code from [enum smw_status_code](#)

6.1.3.2.1.2 struct smw_derive_key_args

struct **smw_derive_key_args**

Key derivation arguments

Definition

```
struct smw_derive_key_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    smw_kdf_t kdf_name;  
    void *kdf_arguments;  
    bool store_derived_key;  
    struct smw_key_descriptor *key_descriptor_base;  
    struct smw_derived_key_descriptor *key_descriptor_derived;  
}
```

Members

version

[in] Version of this structure

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#)

kdf_name

[in] Key derivation function name. See [typedef smw_kdf_t](#)

kdf_arguments

[in] Key derivation function arguments

store_derived_key

[in] If true, store the derived key.

key_descriptor_base

[in] Pointer to a Key base descriptor. See [struct smw_key_descriptor](#)

key_descriptor_derived

[in/out] Pointer to the Key derived descriptor structure. See [struct smw_derived_key_descriptor](#)

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the default configured Secure Subsystem is used.

A new key is derived from a given key base (`@key_descriptor_base`) using the key derivation function `kdf_name`. If the key derivation function requires more arguments, the `kdf_arguments` refers to the associated key derivation function arguments, else this pointer is not used and can be NULL.

Upon successful completion of the key derivation operation, if `store_derived_key` is set to true, a new key ID is set in the `key_descriptor_derived->id`. The shared secret data is exported if `key_descriptor_`

derived->shared_secret and key_descriptor_derived->shared_secret_len are set. Refer to the [Secure Subsystems Features and Capabilities](#) for more details.

The key_descriptor_derived.attributes must be defined.

6.1.3.2.2 TLS 1.2 derivation

6.1.3.2.2.1 struct smw_kdf_tls12_random_data

struct **smw_kdf_tls12_random_data**

TLS 1.2 random data

Definition

```
struct smw_kdf_tls12_random_data {
    unsigned char version;
    unsigned char *client_random;
    unsigned int client_random_length;
    unsigned char *server_random;
    unsigned int server_random_length;
}
```

Members

version

[in] Version of this structure.

client_random

[in] Pointer to Client random data buffer.

client_random_length

[in] Length in bytes of the Client random data buffer.

server_random

[in] Pointer to Server random data buffer.

server_random_length

[in] Length in bytes of the Server random data buffer.

6.1.3.2.2.2 struct smw_kdf_tls12_session_hash

struct **smw_kdf_tls12_session_hash**

TLS 1.2 session hash

Definition

```
struct smw_kdf_tls12_session_hash {
    unsigned char version;
    unsigned char *hash;
    unsigned int hash_length;
}
```

Members

version

[in] Version of this structure.

hash

[in] Pointer to the hash buffer of the session data.

hash_length

[in] Length in bytes of the hash buffer.

6.1.3.2.2.3 struct smw_kdf_tls12_master_secret_args

struct **smw_kdf_tls12_master_secret_args**

TLS 1.2 master secret arguments

Definition

```
struct smw_kdf_tls12_master_secret_args {  
    unsigned char version;  
    smw_tls12_keat key_exchange_name;  
    bool ext_master_key;  
    unsigned char *peer_public_buffer;  
    unsigned int peer_public_buffer_length;  
    union {  
        struct smw_kdf_tls12_random_data *random_data;  
        struct smw_kdf_tls12_session_hash *session_hash;  
    };  
}
```

Members

version

[in] Version of this structure.

key_exchange_name

[in] Name of the key exchange algorithm. See [typedef smw_tls12_keat](#)

ext_master_key

[in] If true, generates an extended master secret key.

peer_public_buffer

[in] Pointer to the Peer public key buffer used for ECDH(E).

peer_public_buffer_length

[in] Length in bytes of the Peer public key buffer.

{unnamed_union}

anonymous

random_data

[in] If `ext_master_key` is *false*, definition of the session random data ([smw_kdf_tls12_random_data](#)).

session_hash

[in] If `ext_master_key` is *true*, definition of the session hash ([smw_kdf_tls12_session_hash](#)).

Description

This structure defines the additional arguments needed for the TLS 1.2 Master Secret, when:

- `smw_derive_key_args->kdf_name` = [SMW_KDF_NAME_TLS12_OP_KEY_EXCHANGE](#)
- `smw_derive_key_args->kdf_arguments` is a type `struct smw_kdf_tls12_op_args` where `smw_kdf_tls12_op_args->op_name` = [SMW_TLS12_OP_NAME_MASTER_SECRET](#).

Note:

As detailed in the [Extended Master Secret - TLS1.2 extension, RFC 7627](#):

- when `ext_master_key` is *true*, the *session_hash* should be set.
- when `ext_master_key` is *false*, the *random_data* should be set.

6.1.3.2.2.4 struct smw_kdf_tls12_key_expansion_args

struct **smw_kdf_tls12_key_expansion_args**

TLS 1.2 key expansion arguments

Definition

```
struct smw_kdf_tls12_key_expansion_args {
    unsigned char version;
    smw_tls12_enc_t encryption_name;
    struct smw_kdf_tls12_random_data *random_data;
    unsigned int client_w_enc_key_id;
    unsigned int server_w_enc_key_id;
    unsigned int client_w_mac_key_id;
    unsigned int server_w_mac_key_id;
    unsigned char *client_w_iv;
    unsigned int client_w_iv_length;
    unsigned char *server_w_iv;
    unsigned int server_w_iv_length;
}
```

Members

version

[in] Version of this structure.

encryption_name

[in] Name of the encryption algorithm. See [typedef smw_tls12_enc_t](#)

random_data

[in] The session random data.

client_w_enc_key_id

[out] Generated client write encryption key identifier.

server_w_enc_key_id

[out] Generated server write encryption key identifier.

client_w_mac_key_id

[out] Generated client write MAC key identifier (see note 1).

server_w_mac_key_id

[out] Generated server write MAC key identifier (see note 1).

client_w_iv

[in/out] Pointer to the Client IV buffer (see note 2).

client_w_iv_length

[in/out] Length in bytes of the Client IV buffer.

server_w_iv

[in/out] Pointer to the Server IV buffer (see note 2).

server_w_iv_length

[in/out] Length in bytes of the Server IV buffer.

Description

This structure defines the additional arguments needed for the TLS 1.2 Key Expansion, when:

- `smw_derive_key_args->kdf_name` = [SMW_KDF_NAME_TLS12_OP_KEY_EXCHANGE](#)
- `smw_derive_key_args->kdf_arguments` is a type `struct smw_kdf_tls12_op_args` where `smw_kdf_tls12_op_args->op_name` = [SMW_TLS12_OP_NAME_KEY_EXPANSION](#).

Note:

1. Client/Server write MAC key are not generated with AEAD cipher encryption (CCM, GCM, CHACHA20_POLY1305).
2. Client/Server write IVs are generated only in case of AEAD cipher modes (CCM, GCM, CHACHA20_POLY1305).

6.1.3.2.2.5 struct smw_kdf_tls12_op_args**struct smw_kdf_tls12_op_args**

TLS 1.2 “operation-based” arguments

Definition

```
struct smw_kdf_tls12_op_args {
    unsigned char version;
    smw_hash_algo_t prf_name;
    smw_tls12_op_t op_name;
    struct smw_op_context *context;
    union {
        struct smw_kdf_tls12_master_secret_args master_secret;
        struct smw_kdf_tls12_key_expansion_args key_expansion;
    };
}
```

Members**version**

[in] Version of this structure.

prf_name

[in] Name of the Pseudo-Random Function (PRF). See [typedef smw_hash_algo_t](#)

op_name

[in] Name of the operation to execute. See [typedef smw_tls12_op_t](#)

context

[in] Pointer to an opaque operation context structure. See `struct smw_op_context`

{unnamed_union}

anonymous

master_secret

[in] The TLS1.2 Master Secret parameters.

key_expansion

[in] The TLS1.2 Key Expansion parameters.

Description

The `context` passed through this structure must be a valid context which is the result of the [smw_allocate_context\(\)](#) function. Subsystems may allocate data internally and associate it with the context. The same context needs to be passed to the master secret and key expansion operations.

Upon completion of the key expansion operation, the context and any associated data is freed.

In previous versions of the SMW library (up to and incl. 5.4), the context was not freed and remained valid, so applications needed to call [smw_cancel_operation\(\)](#). This call is no longer needed.

6.1.3.2.2.6 struct smw_kdf_tls12_args

struct smw_kdf_tls12_args

Key derivation function TLS 1.2 arguments

Definition

```
struct smw_kdf_tls12_args {
    smw_tls12_keas_t key_exchange_name;
    smw_tls12_enc_t encryption_name;
    smw_hash_algo_t prf_name;
    bool ext_master_key;
    unsigned char *kdf_input;
    unsigned int kdf_input_length;
    unsigned int master_sec_key_id;
    unsigned int client_w_enc_key_id;
    unsigned int server_w_enc_key_id;
    unsigned int client_w_mac_key_id;
    unsigned int server_w_mac_key_id;
    unsigned char *client_w_iv;
    unsigned int client_w_iv_length;
    unsigned char *server_w_iv;
    unsigned int server_w_iv_length;
}
```

Members

key_exchange_name

[in] Name of the key exchange algorithm. See [typedef smw_tls12_kea_t](#)

encryption_name

[in] Name of the encryption algorithm. See [typedef smw_tls12_enc_t](#)

prf_name

[in] Name of the Pseudo-Random Function (PRF). See [typedef smw_hash_algo_t](#)

ext_master_key

[in] If true, generates an extended master secret key.

kdf_input

[in] Pointer to the key derivation input data buffer used to generate the master secret key.

kdf_input_length

[in] Length in bytes of the key derivation input data buffer.

master_sec_key_id

[out] Generated master key identifier.

client_w_enc_key_id

[out] Generated client write encryption key identifier.

server_w_enc_key_id

[out] Generated server write encryption key identifier.

client_w_mac_key_id

[out] Generated client write MAC key identifier (see note 1).

server_w_mac_key_id

[out] Generated server write MAC key identifier (see note 1).

client_w_iv

[out] Pointer to the Client IV buffer (see note 2).

client_w_iv_length

[in/out] Length in bytes of the Client IV buffer.

server_w_iv

[out] Pointer to the Server IV buffer (see note 2).

server_w_iv_length

[in/out] Length in bytes of the Server IV buffer.

Description

This structure defines the additional arguments needed for the TLS 1.2 Key derivation when:

- [smw_derive_key_args->kdf_name](#) = [SMW_KDF_NAME_TLS12_KEY_EXCHANGE](#)

Note:

1. Client/Server write MAC key are not generated with AES GCM cipher encryption.
2. Client/Server write IVs are generated only in case of Authentication Encryption with Additional Data Cipher mode (like AES CCM or GCM).

The key derivation [smw_derive_key_args->key_descriptor_derived](#) is filled only if the `key_exchange_name` request for an ephemeral public key.

Following [smw_derive_key_args->key_descriptor_derived](#) fields are:

- `id`: set to 0
- `type_name`: Set the key type name
- `security_size`: Size in bits of the derived key
- `shared_secret`: Shared secret buffer
- `shared_secret_len`: Shared secret buffer length

Warning:

This structure is deprecated and will be removed in a future library release. Please use the new structure [smw_kdf_tls12_op_args](#) instead.

6.1.3.2.3 TLS 1.3 derivation

6.1.3.2.3.1 struct smw_kdf_tls13_args

struct **smw_kdf_tls13_args**

TLS1.3 KDF arguments structure

Definition

```
struct smw_kdf_tls13_args {
    unsigned char version;
    struct smw_key_descriptor *psk;
    smw_hash_algo_t prf_name;
    unsigned char *peer_public_buffer;
    unsigned int peer_public_buffer_length;
    unsigned char *expanded_label;
    unsigned int expanded_label_length;
}
```

Members

version

[in] Version of this structure.

psk

[in, optional] The Pre-Shared Key to use for Early Secret derivation. See [struct smw_key_descriptor](#)

prf_name

[in] Name of the Pseudo-Random Function (PRF). See [typedef smw_hash_algo_t](#)

peer_public_buffer

[in] Pointer to the Peer public buffer.

peer_public_buffer_length

[in] Length in bytes of the Peer public buffer.

expanded_label

[in] Pointer to the expanded label buffer.

expanded_label_length

[in] Length in bytes of the expanded label buffer.

Description

The `expand_label` must be a buffer that contains the output of TLS1.3's "HKDF-Expand_label". The [smw_tls13_expand_label\(\)](#) is a helper function you may use to compute the expanded label, from the input label and context data (the context is usually the Transcript Hash).

The value of the input label controls which actual secrets get derived, e.g.:

- "ext binder" -> `binder_key`
- "c hs traffic" -> `client_handshake_traffic_secret`
- "c ap traffic" -> `client_application_traffic_secret_0`

Please refer to [RFC 8446, section 7.1](#), to see the possible values for the label.

The TLS 1.3 expand label buffer could be created using the following helper function, structure and macro:

6.1.3.2.3.2 smw_tls13_expand_label

enum [smw_status_code](#) [smw_tls13_expand_label](#)(struct [smw_tls13_expand_label_args](#) *args)

Helper function to construct the TLS1.3 "HkdfLabel", used by the "HKDF-Expand-Label" function.

Parameters

- **args** (struct [smw_tls13_expand_label_args](#)*) - Expand label arguments.

Description

`args->expanded_label` should be a buffer that is large enough to hold the expanded label; if that is not the case, this function will return `SMW_STATUS_OUTPUT_TOO_SHORT` and set `args->expanded_label_length` to the minimum required length.

Alternatively, you may also use the [macro SMW_TLS13_EXPANDED_LABEL_LENGTH](#) to get the minimum required length.

This function will also prepend `args->label` with "tls13 " as per [RFC8446 section 7.1](#).

Return

- **SMW_STATUS_OK:**
Success.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->length` is 0.
 - `args->label` is NULL.
 - `args->label_length` is 0.
 - `args->context` is NULL and `args->context_length` is not 0.
 - `args->length` is greater than `UCHAR_MAX`.
 - `args->label_length` is greater than `(UCHAR_MAX - SMW\_TLS13\_PREFIX\_LENGTH)`.
 - `args->context_length` is greater than `UCHAR_MAX`.
- **SMW_STATUS_TOO_LARGE_NUMBER:**

One of the input length parameters is too large.

- **SMW_STATUS_OUTPUT_TOO_SHORT:**

Output buffer is too short.

- Other error code from [enum smw_status_code](#)

6.1.3.2.3.3 struct smw_tls13_expand_label_args

struct **smw_tls13_expand_label_args**

TLS1.3 “HKDF-Expand-Label” arguments structure

Definition

```
struct smw_tls13_expand_label_args {  
    unsigned char version;  
    unsigned int length;  
    unsigned char *label;  
    unsigned int label_length;  
    unsigned char *context;  
    unsigned int context_length;  
    unsigned char *expanded_label;  
    unsigned int expanded_label_length;  
}
```

Members

version

[in] Version of this structure.

length

[in] Length in bytes of the context specific. Usually equal to hash's length of the concatenation of the handshake messages.

label

[in] Input label buffer, without the “tls13 “ prefix.

label_length

[in] Length in bytes of the label buffer.

context

[in] (**optional**) Input context buffer, usually the transcript Hash.

context_length

[in] Length in bytes if the context buffer.

expanded_label

[out] Buffer that holds the expanded label.

expanded_label_length

[in/out] Length in bytes of the expanded label buffer.

6.1.3.2.3.4 macro SMW_TLS13_PREFIX

SMW_TLS13_PREFIX()

TLS 1.3 label prefix

Description

Define the TLS 1.3 label prefix string “**tls13**” used in HKDF-Expand-Label operations.

6.1.3.2.3.5 macro SMW_TLS13_PREFIX_LENGTH

SMW_TLS13_PREFIX_LENGTH()

TLS 1.3 label prefix length

Description

Define the length in bytes of the TLS 1.3 label prefix [SMW_TLS13_PREFIX](#).

6.1.3.2.3.6 macro SMW_TLS13_EXPANDED_LABEL_LENGTH

SMW_TLS13_EXPANDED_LABEL_LENGTH(*arg*)

Return the minimum length needed to construct a TLS1.3 expanded label.

Parameters

- **arg** - An pointer to a [struct smw_tls13_expand_label_args](#).

Description

In order to avoid 2 calls to [smw_tls13_expand_label\(\)](#), you may use this macro to get the minimum required expanded label buffer length, after having filled in the `arg->label_length` and `arg->context_length` values.

Return

Minimum length needed for `arg->expanded_label_length`

6.1.3.2.4 HKDF derivation

6.1.3.2.4.1 struct smw_hkdf_extract_args

struct **smw_hkdf_extract_args**

HKDF extract step arguments structure

Definition

```
struct smw_hkdf_extract_args {  
    unsigned char *salt;  
    unsigned int salt_len;  
    unsigned char *peer_public_buffer;  
    unsigned int peer_public_buffer_len;  
};
```

```
}
```

Members

salt

[in] (optional) Pointer to the Salt buffer.

salt_len

[in] Length in bytes of the Salt buffer.

peer_public_buffer

[in] Pointer to the Peer public buffer in hex format.

peer_public_buffer_len

[in] Length in bytes of the Peer public buffer.

Description

Refer to [RFC5869 section 2.2](#)

6.1.3.2.4.2 struct smw_hkdf_expand_args

struct **smw_hkdf_expand_args**

HKDF expand arguments structure

Definition

```
struct smw_hkdf_expand_args {  
    unsigned char *info;  
    unsigned int  info_len;  
}
```

Members

info

[in] (optional) Context and application specific information buffer.

info_len

[in] Length in bytes of context and application specific information buffer.

Description

Refer to [RFC5869 section 2.3](#)

6.1.3.2.4.3 struct smw_hkdf_args

struct **smw_hkdf_args**

HKDF full arguments structure

Definition

```
struct smw_hkdf_args {  
    struct smw_hkdf_extract_args extract_args;  
    struct smw_hkdf_expand_args expand_args;  
}
```

Members

extract_args

[in] HKDF extract arguments structure. See `struct hkdf_extract_args`

expand_args

[in] HKDF expand arguments structure. See `struct hkdf_expand_args`

Description

This structure is used to operate a 2-steps HKDF key derivation that are extract and expand.

6.1.3.2.4.4 struct smw_kdf_hkdf_args

struct smw_kdf_hkdf_args

HMAC-based Key derivation function arguments

Definition

```
struct smw_kdf_hkdf_args {  
    smw_hash_algo_t hash_algo;  
    union {  
        struct smw_hkdf_args hkdf_args;  
        struct smw_hkdf_extract_args hkdf_extract_args;  
        struct smw_hkdf_expand_args hkdf_expand_args;  
    } ;  
}
```

Members

hash_algo

[in] Hash algorithm name. See [typedef smw_hash_algo_t](#)

{unnamed_union}

anonymous

hkdf_args

[in/out] HKDF full arguments. See [struct smw_hkdf_args](#)

hkdf_extract_args

[in/out] HKDF extract step arguments. See `struct hkdf_extract_args`

hkdf_expand_args

[in] HKDF expand step arguments. See `struct hkdf_expand_args`

Description

This structure defines the additional arguments needed for the HKDF derivation when:

- `smw_derive_key_args->kdf_name` = `SMW_KDF_NAME_HKDF`
- `smw_derive_key_args->kdf_name` = `SMW_KDF_NAME_HKDF_EXTRACT`
- `smw_derive_key_args->kdf_name` = `SMW_KDF_NAME_HKDF_EXPAND`

If user requests to store the derived key, user must provide:

- `smw_derive_key_args.key_descriptor_derived.type_name`
- `smw_derive_key_args.key_descriptor_derived.security_size`
- `smw_derive_key_args.key_attributes.usage_flags`
- `smw_derive_key_args.key_attributes.attributes`
- `smw_derive_key_args.key_attributes.permitted_algo`

The [RFC5869](#) details the HMAC-based Key derivation function (HKDF).

Key derivation using HKDF can be performed either in two dedicated steps (extract and expand) or combined into a single step, but only if the subsystem supports this capability.

- Step #1: Extract (Standalone operation not yet supported)
 - Upon successful completion of this step, resulting pseudorandom key buffer (PRK) is exported if the key derivation `smw_derive_key_args.key_descriptor_derived` Shared Secret buffer is defined. Otherwise a subsystem key is created and the `smw_derive_key_args.key_descriptor_derived` key identifier set with the new created key identifier.
- Step #2: Expand (Standalone operation not yet supported)
 - Upon successful completion of the key derivation operation, derived key descriptor structure `smw_derive_key_args.key_descriptor_derived` is updated. The new key ID is set and shared secret data is exported if `smw_derive_key_args.key_descriptor_derived.shared_secret` defined.
- Full HKDF (step 1 and step 2 combined)
 - Upon successful completion of the key derivation operation, derived key descriptor structure `smw_derive_key_args.key_descriptor_derived` is updated. The new derived key ID is set, and shared secret data is exported if buffer defined.

For the three types of operation supported, the input key material is either a key identifier or a raw buffer. The `smw_derive_key_args.key_descriptor_base` define how the input material IKM (in case of extract or full operation) or PRK (in case of expand operation) is given:

- It's a key identifier, the base key identifier set.
- It's a raw buffer, the base key descriptor's generic key keypair's Public data buffer contains the raw value and the key type name is set to `SMW_KEY_TYPE_NAME_RAW`.

6.1.3.2.5 ECDH derivation

6.1.3.2.5.1 struct smw_kdf_ecdh_args

struct **smw_kdf_ecdh_args**

Key derivation function ECDH arguments

Definition

```
struct smw_kdf_ecdh_args {
    unsigned char *peer_public_buffer;
```

```
    unsigned int peer_public_buffer_length;  
}
```

Members

peer_public_buffer

[in] Pointer to the Peer public key buffer in hex format.

peer_public_buffer_length

[in] Length in bytes of the Peer public key buffer.

Description

This structure defines the additional arguments needed for the ECDH Key derivation when:

- `smw_derive_key_args->kdf_name` = [SMW_KDF_NAME_ECDH](#)

Upon successful completion of the key derivation operation, derived key descriptor structure [smw_derive_key_args.key_descriptor_derived](#) is updated. The new derived key ID is set, and shared secret data is exported if Shared Secret buffer is defined.

6.1.3.2.6 OEM Master Key derivation

6.1.3.2.6.1 struct smw_kdf_oem_master_key_args

struct **smw_kdf_oem_master_key_args**

OEM Master key derivation arguments

Definition

```
struct smw_kdf_oem_master_key_args {  
    unsigned char version;  
    smw_oem_master_key_op_t op;  
    bool use_peer_key_digest_kdf;  
    bool use_oem_srkh_kdf;  
    unsigned char *peer_public_buffer;  
    unsigned int peer_public_buffer_length;  
    unsigned char *info;  
    unsigned int info_len;  
    unsigned char *payload;  
    unsigned int payload_length;  
}
```

Members

version

[in] Version of this structure.

op

[in] OEM Master key derivation operation. See [typedef smw_oem_master_key_op_t](#).

use_peer_key_digest_kdf

[in] True if the peer key digest value is used as salt to derive the OEM Master key.

use_oem_srkh_kdf

[in] True if the OEM SRKH is used as salt to derive OEM wrapping and signing keys.

peer_public_buffer

[in] Key derivation input data used to generate the shared secret key.

peer_public_buffer_length

[in] Length in bytes of the peer public buffer.

info

[in] [optional] Context and application specific information.

info_len

[in] Length in bytes of the information buffer.

payload

[in/out] Depends on the operation type defined by the operation.

payload_length

[in/out] Length in bytes of the payload buffer.

Description

OEM Master key derivation operation is not supported on all subsystems, refer to the [Secure Subsystems Features and Capabilities](#).

This key is used to import keys in the Secure Storage using a blob transporting the key in a secure manner.

The blob is a TLV encoded message in which the key is encrypted and blob is signed. The encryption of the key uses a key derived from the OEM Master key and the blob signature uses a key derived from the OEM Master key. More details are available in the [Secure Subsystems Features and Capabilities](#).

This structure contains the information required to:

- Either prepare the payload data that must be signed.
- Or derive the OEM Master key that will be used to import a key in the Secure Enclave.

This arguments structure is the `kdf_arguments` of the [struct smw_derive_key_args](#) where `kdf_name` is set to [SMW_KDF_NAME_OEM_MASTER_KEY](#)

All [struct smw_derive_key_args](#) fields may not be used for this operation and if set may be overwritten with hardcoded value as described in the subsystem capabilities.

The supported operations `op` are:

- Prepare the OEM Master key payload to sign:
The `payload` is an output, it's built based on the user parameters given for the operation. This payload must be signed to attest the derive operation. It's an optional functionality proposed to facilitate the user process. Other NXP tool, like [SPSDK](#) can be used to build and sign the payload. Calling this operation with `payload` set to NULL will return the expected length of the payload buffer in the `payload_length`.
- Derive the OEM Master key:
The `payload` is an input. It must be a payload signed.

6.1.3.3 Key Exportation

6.1.3.3.1 Function

6.1.3.3.1.1 smw_export_key

enum [smw_status_code](#) smw_export_key(struct [smw_export_key_args](#) *args)

Export a key.

Parameters

- **args** (struct [smw_export_key_args](#)*) - Pointer to the structure that contains the key export arguments.

Description

This function exports asymmetric or symmetric key if subsystem owning the key is supporting the operation.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - args is NULL.
 - args->key_descriptor is NULL.
 - args->key_descriptor->id is 0.
 - Invalid definition of the key buffer(s).
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.3.3.1.2 struct smw_export_key_args

struct **smw_export_key_args**

Key export arguments

Definition

```
struct smw_export_key_args {  
    unsigned char version;  
    struct smw_key_descriptor *key_descriptor;  
}
```

Members

version

[in] Version of this structure

key_descriptor

[in/out] Pointer to a Key descriptor object. See [struct smw_key_descriptor](#)

Description

The `key_descriptor` fields:

- `id`: [in] Key identifier. Identifier returned during the key creation [smw_generate_key\(\)](#) or [smw_import_key\(\)](#).
- `type_name`: [in] **Optional**. If defined but not correct, operation returns with `SMW_STATUS_INVALID_PARAM`. Advice to set it with `SMW_KEY_TYPE_NAME_NONE`.
- `security_size`: [in] **Optional**. If defined but not correct, operation returns with `SMW_STATUS_INVALID_PARAM`. Advice to set it with 0.
- `attributes`: [in] If key is present in the SMW's database, it's ignored. Otherwise, only key's persistency is considered.
- `buffer`: [out] Define the key value to export as follow:
 - For ECC or Edwards asymmetric key:
 - The public key buffer must be set in order to export the public key.
 - The private key buffer must be set in order to export the private key, only if the Secure Subsystem supports it. In that case, the private key may be encrypted, not plaintext.
 - For RSA asymmetric key:
 - The public key buffer must be set in order to export the public key.
 - The modulus buffer must be set in order to export the key's modulus.
 - The private key buffer must be set in order to export the private key, only if the Secure Subsystem supports it. In that case, the private key may be encrypted, not plaintext.
 - For symmetric key:
 - The private key buffer must be set in order to export the secure key, only if the Secure Subsystem supports it. In that case, the secure key may be encrypted, not plaintext.

The user can use [smw_get_key_buffers_lengths\(\)](#) to get the public and/or private key buffer(s) lengths, to allocate corresponding exported buffers.

6.1.3.4 Key Deletion

6.1.3.4.1 Function

6.1.3.4.1.1 smw_delete_key

enum [smw_status_code](#) [smw_delete_key](#)(struct [smw_delete_key_args](#) *args)

Delete a key.

Parameters

- `args` (struct [smw_delete_key_args](#)*) - Pointer to the structure that contains the key deletion arguments.

Description

This function deletes a key.

Return

- `SMW_STATUS_OK`:

Operation succeeded.

- **SMW_STATUS_INVALID_PARAM:**

- args is NULL.
- args->key_descriptor is NULL.
- args->key_descriptor->id is 0.
- Additional invalid parameters returned by the subsystem.

- Other error code from [enum smw_status_code](#)

6.1.3.4.1.2 struct smw_delete_key_args

struct **smw_delete_key_args**

Key deletion arguments

Definition

```
struct smw_delete_key_args {  
    unsigned char version;  
    struct smw_key_descriptor *key_descriptor;  
}
```

Members

version

[in] Version of this structure (must be equal 1).

key_descriptor

[in] Pointer to a Key descriptor object. See [struct smw_key_descriptor](#)

Description

The key_descriptor fields:

- id: [in] Key identifier. Identifier returned during the key creation [smw_generate_key\(\)](#) or [smw_import_key\(\)](#).
- type_name: [in] **Optional**. If defined but not correct, operation returns with SMW_STATUS_INVALID_PARAM. Advice to set it with SMW_KEY_TYPE_NAME_NONE.
- security_size: [in] **Optional**. If defined but not correct, operation returns with SMW_STATUS_INVALID_PARAM. Advice to set it with 0.
- attributes: [in] If key is present in the SMW's database, it's ignored. Otherwise, only key's persistency is considered.
- buffer: Ignored.

6.1.3.5 Key Attributes

6.1.3.5.1 Functions

6.1.3.5.1.1 smw_get_key_attributes

enum [smw_status_code](#) **smw_get_key_attributes**(struct [smw_get_key_attributes_args](#) *args)

Get the key attributes.

Parameters

- **args** (struct [smw_get_key_attributes_args*](#)) - Pointer to the structure that contains the key attributes arguments.

Description

This function gets the Key attributes retrieved for the subsystem owning the given key identifier. If some key attributes are not supported, the output values are empty.

The `args->subsystem_name` field returned is the subsystem name owning the key.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
`args` is NULL.
- Other error code from [enum smw_status_code](#)

6.1.3.5.1.2 struct smw_get_key_attributes_args

struct **smw_get_key_attributes_args**

Get key attributes arguments

Definition

```
struct smw_get_key_attributes_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_key_descriptor *key_descriptor;  
    smw_key_privacy_t key_privacy_name;  
}
```

Members

version

[in] Version of this structure

subsystem_name

[out] Secure Subsystem name. See [typedef smw_subsystem_t](#)

key_descriptor

[in/out] Pointer to a Key descriptor object. See [struct smw_key_descriptor](#)

key_privacy_name

[out] Key privacy name. See [typedef smw_key_privacy_t](#)

Description

The `key_descriptor` fields:

- `id`: [in] key identifier. Identifier returned during the key creation [smw_generate_key\(\)](#) or [smw_import_key\(\)](#).
- `type_name`: [out] Key type name. See [typedef smw_key_type_t](#)
- `security_size`: [out] Security size in bits.
- `attributes`: [out] Key attributes. see [struct smw_key_attributes](#)
- `buffer`: Ignored.

6.1.3.5.1.3 smw_get_key_buffers_lengths

enum [smw_status_code](#) `smw_get_key_buffers_lengths(struct smw\_key\_descriptor *descriptor)`

Gets key buffers lengths.

Parameters

- `descriptor` (struct [smw_key_descriptor](#)*) - [in/out] Pointer to the key descriptor.

Description

Two methods are proposed to get the key buffer lengths:

- Using a valid key identifier, key buffer lengths are key's Secure Subsystem owner value. Only exportable buffer lengths are returned.
- Using a 0's key identifier, key buffer lengths are standard value.

Method 1, the `descriptor` fields:

- `id`: [in] Key identifier. Identifier returned during the key creation [smw_generate_key\(\)](#) or [smw_import_key\(\)](#).
- `type_name`: [in] **Optional**. If defined but not correct, operation returns with `SMW_STATUS_INVALID_PARAM`. Advice to set it with `SMW_KEY_TYPE_NAME_NONE`.
- `security_size`: [in] **Optional**. If defined but not correct, operation returns with `SMW_STATUS_INVALID_PARAM`. Advice to set it with 0.
- `attributes`: [in] If key is present in the SMW's database, it's ignored. Otherwise, only key's persistency is considered.
- `buffer`: [out] Key buffer's length are updated. The `format_name` is ignored, lengths are for hexadecimal key buffer value.

Method 2, the `descriptor` fields:

- `id`: [in] Key identifier must be 0.
- `type_name`: [in] **Mandatory**. Define the key type.
- `security_size`: [in] **Mandatory**. Define the key security size in bits.
- `attributes`: Ignored.
- `buffer`: [out] Key buffer's length are updated. The `format_name` is optional, if set, the lengths are calculated function of the format name.

Return

- `SMW_STATUS_OK`:

Operation succeeded.

- **SMW_STATUS_INVALID_PARAM:**

- descriptor is NULL.
- If `key_descriptor->id` is 0:
 - `descriptor->type_name` is `SMW_KEY_TYPE_NAME_NONE`.
 - `descriptor->security_size` is 0.
- Additional invalid parameters returned by the subsystem.

- Other error code from [enum smw_status_code](#)

6.1.3.5.1.4 smw_get_key_type_name

enum [smw_status_code](#) `smw_get_key_type_name(struct smw_key_descriptor *descriptor)`

Gets the key type name.

Parameters

- `descriptor` (struct [smw_key_descriptor*](#)) - [in/out] Pointer to the key descriptor.

Description

This function gets the Key type name given the key ID.

The `descriptor` fields:

- `id`: [in] Key identifier. Identifier returned during the key creation [smw_generate_key\(\)](#) or [smw_import_key\(\)](#).
- `type_name`: [out] Value updated on success.
- `security_size`: Ignored.
- `attributes`: Ignored.
- `buffer`: Ignored.

Return

- **SMW_STATUS_OK:**

Operation succeeded.

- **SMW_STATUS_INVALID_PARAM:**

Key type name cannot be retrieved.

- Other error code from [enum smw_status_code](#)

6.1.3.5.1.5 smw_get_security_size

enum [smw_status_code](#) `smw_get_security_size(struct smw_key_descriptor *descriptor)`

Gets the key security size.

Parameters

- `descriptor` (struct [smw_key_descriptor*](#)) - [in/out] Pointer to the Key descriptor.

Description

This function gets the Security size given the key ID.

The descriptor fields:

- **id:** [in] Key identifier. Identifier returned during the key creation [smw_generate_key\(\)](#) or [smw_import_key\(\)](#).
- **type_name:** Ignored.
- **security_size:** [out] Value updated on success.
- **attributes:** Ignored.
- **buffer:** Ignored.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
Key security size cannot be retrieved.
- Other error code from [enum smw_status_code](#)

6.1.3.6 Key Storage Protection

6.1.3.6.1 Function

6.1.3.6.1.1 smw_commit_key_storage

enum [smw_status_code](#) **smw_commit_key_storage**(struct [smw_commit_key_storage_args](#) *args)

Commit the active non-volatile key storage

Parameters

- **args** (struct [smw_commit_key_storage_args](#)*) - Pointer to the structure that contains the commit storage arguments.

Description

This function ensures that the non-volatile key storage opened by the subsystem is pushed in physical memory and associated anti-rollback protection counter is incremented.

Warning:

Erasing or replacing the Secure Enclave storage may cause an unrecoverable loss of keys and data. It may also be considered as a security attack and secure subsystem not more accessible.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
args is NULL.
- Other error code from [enum smw_status_code](#)

6.1.3.6.1.2 struct smw_commit_key_storage_args

struct **smw_commit_key_storage_args**

Commit non-volatile key storage arguments

Definition

```
struct smw_commit_key_storage_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
}
```

Members

version

[in] Version of this structure

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#)

6.1.3.7 Key Attestation

6.1.3.7.1 Function

6.1.3.7.1.1 smw_key_attestation

enum [smw_status_code](#) **smw_key_attestation**(struct [smw_key_attestation_args](#) *args)

Get the key attestation certificate.

Parameters

- **args** (struct [smw_key_attestation_args](#)*) - Pointer to the structure that contains the key attestation arguments.

Description

Reads the key attestation certificate.

To query the required certificate buffer length, set args->certificate to NULL. The function will then set the required certificate buffer length in args->certificate_length and return SMW_STATUS_OK.

On operation completion, the args->certificate_length is updated to the correct value when:

- Length is bigger than expected. In this case operation succeeded.
- Length is shorter than expected. In this case, operation fails and returns SMW_STATUS_OUTPUT_TOO_SHORT.

Return

- **SMW_STATUS_OK:**

Operation succeeded.

- **SMW_STATUS_INVALID_PARAM:**

- args is NULL.
- args->key_descriptor is NULL.
- args->key_descriptor->id is 0.
- args->attest_key_descriptor is NULL.
- args->attest_key_descriptor->id is 0.
- args->challenge is NULL and args->certificate is not NULL.
- args->challenge is not NULL and args->challenge_length is 0.

- Other error code from [enum smw_status_code](#)

6.1.3.7.1.2 struct smw_key_attestation_args

struct **smw_key_attestation_args**

Device attestation arguments

Definition

```
struct smw_key_attestation_args {  
    unsigned char version;  
    struct smw_key_descriptor *key_descriptor;  
    struct smw_key_descriptor *attest_key_descriptor;  
    smw_attr_algo_t sign_algo;  
    unsigned char *challenge;  
    unsigned int challenge_length;  
    unsigned char *certificate;  
    unsigned int certificate_length;  
}
```

Members

version

[in] Version of this structure

key_descriptor

[in] Pointer to a Key descriptor object of the key to be attested. See [struct smw_key_descriptor](#)

attest_key_descriptor

[in] Pointer to a Key descriptor object of the attestation key. See [struct smw_key_descriptor](#)

sign_algo

[in] Signature algorithm and attributes. See [typedef smw_attr_algo_t](#)

challenge

[in] Caller unique ephemeral value (e.g. nonce)

challenge_length

[in] Length in bytes of the challenge value

certificate

[out] Device attestation certificate.

certificate_length

[in/out] Length in bytes of the certificate.

Description

The `challenge` length depends on the key (refer to the subsystem capabilities). If the length is bigger than expected, it will be cut to keep only the maximum size. If the length is shorter, the challenge value will be completed with 0's.

The `key_descriptor` and `attest_key_descriptor` fields:

- `id`: [in] Key identifier. Identifier returned during the key creation [smw_generate_key\(\)](#) or [smw_import_key\(\)](#).
- `type_name`: [in] **Optional**. If defined but not correct, operation returns with `SMW_STATUS_INVALID_PARAM`. Advice to set it with `SMW_KEY_TYPE_NAME_NONE`.
- `security_size`: [in] **Optional**. If defined but not correct, operation returns with `SMW_STATUS_INVALID_PARAM`. Advice to set it with 0.
- `attributes`: [in] If key is present in the SMW's database, it's ignored. Otherwise, only key's persistency is considered.
- `buffer`: Ignored.

Note:

Both, key to attest and attestation key, must be owned by the same subsystem supporting the key attestation operation.

6.1.3.8 Key Descriptors**6.1.3.8.1 Structures****6.1.3.8.1.1 struct smw_keypair_gen**

struct **smw_keypair_gen**

Generic Keypair object

Definition

```
struct smw_keypair_gen {  
    unsigned char *public_data;  
    unsigned int public_length;  
    unsigned char *private_data;  
    unsigned int private_length;  
}
```

Members**public_data**

[in/out] Pointer to the public key buffer.

public_length

[in/out] Length in bytes of the public key buffer.

private_data

[in/out] Pointer to the private key buffer.

private_length

[in/out] Length in bytes of the private key buffer.

Description

Generic asymmetric keypair structure used for ECC and Edwards curves.

6.1.3.8.1.2 struct smw_keypair_rsa**struct smw_keypair_rsa**

RSA Keypair object

Definition

```
struct smw_keypair_rsa {  
    unsigned char *public_data;  
    unsigned int public_length;  
    unsigned char *private_data;  
    unsigned int private_length;  
    unsigned char *modulus;  
    unsigned int modulus_length;  
    unsigned char *public_exponent;  
    unsigned int public_exponent_length;  
}
```

Members**public_data**

[in/out] Pointer to the RSA public exponent buffer.

public_length

[in/out] Length in bytes of the RSA public exponent buffer.

private_data

[in/out] Pointer to the RSA private exponent buffer.

private_length

[in/out] Length in bytes of the RSA private exponent buffer.

modulus

[in/out] Pointer to the RSA modulus buffer.

modulus_length

[in/out] Length in bytes of the RSA modulus buffer.

public_exponent

[in] Pointer to the RSA public exponent buffer (key creation only).

public_exponent_length

[in/out] Length in bytes of the RSA public exponent buffer.

Description

Asymmetric keypair structure used for RSA key operations.

First fields are common to the struct `smw_keypair_gen` and must be kept common.

Input parameters `public_exponent` and `public_exponent_length` are only used for key generation. For other operations, they are ignored, and `public_data` and `public_length` must be set instead.

6.1.3.8.1.3 struct smw_keypair_buffer

struct **smw_keypair_buffer**

Keypair buffer

Definition

```
struct smw_keypair_buffer {
    smw_key_format_t format_name;
    union {
        struct smw_keypair_gen gen;
        struct smw_keypair_rsa rsa;
    };
}
```

Members

format_name

[in] Defines the encoding format of all buffers. See [typedef smw_key_format_t](#)

{unnamed_union}

anonymous

gen

Generic keypair object definition. See [struct smw_keypair_gen](#)

rsa

RSA keypair object definition. See [struct smw_keypair_rsa](#)

Description

By default if format name is not specified, it's equivalent to "HEX" format.

6.1.3.8.1.4 struct smw_key_attributes

struct **smw_key_attributes**

Key attributes

Definition

```
struct smw_key_attributes {
    smw_attr_algo_t permitted_algo;
    smw_attr_usage_t usage_flags;
}
```

```
smw_attr_storage_id_t storage_id;  
smw_attr_attributes_t attributes;  
}
```

Members

permitted_algo

[in/out] Permitted algorithm. See [typedef smw_attr_algo_t](#)

usage_flags

[in/out] Permitted usage flags. See [typedef smw_attr_usage_t](#)

storage_id

[in/out] Storage identifier. See [typedef smw_attr_storage_id_t](#)

attributes

[in/out] Attributes. See [typedef smw_attr_attributes_t](#)

Description

Definition of the key attributes members are not all used. It may be function of the subsystem. In case of key creation, it's recommended to define all members correctly even if not used.

6.1.3.8.1.5 struct smw_key_descriptor

struct **smw_key_descriptor**

Key descriptor

Definition

```
struct smw_key_descriptor {  
    smw_key_type_t type_name;  
    unsigned int security_size;  
    unsigned int id;  
    struct smw_key_attributes attributes;  
    struct smw_keypair_buffer *buffer;  
}
```

Members

type_name

[in/out] Key type name. See [typedef smw_key_type_t](#)

security_size

[in/out] Security size in bits.

id

[in/out] Key identifier.

attributes

[in/out] Key attributes. see [struct smw_key_attributes](#)

buffer

[in/out] Pointer to key pair buffer. See [struct smw_keypair_buffer](#)

Description

The `attributes` definition is not used by all APIs. It's documented in API's argument when this field is used.

The `buffer` is optional and may depend on the key operation.

6.1.3.8.1.6 struct smw_derived_key_descriptor**struct smw_derived_key_descriptor**

Derived key descriptor structure

Definition

```
struct smw_derived_key_descriptor {  
    smw_key_type_t type_name;  
    unsigned int security_size;  
    unsigned int id;  
    struct smw_key_attributes attributes;  
    smw_key_format_t format_name;  
    unsigned char *shared_secret;  
    unsigned int shared_secret_len;  
}
```

Members**type_name**

[in] Key type name. See [typedef smw_key_type_t](#)

security_size

[in] Security size in bits.

id

[in/out] Key identifier.

attributes

Key attributes. see [struct smw_key_attributes](#)

format_name

[in] Defines the encoding format of shared secret buffer. See [typedef smw_key_format_t](#)

shared_secret

[in/out] Pointer to shared secret buffer.

shared_secret_len

[in/out] Length in bytes of the shared secret buffer.

6.1.4 Cryptography APIs

The Cryptography APIs allow user of the library to perform cryptographic operations such as hashing, signing, encryption, decryption, MAC and RNG. For some cryptographic operations, both oneshot and multipart modes are supported.

6.1.4.1 Random number generation (RNG)

6.1.4.1.1 Function

6.1.4.1.1.1 smw_rng

enum [smw_status_code](#) **smw_rng**(struct [smw_rng_args](#) *args)

Compute a random number.

Parameters

- **args** (struct [smw_rng_args](#)*) - Pointer to the structure that contains the RNG arguments.

Description

This function computes a random number.

Return

- **SMW_STATUS_OK:**
Random number generated successfully.
- **SMW_STATUS_INVALID_PARAM:**
 - arg is NULL.
 - arg->output is NULL.
 - arg->output_length is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.1.2 Structure

6.1.4.1.2.1 struct smw_rng_args

struct **smw_rng_args**

Random number generator arguments

Definition

```
struct smw_rng_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    unsigned char *output;  
    unsigned int output_length;  
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

output

[out] Pointer to the random number buffer generated.

output_length

[in] Length in bytes of the random number buffer to generate.

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the default configured Secure Subsystem is used.

6.1.4.2 Operation context

The operation context is used in the operation sequence split in multipart calls. The following functions are generic functions regardless the type of cryptographic sequence.

6.1.4.2.1 Functions

6.1.4.2.1.1 smw_allocate_context

enum [smw_status_code](#) **smw_allocate_context**(*struct [smw_context_args](#) *args*)

Allocate SMW operation context.

Parameters

- **args** (struct [smw_context_args](#)*) - Pointer to operation context arguments structure.

Description

This function allocates an operation context for a new cryptographic operation.

This function is intended for use in the multi-part cryptographic operation and it serves as the first step in such operations.

This function does not need to be called for one-shot cryptographic operation.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
`args` is NULL.
- Other error code from [enum smw_status_code](#)

6.1.4.2.1.2 smw_cancel_operation

enum [smw_status_code](#) smw_cancel_operation(struct [smw_context_args](#) *args)

Cancel on-going cryptographic multi-part operation.

Parameters

- **args** (struct [smw_context_args](#)*) - Pointer to operation context arguments structure.

Description

This function cancels the on-going cryptographic multi-part operation and releases all the memory allocated to SMW operation context.

Additionally, this function can be used to release the SMW operation context even if there are no on-going multi-part operations associated with it.

Return

- **SMW_STATUS_OK:**
Operation succeeded and `args->context` is set to NULL.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->context` is NULL.
- Other error code from [enum smw_status_code](#)

6.1.4.2.1.3 smw_copy_context

enum [smw_status_code](#) smw_copy_context(struct [smw_copy_context_args](#) *args)

Copy an operation context.

Parameters

- **args** (struct [smw_copy_context_args](#)*) - Pointer to copy operation context arguments structure.

Description

This function copies the last state of the source operation context to the destination operation context.

Destination context must be allocated using [smw_allocate_context\(\)](#) function prior to invoking this function.

Return

- **SMW_STATUS_OK:**
Operation succeeded and `args->context` is set to NULL.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->src_context` is NULL.
 - `args->dst_context` is NULL.

- Other error code from [enum smw_status_code](#)

6.1.4.2.2 Structures

6.1.4.2.2.1 struct smw_context_args

struct **smw_context_args**

SMW cryptographic operation context arguments

Definition

```
struct smw_context_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_op_context *context;  
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name (**deprecated**). See [typedef smw_subsystem_t](#).

context

[in/out] Pointer to an opaque operation context structure

Description

CAUTION:

The context parameter allocated by SMW should not be modified by the application.

This opaque structure is dynamically allocated by the SMW library upon invoking the function [smw_allocate_context\(\)](#). It is deallocated when any of the following conditions are met:

- Upon successful completion of the associated multi-part final operation.
- In the event of critical failure during the associated operation.
- When [smw_cancel_operation\(\)](#) function is invoked.

CAUTION:

The subsystem_name parameter is deprecated and no more used.

6.1.4.2.2.2 struct smw_copy_context_args

struct **smw_copy_context_args**

SMW cryptographic copy operation context args

Definition

```
struct smw_copy_context_args {  
    unsigned char version;  
    struct smw_op_context *src_context;  
    struct smw_op_context *dst_context;  
}
```

Members

version

[in] Version of this structure.

src_context

[in] Pointer to source opaque operation context structure.

dst_context

[in/out] Pointer to destination opaque operation context structure.

Description

The `dst_context` must be allocated by [smw_allocate_context\(\)](#).

6.1.4.3 Message digests (Hashes)

6.1.4.3.1 Single Part

6.1.4.3.1.1 smw_hash

enum [smw_status_code](#) **smw_hash**(struct [smw_hash_args](#) *args)

Compute hash.

Parameters

- **args** (struct [smw_hash_args](#)*) - Pointer to the structure that contains the Hash arguments.

Description

This function computes a hash.

To query the required digest buffer length, set `args->output` to NULL. The function will then set the required digest buffer length in `args->output_length` and return `SMW_STATUS_OK`. In this condition, the input data can be omitted.

On operation completion, the `args->output_length` is updated to the correct value when:

- Digest buffer length is bigger than expected. In this case, operation succeeds.
- Digest buffer length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - args is NULL.
 - args->input is NULL and args->input_length is not 0.
 - args->input is not NULL and args->input_length is 0.
 - args->output is NULL and args->output_length is not 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.3.2 Multiple Part

6.1.4.3.2.1 smw_hash_init

enum [smw_status_code](#) **smw_hash_init**(struct [smw_hash_init_args](#) *args)

Hash multi-part initialization.

Parameters

- **args** (struct [smw_hash_init_args](#)*) - Pointer to the structure that contains the hash multi-part initialization arguments.

Description

This function executes a hash multi-part initialization.

The operation context must be allocated using [smw_allocate_context\(\)](#) API prior to invoking this API.

The **context** remains valid after this operation if return error code is:

- SMW_STATUS_OK
- SMW_STATUS_INVALID_PARAM
- SMW_STATUS_VERSION_NOT_SUPPORTED
- SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME
- SMW_STATUS_OPERATION_ALREADY_INIT

Otherwise, the **context** is freed and becomes invalid, args->context is set to NULL.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - args is NULL.
 - args->context is NULL.
 - args->input is NULL and args->input_length is not 0.
 - args->input is not NULL and args->input_length is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.3.2.2 smw_hash_update

enum [smw_status_code](#) smw_hash_update(struct [smw_hash_update_args](#) *args)

Hash multi-part update.

Parameters

- **args** (struct [smw_hash_update_args](#)*) - Pointer to the structure that contains the hash multi-part update arguments.

Description

This function executes a hash multi-part update operation.

The context used must be initialized by the hash multi-part initialization [smw_hash_init\(\)](#) API or updated by a previous [smw_hash_update\(\)](#) call.

The **context** remains valid after this operation if return error code is:

- SMW_STATUS_OK
- SMW_STATUS_INVALID_PARAM
- SMW_STATUS_VERSION_NOT_SUPPORTED
- SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME

Otherwise, the **context** is freed and becomes invalid, `args->context` is set to NULL.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - args is NULL.
 - args->context is NULL.
 - args->input is NULL.
 - args->input_length is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.3.2.3 smw_hash_final

enum [smw_status_code](#) smw_hash_final(struct [smw_hash_final_args](#) *args)

Hash multi-part final.

Parameters

- **args** (struct [smw_hash_final_args](#)*) - Pointer to the structure that contains the hash multi-part final arguments.

Description

This function executes a hash multi-part final operation.

The context used must be initialized by the hash multi-part initialization [smw_hash_init\(\)](#) API or updated by a previous [smw_hash_update\(\)](#) call.

Input data field of `args` can be a NULL pointer if no additional data are used.

To query the required digest buffer length, set `args->output` to NULL. The function will then set the required digest buffer length in `args->output_length` and return `SMW_STATUS_OK`. In this condition, the input data can be omitted and the context remains valid.

On operation completion, the `args->output_length` is updated to the correct value when:

- Digest buffer length is bigger than expected. In this case, operation succeeds.
- Digest buffer length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

The **context** remains valid after this operation if return error code is:

- `SMW_STATUS_OK` in case output buffer is set to NULL.
- `SMW_STATUS_OUTPUT_TOO_SHORT`
- `SMW_STATUS_INVALID_PARAM`
- `SMW_STATUS_VERSION_NOT_SUPPORTED`
- `SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME`

Otherwise, the **context** is freed and becomes invalid, `args->context` is set to NULL.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->context` is NULL.
 - `args->input` is not NULL and `args->input_length` is 0.
 - `args->output` is not NULL and `args->output_length` is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.3.3 Structures

6.1.4.3.3.1 struct smw_hash_args

struct **smw_hash_args**

Hash one-shot arguments

Definition

```
struct smw_hash_args {
    unsigned char version;
    smw_subsystem_t subsystem_name;
    smw_hash_algo_t algo_name;
    unsigned char *input;
    unsigned int input_length;
    unsigned char *output;
    unsigned int output_length;
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

algo_name

[in] Algorithm name. See [typedef smw_hash_algo_t](#).

input

[in] Pointer to the input buffer to digest.

input_length

[in] Length in bytes of the input buffer.

output

[out] Pointer to the digest buffer.

output_length

[out] Length in bytes of the digest buffer.

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the default configured Secure Subsystem is used.

6.1.4.3.3.2 struct smw_hash_init_args

struct `smw_hash_init_args`

Hash multi-part initialization arguments

Definition

```
struct smw_hash_init_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    smw_hash_algo_t algo_name;  
    unsigned char *input;  
    unsigned int input_length;  
    struct smw_op_context *context;  
}
```

Members

version

[in] Version of this structure (must be set to 1).

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

algo_name

[in] Algorithm name. See [typedef smw_hash_algo_t](#).

input

[in] Pointer to the input buffer to digest.

input_length

[in] Length in bytes of the input buffer.

context

[in/out] Pointer the multipart operation context.

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the default configured Secure Subsystem is used.

Field `context`:

- Is used only for multi-part operations.
- Must be allocated by [smw_allocate_context\(\)](#).
- Is initialized when initialization operation returned successfully.

6.1.4.3.3 struct smw_hash_update_args

struct **smw_hash_update_args**

Hash multi-part update arguments

Definition

```
struct smw_hash_update_args {  
    unsigned char version;  
    struct smw_op_context *context;  
    unsigned char *input;  
    unsigned int input_length;  
}
```

Members

version

[in] Version of this structure.

context

[in/out] Pointer the multipart operation context updated by [smw_hash_init\(\)](#) or [smw_hash_update\(\)](#).

input

[in] Pointer to the input buffer to digest.

input_length

[in] Length in bytes of the input buffer.

Description

Field `context`:

- Is used only for multi-part operations.
- Is updated when update operation returned successfully.

6.1.4.3.3.4 struct smw_hash_final_args

struct **smw_hash_final_args**

Hash multi-part final arguments

Definition

```
struct smw_hash_final_args {  
    unsigned char version;  
    struct smw_op_context *context;  
    unsigned char *input;  
    unsigned int input_length;  
    unsigned char *output;  
    unsigned int output_length;  
}
```

Members

version

[in] Version of this structure.

context

[in/out] Pointer the multipart operation context updated by [smw_hash_init\(\)](#) or [smw_hash_update\(\)](#).

input

[in] Pointer to the input buffer to digest.

input_length

[in] Length in bytes of the input buffer.

output

[out] Pointer to the digest buffer.

output_length

[out] Length in bytes of the digest buffer.

Description

Field `context`:

- Is used only for multi-part operations.
- Is deleted when final operation returned except under conditions detailed in the [smw_hash_final\(\)](#) function description.

6.1.4.4 Message authentication code (MAC)

6.1.4.4.1 Single Part

6.1.4.4.1.1 smw_mac

enum [smw_status_code](#) **smw_mac**(struct [smw_mac_args](#) *args)

Compute a MAC.

Parameters

- **args** (struct [smw_mac_args](#)*) - Pointer to the structure that contains the MAC arguments.

Description

This function computes a Message Authentication Code using a key present in the Secure Subsystem storage identified by the key_descriptor identifier or a plaintext key value filled in the key descriptor private buffer.

To query the required MAC buffer length, set `args->mac` to NULL. The function will then set the required MAC buffer length in `args->mac_length` and return `SMW_STATUS_OK`.

On operation completion, the `args->mac_length` is updated to the correct value when:

- MAC buffer length is bigger than expected. In this case, operation succeeds.
- MAC buffer length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->key_descriptor` is NULL.
 - `args->input` is NULL and `args->input_length` is not 0.
 - `args->input` is not NULL and `args->input_length` is 0.
 - `args->mac` is NULL and `args->mac_length` is not 0.
 - `args->mac` is not NULL and `args->mac_length` is 0.
 - In case of using plaintext key, private buffer is NULL or length is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.4.1.2 smw_mac_verify

enum [smw_status_code](#) **smw_mac_verify**(struct [smw_mac_args](#) *args)

Compute and verify a MAC.

Parameters

- **args** (struct [smw_mac_args](#)*) - Pointer to the structure that contains the MAC arguments.

Description

This function computes then verifies a Message Authentication Code using a key present in the Secure Subsystem storage identified by the `key_descriptor` or a plaintext key value filled in the key descriptor private buffer.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->key_descriptor` is NULL.
 - `args->input` is NULL and `args->input_length` is not 0.
 - `args->input` is not NULL and `args->input_length` is 0.
 - `args->mac` is NULL and `args->mac_length` is not 0.
 - `args->mac` is not NULL and `args->mac_length` is 0.
 - In case of using plaintext key, private buffer is NULL or length is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.4.2 Structures

6.1.4.4.2.1 struct smw_mac_args

struct **smw_mac_args**

MAC arguments

Definition

```
struct smw_mac_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_key_descriptor *key_descriptor;  
    smw_mac_algo_t algo_name;  
    smw_hash_algo_t hash_name;  
    unsigned char *input;  
    unsigned int input_length;  
    unsigned char *mac;  
    unsigned int mac_length;  
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

key_descriptor

[in] Pointer to a Key descriptor object. See [struct smw_key_descriptor](#).

algo_name

[in] MAC algorithm name. See [typedef smw_mac_algo_t](#).

hash_name

[in] Hash algorithm name. See [typedef smw_hash_algo_t](#).

input

[in] Pointer to the message to be authenticated.

input_length

[in] Length in bytes of the message.

mac

- [in] Pointer to the MAC buffer to compare.
- [out] Pointer to the MAC buffer to be written.

mac_length

- [in] Length in bytes of the MAC buffer to compare.
- [out] Length in bytes of the MAC buffer generated.

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the Secure Subsystem is the default one defined in the library configuration or it's the one handling the key is key identifier is defined.

6.1.4.5 Asymmetric signature**6.1.4.5.1 Single Part****6.1.4.5.1.1 smw_sign**

enum [smw_status_code](#) **smw_sign**(struct [smw_sign_verify_args](#) *args)

Generate a signature.

Parameters

- **args** (struct [smw_sign_verify_args](#)*) - Pointer to the structure that contains the sign arguments.

Description

This function generates a signature using a key present in the Secure Subsystem storage identified by the key descriptor identifier or a plaintext key value filled in the key descriptor private data buffer.

To query the required signature buffer length, set `args->signature` to NULL. The function will then set the required signature buffer length in `args->signature_length` and return `SMW_STATUS_OK`. In this condition, the input message can be omitted.

On operation completion, the `args->signature_length` is updated to the correct value when:

- Signature buffer length is bigger than expected. In this case, operation succeeds.

- Signature buffer length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->key_descriptor` is NULL.
 - `args->message` is NULL.
 - `args->message_length` is 0.
 - `args->signature` is not NULL and `args->signature_length` is 0.
 - Signature algorithm is not an asymmetric signature algorithm.
 - In case of using plaintext key, private buffer is NULL or length is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.5.1.2 smw_verify

enum [smw_status_code](#) **smw_verify**(struct [smw_sign_verify_args](#) *args)

Verify a signature.

Parameters

- **args** (struct [smw_sign_verify_args](#)*) - Pointer to the structure that contains the verify arguments.

Description

This function verifies a signature using a key present in the Secure Subsystem storage identified by the key descriptor identifier or a plaintext key value filled in the key descriptor public data buffer.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->key_descriptor` is NULL.
 - `args->message` is NULL.
 - `args->message_length` is 0.
 - `args->signature` is NULL.
 - `args->signature` is not NULL and `args->signature_length` is 0.
 - Signature algorithm is not an asymmetric signature algorithm.
 - In case of using plaintext key, public buffer is NULL or length is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.5.2 Multiple Part

6.1.4.5.2.1 smw_sign_init

enum [smw_status_code](#) **smw_sign_init**(struct [smw_sign_verify_init_args](#) *args)

Signature generation multi-part initialization.

Parameters

- **args** (struct [smw_sign_verify_init_args](#)*) - Pointer to the structure that contains the signature multi-part initialization arguments.

Description

This function executes a signature generation multi-part initialization using a key present in the Secure Subsystem storage identified by the key descriptor identifier or a plaintext key value filled in the key descriptor private data buffer.

The operation context must be allocated using [smw_allocate_context\(\)](#) API prior to invoking this API.

The **context** remains valid after this operation if return error code is:

- SMW_STATUS_OK
- SMW_STATUS_INVALID_PARAM
- SMW_STATUS_VERSION_NOT_SUPPORTED
- SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME
- SMW_STATUS_OPERATION_ALREADY_INIT
- SMW_STATUS_UNKNOWN_ID
- SMW_STATUS_OPERATION_NOT_SUPPORTED

Otherwise, the **context** is freed and becomes invalid, `args->context` is set to NULL.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->key_descriptor` is NULL.
 - `args->context` is NULL.
 - `args->message` is NULL and `args->message_length` is not 0.
 - `args->message` is not NULL and `args->message_length` is 0.
 - Signature algorithm is not an asymmetric signature algorithm.
 - In case of using plaintext key, private buffer is NULL or length is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.5.2.2 smw_sign_update

enum [smw_status_code](#) **smw_sign_update**(struct [smw_sign_verify_update_args](#) *args)

Signature generation multi-part update.

Parameters

- **args** (struct [smw_sign_verify_update_args*](#)) - Pointer to the structure that contains the signature multi-part update arguments.

Description

This function executes a signature generation multi-part update operation.

The context used must be initialized by the signature generation multi-part initialization [smw_sign_init\(\)](#) API or updated by a previous [smw_sign_update\(\)](#) call.

The **context** remains valid after this operation if return error code is:

- SMW_STATUS_OK
- SMW_STATUS_INVALID_PARAM
- SMW_STATUS_VERSION_NOT_SUPPORTED
- SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME

Otherwise, the **context** is freed and becomes invalid, `args->context` is set to NULL.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->context` is NULL.
 - `args->message` is NULL
 - `args->message_length` is not 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.5.2.3 smw_sign_final

enum [smw_status_code](#) **smw_sign_final**(struct [smw_sign_verify_final_args](#) *args)

Signature generation multi-part final.

Parameters

- **args** (struct [smw_sign_verify_final_args*](#)) - Pointer to the structure that contains the signature multi-part final arguments.

Description

This function executes a signature generation multi-part final operation.

The context used must be initialized by the signature generation multi-part initialization [smw_sign_init\(\)](#) API or updated by a previous [smw_sign_update\(\)](#) call.

Input message field of `args` can be a NULL pointer if no additional data are used.

To query the required signature buffer length, set `args->signature` to NULL. The function will then set the required signature buffer length in `args->signature_length` and return SMW_STATUS_OK. In this condition, the input data can be omitted and the context remains valid.

On operation completion, the `args->signature_length` is updated to the correct value when:

- Signature buffer length is bigger than expected. In this case, operation succeeds.
- Signature buffer length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

The **context** remains valid after this operation if return error code is:

- `SMW_STATUS_OK` in case signature buffer is set to `NULL`.
- `SMW_STATUS_OUTPUT_TOO_SHORT`
- `SMW_STATUS_INVALID_PARAM`
- `SMW_STATUS_VERSION_NOT_SUPPORTED`
- `SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME`

Otherwise, the **context** is freed and becomes invalid, `args->context` is set to `NULL`.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is `NULL`.
 - `args->context` is `NULL`.
 - `args->message` is `NULL` and `args->message_length` is not 0.
 - `args->message` is not `NULL` and `args->message_length` is 0.
 - `args->signature` is not `NULL` and `args->signature_length` is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.5.2.4 smw_verify_init

enum [smw_status_code](#) `smw_verify_init(struct smw\_sign\_verify\_init\_args *args)`

Signature verification multi-part initialization.

Parameters

- **args** (struct [smw_sign_verify_init_args](#)*) - Pointer to the structure that contains the signature multi-part initialization arguments.

Description

This function executes a signature verification multi-part initialization using a key present in the Secure Subsystem storage identified by the key descriptor identifier or a plaintext key value filled in the key descriptor public data buffer.

The operation context must be allocated using [smw_allocate_context\(\)](#) API prior to invoking this API.

The **context** remains valid after this operation if return error code is:

- `SMW_STATUS_OK`
- `SMW_STATUS_INVALID_PARAM`
- `SMW_STATUS_VERSION_NOT_SUPPORTED`
- `SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME`
- `SMW_STATUS_OPERATION_ALREADY_INIT`

- SMW_STATUS_UNKNOWN_ID
- SMW_STATUS_OPERATION_NOT_SUPPORTED

Otherwise, the **context** is freed and becomes invalid, `args->context` is set to NULL.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->key_descriptor` is NULL.
 - `args->context` is NULL.
 - `args->message` is NULL and `args->message_length` is not 0.
 - `args->message` is not NULL and `args->message_length` is 0.
 - Signature algorithm is not an asymmetric signature algorithm.
 - In case of using plaintext key, private buffer is NULL or length is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.5.2.5 smw_verify_update

enum [smw_status_code](#) **smw_verify_update**(struct [smw_sign_verify_update_args](#) *args)

Signature verification multi-part update.

Parameters

- **args** (struct [smw_sign_verify_update_args](#)*) - Pointer to the structure that contains the signature multi-part update arguments.

Description

This function executes a signature verification multi-part update operation.

The context used must be initialized by the signature verification multi-part initialization.

The **context** remains valid after this operation if return error code is:

- SMW_STATUS_OK
- SMW_STATUS_INVALID_PARAM
- SMW_STATUS_VERSION_NOT_SUPPORTED
- SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME

Otherwise, the **context** is freed and becomes invalid, `args->context` is set to NULL.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->context` is NULL.

- `args->message` is NULL
 - `args->message_length` is not 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.5.2.6 smw_verify_final

enum [smw_status_code](#) **smw_verify_final**(struct [smw_sign_verify_final_args](#) *args)

Signature verification multi-part final.

Parameters

- **args** (struct [smw_sign_verify_final_args](#)*) - Pointer to the structure that contains the signature multi-part final arguments.

Description

This function executes a signature verification multi-part final operation.

The context used must be initialized by the signature generation multi-part initialization.

Input message field of `args` can be a NULL pointer if no additional data are used.

Input signature field of `args` must be set with a correct length to perform the signature verification of message given in all multi-part steps.

The **context** remains valid after this operation if return error code is:

- SMW_STATUS_INVALID_PARAM
- SMW_STATUS_VERSION_NOT_SUPPORTED
- SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME

Otherwise, the **context** is freed and becomes invalid, `args->context` is set to NULL.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->context` is NULL.
 - `args->message` is NULL and `args->message_length` is not 0.
 - `args->message` is not NULL and `args->message_length` is 0.
 - `args->signature` is NULL.
 - `args->signature` is not NULL and `args->signature_length` is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.5.3 Structures

6.1.4.5.3.1 struct smw_eddsa_params

struct **smw_eddsa_params**

eddsa signature parameters

Definition

```
struct smw_eddsa_params {  
    unsigned char *context;  
    unsigned int context_length;  
}
```

Members

context

[in] Pointer to the context parameters.

context_length

[in] Length of the context parameters.

6.1.4.5.3.2 struct smw_sm2_params

struct **smw_sm2_params**

SM2 signature parameters

Definition

```
struct smw_sm2_params {  
    unsigned char *identifier;  
    unsigned short identifier_length;  
}
```

Members

identifier

[in] Distinguishable identifier.

identifier_length

[in] Length of the identifier.

6.1.4.5.3.3 struct smw_sign_verify_args

struct **smw_sign_verify_args**

Sign or verify arguments

Definition

```
struct smw_sign_verify_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_key_descriptor *key_descriptor;  
    smw_attr_algo_t sign_algo;
```

```
unsigned char *message;
unsigned int message_length;
unsigned char *signature;
unsigned int signature_length;
union {
    struct smw_eddsa_params *eddsa_params;
    struct smw_sm2_params *sm2_params;
} ;
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

key_descriptor

[in] Pointer to a Key descriptor object. See [struct smw_key_descriptor](#).

sign_algo

[in] Signature algorithm and attributes. See [typedef smw_attr_algo_t](#).

message

[in] Pointer to the message to sign or verify.

message_length

[in] Length in bytes of the message.

signature

- [in] Pointer to the signature buffer to verify.
- [out] Pointer to the signature buffer generated.

signature_length

- [in] Length in bytes of the signature buffer.
- [out] Length in bytes of the signature buffer generated.

{unnamed_union}

anonymous

eddsa_params

[in] (**optional**) Pointer to edwards signature parameters. See [typedef smw_eddsa_params](#).

sm2_params

[in] Pointer to SM2 signature parameters. See [typedef smw_sm2_params](#).

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the Secure Subsystem is the default one defined in the library configuration or it's the one handling the key is key identifier is defined.

6.1.4.5.3.4 struct smw_sign_verify_init_args

struct **smw_sign_verify_init_args**

Sign or verify multi-part initialization arguments

Definition

```
struct smw_sign_verify_init_args {
    unsigned char version;
    smw_subsystem_t subsystem_name;
    struct smw_key_descriptor *key_descriptor;
    smw_attr_algo_t sign_algo;
    unsigned char *message;
    unsigned int message_length;
    union {
        struct smw_eddsa_params *eddsa_params;
        struct smw_sm2_params *sm2_params;
    };
    struct smw_op_context *context;
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

key_descriptor

[in] Pointer to a Key descriptor object. See [struct smw_key_descriptor](#).

sign_algo

[in] Signature algorithm and attributes. See [typedef smw_attr_algo_t](#).

message

[in] Pointer to the message to sign or verify.

message_length

[in] Length in bytes of the message.

{unnamed_union}

anonymous

eddsa_params

[in] (optional) Pointer to edwards signature parameters.

sm2_params

[in] Pointer to SM2 signature parameters.

context

[in/out] Pointer the multipart operation context.

Description

Field `context`:

- Is used only for multi-part operations.
- Must be allocated by [smw_allocate_context\(\)](#).
- Is initialized when initialization operation returned successfully.

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the Secure Subsystem is the default one defined in the library configuration or it's the one handling the key is key identifier is defined.

6.1.4.5.3.5 struct smw_sign_verify_update_args

struct **smw_sign_verify_update_args**

Sign or verify multi-part update arguments

Definition

```
struct smw_sign_verify_update_args {  
    unsigned char version;  
    struct smw_op_context *context;  
    unsigned char *message;  
    unsigned int message_length;  
}
```

Members

version

[in] Version of this structure.

context

[in/out] Pointer the multipart operation context updated by [smw_sign_init\(\)](#) or [smw_sign_update\(\)](#) in case of signature generation, [smw_verify_init\(\)](#) or [smw_verify_update\(\)](#) in case of signature verification.

message

[in] Pointer to the message to sign or verify.

message_length

[in] Length in bytes of the message.

Description

Field `context`:

- Is used only for multi-part operations.
- Is updated when update operation returned successfully.

6.1.4.5.3.6 struct smw_sign_verify_final_args

struct **smw_sign_verify_final_args**

Sign or verify multi-part final arguments

Definition

```
struct smw_sign_verify_final_args {  
    unsigned char version;  
    struct smw_op_context *context;  
    unsigned char *message;  
    unsigned int message_length;  
    unsigned char *signature;  
    unsigned int signature_length;  
}
```

Members

version

[in] Version of this structure.

context

[in/out] Pointer the multipart operation context updated by [smw_sign_init\(\)](#) or [smw_sign_update\(\)](#) in case of signature generation, [smw_verify_init\(\)](#) or [smw_verify_update\(\)](#) in case of signature verification.

message

[in] Pointer to the message to sign or verify.

message_length

[in] Length in bytes of the message.

signature

- [in] Pointer to the signature buffer to verify.
- [out] Pointer to the signature buffer generated.

signature_length

- [in] Length in bytes of the signature buffer.
- [out] Length in bytes of the signature buffer generated.

Description

Field `context`:

- Is used only for multi-part operations.
- Is deleted when final operation returned except under conditions detailed in the [smw_sign_final\(\)](#) and [smw_verify_final\(\)](#) function descriptions.

6.1.4.5.4 Macros

6.1.4.5.4.1 macro `TLS12_MAC_FINISH_DEFAULT_LEN`

`TLS12_MAC_FINISH_DEFAULT_LEN()`

Default TLS 1.2 verify data length for finished message

6.1.4.6 Authentication Encryption/Decryption with associated data (AEAD)

6.1.4.6.1 Single Part

6.1.4.6.1.1 smw_aead

enum [smw_status_code](#) **smw_aead**(struct [smw_aead_args](#) *args)

AEAD encryption or decryption.

Parameters

- **args** (struct [smw_aead_args](#)*) - Pointer to the structure that contains the AEAD one-shot arguments.

Description

This function executes one-shot AEAD encryption or decryption operation using a key present in the Secure Subsystem storage identified by the key descriptor identifier or a plaintext key value filled in the key descriptor private data buffer.

Encryption operation:

- This function encrypts a message and computes the tag.
- If the `args->final->tag` field is set, the computed tag will be stored in the dedicated `args->final->tag` field. The output buffer should be large enough to contain the computed ciphertext.
- If the `args->final->tag` field is not set, `args->final->data->output` field will contain the ciphertext followed by the tag. The output buffer should be large enough to contain concatenated value.

Decryption operation:

- This function authenticates and decrypts the ciphertext.
- If the `args->final->tag` field is not set, the `args->final->data->input` should contain the ciphertext followed by the tag.
- If the computed tag does not match the supplied tag, the operation will be terminated.
- The `args->final->data->output` buffer should be large enough to contain the computed plaintext.

To query the required AEAD encryption or decryption final result output buffer length, set `args->final->data->output` to NULL. The function will then set the required output buffer length in `args->final->data->output_length` and return `SMW_STATUS_OK`. In case of encryption, the `args->tag_length` and `args->output_iv_length` are also set with expected values.

On operation completion, the `args->final->data->output_length` is updated to the correct value when:

- Output length is bigger than expected. In this case, operation succeeds.
- Output length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.
- In the above mentioned two scenarios, if the operation is encryption, the function also updates the required tag buffer length `args->final->tag_length` and output IV length `args->final->output_iv_length`.

Additionally, on encryption operation completion:

- The tag length `args->final->tag_length` is updated with the generated tag value length even if the tag value is concatenated in the output resulting buffer.
- The output IV length `args->final->output_iv` contains the IV used during the operation and the `args->final->output_iv_length` is updated with the IV length.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - args is NULL.
 - args->init is NULL.
 - args->final is NULL.
 - args->final->data is NULL.
 - args->final->data->input is NULL.
 - args->final->data->input_length is 0.
 - args->final->data->output is NULL and args->final->data->output_length is not 0.
 - args->final->output_iv is NULL, when encryption operation.
 - args->init->key_descriptor is NULL.
 - args->init->mode is SMW_AEAD_MODE_NAME_NONE.
 - args->init->op_type_name is SMW_AEAD_OP_TYPE_NAME_NONE.
 - args->init->user_iv is NULL and args->init->user_iv_length is not 0.
 - If tag length is not given in one of init or final structure.
 - If key descriptor is not correctly defined. No key id and no key buffer.
 - In case of using plaintext key, private buffer is NULL or length is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.6.2 Multiple Part

6.1.4.6.2.1 smw_aead_init

enum [smw_status_code](#) **smw_aead_init**(struct [smw_aead_init_args](#) *args)

AEAD multi-part initialization.

Parameters

- **args** (struct [smw_aead_init_args](#)*) - Pointer to the structure that contains the AEAD initialization arguments.

Description

This function initializes AEAD multi-part encryption or decryption operation using a key present in the Secure Subsystem storage identified by the key descriptor identifier or a plaintext key value filled in the key descriptor private data buffer.

The operation context must be allocated using [smw_allocate_context\(\)](#) API prior to invoking this API.

The **context** remains valid after this operation if return error code is:

- SMW_STATUS_OK
- SMW_STATUS_INVALID_PARAM
- SMW_STATUS_VERSION_NOT_SUPPORTED
- SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME
- SMW_STATUS_UNKNOWN_MODE_NAME
- SMW_STATUS_UNKNOWN_OP_TYPE_NAME
- SMW_STATUS_OPERATION_ALREADY_INIT

- SMW_STATUS_UNKNOWN_ID

Otherwise, the **context** is freed and becomes invalid, `args->context` is set to NULL.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->key_descriptor` is NULL.
 - `args->context` is NULL.
 - `args->mode` is SMW_AEAD_MODE_NAME_NONE.
 - `args->op_type_name` is SMW_AEAD_OP_TYPE_NAME_NONE.
 - `args->user_iv` is NULL and `args->user_iv_length` is not 0.
 - `args->plaintext_length` is 0 for the AEAD CCM mode.
 - `args->tag_length` is 0.
 - If key descriptor is not correctly defined. No key id and no key buffer.
 - In case of using plaintext key, private buffer is NULL or length is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.6.2.2 smw_aead_update_aad

enum [smw_status_code](#) smw_aead_update_aad(struct [smw_aead_aad_args](#) *args)

Update AEAD additional authenticated data.

Parameters

- **args** (struct [smw_aead_aad_args](#)*) - Pointer to the structure that contains the AEAD additional data arguments.

Description

This function can be called multiple time while the update data (to encrypt or to decrypt) step is not called.

The context used must be initialized by the AEAD multi-part initialization [smw_aead_init\(\)](#) API or updated by a previous [smw_aead_update_aad\(\)](#) call.

The **context** remains valid after this operation if return error code is:

- SMW_STATUS_OK
- SMW_STATUS_INVALID_PARAM
- SMW_STATUS_VERSION_NOT_SUPPORTED
- SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME

Otherwise, the **context** is freed and becomes invalid, `args->context` is set to NULL.

Return

- **SMW_STATUS_OK:**

Operation succeeded.

- **SMW_STATUS_INVALID_PARAM:**

- args is NULL.
- args->context is NULL.
- args->data is NULL.
- args->data_length is 0.
- Additional invalid parameters returned by the subsystem.

- Other error code from [enum smw_status_code](#)

6.1.4.6.2.3 smw_aead_update

enum [smw_status_code](#) smw_aead_update(struct [smw_aead_data_args](#) *args)

AEAD multi-part update.

Parameters

- **args** (struct [smw_aead_data_args](#)*) - Pointer to the structure that contains the AEAD multi-part data arguments.

Description

This function executes a AEAD multi-part encryption or decryption update operation.

The context used must be initialized by the AEAD multi-part initialization [smw_aead_init\(\)](#) API or updated by a previous [smw_aead_update_aad\(\)](#) or [smw_aead_update\(\)](#) call.

To query the required AEAD encryption or decryption update result output buffer length, set args->output to NULL. The function will then set the required output buffer length in args->output_length and return SMW_STATUS_OK.

On operation completion, the args->output_length is updated to the correct value when:

- Output buffer length is bigger than expected. In this case, operation succeeds.
- Output buffer length is shorter than expected. In this case, operation fails and returns SMW_STATUS_OUTPUT_TOO_SHORT.

The **context** remains valid after this operation if return error code is:

- SMW_STATUS_OK
- SMW_STATUS_INVALID_PARAM
- SMW_STATUS_VERSION_NOT_SUPPORTED
- SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME
- SMW_STATUS_OUTPUT_TOO_SHORT

Otherwise, the **context** is freed and becomes invalid, args->context is set to NULL.

Return

- **SMW_STATUS_OK:**

Operation succeeded.

- **SMW_STATUS_INVALID_PARAM:**

- args is NULL.
- args->context is NULL.
- args->input is NULL.

- `args->input_length` is 0.
 - `args->output` is not NULL and `args->output_length` is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.6.2.4 smw_aead_final

enum [smw_status_code](#) **smw_aead_final**(struct [smw_aead_final_args](#) *args)

AEAD multi-part encryption/decryption final.

Parameters

- **args** (struct [smw_aead_final_args](#)*) - Pointer to the structure that contains the AEAD multi-part final arguments.

Description

This function completes the active AEAD multi-part encryption/decryption operation.

The context used must be initialized by the AEAD multi-part initialization [smw_aead_init\(\)](#) API or updated by a previous [smw_aead_update\(\)](#) call.

Encryption operation:

- This function finalizes the message encryption and computes the tag.
- If the `args->tag` field is set, the computed tag will be stored in the dedicated `args->tag` field. The output buffer should be large enough to contain the computed ciphertext.
- If the `args->tag` field is not set, `args->data->output` field will contain the ciphertext followed by the tag. The output buffer should be large enough to contain concatenated value.

Decryption operation:

- This function finalizes authenticating and decrypting the ciphertext.
- If the `args->tag` field is not set, the `args->data->input` should contain the ciphertext followed by the tag.
- If the computed tag does not match the supplied tag, the operation will be terminated.
- The `args->data->output` buffer should be large enough to contain the computed plaintext.

To query the required AEAD encryption or decryption final result output buffer length, set `args->data->output` to NULL. The function will then set the required output buffer length in `args->data->output_length` and return `SMW_STATUS_OK`. In case of encryption, the `args->tag_length` and `args->output_iv_length` are also set with expected values.

On operation completion, the `args->data->output_length` is updated to the correct value when:

- Output length is bigger than expected. In this case, operation succeeds.
- Output length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.
- In the above mentioned two scenarios, if the operation is encryption, the function also updates the required tag buffer length `args->tag_length` and output IV length `args->output_iv_length`.

Additionally, on encryption operation completion:

- The tag length `args->tag_length` is updated with the generated tag value length even if the tag value is concatenated in the output resulting buffer.
- The output IV length `args->output_iv` contains the IV used during the operation and the `args->output_iv_length` is updated with the IV length.

The **context** remains valid after this operation if return error code is:

- SMW_STATUS_OK in case output buffer is set to NULL.
- SMW_STATUS_INVALID_PARAM
- SMW_STATUS_VERSION_NOT_SUPPORTED
- SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME
- SMW_STATUS_OUTPUT_TOO_SHORT
- SMW_STATUS_UNKNOWN_OP_TYPE_NAME

Otherwise, the **context** is freed and becomes invalid, `args->context` is set to NULL.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->data` is NULL.
 - `args->data->context` is NULL.
 - `args->data->input` is not NULL and `args->data->input_length` is 0.
 - `args->data->output` is not NULL and `args->output_length` is 0.
 - `args->tag_length` is 0.
 - `args->final->output_iv` is NULL, when encryption operation.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.6.3 Structures

6.1.4.6.3.1 struct smw_aead_init_args

struct **smw_aead_init_args**

AEAD initialization arguments

Definition

```
struct smw_aead_init_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_key_descriptor *key_desc;  
    smw_aead_mode_t mode_name;  
    smw_aead_op_type_t op_type_name;  
    unsigned char *user_iv;  
    unsigned int user_iv_length;  
    unsigned int iv_length;  
    unsigned int aad_length;  
    unsigned int tag_length;  
    unsigned int plaintext_length;  
    struct smw_op_context *context;  
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#)

key_desc

[in] Pointer to a key descriptor object. See [struct smw_key_descriptor](#)

mode_name

[in] AEAD mode name. See [typedef smw_aead_mode_t](#)

op_type_name

[in] AEAD operation name. See [typedef smw_aead_op_type_t](#)

user_iv

[in] Pointer to user initialization vector.

user_iv_length

[in] Length in bytes of the user IV buffer.

iv_length

[in] Length in bytes of the requested IV buffer.

aad_length

[in] Length in bytes of the additional authentication data.

tag_length

[in] Length in bytes of the tag.

plaintext_length

[in] Length in bytes of the data to encrypt.

context

[in/out] Pointer the multipart operation context.

Description

Field `context`:

- Is used only for multi-part operations.
- Must be allocated by [smw_allocate_context\(\)](#).
- Is initialized when initialization operation returned successfully.

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the Secure Subsystem is the default one defined in the library configuration or it's the one handling the key is key identifier is defined.

If subsystem offers the capability to generate partial or complete IV:

- To request the full generation of the IV, the `user_iv_length` must be 0.
- To request the partial generation of the IV, the `user_iv_length` must be less than subsystem IV maximum length (usually 12 bytes) and `iv_length` must be the requested length to be generated by the subsystem.

Otherwise, if `user_iv_length` is set to the maximum IV size supported by the subsystem, `iv_length` is not taking into consideration. Please refer to the AEAD description of the [Secure Subsystems Features and Capabilities](#).

6.1.4.6.3.2 struct smw_aead_data_args

struct **smw_aead_data_args**

AEAD data arguments

Definition

```
struct smw_aead_data_args {  
    unsigned char version;  
    struct smw_op_context *context;  
    unsigned char *input;  
    unsigned int input_length;  
    unsigned char *output;  
    unsigned int output_length;  
}
```

Members

version

[in] Version of this structure.

context

[in/out] Pointer the multipart operation context updated by [smw_aead_init\(\)](#), [smw_aead_update\(\)](#) or [smw_aead_update_aad\(\)](#).

input

[in] Pointer to input data buffer to be encrypted or decrypted

input_length

[in] Length in bytes of the input data buffer.

output

[out] Pointer to output buffer.

output_length

[out] Length in bytes of the output buffer.

Description

Field `context`:

- Is used only for multi-part operations.
- Is updated when update operation returned successfully.
- Is deleted when final operation returned except under conditions detailed in the [smw_aead_final\(\)](#) function description.

6.1.4.6.3.3 struct smw_aead_aad_args

struct **smw_aead_aad_args**

Authentication Encryption AAD arguments

Definition

```
struct smw_aead_aad_args {  
    unsigned char version;  
    unsigned char *data;  
    unsigned int data_length;  
    struct smw_op_context *context;  
}
```

Members

version

[in] Version of this structure.

data

[in] Pointer to additional authentication data.

data_length

[in] Length in bytes of the additional authentication data.

context

[in/out] Pointer the multipart operation context updated by [smw_aead_init\(\)](#), [smw_aead_update\(\)](#) or [smw_aead_update_aad\(\)](#).

Description

Field `context`:

- Is used only for multi-part operations.
- Is updated when update operation returned successfully.

6.1.4.6.3.4 struct smw_aead_final_args

struct **smw_aead_final_args**

AEAD final arguments

Definition

```
struct smw_aead_final_args {  
    unsigned char version;  
    struct smw_aead_data_args *data;  
    smw_aead_op_type_t op_type_name;  
    unsigned char *tag;  
    unsigned int tag_length;  
    unsigned int output_iv_length;  
    unsigned char *output_iv;  
}
```

```
}
```

Members

version

[in] Version of this structure.

data

[in] Pointer to AEAD data arguments. See [struct smw_aead_data_args](#)

op_type_name

[in] AEAD operation name. See [typedef smw_aead_op_type_t](#)

tag

- [in] Pointer to tag buffer to be compared.
- [out] Pointer to tag buffer generated.

tag_length

- [in] Length in bytes of the tag buffer.
- [out] Length in bytes of the tag buffer generated.

output_iv_length

- [in] Length in bytes of output IV buffer.
- [out] Length in bytes of output IV generated.

output_iv

[out] Pointer to output IV buffer

Description

The `output_iv` buffer should be allocated by the caller application for encryption operation. Upon successful encryption operation, `output_iv` will contain the IV used by subsystem during operation.

Fields `output_iv_length` and `output_iv` are ignored for decryption operation.

6.1.4.6.3.5 struct smw_aead_args

struct **smw_aead_args**

AEAD one-shot arguments

Definition

```
struct smw_aead_args {  
    unsigned char version;  
    struct smw_aead_aad_args *aad;  
    struct smw_aead_init_args *init;  
    struct smw_aead_final_args *final;  
}
```

Members

version

[in] Version of this structure.

aad

[in] Pointer to AAD arguments. See [struct smw_aead_aad_args](#)

init

[in/out] Pointer to initialization arguments. See [struct smw_aead_init_args](#)

final

[in/out] Pointer to final arguments. See [struct smw_aead_final_args](#)

Description

This structure is defined with the multi-part operation but:

- Field `context` present in `init`, `aad` and `final` is ignored.
- Field `op_type_name` present in `final` is ignored, only the one in `init` is used.

6.1.4.6.4 Examples

6.1.4.6.4.1 Example 1: AEAD one-shot encryption operation

```
#define IV_LEN 12
#define AAD_LEN 20
#define DATA_LEN 32
#define CIPHER_LEN 32
#define TAG_LEN 16

int main(int argc, char *argv[])
{
    int res = SMW_STATUS_OPERATION_FAILURE;
    unsigned char iv[IV_LEN] = {...};
    unsigned char aad[AAD_LEN] = {...};
    unsigned char input[DATA_LEN] = {...};
    unsigned char output[CIPHER_LEN + TAG_LEN] = {0};
    unsigned char output_iv[IV_LEN] = {0};

    struct smw_aead_args args = {0};
    struct smw_aead_init_args init_args = {0};
    struct smw_aead_final_args final_args = {0};
    struct smw_aead_data_args data_args = {0};
    struct smw_aead_aad_args aad_args = {0};
    struct smw_key_descriptor key_desc = {0};

    init_args.subsystem_name = SMW_SUBSYSTEM_NAME_TEE;
    init_args.op_type_name = SMW_AEAD_OP_TYPE_NAME_ENCRYPT;
    init_args.mode_name = SMW_AEAD_MODE_NAME_GCM;
    init_args.plaintext_length = DATA_LEN;
    init_args.user_iv = iv;
    init_args.user_iv_length = IV_LEN;
    init_args.iv_length = IV_LEN;
    init_args.key_desc = &key_desc;

    data_args.input_length = DATA_LEN;
    data_args.input = input;
    data_args.output_length = CIPHER_LEN + TAG_LEN;
    data_args.output = output;
```

```

final_args.data = &data_args;
final_args.tag_length = TAG_LEN;
final_args.output_iv_length = IV_LEN;
final_args.output_iv = output_iv;

aad_args.data = aad;
aad_args.data_length = AAD_LEN;

// One-shot AE encryption operation
args.init = &init_args;
args.final = &final_args;
args.aad = &aad_args;

res = smw_aead(&args);
return res;
}

```

6.1.4.6.4.2 Example 2: AEAD one-shot encryption operation (Tag stored in tag field)

```

#define IV_LEN 12
#define AAD_LEN 20
#define DATA_LEN 32
#define CIPHER_LEN 32
#define TAG_LEN 16

int main(int argc, char *argv[])
{
    int res = SMW_STATUS_OPERATION_FAILURE;
    unsigned char iv[IV_LEN] = {...};
    unsigned char aad[AAD_LEN] = {...};
    unsigned char input[DATA_LEN] = {...};
    unsigned char output[CIPHER_LEN] = {0};
    unsigned char tag[TAG_LEN] = {0};
    unsigned char output_iv[IV_LEN] = {0};

    struct smw_aead_args args = {0};
    struct smw_aead_init_args init_args = {0};
    struct smw_aead_final_args final_args = {0};
    struct smw_aead_data_args data_args = {0};
    struct smw_aead_aad_args aad_args = {0};
    struct smw_key_descriptor key_desc = {0};

    init_args.subsystem_name = SMW_SUBSYSTEM_NAME_TEE;
    init_args.op_type_name = SMW_AEAD_OP_TYPE_NAME_ENCRYPT;
    init_args.mode_name = SMW_AEAD_MODE_NAME_GCM;
    init_args.plaintext_length = DATA_LEN;
    init_args.user_iv = iv;
    init_args.user_iv_length = IV_LEN;
    init_args.iv_length = IV_LEN;
    init_args.key_desc = &key_desc;

    data_args.input_length = DATA_LEN;
    data_args.input = input;
    data_args.output_length = CIPHER_LEN;
    data_args.output = output;

    final_args.data = &data_args;

```

```

    final_args.tag_length = TAG_LEN;
    final_args.tag = tag;
    final_args.output_iv_length = IV_LEN;
    final_args.output_iv = output_iv;

    aad_args.data = aad;
    aad_args.data_length = AAD_LEN;

    // One-shot AE encryption operation
    args.init = &init_args;
    args.final = &final_args;
    args.aad = &aad_args;

    res = smw_aead(&args);
    return res;
}

```

6.1.4.6.4.3 Example 3: AEAD one-shot decryption operation (Tag stored in tag field)

```

#define IV_LEN 12
#define AAD_LEN 20
#define DATA_LEN 32
#define CIPHER_LEN 32
#define TAG_LEN 16

int main(int argc, char *argv[])
{
    int res = SMW_STATUS_OPERATION_FAILURE;
    unsigned char iv[IV_LEN] = {...};
    unsigned char aad[AAD_LEN] = {...};
    unsigned char input[DATA_LEN] = {...};
    unsigned char output[CIPHER_LEN] = {0};
    unsigned char tag[TAG_LEN] = {...};

    struct smw_aead_args args = {0};
    struct smw_aead_init_args init_args = {0};
    struct smw_aead_final_args final_args = {0};
    struct smw_aead_data_args data_args = {0};
    struct smw_aead_aad_args aad_args = {0};
    struct smw_key_descriptor key_desc = {0};

    init_args.subsystem_name = SMW_SUBSYSTEM_NAME_TEE;
    init_args.op_type_name = SMW_AEAD_OP_TYPE_NAME_DECRYPT;
    init_args.mode_name = SMW_AEAD_MODE_NAME_GCM;
    init_args.plaintext_length = DATA_LEN;
    init_args.user_iv = iv;
    init_args.user_iv_length = IV_LEN;
    init_args.iv_length = IV_LEN;
    init_args.key_desc = &key_desc;

    data_args.input_length = CIPHER_LEN;
    data_args.input = input;
    data_args.output_length = DATA_LEN;
    data_args.output = output;

    final_args.data = &data_args;
    final_args.tag_length = TAG_LEN;
    final_args.tag = tag;
}

```

```

    aad_args.data = aad;
    aad_args.data_length = AAD_LEN;

    // One-shot AE encryption operation
    args.init = &init_args;
    args.final = &final_args;
    args.aad = &aad_args;

    res = smw_aead(&args);
    return res;
}

```

6.1.4.6.4.4 Example 4: AEAD multi-part encryption operation

```

#define IV_LEN 12
#define AAD_LEN 20
#define DATA_LEN 32
#define TAG_LEN 16
#define CIPHER_LEN 32

int main(int argc, char *argv[])
{
    int res = SMW_STATUS_OPERATION_FAILURE;

    unsigned char iv[IV_LEN] = {...};
    unsigned char aad[AAD_LEN] = {...};
    unsigned char input[DATA_LEN] = {...};
    unsigned char output[CIPHER_LEN + TAG_LEN] = {0};

    struct smw_aead_data_args data_args = {0};
    struct smw_aead_aad_args aad_args = {0};
    struct smw_aead_final_args final_args = {0};
    struct smw_aead_init_args init_args = {0};
    struct smw_key_descriptor key_desc = {0};
    struct smw_context_args op_ctx = {0};

    init_args.subsystem_name = SMW_SUBSYSTEM_NAME_TEE;
    init_args.op_type_name = SMW_AEAD_OP_TYPE_NAME_ENCRYPT;
    init_args.mode_name = SMW_AEAD_MODE_NAME_GCM;
    init_args.aad_length = AAD_LEN;
    init_args.tag_length = TAG_LEN;
    init_args.plaintext_length = DATA_LEN;
    init_args.user_iv = iv;
    init_args.user_iv_length = IV_LEN;
    init_args.iv_length = IV_LEN;

    // Allocate memory to operation context
    res = smw_allocate_context(&op_ctx);
    if (res != SMW_STATUS_OK)
        goto exit;

    init_args.context = op_ctx.context;
    init_args.key_desc = &key_desc;

    // Initialize multi-part AEAD operation
    res = smw_aead_init(&init_args);
}

```

```
if (res != SMW_STATUS_OK)
    goto exit;

// Add additional data to an active AEAD operation.
aad_args.data = aad;
aad_args.data_length = AAD_LEN;
aad_args.context = init_args.context;
res = smw_aead_update_aad(&aad_args);
if (res != SMW_STATUS_OK)
    goto exit;

/*
 * Encrypt 1st message fragment in an active
 * multi-part AEAD encryption operation.
 */
data_args.input_length = 16;
data_args.input = input;
data_args.output_length = 16;
data_args.output = output;
data_args.context = init_args.context;
res = smw_aead_update(&data_args);
if (res != SMW_STATUS_OK)
    goto exit;

/*
 * Encrypt 2nd message fragment in an active
 * multi-part AEAD encryption operation.
 */
data_args.input_length = 16;
data_args.input = &input[16];
data_args.output_length = 16;
data_args.output = &output[16];
data_args.context = init_args.context;
res = smw_aead_update(&data_args);
if (res != SMW_STATUS_OK)
    goto exit;

/*
 * Finish encrypting the message in an active
 * multi-part AEAD operation.
 */
final_args.op_type_name = SMW_AEAD_OP_TYPE_NAME_ENCRYPT;
final_args.data->context = init_args.context;
final_args.data->input = NULL;
final_args.data->input_length = 0;
final_args.data->output = &output[32];
final_args.data->output_length = TAG_LEN;
final_args.tag_length = TAG_LEN;
final_args.output_iv_length = IV_LEN;
final_args.output_iv = output_iv;

res = smw_aead_final(&final_args);

exit:
return res;
}
```

6.1.4.6.4.5 Example 5: AEAD multi-part decryption operation

```

#define IV_LEN 12
#define AAD_LEN 20
#define DATA_LEN 32
#define TAG_LEN 16
#define CIPHER_LEN 32

int main(int argc, char *argv[])
{
    int res = SMW_STATUS_OPERATION_FAILURE;

    unsigned char iv[IV_LEN] = {...};
    unsigned char aad[AAD_LEN] = {...};
    unsigned char input[CIPHER_LEN + TAG_LEN] = {...};
    unsigned char output[DATA_LEN] = {0};

    struct smw_aead_data_args data_args = {0};
    struct smw_aead_aad_args aad_args = {0};
    struct smw_aead_final_args final_args = {0};
    struct smw_aead_init_args init_args = {0};
    struct smw_key_descriptor key_desc = {0};
    struct smw_context_args op_ctx = {0};

    init_args.subsystem_name = SMW_SUBSYSTEM_NAME_TEE;
    init_args.op_type_name = SMW_AEAD_OP_TYPE_NAME_DECRYPT;
    init_args.mode_name = SMW_AEAD_MODE_NAME_GCM;
    init_args.aad_length = AAD_LEN;
    init_args.tag_length = TAG_LEN;
    init_args.plaintext_length = DATA_LEN;
    init_args.user_iv = iv;
    init_args.user_iv_length = IV_LEN;
    init_args.iv_length = IV_LEN;

    // Allocate memory to operation context
    res = smw_allocate_context(&op_ctx);
    if (res != SMW_STATUS_OK)
        goto exit;

    init_args.context = op_ctx.context;
    init_args.key_desc = &key_desc;

    // Initialize multi-part AEAD operation
    res = smw_aead_init(&init_args);
    if (res != SMW_STATUS_OK)
        goto exit;

    // Add additional data to an active AEAD operation.
    aad_args.data = aad;
    aad_args.data_length = AAD_LEN;
    aad_args.context = init_args.context;
    res = smw_aead_update_aad(&aad_args);
    if (res != SMW_STATUS_OK)
        goto exit;

    /*
     * Decrypt 1st message fragment in an active
     * multi-part AEAD decryption operation.

```

```

    */
    data_args.input_length = 16;
    data_args.input = input;
    data_args.output_length = 16;
    data_args.output = output;
    data_args.context = init_args.context;
    res = smw_aead_update(&data_args);
    if (res != SMW_STATUS_OK)
        goto exit;

    /*
     * Decrypt 2nd message fragment in an active
     * multi-part AEAD decryption operation.
     */
    data_args.input_length = 16;
    data_args.input = &input[16];
    data_args.output_length = 16;
    data_args.output = &output[16];
    data_args.context = init_args.context;
    res = smw_aead_update(&data_args);
    if (res != SMW_STATUS_OK)
        goto exit;

    /*
     * Finish authenticating and decrypting the message
     * in an active multi-part AEAD operation.
     */
    final_args.op_type_name = SMW_AEAD_OP_TYPE_NAME_DECRYPT;
    final_args.data.context = init_args.context;
    // Pass the tag
    final_args.data->input = &input[32];
    final_args.data->input_length = TAG_LEN;
    final_args.data->output = NULL;
    final_args.data->output_length = 0;
    final_args.tag_length = TAG_LEN;
    res = smw_aead_final(&final_args);

exit:
    return res;
}

```

6.1.4.7 Asymmetric Encryption/Decryption

6.1.4.7.1 Single Part

6.1.4.7.1.1 smw_asymmetric_encrypt

enum [smw_status_code](#) **smw_asymmetric_encrypt**(struct [smw_asymmetric_encryption_args](#) *args)

Encrypt a message.

Parameters

- **args** (struct [smw_asymmetric_encryption_args](#)*) - Pointer to asymmetric encryption and decryption arguments structure.

Description

This function encrypts a message using a RSA key present in the Secure Subsystem storage identified by the key descriptor identifier or a plaintext key value filled in the key descriptor RSA public buffers; modulus and public data.

To query the required asymmetric encryption result output buffer length, set `args->output` to `NULL`. The function will then set the required output buffer length in `args->output_length` and return `SMW_STATUS_OK`.

On operation completion, the `args->output_length` is updated to the correct value when:

- Output length is bigger than expected. In this case, operation succeeds.
- Output length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is `NULL`.
 - `args->key_descriptor` is `NULL`.
 - `args->input` is `NULL`.
 - `args->input_length` is 0.
 - `args->output` is not `NULL` and `args->output_length` is 0.
 - `args->salt` is `NULL` and `args->salt_length` is not 0, OEAP encryption.
 - `args->salt` is not `NULL` and `args->salt_length` is 0, OEAP encryption.
 - If key descriptor is not correctly defined. No key id and no key buffer or not a RSA key type.
 - In case of using plaintext key, modulus and public buffer are `NULL` or corresponding length are 0.
 - Signature algorithm is not an asymmetric encryption algorithm.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.7.1.2 smw_asymmetric_decrypt

enum [smw_status_code](#) **smw_asymmetric_decrypt**(struct [smw_asymmetric_encryption_args](#) *args)

Decrypt a message

Parameters

- **args** (struct [smw_asymmetric_encryption_args](#)*) - Pointer to asymmetric encryption and decryption arguments structure.

Description

This function decrypts a message using a RSA key present in the Secure Subsystem storage identified by the key descriptor identifier or a plaintext key value filled in the key descriptor RSA private buffers; modulus and private data.

On operation completion, the `args->output_length` is updated to the correct value when:

- Output length is bigger than expected. In this case, operation succeeds.

- Output length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->key_descriptor` is NULL.
 - `args->input` is NULL.
 - `args->input_length` is 0.
 - `args->output` is NULL.
 - `args->output` is not NULL and `args->output_length` is 0.
 - `args->salt` is NULL and `args->salt_length` is not 0, OEAP encryption.
 - `args->salt` is not NULL and `args->salt_length` is 0, OEAP encryption.
 - If key descriptor is not correctly defined. No key id and no key buffer or not a RSA key type.
 - In case of using plaintext key, modulus and private buffer are NULL or corresponding length are 0.
 - Signature algorithm is not an asymmetric encryption algorithm.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.7.2 Structures

6.1.4.7.2.1 struct smw_asymmetric_encryption_args

struct **smw_asymmetric_encryption_args**

Asymmetric encryption and decryption arguments structure

Definition

```
struct smw_asymmetric_encryption_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_key_descriptor *key_descriptor;  
    smw_attr_algo_t algo;  
    unsigned char *input;  
    unsigned int input_length;  
    unsigned char *output;  
    unsigned int output_length;  
    unsigned char *salt;  
    unsigned int salt_length;  
}
```

Members

version

[in] Version of this structure

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

key_descriptor

[in] Pointer to a Key descriptor object. See [struct smw_key_descriptor](#).

algo

[in] Encryption/decryption algorithm and attributes. See [typedef smw_attr_algo_t](#).

input

[in] Pointer to input data buffer.

input_length

[in] Length in bytes of the input data buffer.

output

[out] Pointer to output buffer

output_length

[out] Length in bytes of the output buffer.

salt

[in] (**optional**) Pointer to salt or label buffer

salt_length

[in] (**optional**) Length in bytes of the salt buffer length.

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the Secure Subsystem is the default one defined in the library configuration or it's the one handling the key is key identifier is defined.

6.1.4.8 Symmetric Encryption (Cipher)

6.1.4.8.1 Single Part

6.1.4.8.1.1 smw_cipher

enum [smw_status_code](#) **smw_cipher**(struct [smw_cipher_args](#) *args)

Cipher encryption or decryption.

Parameters

- **args** (struct [smw_cipher_args](#)*) - Pointer to the structure that contains the cipher arguments.

Description

This function executes a cipher encryption or decryption using a key present in the Secure Subsystem storage identified by the key descriptor identifier or a plaintext key value filled in the key descriptor private data buffer.

To query the required output buffer length, set `args->data.output` to NULL. The function will then set the required output buffer length in `args->data.output_length` and return `SMW_STATUS_OK`.

On operation completion, the `args->data.output_length` is updated to the correct value when:

- Output buffer length is bigger than expected. In this case, operation succeeds.
- Output buffer length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->data.input` is NULL.
 - `args->data.input_length` is 0.
 - `args->data.output` is not NULL and `args->data.output_length` is 0.
 - `args->init.key_desc` is NULL.
 - `args->init.nb_keys` is 0.
 - `args->init.nb_keys` is not equal to 1 in case of mode other than XTS.
 - `args->init.nb_keys` is not equal to 2 in case of XTS mode.
 - In case of multiple keys, key type are not identical, keys are not owned by the same Secure Subsystem.
 - If key descriptor is not correctly defined. No key id and no key buffer.
 - `args->init.mode_name` is `SMW_CIPHER_MODE_NAME_NONE`.
 - `args->init.op_type_name` is `SMW_CIPHER_OP_TYPE_NAME_NONE`.
 - `args->init.iv` is NULL in case of mode requiring an IV/counter/tweak.
 - `args->init.iv_length` is 0 in case of mode requiring an IV/counter/tweak.
 - In case of using plaintext key(s), private buffer is NULL or length is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.8.2 Multiple Part

6.1.4.8.2.1 smw_cipher_init

enum [smw_status_code](#) `smw_cipher_init(struct smw\_cipher\_init\_args *args)`

Cipher multi-part initialization.

Parameters

- **args** (struct [smw_cipher_init_args](#)*) - Pointer to the structure that contains the cipher multi-part initialization arguments.

Description

This function executes a cipher multi-part encryption or decryption initialization using a key present in the Secure Subsystem storage identified by the key descriptor identifier or a plaintext key value filled in the key descriptor private data buffer.

The operation context must be allocated using [smw_allocate_context\(\)](#) API prior to invoking this API.

The **context** remains valid after this operation if return error code is:

- `SMW_STATUS_OK`
- `SMW_STATUS_INVALID_PARAM`
- `SMW_STATUS_VERSION_NOT_SUPPORTED`

- SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME
- SMW_STATUS_UNKNOWN_MODE_NAME
- SMW_STATUS_UNKNOWN_OP_TYPE_NAME
- SMW_STATUS_OPERATION_ALREADY_INIT
- SMW_STATUS_UNKNOWN_ID
- SMW_STATUS_OPERATION_NOT_SUPPORTED

Otherwise, the **context** is freed and becomes invalid, `args->context` is set to NULL.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->context` is NULL.
 - `args->key_desc` is NULL.
 - `args->nb_keys` is 0.
 - `args->nb_keys` is not equal to 1 in case of mode other than XTS.
 - `args->nb_keys` is not equal to 2 in case of XTS mode.
 - In case of multiple keys, key type are not identical, keys are not owned by the same Secure Subsystem.
 - If key descriptor is not correctly defined. No key id and no key buffer.
 - `args->mode_name` is SMW_CIPHER_MODE_NAME_NONE.
 - `args->op_type_name` is SMW_CIPHER_OP_TYPE_NAME_NONE.
 - `args->iv` is NULL in case of mode requiring an IV/counter/tweak.
 - `args->iv_length` is 0 in case of mode requiring an IV/counter/tweak.
 - In case of using plaintext key(s), private buffer is NULL or length is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.8.2.2 smw_cipher_update

enum [smw_status_code](#) **smw_cipher_update**(struct [smw_cipher_data_args](#) *args)

Cipher multi-part update.

Parameters

- **args** (struct [smw_cipher_data_args](#)*) - Pointer to the structure that contains the cipher multi-part update arguments.

Description

This function executes a cipher multi-part encryption or decryption update operation.

The context used must be initialized by the cipher multi-part initialization [smw_cipher_init\(\)](#) API or updated by a previous [smw_cipher_update\(\)](#) call.

To query the required output buffer length, set `args->output` to NULL. The function will then set the required output buffer length in `args->output_length` and return SMW_STATUS_OK.

On operation completion, the `args->output_length` is updated to the correct value when:

- Output buffer length is bigger than expected. In this case, operation succeeds.

- Output buffer length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

The **context** remains valid after this operation if return error code is:

- `SMW_STATUS_OK`
- `SMW_STATUS_INVALID_PARAM`
- `SMW_STATUS_VERSION_NOT_SUPPORTED`
- `SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME`
- `SMW_STATUS_OUTPUT_TOO_SHORT`

Otherwise, the **context** is freed and becomes invalid, `args->context` is set to `NULL`.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is `NULL`.
 - `args->context` is `NULL`.
 - `args->input` is `NULL`.
 - `args->input_length` is `NULL`.
 - `args->output` is not `NULL` and `args->output_length` is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.8.2.3 smw_cipher_final

enum [smw_status_code](#) `smw_cipher_final(struct smw\_cipher\_data\_args *args)`

Cipher multi-part final.

Parameters

- **args** (struct [smw_cipher_data_args](#)*) - Pointer to the structure that contains the cipher multi-part final arguments.

Description

This function executes a cipher multi-part encryption or decryption final operation.

The context used must be initialized by the cipher multi-part initialization [smw_cipher_init\(\)](#) API or updated by a previous [smw_cipher_update\(\)](#) call.

Input data field of `args` can be a `NULL` pointer if no additional data are used.

To query the required output buffer length, set `args->output` to `NULL`. The function will then set the required output buffer length in `args->output_length` and return `SMW_STATUS_OK`.

On operation completion, the `args->output_length` is updated to the correct value when:

- Output buffer length is bigger than expected. In this case, operation succeeds.
- Output buffer length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

The **context** remains valid after this operation if return error code is:

- SMW_STATUS_OK in case output buffer is set to NULL.
- SMW_STATUS_INVALID_PARAM
- SMW_STATUS_VERSION_NOT_SUPPORTED
- SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME
- SMW_STATUS_OUTPUT_TOO_SHORT

Otherwise, the **context** is freed and becomes invalid, `args->context` is set to NULL.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->context` is NULL.
 - `args->input` is not NULL and `args->input_length` is 0.
 - `args->output` is not NULL and `args->output_length` is 0.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.4.8.3 Structures

6.1.4.8.3.1 struct smw_cipher_init_args

struct **smw_cipher_init_args**

Cipher multi-part initialization arguments

Definition

```
struct smw_cipher_init_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_key_descriptor **keys_desc;  
    unsigned int nb_keys;  
    smw_cipher_mode_t mode_name;  
    smw_cipher_op_type_t op_type_name;  
    unsigned char *iv;  
    unsigned int iv_length;  
    struct smw_op_context *context;  
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

keys_desc

[in] Pointer to an array of pointers to key descriptors. See [struct smw_key_descriptor](#).

nb_keys

[in] Number of entries of the key descriptors array `keys_desc`.

mode_name

[in] Cipher mode name. See [typedef smw_cipher_mode_t](#).

op_type_name

[in] Cipher operation type name. See [typedef smw_cipher_op_type_t](#).

iv

[in] Pointer to initialization vector. Not used in ECB mode.

iv_length

[in] Length in bytes of the initialization vector. Not used in ECB mode.

context

[in/out] Pointer the multipart operation context.

Description

Field `context`:

- Is used only for multi-part operations.
- Must be allocated by [smw_allocate_context\(\)](#).
- Is initialized when initialization operation returned successfully.

The number of keys `nb_keys` is usually 1 key, but for the XTS mode, 2 keys are required. In the case of multi-key cipher operation, the keys must be owned by the same Secure Subsystem and must be the same key type.

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the Secure Subsystem is the default one defined in the library configuration or it's the one handling the key is key identifier is defined.

6.1.4.8.3.2 struct smw_cipher_data_args

struct **smw_cipher_data_args**

Cipher data arguments

Definition

```
struct smw_cipher_data_args {
    unsigned char version;
    struct smw_op_context *context;
    unsigned char *input;
    unsigned int input_length;
    unsigned char *output;
    unsigned int output_length;
}
```

Members

version

[in] Version of this structure.

context

[in/out] Pointer the multipart operation context updated by [smw_cipher_init\(\)](#) or [smw_cipher_update\(\)](#).

input

[in] Pointer to the input buffer to encrypt or decrypt.

input_length

[in] Length in bytes of the input buffer.

output

[out] Pointer to the output buffer decrypted or encrypted.

output_length

[out] Length in bytes of the output buffer.

Description

Field `context`:

- Is used only for multi-part operations.
- Is updated when update operation returned successfully.
- Is deleted when final operation returned except under conditions detailed in the [smw_cipher_final\(\)](#) function description.

In case of final operation, `input` and `input_length` are optional.

6.1.4.8.3.3 struct smw_cipher_args

struct **smw_cipher_args**

Cipher one-shot arguments

Definition

```
struct smw_cipher_args {  
    struct smw_cipher_init_args init;  
    struct smw_cipher_data_args data;  
}
```

Members

init

Initialization arguments. See [struct smw_cipher_init_args](#).

data

Data arguments. See [struct smw_cipher_data_args](#).

Description

This structure is defined with the multi-part operation but field `context` present in `init` and `data` is ignored.

6.1.5 Device management APIs

The Device management APIs allow users of the library to manage device operations such as information, configuration, and lifecycle management.

6.1.5.1 Device information

6.1.5.1.1 Functions

6.1.5.1.1.1 smw_device_get_info

enum [smw_status_code](#) smw_device_get_info(struct [smw_device_info_args](#) *args)

Get the device information.

Parameters

- **args** (struct [smw_device_info_args](#)*) - Pointer to the structure that contains the device information arguments.

Description

Reads the device information including the SoC ID, SoC revision and whether the OEM SRKH is fused.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - args is NULL.
- Other error code from [enum smw_status_code](#)

6.1.5.1.1.2 smw_device_attestation

enum [smw_status_code](#) smw_device_attestation(struct [smw_device_attestation_args](#) *args)

Get the device attestation certificate.

Parameters

- **args** (struct [smw_device_attestation_args](#)*) - Pointer to the structure that contains the device attestation arguments.

Description

Reads the device attestation.

To query the required certificate buffer length, set `args->certificate` to NULL. The function will then set the required certificate buffer length in `args->certificate_length` and return `SMW_STATUS_OK`.

On operation completion, the `args->certificate_length` field is updated to the correct value when:

- Certificate buffer length is bigger than expected. In this case operation succeeds.
- Certificate buffer length is shorter than expected. In this case operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - Additional invalid parameters returned by the subsystem.
- **SMW_STATUS_OPERATION_DISABLED:**
EL2GO provisioned keys are missing their Key Check Values (KCV) and the device attestation service is disabled.
- Other error code from [enum smw_status_code](#)

6.1.5.1.1.3 smw_device_get_uuid

enum [smw_status_code](#) smw_device_get_uuid(struct [smw_device_uuid_args](#) *args)

Get the device UUID.

Parameters

- **args** (struct [smw_device_uuid_args](#)*) - Pointer to the structure that contains the device UUID arguments.

Description

Extracts device UUID from the device certificate, if `args->certificate` is set or reads the device UUID from device if `args->certificate` is NULL.

Device UUID buffer is in big endian format.

To query the required UUID buffer length, set `args->uuid` to NULL. The function will then set the required UUID buffer length in `args->uuid_length` and return **SMW_STATUS_OK**.

On operation completion, the `args->uuid_length` field is updated to the correct value when:

- UUID buffer length is bigger than expected. In this case operation succeeds.
- UUID buffer length is shorter than expected. In this case operation fails and returns **SMW_STATUS_OUTPUT_TOO_SHORT**.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.5.1.2 Structures

6.1.5.1.2.1 struct smw_device_attestation_args

struct **smw_device_attestation_args**

Device attestation arguments

Definition

```
struct smw_device_attestation_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    unsigned char *challenge;  
    unsigned int challenge_length;  
    unsigned char *certificate;  
    unsigned int certificate_length;  
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

challenge

[in] Pointer to caller unique ephemeral value (e.g. nonce).

challenge_length

[in] Length in bytes of the challenge buffer.

certificate

[out] Pointer to the generated device attestation certificate.

certificate_length

[in/out] Length in bytes of the certificate buffer.

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the default configured Secure Subsystem is used.

`challenge` length depends of the device (refer to the subsystem capabilities). If the length is bigger than expected, it will be cut to keep only the device maximum size. If the length is shorter, the challenge value will be completed with 0's.

6.1.5.1.2.2 struct smw_device_uuid_args

struct `smw_device_uuid_args`

Device UUID arguments

Definition

```
struct smw_device_uuid_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
}
```

```
    unsigned char *certificate;  
    unsigned int  certificate_length;  
    unsigned char *uuid;  
    unsigned int  uuid_length;  
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

certificate

[in] Device attestation certificate.

certificate_length

[in] Length in bytes of the certificate.

uuid

[out] Device UUID buffer.

uuid_length

[out] Length in bytes of the Device UUID buffer.

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the default configured Secure Subsystem is used.

Two methods are allowed to get the Device UUID:

- **Method #1**

Extract the device UUID from the device certificate. The Device Certificate (@certificate) is previously read using the [smw_device_attestation\(\)](#) API.

- **Method #2**

Read the device UUID without providing the Device Certificate. The field `certificate` must be set to NULL.

6.1.5.1.2.3 struct smw_device_info_args

struct **smw_device_info_args**

Device information arguments

Definition

```
struct smw_device_info_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    smw_soc_id_t soc_id;  
    smw_soc_revision_t soc_rev;  
    smw_lifecycle_t lifecycle;  
}
```

```
    bool srkh_fused;  
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

soc_id

[out] SoC identifier. See [typedef smw_soc_id_t](#).

soc_rev

[out] SoC revision. See [typedef smw_soc_revision_t](#).

lifecycle

[out] SoC lifecycle. See [typedef smw_lifecycle_t](#).

srkh_fused

[out] True if the OEM SRKH is fused, false otherwise.

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the default configured Secure Subsystem is used.

6.1.5.1.3 Typedefs

6.1.5.1.3.1 typedef smw_soc_id_t

type **smw_soc_id_t**

NXP SoC Identifiers

Description

List of all NXP SoC identifiers handled in the library. List may be non-exhaustive.

Values

- SOC_UNKNOWN: Device not identified.
- SOC_IMX8ULP: i.MX8ULP device.
- SOC_IMX91: i.MX91 device.
- SOC_IMX93: i.MX93 device.
- SOC_IMX95: i.MX95 device.
- SOC_IMX941: i.MX941 device.
- SOC_IMX942: i.MX942 device.
- SOC_IMX943: i.MX943 device.
- SOC_IMX937: i.MX937 device (equivalent to i.MX952).
- SOC_IMX952: i.MX952 device.

6.1.5.1.3.2 typedef smw_soc_revision_t

type **smw_soc_revision_t**

NXP SOC Revision

Description

List of revision handled in the library. List may be non-exhaustive.

Values

- SOC_REV_A0: SoC revision A0.
- SOC_REV_A1: SoC revision A1.
- SOC_REV_A2: SoC revision A2.
- SOC_REV_B0: SoC revision B0.
- SOC_REV_B1: SoC revision B1.
- SOC_REV_C0: SoC revision C0.

6.1.5.2 Device lifecycle

6.1.5.2.1 Functions

6.1.5.2.1.1 smw_device_set_lifecycle

enum [smw_status_code](#) **smw_device_set_lifecycle**(struct [smw_device_lifecycle_args](#) *args)

Set the device to given lifecycle.

Parameters

- **args** (struct [smw_device_lifecycle_args](#)*) - Pointer to the structure that contains the device lifecycle arguments.

Description

Forward the device lifecycle to the given value. The device must be reset to propagate the new lifecycle.

Warning:

Forwarding device lifecycle is not reversible.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - **args** is NULL.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.5.2.1.2 smw_device_get_lifecycle

enum [smw_status_code](#) smw_device_get_lifecycle(struct [smw_device_lifecycle_args](#) *args)

Get the device active lifecycle.

Parameters

- **args** (struct [smw_device_lifecycle_args](#)*) - Pointer to the structure that contains the device lifecycle arguments.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - args is NULL.
- Other error code from [enum smw_status_code](#)

6.1.5.2.2 Structures

6.1.5.2.2.1 struct smw_device_lifecycle_args

struct **smw_device_lifecycle_args**

Device lifecycle arguments

Definition

```
struct smw_device_lifecycle_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    smw_lifecycle_t lifecycle_name;  
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

lifecycle_name

- [in] Name of the device lifecycle to set.
- [out] Name of the current device lifecycle.
- See [typedef smw_lifecycle_t](#).

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the default configured Secure Subsystem is used.

6.1.5.3 Device reprovisioning

The Device reprovisioning allows clearing and reinitializing the secure storage. This operation removes all stored keys and data objects, resetting anti-rollback protection.

Details on the reprovisioning support and procedure are available in the [Key and Data Storage](#) section.

6.1.5.3.1 Functions

6.1.5.3.1.1 smw_device_reprovision_prepare

enum [smw_status_code](#) smw_device_reprovision_prepare(struct [smw_device_reprovision_args](#) *args)

Fill the device reprovisioning message.

Parameters

- **args** (struct [smw_device_reprovision_args](#)*) - Pointer to the structure that contains the reprovisioning arguments.

Description

This function is used to fill the reprovisioning message that will have to be signed by the user using the device keys.

Warning:

This function is only available for the ELE (EdgeLock Enclave) Secure Subsystem.

To query the required data buffer length, set `args->data` to NULL. The function will then set the required data buffer length in `args->data_length` and return `SMW_STATUS_OK`.

On operation completion, the `args->data_length` field is updated to the correct value when:

- Data buffer length is bigger than expected. In this case operation succeeds.
- Data buffer length is shorter than expected. In this case operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

Return

- **SMW_STATUS_OK:**
 - Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - Additional invalid parameters returned by the subsystem.
- **Other error code from [enum smw_status_code](#)**
 - Common return codes

6.1.5.3.1.2 smw_device_reprovision

enum [smw_status_code](#) smw_device_reprovision(struct [smw_device_reprovision_args](#) *args)

Request device storage reprovisioning.

Parameters

- **args** (struct [smw_device_reprovision_args*](#)) - Pointer to the structure that contains the reprovisioning arguments.

Description

This function is used to request the subsystem to enable the secure storage re-provisioning. In this case, the rollback protection of the storage is reset and consequently all objects previously stored are lost.

Warning:

This function is only available for the ELE (EdgeLock Enclave) Secure Subsystem.

The field data of `args->data` might contain the ELE reprovisioning signed message to be passed to the ELE Secure Subsystem.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - args is NULL.
 - args->data is NULL.
 - args->data_length is 0.
 - Additional invalid parameters returned by the subsystem.
- **Other error code from [enum smw_status_code](#)**
 - Common return codes

6.1.5.3.2 Structures

6.1.5.3.2.1 struct smw_device_reprovision_args

struct **smw_device_reprovision_args**

Device storage reprovisioning arguments

Definition

```
struct smw_device_reprovision_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    unsigned char *data;  
    unsigned int data_length;  
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

data

- [in] Pointer to the data message signed sent to the device via the [smw_device_reprovision\(\)](#).
- [out] Pointer to the data message filled by the [smw_device_reprovision_prepare\(\)](#).

data_length

- [in] Length in bytes of the data buffer.
- [out] Length in bytes of the data buffer generated by [smw_device_reprovision_prepare\(\)](#).

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the default configured Secure Subsystem is used.

6.1.6 Data Storage APIs

The Storage APIs allow users of the library to manage secure storage operations such as creation, deletion, and data access.

Storing the data can be done either in plain text or encrypted and/or signed. If the data is encrypted data will be returned as an encrypted blob. If the data is signed, data will be returned as a signed blob. Refer to the subsystem capabilities for more details of the supported features and blob format. Signature is limited to MAC signature.

6.1.6.1 Data Management

6.1.6.1.1 Data Store

6.1.6.1.1.1 smw_store_data

enum [smw_status_code](#) [smw_store_data](#)(*struct smw_store_data_args* *args)

Store data.

Parameters

- **args** (*struct smw_store_data_args**) - Pointer to the structure that contains the store data arguments.

Description

Stores the data in the Secure Subsystem. The data can be optionally encrypted and/or signed by the Secure Subsystem before storage. In case of encryption and/or signature, the data retrieved is in blob i.e. encrypted and/or signed.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->data_descriptor` is NULL.

- args->data_descriptor->data is NULL.
 - args->data_descriptor->length is 0.
 - if args->encryption_args is defined:
 - args->encryption_args->nb_keys is 0.
 - args->encryption_args->keys_desc is NULL.
 - args->encryption_args->mode_name is SMW_CIPHER_MODE_NAME_NONE.
 - if args->sign_args is defined:
 - args->sign_args->key_descriptor is NULL.
 - args->sign_args->algo_name is SMW_SIGN_MODE_NAME_NONE.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.6.1.1.2 struct smw_encryption_args

struct **smw_encryption_args**

Encryption arguments

Definition

```
struct smw_encryption_args {
    struct smw_key_descriptor **keys_desc;
    unsigned int nb_keys;
    smw_cipher_mode_t mode_name;
    unsigned char *iv;
    unsigned int iv_length;
}
```

Members

keys_desc

[in] Pointer to an array of pointers to key descriptors used to encrypt the data. See [struct smw_key_descriptor](#).

nb_keys

[in] Number of entries of key descriptors array.

mode_name

[in] Cipher mode name. See [typedef smw_cipher_mode_t](#).

iv

[in] Pointer to initialization vector. Not used in ECB mode.

iv_length

[in] Length in bytes of the initialization vector. Not used in ECB mode.

Description

The number of keys `nb_keys` is usually 1 key, but for the XTS mode, 2 keys are required. In the case of multi-key cipher operation, the keys must be owned by the same Secure Subsystem and must be the same key type.

6.1.6.1.1.3 struct smw_sign_args

struct **smw_sign_args**

Sign arguments

Definition

```
struct smw_sign_args {  
    struct smw_key_descriptor *key_descriptor;  
    smw_mac_algo_t algo_name;  
    smw_hash_algo_t hash_name;  
}
```

Members

key_descriptor

[in] Pointer to key descriptor used for signing. See [struct smw_key_descriptor](#)

algo_name

[in] MAC algorithm name. See [typedef smw_mac_algo_t](#).

hash_name

[in] Hash algorithm name. See [typedef smw_hash_algo_t](#).

6.1.6.1.1.4 struct smw_store_data_args

struct **smw_store_data_args**

Store data arguments

Definition

```
struct smw_store_data_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_data_descriptor *data_descriptor;  
    struct smw_encryption_args *encryption_args;  
    struct smw_sign_args *sign_args;  
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

data_descriptor

[in] Data descriptor. See [struct smw_data_descriptor](#).

encryption_args

[in] (optional) Encryption arguments. See [struct smw_encryption_args](#).

sign_args

[in] (optional) Sign arguments. See [struct smw_sign_args](#).

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the default configured Secure Subsystem is used. In case, user requests to encrypt and/or sign the data, the subsystem is the Secure Subsystem handling the key(s).

The `encryption_args` and `sign_args` arguments are optional. If defined the operation consists respectively in encrypting and/or signing the data. The capability to encrypt and/or sign data is function of the subsystem. More details are available in the [Secure Subsystems Features and Capabilities](#).

Note:

If the data is encrypted and/or signed:

- Key must be present in the same Secure Subsystem. Only opaque key(s) is supported for encryption/signature operations.
- The key must be capable to do the cryptographic operation (i.e usage and permitted algorithm correctly defined when supported).

6.1.6.1.2 Data Retrieve

6.1.6.1.2.1 smw_retrieve_data

enum [smw_status_code](#) `smw_retrieve_data`(struct [smw_retrieve_data_args](#) *args)

Retrieve data.

Parameters

- **args** (struct [smw_retrieve_data_args](#)*) - Pointer to the structure that contains the retrieve data arguments.

Description

Retrieves the data in the format that was stored. If the data was encrypted and/or signed, the data is returned in blob format (encrypted and/or signed), as detailed in the [Secure Subsystems Features and Capabilities](#). If the data was not encrypted and/or signed, the data is returned in plain format.

To query the required data buffer length for data retrieval, set `args->data_descriptor->data` to NULL. The function will then set the required data buffer length in `args->data_descriptor->length` and return `SMW_STATUS_OK`.

On operation completion, the `args->data_descriptor->length` is updated to the correct value when

- Data buffer length is bigger than expected. In this case, operation succeeds.
- Data buffer length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

Return

- **SMW_STATUS_OK:**

Operation succeeded.

- **SMW_STATUS_INVALID_PARAM:**

- args is NULL.
- args->data_descriptor is NULL.
- args->data_descriptor->data is NULL.
- args->data_descriptor->identifier is 0.
- args->subsystem_name is specified (other than SMW_SUBSYSTEM_NAME_NONE) and does not match the subsystem handling the data identifier.
- Additional invalid parameters returned by the subsystem.

- Other error code from [enum smw_status_code](#)

6.1.6.1.2.2 struct smw_retrieve_data_args

struct **smw_retrieve_data_args**

Retrieve data arguments

Definition

```
struct smw_retrieve_data_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_data_descriptor *data_descriptor;  
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

data_descriptor

[in/out] Data descriptor. See [struct smw_data_descriptor](#).

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the Secure Subsystem is the one handling the data identifier.

6.1.6.1.3 Data Delete

6.1.6.1.3.1 smw_delete_data

enum [smw_status_code](#) **smw_delete_data**(struct [smw_delete_data_args](#) *args)

Delete data.

Parameters

- **args** (struct [smw_delete_data_args](#)*) - Pointer to the structure that contains the store data arguments.

Description

Deletes the data.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - args is NULL.
 - args->data_descriptor is NULL.
 - args->data_descriptor->data is NULL.
 - args->data_descriptor->identifier is 0.
 - args->subsystem_name is specified (other than SMW_SUBSYSTEM_NAME_NONE) and does not match the subsystem handling the data identifier.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.6.1.3.2 struct smw_delete_data_args

struct **smw_delete_data_args**

Delete data arguments

Definition

```
struct smw_delete_data_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_data_descriptor *data_descriptor;  
}
```

Members

version

[in] Version of this structure.

subsystem_name

[in] Secure Subsystem name. See [typedef smw_subsystem_t](#).

data_descriptor

[in] Data descriptor. See [struct smw_data_descriptor](#).

Description

The `subsystem_name` designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the Secure Subsystem is the one handling the data identifier.

6.1.6.1.4 Data Information

6.1.6.1.4.1 `smw_get_data_info`

enum [smw_status_code](#) `smw_get_data_info`(struct [smw_data_info_args](#) *args)

Get data information.

Parameters

- `args` (struct [smw_data_info_args](#)*) - Pointer to the data information arguments.

Description

Returns the data information extracts from the subsystem where data is stored combined with the internal database if data identifier is present.

The `args.subsystem_name` field returned is the subsystem name that owns the data.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->data_descriptor` is NULL.
 - `args->data_descriptor->data` is NULL.
 - `args->data_descriptor->identifier` is 0.
 - `args->subsystem_name` is specified (other than `SMW_SUBSYSTEM_NAME_NONE`) and does not match the subsystem handling the data identifier.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.6.1.4.2 struct `smw_data_info_args`

struct `smw_data_info_args`

Data information arguments

Definition

```
struct smw_data_info_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_data_descriptor *data_descriptor;  
}
```


Members

version

[in] Version of this structure.

subsystem_name

[in/out] Secure Subsystem name. See [typedef smw_subsystem_t](#).

data_descriptor

[in/out] Data descriptor. See [struct smw_data_descriptor](#).

Description

This function gets the data attributes retrieved for the subsystem owning the given data identifier.

The `subsystem_name` (input) designates the Secure Subsystem to be used. If this field is [SMW_SUBSYSTEM_NAME_NONE](#), the Secure Subsystem is the one handling the data identifier.

The `subsystem_name` (output) when input value is set to [SMW_SUBSYSTEM_NAME_NONE](#) is updated with the Secure Subsystem name handling the data identifier.

If some data attributes are not supported, the output values are empty.

6.1.6.2 Data Descriptor

6.1.6.2.1 Structures

6.1.6.2.1.1 struct smw_data_attributes

struct **smw_data_attributes**

Data attributes

Definition

```
struct smw_data_attributes {
    smw_attr_storage_id_t storage_id;
    smw_attr_attributes_t attributes;
}
```

Members

storage_id

[in/out] Storage identifier. See [typedef smw_attr_storage_id_t](#).

attributes

[in/out] Attributes. See [typedef smw_attr_attributes_t](#).

6.1.6.2.1.2 struct smw_data_descriptor

struct **smw_data_descriptor**

Data descriptor

Definition

```
struct smw_data_descriptor {
    unsigned int identifier;
    unsigned char *data;
    unsigned int length;
    struct smw_data_attributes attributes;
}
```

Members

identifier

[in] Data identifier.

data

[in/out] Pointer to the data buffer.

length

[in/out] Length in bytes of data buffer.

attributes

Data attributes. See [typedef smw_data_attributes](#).

6.1.7 Object Database APIs

The object database APIs allows users to find object(s) and update some properties of an object in the SMW database.

6.1.7.1 Finding object(s)

Two methods are available to find object(s):

- By its identifier
- By its attribute(s)

6.1.7.1.1 Find by identifier

6.1.7.1.1.1 smw_find_object_db

enum [smw_status_code](#) **smw_find_object_db**(struct [smw_find_object_db_args](#) *args)

Find an object by its id.

Parameters

- **args** (struct [smw_find_object_db_args](#)*) - Pointer to the structure that contains the find object arguments.

Description

This function returns the object descriptor matching the following object defined by the `args->object_descriptor` following fields:

- **id:** [in] (**mandatory**) Object identifier to update.

- `type`: [in] (**optional**) Object type.
- `persistence`: [in] (**optional**) Object persistence attributes.
- `key.attributes.attributes`: [in] (**optional**) Key persistence, if the `type` is a key type.
- `data.attributes.attributes`: [in] (**optional**) Data persistence, if the `type` is a data type.

On successful return, the `args->object_descriptor` is filled with the object information. The following fields are output fields:

- `id`: [out] Object identifier.
- `type`: [out] Object type.
- `subsystem_name`: [out] Secure Subsystem name owning the object.
- `label`: [out] User defined object string label.
- `user_id`: [out] User defined ID in base64 null terminated string.
- `group`: [out] Object storage group (if supported by the Secure Subsystem).
- If the `type` is a key type:
 - `key.type_name`: [out] Key type name.
 - `key.security_size`: [out] Key security size.
 - `key.id`: [out] Key identifier (same as the `id`).
 - `key.attributes`: [out] Key attributes; permitted algorithm, usage, storage id, attributes.
 - `buffer`: Nothing returned.
- If the `type` is a data type:
 - `data.identifier`: [out] Data identifier (same as the `id`).
 - `data.attributes`: [out] Data attributes; storage id, attributes.
 - `data.data`: Nothing returned.
 - `data.length`: Nothing returned.

Return

- **SMW_STATUS_OK**:
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM**:
 - `args` is NULL.
 - `args->object_descriptor` is NULL.
 - `args->object_descriptor.data.data` is not NULL, if `args->object_descriptor.type` is not a key type.
 - Additional invalid parameters returned by the subsystem.
- Other error code from [enum smw_status_code](#)

6.1.7.1.2 Find by attributes

6.1.7.1.2.1 smw_find_object_db_init

enum [smw_status_code](#) `smw_find_object_db_init`(struct [smw_find_object_db_args](#) *args)

Initialize the find object context.

Parameters

- **args** (struct [smw_find_object_db_args](#)*) - Pointer to the structure that contains the find object arguments.

Description

This function allocates a new find context for multi-part object search in the SMW's database. Next, the [smw_find_object_db_next\(\)](#) function must be called in a loop to retrieve all matching objects.

Any of the `args->object_descriptor` fields can be used to filter the objects to be found.

The search is limited to persistency attribute defined in the `args->object_descriptor` by one of the following field:

- `persistency`: [in] Object persistency attributes.
- `key.attributes.attributes`: [in] Key persistency if `type` is a key type.
- `data.attributes.attributes`: [in] Data persistency if `type` is a data type.

Note:

If the input `@args->object_descriptor.id` is set, the search in the database will be limited to the object with that identifier. If the object identifier is not present in the database, the function queries all subsystems to find the object (key and/or data as defined by the `@args->object_descriptor.type` field). As the object identifier is known, it's advised to use the function `smw_find_object_db()` instead.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->object_descriptor` is NULL.
- Other error code from [enum smw_status_code](#)

6.1.7.1.2.2 smw_find_object_db_next

enum [smw_status_code](#) `smw_find_object_db_next(struct smw_find_object_db_args *args)`

Find next matching object.

Parameters

- **args** (struct [smw_find_object_db_args*](#)) - Pointer to the structure that contains the find object arguments.

Description

This function returns one by one the object matching the query setup by the [smw_find_object_db_init\(\)](#) function.

The operation context must be by initialized by the multi-part initialization [smw_find_object_db_init\(\)](#) API or updated by a previous [smw_find_object_db_next\(\)](#) call.

On successful return, the `args->object_descriptor` is filled with the object information. The following fields are output fields:

- `id`: [out] Object identifier.
- `type`: [out] Object type.
- `subsystem_name`: [out] Secure Subsystem name owning the object.
- `label`: [out] User defined object string label.

- `user_id`: [out] User defined ID in base64 null terminated string.
- `group`: [out] Object storage group (if supported by the Secure Subsystem).
- If the type is a key type:
 - `key.type_name`: [out] Key type name.
 - `key.security_size`: [out] Key security size.
 - `key.id`: [out] Key identifier (same as the `id`).
 - `key.attributes`: [out] Key attributes; permitted algorithm, usage, storage id, attributes.
 - `buffer`: Nothing returned.
- If the type is a data type:
 - `data.identifier`: [out] Data identifier (same as the `id`).
 - `data.attributes`: [out] Data attributes; storage id, attributes.
 - `data.data`: Nothing returned.
 - `data.length`: Nothing returned.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
 - `args->object_descriptor` is NULL.
- Other error code from [enum smw_status_code](#)

6.1.7.1.2.3 smw_find_object_db_final

enum [smw_status_code](#) `smw_find_object_db_final`(struct [smw_find_object_db_args](#) *args)

Finalize the find context.

Parameters

- **args** (struct [smw_find_object_db_args](#)*) - Pointer to the structure that contains the find object arguments.

Description

This function close the find query and destroys the operation context.

Return

- **SMW_STATUS_OK:**
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM:**
 - `args` is NULL.
- Other error code from [enum smw_status_code](#)

6.1.7.1.3 Structures

6.1.7.1.3.1 struct smw_find_object_db_args

struct **smw_find_object_db_args**

Object find operation arguments

Definition

```
struct smw_find_object_db_args {  
    unsigned char version;  
    struct smw_object_descriptor *object_descriptor;  
    void *ctx;  
}
```

Members

version

[in] Version of this structure

object_descriptor

[in/out] Object descriptor. See [struct smw_object_descriptor](#).

ctx

[in/out] Find operation opaque context.

Description

Field `ctx`:

- Is used only for multi-part operations.
- Is allocated and initialized by [smw_find_object_db_init\(\)](#).
- Must be passed to each [smw_find_object_db_next\(\)](#) call.
- Is deleted when final operation [smw_find_object_db_final\(\)](#) is called.

6.1.7.2 Updating object properties

6.1.7.2.1 Function

6.1.7.2.1.1 smw_update_object_db

enum [smw_status_code](#) **smw_update_object_db**(struct [smw_object_descriptor](#) *descriptor)

Update a database object.

Parameters

- **descriptor** (struct [smw_object_descriptor](#)*) - Object descriptor to update.

Description

This function updates the object attribute list.

The `descriptor` following fields should be used to identify the object to be updated:

- `id`: [in] (**mandatory**) Object identifier to update.
- `type`: [in] (**optional**) Object type.
- `subsystem_name`: [in] (**optional**) Secure Subsystem name.
- `persistence`: [in] (**optional**) Object persistence attributes.
- `key.attributes.attributes`: [in] (**optional**) Key persistence, if the `type` is a key type.
- `data.attributes.attributes`: [in] (**optional**) Data persistence, if the `type` is a data type.

The `descriptor` following fields could be updated:

- `label`: [in] User defined object string label.
- `user_id`: [in] User defined ID in base64 null terminated string.

The other `descriptor` fields are ignored during the update operation.

Return

- **SMW_STATUS_OK**:
Operation succeeded.
- **SMW_STATUS_INVALID_PARAM**:
 - `descriptor` is NULL.
- Other error code from [enum smw_status_code](#)

6.1.7.3 Object Descriptor

6.1.7.3.1 Structure

6.1.7.3.1.1 struct smw_object_descriptor

struct **smw_object_descriptor**

Generic SMW object descriptor

Definition

```
struct smw_object_descriptor {
    unsigned int id;
    smw_object_type_t type;
    smw_subsystem_t subsystem_name;
    smw_attr_attributes_t persistence;
    char *label;
    char *user_id;
    unsigned int group;
    union {
        struct smw_key_descriptor key;
        struct smw_data_descriptor data;
    };
};
```

Members

id

[in/out] Object identifier.

type

[in/out] Defines the object type. See [typedef smw_object_type_t](#).

subsystem_name

[in/out] Secure Subsystem name. See [typedef smw_subsystem_t](#).

persistence

[in/out] Object persistence attributes. See [typedef smw_attr_attributes_t](#).

label

[in/out] User defined object string label.

user_id

[in/out] User defined ID in base64 null terminated string.

group

[in/out] Object storage group (may not be used by all subsystems).

{unnamed_union}

anonymous

key

[in/out] Key descriptor. See [struct smw_key_descriptor](#).

data

[in/out] Data descriptor. See [struct smw_data_descriptor](#).

Description

Depending on the type of object (key or data), the object persistence is retrieved from either the `key.attributes` or the `data.attributes`.

6.1.8 Status APIs return codes

This chapter lists and describes all the return codes and status values returned by the SMW APIs.

6.1.8.1 Status codes

6.1.8.1.1 enum smw_status_code

enum smw_status_code

Security Middleware status codes

6.1.8.1.1.1 Definition

```
enum smw_status_code {
    SMW_STATUS_OK,
    SMW_STATUS_INVALID_VERSION,
    SMW_STATUS_INVALID_BUFFER,
```



```
SMW_STATUS_EOF,  
SMW_STATUS_SYNTAX_ERROR,  
SMW_STATUS_UNKNOWN_NAME,  
SMW_STATUS_UNKNOWN_ID,  
SMW_STATUS_TOO_LARGE_NUMBER,  
SMW_STATUS_ALLOC_FAILURE,  
SMW_STATUS_INVALID_PARAM,  
SMW_STATUS_VERSION_NOT_SUPPORTED,  
SMW_STATUS_SUBSYSTEM_LOAD_FAILURE,  
SMW_STATUS_SUBSYSTEM_UNLOAD_FAILURE,  
SMW_STATUS_SUBSYSTEM_FAILURE,  
SMW_STATUS_SUBSYSTEM_NOT_CONFIGURED,  
SMW_STATUS_OPERATION_NOT_SUPPORTED,  
SMW_STATUS_OPERATION_NOT_CONFIGURED,  
SMW_STATUS_OPERATION_FAILURE,  
SMW_STATUS_SIGNATURE_INVALID,  
SMW_STATUS_NO_KEY_BUFFER,  
SMW_STATUS_OUTPUT_TOO_SHORT,  
SMW_STATUS_SIGNATURE_LEN_INVALID,  
SMW_STATUS_OPS_INVALID,  
SMW_STATUS_MUTEX_INIT_FAILURE,  
SMW_STATUS_MUTEX_DESTROY_FAILURE,  
SMW_STATUS_INVALID_TAG,  
SMW_STATUS_RANGE_DUPLICATE,  
SMW_STATUS_ALGO_NOT_CONFIGURED,  
SMW_STATUS_CONFIG_ALREADY_LOADED,  
SMW_STATUS_NO_CONFIG_LOADED,  
SMW_STATUS_SUBSYSTEM_OUT_OF_MEMORY,  
SMW_STATUS_SUBSYSTEM_STORAGE_NO_SPACE,  
SMW_STATUS_SUBSYSTEM_STORAGE_ERROR,  
SMW_STATUS_SUBSYSTEM_CORRUPT_OBJECT,  
SMW_STATUS_LOAD_METHOD_DUPLICATE,  
SMW_STATUS_LIBRARY_ALREADY_INIT,  
SMW_STATUS_SUBSYSTEM_LOADED,  
SMW_STATUS_SUBSYSTEM_NOT_LOADED,  
SMW_STATUS_OBJ_DB_INIT,  
SMW_STATUS_OBJ_DB_CREATE,  
SMW_STATUS_OBJ_DB_UPDATE,  
SMW_STATUS_OBJ_DB_DELETE,  
SMW_STATUS_OBJ_DB_GET_INFO,  
SMW_STATUS_KEY_POLICY_WARNING_IGNORED,  
SMW_STATUS_KEY_INVALID,  
SMW_STATUS_MUTEX_LOCK_FAILURE,  
SMW_STATUS_MUTEX_UNLOCK_FAILURE,  
SMW_STATUS_INVALID_LIBRARY_CONTEXT,  
SMW_STATUS_INVALID_CONFIG_DATABASE,  
SMW_STATUS_INVALID_LIFECYCLE,  
SMW_STATUS_UNKNOWN_MODE_NAME,  
SMW_STATUS_UNKNOWN_OP_TYPE_NAME,  
SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME,  
SMW_STATUS_UNKNOWN_ALGO_NAME,  
SMW_STATUS_UNKNOWN_SIGN_ALGO_NAME,  
SMW_STATUS_UNKNOWN_SIGN_TYPE_NAME,  
SMW_STATUS_UNKNOWN_CONFIG_OP_NAME,  
SMW_STATUS_UNKNOWN_LOAD_METHOD_NAME,  
SMW_STATUS_UNKNOWN_KEY_OP_NAME,  
SMW_STATUS_UNKNOWN_KEY_TYPE_NAME,  
SMW_STATUS_UNKNOWN_FORMAT_NAME,  
SMW_STATUS_UNKNOWN_KDF_NAME,  
SMW_STATUS_UNKNOWN_TLS12_KEYA_NAME,
```

```
SMW_STATUS_UNKNOWN_TLS12_ENC_NAME,  
SMW_STATUS_OEM_SRKH_NOT_FUSED,  
SMW_STATUS_CONFIGURATION_FAILURE,  
SMW_STATUS_INVALID_IV_SIZE,  
SMW_STATUS_UNKNOWN_KEY_PRIVACY_NAME,  
SMW_STATUS_OBJ_DB_FIND,  
SMW_STATUS_PUBLIC_EXPONENT_NOT_SUPPORTED,  
SMW_STATUS_KEY_ID_ALREADY_EXIST,  
SMW_STATUS_PERMITTED_ALGO_INVALID,  
SMW_STATUS_OPERATION_ALREADY_INIT,  
SMW_STATUS_INPUT_TOO_LARGE,  
SMW_STATUS_KEY_RANGE_NOT_CONFIGURABLE,  
SMW_STATUS_OBJ_DB_CAPABILITY_NOT_SUPPORTED,  
SMW_STATUS_OPERATION_DISABLED  
};
```

6.1.8.1.1.2 Constants

SMW_STATUS_OK

Function returned successfully.

SMW_STATUS_INVALID_VERSION

The version of the configuration file is not supported.

SMW_STATUS_INVALID_BUFFER

The configuration file passed by OSAL to the library is not valid.

SMW_STATUS_EOF

The configuration file is syntactically too short.

SMW_STATUS_SYNTAX_ERROR

The configuration file is syntactically wrong.

SMW_STATUS_UNKNOWN_NAME

Generic status code indicating that one of the name arguments is not valid.

SMW_STATUS_UNKNOWN_ID

One of the identifier arguments is not valid.

SMW_STATUS_TOO_LARGE_NUMBER

The configuration file defines a too big numeral value.

SMW_STATUS_ALLOC_FAILURE

Internal allocation failure.

SMW_STATUS_INVALID_PARAM

Generic status code indicating that one of the argument parameter is not valid.

SMW_STATUS_VERSION_NOT_SUPPORTED

Argument version not compatible.

SMW_STATUS_SUBSYSTEM_LOAD_FAILURE

Load of the Secure Subsystem failed.

SMW_STATUS_SUBSYSTEM_UNLOAD_FAILURE

Unload of the Secure Subsystem failed.

SMW_STATUS_SUBSYSTEM_FAILURE

Secure Subsystem operation general failure.

SMW_STATUS_SUBSYSTEM_NOT_CONFIGURED

Secure Subsystem is not configured in the user configuration.

SMW_STATUS_OPERATION_NOT_SUPPORTED

Operation is not supported by the Secure Subsystem.

SMW_STATUS_OPERATION_NOT_CONFIGURED

Operation is not configured in the user configuration.

SMW_STATUS_OPERATION_FAILURE

Operation general failure. Error returned before calling the Secure Subsystem.

SMW_STATUS_SIGNATURE_INVALID

The Signature is not valid.

SMW_STATUS_NO_KEY_BUFFER

No Key buffer is set in the Key descriptor structure.

SMW_STATUS_OUTPUT_TOO_SHORT

Output buffer is too small. Output size field is updated with the expected size.

SMW_STATUS_SIGNATURE_LEN_INVALID

The Signature length is not valid.

SMW_STATUS_OPS_INVALID

OSAL operations structure is invalid.

SMW_STATUS_MUTEX_INIT_FAILURE

Mutex initialization has failed.

SMW_STATUS_MUTEX_DESTROY_FAILURE

Mutex destruction has failed.

SMW_STATUS_INVALID_TAG

Tag is invalid.

SMW_STATUS_RANGE_DUPLICATE

Size range is defined more than once for a given algorithm.

SMW_STATUS_ALGO_NOT_CONFIGURED

Size range is defined but the corresponding algorithm is not configured.

SMW_STATUS_CONFIG_ALREADY_LOADED

User configuration is already loaded. To load another one, the Unload configuration API must be called first.

SMW_STATUS_NO_CONFIG_LOADED

No user configuration is loaded.

SMW_STATUS_SUBSYSTEM_OUT_OF_MEMORY

Subsystem memory allocation failure.

SMW_STATUS_SUBSYSTEM_STORAGE_NO_SPACE

Not enough space in the secure subsystem to handle the requested operation.

SMW_STATUS_SUBSYSTEM_STORAGE_ERROR

Generic secure subsystem storage error.

SMW_STATUS_SUBSYSTEM_CORRUPT_OBJECT

An object stored in the secure subsystem is corrupted.

SMW_STATUS_LOAD_METHOD_DUPLICATE

The load/unload method is defined more than once.

SMW_STATUS_LIBRARY_ALREADY_INIT

Library is already initialized.

SMW_STATUS_SUBSYSTEM_LOADED

Secure Subsystem is loaded.

SMW_STATUS_SUBSYSTEM_NOT_LOADED

Secure Subsystem is not loaded.

SMW_STATUS_OBJ_DB_INIT

Initialization error of the object database.

SMW_STATUS_OBJ_DB_CREATE

Object database creation error.

SMW_STATUS_OBJ_DB_UPDATE

Object database update error.

SMW_STATUS_OBJ_DB_DELETE

Object database delete error.

SMW_STATUS_OBJ_DB_GET_INFO

Object database get information error.

SMW_STATUS_KEY_POLICY_WARNING_IGNORED

At least one element of the key policy is ignored.

SMW_STATUS_KEY_INVALID

Key used for the operation is not valid.

SMW_STATUS_MUTEX_LOCK_FAILURE

Mutex lock has failed.

SMW_STATUS_MUTEX_UNLOCK_FAILURE

Mutex unlock has failed.

SMW_STATUS_INVALID_LIBRARY_CONTEXT

Library context is not valid.

SMW_STATUS_INVALID_CONFIG_DATABASE

Configuration database is not valid.

SMW_STATUS_INVALID_LIFECYCLE

Device lifecycle not valid, or object not accessible in current device lifecycle.

SMW_STATUS_UNKNOWN_MODE_NAME

Mode name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_OP_TYPE_NAME

Operation type name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME

Secure Subsystem name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_ALGO_NAME

Algorithm name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_SIGN_ALGO_NAME

Signature algo name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_SIGN_TYPE_NAME

Signature type name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_CONFIG_OP_NAME

Operation name set in the configuration file is not recognized by SMW.

SMW_STATUS_UNKNOWN_LOAD_METHOD_NAME

String value of the load/unload method set in the configuration file is not recognized by SMW.

SMW_STATUS_UNKNOWN_KEY_OP_NAME

Key operation name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_KEY_TYPE_NAME

Key type name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_FORMAT_NAME

Key format name provided by the user is not recognized by SMW.

SMW_STATUS_UNKNOWN_KDF_NAME

Key derivation function name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_TLS12_KEYA_NAME

TLS 1.2 Key exchange algorithm name provided by the user is not recognized by SMW.

SMW_STATUS_UNKNOWN_TLS12_ENC_NAME

TLS 1.2 encryption algorithm name provided by the user is not recognized by SMW.

SMW_STATUS_OEM_SRKH_NOT_FUSED

Device OEM SRKH is not fused.

SMW_STATUS_CONFIGURATION_FAILURE

Library configuration failure.

SMW_STATUS_INVALID_IV_SIZE

IV size used for the operation is not valid

SMW_STATUS_UNKNOWN_KEY_PRIVACY_NAME

Key privacy name set in the object database is not recognized by SMW.

SMW_STATUS_OBJ_DB_FIND

Object database find object error.

SMW_STATUS_PUBLIC_EXPONENT_NOT_SUPPORTED

Provided RSA public exponent is unsupported.

SMW_STATUS_KEY_ID_ALREADY_EXIST

Key identifier already exist.

SMW_STATUS_PERMITTED_ALGO_INVALID

Permitted algorithm missing one or more algorithm parameters.

SMW_STATUS_OPERATION_ALREADY_INIT

Operation is already initialized.

SMW_STATUS_INPUT_TOO_LARGE

Input buffer is too large.

SMW_STATUS_KEY_RANGE_NOT_CONFIGURABLE

The key range can not be set for this type of key.

SMW_STATUS_OBJ_DB_CAPABILITY_NOT_SUPPORTED

undescribed

SMW_STATUS_OPERATION_DISABLED

Operation is disabled by the secure subsystem (e.g., EL2GO key KCV absent).

6.1.8.1.1.3 Status code classification

- Common return codes
 - SMW_STATUS_OK
 - SMW_STATUS_UNKNOWN_NAME
 - SMW_STATUS_UNKNOWN_ID
 - SMW_STATUS_ALLOC_FAILURE
 - SMW_STATUS_INVALID_PARAM
 - SMW_STATUS_VERSION_NOT_SUPPORTED
 - SMW_STATUS_SUBSYSTEM_LOAD_FAILURE
 - SMW_STATUS_SUBSYSTEM_UNLOAD_FAILURE
 - SMW_STATUS_SUBSYSTEM_FAILURE
 - SMW_STATUS_SUBSYSTEM_NOT_CONFIGURED
 - SMW_STATUS_OPERATION_NOT_SUPPORTED
 - SMW_STATUS_OPERATION_NOT_CONFIGURED
 - SMW_STATUS_OPERATION_FAILURE
 - SMW_STATUS_NO_KEY_BUFFER
 - SMW_STATUS_OUTPUT_TOO_SHORT
 - SMW_STATUS_INPUT_TOO_LARGE
 - SMW_STATUS_SUBSYSTEM_OUT_OF_MEMORY
 - SMW_STATUS_SUBSYSTEM_STORAGE_NO_SPACE
 - SMW_STATUS_SUBSYSTEM_STORAGE_ERROR
 - SMW_STATUS_SUBSYSTEM_CORRUPT_OBJECT
 - SMW_STATUS_SUBSYSTEM_LOADED
 - SMW_STATUS_SUBSYSTEM_NOT_LOADED
 - SMW_STATUS_KEY_INVALID
 - SMW_STATUS_INVALID_LIFECYCLE
 - SMW_STATUS_INVALID_IV_SIZE
 - SMW_STATUS_UNKNOWN_MODE_NAME
 - SMW_STATUS_UNKNOWN_OP_TYPE_NAME
 - SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME
 - SMW_STATUS_UNKNOWN_ALGO_NAME
 - SMW_STATUS_UNKNOWN_SIGN_ALGO_NAME

- SMW_STATUS_UNKNOWN_SIGN_TYPE_NAME
- SMW_STATUS_OPERATION_ALREADY_INIT
- SMW_STATUS_OPERATION_DISABLED
- Specific return codes - Library initialization
 - SMW_STATUS_OPS_INVALID
 - SMW_STATUS_MUTEX_INIT_FAILURE
 - SMW_STATUS_MUTEX_DESTROY_FAILURE
 - SMW_STATUS_LIBRARY_ALREADY_INIT
 - SMW_STATUS_MUTEX_LOCK_FAILURE
 - SMW_STATUS_MUTEX_UNLOCK_FAILURE
 - SMW_STATUS_INVALID_LIBRARY_CONTEXT
 - SMW_STATUS_INVALID_CONFIG_DATABASE
 - SMW_STATUS_CONFIGURATION_FAILURE
- Specific return codes - Configuration file
 - SMW_STATUS_INVALID_VERSION
 - SMW_STATUS_INVALID_BUFFER
 - SMW_STATUS_EOF
 - SMW_STATUS_SYNTAX_ERROR
 - SMW_STATUS_TOO_LARGE_NUMBER
 - SMW_STATUS_INVALID_TAG
 - SMW_STATUS_RANGE_DUPLICATE
 - SMW_STATUS_ALGO_NOT_CONFIGURED
 - SMW_STATUS_CONFIG_ALREADY_LOADED
 - SMW_STATUS_NO_CONFIG_LOADED
 - SMW_STATUS_LOAD_METHOD_DUPLICATE
 - SMW_STATUS_UNKNOWN_CONFIG_OP_NAME
 - SMW_STATUS_UNKNOWN_LOAD_METHOD_NAME
 - SMW_STATUS_KEY_RANGE_NOT_CONFIGURABLE
- Specific return codes - Signature
 - SMW_STATUS_SIGNATURE_INVALID
 - SMW_STATUS_SIGNATURE_LEN_INVALID
- Specific return codes - Object database
 - SMW_STATUS_ERROR_OBJ_DB_INIT
 - SMW_STATUS_ERROR_OBJ_DB_CREATE
 - SMW_STATUS_ERROR_OBJ_DB_UPDATE
 - SMW_STATUS_ERROR_OBJ_DB_DELETE
 - SMW_STATUS_ERROR_OBJ_DB_GET_INFO
 - SMW_STATUS_ERROR_OBJ_DB_FIND
- Specific return codes - Key manager
 - SMW_STATUS_KEY_POLICY_WARNING_IGNORED
 - SMW_STATUS_UNKNOWN_KEY_OP_NAME
 - SMW_STATUS_UNKNOWN_KEY_TYPE_NAME
 - SMW_STATUS_UNKNOWN_FORMAT_NAME
 - SMW_STATUS_UNKNOWN_KDF_NAME
 - SMW_STATUS_UNKNOWN_TLS12_KEY_NAME
 - SMW_STATUS_UNKNOWN_TLS12_ENC_NAME
 - SMW_STATUS_UNKNOWN_KEY_PRIVACY_NAME

- SMW_STATUS_PUBLIC_EXPONENT_NOT_SUPPORTED
- SMW_STATUS_KEY_ID_ALREADY_EXIST
- SMW_STATUS_PERMITTED_ALGO_INVALID
- Specific return codes - Device manager
 - SMW_STATUS_OEM_SRKH_NOT_FUSED

6.1.9 Cryptographic Algorithm (smw_attr_algo_t) encoding

This chapter intends to detail the algorithm encoding definitions used to describe cryptographic algorithms and their parameters within the SMW API.

The algorithm encoding is a 64-bit value that encapsulates all necessary information to define and restrict cryptographic operations.

The same 64-bit encoding is used throughout the SMW API to specify:

- Cryptographic algorithms (symmetric, asymmetric, hash functions).
- Algorithm modes and parameters.
- Permitted key algorithm when requested by the Security Subsystem.

6.1.9.1 Algorithm Encoding

6.1.9.1.1 Typedef

6.1.9.1.1.1 typedef smw_attr_algo_t

type **smw_attr_algo_t**
Algorithm parameters

Description

64-bits field used to define an algorithm and its parameters.

Table 139. Algorithm encoding bit fields

Bits	Description
[63:40]	Reserved
[39:32]	Additional Algorithm Parameters:
[31:24]	Operation Class Algorithm Operation value table
[23:16]	Hash Algorithm Hash value table
[15:8]	Define either the algorithm:
[7:0]	Main Algorithm Main Algorithm value table

6.1.9.2 Main algorithm

Table 140. Main Algorithm value

Value	Define	Description
0x0	SMW_ATTR_ALGO_NONE	No algorithm defined.
0x1	SMW_ATTR_ALGO_AES	Advanced Encryption Standard.

Table 140. Main Algorithm value...continued

Value	Define	Description
0x2	SMW_ATTR_ALGO_DES	Data Encryption Standard.
0x3	SMW_ATTR_ALGO_DES3	Triple DES.
0x4	SMW_ATTR_ALGO_CHACHA20	ChaCha20-Poly1305.
0x5	SMW_ATTR_ALGO_SM4	ShāngMi 4.
0x6	SMW_ATTR_ALGO_RSA	Rivest-Shamir-Adleman.
0x7	SMW_ATTR_ALGO_SM2	ShāngMi 2.
0x8	SMW_ATTR_ALGO_HMAC	Hash-based message authentication code.
0x9	SMW_ATTR_ALGO_DSA	Digital Signature Algorithm.
0xA	SMW_ATTR_ALGO_ECDSA	Elliptic Curve Digital Signature Algorithm.
0xB	SMW_ATTR_ALGO_EDDSA	Edwards-curve Digital Signature Algorithm.
0xC	SMW_ATTR_ALGO_DH	Diffie-Hellman.
0xD	SMW_ATTR_ALGO_ECDH	Elliptic-curve Diffie-Hellman.
0xE	SMW_ATTR_ALGO_HKDF	HMAC-based Key Derivation Function.
0xF	SMW_ATTR_ALGO_HKDF_EXTRACT	HMAC-based Key Derivation Function Extract step.
0x10	SMW_ATTR_ALGO_HKDF_EXPAND	HMAC-based Key Derivation Function Expand step.
0x11	SMW_ATTR_ALGO_TLS_1_2	Transport Layer Security 1.2 Key Derivation.
0x12	SMW_ATTR_ALGO_TLS_1_3	Transport Layer Security 1.3 Key Derivation.
0x80	SMW_ATTR_ALGO_CKDF	Custom Key Derivation Function.
0xFF	SMW_ATTR_ALGO_HASH	Hash.

6.1.9.2.1 Macro

6.1.9.2.1.1 macro SMW_ATTR_GET_ALGO

SMW_ATTR_GET_ALGO(*algo*)

Get the main algorithm.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro extracts the main algorithm of *algo*.

Return

The main algorithm.

6.1.9.3 Mode

Table 141. Algorithm Mode value

Value	Define	Description
0x0	SMW_ATTR_MODE_NONE	No mode defined.
0x1	SMW_ATTR_MODE_ECB_NO_PAD	Electronic Code Book No Padding.
0x2	SMW_ATTR_MODE_CBC_NO_PAD	Cipher block chaining No Padding.
0x3	SMW_ATTR_MODE_CFB	Ciphertext Feedback.
0x4	SMW_ATTR_MODE_CTR	Counter Mode.
0x5	SMW_ATTR_MODE_CTS	Ciphertext Stealing.
0x6	SMW_ATTR_MODE_OFB	Output Feedback.
0x7	SMW_ATTR_MODE_XTS	XEX Tweakable Block Ciphertext Stealing.
0x8	SMW_ATTR_MODE_CCM	Counter with CBC-MAC Mode.
0x9	SMW_ATTR_MODE_GCM	Galois/Counter Mode.
0xA	SMW_ATTR_MODE_PKCS1_1_5	Public-Key Cryptography Standards 1.5.
0xB	SMW_ATTR_MODE_OAEP	Optimal Asymmetric Encryption Padding.
0xC	SMW_ATTR_MODE_PSS	Probabilistic Signature Scheme.
0xD	SMW_ATTR_MODE_PKCS5	Password-Based Cryptography Specification.
0xE	SMW_ATTR_MODE_CMAC	Cipher-based Message Authentication Code.
0xF	SMW_ATTR_MODE_POLY1305	Poly1305-AES.
0x10	SMW_ATTR_MODE_CLIENT	Client (TLS 1.2).
0x11	SMW_ATTR_MODE_SERVER	Server (TLS 1.2).
0x12	SMW_ATTR_MODE_NO_PAD	Asymmetric Encryption with no padding.
0xFF	SMW_ATTR_MODE_ANY	Any mode.

6.1.9.3.1 Macros

6.1.9.3.1.1 macro SMW_ATTR_GET_MODE

SMW_ATTR_GET_MODE(*algo*)

Get the mode.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro extracts the mode of *algo*.

Return

The mode.

6.1.9.3.1.2 macro SMW_ATTR_SET_MODE

SMW_ATTR_SET_MODE(*algo*, *mode*)

Set the algorithm mode.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).
- **mode** - A valid algorithm mode. See [typedef smw_attr_algo_t](#).

Description

This macro sets the algorithm mode of *algo*.

Return

A valid algorithm.

6.1.9.4 Curve

Table 142. Algorithm Curve value

Value	Define	Description
0x0	SMW_ATTR_CURVE_NONE	No curve defined.
0x1	SMW_ATTR_CURVE_SECP_R1	Secp R1 curve (aka NIST P).
0x2	SMW_ATTR_CURVE_BRAINPOOL_R1	Brainpool R1 curve.
0x3	SMW_ATTR_CURVE_BRAINPOOL_T1	Brainpool T1 curve.
0x4	SMW_ATTR_CURVE_ED25519	Twisted Edwards25519.
0x5	SMW_ATTR_CURVE_ED448	Twisted Edwards448.
0xFF	SMW_ATTR_CURVE_ANY	Any curve.

6.1.9.4.1 Macro

6.1.9.4.1.1 macro SMW_ATTR_GET_CURVE

SMW_ATTR_GET_CURVE(*algo*)

Get the curve.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro extracts the curve of *algo*.

Return

The curve.

6.1.9.5 Key Derivation Function

Table 143. Algorithm Key Derivation Function value

Value	Define	Description
0xE	SMW_ATTR_ALGO_HKDF	HMAC Key derivation function

6.1.9.5.1 Macro

6.1.9.5.1.1 macro SMW_ATTR_GET_KDF

SMW_ATTR_GET_KDF(*algo*)

Get the key agreement's key derivation algorithm.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro extracts the key derivation of key agreement *algo*.

Return

The curve.

6.1.9.6 Hash

Table 144. Algorithm Hash value

Value	Define	Description
0x0	SMW_ATTR_HASH_NONE	No hash algorithm defined.
0x1	SMW_ATTR_HASH_MD5	Message Digest 5.
0x2	SMW_ATTR_HASH_SHA1	Secure Hash Algorithm 1.
0x3	SMW_ATTR_HASH_SHA224	Secure Hash Algorithm 2, 224 bits.
0x4	SMW_ATTR_HASH_SHA256	Secure Hash Algorithm 2, 256 bits.
0x5	SMW_ATTR_HASH_SHA384	Secure Hash Algorithm 2, 384 bits.
0x6	SMW_ATTR_HASH_SHA512	Secure Hash Algorithm 2, 512 bits.
0x7	SMW_ATTR_HASH_SHA3_224	Secure Hash Algorithm 3, 224 bits.
0x8	SMW_ATTR_HASH_SHA3_256	Secure Hash Algorithm 3, 256 bits.
0x9	SMW_ATTR_HASH_SHA3_384	Secure Hash Algorithm 3, 384 bits.
0xA	SMW_ATTR_HASH_SHA3_512	Secure Hash Algorithm 3, 512 bits.
0xB	SMW_ATTR_HASH_SM3	ShangMi 3.
0xC	SMW_ATTR_HASH_SHAKE128	Shake128.
0xD	SMW_ATTR_HASH_SHAKE256	Shake256.
0xFF	SMW_ATTR_HASH_ANY	Any hash algorithm.

6.1.9.6.1 Macros

6.1.9.6.1.1 macro SMW_ATTR_GET_HASH

SMW_ATTR_GET_HASH(*algo*)

Get the hash algorithm.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro extracts the hash algorithm of *algo*.

Return

The hash algorithm.

6.1.9.6.1.2 macro SMW_ATTR_SET_HASH

SMW_ATTR_SET_HASH(*algo*, *hash*)

Set the hash algorithm.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).
- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro sets the hash algorithm of *algo*.

Return

A valid algorithm.

6.1.9.7 Class

Table 145. Algorithm Operation value

Value	Define	Description
0x0	SMW_ATTR_CLASS_NONE	No class of operation defined.
0x1	SMW_ATTR_CLASS_DIGEST	Digest calculation.
0x2	SMW_ATTR_CLASS_SYMMETRIC_ENCRYPTION	Symmetric encryption.
0x3	SMW_ATTR_CLASS_ASYMMETRIC_ENCRYPTION	Asymmetric encryption.
0x4	SMW_ATTR_CLASS_ASYMMETRIC_SIGNATURE	Asymmetric signature.
0x5	SMW_ATTR_CLASS_MAC	Message Authentication Code.
0x6	SMW_ATTR_CLASS_AEAD	Authenticated Encryption with Associated Data.
0x7	SMW_ATTR_CLASS_KEY_DERIVATION	Key derivation.

Table 145. Algorithm Operation value...continued

Value	Define	Description
0x8	SMW_ATTR_CLASS_KEY_ATTESTATION	Key attestation.
0x9	SMW_ATTR_CLASS_KEY_AGREEMENT	Key agreement.

6.1.9.7.1 Macros

6.1.9.7.1.1 macro SMW_ATTR_GET_CLASS

SMW_ATTR_GET_CLASS(*algo*)

Get the class.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro extracts the class of *algo*.

Return

The class.

6.1.9.7.1.2 macro SMW_ATTR_ALGO_DIGEST

SMW_ATTR_ALGO_DIGEST(*hash*)

Build a digest algorithm.

Parameters

- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro builds a digest algorithm using the *hash* algorithm.

Return

A valid digest algorithm.

6.1.9.7.1.3 macro SMW_ATTR_ALGO_SYMMETRIC_ENCRYPTION

SMW_ATTR_ALGO_SYMMETRIC_ENCRYPTION(*algo*, *mode*)

Build a symmetric encryption algorithm.

Parameters

- **algo** - A valid main algorithm for symmetric encryption. See [typedef smw_attr_algo_t](#).
- **mode** - A valid mode. See [typedef smw_attr_algo_t](#).

Description

This macro builds a symmetric encryption algorithm given the main algorithm `algo` and the `mode`.

Return

A valid symmetric encryption algorithm.

6.1.9.7.1.4 macro SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_ECDSA

SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_ECDSA(*curve*, *hash*)

Build an asymmetric signature ECDSA algorithm.

Parameters

- **curve** - A valid curve. See [typedef smw_attr_algo_t](#).
- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro builds an asymmetric signature ECDSA algorithm given the `curve` and the `hash` algorithm.

Return

A valid asymmetric signature ECDSA algorithm.

6.1.9.7.1.5 macro SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_EDDSA

SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_EDDSA(*curve*, *hash*, *param*)

Build an asymmetric signature EDDSA algorithm.

Parameters

- **curve** - A valid curve. See [typedef smw_attr_algo_t](#).
- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).
- **param** - A valid EDDSA parameter algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro builds an asymmetric signature EDDSA algorithm given the `curve`, the `hash` algorithm and the `param` parameter.

The `hash` algorithm function of the signature scheme and may not be used.

Return

A valid asymmetric signature EDDSA algorithm.

6.1.9.7.1.6 macro SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_DSA

SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_DSA(*hash*)

Build an asymmetric signature DSA algorithm.

Parameters

- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro builds an asymmetric signature DSA algorithm given the `hash` algorithm.

Return

A valid asymmetric signature DSA algorithm.

6.1.9.7.1.7 macro SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_RSA

SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_RSA(*mode, hash, salt*)

Build an asymmetric signature RSA algorithm.

Parameters

- **mode** - A valid mode. See [typedef smw_attr_algo_t](#).
- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).
- **salt** - Salt length.

Description

This macro builds an asymmetric signature RSA algorithm given the `mode`, the `hash` algorithm and the `salt` length.

Return

A valid asymmetric signature RSA algorithm.

6.1.9.7.1.8 macro SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_NO_LABEL

SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_NO_LABEL(*hash*)

Build an asymmetric signature TLS1.2 algorithm without label.

Parameters

- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro builds an asymmetric signature TLS1.2 algorithm given the `hash` algorithm.

Return

A valid asymmetric signature TLS1.2 algorithm.

6.1.9.7.1.9 macro SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_CLIENT

SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_CLIENT(*hash*)

Build an asymmetric signature TLS1.2 algorithm with label *CLIENT*.

Parameters

- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro builds an asymmetric signature TLS1.2 algorithm with *CLIENT* label given the `hash` algorithm.

Return

A valid asymmetric signature TLS1.2 algorithm with label *CLIENT*.

6.1.9.7.1.10 macro SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_SERVER

SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_SERVER(*hash*)

Build an asymmetric signature TLS1.2 algorithm with label *SERVER*.

Parameters

- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro builds an asymmetric signature TLS1.2 algorithm with *SERVER* label given the `hash` algorithm.

Return

A valid asymmetric signature TLS1.2 algorithm with label *SERVER*.

6.1.9.7.1.11 macro SMW_ATTR_ALGO_MAC

SMW_ATTR_ALGO_MAC(*algo, mode, mac*)

Build a MAC algorithm.

Parameters

- **algo** - A valid main algorithm. See [typedef smw_attr_algo_t](#).
- **mode** - A valid mode. See [typedef smw_attr_algo_t](#).
- **mac** - MAC length.

Description

This macro builds a MAC algorithm given the main algorithm `algo`, the `mode` and the `hash` algorithm. The main algorithm `algo` is not HMAC.

Return

A valid MAC algorithm.

6.1.9.7.1.12 macro SMW_ATTR_ALGO_MAC_HMAC

SMW_ATTR_ALGO_MAC_HMAC(*hash, mac*)

Build a HMAC algorithm.

Parameters

- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).
- **mac** - MAC length.

Description

This macro builds a HMAC algorithm given the `hash` algorithm and the `mac` length.

Return

A valid HMAC algorithm.

6.1.9.7.1.13 macro SMW_ATTR_ALGO_AEAD

SMW_ATTR_ALGO_AEAD(*algo, mode, tag*)

Build an AEAD algorithm.

Parameters

- **algo** - A valid main algorithm. See [typedef smw_attr_algo_t](#).
- **mode** - A valid mode. See [typedef smw_attr_algo_t](#).
- **tag** - Tag length.

Description

This macro builds an AEAD algorithm given the main algorithm `algo`, the `mode` and the `tag` length.

Return

A valid AEAD algorithm.

6.1.9.7.1.14 macro SMW_ATTR_ALGO_KEY_DERIVATION_HKDF

SMW_ATTR_ALGO_KEY_DERIVATION_HKDF(*hash*)

Build an HKDF key derivation algorithm.

Parameters

- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro builds an HKDF key derivation algorithm given the `hash` algorithm.

Return

A valid HKDF key derivation algorithm.

6.1.9.7.1.15 macro SMW_ATTR_ALGO_KEY_DERIVATION_DH

SMW_ATTR_ALGO_KEY_DERIVATION_DH(*void*)

Build the DH key derivation algorithm.

Parameters

- **void** - no arguments

Description

This macro builds the DH key derivation algorithm.

Return

DH key derivation algorithm.

6.1.9.7.1.16 macro SMW_ATTR_ALGO_KEY_DERIVATION_ECDH

SMW_ATTR_ALGO_KEY_DERIVATION_ECDH(*void*)

Build the ECDH key derivation algorithm.

Parameters

- **void** - no arguments

Description

This macro builds the ECDH key derivation algorithm.

Return

ECDH key derivation algorithm.

6.1.9.7.1.17 macro SMW_ATTR_ALGO_KEY_DERIVATION_EDDSA

SMW_ATTR_ALGO_KEY_DERIVATION_EDDSA(*curve*)

Build an EDDSA key derivation algorithm.

Parameters

- **curve** - A valid curve. See [typedef smw_attr_algo_t](#).

Description

This macro builds an EDDSA key derivation algorithm given the *curve*.

Return

A valid EDDSA key derivation algorithm.

6.1.9.7.1.18 macro SMW_ATTR_ALGO_KEY_DERIVATION_TLS12

SMW_ATTR_ALGO_KEY_DERIVATION_TLS12(*hash*)

Build the TLS 1.2 derivation algorithm.

Parameters

- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro builds the TLS 1.2 key derivation algorithm.

Return

TLS 1.2 key derivation algorithm.

6.1.9.7.1.19 macro `SMW_ATTR_ALGO_KEY_DERIVATION_TLS13`

SMW_ATTR_ALGO_KEY_DERIVATION_TLS13(*hash*)

Build the TLS 1.3 derivation algorithm.

Parameters

- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro builds the TLS 1.3 key derivation algorithm.

Return

TLS 1.3 key derivation algorithm.

6.1.9.7.1.20 macro `SMW_ATTR_ALGO_KEY_ATTESTATION_MAC`

SMW_ATTR_ALGO_KEY_ATTESTATION_MAC(*algo, mode, mac*)

Build a MAC key attestation algorithm.

Parameters

- **algo** - A valid main algorithm. See [typedef smw_attr_algo_t](#).
- **mode** - A valid mode. See [typedef smw_attr_algo_t](#).
- **mac** - MAC length.

Description

This macro builds a MAC key attestation algorithm given the main algorithm `algo`, the `mode` and the `mac` length.

Return

A valid MAC key attestation algorithm.

6.1.9.7.1.21 macro `SMW_ATTR_ALGO_KEY_ATTESTATION_ECDSA`

SMW_ATTR_ALGO_KEY_ATTESTATION_ECDSA(*curve, hash*)

Build an ECDSA key attestation algorithm.

Parameters

- **curve** - A valid curve. See [typedef smw_attr_algo_t](#).
- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro builds an ECDSA key attestation algorithm given the `curve` and the `hash` algorithm.

Return

A valid ECDSA key attestation algorithm.

6.1.9.7.1.22 macro SMW_ATTR_ALGO_KEY_AGREEMENT

SMW_ATTR_ALGO_KEY_AGREEMENT(*algo, kdf, hash*)

Build a key agreement algorithm.

Parameters

- **algo** - Key agreement algorithm. See [typedef smw_attr_algo_t](#).
- **kdf** - Combined key derivation algorithm. See [typedef smw_attr_algo_t](#).
- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro builds a key agreement with or without combined key derivation algorithm.

Return

A valid key agreement algorithm with or without combined key derivation.

6.1.9.7.1.23 macro SMW_ATTR_ALGO_ASYMMETRIC_ENCRYPTION_RSA

SMW_ATTR_ALGO_ASYMMETRIC_ENCRYPTION_RSA(*mode, hash*)

Build an RSA asymmetric encryption algorithm.

Parameters

- **mode** - A valid mode. See [typedef smw_attr_algo_t](#).
- **hash** - A valid hash algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro builds an RSA asymmetric encryption algorithm given the `mode` and the `hash` algorithm.

Return

A valid RSA asymmetric encryption algorithm.

6.1.9.8 Additional Parameters: Salt length

The Salt length parameter defines the number of bytes used as RSA PSS salt output length in RSA PSS signature operations.

Table 146. Additional Algorithm Salt length bitmask value

Parameter	Bit[39]	Bits[38:32]	Comment
Salt length bytes	0x0	0 <= len <= SaltLen	

Table 146. Additional Algorithm Salt length bitmask value...continued

Parameter	Bit[39]	Bits[38:32]	Comment
Min Salt length	0x1	0 <= len <= SaltLen	Use for the key permitted algorithm definition only.

The `SaltLen` is the maximum salt length in bytes that can be used with the RSA PSS algorithm.

The [RFC8017](#) standard defines the RSA PSS signature scheme. The maximum `SaltLen` parameter is defined in the RFC8017, section 9.1.1 by the formula:

$$sLen \leq emLen - hLen - 2$$

Where `emLen` is the length in bytes of the RSA encoded message, and `hLen` is the length in bytes of the hash algorithm output.

The typical salt lengths are the hash algorithm output lengths in bytes.

The Bit[39] restricts the key usage to the RSA PSS algorithm with a Salt length of at least the `SaltLen` bytes size. `SaltLen` can't be 0.

If this additional parameter is not set, the default salt length is the selected operation hash algorithm output length.

6.1.9.8.1 Macros

6.1.9.8.1.1 macro SMW_ATTR_SET_SALT_LENGTH

SMW_ATTR_SET_SALT_LENGTH(*algo*, *length*)

Set salt length.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).
- **length** - Salt length associated to `algo`.

Description

This macro sets the salt length.

Return

A valid algorithm with salt length bits set to `length`.

6.1.9.8.1.2 macro SMW_ATTR_GET_SALT_LENGTH

SMW_ATTR_GET_SALT_LENGTH(*algo*)

Get salt length.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro returns the salt length value set for `algo`.

Return

Length set for `algo`.

6.1.9.8.1.3 macro `SMW_ATTR_SET_MIN_SALT_LENGTH`

`SMW_ATTR_SET_MIN_SALT_LENGTH(algo, length)`

Set min salt length.

Parameters

- `algo` - A valid algorithm. See [typedef `smw\_attr\_algo\_t`](#).
- `length` - Salt length associated to `algo`.

Description

This macro sets the min salt length.

Return

A valid algorithm with min salt length bits set to `length`.

6.1.9.8.1.4 macro `SMW_ATTR_IS_MIN_SALT_LENGTH`

`SMW_ATTR_IS_MIN_SALT_LENGTH(algo)`

Whether the min salt length bit is set.

Parameters

- `algo` - A valid algorithm. See [typedef `smw\_attr\_algo\_t`](#).

Description

This macro returns whether the min salt length bit is set or not.

Return

1 if the min salt length bit is set, 0 otherwise.

6.1.9.9 Additional Parameters: MAC length

The MAC length parameter defines the number of bytes used as MAC output length in MAC. It's used to truncate the MAC output to a shorter length.

Table 147. Additional Algorithm MAC length bitmask value

Parameter	Bit[39]	Bits[38:32]	Comment
MAC length bytes	0x0	0 <= len <= MacLen	
Min MAC length bytes	0x1	0 <= len <= MacLen	Use for the key permitted algorithm

The `MacLen` value depends on the selected MAC algorithm:

- For HMAC algorithms, `MacLen` is the hash output length. Refer to [Digest \(Hash\)](#) chapter.
- For CMAC algorithms, `MacLen` is the block size (16 bytes) of the AES cipher.

The Bit[39] restricts the key usage to the MAC algorithm with a MAC length of at least the `MacLen` bytes size. `MacLen` can't be 0.

If this additional algorithm parameter is not set, the default MAC length is the full MAC output length:

- For HMAC algorithms, the MAC output length is the hash output length.
- For CMAC algorithms, the MAC output length is the block size of the underlying cipher.

6.1.9.9.1 Macros

6.1.9.9.1.1 macro `SMW_ATTR_SET_MAC_LENGTH`

`SMW_ATTR_SET_MAC_LENGTH(algo, length)`

Set MAC length.

Parameters

- `algo` - A valid algorithm. See [typedef `smw\_attr\_algo\_t`](#).
- `length` - MAC length associated to `algo`.

Description

This macro sets the MAC length.

Return

A valid algorithm with MAC length bits set to `length`.

6.1.9.9.1.2 macro `SMW_ATTR_GET_MAC_LENGTH`

`SMW_ATTR_GET_MAC_LENGTH(algo)`

Get MAC length.

Parameters

- `algo` - A valid algorithm. See [typedef `smw\_attr\_algo\_t`](#).

Description

This macro returns the MAC length value set for `algo`.

Return

Length set for `algo`.

6.1.9.9.1.3 macro `SMW_ATTR_SET_MIN_MAC_LENGTH`

`SMW_ATTR_SET_MIN_MAC_LENGTH(algo, length)`

Set min MAC length.

Parameters

- `algo` - A valid algorithm. See [typedef `smw\_attr\_algo\_t`](#).
- `length` - MAC length associated to `algo`.

Description

This macro sets the min MAC length.

Return

A valid algorithm with min MAC length bits set to `length`.

6.1.9.9.1.4 macro SMW_ATTR_IS_MIN_MAC_LENGTH

SMW_ATTR_IS_MIN_MAC_LENGTH(*algo*)

Whether the min MAC length bit is set.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro returns whether the min MAC length bit is set or not.

Return

1 if the min MAC length bit is set, 0 otherwise.

6.1.9.10 Additional Parameters: Tag length

The Tag length parameter defines the number of bytes used as AEAD output Tag length in AEAD. It's used to truncate the AEAD Tag length to a shorter length.

Table 148. Additional Algorithm Tag length bitmask value

Parameter	Bit[39]	Bits[38:32]	Comment
Tag length bytes	0x0	0 <= len <= TagLen	
Min Tag length bytes	0x1	0 <= len <= TagLen	Use for the key permitted algorithm definition only.

The `TagLen` is the maximum tag length supported by the AEAD algorithm.

The AEAD CCM supports authentication tag sizes of 4, 6, 8, 10, 12, 14, and 16 bytes. The default tag length is 16 bytes.

Refer to the [RFC3610](#) for more details on AEAD CCM tag length specifications.

The AEAD GCM supports authentication tag sizes of 4, 8, 12, 13, 14, 15, and 16 bytes. The default tag length is 16 bytes.

Refer to the [NIST800-38D](#) for more details on AEAD GCM tag length specifications.

The Bit[39] restricts the key usage to the AEAD algorithm with a Tag length of at least the `TagLen` bytes size. `TagLen` can't be 0.

If this additional algorithm parameter is not set, the default Tag length is the full Tag output length which is 16 bytes.

6.1.9.10.1 Macros

6.1.9.10.1.1 macro SMW_ATTR_SET_TAG_LENGTH

SMW_ATTR_SET_TAG_LENGTH(*algo*, *length*)

Set tag length.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).
- **length** - Tag length associated to *algo*.

Description

This macro sets the tag length.

Return

A valid algorithm with tag length bits set to *length*.

6.1.9.10.1.2 macro SMW_ATTR_GET_TAG_LENGTH

SMW_ATTR_GET_TAG_LENGTH(*algo*)

Get tag length.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro returns the tag length value set for *algo*.

Return

Length set for *algo*.

6.1.9.10.1.3 macro SMW_ATTR_SET_MIN_TAG_LENGTH

SMW_ATTR_SET_MIN_TAG_LENGTH(*algo*, *length*)

Set min tag length.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).
- **length** - Tag length associated to *algo*.

Description

This macro sets the min tag length.

Return

A valid algorithm with min tag length bits set to *length*.

6.1.9.10.1.4 macro SMW_ATTR_IS_MIN_TAG_LENGTH

SMW_ATTR_IS_MIN_TAG_LENGTH(*algo*)

Whether the min tag length bit is set.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro returns whether the min tag length bit is set or not.

Return

1 if the min tag length bit is set, 0 otherwise.

6.1.9.11 Additional Parameters: Signature Message

The Signature Message parameter defines the input message type for asymmetric signature operations. It specifies whether the input is a full message or a pre-computed hash digest.

Table 149. Additional Algorithm Signature Message bitmask value

Parameter	Bit[39]	Bits[38:32]
Signature Message Full	0x0	-
Signature Message Hashed	0x1	-

6.1.9.11.1 Macros

6.1.9.11.1.1 macro SMW_ATTR_SET_MSG_HASHED

SMW_ATTR_SET_MSG_HASHED(*algo*)

Set Asymmetric signature message hashed flag.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro sets the signature flag indicating input message is hashed.

Return

A valid algorithm with signature message flag set.

6.1.9.11.1.2 macro SMW_ATTR_IS_MSG_HASHED

SMW_ATTR_IS_MSG_HASHED(*algo*)

Whether signature message hashed flag is set.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro returns whether the signature input message hashed flag is set or not.

Return

1 if the message hashed flag is set, 0 otherwise.

6.1.9.12 Additional Parameters: EdDSA Signature

The EDDSA Signature parameter defines the signature scheme variant for Edwards-curve Digital Signature Algorithm operations. It specifies whether the signature uses the pre-hashed PureEDDSA variant or the context variant.

Table 150. Additional Algorithm EdDSA Signature bitmask value

Parameter	Bit[39]	Bits[38:32]
EDDSA pre-hashed	-	0x1
EDDSA context	-	0x2

6.1.9.12.1 Macros

6.1.9.12.1.1 macro SMW_ATTR_SET_SIGN_EDDSA_PREHASHED

SMW_ATTR_SET_SIGN_EDDSA_PREHASHED(*algo*)

Set EDDSA pre-hashed signature.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro sets the asymmetric EDDSA signature parameter pre-hashed.

Return

A valid algorithm with EDDSA signature parameter pre-hashed set.

6.1.9.12.1.2 macro SMW_ATTR_IS_SIGN_EDDSA_PREHASHED

SMW_ATTR_IS_SIGN_EDDSA_PREHASHED(*algo*)

Whether the signature EDDSA is pre-hashed.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro returns whether the asymmetric EDDSA signature parameter pre-hashed is set or not.

Return

1 if the signature message is pre-hashed type, 0 otherwise.

6.1.9.12.1.3 macro SMW_ATTR_SET_SIGN_EDDSA_CONTEXT

SMW_ATTR_SET_SIGN_EDDSA_CONTEXT(*algo*)

Set EDDSA context signature.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro sets the asymmetric EDDSA signature parameter context.

Return

A valid algorithm with EDDSA signature parameter context set.

6.1.9.12.1.4 macro SMW_ATTR_IS_SIGN_EDDSA_CONTEXT

SMW_ATTR_IS_SIGN_EDDSA_CONTEXT(*algo*)

Whether the EDDSE signature uses a context.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro returns whether the asymmetric EDDSA signature parameter context is set or not.

Return

1 if the signature message uses a context, 0 otherwise.

6.1.9.13 Additional Parameter: generic length

6.1.9.13.1 Macros

6.1.9.13.1.1 macro SMW_ATTR_SET_LENGTH

SMW_ATTR_SET_LENGTH(*algo*, *length*)

Set length.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).
- **length** - Length associated to *algo*.

Description

This macro sets the length.

Return

A valid algorithm with length bits set to `length`.

6.1.9.13.1.2 macro `SMW_ATTR_GET_LENGTH`

`SMW_ATTR_GET_LENGTH(algo)`

Get length.

Parameters

- **`algo`** - A valid algorithm. See [typedef `smw\_attr\_algo\_t`](#).

Description

This macro returns the length value set for `algo`.

Return

Length set for `algo`.

6.1.9.13.1.3 macro `SMW_ATTR_SET_MIN_LENGTH`

`SMW_ATTR_SET_MIN_LENGTH(algo, length)`

Set min length.

Parameters

- **`algo`** - A valid algorithm. See [typedef `smw\_attr\_algo\_t`](#).
- **`length`** - Length associated to `algo`.

Description

This macro sets the min length.

Return

A valid algorithm with min length bits set to `length`.

6.1.9.13.1.4 macro `SMW_ATTR_IS_MIN_LENGTH`

`SMW_ATTR_IS_MIN_LENGTH(algo)`

Whether the min length bit is set.

Parameters

- **`algo`** - A valid algorithm. See [typedef `smw\_attr\_algo\_t`](#).

Description

This macro returns whether the min length bit is set or not.

Return

1 if the min length bit is set, 0 otherwise.

6.1.9.14 Additional Parameter: generic signature

The following defines can be used to set or get signature parameters in the algorithm attributes.

Table 151. Additional Algorithm Signature Parameter bitmask value

Define	Description
SMW_ATTR_SIGN_PARAM_EDDSA_NONE	Pure-EdDSA Signature
SMW_ATTR_SIGN_PARAM_EDDSA_PREHASHED	EdDSA Pre-hashed Signature
SMW_ATTR_SIGN_PARAM_EDDSA_CONTEXT	EdDSA Context Signature
SMW_ATTR_SIGN_HASHED_FLAG	Message to sign is hashed

6.1.9.14.1 Macros

6.1.9.14.1.1 macro SMW_ATTR_GET_SIGN_PARAM

SMW_ATTR_GET_SIGN_PARAM(*algo*)

Get Asymmetric signature parameter.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).

Description

This macro returns the signature parameter set for *algo*.

Return

Signature parameter set for *algo*.

6.1.9.14.1.2 macro SMW_ATTR_SET_SIGN_PARAM

SMW_ATTR_SET_SIGN_PARAM(*algo*, *param*)

Set Asymmetric signature parameter.

Parameters

- **algo** - A valid algorithm. See [typedef smw_attr_algo_t](#).
- **param** - Parameter associated to *algo*.

Description

This macro sets the signature parameter.

Return

A valid algorithm with signature parameter set to *param*.

6.1.10 Objects Attributes definitions

Objects Attributes definitions provide bitmasks and macros to manage attributes associated to keys and data objects.

Macros are provided to set, get, and test attribute values.

A key is defined by the following attributes:

- [Permitted algorithm](#): Defines which algorithm(s) can use this key.
- [Usage restriction](#): Defines how the key can be used.
- [Object Attributes](#):
 - Persistence: Defines if the key is permanent, persistent or transient.
 - Lifecycle: Defines in which device lifecycle(s) the object is usable.
 - Sensitivity: Defines if the object is sensitive (plain text retrieval restricted).
- [Storage location](#): Defines where the object is stored.

A data object is defined by the following attributes:

- [Object Attributes](#):
 - Persistence: Defines if the data object is permanent, persistent or transient.
 - Lifecycle: Defines in which device lifecycle(s) the object is usable.
 - Sensitivity: Defines if the object is sensitive (plain text retrieval restricted).
- [Storage location](#): Defines where the object is stored.

6.1.10.1 Permitted algorithm

Definition of the key permitted algorithm restriction if supported by the subsystem.

Refer to the [Cryptographic Algorithm \(smw_attr_algo_t\) encoding](#), the 64-bit encoding is same as the cryptographic 64-bit algorithm encoding format with some exception detailed in the notes.

6.1.10.2 Usage restriction

Definition of the key usage restriction if supported by the subsystem.

6.1.10.2.1 Typedef

6.1.10.2.1.1 typedef smw_attr_usage_t

type **smw_attr_usage_t**

Key usages

Description

32-bits field used to define the permitted usages.

Table 152. Key Usage bitmask value

Value	Define	Description
0x00000000	SMW_ATTR_USAGE_NONE	No key usage defined.
0x00000001	SMW_ATTR_USAGE_CACHE	Permission to cache the key.
0x00000002	SMW_ATTR_USAGE_COPY	Permission to copy the key.

Table 152. Key Usage bitmask value...continued

Value	Define	Description
0x00000004	SMW_ATTR_USAGE_EXPORT	Permission to export the key.
0x00000008	SMW_ATTR_USAGE_ENCRYPT	Permission to encrypt a message with the key.
0x00000010	SMW_ATTR_USAGE_DECRYPT	Permission to decrypt a message with the key.
0x00000020	SMW_ATTR_USAGE_SIGN_MESSAGE	Permission to sign a message with the key.
0x00000040	SMW_ATTR_USAGE_VERIFY_MESSAGE	Permission to verify a message signature with the key.
0x00000080	SMW_ATTR_USAGE_SIGN_HASH	Permission to sign a message hash with the key.
0x00000100	SMW_ATTR_USAGE_VERIFY_HASH	Permission to verify a message hash with the key.
0x00000200	SMW_ATTR_USAGE_DERIVE	Permission to derive other keys from this key.

6.1.10.2.2 Macros

6.1.10.2.2.1 macro SMW_ATTR_USAGE_SET_CACHE

SMW_ATTR_USAGE_SET_CACHE(usage)

Set cache operation.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro sets cache operation as permitted usage.

Return

A valid usage.

6.1.10.2.2.2 macro SMW_ATTR_USAGE_SET_COPY

SMW_ATTR_USAGE_SET_COPY(usage)

Set copy operation.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro sets copy operation as permitted usage.

Return

A valid usage.

6.1.10.2.2.3 macro SMW_ATTR_USAGE_SET_EXPORT

SMW_ATTR_USAGE_SET_EXPORT(*usage*)

Set export operation.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro sets export operation as permitted usage.

Return

A valid usage.

6.1.10.2.2.4 macro SMW_ATTR_USAGE_SET_ENCRYPT

SMW_ATTR_USAGE_SET_ENCRYPT(*usage*)

Set encrypt operation.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro sets encrypt operation as permitted usage.

Return

A valid usage.

6.1.10.2.2.5 macro SMW_ATTR_USAGE_SET_DECRYPT

SMW_ATTR_USAGE_SET_DECRYPT(*usage*)

Set decrypt operation.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro sets decrypt operation as permitted usage.

Return

A valid usage.

6.1.10.2.2.6 macro SMW_ATTR_USAGE_SET_SIGN_MESSAGE

SMW_ATTR_USAGE_SET_SIGN_MESSAGE(*usage*)

Set sign message operation.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro sets sign message operation as permitted usage.

Return

A valid usage.

6.1.10.2.2.7 macro SMW_ATTR_USAGE_SET_VERIFY_MESSAGE

SMW_ATTR_USAGE_SET_VERIFY_MESSAGE(usage)

Set verify message operation.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro sets verify message operation as permitted usage.

Return

A valid usage.

6.1.10.2.2.8 macro SMW_ATTR_USAGE_SET_SIGN_HASH

SMW_ATTR_USAGE_SET_SIGN_HASH(usage)

Set sign hash operation.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro sets sign hash operation as permitted usage.

Return

A valid usage.

6.1.10.2.2.9 macro SMW_ATTR_USAGE_SET_VERIFY_HASH

SMW_ATTR_USAGE_SET_VERIFY_HASH(usage)

Set verify hash operation.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro sets verify hash operation as permitted usage.

Return

A valid usage.

6.1.10.2.2.10 macro SMW_ATTR_USAGE_SET_DERIVE

SMW_ATTR_USAGE_SET_DERIVE(usage)

Set derive operation.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro sets derive operation as permitted usage.

Return

A valid usage.

6.1.10.2.2.11 macro SMW_ATTR_USAGE_IS_CACHE

SMW_ATTR_USAGE_IS_CACHE(usage)

Whether the cache operation is permitted.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro returns whether the cache operation is permitted or not.

Return

1 if the cache operation is permitted, 0 otherwise.

6.1.10.2.2.12 macro SMW_ATTR_USAGE_IS_COPY

SMW_ATTR_USAGE_IS_COPY(usage)

Whether the copy operation is permitted.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro returns whether the copy operation is permitted or not.

Return

1 if the copy operation is permitted, 0 otherwise.

6.1.10.2.2.13 macro SMW_ATTR_USAGE_IS_EXPORT

SMW_ATTR_USAGE_IS_EXPORT(*usage*)

Whether the export operation is permitted.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro returns whether the export operation is permitted or not.

Return

1 if the export operation is permitted, 0 otherwise.

6.1.10.2.2.14 macro SMW_ATTR_USAGE_IS_ENCRYPT

SMW_ATTR_USAGE_IS_ENCRYPT(*usage*)

Whether the encrypt operation is permitted.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro returns whether the encrypt operation is permitted or not.

Return

1 if the encrypt operation is permitted, 0 otherwise.

6.1.10.2.2.15 macro SMW_ATTR_USAGE_IS_DECRYPT

SMW_ATTR_USAGE_IS_DECRYPT(*usage*)

Whether the decrypt operation is permitted.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro returns whether the decrypt operation is permitted or not.

Return

1 if the decrypt operation is permitted, 0 otherwise.

6.1.10.2.2.16 macro SMW_ATTR_USAGE_IS_SIGN_MESSAGE

SMW_ATTR_USAGE_IS_SIGN_MESSAGE(*usage*)

Whether the sign message operation is permitted.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro returns whether the sign message operation is permitted or not.

Return

1 if the sign message operation is permitted, 0 otherwise.

6.1.10.2.2.17 macro SMW_ATTR_USAGE_IS_VERIFY_MESSAGE

SMW_ATTR_USAGE_IS_VERIFY_MESSAGE(*usage*)

Whether the verify message operation is permitted.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro returns whether the verify message operation is permitted or not.

Return

1 if the verify message operation is permitted, 0 otherwise.

6.1.10.2.2.18 macro SMW_ATTR_USAGE_IS_SIGN_HASH

SMW_ATTR_USAGE_IS_SIGN_HASH(*usage*)

Whether the sign hash operation is permitted.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro returns whether the sign hash operation is permitted or not.

Return

1 if the sign hash operation is permitted, 0 otherwise.

6.1.10.2.2.19 macro SMW_ATTR_USAGE_IS_VERIFY_HASH

SMW_ATTR_USAGE_IS_VERIFY_HASH(*usage*)

Whether the verify hash operation is permitted.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro returns whether the verify hash operation is permitted or not.

Return

1 if the verify hash operation is permitted, 0 otherwise.

6.1.10.2.20 macro SMW_ATTR_USAGE_IS_DERIVE**SMW_ATTR_USAGE_IS_DERIVE(usage)**

Whether the derive operation is permitted.

Parameters

- **usage** - A usage. See [typedef smw_attr_usage_t](#).

Description

This macro returns whether the derive operation is permitted or not.

Return

1 if the derive operation is permitted, 0 otherwise.

6.1.10.3 Object attributes

Definition of the object (key or data) attributes as Persistence, Lifecycle usage restriction (if supported by subsystem), ...

6.1.10.3.1 Typedef**6.1.10.3.1.1 typedef smw_attr_attributes_t**

type **smw_attr_attributes_t**

Object attributes

Description

32-bits field used to define the attributes.

Table 153. Object attributes bit field

Bits	Description
[31:19]	Reserved
[18]	Sensitive, object is sensitive, plain text value can't be retrieved.
[17:16]	Read/Write data object, not applicable to a key object.

Table 153. Object attributes bit field...continued

Bits	Description		
	Value	Define	Description
	0x00	SMW_ATTR_RW_FLAG_NONE	No restriction. Can be read and write.
	0x01	SMW_ATTR_RW_FLAG_READ_ONLY	Read only. Can't be overwritten.
	0x02	SMW_ATTR_RW_FLAG_READ_ONCE	Read only one time and it's removed after read.
[15:8]	Device Lifecycle when object is usable. It's a bitmask value. One or more lifecycle can be selected.		
	Value	Define	Description
	0x01	SMW_ATTR_LIFECYCLE_CURRENT	Current Lifecycle
	0x02	SMW_ATTR_LIFECYCLE_OPEN	OEM Open Lifecycle
	0x04	SMW_ATTR_LIFECYCLE_CLOSED	OEM Closed Lifecycle
	0x08	SMW_ATTR_LIFECYCLE_CLOSED_LOCKED	OEM Closed and Locked Lifecycle
[7:4]	Reserved		
[3:0]	Object persistency.		
	Value	Define	Description
	0x0	SMW_ATTR_PERSISTENCE_TRANSIENT	Object is transient. Does not resist a device reset.
	0x1	SMW_ATTR_PERSISTENCE_PERSISTENT	Object is persistent. Resists device reset.
	0x2	SMW_ATTR_PERSISTENCE_PERMANENT	Object is permanent. Can't be deleted.

6.1.10.3.2 Macros

6.1.10.3.2.1 macro SMW_ATTR_SET_PERSISTENCE

SMW_ATTR_SET_PERSISTENCE(*attr*, *persistence*)

Set persistence.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).
- **persistence** - Persistence.

Description

This macro sets *persistence*.

Return

Attributes

6.1.10.3.2.2 macro SMW_ATTR_GET_PERSISTENCE

SMW_ATTR_GET_PERSISTENCE(*attr*)

Get persistence.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro gets persistence.

Return

Persistence

6.1.10.3.2.3 macro SMW_ATTR_SET_TRANSIENT

SMW_ATTR_SET_TRANSIENT(*attr*)

Set transient persistence.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro sets transient persistence.

Return

Attributes

6.1.10.3.2.4 macro SMW_ATTR_SET_PERSISTENT

SMW_ATTR_SET_PERSISTENT(*attr*)

Set persistent persistence.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro sets persistent persistence.

Return

Attributes

6.1.10.3.2.5 macro SMW_ATTR_SET_PERMANENT

SMW_ATTR_SET_PERMANENT(*attr*)

Set permanent persistence.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro sets permanent persistence.

Return

Attributes

6.1.10.3.2.6 macro SMW_ATTR_IS_TRANSIENT**SMW_ATTR_IS_TRANSIENT(attr)**

Whether the persistence is transient.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro returns whether the persistence is transient or not.

Return

1 if the persistence is transient, 0 otherwise.

6.1.10.3.2.7 macro SMW_ATTR_IS_PERSISTENT**SMW_ATTR_IS_PERSISTENT(attr)**

Whether the persistence is persistent.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro returns whether the persistence is persistent or not.

Return

1 if the persistence is persistent, 0 otherwise.

6.1.10.3.2.8 macro SMW_ATTR_IS_PERMANENT**SMW_ATTR_IS_PERMANENT(attr)**

Whether the persistence is permanent.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro returns whether the persistence is permanent or not.

Return

1 if the persistence is permanent, 0 otherwise.

6.1.10.3.2.9 macro SMW_ATTR_SET_LC_CURRENT

SMW_ATTR_SET_LC_CURRENT(*attr*)

Set current lifecycle.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro sets current lifecycle.

Return

Attributes

6.1.10.3.2.10 macro SMW_ATTR_SET_LC_OPEN

SMW_ATTR_SET_LC_OPEN(*attr*)

Set open lifecycle.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro sets open lifecycle.

Return

Attributes

6.1.10.3.2.11 macro SMW_ATTR_SET_LC_CLOSED

SMW_ATTR_SET_LC_CLOSED(*attr*)

Set closed lifecycle.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro sets closed lifecycle.

Return

Attributes

6.1.10.3.2.12 macro SMW_ATTR_SET_LC_CLOSED_LOCKED

SMW_ATTR_SET_LC_CLOSED_LOCKED(*attr*)

Set closed-locked lifecycle.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro sets closed-locked lifecycle.

Return

Attributes

6.1.10.3.2.13 macro SMW_ATTR_IS_LC_CURRENT

SMW_ATTR_IS_LC_CURRENT(*attr*)

Whether the lifecycle is current.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro returns whether the lifecycle is current or not.

Return

1 if the lifecycle is current, 0 otherwise.

6.1.10.3.2.14 macro SMW_ATTR_IS_LC_OPEN

SMW_ATTR_IS_LC_OPEN(*attr*)

Whether the lifecycle is open.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro returns whether the lifecycle is open or not.

Return

1 if the lifecycle is open, 0 otherwise.

6.1.10.3.2.15 macro SMW_ATTR_IS_LC_CLOSED

SMW_ATTR_IS_LC_CLOSED(*attr*)

Whether the lifecycle is closed.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro returns whether the lifecycle is closed or not.

Return

1 if the lifecycle is closed, 0 otherwise.

6.1.10.3.2.16 macro SMW_ATTR_IS_LC_CLOSED_LOCKED

SMW_ATTR_IS_LC_CLOSED_LOCKED(*attr*)

Whether the lifecycle is closed-locked.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro returns whether the lifecycle is closed-locked or not.

Return

1 if the lifecycle is closed-locked, 0 otherwise.

6.1.10.3.2.17 macro SMW_ATTR_SET_READ_ONLY

SMW_ATTR_SET_READ_ONLY(*attr*)

Set read-only flag.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro sets read-only flag.

Return

Attributes

6.1.10.3.2.18 macro SMW_ATTR_SET_READ_ONCE

SMW_ATTR_SET_READ_ONCE(*attr*)

Set read-once flag.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro sets read-once flag.

Return

Attributes

6.1.10.3.2.19 macro SMW_ATTR_IS_READ_ONLY

SMW_ATTR_IS_READ_ONLY(attr)

Whether the read-only flag is set.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro returns whether the read-only flag is set or not.

Return

1 if the read-only flag is set, 0 otherwise.

6.1.10.3.2.20 macro SMW_ATTR_IS_READ_ONCE

SMW_ATTR_IS_READ_ONCE(attr)

Whether the read-once flag is set.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro returns whether the read-once flag is set or not.

Return

1 if the read-once flag is set, 0 otherwise.

6.1.10.3.2.21 macro SMW_ATTR_SET_SENSITIVE

SMW_ATTR_SET_SENSITIVE(attr)

Set sensitive flag.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro sets sensitive flag.

Return

Attributes

6.1.10.3.2.22 macro SMW_ATTR_CLEAR_SENSITIVE

SMW_ATTR_CLEAR_SENSITIVE(*attr*)

Clear sensitive flag.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro clear sensitive flag.

Return

Attributes

6.1.10.3.2.23 macro SMW_ATTR_IS_SENSITIVE

SMW_ATTR_IS_SENSITIVE(*attr*)

Whether the key is sensitive.

Parameters

- **attr** - Attributes. See [typedef smw_attr_attributes_t](#).

Description

This macro returns whether the sensitivity is true or not.

Return

1 if the sensitivity is set, 0 otherwise.

6.1.10.4 Storage location

Definition of the object (key or data) storage location. Giving Information such as storage location, type of object.

The storage location attribute is used only by the ELE subsystem and only part of the bit field is used, other bits are reserved for future use.

6.1.10.4.1 Typedef

6.1.10.4.1.1 typedef smw_attr_storage_id_t

type **smw_attr_storage_id_t**

Storage identifier

Description

32-bits field used to define the storage identifier.

The storage identifier is related to the Secure Subsystem. It is not the purpose of this API to modify it.

Table 154. Object storage identifier bit field

Bits	Description
[31:24]	Reserved
[23]	Set to 1 to use this NXP vendor encoding.
[22]	Vendor NXP Identifier.
[21]	NXP's EdgeLock 2GO object.
[20:16]	Reserved must be 0s.
[15]	Object type:
[14:8]	NXP's Secure Enclave storage identifier:
[7:0]	NXP's Secure Enclave identifier. Defined For future use

6.1.10.4.2 Examples

In case of *EdgeLock 2GO* object, the storage location attribute should be set as follows:

- Key type object: 0x00E000__
- Data type object: 0x00E080__

where __ means any value.

In case of *EdgeLock Key Import* object, the storage location attribute should be set as follows:

- Key type object: 0x00C00200

For other key creation operations (generate, derivation), the storage location attribute can be any value. The ELE Subsystem is not interpreting this attribute. Attribute is stored and returned as-is to the user. It's advice to set it to 0x00000000 for compatibility with future versions.

6.1.11 Names type definitions

Names type definitions provide constant names used in APIs arguments such as subsystem names, operation names, key type names, etc.

6.1.11.1 Names

6.1.11.1.1 typedef smw_subsystem_t

type **smw_subsystem_t**

Secure Subsystem name

6.1.11.1.1 Values

- SMW_SUBSYSTEM_NAME_NONE: No Secure Subsystem specified
- SMW_SUBSYSTEM_NAME_TEE: Trusted Execution Environment
- SMW_SUBSYSTEM_NAME_SECO: SECO
- SMW_SUBSYSTEM_NAME_ELE: Edge Lock Enclave
- SMW_SUBSYSTEM_NAME_NB: Number of Secure Subsystems

6.1.11.1.2 typedef smw_object_type_t

type **smw_object_type_t**

Object type name

6.1.11.1.2.1 Values

- SMW_OBJECT_TYPE_NAME_NONE: No object type specified
- SMW_OBJECT_TYPE_NAME_DATA: Data Object
- SMW_OBJECT_TYPE_NAME_SECRET_KEY: Symmetric Secret Key Object
- SMW_OBJECT_TYPE_NAME_PUBLIC_KEY: Asymmetric Public Key Object
- SMW_OBJECT_TYPE_NAME_KEY_PAIR: Asymmetric Key Pair Object
- SMW_OBJECT_TYPE_NAME_CERT: Certificate Object
- SMW_OBJECT_TYPE_NAME_NB: Number of object types

6.1.11.1.3 typedef smw_key_type_t

type **smw_key_type_t**

Key type name

6.1.11.1.3.1 Values

- SMW_KEY_TYPE_NAME_NONE: No key type specified
- SMW_KEY_TYPE_NAME_SECP_R1: ECC Secp R1 key
- SMW_KEY_TYPE_NAME_BRAINPOOL_R1: ECC Brainpool R1 key
- SMW_KEY_TYPE_NAME_BRAINPOOL_T1: ECC Brainpool T1 key
- SMW_KEY_TYPE_NAME_ED25519: Twisted Edwards 25519 key
- SMW_KEY_TYPE_NAME_AES: AES key
- SMW_KEY_TYPE_NAME_DES: DES key
- SMW_KEY_TYPE_NAME_DES3: Triple DES key
- SMW_KEY_TYPE_NAME_SM2: SM2 key
- SMW_KEY_TYPE_NAME_SM4: SM4 key
- SMW_KEY_TYPE_NAME_HMAC: HMAC key
- SMW_KEY_TYPE_NAME_RSA: RSA key
- SMW_KEY_TYPE_NAME_DH: DH key
- SMW_KEY_TYPE_NAME_TLS_MASTER: TLS master key
- SMW_KEY_TYPE_NAME_RAW: Raw key
- SMW_KEY_TYPE_NAME_DERIVE: Derived key

- SMW_KEY_TYPE_NAME_HKDF_IKM: HKDF IKM key
- SMW_KEY_TYPE_NAME_X25519: Montgomery Elliptic Curve25519 key
- SMW_KEY_TYPE_NAME_EL2GO_PROV_OEM_KEY: EdgeLock 2GO Provisioning OEM Key
- SMW_KEY_TYPE_NAME_ED448: Twisted Edwards 448 key
- SMW_KEY_TYPE_NAME_X448: Montgomery Elliptic Curve448 key
- SMW_KEY_TYPE_NAME_NB: Number of key types

6.1.11.1.4 typedef smw_key_privacy_t

type **smw_key_privacy_t**

Key privacy name

6.1.11.1.4.1 Values

- SMW_KEY_PRIVACY_NAME_NONE: No key privacy specified
- SMW_KEY_PRIVACY_NAME_PUBLIC: Public key
- SMW_KEY_PRIVACY_NAME_PRIVATE: Private key
- SMW_KEY_PRIVACY_NAME_PAIR: Key pair
- SMW_KEY_PRIVACY_NAME_SHARED_SECRET: Shared secret
- SMW_KEY_PRIVACY_NAME_NB: Number of key privacies

6.1.11.1.5 typedef smw_hash_algo_t

type **smw_hash_algo_t**

Hash algorithm name

6.1.11.1.5.1 Values

- SMW_HASH_ALGO_NAME_NONE: No hash algorithm specified
- SMW_HASH_ALGO_NAME_MD5: Message Digest 5
- SMW_HASH_ALGO_NAME_SHA1: Secure Hash Algorithm 1
- SMW_HASH_ALGO_NAME_SHA224: Secure Hash Algorithm 2, 224 bits
- SMW_HASH_ALGO_NAME_SHA256: Secure Hash Algorithm 2, 256 bits
- SMW_HASH_ALGO_NAME_SHA384: Secure Hash Algorithm 2, 384 bits
- SMW_HASH_ALGO_NAME_SHA512: Secure Hash Algorithm 2, 512 bits
- SMW_HASH_ALGO_NAME_SHA3_224: Secure Hash Algorithm 3, 224 bits
- SMW_HASH_ALGO_NAME_SHA3_256: Secure Hash Algorithm 3, 256 bits
- SMW_HASH_ALGO_NAME_SHA3_384: Secure Hash Algorithm 3, 384 bits
- SMW_HASH_ALGO_NAME_SHA3_512: Secure Hash Algorithm 3, 512 bits
- SMW_HASH_ALGO_NAME_SM3: ShangMi 3
- SMW_HASH_ALGO_NAME_SHAKE256: Secure Hash Algorithm KECCAK, 256 bits
- SMW_HASH_ALGO_NAME_NB: Number of Hash algorithms

6.1.11.1.6 typedef smw_mac_algo_t

type **smw_mac_algo_t**

MAC algorithm name

6.1.11.1.6.1 Values

- SMW_MAC_ALGO_NAME_NONE: No Message Authentication Code algorithm specified
- SMW_MAC_ALGO_NAME_CMAC: Cipher-based Message Authentication Code algorithm
- SMW_MAC_ALGO_NAME_CMAC_TRUNCATED: Cipher-based Message Authentication Code truncated algorithm
- SMW_MAC_ALGO_NAME_HMAC: Hash-Based Message Authentication Code algorithm
- SMW_MAC_ALGO_NAME_HMAC_TRUNCATED: Hash-Based Message Authentication Code truncated algorithm
- SMW_MAC_ALGO_NAME_NB: Number of Message Authentication Code algorithms

6.1.11.1.7 typedef smw_cipher_mode_t

type **smw_cipher_mode_t**

Cipher mode name

6.1.11.1.7.1 Values

- SMW_CIPHER_MODE_NAME_NONE: No cipher mode specified
- SMW_CIPHER_MODE_NAME_CBC: Cipher Block Chaining mode
- SMW_CIPHER_MODE_NAME_CFB: Cipher Feedback Block mode
- SMW_CIPHER_MODE_NAME_CTR: Counter mode
- SMW_CIPHER_MODE_NAME_CTS: Ciphertext Stealing mode
- SMW_CIPHER_MODE_NAME_ECB: Electronic Codebook Block mode
- SMW_CIPHER_MODE_NAME_XTS: XEX Tweakable Block Cipher with Ciphertext Stealing mode
- SMW_CIPHER_MODE_NAME_OFB: Output Feedback Block mode
- SMW_CIPHER_MODE_NAME_NB: Number of cipher modes

6.1.11.1.8 typedef smw_aead_mode_t

type **smw_aead_mode_t**

AEAD mode name

6.1.11.1.8.1 Values

- SMW_AEAD_MODE_NAME_NONE: No AEAD mode specified
- SMW_AEAD_MODE_NAME_CCM: Counter with CBC-MAC mode
- SMW_AEAD_MODE_NAME_GCM: Galois Counter Mode
- SMW_AEAD_MODE_NAME_CHACHA20_POLY1305: ChaCha20 stream cipher with Poly1305 MAC mode
- SMW_AEAD_MODE_NAME_NB: Number of AEAD modes

6.1.11.1.9 typedef smw_cipher_op_type_t

type **smw_cipher_op_type_t**

Cipher operation type name

6.1.11.1.9.1 Values

- SMW_CIPHER_OP_TYPE_NAME_NONE: No cipher operation type specified

- SMW_CIPHER_OP_TYPE_NAME_ENCRYPT: Encrypt operation type
- SMW_CIPHER_OP_TYPE_NAME_DECRYPT: Decrypt operation type
- SMW_CIPHER_OP_TYPE_NAME_NB: Number of cipher operation types

6.1.11.1.10 typedef smw_aead_op_type_t

type **smw_aead_op_type_t**

AEAD operation type name

6.1.11.1.10.1 Values

- SMW_AEAD_OP_TYPE_NAME_NONE: No AEAD operation specified
- SMW_AEAD_OP_TYPE_NAME_ENCRYPT: AEAD encrypt operation
- SMW_AEAD_OP_TYPE_NAME_DECRYPT: AEAD decrypt operation
- SMW_AEAD_OP_TYPE_NAME_NB: Number of AEAD operations

6.1.11.1.11 typedef smw_key_format_t

type **smw_key_format_t**

Key format name

6.1.11.1.11.1 Values

- SMW_KEY_FORMAT_NAME_NONE: No key format specified
- SMW_KEY_FORMAT_NAME_HEX: Hexadecimal value (no encoding)
- SMW_KEY_FORMAT_NAME_BASE64: Base 64 encoding value
- SMW_KEY_FORMAT_NAME_NB: Number of key formats

6.1.11.1.12 typedef smw_signature_algo_t

type **smw_signature_algo_t**

Signature algorithm name

6.1.11.1.12.1 Values

- SMW_SIGNATURE_ALGO_NAME_NONE: No signature algorithm specified
- SMW_SIGNATURE_ALGO_NAME_DEFAULT: Signature algorithm is given by the key type
- SMW_SIGNATURE_ALGO_NAME_ECDSA: Elliptic Curve Digital signature algorithm
- SMW_SIGNATURE_ALGO_NAME_EDDSA: Edwards-curve Digital signature algorithm
- SMW_SIGNATURE_ALGO_NAME_DSA: Digital signature algorithm
- SMW_SIGNATURE_ALGO_NAME_RSA: Rivest, Shamir and Adleman signature algorithm
- SMW_SIGNATURE_ALGO_NAME_TLS_1_2: Transport Layer Security 1.2 signature algorithm
- SMW_SIGNATURE_ALGO_NAME_SM2: ShāngMì 2 signature algorithm
- SMW_SIGNATURE_ALGO_NAME_NB: Number of signature algorithms

6.1.11.1.13 typedef smw_signature_type_t

type **smw_signature_type_t**

Signature type name

6.1.11.1.13.1 Values

- SMW_SIGNATURE_TYPE_NAME_NONE: No signature type specified
- SMW_SIGNATURE_TYPE_NAME_DEFAULT: Default signature type
- SMW_SIGNATURE_TYPE_NAME_PKCS1_1_5: Public-Key Cryptography Standards #1 v1.5
- SMW_SIGNATURE_TYPE_NAME_PSS: Probabilistic Signature Scheme
- SMW_SIGNATURE_TYPE_NAME_CLIENT: TLS client signature
- SMW_SIGNATURE_TYPE_NAME_SERVER: TLS server signature
- SMW_SIGNATURE_TYPE_NAME_NB: Number of signature types

6.1.11.1.14 typedef smw_tls12_kec_t

type **smw_tls12_kec_t**

TLS 1.2 Key exchange algorithm name

6.1.11.1.14.1 Values

- SMW_TLS12_KEC_NAME_NONE: No TLS 1.2 Key exchange algorithm specified
- SMW_TLS12_KEC_NAME_DH_DSS: Diffie-Hellman signed with DSS
- SMW_TLS12_KEC_NAME_DH_RSA: Diffie-Hellman signed with RSA
- SMW_TLS12_KEC_NAME_DHE_DSS: Diffie-Hellman key Exchange signed with DSS
- SMW_TLS12_KEC_NAME_DHE_RSA: Diffie-Hellman key Exchange signed with RSA
- SMW_TLS12_KEC_NAME_ECDH_ECDSA: Elliptic Curve Diffie-Hellman signed with ECDSA
- SMW_TLS12_KEC_NAME_ECDH_RSA: Elliptic Curve Diffie-Hellman signed with RSA
- SMW_TLS12_KEC_NAME_ECDHE_ECDSA: Elliptic Curve Diffie-Hellman key Exchange signed with ECDSA
- SMW_TLS12_KEC_NAME_ECDHE_RSA: Elliptic Curve Diffie-Hellman key Exchange signed with RSA
- SMW_TLS12_KEC_NAME_RSA: Rivest-Shamir-Adleman
- SMW_TLS12_KEC_NAME_NB: Number of TLS 1.2 Key exchange algorithms

6.1.11.1.15 typedef smw_tls12_enc_t

type **smw_tls12_enc_t**

TLS 1.2 encryption algorithm name

6.1.11.1.15.1 Values

- SMW_TLS12_ENC_NAME_NONE: No TLS 1.2 encryption algorithm specified
- SMW_TLS12_ENC_NAME_3DES_EDE_CBC: Triple DES Encrypt-Decrypt-Encrypt with CBC
- SMW_TLS12_ENC_NAME_AES_128_CBC: Advanced Encryption Standard 128 bits with CBC
- SMW_TLS12_ENC_NAME_AES_128_GCM: Advanced Encryption Standard 128 bits with GCM
- SMW_TLS12_ENC_NAME_AES_256_CBC: Advanced Encryption Standard 256 bits with CBC
- SMW_TLS12_ENC_NAME_AES_256_GCM: Advanced Encryption Standard 256 bits with GCM
- SMW_TLS12_ENC_NAME_RC4_128: Rivest Cipher 4 128 bits
- SMW_TLS12_ENC_NAME_AES_128_CCM: Advanced Encryption Standard 128 bits with CCM
- SMW_TLS12_ENC_NAME_AES_256_CCM: Advanced Encryption Standard 256 bits with CCM
- SMW_TLS12_ENC_NAME_CHACHA20_POLY1305: ChaCha20 stream cipher with Poly1305 MAC

- SMW_TLS12_ENC_NAME_NB: Number of TLS 1.2 encryption algorithms

6.1.11.1.16 typedef smw_tls12_op_t

type **smw_tls12_op_t**

TLS 1.2 operation name

6.1.11.1.16.1 Values

- SMW_TLS12_OP_NAME_NONE: No TLS 1.2 operation name specified
- SMW_TLS12_OP_NAME_MASTER_SECRET: TLS 1.2 master secret
- SMW_TLS12_OP_NAME_KEY_EXPANSION: TLS 1.2 key expansion
- SMW_TLS12_OP_NAME_NB: Number of TLS 1.2 operation names

6.1.11.1.17 typedef smw_oem_master_key_op_t

type **smw_oem_master_key_op_t**

OEM Master key derivation operation name

6.1.11.1.17.1 Values

- SMW_OEM_MK_OP_NAME_NONE: No operation name specified
- SMW_OEM_MK_OP_NAME_DERIVE: Derive the OEM Master key
- SMW_OEM_MK_OP_NAME_PREPARE: Prepare the OEM Master key payload to sign
- SMW_OEM_MK_OP_NAME_NB: Number of operation names

6.1.11.1.18 typedef smw_lifecycle_t

type **smw_lifecycle_t**

Device lifecycle name

6.1.11.1.18.1 Values

- SMW_LIFECYCLE_NAME_NONE: No lifecycle specified
- SMW_LIFECYCLE_NAME_CURRENT: Current
- SMW_LIFECYCLE_NAME_OEM_OPEN: OEM Open
- SMW_LIFECYCLE_NAME_OPEN: **Deprecated**, renamed as SMW_LIFECYCLE_NAME_OEM_OPEN
- SMW_LIFECYCLE_NAME_OEM_CLOSED: OEM Closed
- SMW_LIFECYCLE_NAME_CLOSED: **Deprecated**, renamed as SMW_LIFECYCLE_NAME_OEM_CLOSED
- SMW_LIFECYCLE_NAME_OEM_LOCKED: OEM Locked
- SMW_LIFECYCLE_NAME_CLOSED_LOCKED: **Deprecated**, renamed as SMW_LIFECYCLE_NAME_OEM_LOCKED
- SMW_LIFECYCLE_NAME_OEM_RETURN: OEM return
- SMW_LIFECYCLE_NAME_NXP_RETURN: NXP return
- SMW_LIFECYCLE_NAME_NB: Number of lifecycles

6.1.11.1.19 typedef smw_kdf_t

type **smw_kdf_t**

Key Derivation Function name

6.1.11.1.19.1 Values

- SMW_KDF_NAME_NONE: No Key Derivation Function name specified
- SMW_KDF_NAME_HKDF: HMAC-Based Key Derivation Function
- SMW_KDF_NAME_HKDF_EXTRACT: HMAC-Based Key Derivation Extract step Function
- SMW_KDF_NAME_HKDF_EXPAND: HMAC-Based Key Derivation Expand step Function
- SMW_KDF_NAME_TLS12_KEY_EXCHANGE: TLS 1.2 Key Exchange **deprecated**, prefer *SMW_KDF_NAME_TLS12_OP_KEY_EXCHANGE*
- SMW_KDF_NAME_ECDH: ECDH Key Exchange
- SMW_KDF_NAME_TLS12_OP_KEY_EXCHANGE: TLS 1.2 “Operation-based” Key Exchange
- SMW_KDF_NAME_TLS13_KEY_EXCHANGE: TLS 1.3 Key Exchange
- SMW_KDF_NAME_OEM_MASTER_KEY: OEM Master key derivation
- SMW_KDF_NAME_NB: Number of Key Derivation Functions

6.1.11.1.20 typedef smw_asymmetric_encryption_algo_t

type **smw_asymmetric_encryption_algo_t**

Asymmetric encryption/decryption algo name

6.1.11.1.20.1 Values

- SMW_ASYMMETRIC_ENCRYPTION_ALGO_NAME_NONE: No encryption/decryption algorithm specified
- SMW_ASYMMETRIC_ENCRYPTION_ALGO_NAME_RSA: RSA encryption/decryption algorithm
- SMW_ASYMMETRIC_ENCRYPTION_ALGO_NAME_NB: Number of encryption/decryption algorithms

6.1.11.1.21 typedef smw_asymmetric_encryption_mode_t

type **smw_asymmetric_encryption_mode_t**

Asymmetric encryption/decryption padding scheme name

6.1.11.1.21.1 Values

- SMW_ASYMMETRIC_ENCRYPTION_MODE_NAME_NONE: No Encryption padding scheme name specified
- SMW_ASYMMETRIC_ENCRYPTION_MODE_NAME_PKCS1_1_5: Public-Key Cryptography Standards #1 v1.5
- SMW_ASYMMETRIC_ENCRYPTION_MODE_NAME_OAEP: Optimal Asymmetric Encryption Padding
- SMW_ASYMMETRIC_ENCRYPTION_MODE_NAME_NO_PAD: Asymmetric Encryption/decryption with no padding
- SMW_ASYMMETRIC_ENCRYPTION_MODE_NAME_NB: Number of encryption padding schemes

6.1.11.1.22 typedef smw_osal_db_capability_t

type **smw_osal_db_capability_t**

OSAL database capability names

6.1.11.1.22.1 Values

- SMW_OSAL_DB_CAPABILITY_NONE: No capability specified
- SMW_OSAL_DB_CAPABILITY_PUBLIC_KEY_IMPORT: Public key import capability
- SMW_OSAL_DB_CAPABILITY_NB: Number of database capabilities

6.2 PSA APIs

6.2.1 Initialization

6.2.1.1 library initialization

6.2.1.1.1 Function

6.2.1.1.1.1 `psa_crypto_init`

[psa_status_t](#) `psa_crypto_init(void)`

Library initialization.

Parameters

- `void` - no arguments

Description

Applications must call this function before calling any other function in this module.

Applications are permitted to call this function more than once. Once a call succeeds, subsequent calls are guaranteed to succeed.

If the application calls any functions before calling `psa_crypto_init()`, the function returns `PSA_ERROR_BAD_STATE`.

Return

- `PSA_SUCCESS`
- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_INSUFFICIENT_ENTROPY`

6.2.2 Key Manager APIs

The Key Manager APIs allow users of the library to manage cryptographic keys. such as generation, derivation, import, export, and deletion.

6.2.2.1 Key Creation

6.2.2.1.1 Key Generate

6.2.2.1.1.1 `psa_generate_key`

`psa_status_t` `psa_generate_key`(const `psa_key_attributes_t` *attributes, `psa_key_id_t` *key)

Generate a key or key pair.

Parameters

- **attributes** (const `psa_key_attributes_t` *) - [in] The attributes for the new key.
- **key** (`psa_key_id_t` *) - [out] On success, an identifier for the newly created key. `PSA_KEY_ID_NULL` on failure.

Description

The key is generated randomly. Its location, policy, type and size are taken from `attributes`.

The following type-specific considerations apply:

- For RSA keys (`PSA_KEY_TYPE_RSA_KEY_PAIR`), the public exponent is 65537. The modulus is a product of two probabilistic primes between 2^{n-1} and 2^n where n is the bit size specified in the attributes.

The `attributes` parameter must set the new key:

- Permitted algorithm
- Usage
- Lifetime (if a persistent key is desired)
- Key identifier (if a persistent key is desired)
- Key type
- Key size

Note:

The attributes is an input parameter: it is not updated with the final key attributes. The final attributes of the new key can be queried by calling `psa_get_key_attributes()` with the key's identifier.

Return

- **PSA_SUCCESS:**

Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`.

- **PSA_ERROR_NOT_PERMITTED:**

Creating a key with the specified attributes is not permitted.

- **PSA_ERROR_ALREADY_EXISTS:**

This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.

- **PSA_ERROR_INVALID_ARGUMENT:**

- The key type is invalid, or is an asymmetric public-key type.
- The key size is not valid for the key type.
- The key lifetime is invalid.

- The key identifier is not valid for the key lifetime.
- The key usage flags include invalid values.
- The key's permitted-usage algorithm is invalid.
- The key attributes, as a whole, are invalid.

- **PSA_ERROR_NOT_SUPPORTED:**

The key attributes, as a whole are not supported.

- PSA_ERROR_INSUFFICIENT_ENTROPY
- PSA_ERROR_INSUFFICIENT_MEMORY
- PSA_ERROR_INSUFFICIENT_STORAGE
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- PSA_ERROR_STORAGE_FAILURE
- PSA_ERROR_DATA_CORRUPT
- PSA_ERROR_DATA_INVALID

6.2.2.1.1.2 psa_generate_key_custom

[psa_status_t](#) **psa_generate_key_custom**(const [psa_key_attributes_t](#) *attributes, const [psa_custom_key_parameters_t](#) *custom, const uint8_t *custom_data, size_t custom_data_length, [psa_key_id_t](#) *key)

Generate a key or key pair using custom production parameters.

Parameters

- **attributes** (const [psa_key_attributes_t](#) *) - [in] The attributes for the new key.
- **custom** (const [psa_custom_key_parameters_t](#) *) - [in] Customized production parameters for the key generation.
- **custom_data** (const uint8_t *) - [in] A buffer containing additional variable-sized production parameters.
- **custom_data_length** (size_t) - [in] Length of `custom_data` in bytes.
- **key** ([psa_key_id_t](#) *) - [out] On success, an identifier for the newly created key. PSA_KEY_ID_NULL on failure.

Description

Warning:

Not supported

Use this function to provide explicit production parameters when generating a key. See the description of [psa_generate_key\(\)](#) for the operation of this function with the default production parameters.

The key is generated randomly. Its location, policy, type and size are taken from `attributes`.

The `attributes` parameter must set the new key:

- Permitted algorithm
- Usage
- Lifetime (if a persistent key is desired)
- Key identifier (if a persistent key is desired)
- Key type
- Key size

Note:

The *attributes* is an input parameter: it is not updated with the final key attributes. The final attributes of the new key can be queried by calling `psa_get_key_attributes()` with the key's identifier.

If custom parameters are not used, this function is `psa_generate_key()` equivalent.

Return

- **PSA_SUCCESS:**

Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`.

- **PSA_ERROR_NOT_PERMITTED:**

Creating a key with the specified attributes is not permitted.

- **PSA_ERROR_ALREADY_EXISTS:**

This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.

- **PSA_ERROR_INVALID_ARGUMENT:**

- The key type is invalid, or is an asymmetric public-key type.
- The key size is not valid for the key type.
- The key lifetime is invalid.
- The key identifier is not valid for the key lifetime.
- The key usage flags include invalid values.
- The key's permitted-usage algorithm is invalid.
- The key attributes, as a whole, are invalid.
- The production parameters are invalid.

- **PSA_ERROR_NOT_SUPPORTED:**

The key attributes, as a whole are not supported.

- PSA_ERROR_INSUFFICIENT_ENTROPY
- PSA_ERROR_INSUFFICIENT_MEMORY
- PSA_ERROR_INSUFFICIENT_STORAGE
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- PSA_ERROR_STORAGE_FAILURE
- PSA_ERROR_DATA_CORRUPT
- PSA_ERROR_DATA_INVALID

6.2.2.1.1.3 psa_copy_key

`psa_status_t` **psa_copy_key**(`psa_key_id_t` source_key, const `psa_key_attributes_t` *attributes, `psa_key_id_t` *target_key)

Make a copy of a key.

Parameters

- **source_key** (`psa_key_id_t`) - [in] The key to copy. It must allow the usage PSA_KEY_USAGE_COPY. If a private or secret key is being copied outside of a secure element it must also allow PSA_KEY_USAGE_EXPORT.
- **attributes** (const `psa_key_attributes_t` *) - [in] The attributes for the new key.
- **target_key** (`psa_key_id_t` *) - [out] On success, an identifier for the newly created key. PSA_KEY_ID_NULL on failure.

Description

Warning:

Not supported

Copy key material from one location to another. Its location is taken from attributes, its policy is the intersection of the policy in `attributes` and the source key policy, and its type and size are taken from the source key.

This function is primarily useful to copy a key from one location to another, as it populates a key using the material from another key which can have a different lifetime.

The policy on the source key must have the usage flag `PSA_KEY_USAGE_COPY` set. This flag is sufficient to permit the copy if the key has the lifetime `PSA_KEY_LIFETIME_VOLATILE` or `PSA_KEY_LIFETIME_PERSISTENT`. Some secure elements do not provide a way to copy a key without making it extractable from the secure element. If a key is located in such a secure element, then the key must have both usage flags `PSA_KEY_USAGE_COPY` and `PSA_KEY_USAGE_EXPORT` in order to make a copy of the key outside the secure element.

The resulting key can only be used in a way that conforms to both the policy of the original key and the policy specified in the attributes parameter:

- The usage flags on the resulting key are the bitwise-and of the usage flags on the source policy and the usage flags in attributes.
- If both permit the same algorithm or wildcard-based algorithm, the resulting key has the same permitted algorithm.
- If either of the policies permits an algorithm and the other policy permits a wildcard-based permitted algorithm that includes this algorithm, the resulting key uses this permitted algorithm.
- If the policies do not permit any algorithm in common, this function fails with the status `PSA_ERROR_INVALID_ARGUMENT`.

As a result, the new key cannot be used for operations that were not permitted on the source key.

The `attributes` parameter must set the new key:

- Permitted algorithm
- Usage: Flags are combined with the source key usage flags.
- Lifetime (if a persistent key is desired)
- Key identifier (if a persistent key is desired)
- key type and key size: (optional) These are taken from the source key, if set they must be identical as the source key.

Note:

The attributes is an input parameter: it is not updated with the final key attributes. The final attributes of the new key can be queried by calling `psa_get_key_attributes()` with the key's identifier.

Return

- **PSA_SUCCESS:**

Success. If the new key is persistent, the key material and the key's metadata have been saved to persistent storage.

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`.

- **PSA_ERROR_INVALID_HANDLE:**

`source_key` is invalid.

- **PSA_ERROR_NOT_PERMITTED:**

- `source_key` does not have the `PSA_KEY_USAGE_COPY` usage flag.
- `source_key` does not have the `PSA_KEY_USAGE_EXPORT` usage flag, and the location of `target_key` is outside the security boundary of the `source_key` storage location.
- Creating a key with the specified attributes is not permitted.

- **PSA_ERROR_ALREADY_EXISTS:**

This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `attributes` specifies a key type or key size which does not match the attributes of source key.
- The lifetime or identifier in `attributes` are invalid.
- The key policies from `source_key` and those specified in `attributes` are incompatible.

- **PSA_ERROR_NOT_SUPPORTED:**

- The `source_key` storage location does not support copying to the target key's storage location.
- The key attributes, as a whole, are not supported in the target key's storage location.

- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_INSUFFICIENT_STORAGE`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`

6.2.2.1.2 Key Import

6.2.2.1.2.1 `psa_import_key`

`psa_status_t` `psa_import_key`(const `psa_key_attributes_t` *`attributes`, const `uint8_t` *`data`, `size_t` `data_length`, `psa_key_id_t` *`key`)

Import a key in binary format.

Parameters

- **`attributes`** (const `psa_key_attributes_t` *) - [in] The attributes for the new key.
- **`data`** (const `uint8_t` *) - [in] Buffer containing the key data. The content of this buffer is interpreted according to the type declared in `attributes`.
- **`data_length`** (`size_t`) - [in] Size of the `data` buffer in bytes.
- **`key`** (`psa_key_id_t` *) - [out] On success, an identifier for the newly created key. `PSA_KEY_ID_NULL` on failure.

Description

Warning:

Import of private plaintext key may be not supported depending on the secure subsystem in use, in this case the key must be wrapped.

The key data determines the key size. The attributes can optionally specify a key size; in this case it must match the size determined from the key data. A key size of 0 in attributes indicates that the key size is solely determined by the key data.

The following new key `attributes` are required:

- The key type determines how the data buffer is interpreted.
- The key permitted algorithm.
- The key usage flags.

The following new key `attributes` must be set in case of a persistent key:

- The key lifetime.
- The key identifier.

The following new key `attributes` are optional:

- The key size, if not zero, must match the size of the key data.

Return

- **PSA_SUCCESS:**

Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.

- **PSA_ERROR_ALREADY_EXISTS:**

This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.

- **PSA_ERROR_NOT_PERMITTED:**

Creating a key with the specified attributes is not permitted.

- **PSA_ERROR_NOT_SUPPORTED:**

The key type or key size is not supported.

- **PSA_ERROR_INVALID_ARGUMENT:**

- The key type is invalid.
- The key size is nonzero, and is incompatible with the key data in data.
- The key lifetime is invalid.
- The key identifier is not valid for the key lifetime.
- The key usage flags include invalid values.
- The key's permitted-usage algorithm is invalid.
- The key attributes, as a whole, are invalid.
- The key data is not correctly formatted for the key type.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_INSUFFICIENT_STORAGE**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.2.2 Key Derivation

6.2.2.2.1 Multiple Part

The sequence of operations to derive a key is as follows:

1. Allocate a key-derivation operation object [psa_key_derivation_operation_t](#) which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for [psa_key_derivation_operation_t](#), e.g. [PSA_KEY_DERIVATION_OPERATION_INIT](#).
3. Call [psa_key_derivation_setup\(\)](#) to select the algorithm.
4. Provide the inputs for the key derivation by calling [psa_key_derivation_input_bytes\(\)](#) or [psa_key_derivation_input_key\(\)](#) as appropriate. Which inputs are needed, in what order, whether keys are permitted, and what type of keys depends on the algorithm.
5. Optionally set the operation's maximum capacity with [psa_key_derivation_set_capacity\(\)](#). This can be done before, in the middle of, or after providing inputs. For some algorithms, this step is mandatory because the output depends on the maximum capacity.
6. To derive a key, call [psa_key_derivation_output_key\(\)](#). To derive a byte string for a different purpose, call [psa_key_derivation_output_bytes\(\)](#). Successive calls to these functions use successive output bytes calculated by the key derivation algorithm.
7. Clean up the key derivation operation object with [psa_key_derivation_abort\(\)](#).

6.2.2.2.1.1 typedef psa_key_derivation_operation_t

type **psa_key_derivation_operation_t**

The type of the state object for key derivation operations.

Description

Before calling any function on a key derivation operation object, the application must initialize it by any of the following means:

- Set the object to all-bits-zero, for example:

```
psa_key_derivation_operation_t operation;
memset(&operation, 0, sizeof(operation));
```

- Initialize the object to logical zero values by declaring the object as static or global without an explicit initializer, for example:

```
static psa_key_derivation_operation_t operation;
```

- Initialize the object to the initializer [PSA_KEY_DERIVATION_OPERATION_INIT](#), for example:

```
psa_key_derivation_operation_t operation = PSA_KEY_DERIVATION_OPERATION_INIT;
```

- Assign the result of the function [psa_key_derivation_operation_init\(\)](#) to the object, for example:

```
psa_key_derivation_operation_t operation;
operation = psa_key_derivation_operation_init();
```

6.2.2.2.1.2 macro `PSA_KEY_DERIVATION_OPERATION_INIT`

`PSA_KEY_DERIVATION_OPERATION_INIT()`

Initializer for a key derivation operation object of [typedef `psa\_key\_derivation\_operation\_t`](#).

6.2.2.2.1.3 `psa_key_derivation_operation_init`

[psa_key_derivation_operation_t](#) `psa_key_derivation_operation_init(void)`

Return an initial value for a key derivation operation object.

Parameters

- `void` - no arguments

Return

Initialized value of a key derivation operation object.

6.2.2.2.1.4 `psa_key_derivation_setup`

[psa_status_t](#) `psa_key_derivation_setup(psa_key_derivation_operation_t *operation, psa_algorithm_t alg)`

Set up a key derivation operation.

Parameters

- `operation` ([psa_key_derivation_operation_t](#)*) - [in] The key derivation operation object to set up. It must have been initialized but not set up yet.
- `alg` ([psa_algorithm_t](#)) - [in] The algorithm to compute.

Description

A key derivation algorithm takes some inputs and uses them to generate a byte stream in a deterministic way. This byte stream can be used to produce keys and other cryptographic material.

The `alg` must be an algorithm where:

- [PSA_ALG_IS_KEY_DERIVATION\(\)](#) is true, for a key derivation algorithm.
- [PSA_ALG_IS_KEY_AGREEMENT\(\)](#) is true and [PSA_ALG_IS_RAW_KEY_AGREEMENT\(\)](#) is false, for a key agreement and key derivation combined algorithm.

A key-agreement and key-derivation algorithm uses a key-agreement protocol to provide a shared secret which is used for the key derivation. See [psa_key_derivation_key_agreement\(\)](#).

After a successful call to [psa_key_derivation_setup\(\)](#), the operation is active, and the application must eventually terminate the operation with a call to [psa_key_derivation_abort\(\)](#).

If [psa_key_derivation_setup\(\)](#) returns an error, the operation object is unchanged. If a subsequent function call with an active operation returns an error, the operation enters an error state.

To abandon an active operation, or reset an operation in an error state, call [psa_key_derivation_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_INVALID_ARGUMENT:**

`alg` is neither a key-derivation algorithm, nor a key-agreement and key-derivation algorithm.

- **PSA_ERROR_NOT_SUPPORTED:**

`alg` is not supported.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid: it must be inactive.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.2.2.1.5 `psa_key_derivation_get_capacity`

[psa_status_t](#) `psa_key_derivation_get_capacity`(const [psa_key_derivation_operation_t](#) *operation, [size_t](#) *capacity)

Retrieve the current capacity of a key derivation operation.

Parameters

- **operation** (const [psa_key_derivation_operation_t](#) *) - [in] The operation to query.
- **capacity** ([size_t](#) *) - [out] On success, the capacity of the operation.

Description

Warning:

Not supported.

The capacity of a key derivation is the maximum number of bytes that it can return. Reading **N** bytes of output from a key derivation operation reduces its capacity by at least **N**. The capacity can be reduced by more than **N** in the following situations:

- Calling [psa_key_derivation_output_key\(\)](#) or [psa_key_derivation_key_custom\(\)](#) can reduce the capacity by more than the key size, depending on the type of key being generated. See [psa_key_derivation_output_key\(\)](#) for details of the key derivation process.
- When the [typedef psa_key_derivation_operation_t](#) object is operating as a deterministic random bit generator (DRBG), which reduces capacity in whole blocks, even when less than a block is read.

Return

- **PSA_SUCCESS**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be active.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.2.2.1.6 psa_key_derivation_set_capacity

[psa_status_t](#) [psa_key_derivation_set_capacity](#)([psa_key_derivation_operation_t](#) *operation, [size_t](#) capacity)

Set the maximum capacity of a key derivation operation.

Parameters

- **operation** ([psa_key_derivation_operation_t](#)*) - [in] The key derivation operation object to modify.
- **capacity** ([size_t](#)) - [in] The new capacity of the operation. It must be less or equal to the operation's current capacity.

Description

Warning:

Not supported.

The capacity of a key derivation operation is the maximum number of bytes that the key derivation operation can return from this point onwards.

Return

- PSA_SUCCESS
- **PSA_ERROR_INVALID_ARGUMENT:**
 - `capacity` is larger than the operation's current capacity. In this case, the operation object remains valid and its capacity remains unchanged.
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_HARDWARE_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be active.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.2.2.1.7 psa_key_derivation_input_bytes

[psa_status_t](#) [psa_key_derivation_input_bytes](#)([psa_key_derivation_operation_t](#) *operation, [psa_key_derivation_step_t](#) step, [const uint8_t](#) *data, [size_t](#) data_length)

Provide an input for key derivation or key agreement.

Parameters

- **operation** ([psa_key_derivation_operation_t](#)*) - [in] The key derivation operation object to use. It must have been set up with [psa_key_derivation_setup\(\)](#) and must not have produced any output yet.
- **step** ([psa_key_derivation_step_t](#)) - [in] Which step the input data is for.
- **data** ([const uint8_t](#)*) - [in] Input data to use.
- **data_length** ([size_t](#)) - [in] Size of the `data` buffer in bytes.

Description

Which inputs are required and in what order depends on the algorithm. Refer to the documentation of each key derivation or key agreement algorithm for information.

This function passes direct inputs, which is usually correct for non-secret inputs. To pass a secret input, which is normally in a key object, call [psa_key_derivation_input_key\(\)](#) instead of this function. Refer to the documentation of individual step types (PSA_KEY_DERIVATION_INPUT_XXX values of [typedef psa_key_derivation_step_t](#)) for more information.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_key_derivation_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `step` is not compatible with the operation's algorithm.
- `step` does not permit direct inputs.
- `data_length` is too small or too large for `step` in this particular algorithm.

- **PSA_ERROR_NOT_SUPPORTED:**

- `step` is not supported with the operation's algorithm.
- `data_length` is not supported for `step` in this particular algorithm.

- **PSA_ERROR_INSUFFICIENT_MEMORY**

- **PSA_ERROR_COMMUNICATION_FAILURE**

- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_STORAGE_FAILURE**

- **PSA_ERROR_DATA_CORRUPT**

- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid for this input step. This can happen if the application provides a step out of order or repeats a step that may not be repeated.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.2.2.1.8 psa_key_derivation_input_integer

[psa_status_t](#) [psa_key_derivation_input_integer](#)([psa_key_derivation_operation_t](#) *operation, [psa_key_derivation_step_t](#) step, [uint64_t](#) value)

Provide a numeric input for key derivation or key agreement.

Parameters

- **operation** ([psa_key_derivation_operation_t](#)*) - [in] The key derivation operation object to use. It must have been set up with [psa_key_derivation_setup\(\)](#) and must not have produced any output yet.
- **step** ([psa_key_derivation_step_t](#)) - [in] Which step the input data is for.
- **value** ([uint64_t](#)) - [in] The value of the numeric input.

Description

Warning:

Not supported.

Which inputs are required and in what order depends on the algorithm. However, when an algorithm requires a particular order, numeric inputs usually come first as they tend to be configuration parameters. Refer to the documentation of each key derivation or key agreement algorithm for information.

This function is used for inputs which are fixed-size non-negative integers.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_key_derivation_abort\(\)](#).

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid for this input `step`. This can happen if the application provides a step out of order or repeats a step that may not be repeated.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).
- **PSA_ERROR_INVALID_ARGUMENT:**
 - `step` is not compatible with the operation's algorithm.
 - `step` does not allow numerical inputs.
 - `value` is not valid for `step` in the operation's algorithm.
- **PSA_ERROR_NOT_SUPPORTED:**
 - `step` is not supported with the operation's algorithm.
 - `value` is not supported for `step` in the operation's algorithm.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**

6.2.2.2.1.9 psa_key_derivation_input_key

[psa_status_t](#) [psa_key_derivation_input_key](#)([psa_key_derivation_operation_t](#) *operation, [psa_key_derivation_step_t](#) step, [psa_key_id_t](#) key)

Provide an input for key derivation in the form of a key.

Parameters

- **operation** ([psa_key_derivation_operation_t](#)*) - [in] The key derivation operation object to use. It must have been set up with [psa_key_derivation_setup\(\)](#) and must not have produced any output yet.
- **step** ([psa_key_derivation_step_t](#)) - [in] Which step the input data is for.
- **key** ([psa_key_id_t](#)) - [in] Identifier of the key. It must have an appropriate type for step and must allow the usage `PSA_KEY_USAGE_DERIVE` or `PSA_KEY_USAGE_VERIFY_DERIVATION`.

Description

Which inputs are required and in what order depends on the algorithm. Refer to the documentation of each key derivation or key agreement algorithm for information.

This function obtains input from a key object, which is usually correct for secret inputs or for non-secret personalization strings kept in the key store. To pass a non-secret parameter which is not in the key store, call [psa_key_derivation_input_bytes\(\)](#) instead of this function. Refer to the documentation of individual step types (PSA_KEY_DERIVATION_INPUT_xxx values of [typedef psa_key_derivation_step_t](#)) for more information.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_key_derivation_abort\(\)](#).

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE:**
`key` is not a valid key identifier.
- **PSA_ERROR_NOT_PERMITTED:**
 - The key has neither the PSA_KEY_USAGE_DERIVE nor the PSA_KEY_USAGE_VERIFY_DERIVATION usage flag.
 - The key does not permit the operation's algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
 - `step` is not compatible with the operation's algorithm.
 - `step` does not allow key inputs of the given type or does not allow key inputs at all.
- **PSA_ERROR_NOT_SUPPORTED:**
 - `step` is not supported with the operation's algorithm.
 - Key inputs of the given type are not supported for `step` in the operation's algorithm.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid for this input step. This can happen if the application provides a step out of order or repeats a step that may not be repeated.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.2.2.1.10 psa_key_derivation_output_bytes

[psa_status_t](#) **psa_key_derivation_output_bytes**([psa_key_derivation_operation_t](#) *operation, [uint8_t](#) *output, [size_t](#) output_length)

Read some data from a key derivation operation.

Parameters

- **operation** ([psa_key_derivation_operation_t*](#)) - [in] The key derivation operation object to read from.
- **output** ([uint8_t*](#)) - [out] Buffer where the output will be written.
- **output_length** ([size_t](#)) - [in] Number of bytes to `output`.

Description

This function calculates output bytes from a key derivation algorithm and returns those bytes. If the key derivation's output is viewed as a stream of bytes, this function consumes the requested number of bytes from the stream and returns them to the caller. The operation's capacity decreases by the number of bytes read.

If this function returns an error status other than `PSA_ERROR_INSUFFICIENT_DATA`, the operation enters an error state and must be aborted by calling [psa_key_derivation_abort\(\)](#).

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_NOT_PERMITTED:**
One of the inputs was a key whose policy did not permit `PSA_KEY_USAGE_DERIVE`.
- **PSA_ERROR_INSUFFICIENT_DATA:**
The operation's capacity was less than `output_length` bytes. In this case:
 - No output is written to the `output` buffer.
 - The operation's capacity is set to 0.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be active and completed all required input steps.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.2.2.11 psa_key_derivation_output_key

[psa_status_t](#) **psa_key_derivation_output_key**(const [psa_key_attributes_t](#) *attributes, [psa_key_derivation_operation_t](#) *operation, [psa_key_id_t](#) *key)

Derive a key from an ongoing key derivation operation.

Parameters

- **attributes** (const [psa_key_attributes_t](#)*) - [in] The attributes for the new key.
- **operation** ([psa_key_derivation_operation_t](#)*) - [in] The key derivation operation object to read from.
- **key** ([psa_key_id_t](#)*) - [out] On success, an identifier for the newly created key. `PSA_KEY_ID_NULL` on failure.

Description

This function calculates output bytes from a key derivation algorithm and uses those bytes to generate a key deterministically. The key's location, policy, type and size are taken from `attributes`.

The following new key `attributes` are required:

- The key type. It cannot be an asymmetric public key.
- The key size. It must be a valid size for the key type.
- The key permitted algorithm policy. If the key type to be created is `PSA_KEY_TYPE_PASSWORD_HASH`, then the permitted algorithm policy must be either the same as the current operation's algorithm, or `PSA_ALG_NONE`.
- The key usage flags.

The following new key attributes must be set in case of a persistent key:

- The key lifetime.
- The key identifier.

If the key derivation's output is viewed as a stream of bytes, this function consumes the required number of bytes from the stream. The operation's capacity decreases by the number of bytes used to derive the key.

If this function returns an error status other than `PSA_ERROR_INSUFFICIENT_DATA`, the operation enters an error state and must be aborted by calling [psa_key_derivation_abort\(\)](#).

How much output is produced and consumed from the operation, and how the key is derived, depends on the key type.

For algorithms that take a `PSA_KEY_DERIVATION_INPUT_SECRET` or `PSA_KEY_DERIVATION_INPUT_PASSWORD` input step, the input to that step must be provided with [psa_key_derivation_input_key\(\)](#).

Note:

This function is equivalent to calling `psa_key_derivation_output_key_custom()` with the production parameters `PSA_CUSTOM_KEY_PARAMETERS_INIT` and `custom_data_length == 0` (`custom_data` is ignored).

Return

• **PSA_SUCCESS:**

Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.

• **PSA_ERROR_ALREADY_EXISTS:**

This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.

• **PSA_ERROR_INSUFFICIENT_DATA:**

The operation's capacity was less than `output_length` bytes. In this case:

- No output is written to the `output` buffer.
- The operation's capacity is set to 0.

• **PSA_ERROR_NOT_SUPPORTED:**

The key attributes, as a whole, are not supported.

• **PSA_ERROR_INVALID_ARGUMENT:**

- The key type is invalid, or is an asymmetric public-key type.
- The key type is `PSA_KEY_TYPE_PASSWORD_HASH`, and the permitted algorithm policy is not the same as the current operation's algorithm.
- The key size is not valid for the key type.

- The key lifetime is invalid.
 - The key identifier is not valid for the key lifetime.
 - The key usage flags include invalid values.
 - The key's permitted-usage algorithm is invalid.
 - The key attributes, as a whole, are invalid.
- **PSA_ERROR_NOT_PERMITTED:**
 - A PSA_KEY_DERIVATION_INPUT_SECRET or PSA_KEY_DERIVATION_INPUT_PASSWORD input was neither provided through a key nor the result of a key agreement.
 - One of the inputs was a key whose policy did not permit PSA_KEY_USAGE_DERIVE.
 - Creating a key with the specified attributes is not permitted.
 - PSA_ERROR_INSUFFICIENT_MEMORY
 - PSA_ERROR_INSUFFICIENT_STORAGE
 - PSA_ERROR_COMMUNICATION_FAILURE
 - PSA_ERROR_CORRUPTION_DETECTED
 - PSA_ERROR_STORAGE_FAILURE
 - PSA_ERROR_DATA_CORRUPT
 - PSA_ERROR_DATA_INVALID
 - **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be active and completed all required input steps.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.2.2.1.12 psa_key_derivation_output_key_custom

[psa_status_t](#) [psa_key_derivation_output_key_custom](#)(const [psa_key_attributes_t](#) *attributes, [psa_key_derivation_operation_t](#) *operation, const [psa_custom_key_parameters_t](#) *custom, const uint8_t *custom_data, size_t custom_data_length, [psa_key_id_t](#) *key)

Derive a key from an ongoing key derivation operation with custom production parameters.

Parameters

- **attributes** (const [psa_key_attributes_t](#) *) - [in] The attributes for the new key.
- **operation** ([psa_key_derivation_operation_t](#) *) - [in] The key derivation operation object to read from.
- **custom** (const [psa_custom_key_parameters_t](#) *) - [in] Customized production parameters for the key derivation.
- **custom_data** (const uint8_t *) - [in] A buffer containing additional variable-sized production parameters.
- **custom_data_length** (size_t) - [in] Length of custom_data in bytes.
- **key** ([psa_key_id_t](#) *) - [out] On success, an identifier for the newly created key. PSA_KEY_ID_NULL on failure.

Description

Warning:

Not supported

This function calculates output bytes from a key derivation algorithm and uses those bytes to generate a key deterministically. The key's location, policy, type and size are taken from `attributes`.

The following new key `attributes` are required:

- The key type. It cannot be an asymmetric public key.
- The key size. It must be a valid size for the key type.
- The key permitted algorithm policy. If the key type to be created is `PSA_KEY_TYPE_PASSWORD_HASH`, then the permitted algorithm policy must be either the same as the current operation's algorithm, or `PSA_ALG_NONE`.
- The key usage flags.

The following new key attributes must be set in case of a persistent key:

- The key lifetime.
- The key identifier.

This function operates in a similar way to [psa_key_derivation_output_key\(\)](#), but enables explicit production parameters to be provided when deriving a key. For example, the production parameters can be used to select an alternative key derivation process, or configure additional key parameters. See [psa_key_derivation_output_key\(\)](#) for the operation of this function with the default production parameters.

See [typedef psa_custom_key_parameters_t](#) for a list of non-default production parameters. See the key type definitions in Key types for details of the custom production parameters used for key derivation.

Note:

When the custom parameter is `PSA_CUSTOM_KEY_PARAMETERS_INIT` with `custom_data_length == 0`, this function is equivalent to the `psa_key_derivation_output_key()` (custom_data is ignored).

Return

- **PSA_SUCCESS:**

Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.

- **PSA_ERROR_ALREADY_EXISTS:**

This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.

- **PSA_ERROR_INSUFFICIENT_DATA:**

The operation's capacity was less than `output_length` bytes. In this case:

- No output is written to the `output` buffer.
- The operation's capacity is set to 0.

- **PSA_ERROR_NOT_SUPPORTED:**

- The key attributes, as a whole, are not supported.
- The production parameters are not supported.

- **PSA_ERROR_INVALID_ARGUMENT:**

- The key type is invalid, or is an asymmetric public-key type.
- The key type is `PSA_KEY_TYPE_PASSWORD_HASH`, and the permitted algorithm policy is not the same as the current operation's algorithm.
- The key size is not valid for the key type.
- The key lifetime is invalid.
- The key identifier is not valid for the key lifetime.
- The key usage flags include invalid values.
- The key's permitted-usage algorithm is invalid.
- The key attributes, as a whole, are invalid.

- The production parameters are invalid.
- **PSA_ERROR_NOT_PERMITTED:**
 - A PSA_KEY_DERIVATION_INPUT_SECRET or PSA_KEY_DERIVATION_INPUT_PASSWORD input was neither provided through a key nor the result of a key agreement.
 - One of the inputs was a key whose policy did not permit PSA_KEY_USAGE_DERIVE.
 - Creating a key with the specified attributes is not permitted.
- PSA_ERROR_INSUFFICIENT_MEMORY
- PSA_ERROR_INSUFFICIENT_STORAGE
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- PSA_ERROR_STORAGE_FAILURE
- PSA_ERROR_DATA_CORRUPT
- PSA_ERROR_DATA_INVALID
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be active and completed all required input steps.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.2.2.13 psa_key_derivation_verify_bytes

[psa_status_t](#) **psa_key_derivation_verify_bytes**([psa_key_derivation_operation_t](#) *operation, const uint8_t *expected_output, size_t output_length)

Compare output data from a key derivation operation to an expected value.

Parameters

- **operation** ([psa_key_derivation_operation_t](#)*) - [in] The key derivation operation object to read from.
- **expected_output** (const uint8_t*) - [in] Buffer containing the expected derivation output.
- **output_length** (size_t) - [in] Length of the expected output. This is also the number of bytes that will be read.

Description

Warning:

Not supported.

This function calculates output bytes from a key derivation algorithm and compares those bytes to an expected value. If the key derivation's output is viewed as a stream of bytes, this function destructively reads `output_length` bytes from the stream before comparing them with `expected_output`. The operation's capacity decreases by the number of bytes read.

This is functionally equivalent to the following code:

```
uint8_t tmp[output_length];

psa_key_derivation_output_bytes(operation, tmp, output_length);

if (memcmp(expected_output, tmp, output_length) != 0)
    return PSA_ERROR_INVALID_SIGNATURE;
```

However, calling [psa_key_derivation_verify_bytes\(\)](#) works even if the key's policy does not allow output of the bytes.

If this function returns an error status other than `PSA_ERROR_INSUFFICIENT_DATA`, the operation enters an error state and must be aborted by calling [psa_key_derivation_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success. The output of the key derivation operation matches `expected_output`.

- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid: it must be active, with all required input steps complete.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

- **PSA_ERROR_INVALID_SIGNATURE:**

The output of the key derivation operation does not match the value in `expected_output`.

- **PSA_ERROR_INSUFFICIENT_DATA:**

The operation's capacity was less than `output_length` bytes. In this case, the operation's capacity is set to zero.

- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`

6.2.2.2.1.14 psa_key_derivation_verify_key

[psa_status_t](#) [psa_key_derivation_verify_key](#)([psa_key_derivation_operation_t](#) *operation, [psa_key_id_t](#) expected)

Compare output data from a key derivation operation to an expected value stored in a key.

Parameters

- **operation** ([psa_key_derivation_operation_t](#)*) - [in] The key derivation operation object to read from.
- **expected** ([psa_key_id_t](#)) - [in] key of type `PSA_KEY_TYPE_PASSWORD_HASH` containing the expected output. The key must allow the usage `PSA_KEY_USAGE_VERIFY_DERIVATION`, and the permitted algorithm must match the operation's algorithm. The value of this key is typically computed by a previous call to [psa_key_derivation_output_key\(\)](#).

Description

Warning:

Not supported.

This function calculates output bytes from a key derivation algorithm and compares those bytes to an expected value, provided as key of type `PSA_KEY_TYPE_PASSWORD_HASH`. If the key derivation's output is viewed as a stream of bytes, this function destructively reads the number of bytes corresponding to the length of the expected key from the stream before comparing them with the key value. The operation's capacity decreases by the number of bytes read.

Note:

This is functionally equivalent to exporting the expected key and calling `psa_key_derivation_verify_bytes()` on the result, except that it works when the key cannot be exported.

If this function returns an error status other than `PSA_ERROR_INSUFFICIENT_DATA`, the operation enters an error state and must be aborted by calling [psa_key_derivation_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success. The output of the key derivation operation matches the expected key value.

- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid: it must be active, with all required input steps complete.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

- **PSA_ERROR_INVALID_HANDLE:**

`expected` is not a valid key identifier.

- **PSA_ERROR_NOT_PERMITTED:**

The key does not have the `PSA_KEY_USAGE_VERIFY_DERIVATION` flag, or it does not permit the requested algorithm.

- **PSA_ERROR_INVALID_SIGNATURE:**

The output of the key derivation operation does not match the value of the expected key.

- **PSA_ERROR_INSUFFICIENT_DATA:**

The operation's capacity was less than the length of the `expected` key. In this case, the operation's capacity is set to zero.

- **PSA_ERROR_INVALID_ARGUMENT:**

The key type is not `PSA_KEY_TYPE_PASSWORD_HASH`.

- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`

6.2.2.2.1.15 `psa_key_derivation_abort`

[psa_status_t](#) `psa_key_derivation_abort`([psa_key_derivation_operation_t](#) *operation)

Abort a key derivation operation.

Parameters

- **operation** ([psa_key_derivation_operation_t](#)*) - [in] The operation to abort.

Description

Aborting an operation frees all associated resources except for the operation object itself. Once aborted, the operation object can be reused for another operation by calling [psa_key_derivation_setup\(\)](#) again.

This function can be called at any time after the operation object has been initialized as described in [typedef `psa\_key\_derivation\_operation\_t`](#).

In particular, it is valid to call [`psa\_key\_derivation\_abort\(\)`](#) twice, or to call [`psa\_key\_derivation\_abort\(\)`](#) on an operation that has not been set up.

Return

- `PSA_SUCCESS`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- **`PSA_ERROR_BAD_STATE`:**

The library has not been previously initialized by [`psa\_crypto\_init\(\)`](#).

6.2.2.2.16 typedef `psa_key_derivation_step_t`

type `psa_key_derivation_step_t`

Encoding of the step of a key derivation.

6.2.2.3 Standalone key agreement

6.2.2.3.1 Functions

6.2.2.3.1.1 `psa_key_agreement`

[`psa\_status\_t`](#) `psa_key_agreement`([`psa\_key\_id\_t`](#) `private_key`, [const `uint8\_t\*`](#) `peer_key`, [size_t](#) `peer_key_length`, [psa_algorithm_t](#) `alg`, [const `psa\_key\_attributes\_t\*`](#) `attributes`, [psa_key_id_t*](#) `key`)

Perform a key agreement and return the shared secret as a derivation key.

Parameters

- **`private_key`** ([psa_key_id_t](#)) - [in] Identifier of the private key to use. It must permit the usage `PSA_KEY_USAGE_DERIVE`.
- **`peer_key`** (const [uint8_t*](#)) - [in] Public key of the peer.
- **`peer_key_length`** (size_t) - [in] Size of `peer_key` in bytes.
- **`alg`** ([psa_algorithm_t](#)) - [in] The standalone key agreement algorithm to compute: a value of [typedef `psa\_algorithm\_t`](#) such that [PSA_ALG_IS_STANDALONE_KEY_AGREEMENT](#) is true.
- **`attributes`** (const [psa_key_attributes_t*](#)) - [in] The attributes for the new key.
- **`key`** ([psa_key_id_t*](#)) - [out] On success, an identifier for the newly created key. `PSA_KEY_ID_NULL` on failure.

Description

Warning:

Not supported.

A key agreement algorithm takes two inputs: a private key `private_key`, and a public key `peer_key`. The result of this function is a shared secret, returned as a derivation key.

The `peer_key` data is parsed with the type `PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR` where `type` is the type of `private_key`, and with the same bit-size as `private_key`. The peer key must be in the format that `psa_import_key()` accepts for this public key type.

This key can be input to a key derivation operation using `psa_key_derivation_input_key()`.

Warning:

The shared secret resulting from a key agreement algorithm such as finite-field Diffie-Hellman or elliptic curve Diffie-Hellman has biases. This makes it unsuitable for use as key material, for example, as an AES key. Instead, it is recommended that a key derivation algorithm is applied to the result, to derive unbiased cryptographic keys.

The following new key `attributes` are required:

- The key type must be one of `PSA_KEY_TYPE_DERIVE`, `PSA_KEY_TYPE_RAW_DATA`, `PSA_KEY_TYPE_HMAC`, or `PSA_KEY_TYPE_PASSWORD`.
- The key permitted algorithm policy.
- The key usage flags.

The following new key attributes must be set in case of a persistent key:

- The key lifetime.
- The key identifier.

The following new key `attributes` is optional:

- The key size if nonzero, it must be equal to the output size of the key agreement, in bits.
The output size, in bits, of the key agreement is $8 * \text{PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE}(\text{type}, \text{bits})$, where `type` and `bits` are the type and bit-size of `private_key`.

Return

- **PSA_SUCCESS:**

Success. The new key contains the share secret. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.

- **PSA_ERROR_BAD_STATE:**

- The library has not been previously initialized by `psa_crypto_init()`.

- **PSA_ERROR_INVALID_HANDLE:**

`private_key` is not a valid key identifier.

- **PSA_ERROR_NOT_PERMITTED:**

- `private_key` does not have the `PSA_KEY_USAGE_DERIVE` flag, or it does not permit the requested algorithm.
- Creating a key with the specified attributes is not permitted.

- **PSA_ERROR_ALREADY_EXISTS:**

This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not a key agreement algorithm.
- `private_key` is not compatible with `alg`.
- `peer_key` is not a valid public key corresponding to `private_key`.
- The output key attributes in `attributes` are not valid:
 - The key type is not valid for key agreement output.
 - The key size is nonzero, and is not the size of the shared secret.

- The key lifetime is invalid.
- The key identifier is not valid for the key lifetime.
- The key usage flags include invalid values.
- The key's permitted-usage algorithm is invalid.
- The key attributes, as a whole, are invalid.

• **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported.
- `private_key` is not supported for use with `alg`.
- The output key attributes, as a whole, are not supported.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_INSUFFICIENT_STORAGE**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**

6.2.2.3.1.2 `psa_raw_key_agreement`

`psa_status_t` `psa_raw_key_agreement`(`psa_algorithm_t` `alg`, `psa_key_id_t` `private_key`, `const uint8_t*` `peer_key`, `size_t` `peer_key_length`, `uint8_t*` `output`, `size_t` `output_size`, `size_t*` `output_length`)

Perform a key agreement and return the raw shared secret.

Parameters

- **`alg`** (`psa_algorithm_t`) - [in] The key agreement algorithm to compute (PSA_ALG_XXX value such that `PSA_ALG_IS_RAW_KEY_AGREEMENT` is true).
- **`private_key`** (`psa_key_id_t`) - [in] Identifier of the private key to use. It must allow the usage `PSA_KEY_USAGE_DERIVE`.
- **`peer_key`** (`const uint8_t*`) - [in] Public key of the peer.
- **`peer_key_length`** (`size_t`) - [in] Size of `peer_key` in bytes.
- **`output`** (`uint8_t*`) - [out] Buffer where the raw shared secret is to be written.
- **`output_size`** (`size_t`) - [in] Size of the `output` buffer in bytes.
- **`output_length`** (`size_t*`) - [out] On success, the number of bytes that make up the returned output.

Description

Warning:

Not supported.

A key agreement algorithm takes two inputs: a private key `private_key`, and a public key `peer_key`. The result of this function is a shared secret, returned as a derivation key.

The `peer_key` data is parsed with the type `PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR` where type is the type of `private_key`, and with the same bit-size as `private_key`. The peer key must be in the format that `psa_import_key()` accepts for this public key type.

Warning:

The raw result of a key agreement algorithm such as finite-field Diffie-Hellman or elliptic curve Diffie-Hellman has biases, and is not suitable for use as key material. Instead it is recommended that the result is used as input to a key derivation algorithm.

To chain a key agreement with a key derivation, use `psa_key_derivation_key_agreement()` and other functions from the key derivation interface.

The `output_size` parameter must be appropriate for the keys:

- The required output size is `PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE(type, bits)` where `type` is the type of `private_key` and `bits` is the bit-size of either `private_key` or the `peer_key`.
- `PSA_RAW_KEY_AGREEMENT_OUTPUT_MAX_SIZE` evaluates to the maximum output size of any supported raw key agreement algorithm.

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_INVALID_HANDLE:**

`private_key` is not a valid key identifier.

- **PSA_ERROR_NOT_PERMITTED:**

`private_key` does not have the `PSA_KEY_USAGE_DERIVE` flag, or it does not permit the requested algorithm.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not a key agreement algorithm.
- `private_key` is not compatible with `alg`.
- `peer_key` is not valid public key corresponding to `private_key`.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the `output` buffer is too small. `PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE()` or `PSA_RAW_KEY_AGREEMENT_OUTPUT_MAX_SIZE` can be used to determine the required buffer size.

- **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not a supported key agreement algorithm.
- `private_key` is not supported for use with `alg`.

- **PSA_ERROR_INSUFFICIENT_MEMORY**

- **PSA_ERROR_COMMUNICATION_FAILURE**

- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_STORAGE_FAILURE**

- **PSA_ERROR_DATA_CORRUPT**

- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

- The library has not been previously initialized by `psa_crypto_init()`.

6.2.2.4 Combining key agreement and key derivation

6.2.2.4.1 Function

6.2.2.4.1.1 psa_key_derivation_key_agreement

[psa_status_t](#) [psa_key_derivation_key_agreement](#)([psa_key_derivation_operation_t](#) *operation, [psa_key_derivation_step_t](#) step, [psa_key_id_t](#) private_key, const uint8_t *peer_key, size_t peer_key_length)

Perform a key agreement and use the shared secret as input to a key derivation.

Parameters

- **operation** ([psa_key_derivation_operation_t](#)*) - [in] The key derivation operation object to use. It must have been set up with [psa_key_derivation_setup\(\)](#) with a key agreement and derivation algorithm (PSA_ALG_XXX value such that [PSA_ALG_IS_KEY_AGREEMENT](#) is true and [PSA_ALG_IS_RAW_KEY_AGREEMENT](#) is false). The operation must be ready for an input of the type given by step.
- **step** ([psa_key_derivation_step_t](#)) - [in] Which step the input data is for.
- **private_key** ([psa_key_id_t](#)) - [in] Identifier of the private key to use. It must allow the usage PSA_KEY_USAGE_DERIVE.
- **peer_key** (const uint8_t*) - [in] Public key of the peer.
- **peer_key_length** (size_t) - [in] Size of peer_key in bytes.

Description

A key agreement algorithm takes two inputs: a private key `private_key` a public key `peer_key`. The result of this function is passed as input to a key derivation. The output of this key derivation can be extracted by reading from the resulting operation to produce keys and other cryptographic material.

The `peer_key` data is parsed with the type [PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR](#) where type is the type of `private_key`, and with the same bit-size as `private_key`. The peer key must be in the format that [psa_import_key\(\)](#) accepts for this public key type.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_key_derivation_abort\(\)](#).

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE:**
`private_key` is not a valid key identifier.
- **PSA_ERROR_NOT_PERMITTED:**
`private_key` does not have the PSA_KEY_USAGE_DERIVE flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
 - The operation's algorithm is not a key-agreement algorithm.
 - `step` does not permit an input resulting from a key agreement.
 - `private_key` is not compatible with the operation's algorithm.
 - `peer_key` is not a valid public key corresponding to `private_key`.
- **PSA_ERROR_NOT_SUPPORTED:**

`private_key` is not supported for use with the operation's algorithm.

- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- **`PSA_ERROR_BAD_STATE`:**
 - The operation state is not valid for this key agreement step.
 - The library has not been previously initialized by `psa_crypto_init()`.

6.2.2.5 Key Exportation

6.2.2.5.1 Function

6.2.2.5.1.1 `psa_export_key`

`psa_status_t` `psa_export_key`(`psa_key_id_t` `key`, `uint8_t *``data`, `size_t` `data_size`, `size_t *``data_length`)

Export a key in binary format.

Parameters

- **`key`** (`psa_key_id_t`) - [in] Identifier of the key to export. It must allow the usage `PSA_KEY_USAGE_EXPORT`, unless it is a public key.
- **`data`** (`uint8_t *`) - [out] Buffer where the key data is to be written.
- **`data_size`** (`size_t`) - [in] Size of the `data` buffer in bytes.
- **`data_length`** (`size_t *`) - [out] On success, the number of bytes that make up the key data.

Description

Warning:

Private or secure key are not exportable in plain text, only asymmetric public key can be exported.

In the current implementation only the asymmetric public keys is exported in a standard binary format.

Parameter `data_size` must be appropriate for the key:

- The required output size is `PSA_EXPORT_KEY_OUTPUT_SIZE()` where `type` is the key type and `bits` is the key size in bits.
- `PSA_EXPORTASYMMETRICKEYMAXSIZE` evaluates to the maximum output size of any supported public key or key pair.
- `PSA_EXPORTKEYPAIRMAXSIZE` evaluates to the maximum output size of any supported key pair.
- `PSA_EXPORTPUBLICKEYMAXSIZE` evaluates to the maximum output size of any supported public key.
- This API defines no maximum size for symmetric keys. Arbitrarily large data items can be stored in the key store, for example certificates that correspond to a stored private key or input material for key derivation.

Return

- **`PSA_SUCCESS`:**

Success. The first `data_length` bytes of data contain the exported key.

- **PSA_ERROR_INVALID_HANDLE:**

The `key` handle is invalid.

- **PSA_ERROR_NOT_PERMITTED:**

The key does not have the `PSA_KEY_USAGE_EXPORT` flag.

- **PSA_ERROR_NOT_SUPPORTED:**

- The key's storage location does not support export of the key.
- This type of key can't be exported.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the `data` buffer is too small.

- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.2.5.1.2 `psa_export_public_key`

[psa_status_t](#) `psa_export_public_key`([psa_key_id_t](#) `key`, [uint8_t*](#) `data`, [size_t](#) `data_size`, [size_t*](#) `data_length`)

Export a public key or the public part of a key pair in binary format.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to export.
- **data** ([uint8_t*](#)) - [out] Buffer where the key data is to be written.
- **data_size** ([size_t](#)) - [in] Size of the `data` buffer in bytes.
- **data_length** ([size_t*](#)) - [out] On success, the number of bytes that make up the key data.

Description

Exporting a public key object or the public part of a key pair is always permitted, regardless of the key's usage flags.

Parameter `data_size` must be appropriate for the key:

- The required output size is [PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE\(\)](#) where `type` is the key type and `bits` is the key size in bits.
- [PSA_EXPORT_PUBLIC_KEY_MAX_SIZE](#) evaluates to the maximum output size of any supported public key or public part of a key pair.
- [PSA_EXPORTASYMMETRICKEY_MAX_SIZE](#) evaluates to the maximum output size of any supported public key or key pair

Return

- **PSA_SUCCESS**
- **PSA_ERROR_INVALID_HANDLE**

- **PSA_ERROR_INVALID_ARGUMENT:**

The key is neither a public key nor a key pair.

- **PSA_ERROR_NOT_SUPPORTED:**

- The key's storage location does not support export of the key.
- This type of key can't be exported.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the `data` buffer is too small.

- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_INSUFFICIENT_MEMORY**

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.2.5.2 Macros

6.2.2.5.2.1 macro PSA_EXPORT_KEY_OUTPUT_SIZE

PSA_EXPORT_KEY_OUTPUT_SIZE(*key_type*, *key_bits*)

Sufficient output buffer size for [psa_export_key\(\)](#).

Parameters

- **key_type** - [in] A supported key type.
- **key_bits** - [in] The size of the key in bits.

Description

The following code illustrates how to allocate enough memory to export a key by querying the key type and size at runtime.

```
psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
psa_status_t status;

status = psa_get_key_attributes(key, &attributes);
if (status != PSA_SUCCESS)
    handle_error(...);

psa_key_type_t key_type = psa_get_key_type(&attributes);
size_t key_bits = psa_get_key_bits(&attributes);
size_t buffer_size = PSA_EXPORT_KEY_OUTPUT_SIZE(key_type, key_bits);

psa_reset_key_attributes(&attributes);

uint8_t *buffer = malloc(buffer_size);
if (buffer == NULL)
    handle_error(...);
```

```
size_t buffer_length;
status = psa_export_key(key, buffer, buffer_size, &buffer_length);
if (status != PSA_SUCCESS)
    handle_error(...);
```

See also [PSA_EXPORT_KEY_PAIR_MAX_SIZE](#), [PSA_EXPORT_PUBLIC_KEY_MAX_SIZE](#), and [PSA_EXPORTASYMMETRIC_KEY_MAX_SIZE](#).

Return

If the parameters are valid and supported, return a buffer size in bytes that guarantees that [psa_export_key\(\)](#) or [psa_export_public_key\(\)](#) will not fail with `PSA_ERROR_BUFFER_TOO_SMALL`. If the parameters are a valid combination that is not supported by the implementation, this macro must return either a sensible size or 0. If the parameters are not valid, the return value is unspecified.

6.2.2.5.2.2 macro `PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE`

`PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE`(*key_type*, *key_bits*)

Sufficient output buffer size for [psa_export_public_key\(\)](#).

Parameters

- **`key_type`** - [in] A public key or key pair key type.
- **`key_bits`** - [in] The size of the key in bits.

Description

The following code illustrates how to allocate enough memory to export a public key by querying the key type and size at runtime.

```
psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
psa_status_t status;

status = psa_get_key_attributes(key, &attributes);
if (status != PSA_SUCCESS)
    handle_error(...);

psa_key_type_t key_type = psa_get_key_type(&attributes);
size_t key_bits = psa_get_key_bits(&attributes);
size_t buffer_size = PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE(key_type, key_bits);

psa_reset_key_attributes(&attributes);

uint8_t *buffer = malloc(buffer_size);
if (buffer == NULL)
    handle_error(...);

size_t buffer_length;
status = psa_export_public_key(key, buffer, buffer_size, &buffer_length);
if (status != PSA_SUCCESS)
    handle_error(...);
```

See also [PSA_EXPORT_PUBLIC_KEY_MAX_SIZE](#) and [PSA_EXPORTASYMMETRIC_KEY_MAX_SIZE](#).

Return

If the parameters are valid and supported, return a buffer size in bytes that guarantees that [psa_export_public_key\(\)](#) will not fail with `PSA_ERROR_BUFFER_TOO_SMALL`. Otherwise return 0.

6.2.2.5.2.3 macro `PSA_EXPORT_KEY_PAIR_MAX_SIZE`

`PSA_EXPORT_KEY_PAIR_MAX_SIZE()`

Sufficient buffer size for exporting any asymmetric key pair.

Description

This value must be a sufficient buffer size when calling [psa_export_key\(\)](#) to export any asymmetric key pair that is supported by the implementation, regardless of the exact key type and key size.

See also [PSA_EXPORT_KEY_OUTPUT_SIZE\(\)](#), [PSA_EXPORT_PUBLIC_KEY_MAX_SIZE](#), and [PSA_EXPORT_ASYMMETRIC_KEY_MAX_SIZE](#).

6.2.2.5.2.4 macro `PSA_EXPORT_PUBLIC_KEY_MAX_SIZE`

`PSA_EXPORT_PUBLIC_KEY_MAX_SIZE()`

Sufficient buffer size for exporting any asymmetric public key.

Description

This value must be a sufficient buffer size when calling [psa_export_key\(\)](#) or [psa_export_public_key\(\)](#) to export any asymmetric public key that is supported by the implementation, regardless of the exact key type and key size.

See also [PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE\(\)](#), [PSA_EXPORT_KEY_OUTPUT_SIZE\(\)](#), [PSA_EXPORT_KEY_PAIR_MAX_SIZE](#), and [PSA_EXPORT_ASYMMETRIC_KEY_MAX_SIZE](#).

6.2.2.5.2.5 macro `PSA_EXPORT_ASYMMETRIC_KEY_MAX_SIZE`

`PSA_EXPORT_ASYMMETRIC_KEY_MAX_SIZE()`

Sufficient buffer size for exporting any asymmetric key pair or public key.

Description

This value must be a sufficient buffer size when calling [psa_export_key\(\)](#) or [psa_export_public_key\(\)](#) to export any asymmetric key pair or public key that is supported by the implementation, regardless of the exact key type and key size.

See also [PSA_EXPORT_KEY_PAIR_MAX_SIZE](#), [PSA_EXPORT_PUBLIC_KEY_MAX_SIZE](#), and [PSA_EXPORT_KEY_OUTPUT_SIZE\(\)](#).

6.2.2.6 Key Deletion

6.2.2.6.1 Functions

6.2.2.6.1.1 `psa_destroy_key`

[psa_status_t](#) `psa_destroy_key`([psa_key_id_t](#) key)

Destroy a key.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to erase. If this is `PSA_KEY_ID_NULL`, do nothing and return `PSA_SUCCESS`.

Description

This function destroys a key from both volatile memory and, if applicable, non-volatile storage.

Destroying the key makes the key identifier invalid, and the key identifier must not be used again by the application.

If a key is currently in use in a multi-part operation, then destroying the key will cause the multi-part operation to fail.

Return

- **PSA_SUCCESS:**

`key` was a valid key identifier and the key material that it referred to has been erased. Alternatively, key is `PSA_KEY_ID_NULL`.

- **PSA_ERROR_NOT_PERMITTED:**

The key cannot be erased because it is read-only, either due to a policy or due to physical restrictions.

- **PSA_ERROR_INVALID_HANDLE:**

`key` is not a valid handle nor `PSA_KEY_ID_NULL`.

- **PSA_ERROR_COMMUNICATION_FAILURE:**

There was an failure in communication with the cryptoprocessor. The key material might still be present in the cryptoprocessor.

- **PSA_ERROR_STORAGE_FAILURE:**

The storage operation failed.

- **PSA_ERROR_DATA_CORRUPT:**

The storage is corrupted.

- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_CORRUPTION_DETECTED:**

An unexpected condition which is not a storage corruption or a communication failure occurred. The cryptoprocessor might have been compromised.

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.2.6.1.2 psa_purge_key

[psa_status_t](#) **psa_purge_key**([psa_key_id_t](#) key)

Remove non-essential copies of key material from memory.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to purge.

Description

Warning:

Not supported.

For keys that have been created with the PSA_KEY_USAGE_CACHE usage flag, an implementation is permitted to make additional copies of the key material that are not in storage and not for the purpose of ongoing operations.

This function will remove these extra copies of the key material from memory.

This function is not required to remove key material from memory in any of the following situations:

- The key is currently in use in a cryptographic operation.
- The key is volatile.

Return

- **PSA_SUCCESS:**

The key material will have been removed from memory if it is not currently required.

- PSA_ERROR_INVALID_HANDLE
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_STORAGE_FAILURE
- PSA_ERROR_DATA_CORRUPT
- PSA_ERROR_DATA_INVALID
- PSA_ERROR_CORRUPTION_DETECTED
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.2.7 Key Encapsulate

6.2.2.7.1 Functions

6.2.2.7.1.1 psa_encapsulate

[psa_status_t](#) **psa_encapsulate**([psa_key_id_t](#) key, [psa_algorithm_t](#) alg, const [psa_key_attributes_t](#) *attributes, [psa_key_id_t](#) *output_key, [uint8_t](#) *ciphertext, [size_t](#) ciphertext_size, [size_t](#) *ciphertext_length)

Use a public key to generate a new shared secret key and associated ciphertext.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the encapsulation. It must be a public key or an asymmetric key pair. It must permit the usage `PSA_KEY_USAGE_ENCRYPT`.
- **alg** ([psa_algorithm_t](#)) - [in] The key-encapsulation algorithm such that [PSA_ALG_IS_KEY_ENCAPSULATION](#) is true.
- **attributes** (const [psa_key_attributes_t*](#)) - [in] The attributes for the output key.
- **output_key** ([psa_key_id_t*](#)) - [out] On success, an identifier for the newly created shared secret key. `PSA_KEY_ID_NULL` on failure.
- **ciphertext** ([uint8_t*](#)) - [out] Buffer where the ciphertext output is to be written.
- **ciphertext_size** ([size_t](#)) - [in] Size of the `ciphertext` buffer in bytes.
- **ciphertext_length** ([size_t*](#)) - [out] On success, the number of bytes that make up the ciphertext value.

Description

Warning:

Not supported.

The following new key `attributes` are required:

- The key type. All key-encapsulation algorithms can output a key of type `PSA_KEY_TYPE_DERIVE` or `PSA_KEY_TYPE_HMAC`. Key encapsulation algorithms that produce a uniformly pseudorandom shared secret, can also output block-cipher key types, for example `PSA_KEY_TYPE_AES`.
- The key permitted algorithm policy.
- The key usage flags.

The following new key attributes must be set in case of a persistent key:

- The key lifetime.
- The key identifier.

The following new key `attributes` is optional:

- The key size, if is nonzero, it must be equal to the size, in bits, of the shared secret.

The `output_key` location, policy, and type are taken from `attributes`.

The size of the returned key is always the bit-size of the shared secret, rounded up to a whole number of bytes. The size of the shared secret is dependent on the key encapsulation algorithm and the type and size of `key`.

It is recommended that the shared secret key is used as an input to a key derivation operation to produce additional cryptographic keys. For some key encapsulation algorithms, the shared secret key is also suitable for use as a key in cryptographic operations such as encryption.

The output `ciphertext` is to be sent to the other participant, who uses the decapsulation key to extract another copy of the shared secret key.

Return

• **PSA_SUCCESS:**

Success. The bytes of `ciphertext` contain the data to be sent to the other participant, and `output_key` contains the identifier for the shared secret key.

• **PSA_ERROR_INVALID_HANDLE:**

`key` is not a valid key identifier.

- **PSA_ERROR_NOT_PERMITTED:**

- The key does not have the `PSA_KEY_USAGE_ENCRYPT` flag, or it does not permit the requested algorithm.
- Creating a key with the specified attributes is not permitted.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not a key encapsulation algorithm.
- `key` is not a public key or an asymmetric key pair, that is compatible with `alg`.
- The output key attributes in `attributes` are not valid:
 - The key type is not valid for the shared secret.
 - The key size is nonzero, and is not the size of the shared secret.
 - The key lifetime is invalid.
 - The key identifier is not valid for the key lifetime.
 - The key usage flags include invalid values.
 - The key's permitted-usage algorithm is invalid.
 - The key attributes, as a whole, are invalid.

- **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported.
- `key` is not supported for use with `alg`.
- The output key attributes, as a whole, are not supported.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`.

6.2.2.7.1.2 `psa_decapsulate`

`psa_status_t` `psa_decapsulate`(`psa_key_id_t` `key`, `psa_algorithm_t` `alg`, `const uint8_t*` `ciphertext`, `size_t` `ciphertext_length`, `const psa_key_attributes_t*` `attributes`, `psa_key_id_t*` `output_key`)

Use a private key to decapsulate a shared secret key from a ciphertext.

Parameters

- **key** (`psa_key_id_t`) - [in] Identifier of the key to use for the decapsulation. It must be an asymmetric key pair. It must permit the usage `PSA_KEY_USAGE_DECRYPT`.
- **alg** (`psa_algorithm_t`) - [in] The key-encapsulation algorithm such that `PSA_ALG_IS_KEY_ENCAPSULATION` is true.
- **ciphertext** (`const uint8_t*`) - [in] The ciphertext received from the other participant.
- **ciphertext_length** (`size_t`) - [in] Size of the `ciphertext` buffer in bytes.
- **attributes** (`const psa_key_attributes_t*`) - [in] The attributes for the output key.
- **output_key** (`psa_key_id_t*`) - [out] On success, an identifier for the newly created shared secret key. `PSA_KEY_ID_NULL` on failure.

Description

Warning:

Not supported.

The following new key `attributes` are required:

- The key type. All key-encapsulation algorithms can output a key of type `PSA_KEY_TYPE_DERIVE` or `PSA_KEY_TYPE_HMAC`. Key encapsulation algorithms that produce a uniformly pseudorandom shared secret, can also output block-cipher key types, for example `PSA_KEY_TYPE_AES`.
- The key permitted algorithm policy.
- The key usage flags.

The following new key attributes must be set in case of a persistent key:

- The key lifetime.
- The key identifier.

The following new key `attributes` is optional:

- The key size, if is nonzero, it must be equal to the size, in bits, of the shared secret.

The `output_key` location, policy, and type are taken from `attributes`.

The size of the returned key is always the bit-size of the shared secret, rounded up to a whole number of bytes. The size of the shared secret is dependent on the key-encapsulation algorithm and the type and size of key.

It is recommended that the shared secret key is used as an input to a key derivation operation to produce additional cryptographic keys. For some key encapsulation algorithms, the shared secret key is also suitable for use as a key in cryptographic operations such as encryption.

If the key encapsulation protocol is executed correctly then, with overwhelming probability, the two copies of the shared secret are identical. However, the protocol does not protect one participant against the other participant executing it incorrectly, or against a third party modifying data in transit.

Warning:

A `PSA_SUCCESS` result from `psa_decapsulate()` does not guarantee that the output key is identical to the key produced by the call to `psa_encapsulate()`. For example, `PSA_SUCCESS` can be returned with a mismatched shared secret key value in the following situations:

- *The key encapsulation algorithm does not authenticate the ciphertext. Manipulated or corrupted ciphertext will not be detected during decapsulation.*
- *The key encapsulation algorithm reports authentication failure implicitly, by returning a pseudorandom key value. This is done to prevent disclosing information to an attacker that has manipulated the ciphertext.*
- *The key encapsulation algorithm is probabilistic, and will extremely rarely result in non-identical key values.*

It is strongly recommended that the application uses the output key in a way that will confirm that the shared secret keys are identical.

Return

- **PSA_SUCCESS:**

Success. `output_key` contains the identifier for the shared secret key.

- **PSA_ERROR_INVALID_HANDLE:**

`key` is not a valid key identifier.

- **PSA_ERROR_NOT_PERMITTED:**

- The key does not have the `PSA_KEY_USAGE_DECRYPT` flag, or it does not permit the requested algorithm.
- Creating a key with the specified attributes is not permitted.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not a key encapsulation algorithm.
- `key` is not an asymmetric key pair, that is compatible with `alg`.
- The output key attributes in `attributes` are not valid:
 - The key type is not valid for the shared secret.
 - The key size is nonzero, and is not the size of the shared secret.
 - The key lifetime is invalid.
 - The key identifier is not valid for the key lifetime.
 - The key usage flags include invalid values.
 - The key's permitted-usage algorithm is invalid.
 - The key attributes, as a whole, are invalid.
- `ciphertext` is obviously invalid for the selected algorithm and key.

- **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported.
- `key` is not supported for use with `alg`.
- The output key attributes, as a whole, are not supported.

- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`.

6.2.2.7.2 Macros

6.2.2.7.2.1 macro `PSA_ENCAPSULATE_CIPHERTEXT_SIZE`

`PSA_ENCAPSULATE_CIPHERTEXT_SIZE`(*key_type*, *key_bits*, *alg*)

Sufficient ciphertext buffer size for `psa_encapsulate()`, in bytes.

Parameters

- **`key_type`** - [in] A key type that is compatible with algorithm `alg`.
- **`key_bits`** - [in] The size of the key in bits.
- **`alg`** - [in] A key-encapsulation algorithm such that `PSA_ALG_IS_KEY_ENCAPSULATION` is true.

Description

Warning:

Not supported.

If the size of the ciphertext buffer is at least this large, it is guaranteed that [psa_encapsulate\(\)](#) will not fail due to an insufficient buffer size. The actual size of the ciphertext might be smaller in any given call.

See also [PSA_ENCAPSULATE_CIPHERTEXT_MAX_SIZE](#).

Return

A sufficient ciphertext buffer size for the specified algorithm, key type, and size. If algorithm, key type and size is not recognize, return 0.

6.2.2.7.2.2 macro PSA_ENCAPSULATE_CIPHERTEXT_MAX_SIZE

PSA_ENCAPSULATE_CIPHERTEXT_MAX_SIZE()

The maximum ciphertext buffer size for [psa_encapsulate\(\)](#), for any of the supported key types and key encapsulation algorithms.

Description

Warning:

Not supported.

If the size of the ciphertext buffer is at least this large, it is guaranteed that [psa_encapsulate\(\)](#) will not fail due to an insufficient buffer size.

See also [PSA_ENCAPSULATE_CIPHERTEXT_SIZE](#).

6.2.2.8 Key Attributes

6.2.2.8.1 Attributes

6.2.2.8.1.1 typedef psa_key_attributes_t

type **psa_key_attributes_t**

The type of an object containing key attributes.

Description

This is the object that represents the metadata of a key object. Metadata that can be stored in attributes includes:

- The location of the key in storage, indicated by its key identifier and its lifetime.
- The key's policy, comprising usage flags and a specification of the permitted algorithm(s).
- Information about the key itself: the key type and its size.
- Implementation specific attributes.

The actual key material is not considered an attribute of a key. Key attributes do not contain information that is generally considered highly confidential.

Each attribute of this object is set with a function [psa_set_key_xxx\(\)](#) and retrieved with a function [psa_get_key_xxx\(\)](#).

An attribute object can contain references to auxiliary resources, for example pointers to allocated memory or indirect references to pre-calculated values. In order to free such resources, the application must call [psa_reset_key_attributes\(\)](#). As an exception, calling [psa_reset_key_attributes\(\)](#) on an attribute

object is optional if the object has only been modified by the following functions since it was initialized or last reset with [psa_reset_key_attributes\(\)](#):

- [psa_set_key_id\(\)](#)
- [psa_set_key_lifetime\(\)](#)
- [psa_set_key_type\(\)](#)
- [psa_set_key_bits\(\)](#)
- [psa_set_key_usage_flags\(\)](#)
- [psa_set_key_algorithm\(\)](#)

Before calling any function on a key attribute object, the application must initialize it by any of the following means:

- Set the object to all-bits-zero, for example:

```
psa_key_attributes_t attributes;
memset(&attributes, 0, sizeof(attributes));
```

- Initialize the object to logical zero values by declaring the object as static or global without an explicit initializer, for example:

```
static psa_key_attributes_t attributes;
```

- Initialize the object to the initializer [PSA_KEY_ATTRIBUTES_INIT](#), for example:

```
psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
```

- Assign the result of the function [psa_key_attributes_init\(\)](#) to the object, for example:

```
psa_key_attributes_t attributes;
attributes = psa_key_attributes_init();
```

A freshly initialized attribute object contains the following values:

Attribute	Value
lifetime	PSA_KEY_LIFETIME_VOLATILE.
key identifier	PSA_KEY_ID_NULL - which is not a valid key identifier.
type	PSA_KEY_TYPE_NONE - meaning that the type is unspecified.
key size	0 - meaning that the size is unspecified.
usage flags	0 - which allows no usage except exporting a public key.
algorithm	PSA_ALG_NONE - which does not allow cryptographic usage, but allows exporting.

Usage

A typical sequence to create a key is as follows:

1. Create and initialize an attribute object.
2. If the key is persistent, call [psa_set_key_id\(\)](#). Also call [psa_set_key_lifetime\(\)](#) to place the key in a non-default location.
3. Set the key policy with [psa_set_key_usage_flags\(\)](#) and [psa_set_key_algorithm\(\)](#).
4. Set the key type with [psa_set_key_type\(\)](#). Skip this step if copying an existing key with [psa_copy_key\(\)](#).

5. When generating a random key with [psa_generate_key\(\)](#) or deriving a key with [psa_key_derivation_output_key\(\)](#), set the desired key size with [psa_set_key_bits\(\)](#).
6. Call a key creation function: [psa_import_key\(\)](#), [psa_generate_key\(\)](#), [psa_key_derivation_output_key\(\)](#) or [psa_copy_key\(\)](#). This function reads the attribute object, creates a key with these attributes, and outputs an identifier for the newly created key.
7. Optionally call [psa_reset_key_attributes\(\)](#), now that the attribute object is no longer needed. Currently this call is not required as the attributes defined in this specification do not require additional resources beyond the object itself.

A typical sequence to query a key's attributes is as follows:

1. Call [psa_get_key_attributes\(\)](#).
2. Call [psa_get_key_xxx\(\)](#) functions to retrieve the required attribute(s).
3. Call [psa_reset_key_attributes\(\)](#) to free any resources that can be used by the attribute object.

Once a key has been created, it is impossible to change its attributes.

6.2.2.8.1.2 macro PSA_KEY_ATTRIBUTES_INIT

PSA_KEY_ATTRIBUTES_INIT()

Initializer for a key attribute object of [typedef psa_key_attributes_t](#).

6.2.2.8.1.3 psa_reset_key_attributes

`void psa_reset_key_attributes(psa\_key\_attributes\_t *attributes)`

Reset a key attribute object to a freshly initialized state.

Parameters

- **attributes** ([psa_key_attributes_t](#)*) - [in/out] The attribute object to reset.

Description

The attribute object must be initialized as described in the documentation of the [typedef `psa\_key\_attributes\_t`](#) before calling this function. Once the object has been initialized, this function can be called at any time.

This function frees any auxiliary resources that the object might contain.

Return

void

6.2.2.8.1.4 psa_key_attributes_init

`psa\_key\_attributes\_t psa_key_attributes_init(void)`

Return an initial value for a key attribute object.

Parameters

- **void** - no arguments

Return

Initialized value of a key attribute.

6.2.2.8.1.5 psa_get_key_attributes

[psa_status_t](#) **psa_get_key_attributes**([psa_key_id_t](#) key, [psa_key_attributes_t](#) *attributes)

Retrieve the attributes of a key.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to query.
- **attributes** ([psa_key_attributes_t](#)*) - [out] Attributes of the key.

Description

This function first resets the attribute object as with [psa_reset_key_attributes\(\)](#). It then copies the key attributes retrieved from the secure subsystem owning the key. If some key attributes are not supported, the returned values are empty.

On failure, it is equivalent to a freshly-initialized attribute object.

Note:

This function clears any previous content from the attribute object and therefore expects it to be in a valid state. In particular, if this function is called on a newly allocated attribute object, the attribute object must be initialized before calling this function.

Note:

This function might allocate memory or other resources. Once this function has been called on an attribute object, [psa_reset_key_attributes\(\)](#) must be called to free these resources.

Return

- PSA_SUCCESS
- **PSA_ERROR_INVALID_ARGUMENT:**
 - attributes is NULL.
 - key is 0.
- PSA_ERROR_INVALID_HANDLE
- PSA_ERROR_INSUFFICIENT_MEMORY
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- PSA_ERROR_STORAGE_FAILURE
- PSA_ERROR_DATA_CORRUPT
- PSA_ERROR_DATA_INVALID
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.2.8.1.6 psa_set_key_id

void **psa_set_key_id**([psa_key_attributes_t](#) *attributes, [psa_key_id_t](#) id)

Declare a key as persistent and set its key identifier.

Parameters

- **attributes** ([psa_key_attributes_t](#)*) - [in/out] The attribute object to write to.
- **id** ([psa_key_id_t](#)) - [in] The persistent identifier for the key.

Description

The application must choose a value for `id` between [PSA_KEY_ID_USER_MIN](#) and [PSA_KEY_ID_USER_MAX](#).

If the attribute object currently declares the key as volatile, which is the default lifetime of an attribute object, this function sets the lifetime attribute to [PSA_KEY_LIFETIME_PERSISTENT](#).

This function does not access storage, it merely stores the given value in the attribute object.

The persistent key will be written to storage when the attribute object is passed to a key creation function such as [psa_import_key\(\)](#), [psa_generate_key\(\)](#), [psa_key_derivation_output_key\(\)](#), [psa_copy_key\(\)](#), ...

Return

void

6.2.2.8.1.7 psa_get_key_id

[psa_key_id_t](#) **psa_get_key_id**(const [psa_key_attributes_t](#) *attributes)

Retrieve the key identifier from key attributes.

Parameters

- **attributes** (const [psa_key_attributes_t](#) *) - [in] The key attribute object to query.

Return

The persistent identifier stored in the attribute object. This value is unspecified if the attribute object declares the key as volatile.

6.2.2.8.1.8 psa_set_key_algorithm

void **psa_set_key_algorithm**([psa_key_attributes_t](#) *attributes, [psa_algorithm_t](#) alg)

Declare the permitted algorithm policy for a key.

Parameters

- **attributes** ([psa_key_attributes_t](#) *) - [in/out] The attribute object to write to.
- **alg** ([psa_algorithm_t](#)) - [in] The permitted algorithm to write.

Description

The permitted algorithm policy of a key encodes which algorithm or algorithms are permitted to be used with this key.

This function overwrites any permitted algorithm policy previously set in `attributes`.

6.2.2.8.1.9 psa_set_key_bits

void **psa_set_key_bits**([psa_key_attributes_t](#) *attributes, [size_t](#) bits)

Declare the size of a key.

Parameters

- **attributes** ([psa_key_attributes_t](#)*) - [in/out] The attribute object to write to.
- **bits** ([size_t](#)) - [in] The key size in bits. If this is 0, the key size in `attributes` becomes unspecified. Keys of size 0 are not supported.

Description

This function overwrites any key size previously set in `attributes`.

6.2.2.8.1.10 `psa_get_key_algorithm`

[psa_algorithm_t](#) `psa_get_key_algorithm(const psa\_key\_attributes\_t *attributes)`

Retrieve the permitted algorithm policy from key attributes.

Parameters

- **attributes** (const [psa_key_attributes_t](#)*) - [in] The key attribute object to query.

Return

The algorithm stored in the attribute object.

6.2.2.8.1.11 `psa_get_key_bits`

[size_t](#) `psa_get_key_bits(const psa\_key\_attributes\_t *attributes)`

Retrieve the key size from key attributes.

Parameters

- **attributes** (const [psa_key_attributes_t](#)*) - [in] The key attribute object to query.

Return

The key size stored in the attribute object, in bits.

6.2.2.8.2 Key identifier

6.2.2.8.2.1 `typedef psa_key_id_t`

type `psa_key_id_t`

Key identifier.

Description

A key identifiers can be a permanent name for a persistent key, or a transient reference to volatile key.

6.2.2.8.2.2 macro `PSA_KEY_ID_USER_MAX`

`PSA_KEY_ID_USER_MAX()`

The maximum value for a key identifier chosen by the application.

6.2.2.8.2.3 macro PSA_KEY_ID_USER_MIN

PSA_KEY_ID_USER_MIN()

The minimum value for a key identifier chosen by the application.

6.2.2.8.2.4 macro PSA_KEY_ID_VENDOR_MAX

PSA_KEY_ID_VENDOR_MAX()

The maximum value for a key identifier chosen by the implementation.

6.2.2.8.2.5 macro PSA_KEY_ID_VENDOR_MIN

PSA_KEY_ID_VENDOR_MIN()

The minimum value for a key identifier chosen by the implementation.

6.2.2.9 Key custom parameters

6.2.2.9.1 Typedef

6.2.2.9.1.1 typedef psa_custom_key_parameters_t

type **psa_custom_key_parameters_t**

Custom production parameters for key generation or key derivation.

Description

The interpretation of this structure depends on the type of the key.

The [Custom key parameters](#) table shows the custom production parameters for each type of key. See the key type definitions for details of the valid parameter values.

Table 155. Custom key parameters

Key type	Custom key parameters
RSA	Use the production parameters to select an exponent value that is different from the default value of 65537. See PSA_KEY_TYPE_RSA_KEY_PAIR.
Other key types	Reserved for future use.

Note:

Future versions of the PSA Certified Crypto API, and implementations, may add other fields in this structure.

6.2.2.9.2 Macros

6.2.2.9.2.1 PSA_CUSTOM_KEY_PARAMETERS_INIT

This macro returns a suitable initializer for a object of type [typedef psa_custom_key_parameters_t](#).

6.2.3 Cryptography APIs

The Cryptography APIs allow user of the library to perform cryptographic operations such as hashing, signing, encryption, decryption, MAC and RNG. For some cryptographic operations, both oneshot and multipart modes are supported.

6.2.3.1 Random Number Generation

6.2.3.1.1 Function

6.2.3.1.1.1 `psa_generate_random`

[psa_status_t](#) `psa_generate_random(uint8_t *output, size_t output_size)`

Generate random bytes.

Parameters

- **output** (uint8_t*) - [out] Output buffer for the generated data.
- **output_size** (size_t) - [in] Number of bytes to generate and output.

Description

Warning:

This function can fail! Callers MUST check the return status and MUST NOT use the content of the @output buffer if the return status is not PSA_SUCCESS.

Return

- **PSA_SUCCESS**

Success. output contains `output_size` bytes of generated random data.

- **PSA_ERROR_NOT_SUPPORTED**
- **PSA_ERROR_INSUFFICIENT_ENTROPY**
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.2 Message digests (Hashes)

6.2.3.2.1 Single Part

6.2.3.2.1.1 `psa_hash_compute`

[psa_status_t](#) `psa_hash_compute(psa\_algorithm\_t alg, const uint8_t *input, size_t input_length, uint8_t *hash, size_t hash_size, size_t *hash_length)`

Calculate the hash (digest) of a message.

Parameters

- **alg** ([psa_algorithm_t](#)) - [in] The hash algorithm to compute such that [PSA_ALG_IS_HASH](#) is true.
- **input** (const uint8_t*) - [in] Buffer containing the message to hash.
- **input_length** (size_t) - [in] Size of the `input` buffer in bytes.
- **hash** (uint8_t*) - [out] Buffer where the hash is to be written.
- **hash_size** (size_t) - [in] Size of the `hash` buffer in bytes. This must be at least [PSA_HASH_LENGTH](#).
- **hash_length** (size_t*) - [out] On success, the number of bytes that make up the hash value.

Description

Note:

To verify the hash of a message against an expected value, use `psa_hash_compare()` instead.

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported.
- `input_length` is too large.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not a hash algorithm.
- `input_length` is too large for `alg`.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The `hash_size` is too small. [PSA_HASH_LENGTH](#) can be used to determine the required buffer size.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.2.1.2 `psa_hash_compare`

[psa_status_t](#) `psa_hash_compare`([psa_algorithm_t](#) `alg`, const uint8_t* `input`, size_t `input_length`, const uint8_t* `hash`, size_t `hash_length`)

Calculate the hash (digest) of a message and compare it with a reference value.

Parameters

- **alg** ([psa_algorithm_t](#)) - [in] The hash algorithm to compute such that [PSA_ALG_IS_HASH](#) is true.
- **input** (const uint8_t*) - [in] Buffer containing the message to hash.
- **input_length** (size_t) - [in] Size of the `input` buffer in bytes.
- **hash** (const uint8_t*) - [in] Buffer containing the expected hash value.
- **hash_length** (size_t) - [in] Size of the `hash` buffer in bytes.

Return• **PSA_SUCCESS:**

The expected hash is identical to the actual hash of the input.

• **PSA_ERROR_INVALID_SIGNATURE:**

The calculated hash of the message does not match the value in `hash`.

• **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported.
- `input_length` is too large.

• **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not a hash algorithm.
- `input_length` is too large for `alg`.

• **PSA_ERROR_INSUFFICIENT_MEMORY**• **PSA_ERROR_COMMUNICATION_FAILURE**• **PSA_ERROR_HARDWARE_FAILURE**• **PSA_ERROR_CORRUPTION_DETECTED**• **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.2.2 Multiple Part

The sequence of operations to calculate a hash (message digest) is as follows:

1. Allocate an operation object [psa_hash_operation_t](#) which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for [psa_hash_operation_t](#), e.g. [PSA_HASH_OPERATION_INIT](#).
3. Call [psa_hash_setup\(\)](#) to specify the algorithm.
4. Call [psa_hash_update\(\)](#) zero, one or more times, passing a fragment of the message each time. The hash that is calculated is the hash of the concatenation of these messages in order.
5. To calculate the hash, call [psa_hash_finish\(\)](#). To compare the hash with an expected value, call [psa_hash_verify\(\)](#). To suspend the hash operation and extract the current state, call [psa_hash_suspend\(\)](#).

6.2.3.2.2.1 typedef psa_hash_operation_t

type **psa_hash_operation_t**

The type of the state object for multi-part hash operations.

Description

Before calling any function on a hash operation object, the application must initialize it by any of the following means:

- Set the object to all-bits-zero, for example:

```
psa_hash_operation_t operation;
memset(&operation, 0, sizeof(operation));
```

- Initialize the object to logical zero values by declaring the object as static or global without an explicit initializer, for example:

```
static psa_hash_operation_t operation;
```

- Initialize the object to the initializer [PSA_HASH_OPERATION_INIT](#), for example:

```
psa_hash_operation_t operation = PSA_HASH_OPERATION_INIT;
```

- Assign the result of the function `psa_hash_operation_init()` to the object, for example:

```
psa_hash_operation_t operation;
operation = psa_hash_operation_init();
```

6.2.3.2.2 macro `PSA_HASH_OPERATION_INIT`

`PSA_HASH_OPERATION_INIT()`

Returns a suitable initializer for a hash operation object of [typedef `psa\_hash\_operation\_t`](#).

6.2.3.2.3 `psa_hash_operation_init`

[psa_hash_operation_t](#) `psa_hash_operation_init(void)`

Return an initial value for a hash operation object.

Parameters

- `void` - no arguments

Return

Initialized value of hash operation object.

6.2.3.2.4 `psa_hash_setup`

[psa_status_t](#) `psa_hash_setup(psa_hash_operation_t *operation, psa_algorithm_t alg)`

Set up a multi-part hash operation.

Parameters

- `operation` ([psa_hash_operation_t*](#)) - [in] The operation object to set up. It must have been initialized as per the documentation for [typedef `psa\_hash\_operation\_t`](#) and not yet in use.
- `alg` ([psa_algorithm_t](#)) - [in] The hash algorithm to compute such that [PSA_ALG_IS_HASH](#) is true.

Description

After a successful call to [psa_hash_setup\(\)](#), the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to [psa_hash_finish\(\)](#) or [psa_hash_verify\(\)](#) or [psa_hash_suspend\(\)](#).
- A call to [psa_hash_abort\(\)](#).

If [psa_hash_setup\(\)](#) returns an error, the operation object is unchanged. If a subsequent function call with an active operation returns an error, the operation enters an error state.

To abandon an active operation, or reset an operation in an error state, call [psa_hash_abort\(\)](#).

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_NOT_SUPPORTED:**
`alg` is not a supported hash algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
`alg` is not a hash algorithm.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be inactive.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.2.2.5 psa_hash_update

[psa_status_t](#) [psa_hash_update](#)([psa_hash_operation_t](#) *operation, const uint8_t *input, size_t input_length)

Add a message fragment to a multi-part hash operation.

Parameters

- **operation** ([psa_hash_operation_t](#)*) - [in] Active hash operation.
- **input** (const uint8_t*) - [in] Buffer containing the message fragment to hash.
- **input_length** (size_t) - [in] Size of the `input` buffer in bytes.

Description

The application must call [psa_hash_setup\(\)](#) or [psa_hash_resume\(\)](#) before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_hash_abort\(\)](#).

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INSUFFICIENT_MEMORY**

- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- **`PSA_ERROR_BAD_STATE`:**
 - The operation state is not valid: it must be active.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.2.2.6 `psa_hash_finish`

[psa_status_t](#) `psa_hash_finish`([psa_hash_operation_t](#) *operation, [uint8_t](#) *hash, [size_t](#) hash_size, [size_t](#) *hash_length)

Finish the calculation of the hash of a message.

Parameters

- **operation** ([psa_hash_operation_t](#)*) - [in] Active hash operation.
- **hash** ([uint8_t](#)*) - [out] Buffer where the hash is to be written.
- **hash_size** ([size_t](#)) - [in] Size of the `hash` buffer in bytes. This must be at least [PSA_HASH_LENGTH](#) where `alg` is the algorithm that the operation performs.
- **hash_length** ([size_t](#)*) - [out] On success, the number of bytes that make up the hash value.

Description

The application must call [psa_hash_setup\(\)](#) or [psa_hash_resume\(\)](#) before calling this function. This function calculates the hash of the message formed by concatenating the inputs passed to preceding calls to [psa_hash_update\(\)](#).

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_hash_abort\(\)](#).

Warning:

It is not recommended to use this function when a specific value is expected for the hash. Call [psa_hash_verify\(\)](#) instead with the expected hash value.

Return

- **`PSA_SUCCESS`:**
 - Success.
- **`PSA_ERROR_BUFFER_TOO_SMALL`:**
 - The size of the `hash` buffer is too small. `:c:macro`PSA_HASH_LENGTH`` can be used to determine the required buffer size.
- **`PSA_ERROR_INSUFFICIENT_MEMORY`**
- **`PSA_ERROR_COMMUNICATION_FAILURE`**
- **`PSA_ERROR_CORRUPTION_DETECTED`**
- **`PSA_ERROR_BAD_STATE`:**
 - The operation state is not valid: it must be active.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.2.2.7 `psa_hash_verify`

[psa_status_t](#) `psa_hash_verify`([psa_hash_operation_t](#) *operation, [const uint8_t](#) *hash, [size_t](#) hash_length)

Finish the calculation of the hash of a message and compare it with an expected value.

Parameters

- **operation** ([psa_hash_operation_t](#)*) - [in] Active hash operation.
- **hash** (const uint8_t*) - [in] Buffer containing the expected hash value.
- **hash_length** (size_t) - [in] Size of the `hash` buffer in bytes.

Description

The application must call [psa_hash_setup\(\)](#) before calling this function. This function calculates the hash of the message formed by concatenating the inputs passed to preceding calls to [psa_hash_update\(\)](#). It then compares the calculated hash with the expected hash passed as a parameter to this function.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_hash_abort\(\)](#).

Return

- **PSA_SUCCESS:**
Success. The expected hash is identical to the actual hash of the message.
- **PSA_ERROR_INVALID_SIGNATURE:**
The calculated hash of the message doesn't match the value in `hash`.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be active.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.2.2.8 psa_hash_abort

[psa_status_t](#) **psa_hash_abort**([psa_hash_operation_t](#) *operation)

Abort a hash operation.

Parameters

- **operation** ([psa_hash_operation_t](#)*) - [in] Initialized hash operation.

Description

Aborting an operation frees all associated resources except for the operation object itself. Once aborted, the operation object can be reused for another operation by calling [psa_hash_setup\(\)](#) again.

This function can be called any time after the operation object has been initialized by one of the methods described in [typedef psa_hash_operation_t](#).

In particular, calling [psa_hash_abort\(\)](#) after the operation has been terminated by a call to [psa_hash_abort\(\)](#), [psa_hash_finish\(\)](#) or [psa_hash_verify\(\)](#) is safe and has no effect.

Return

- **PSA_SUCCESS**

- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- **`PSA_ERROR_BAD_STATE:`**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.2.9 `psa_hash_clone`

[psa_status_t](#) `psa_hash_clone(const psa\_hash\_operation\_t *source_operation, psa\_hash\_operation\_t *target_operation)`

Clone a hash operation.

Parameters

- **`source_operation`** (const [psa_hash_operation_t](#)*) - [in] The active hash operation to clone.
- **`target_operation`** ([psa_hash_operation_t](#)*) - [out] The operation object to set up. It must be initialized but not active.

Description

This function copies the state of an ongoing hash operation to a new operation object. In other words, this function is equivalent to calling [psa_hash_setup\(\)](#) on `target_operation` with the same algorithm that `source_operation` was set up for, then [psa_hash_update\(\)](#) on `target_operation` with the same input that was passed to `source_operation`. After this function returns, the two objects are independent, i.e. subsequent calls involving one of the objects do not affect the other object.

Return

- **`PSA_SUCCESS`**
Success. `target_operation` is ready to continue the same hash operation as `source_operation`.
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_INSUFFICIENT_MEMORY`
- **`PSA_ERROR_BAD_STATE:`**
 - The `source_operation` state is not valid: it must be active.
 - The `target_operation` state is not valid: it must be inactive.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.2.3 Suspend and Resume operation

Warning:

The HASH multiple part suspend and resume operations are not supported in the current implementation.

The sequence of operations to suspend and resume a hash operation is as follows:

1. Compute the first part of the hash.
 - a. Allocate an operation object and initialize it as described in the documentation for [psa_hash_operation_t](#).
 - b. Call [psa_hash_setup\(\)](#) to specify the algorithm.
 - c. Call [psa_hash_update\(\)](#) zero, one or more times, passing a fragment of the message each time.

- d. Call [psa_hash_suspend\(\)](#) to extract the hash suspend state into a buffer.
2. Pass the hash state buffer to the application which will resume the operation.
3. Compute the rest of the hash.
 - a. Allocate an operation object and initialize it as described in the documentation for [psa_hash_operation_t](#).
 - b. Call [psa_hash_resume\(\)](#) with the extracted hash state.
 - c. Call [psa_hash_update\(\)](#) zero, one or more times, passing a fragment of the message each time.
 - d. To calculate the hash, call [psa_hash_finish\(\)](#). To compare the hash with an expected value, call [psa_hash_verify\(\)](#).

If an error occurs at any step after a call to [psa_hash_setup\(\)](#) or [psa_hash_resume\(\)](#), the operation will need to be reset by a call to [psa_hash_abort\(\)](#). The application can call [psa_hash_abort\(\)](#) at any time after the operation has been initialized.

6.2.3.2.3.1 psa_hash_suspend

[psa_status_t](#) [psa_hash_suspend](#)([psa_hash_operation_t](#) *operation, [uint8_t](#) *hash_state, [size_t](#) hash_state_size, [size_t](#) *hash_state_length)

Halt the hash operation and extract the intermediate state of the hash computation.

Parameters

- **operation** ([psa_hash_operation_t](#)*) - [in] Active hash operation.
- **hash_state** ([uint8_t](#)*) - [out] Buffer where the hash suspend state is to be written.
- **hash_state_size** ([size_t](#)) - [in] Size of the `hash_state` buffer in bytes.
- **hash_state_length** ([size_t](#)*) - [out] On success, the number of bytes that make up the hash suspend state.

Description

Warning:

Not supported.

The application must call [psa_hash_setup\(\)](#) or [psa_hash_resume\(\)](#) before calling this function. This function extracts an intermediate state of the hash computation of the message formed by concatenating the inputs passed to preceding calls to [psa_hash_update\(\)](#).

This function can be used to halt a hash operation, and then resume the hash operation at a later time, or in another application, by transferring the extracted hash suspend state to a call to [psa_hash_resume\(\)](#).

The `hash_state_size` must be appropriate for the selected algorithm:

- A sufficient output size is `PSA_HASH_SUSPEND_OUTPUT_SIZE(alg)` where `alg` is the algorithm that was used to set up the operation.
- `PSA_HASH_SUSPEND_OUTPUT_MAX_SIZE` evaluates to the maximum output size of any supported hash algorithm.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_hash_abort\(\)](#).

Hash suspend and resume is not defined for the SHA3 family of hash algorithms. Hash suspend state defines the format of the output from [psa_hash_suspend\(\)](#).

Warning:

Applications must not use any of the hash suspend state as if it was a hash output. Instead, the suspend state must only be used to resume a hash operation, and `psa_hash_finish()` or `psa_hash_verify()` can then calculate or verify the final hash value.

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the `hash_state` buffer is too small. [PSA_HASH_SUSPEND_OUTPUT_SIZE](#) or [PSA_HASH_SUSPEND_OUTPUT_MAX_SIZE](#) can be used to determine the required buffer size.

- **PSA_ERROR_NOT_SUPPORTED:**

The hash algorithm being computed does not support suspend and resume.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid: it must be active.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.2.3.2 `psa_hash_resume`

[psa_status_t](#) `psa_hash_resume`([psa_hash_operation_t](#) *operation, const uint8_t *hash_state, size_t hash_state_length)

Set up a multi-part hash operation using the hash suspend state from a previously suspended hash operation.

Parameters

- **operation** ([psa_hash_operation_t](#)*) - [in] The operation object to set up. It must have been initialized as per the documentation for [typedef psa_hash_operation_t](#) and not yet in use.
- **hash_state** (const uint8_t*) - [in] A buffer containing the suspended hash state which is to be resumed. This must be in the format output by [psa_hash_suspend\(\)](#).
- **hash_state_length** (size_t) - [in] Length of `hash_state` in bytes.

Description

Warning:

Not supported.

After a successful call to [psa_hash_resume\(\)](#), the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to [psa_hash_finish\(\)](#), [psa_hash_verify\(\)](#) or [psa_hash_suspend\(\)](#).
- A call to [psa_hash_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_NOT_SUPPORTED:**

The provided hash suspend state is for an algorithm that is not supported.

- **PSA_ERROR_INVALID_ARGUMENT:**

`hash_state` does not correspond to a valid hash suspend state. See Hash suspend state format for the definition.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid: it must be inactive.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.2.4 Macros

6.2.3.2.4.1 macro **PSA_HASH_BLOCK_LENGTH**

PSA_HASH_BLOCK_LENGTH(*alg*)

The input block size of a hash algorithm, in bytes.

Parameters

- **alg** - [in] A hash algorithm such that [PSA_ALG_IS_HASH](#) is true.

Description

Hash algorithms process their input data in blocks. Hash operations will retain any partial blocks until they have enough input to fill the block or until the operation is finished.

This affects the output from [psa_hash_suspend\(\)](#).

Return

The block size in bytes for the specified hash algorithm. If the hash algorithm is not recognized, return 0.

6.2.3.2.4.2 macro **PSA_HASH_LENGTH**

PSA_HASH_LENGTH(*alg*)

The size of the output of [psa_hash_compute\(\)](#) and [psa_hash_finish\(\)](#), in bytes.

Parameters

- **alg** - [in] A hash algorithm such that [PSA_ALG_IS_HASH](#) is true, or an HMAC algorithm such that [PSA_ALG_IS_HMAC](#) is true.

Description

This is also the hash length that [psa_hash_compare\(\)](#) and [psa_hash_verify\(\)](#) expect.

See also [PSA_HASH_MAX_SIZE](#).

Return

The hash length for the specified hash algorithm. If the hash algorithm is not recognized, return 0.

6.2.3.2.4.3 macro PSA_HASH_MAX_SIZE

PSA_HASH_MAX_SIZE()

Maximum size of a hash.

Description

This macro must expand to a compile-time constant integer.

See also [PSA_HASH_LENGTH](#).

6.2.3.2.4.4 macro PSA_HASH_SUSPEND_OUTPUT_MAX_SIZE

PSA_HASH_SUSPEND_OUTPUT_MAX_SIZE()

A sufficient hash suspend state buffer size for [psa_hash_suspend\(\)](#), for any supported hash algorithms.

Description

Warning:

Not supported.

See also [PSA_HASH_SUSPEND_OUTPUT_SIZE](#).

6.2.3.2.4.5 macro PSA_HASH_SUSPEND_OUTPUT_SIZE

PSA_HASH_SUSPEND_OUTPUT_SIZE(*alg*)

A sufficient hash suspend state buffer size for [psa_hash_suspend\(\)](#).

Parameters

- **alg** - [in] A hash algorithm such that [PSA_ALG_IS_HASH](#) is true.

Description

If the size of the hash state buffer is at least this large, it is guaranteed that [psa_hash_suspend\(\)](#) will not fail due to an insufficient buffer size. The actual size of the output might be smaller in any given call.

See also [PSA_HASH_SUSPEND_OUTPUT_MAX_SIZE](#).

Return

A sufficient output size for the algorithm. If the hash algorithm is not recognized, or is not supported by [psa_hash_suspend\(\)](#), return 0.

6.2.3.3 Message Authentication Code (MAC)

6.2.3.3.1 Single Part

6.2.3.3.1.1 psa_mac_compute

[psa_status_t](#) **psa_mac_compute**([psa_key_id_t](#) key, [psa_algorithm_t](#) alg, const uint8_t *input, size_t input_length, uint8_t *mac, size_t mac_size, size_t *mac_length)

Calculate the message authentication code (MAC) of a message.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must allow the usage `PSA_KEY_USAGE_SIGN_MESSAGE`.
- **alg** ([psa_algorithm_t](#)) - [in] The MAC algorithm to compute such that [PSA_ALG_IS_MAC](#) is true.
- **input** (const uint8_t*) - [in] Buffer containing the input message.
- **input_length** (size_t) - [in] Size of the `input` buffer in bytes.
- **mac** (uint8_t*) - [out] Buffer where the MAC value is to be written.
- **mac_size** (size_t) - [in] Size of the `mac` buffer in bytes.
- **mac_length** (size_t*) - [out] On success, the number of bytes that make up the MAC value.

Description

Note:

To verify the MAC of a message against an expected value, use `psa_mac_verify()` instead.

The `mac_size` parameter must be appropriate for the selected algorithm and key:

- The exact MAC size is [PSA_MAC_LENGTH](#) where `key_type` and `key_bits` are attributes of the key used to compute the MAC.
- [PSA_MAC_MAX_SIZE](#) evaluates to the maximum MAC size of any supported MAC algorithm.

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE:**
`key` is not a valid key identifier.
- **PSA_ERROR_NOT_PERMITTED:**
The `key` does not have the `PSA_KEY_USAGE_SIGN_MESSAGE` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
 - `alg` is not a MAC algorithm.
 - `key` is not compatible with `alg`.
 - `input_length` is too large for `alg`.
- **PSA_ERROR_NOT_SUPPORTED:**
 - `alg` is not supported.

- `key` is not supported for use with `alg`.
- `input_length` is too large.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the `mac` buffer is too small. [PSA_MAC_LENGTH](#) or [PSA_MAC_MAX_SIZE](#) can be used to determine the required buffer size.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE:**
- **PSA_ERROR_DATA_CORRUPT:**
- **PSA_ERROR_DATA_INVALID:**
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.3.1.2 psa_mac_verify

[psa_status_t](#) **psa_mac_verify**([psa_key_id_t](#) `key`, [psa_algorithm_t](#) `alg`, [const uint8_t*](#) `input`, [size_t](#) `input_length`, [const uint8_t*](#) `mac`, [size_t](#) `mac_length`)

Calculate the MAC of a message and compare it with a reference value.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must allow the usage `PSA_KEY_USAGE_VERIFY_MESSAGE`.
- **alg** ([psa_algorithm_t](#)) - [in] The MAC algorithm to compute such that [PSA_ALG_IS_MAC](#) is true.
- **input** ([const uint8_t*](#)) - [in] Buffer containing the input message.
- **input_length** ([size_t](#)) - [in] Size of the `input` buffer in bytes.
- **mac** ([const uint8_t*](#)) - [in] Buffer containing the expected MAC value.
- **mac_length** ([size_t](#)) - [in] Size of the `mac` buffer in bytes.

Return

- **PSA_SUCCESS:**

Success. The expected MAC is identical to the actual MAC of the input.

- **PSA_ERROR_INVALID_SIGNATURE:**

The calculated MAC of the message does not match the expected value.

- **PSA_ERROR_INVALID_HANDLE:**

`key` is not a valid key identifier.

- **PSA_ERROR_NOT_PERMITTED:**

The `key` does not have the `PSA_KEY_USAGE_VERIFY_MESSAGE` flag, or it does not permit the requested algorithm.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not a MAC algorithm.
- `key` is not compatible with `alg`.

- `input_length` is too large for `alg`.

- **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported.
- `key` is not supported for use with `alg`.
- `input_length` is too large.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE:**
- **PSA_ERROR_DATA_CORRUPT:**
- **PSA_ERROR_DATA_INVALID:**
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.3.2 Multiple Part

Warning:

The MAC signature generation and verification multiple part sequences are not supported in the current implementation.

MAC signature generation sequence

The sequence of operations to calculate a MAC is as follows:

1. Allocate an operation object [psa_mac_operation_t](#) which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for [psa_mac_operation_t](#), e.g. [PSA_MAC_OPERATION_INIT](#).
3. Call [psa_mac_sign_setup\(\)](#) to specify the algorithm and key.
4. Call [psa_mac_update\(\)](#) zero, one or more times, passing a fragment of the message each time. The MAC that is calculated is the MAC of the concatenation of these messages in order.
5. At the end of the message, call [psa_mac_sign_finish\(\)](#) to finish calculating the MAC value and retrieve it.

MAC signature verification sequence

The sequence of operations to verify a MAC is as follows:

1. Allocate an operation object [psa_mac_operation_t](#) which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for [psa_mac_operation_t](#), e.g. [PSA_MAC_OPERATION_INIT](#).
3. Call [psa_mac_verify_setup\(\)](#) to specify the algorithm and key.
4. Call [psa_mac_update\(\)](#) zero, one or more times, passing a fragment of the message each time. The MAC that is calculated is the MAC of the concatenation of these messages in order.
5. At the end of the message, call [psa_mac_verify_finish\(\)](#) to finish calculating the actual MAC of the message and verify it against the expected value.

6.2.3.3.2.1 typedef `psa_mac_operation_t`

type `psa_mac_operation_t`

The type of the state object for multi-part MAC operations.

Description

Before calling any function on a MAC operation object, the application must initialize it by any of the following means:

- Set the object to all-bits-zero, for example:

```
psa_mac_operation_t operation;  
memset(&operation, 0, sizeof(operation));
```

- Initialize the object to logical zero values by declaring the object as static or global without an explicit initializer, for example:

```
static psa_mac_operation_t operation;
```

- Initialize the object to the initializer [PSA_MAC_OPERATION_INIT](#), for example:

```
psa_mac_operation_t operation = PSA_MAC_OPERATION_INIT;
```

- Assign the result of the function [psa_mac_operation_init\(\)](#) to the object, for example:

```
psa_mac_operation_t operation;  
operation = psa_mac_operation_init();
```

6.2.3.3.2.2 macro PSA_MAC_OPERATION_INIT

PSA_MAC_OPERATION_INIT()

Returns a suitable initializer for a MAC operation object of [typedef psa_mac_operation_t](#).

6.2.3.3.2.3 psa_mac_operation_init

[psa_mac_operation_t](#) **psa_mac_operation_init**(void)

Return an initial value for a MAC operation object.

Parameters

- **void** - no arguments

Return

Initialized value of mac operation object.

6.2.3.3.2.4 psa_mac_sign_setup

[psa_status_t](#) **psa_mac_sign_setup**([psa_mac_operation_t](#) *operation, [psa_key_id_t](#) key, [psa_algorithm_t](#) alg)

Set up a multi-part MAC calculation operation.

Parameters

- **operation** ([psa_mac_operation_t*](#)) - [in] The operation object to set up. It must have been initialized as per the documentation for [typedef psa_mac_operation_t](#) and not yet in use.
- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage `PSA_KEY_USAGE_SIGN_MESSAGE`.
- **alg** ([psa_algorithm_t](#)) - [in] The MAC algorithm to compute such that [PSA_ALG_IS_MAC](#) is true.

Description

Warning:

Not supported.

This function sets up the calculation of the message authentication code (MAC) of a byte string. To verify the MAC of a message against an expected value, use [psa_mac_verify_setup\(\)](#) instead.

After a successful call to [psa_mac_sign_setup\(\)](#), the operation is active, and the application must eventually terminate the operation through one of the following methods:

- A successful call to [psa_mac_sign_finish\(\)](#).
- A call to [psa_mac_abort\(\)](#).

If [psa_mac_sign_setup\(\)](#) returns an error, the operation object is unchanged. If a subsequent function call with an active operation returns an error, the operation enters an error state.

To abandon an active operation, or reset an operation in error state, call [psa_mac_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success. The operation is now active.

- **PSA_ERROR_INVALID_HANDLE**

`key` is not a valid key identifier.

- **PSA_ERROR_NOT_PERMITTED:**

The key does not have the `PSA_KEY_USAGE_SIGN_MESSAGE` flag, or it does not permit the requested algorithm.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not a MAC algorithm.
- `key` is not compatible with `alg`.

- **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported.
- `key` is not supported for use with `alg`.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE:**
- **PSA_ERROR_DATA_CORRUPT:**

- **PSA_ERROR_DATA_INVALID:**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be inactive.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.3.2.5 psa_mac_verify_setup

[psa_status_t](#) **psa_mac_verify_setup**([psa_mac_operation_t](#) *operation, [psa_key_id_t](#) key, [psa_algorithm_t](#) alg)

Set up a multi-part MAC verification operation.

Parameters

- **operation** ([psa_mac_operation_t](#)*) - [in] The operation object to set up. It must have been initialized as per the documentation for [typedef psa_mac_operation_t](#) and not yet in use.
- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage `PSA_KEY_USAGE_VERIFY_MESSAGE`.
- **alg** ([psa_algorithm_t](#)) - [in] The MAC algorithm to compute such that [PSA_ALG_IS_MAC](#) is true.

Description

Warning:

Not supported.

This function sets up the verification of the message authentication code (MAC) of a byte string against an expected value.

After a successful call to [psa_mac_verify_setup\(\)](#), the application must eventually terminate the operation through one of the following methods:

- A successful call to [psa_mac_verify_finish\(\)](#).
- A call to [psa_mac_abort\(\)](#).

If [psa_mac_verify_setup\(\)](#) returns an error, the operation object is unchanged. If a subsequent function call with an active operation returns an error, the operation enters an error state.

To abandon an active operation, or reset an operation in an error state, call [psa_mac_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success. The operation is now active.
- **PSA_ERROR_INVALID_HANDLE:**

`key` is not a valid key identifier.
- **PSA_ERROR_NOT_PERMITTED:**

The key does not have the `PSA_KEY_USAGE_VERIFY_MESSAGE` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
 - `alg` is not a MAC algorithm.

- `key` is not compatible with `alg`.
- **PSA_ERROR_NOT_SUPPORTED:**
 - `alg` is not supported.
 - `key` is not supported for use with `alg`.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE:**
- **PSA_ERROR_DATA_CORRUPT:**
- **PSA_ERROR_DATA_INVALID:**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be inactive.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.3.2.6 `psa_mac_update`

[psa_status_t](#) `psa_mac_update`([psa_mac_operation_t](#) *`operation`, [const uint8_t](#) *`input`, [size_t](#) `input_length`)

Add a message fragment to a multi-part MAC operation.

Parameters

- `operation` ([psa_mac_operation_t](#)*) - [in] Active MAC operation.
- `input` ([const uint8_t](#)*) - [in] Buffer containing the message fragment to add to the MAC calculation.
- `input_length` ([size_t](#)) - [in] Size of the `input` buffer in bytes.

Description

Warning:

Not supported.

The application must call [psa_mac_sign_setup\(\)](#) or [psa_mac_verify_setup\(\)](#) before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_mac_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success.
- **PSA_ERROR_INVALID_ARGUMENT:**

The total input for the operation is too large for the MAC algorithm.
- **PSA_ERROR_NOT_SUPPORTED:**

The total input for the operation is not supported.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**

- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- **`PSA_ERROR_BAD_STATE`:**
 - The operation state is not valid: it must be active.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.3.2.7 `psa_mac_sign_finish`

[psa_status_t](#) `psa_mac_sign_finish`([psa_mac_operation_t](#) *operation, [uint8_t](#) *mac, [size_t](#) mac_size, [size_t](#) *mac_length)

Finish the calculation of the MAC of a message.

Parameters

- **operation** ([psa_mac_operation_t](#)*) - [in] Active MAC operation.
- **mac** ([uint8_t](#)*) - [in] Buffer where the MAC value is to be written.
- **mac_size** ([size_t](#)) - [in] Size of the `mac` buffer in bytes.
- **mac_length** ([size_t](#)*) - [out] On success, the number of bytes that make up the MAC value.

Description

Warning:

Not supported.

The application must call [psa_mac_sign_setup\(\)](#) before calling this function. This function calculates the MAC of the message formed by concatenating the inputs passed to preceding calls to [psa_mac_update\(\)](#).

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_mac_abort\(\)](#).

The `mac_size` parameter must be appropriate for the selected algorithm and key:

- The exact MAC size is [PSA_MAC_LENGTH](#) where `key_type` and `key_bits` are attributes of the key, and `alg` is the algorithm used to compute the MAC.
- [PSA_MAC_MAX_SIZE](#) evaluates to the maximum MAC size of any supported MAC algorithm.

Return

- **`PSA_SUCCESS`:**

Success.

- **`PSA_ERROR_BUFFER_TOO_SMALL`:**

The size of the `mac` buffer is too small. [PSA_MAC_LENGTH](#) or [PSA_MAC_MAX_SIZE](#) can be used to determine the required buffer size.

- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`

- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be an active mac sign operation.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.3.2.8 psa_mac_verify_finish

[psa_status_t](#) **psa_mac_verify_finish**([psa_mac_operation_t](#) *operation, const uint8_t *mac, size_t mac_length)

Finish the calculation of the MAC of a message and compare it with an expected value.

Parameters

- **operation** ([psa_mac_operation_t](#)*) - [in] Active MAC operation.
- **mac** (const uint8_t*) - [in] Buffer containing the expected MAC value.
- **mac_length** (size_t) - [in] Size of the `mac` buffer in bytes.

Description

Warning:

Not supported.

The application must call [psa_mac_verify_setup\(\)](#) before calling this function. This function calculates the MAC of the message formed by concatenating the inputs passed to preceding calls to [psa_mac_update\(\)](#). It then compares the calculated MAC with the expected MAC passed as a parameter to this function.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_mac_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success. The expected MAC is identical to the actual MAC of the message.
- **PSA_ERROR_INVALID_SIGNATURE:**

The calculated MAC of the message does not match the value in `mac`.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be an active mac verify operation.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.3.2.9 psa_mac_abort

[psa_status_t](#) **psa_mac_abort**([psa_mac_operation_t](#) *operation)

Abort a MAC operation.

Parameters

- **operation** ([psa_mac_operation_t*](#)) - [in] Initialized MAC operation.

Description

Warning:

Not supported.

Aborting an operation frees all associated resources except for the operation object itself. Once aborted, the operation object can be reused for another operation by calling [psa_mac_sign_setup\(\)](#) or [psa_mac_verify_setup\(\)](#) again.

This function can be called any time after the operation object has been initialized by one of the methods described in [typedef psa_mac_operation_t](#).

In particular, calling [psa_mac_abort\(\)](#) after the operation has been terminated by a call to [psa_mac_abort\(\)](#), [psa_mac_sign_finish\(\)](#) or [psa_mac_verify_finish\(\)](#) is safe and has no effect.

Return

- **PSA_SUCCESS** Success. The operation object can now be discarded or reused.
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.3.3 Macros

6.2.3.3.3.1 macro PSA_MAC_LENGTH

PSA_MAC_LENGTH(*key_type*, *key_bits*, *alg*)

The size of the output of [psa_mac_compute\(\)](#) and [psa_mac_sign_finish\(\)](#), in bytes.

Parameters

- **key_type** - [in] The type of the MAC key.
- **key_bits** - [in] The size of the MAC key in bits.
- **alg** - [in] A MAC algorithm such that [PSA_ALG_IS_MAC](#) is true.

Description

This is also the MAC length that [psa_mac_verify\(\)](#) and [psa_mac_verify_finish\(\)](#) expect.

See also [PSA_MAC_MAX_SIZE](#).

Return

The MAC length for the specified algorithm with the specified key parameters. 0 if the MAC algorithm is not recognized.

6.2.3.3.3.2 macro PSA_MAC_MAX_SIZE

PSA_MAC_MAX_SIZE()

Maximum size of a MAC.

Description

This macro must expand to a compile-time constant integer. The maximum MAC size is the maximum size supported by HMAC and by CMAC.

See also [PSA_MAC_LENGTH](#).

6.2.3.3.3.3 macro PSA_ALG_FULL_LENGTH_MAC

PSA_ALG_FULL_LENGTH_MAC(*mac_alg*)

Macro to construct the MAC algorithm with a full length MAC, from a truncated MAC algorithm.

Parameters

- **mac_alg** - [in] A MAC algorithm identifier such that [PSA_ALG_IS_MAC](#) is true. This can be a truncated or untruncated MAC algorithm.

Return

The corresponding MAC algorithm with a full length MAC.

Unspecified if *mac_alg* is not a supported MAC algorithm.

6.2.3.3.3.4 macro PSA_ALG_AT_LEAST_THIS_LENGTH_MAC

PSA_ALG_AT_LEAST_THIS_LENGTH_MAC(*mac_alg*, *min_mac_length*)

Macro to build a MAC minimum-MAC-length wildcard algorithm.

Parameters

- **mac_alg** - [in] A MAC algorithm such that [PSA_ALG_IS_MAC](#) is true. This can be a truncated or untruncated MAC algorithm.
- **min_mac_length** - [in] Desired minimum length of the message authentication code in bytes. This must be at most the untruncated length of the MAC and must be at least 1.

Description

A key with a minimum-MAC-length MAC wildcard algorithm as permitted algorithm policy can be used with all MAC algorithms sharing the same base algorithm, and where the (potentially truncated) MAC length of the specific algorithm is equal to or larger than the wildcard algorithm's minimum MAC length.

Note:

When setting the minimum required MAC length to less than the smallest MAC length allowed by the base algorithm, this effectively becomes an 'any-MAC-length-allowed' policy for that base algorithm.

The untruncated MAC algorithm can be recovered using [PSA_ALG_FULL_LENGTH_MAC](#).

Compatible key types:

The resulting wildcard MAC algorithm is compatible with the same key types as the MAC algorithm used to construct it.

Return

The corresponding MAC wildcard algorithm with the specified minimum MAC length.

Unspecified if `mac_alg` is not a supported MAC algorithm or if `min_mac_length` is less than 1 or too large for the specified MAC algorithm.

6.2.3.4 Asymmetric Signature

6.2.3.4.1 Single Part

6.2.3.4.1.1 psa_sign_message

[psa_status_t](#) **psa_sign_message**([psa_key_id_t](#) key, [psa_algorithm_t](#) alg, const uint8_t *input, size_t input_length, uint8_t *signature, size_t signature_size, size_t *signature_length)

Sign a message with a private key. For hash-and-sign algorithms, this includes the hashing step.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must be an asymmetric key pair. The key must allow the usage `PSA_KEY_USAGE_SIGN_MESSAGE`.
- **alg** ([psa_algorithm_t](#)) - [in] An asymmetric signature algorithm such that [PSA_ALG_IS_SIGN_MESSAGE](#) is true.
- **input** (const uint8_t*) - [in] The input message to sign.
- **input_length** (size_t) - [in] Size of the `input` buffer in bytes.
- **signature** (uint8_t*) - [out] Buffer where the signature is to be written.
- **signature_size** (size_t) - [in] Size of the `signature` buffer in bytes.
- **signature_length** (size_t*) - [out] On success, the number of bytes that make up the returned signature value.

Description

The `signature_size` parameter must be appropriate for the selected algorithm and key:

- The required signature size is [PSA_SIGN_OUTPUT_SIZE](#) where `key_type` and `key_bits` are the type and bit-size respectively of key.
- [PSA_SIGNATURE_MAX_SIZE](#) evaluates to the maximum signature size of any supported signature algorithm.

Return

• PSA_SUCCESS

Success.

• PSA_ERROR_INVALID_HANDLE

`key` is not a valid key identifier.

• PSA_ERROR_NOT_PERMITTED:

The `key` does not have the `PSA_KEY_USAGE_SIGN_MESSAGE` flag, or it does not permit the requested algorithm.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the signature buffer is too small. [PSA_SIGN_OUTPUT_SIZE](#) or [PSA_SIGNATURE_MAX_SIZE](#) can be used to determine the required buffer size.

- **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported, or is not an asymmetric signature algorithm that permits signing a message.
- `key` is not supported for use with `alg`.
- `input_length` is too large.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not an asymmetric signature algorithm that permits signing a message.
- `key` is not an asymmetric key pair, that is compatible with `alg`.
- `input_length` is too large for the algorithm and key type.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_INSUFFICIENT_ENTROPY**
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.4.1.2 `psa_verify_message`

[psa_status_t](#) **psa_verify_message**([psa_key_id_t](#) key, [psa_algorithm_t](#) alg, const uint8_t *input, size_t input_length, const uint8_t *signature, size_t signature_length)

Verify the signature of a message with a public key. For hash-and-sign verification algorithm, this includes the hashing step.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must be a public key or an asymmetric key pair. The key must allow the usage `PSA_KEY_USAGE_VERIFY_MESSAGE`.
- **alg** ([psa_algorithm_t](#)) - [in] An asymmetric signature algorithm such that [PSA_ALG_IS_SIGN_MESSAGE](#) is true.
- **input** (const uint8_t*) - [in] The message whose signature is to be verified.
- **input_length** (size_t) - [in] Size of the `input` buffer in bytes.
- **signature** (const uint8_t*) - [in] Buffer containing the signature to verify.
- **signature_length** (size_t) - [in] Size of the `signature` buffer in bytes.

Return

- **PSA_SUCCESS:**

Success. The signature is valid.

- **PSA_ERROR_INVALID_HANDLE**

`key` is not a valid key identifier.

- **PSA_ERROR_NOT_PERMITTED:**

The key does not have the `PSA_KEY_USAGE_VERIFY_MESSAGE` flag, or it does not permit the requested algorithm.

- **PSA_ERROR_INVALID_SIGNATURE:**

The calculated signature of the message does not match the value in `signature`.

- **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported, or is not an asymmetric signature algorithm that permits verifying a message.
- `key` is not supported for use with `alg`.
- `input_length` is too large.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not an asymmetric signature algorithm that permits verifying a message.
- `key` is not an asymmetric key pair, that is compatible with `alg`.
- `input_length` is too large for the algorithm and key type.

- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.4.1.3 psa_sign_hash

[psa_status_t](#) **psa_sign_hash**([psa_key_id_t](#) key, [psa_algorithm_t](#) alg, const [uint8_t](#) *hash, [size_t](#) hash_length, [uint8_t](#) *signature, [size_t](#) signature_size, [size_t](#) *signature_length)

Sign a pre-computed hash with a private key.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must be an asymmetric key pair. The key must allow the usage `PSA_KEY_USAGE_SIGN_HASH`.
- **alg** ([psa_algorithm_t](#)) - [in] An asymmetric signature algorithm that separates the hash and sign operations such that [PSA_ALG_IS_SIGN_HASH](#) is true.
- **hash** (const [uint8_t](#) *) - [in] The input to sign. This is usually the hash of a message.
- **hash_length** ([size_t](#)) - [in] Size of the hash buffer in bytes.
- **signature** ([uint8_t](#) *) - [out] Buffer where the signature is to be written.
- **signature_size** ([size_t](#)) - [in] Size of the `signature` buffer in bytes.
- **signature_length** ([size_t](#) *) - [out] On success, the number of bytes that make up the returned signature value.

Description

For hash-and-sign signature algorithms, the hash input to this function is the hash of the message to sign. The algorithm used to calculate this hash is encoded in the signature algorithm. For such algorithms, `hash_length` must equal the length of the hash output:

```
hash_length == PSA_HASH_LENGTH(PSA_ALG_GET_HASH(alg))
```

Specialized signature algorithms can apply a padding or encoding to the hash. In such cases, the encoded hash must be passed to this function. For example, see `PSA_ALG_RSA_PKCS1V15_SIGN_RAW`.

The `signature_size` parameter must be appropriate for the selected algorithm and key:

- The required signature size is [PSA_SIGN_OUTPUT_SIZE](#) where `key_type` and `key_bits` are the type and bit-size respectively of `key`.
- [PSA_SIGNATURE_MAX_SIZE](#) evaluates to the maximum signature size of any supported signature algorithm.

Return

- **PSA_SUCCESS**

Success.

- **PSA_ERROR_INVALID_HANDLE**

`key` is not a valid key identifier.

- **PSA_ERROR_NOT_PERMITTED:**

The key does not have the `PSA_KEY_USAGE_SIGN_HASH` flag, or it does not permit the requested algorithm.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the `signature` buffer is too small. [PSA_SIGN_OUTPUT_SIZE](#) or [PSA_SIGNATURE_MAX_SIZE](#) can be used to determine the required buffer size.

- **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported, or is not an asymmetric signature algorithm that permits signing a pre-computed hash.
- `key` is not supported for use with `alg`.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not an asymmetric signature algorithm that permits signing a pre-computed hash.
- `key` is not an asymmetric key pair, that is compatible with `alg`.
- `hash_length` is not valid for the algorithm and key type.
- `hash` is not a valid input value for the algorithm and key type.

- **PSA_ERROR_INSUFFICIENT_MEMORY**

- **PSA_ERROR_COMMUNICATION_FAILURE**

- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_STORAGE_FAILURE**

- **PSA_ERROR_DATA_CORRUPT**

- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_INSUFFICIENT_ENTROPY**

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.4.1.4 psa_verify_hash

[psa_status_t](#) **psa_verify_hash**([psa_key_id_t](#) key, [psa_algorithm_t](#) alg, const uint8_t *hash, size_t hash_length, const uint8_t *signature, size_t signature_length)

Verify the signature of a pre-computed hash using a public key.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must be a public key or an asymmetric key pair. The key must allow the usage `PSA_KEY_USAGE_VERIFY_HASH`.
- **alg** ([psa_algorithm_t](#)) - [in] An asymmetric signature algorithm that separates the hash and sign operations such that `PSA_ALG_IS_SIGN_HASH` is true.
- **hash** (const uint8_t*) - [in] The input whose signature is to be verified. T
- **hash_length** (size_t) - [in] Size of the `hash` buffer in bytes.
- **signature** (const uint8_t*) - [in] Buffer containing the signature to verify.
- **signature_length** (size_t) - [in] Size of the signature buffer in bytes.

Description

For hash-and-sign signature algorithms, the hash input to this function is the hash of the message to verify. The algorithm used to calculate this hash is encoded in the signature algorithm. For such algorithms, `hash_length` must equal the length of the hash output:

```
hash_length == PSA_HASH_LENGTH(PSA_ALG_GET_HASH(alg))
```

Specialized signature algorithms can apply a padding or encoding to the hash. In such cases, the encoded hash must be passed to this function. For example, see `PSA_ALG_RSA_PKCS1V15_SIGN_RAW`.

Return

- **PSA_SUCCESS:**
Success. The signature is valid.
- **PSA_ERROR_INVALID_HANDLE**
`key` is not a valid key identifier.
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_VERIFY_HASH` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_SIGNATURE:**
The calculated signature of the hash does not match the `signature`.
- **PSA_ERROR_NOT_SUPPORTED:**
 - `alg` is not supported, or is not an asymmetric signature algorithm that permits verifying a pre-computed hash.
 - `key` is not supported for use with `alg`.
- **PSA_ERROR_INVALID_ARGUMENT:**
 - `alg` is not an asymmetric signature algorithm that permits verifying a pre-computed hash.
 - `key` is not an asymmetric key pair, that is compatible with `alg`.
 - `hash_length` is not valid for the algorithm and key type.
 - `hash` is not a valid input value for the algorithm and key type.
- **PSA_ERROR_INSUFFICIENT_MEMORY**

- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- **`PSA_ERROR_BAD_STATE`:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.4.2 Multiple Part

There is no dedicated PSA asymmetric signature operation to perform multiple part sequence. But as the multiple part signature generation or verification consists in hashing the message, the multiple part sequence can be done by first use a [hash multiple part](#) operation and then pass the hash to either [psa_sign_hash\(\)](#) or [psa_verify_hash\(\)](#) to respectively generate or verify a message signature.

The [PSA_ALG_GET_HASH](#) can be used to determine the hash algorithm to use.

6.2.3.4.3 Macros

6.2.3.4.3.1 macro `PSA_SIGNATURE_MAX_SIZE`

`PSA_SIGNATURE_MAX_SIZE()`

Maximum size of an asymmetric signature.

Description

This macro must expand to a compile-time constant integer.

Returns a sufficient signature buffer size for [psa_sign_message\(\)](#) and [psa_sign_hash\(\)](#), for any of the supported key types and asymmetric signature algorithms.

See also [PSA_SIGN_OUTPUT_SIZE](#)

6.2.3.4.3.2 macro `PSA_SIGN_OUTPUT_SIZE`

`PSA_SIGN_OUTPUT_SIZE(key_type, key_bits, alg)`

Sufficient signature buffer size for [psa_sign_message\(\)](#) and [psa_sign_hash\(\)](#).

Parameters

- **`key_type`** - [in] An asymmetric key type. This can be a key pair type or a public key type.
- **`key_bits`** - [in] The size of the key in bits.
- **`alg`** - [in] The signature algorithm.

Description

If the size of the signature buffer is at least this large, it is guaranteed that [psa_sign_message\(\)](#) and [psa_sign_hash\(\)](#) will not fail due to an insufficient buffer size. The actual size of the output might be smaller in any given call.

See also [PSA_SIGNATURE_MAX_SIZE](#).

Return

A sufficient signature buffer size for the specified asymmetric signature algorithm and key parameters. 0 if asymmetric signature algorithm and key parameters are not recognized.

6.2.3.5 Authenticated Encryption with Associated Data (AEAD)

6.2.3.5.1 Single Part

6.2.3.5.1.1 psa_aead_encrypt

[psa_status_t](#) **psa_aead_encrypt**([psa_key_id_t](#) key, [psa_algorithm_t](#) alg, const uint8_t *nonce, size_t nonce_length, const uint8_t *additional_data, size_t additional_data_length, const uint8_t *plaintext, size_t plaintext_length, uint8_t *ciphertext, size_t ciphertext_size, size_t *ciphertext_length)

Process an authenticated encryption operation.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must allow the usage `PSA_KEY_USAGE_ENCRYPT`.
- **alg** ([psa_algorithm_t](#)) - [in] The AEAD algorithm to compute such that [PSA_ALG_IS_AEAD](#) is true.
- **nonce** (const uint8_t*) - [in] Nonce or IV to use.
- **nonce_length** (size_t) - [in] Size of the `nonce` buffer in bytes. This must be appropriate for the selected algorithm. The default nonce size is [PSA_AEAD_NONCE_LENGTH](#) where `key_type` is the type of `key`.
- **additional_data** (const uint8_t*) - [in] Additional data that will be authenticated but not encrypted.
- **additional_data_length** (size_t) - [in] Size of `additional_data` in bytes.
- **plaintext** (const uint8_t*) - [in] Data that will be authenticated and encrypted.
- **plaintext_length** (size_t) - [in] Size of plaintext in bytes.
- **ciphertext** (uint8_t*) - [out] Output buffer for the authenticated and encrypted data.
- **ciphertext_size** (size_t) - [in] Size of the `ciphertext` buffer in bytes.
- **ciphertext_length** (size_t*) - [out] On success, the size of the output in the ciphertext buffer.

Description

The `ciphertext_size` parameter must be appropriate for the selected algorithm and key:

- A sufficient output size is [PSA_AEAD_ENCRYPT_OUTPUT_SIZE](#) where `key_type` is the type of `key`.
- [PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE](#) evaluates to the maximum ciphertext size of any supported AEAD encryption.

The `ciphertext` output parameters does not contain the additional data. For algorithms where the encrypted data and the authentication tag are defined as separate outputs, the authentication tag is appended to the encrypted data.

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_INVALID_HANDLE:**

`key` is not a valid key identifier.

- **PSA_ERROR_NOT_PERMITTED:**

The key does not have the `PSA_KEY_USAGE_ENCRYPT` flag, or it does not permit the requested algorithm.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not an AEAD algorithm.
- `key` is not compatible with `alg`.
- `nonce_length` is not valid for use with `alg` and `key`.
- `additional_data_length` or `plaintext_length` are too large for `alg`.

- **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported or is not an AEAD algorithm.
- `key` is not supported for use with `alg`.
- `nonce_length` is not supported for use with `alg` and `key`.
- `additional_data_length` or `plaintext_length` are too large.

- **PSA_ERROR_INSUFFICIENT_MEMORY**

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The `ciphertext_size` is too small. [PSA_AEAD_ENCRYPT_OUTPUT_SIZE](#) or [PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE](#) can be used to determine the required buffer size.

- **PSA_ERROR_COMMUNICATION_FAILURE**

- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_STORAGE_FAILURE**

- **PSA_ERROR_DATA_CORRUPT**

- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.5.1.2 `psa_aead_decrypt`

[psa_status_t](#) `psa_aead_decrypt`([psa_key_id_t](#) `key`, [psa_algorithm_t](#) `alg`, [const uint8_t*](#) `nonce`, [size_t](#) `nonce_length`, [const uint8_t*](#) `additional_data`, [size_t](#) `additional_data_length`, [const uint8_t*](#) `ciphertext`, [size_t](#) `ciphertext_length`, [uint8_t*](#) `plaintext`, [size_t](#) `plaintext_size`, [size_t*](#) `plaintext_length`)

Process an authenticated decryption operation.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must allow the usage `PSA_KEY_USAGE_DECRYPT`.
- **alg** ([psa_algorithm_t](#)) - [in] The AEAD algorithm to compute such that [PSA_ALG_IS_AEAD](#) is true.
- **nonce** (`const uint8_t*`) - [in] Nonce or IV to use.
- **nonce_length** (`size_t`) - [in] Size of the `nonce` buffer in bytes. This must be appropriate for the selected algorithm. The default nonce size is [PSA_AEAD_NONCE_LENGTH](#) where `key_type` is the type of `key`.
- **additional_data** (`const uint8_t*`) - [in] Additional data that has been authenticated but not encrypted.
- **additional_data_length** (`size_t`) - [in] Size of `additional_data` in bytes.
- **ciphertext** (`const uint8_t*`) - [in] Data that has been authenticated and encrypted.
- **ciphertext_length** (`size_t`) - [in] Size of ciphertext in bytes.
- **plaintext** (`uint8_t*`) - [out] Output buffer for the decrypted data.
- **plaintext_size** (`size_t`) - [in] Size of the `plaintext` buffer in bytes.
- **plaintext_length** (`size_t*`) - [out] On success, the size of the output in the plaintext buffer.

Description

The `plaintext_size` parameter must be appropriate for the selected algorithm and key:

- A sufficient output size is [PSA_AEAD_DECRYPT_OUTPUT_SIZE](#) where `key_type` is the type of `key`.
- [PSA_AEAD_DECRYPT_OUTPUT_MAX_SIZE](#) evaluates to the maximum plaintext size of any supported AEAD encryption.

The `ciphertext` input parameters does not contain the additional data. For algorithms where the encrypted data and the authentication tag are defined as separate inputs, the authentication tag is appended to the encrypted data.

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE:**
`key` is not a valid key identifier.
- **PSA_ERROR_INVALID_SIGNATURE:**
The ciphertext is not authentic.
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_DECRYPT` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
 - `alg` is not an AEAD algorithm.
 - `key` is not compatible with `alg`.
 - `nonce_length` is not valid for use with `alg` and `key`.

- `additional_data_length` or `ciphertext_length` are too large for alg.

- **PSA_ERROR_NOT_SUPPORTED:**

- alg is not supported or is not an AEAD algorithm.
- key is not supported for use with alg.
- nonce_length is not supported for use with alg and key.
- additional_data_length or ciphertext_length are too large.

- **PSA_ERROR_INSUFFICIENT_MEMORY**

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The `plaintext_size` is too small. [PSA_AEAD_DECRYPT_OUTPUT_SIZE](#) or [PSA_AEAD_DECRYPT_OUTPUT_MAX_SIZE](#) can be used to determine the required buffer size.

- **PSA_ERROR_COMMUNICATION_FAILURE**

- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_STORAGE_FAILURE**

- **PSA_ERROR_DATA_CORRUPT**

- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.5.2 Multiple Part

AEAD encryption

The sequence of operations to encrypt a message with authentication is as follows:

1. Allocate an operation object [psa_aead_operation_t](#) which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for [psa_aead_operation_t](#), e.g. [PSA_AEAD_OPERATION_INIT](#).
3. Call [psa_aead_encrypt_setup\(\)](#) to specify the algorithm and key.
4. If needed, call [psa_aead_set_lengths\(\)](#) to specify the length of the inputs to the subsequent calls to [psa_aead_update_ad\(\)](#) and [psa_aead_update\(\)](#). See the documentation of [psa_aead_set_lengths\(\)](#) for details.
5. Call either [psa_aead_generate_nonce\(\)](#) or [psa_aead_set_nonce\(\)](#) to generate or set the nonce. It is recommended to use [psa_aead_generate_nonce\(\)](#) unless the protocol being implemented requires a specific nonce value.
6. Call [psa_aead_update_ad\(\)](#) zero, one or more times, passing a fragment of the non-encrypted additional authenticated data each time.
7. Call [psa_aead_update\(\)](#) zero, one or more times, passing a fragment of the message to encrypt each time.
8. Call [psa_aead_finish\(\)](#).

AEAD decryption

The sequence of operations to decrypt a message with authentication is as follows:

1. Allocate an operation object [psa_aead_operation_t](#) which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for [psa_aead_operation_t](#), e.g. [PSA_AEAD_OPERATION_INIT](#).
3. Call [psa_aead_decrypt_setup\(\)](#) to specify the algorithm and key.
4. If needed, call [psa_aead_set_lengths\(\)](#) to specify the length of the inputs to the subsequent calls to [psa_aead_update_ad\(\)](#) and [psa_aead_update\(\)](#). See the documentation of [psa_aead_set_lengths\(\)](#) for details.

5. Call [psa_aead_set_nonce\(\)](#) with the nonce for the decryption.
6. Call [psa_aead_update_ad\(\)](#) zero, one or more times, passing a fragment of the non-encrypted additional authenticated data each time.
7. Call [psa_aead_update\(\)](#) zero, one or more times, passing a fragment of the ciphertext to decrypt each time.
8. Call [psa_aead_verify\(\)](#).

6.2.3.5.2.1 typedef `psa_aead_operation_t`

type **psa_aead_operation_t**

The type of the state object for multi-part AEAD operations.

Description

Before calling any function on an AEAD operation object, the application must initialize it by any of the following means:

- Set the object to all-bits-zero, for example:

```
psa_aead_operation_t operation;
memset(&operation, 0, sizeof(operation));
```

- Initialize the object to logical zero values by declaring the object as static or global without an explicit initializer, for example:

```
static psa_aead_operation_t operation;
```

- Initialize the object to the initializer [PSA_AEAD_OPERATION_INIT](#), for example:

```
psa_aead_operation_t operation = PSA_AEAD_OPERATION_INIT;
```

- Assign the result of the function [psa_aead_operation_init\(\)](#) to the object, for example:

```
psa_aead_operation_t operation;
```

6.2.3.5.2.2 macro `PSA_AEAD_OPERATION_INIT`

PSA_AEAD_OPERATION_INIT()

Suitable initializer for an AEAD operation object of [typedef `psa\_aead\_operation\_t`](#).

6.2.3.5.2.3 `psa_aead_operation_init`

[psa_aead_operation_t](#) **psa_aead_operation_init(void)**

Return an initial value for an AEAD operation object.

Parameters

- **void** - no arguments

Return

Return an initial value for an AEAD operation object.

6.2.3.5.2.4 psa_aead_encrypt_setup

[psa_status_t](#) [psa_aead_encrypt_setup](#)([psa_aead_operation_t](#) *operation, [psa_key_id_t](#) key, [psa_algorithm_t](#) alg)

Set the key for a multi-part authenticated encryption operation.

Parameters

- **operation** ([psa_aead_operation_t](#)*) - [in] The operation object to set up. It must have been initialized as per the documentation for [typedef psa_aead_operation_t](#) and not yet in use.
- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage `PSA_KEY_USAGE_ENCRYPT`.
- **alg** ([psa_algorithm_t](#)) - [in] The AEAD algorithm to compute such that [PSA_ALG_IS_AEAD](#) is true.

Description

After a successful call to [psa_aead_encrypt_setup\(\)](#), the operation is active, and the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to [psa_aead_finish\(\)](#).
- A call to [psa_aead_abort\(\)](#).

If [psa_aead_encrypt_setup\(\)](#) returns an error, the operation object is unchanged. If a subsequent function call with an active operation returns an error, the operation enters an error state.

To abandon an active operation, or reset an operation in an error state, call [psa_aead_abort\(\)](#).

Return

- **PSA_SUCCESS:**
Success. The operation is now active.
- **PSA_ERROR_INVALID_HANDLE:**
`key` is not a valid key identifier.
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_ENCRYPT` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
 - `alg` is not an AEAD algorithm.
 - `key` is not compatible with `alg`.
- **PSA_ERROR_NOT_SUPPORTED:**
 - `alg` is not supported.
 - `key` is not supported for use with `alg`.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**

- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be inactive.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.5.2.5 [psa_aead_decrypt_setup](#)

[psa_status_t](#) [psa_aead_decrypt_setup](#)([psa_aead_operation_t](#) *operation, [psa_key_id_t](#) key, [psa_algorithm_t](#) alg)

Set the key for a multi-part authenticated decryption operation.

Parameters

- **operation** ([psa_aead_operation_t](#)*) - [in] The operation object to set up. It must have been initialized as per the documentation for [typedef psa_aead_operation_t](#) and not yet in use.
- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage `PSA_KEY_USAGE_DECRYPT`.
- **alg** ([psa_algorithm_t](#)) - [in] The AEAD algorithm to compute such that [PSA_ALG_IS_AEAD](#) is true.

Description

After a successful call to [psa_aead_decrypt_setup\(\)](#), the operation is active, and the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to [psa_aead_verify\(\)](#).
- A call to [psa_aead_abort\(\)](#).

If [psa_aead_decrypt_setup\(\)](#) returns an error, the operation object is unchanged. If a subsequent function call with an active operation returns an error, the operation enters an error state.

To abandon an active operation, or reset an operation in an error state, call [psa_aead_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success. The operation is now active.
- **PSA_ERROR_INVALID_HANDLE:**

`key` is not a valid key identifier.
- **PSA_ERROR_NOT_PERMITTED:**

The key does not have the `PSA_KEY_USAGE_DECRYPT` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**

`alg` is not an AEAD algorithm. `key` is not compatible with `alg`.
- **PSA_ERROR_NOT_SUPPORTED:**

`alg` is not supported or is not an AEAD algorithm. `key` is not supported for use with `alg`.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**

- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- **`PSA_ERROR_BAD_STATE`:**
 - The operation state is not valid: it must be inactive.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.5.2.6 `psa_aead_set_lengths`

[psa_status_t](#) `psa_aead_set_lengths`([psa_aead_operation_t](#) *operation, [size_t](#) ad_length, [size_t](#) plaintext_length)

Declare the lengths of the message and additional data for AEAD.

Parameters

- **operation** ([psa_aead_operation_t](#)*) - [in] Active AEAD operation.
- **ad_length** ([size_t](#)) - [in] Size of the non-encrypted additional authenticated data in bytes.
- **plaintext_length** ([size_t](#)) - [in] Size of the plaintext to encrypt in bytes.

Description

The application must call this function before calling [psa_aead_set_nonce\(\)](#) or [psa_aead_generate_nonce\(\)](#), if the algorithm for the operation requires it. If the algorithm does not require it, calling this function is optional.

- For `PSA_ALG_CCM`, calling this function is required.
- For the other AEAD algorithms defined in this specification, calling this function is not required.
- For vendor-defined algorithm, refer to the vendor documentation.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_aead_abort\(\)](#).

Return

- **`PSA_SUCCESS`:**
 - Success.
- **`PSA_ERROR_INVALID_ARGUMENT`:**
 - `ad_length` or `plaintext_length` are too large for the chosen algorithm.
- **`PSA_ERROR_NOT_SUPPORTED`:**
 - `ad_length` or `plaintext_length` are too large.
- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- **`PSA_ERROR_BAD_STATE`:**
 - The operation state is not valid: it must be active, and [psa_aead_set_nonce\(\)](#) and [psa_aead_generate_nonce\(\)](#) must not have been called yet.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.5.2.7 psa_aead_generate_nonce

[psa_status_t](#) **psa_aead_generate_nonce**([psa_aead_operation_t](#) *operation, uint8_t *nonce, size_t nonce_size, size_t *nonce_length)

Generate a random nonce for an authenticated encryption operation.

Parameters

- **operation** ([psa_aead_operation_t](#)*) - [in] Active AEAD operation.
- **nonce** (uint8_t*) - [out] Buffer where the generated nonce is to be written.
- **nonce_size** (size_t) - [in] Size of the `nonce` buffer in bytes.
- **nonce_length** (size_t*) - [out] On success, the number of bytes of the generated nonce.

Description

This function generates a random nonce for the authenticated encryption operation with an appropriate size for the chosen algorithm, key type and key size.

The `nonce_size` parameter must be set appropriate for the selected algorithm and key:

- A sufficient output size is [PSA_AEAD_NONCE_LENGTH](#) where `key_type` is the type of key and `alg` is the algorithm that were used to set up the operation.
- [PSA_AEAD_NONCE_MAX_SIZE](#) evaluates to a sufficient output size for any supported AEAD algorithm.

The application must call [psa_aead_encrypt_setup\(\)](#) before calling this function. If applicable for the algorithm, the application must call [psa_aead_set_lengths\(\)](#) before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_aead_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the `nonce` buffer is too small. [PSA_AEAD_NONCE_LENGTH](#) or [PSA_AEAD_NONCE_MAX_SIZE](#) can be used to determine the required buffer size.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid: it must be an active AEAD encryption operation, with no nonce set.
- The operation state is not valid: this is an algorithm which requires [psa_aead_set_lengths\(\)](#) to be called before setting the nonce.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.5.2.8 psa_aead_set_nonce

[psa_status_t](#) [psa_aead_set_nonce](#)([psa_aead_operation_t](#) *operation, const uint8_t *nonce, size_t nonce_length)

Set the nonce for an authenticated encryption or decryption operation.

Parameters

- **operation** ([psa_aead_operation_t](#)*) - [in] Active AEAD operation.
- **nonce** (const uint8_t*) - [in] Buffer containing the nonce to use.
- **nonce_length** (size_t) - [in] Size of the nonce in bytes. This must be a valid nonce size for the chosen algorithm. The default nonce size is [PSA_AEAD_NONCE_LENGTH](#) where key_type and alg are type of key and the algorithm respectively that were used to set up the AEAD operation.

Description

This function sets the nonce for the authenticated encryption or decryption operation.

The application must call [psa_aead_encrypt_setup\(\)](#) or [psa_aead_decrypt_setup\(\)](#) before calling this function. If applicable for the algorithm, the application must call [psa_aead_set_lengths\(\)](#) before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_aead_abort\(\)](#).

Note:

When encrypting, [psa_aead_generate_nonce\(\)](#) is recommended instead of using this function, unless implementing a protocol that requires a non-random IV.

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_INVALID_ARGUMENT:**

nonce_length is not valid for the chosen algorithm.

- **PSA_ERROR_NOT_SUPPORTED:**

nonce_length is not supported for use with the operation's algorithm and key.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid: it must be active, with no nonce set.
- The operation state is not valid: this is an algorithm which requires [psa_aead_set_lengths\(\)](#) to be called before setting the nonce.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.5.2.9 `psa_aead_update_ad`

[psa_status_t](#) `psa_aead_update_ad`([psa_aead_operation_t](#) *operation, const uint8_t *input, size_t input_length)

Pass additional data to an active AEAD operation.

Parameters

- **operation** ([psa_aead_operation_t](#)*) - [in] Active AEAD operation.
- **input** (const uint8_t*) - [in] Buffer containing the fragment of additional data.
- **input_length** (size_t) - [in] Size of the `input` buffer in bytes.

Description

Additional data is authenticated, but not encrypted.

This function can be called multiple times to pass successive fragments of the additional data. This function must not be called after passing data to encrypt or decrypt with [psa_aead_update\(\)](#).

The following must occur before calling this function:

1. Call either [psa_aead_encrypt_setup\(\)](#) or [psa_aead_decrypt_setup\(\)](#).
2. Set the nonce with [psa_aead_generate_nonce\(\)](#) or [psa_aead_set_nonce\(\)](#).

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_aead_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_INVALID_ARGUMENT:**

Excess additional data: the total input length to [psa_aead_update_ad\(\)](#) is greater than the additional data length that was previously specified with [psa_aead_set_lengths\(\)](#), or is too large for the chosen AEAD algorithm.

- **PSA_ERROR_NOT_SUPPORTED:**

The total additional data length is too large.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid: it must be active, have a nonce set, have lengths set if required by the algorithm, and [psa_aead_update\(\)](#) must not have been called yet.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.5.2.10 `psa_aead_update`

[psa_status_t](#) `psa_aead_update`([psa_aead_operation_t](#) *operation, const uint8_t *input, size_t input_length, uint8_t *output, size_t output_size, size_t *output_length)

Encrypt or decrypt a message fragment in an active AEAD operation.

Parameters

- **operation** ([psa_aead_operation_t*](#)) - [in] Active AEAD operation.
- **input** (const uint8_t*) - [in] Buffer containing the message fragment to encrypt or decrypt.
- **input_length** (size_t) - [in] Size of the `input` buffer in bytes.
- **output** (uint8_t*) - [out] Buffer where the output is to be written.
- **output_size** (size_t) - [in] Size of the `output` buffer in bytes.
- **output_length** (size_t*) - [out] On success, the number of bytes that make up the returned output.

Description

The `output_size` parameter must be appropriate for the selected algorithm and key:

- A sufficient output size is [PSA_AEAD_UPDATE_OUTPUT_SIZE](#) where `key_type` is the type of `key` and `alg` is the algorithm that were used to set up the operation.
- [PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE](#) evaluates to the maximum output size of any supported AEAD algorithm.

The following must occur before calling this function:

1. Call either [psa_aead_encrypt_setup\(\)](#) or [psa_aead_decrypt_setup\(\)](#). The choice of setup function determines whether this function encrypts or decrypts its input.
2. Set the nonce with [psa_aead_generate_nonce\(\)](#) or [psa_aead_set_nonce\(\)](#).
3. Call [psa_aead_update_ad\(\)](#) to pass all the additional data.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_aead_abort\(\)](#).

Note:

This function does not require the input to be aligned to any particular block boundary. If the implementation can only process a whole block at a time, it must consume all the input provided, but it might delay the end of the corresponding output until a subsequent call to [psa_aead_update\(\)](#), [psa_aead_finish\(\)](#) or [psa_aead_verify\(\)](#) provides sufficient input. The amount of data that can be delayed in this way is bounded by [PSA_AEAD_UPDATE_OUTPUT_SIZE](#), [PSA_AEAD_FINISH_OUTPUT_SIZE](#), or [PSA_AEAD_VERIFY_OUTPUT_SIZE](#).

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
 - The size of the `output` buffer is too small. [PSA_AEAD_UPDATE_OUTPUT_SIZE](#) or [PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE](#) can be used to determine the required buffer size.
- **PSA_ERROR_INVALID_ARGUMENT:**
 - Incomplete additional data: the total length of input to [psa_aead_update_ad\(\)](#) is less than the additional data length that was previously specified with [psa_aead_set_lengths\(\)](#).
 - Excess input data: the total length of input to [psa_aead_update\(\)](#) is greater than the plaintext length that was previously specified with [psa_aead_set_lengths\(\)](#), or is too large for the specific AEAD algorithm.

- **PSA_ERROR_NOT_SUPPORTED:**

The total input length is too large.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid: it must be active, have a nonce set, and have lengths set if required by the algorithm.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.5.2.11 `psa_aead_finish`

[psa_status_t](#) `psa_aead_finish`([psa_aead_operation_t](#) *operation, [uint8_t](#) *ciphertext, [size_t](#) ciphertext_size, [size_t](#) *ciphertext_length, [uint8_t](#) *tag, [size_t](#) tag_size, [size_t](#) *tag_length)

Finish encrypting a message in an AEAD operation.

Parameters

- **operation** ([psa_aead_operation_t](#)*) - [in] Active AEAD operation.
- **ciphertext** ([uint8_t](#)*) - [out] Buffer where the last part of the ciphertext is to be written.
- **ciphertext_size** ([size_t](#)) - [in] Size of the `ciphertext` buffer in bytes.
- **ciphertext_length** ([size_t](#)*) - [out] On success, the number of bytes of returned ciphertext.
- **tag** ([uint8_t](#)*) - [out] Buffer where the authentication tag is to be written.
- **tag_size** ([size_t](#)) - [in] Size of the `tag` buffer in bytes.
- **tag_length** ([size_t](#)*) - [out] On success, the number of bytes that make up the returned tag.

Description

The operation must have been set up with [psa_aead_encrypt_setup\(\)](#).

This function finishes the authentication of the additional data formed by concatenating the inputs passed to preceding calls to [psa_aead_update_ad\(\)](#) with the plaintext formed by concatenating the inputs passed to preceding calls to [psa_aead_update\(\)](#).

This function has two output buffers:

- `ciphertext` contains trailing ciphertext that was buffered from preceding calls to [psa_aead_update\(\)](#).
- `tag` contains the authentication tag.

The `ciphertext_size` parameter must be appropriate for the selected algorithm and key:

- A sufficient output size is [PSA_AEAD_FINISH_OUTPUT_SIZE](#) where `key_type` is the type of key and `alg` is the algorithm that were used to set up the operation.
- [PSA_AEAD_FINISH_OUTPUT_MAX_SIZE](#) evaluates to the maximum output size of any supported AEAD algorithm.

The `tag_size` parameter must be appropriate for the selected algorithm and key:

- The exact tag size is `PSA_AEAD_TAG_LENGTH` where `key_type` and `key_bits` are the type and bit-size of the key, and `alg` is the algorithm that were used in the call to `psa_aead_encrypt_setup()`.
- `PSA_AEAD_TAG_MAX_SIZE` evaluates to the maximum tag size of any supported AEAD algorithm.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_aead_abort()`.

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the `ciphertext` or `tag` buffer is too small. `PSA_AEAD_FINISH_OUTPUT_SIZE` or `PSA_AEAD_FINISH_OUTPUT_MAX_SIZE` can be used to determine the required ciphertext buffer size. `PSA_AEAD_TAG_LENGTH` or `PSA_AEAD_TAG_MAX_SIZE` can be used to determine the required `tag` buffer size.

- **PSA_ERROR_INVALID_ARGUMENT:**

- Incomplete additional data: the total length of input to `psa_aead_update_ad()` is less than the additional data length that was previously specified with `psa_aead_set_lengths()`.
- Incomplete plaintext: the total length of input to `psa_aead_update()` is less than the plaintext length that was previously specified with `psa_aead_set_lengths()`.

- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`

- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid: it must be an active encryption operation with a nonce set.
- The library has not been previously initialized by `psa_crypto_init()`.

6.2.3.5.2.12 psa_aead_verify

`psa_status_t` `psa_aead_verify`(`psa_aead_operation_t` *`operation`, `uint8_t` *`plaintext`, `size_t` `plaintext_size`, `size_t` *`plaintext_length`, `const uint8_t` *`tag`, `size_t` `tag_length`)

Finish authenticating and decrypting a message in an AEAD operation.

Parameters

- `operation` (`psa_aead_operation_t` *) - [in] Active AEAD operation.
- `plaintext` (`uint8_t` *) - [out] Buffer where the last part of the plaintext is to be written. This is the remaining data from previous calls to `psa_aead_update()` that could not be processed until the end of the input.
- `plaintext_size` (`size_t`) - [in] Size of the `plaintext` buffer in bytes.
- `plaintext_length` (`size_t` *) - [out] On success, the number of bytes of returned plaintext.
- `tag` (`const uint8_t` *) - [in] Buffer containing the authentication tag.
- `tag_length` (`size_t`) - [in] Size of the `tag` buffer in bytes.

Description

The operation must have been set up with [psa_aead_decrypt_setup\(\)](#).

This function finishes the authenticated decryption of the message components:

- The additional data consisting of the concatenation of the inputs passed to preceding calls to [psa_aead_update_ad\(\)](#).
- The ciphertext consisting of the concatenation of the inputs passed to preceding calls to [psa_aead_update\(\)](#).
- The tag passed to this function call.

The `plaintext_size` parameter must be appropriate for the selected algorithm and key:

- A sufficient output size is [PSA_AEAD_VERIFY_OUTPUT_SIZE](#) where `key_type` is the type of key and `alg` is the algorithm that were used to set up the operation.
- [PSA_AEAD_VERIFY_OUTPUT_MAX_SIZE](#) evaluates to the maximum output size of any supported AEAD algorithm.

If the authentication tag is correct, this function outputs any remaining plaintext and reports success. If the authentication tag is not correct, this function returns `PSA_ERROR_INVALID_SIGNATURE`.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_aead_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_INVALID_SIGNATURE:**

The calculated authentication tag does not match the value in `tag`.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the `plaintext` buffer is too small. [PSA_AEAD_VERIFY_OUTPUT_SIZE](#) or [PSA_AEAD_VERIFY_OUTPUT_MAX_SIZE](#) can be used to determine the required buffer size.

- **PSA_ERROR_INVALID_ARGUMENT:**

- Incomplete additional data: the total length of input to [psa_aead_update_ad\(\)](#) is less than the additional data length that was previously specified with [psa_aead_set_lengths\(\)](#).
- Incomplete ciphertext: the total length of input to [psa_aead_update\(\)](#) is less than the plaintext length that was previously specified with [psa_aead_set_lengths\(\)](#).

- **PSA_ERROR_INSUFFICIENT_MEMORY**

- **PSA_ERROR_COMMUNICATION_FAILURE**

- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_STORAGE_FAILURE**

- **PSA_ERROR_DATA_CORRUPT**

- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid: it must be an active decryption operation with a nonce set.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.5.2.13 `psa_aead_abort`

[psa_status_t](#) `psa_aead_abort`([psa_aead_operation_t](#) *operation)

Abort an AEAD operation.

Parameters

- **operation** ([psa_aead_operation_t](#)*) - Initialized AEAD operation.

Description

Aborting an operation frees all associated resources except for the operation object itself. Once aborted, the operation object can be reused for another operation by calling [psa_aead_encrypt_setup\(\)](#) or [psa_aead_decrypt_setup\(\)](#) again.

This function can be called any time after the operation object has been initialized as described in [typedef `psa\_aead\_operation\_t`](#).

In particular, calling [psa_aead_abort\(\)](#) after the operation has been terminated by a call to [psa_aead_abort\(\)](#), [psa_aead_finish\(\)](#) or [psa_aead_verify\(\)](#) is safe and has no effect.

Return

- **PSA_SUCCESS**
Success. The operation object can now be discarded or reused.
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.5.3 Macros

6.2.3.5.3.1 macro `PSA_AEAD_ENCRYPT_OUTPUT_SIZE`

`PSA_AEAD_ENCRYPT_OUTPUT_SIZE`(*key_type*, *alg*, *plaintext_length*)

A sufficient output buffer size for [psa_aead_encrypt\(\)](#), in bytes.

Parameters

- **key_type** - [in] A symmetric key type that is compatible with algorithm *alg*.
- **alg** - [in] An AEAD algorithm such that [PSA_ALG_IS_AEAD](#) is true.
- **plaintext_length** - [in] Size of the plaintext in bytes.

Description

If the size of the ciphertext buffer is at least this large, it is guaranteed that [psa_aead_encrypt\(\)](#) will not fail due to an insufficient buffer size. Depending on the algorithm, the actual size of the ciphertext might be smaller.

See also [PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE](#).

Return

The AEAD ciphertext size for the specified key type and algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0.

6.2.3.5.3.2 macro `PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE`

`PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE`(*plaintext_length*)

The maximum size of the output of [psa_aead_encrypt\(\)](#), for any of the supported key types and AEAD algorithms.

Parameters

- **`plaintext_length`** - [in] Size of the plaintext in bytes.

Description

If the size of the ciphertext buffer is at least this large, it is guaranteed that [psa_aead_encrypt\(\)](#) will not fail due to an insufficient buffer size.

See also [PSA_AEAD_ENCRYPT_OUTPUT_SIZE](#).

6.2.3.5.3.3 macro `PSA_AEAD_DECRYPT_OUTPUT_SIZE`

`PSA_AEAD_DECRYPT_OUTPUT_SIZE`(*key_type*, *alg*, *ciphertext_length*)

A sufficient output buffer size for [psa_aead_decrypt\(\)](#), in bytes.

Parameters

- **`key_type`** - [in] A symmetric key type that is compatible with algorithm *alg*.
- **`alg`** - [in] An AEAD algorithm such that [PSA_ALG_IS_AEAD](#) is true.
- **`ciphertext_length`** - [in] Size of the ciphertext in bytes.

Description

If the size of the plaintext buffer is at least this large, it is guaranteed that [psa_aead_decrypt\(\)](#) will not fail due to an insufficient buffer size. Depending on the algorithm, the actual size of the plaintext might be smaller.

See also [PSA_AEAD_DECRYPT_OUTPUT_MAX_SIZE](#).

Return

The AEAD plaintext size for the specified key type and algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0.

6.2.3.5.3.4 macro `PSA_AEAD_DECRYPT_OUTPUT_MAX_SIZE`

`PSA_AEAD_DECRYPT_OUTPUT_MAX_SIZE`(*ciphertext_length*)

The maximum size of the output of [psa_aead_decrypt\(\)](#), for any of the supported key types and AEAD algorithms.

Parameters

- **`ciphertext_length`** - [in] Size of the ciphertext in bytes.

Description

If the size of the plaintext buffer is at least this large, it is guaranteed that [psa_aead_decrypt\(\)](#) will not fail due to an insufficient buffer size.

See also [PSA_AEAD_DECRYPT_OUTPUT_SIZE](#).

6.2.3.5.3.5 macro PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE

PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE(*input_length*)

A sufficient output buffer size for [psa_aead_update\(\)](#), for any of the supported key types and AEAD algorithms.

Parameters

- **input_length** - [in] Size of the input in bytes.

Description

If the size of the output buffer is at least this large, it is guaranteed that [psa_aead_update\(\)](#) will not fail due to an insufficient buffer size.

See also [PSA_AEAD_UPDATE_OUTPUT_SIZE](#).

6.2.3.5.3.6 macro PSA_AEAD_UPDATE_OUTPUT_SIZE

PSA_AEAD_UPDATE_OUTPUT_SIZE(*key_type*, *alg*, *input_length*)

A sufficient output buffer size for [psa_aead_update\(\)](#), in bytes.

Parameters

- **key_type** - [in] A symmetric key type that is compatible with algorithm *alg*.
- **alg** - [in] An AEAD algorithm such that [PSA_ALG_IS_AEAD](#) is true.
- **input_length** - [in] Size of the input in bytes.

Description

If the size of the output buffer is at least this large, it is guaranteed that [psa_aead_update\(\)](#) will not fail due to an insufficient buffer size. The actual size of the output might be smaller in any given call.

See also [PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE](#).

Return

A sufficient output buffer size for the specified key type and algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0.

6.2.3.5.3.7 macro PSA_AEAD_FINISH_OUTPUT_SIZE

PSA_AEAD_FINISH_OUTPUT_SIZE(*key_type*, *alg*)

A sufficient ciphertext buffer size for [psa_aead_finish\(\)](#), in bytes.

Parameters

- **key_type** - [in] A symmetric key type that is compatible with algorithm *alg*.
- **alg** - [in] An AEAD algorithm such that [PSA_ALG_IS_AEAD](#) is true.

Description

If the size of the ciphertext buffer is at least this large, it is guaranteed that [psa_aead_finish\(\)](#) will not fail due to an insufficient ciphertext buffer size. The actual size of the output might be smaller in any given call.

See also [PSA_AEAD_FINISH_OUTPUT_MAX_SIZE](#).

Return

A sufficient ciphertext buffer size for the specified key type and algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0.

6.2.3.5.3.8 macro PSA_AEAD_FINISH_OUTPUT_MAX_SIZE

PSA_AEAD_FINISH_OUTPUT_MAX_SIZE()

The maximum the ciphertext buffer size of [psa_aead_finish\(\)](#), for any of the supported key types and AEAD algorithms.

Description

See also [PSA_AEAD_FINISH_OUTPUT_SIZE](#).

6.2.3.5.3.9 macro PSA_AEAD_VERIFY_OUTPUT_SIZE

PSA_AEAD_VERIFY_OUTPUT_SIZE(*key_type*, *alg*)

A sufficient plaintext buffer size for [psa_aead_verify\(\)](#), in bytes.

Parameters

- **key_type** - [in] A symmetric key type that is compatible with algorithm *alg*.
- **alg** - [in] An AEAD algorithm such that [PSA_ALG_IS_AEAD](#) is true.

Description

If the size of the plaintext buffer is at least this large, it is guaranteed that [psa_aead_verify\(\)](#) will not fail due to an insufficient plaintext buffer size. The actual size of the output might be smaller in any given call.

See also [PSA_AEAD_VERIFY_OUTPUT_MAX_SIZE](#).

Return

A sufficient plaintext buffer size for the specified key type and algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0.

6.2.3.5.3.10 macro PSA_AEAD_VERIFY_OUTPUT_MAX_SIZE

PSA_AEAD_VERIFY_OUTPUT_MAX_SIZE()

The maximum plaintext buffer size of [psa_aead_verify\(\)](#), for any of the supported key types and AEAD algorithms.

Description

See also [PSA_AEAD_VERIFY_OUTPUT_SIZE](#).

6.2.3.5.3.11 macro PSA_AEAD_NONCE_LENGTH

PSA_AEAD_NONCE_LENGTH(*key_type*, *alg*)

The default nonce size for an AEAD algorithm, in bytes.

Parameters

- **key_type** - [in] A symmetric key type that is compatible with algorithm *alg*.
- **alg** - [in] An AEAD algorithm such that [PSA_ALG_IS_AEAD](#) is true.

Description

This macro can be used to allocate a buffer of sufficient size to store the nonce output from [psa_aead_generate_nonce\(\)](#).

See also [PSA_AEAD_NONCE_MAX_SIZE](#).

Return

The default nonce size for the specified key type and algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0.

6.2.3.5.3.12 macro PSA_AEAD_NONCE_MAX_SIZE

PSA_AEAD_NONCE_MAX_SIZE()

The maximum nonce size for all supported AEAD algorithms, in bytes.

Description

See also [PSA_AEAD_NONCE_LENGTH](#).

6.2.3.5.3.13 macro PSA_AEAD_TAG_LENGTH

PSA_AEAD_TAG_LENGTH(*key_type*, *key_bits*, *alg*)

The length of a tag for an AEAD algorithm, in bytes.

Parameters

- **key_type** - [in] The type of the AEAD key.
- **key_bits** - [in] The size of the AEAD key in bits.
- **alg** - [in] An AEAD algorithm such that [PSA_ALG_IS_AEAD](#) is true.

Description

This is the size of the tag output from [psa_aead_finish\(\)](#).

If the size of the tag buffer is at least this large, it is guaranteed that [psa_aead_finish\(\)](#) will not fail due to an insufficient tag buffer size.

See also [PSA_AEAD_TAG_MAX_SIZE](#).

Return

The tag length for the specified algorithm and key. If the AEAD algorithm does not have an identified tag that can be distinguished from the rest of the ciphertext, return 0. If the AEAD algorithm is not recognized, return 0.

6.2.3.5.3.14 macro `PSA_AEAD_TAG_MAX_SIZE`

`PSA_AEAD_TAG_MAX_SIZE()`

The maximum tag size for all supported AEAD algorithms, in bytes.

Description

See also [PSA_AEAD_TAG_LENGTH](#).

6.2.3.5.3.15 macro `PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG`

`PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG(aead_alg)`

An AEAD algorithm with the default tag length.

Parameters

- `aead_alg` - [in] An AEAD algorithm such that [PSA_ALG_IS_AEAD](#) is true.

Description

This macro can be used to construct the AEAD algorithm with default tag length from an AEAD algorithm with a shortened tag. See also [PSA_ALG_AEAD_WITH_SHORTENED_TAG](#).

Return

The corresponding AEAD algorithm with the default tag length for that algorithm.

6.2.3.5.3.16 macro `PSA_ALG_AEAD_TAG_LENGTH`

`PSA_ALG_AEAD_TAG_LENGTH(aead_alg)`

Retrieve the tag length of a specified AEAD algorithm.

Parameters

- `aead_alg` - [in] An AEAD algorithm identifier such that [PSA_ALG_IS_AEAD](#) is true.

Return

The tag length specified by the input algorithm. 0 if `aead_alg` is not a supported AEAD algorithm.

6.2.3.5.3.17 macro `PSA_ALG_AEAD_WITH_SHORTENED_TAG`

`PSA_ALG_AEAD_WITH_SHORTENED_TAG(aead_alg, tag_length)`

Macro to build a AEAD algorithm with a shortened tag.

Parameters

- `aead_alg` - [in] An AEAD algorithm identifier such that [PSA_ALG_IS_AEAD](#) is true.
- `tag_length` - [in] Desired length of the authentication tag in bytes.

Description

An AEAD algorithm with a shortened tag is similar to the corresponding AEAD algorithm, but has an authentication tag that consists of fewer bytes. Depending on the algorithm, the tag length might affect the calculation of the ciphertext.

The AEAD algorithm with a default length tag can be recovered using [PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG](#).

Return

The corresponding AEAD algorithm with the specified tag length.

Unspecified if `aead_alg` is not a supported AEAD algorithm or if `tag_length` is not valid for the specified AEAD algorithm.

6.2.3.5.3.18 macro PSA_ALG_AEAD_WITH_AT_LEAST_THIS_LENGTH_TAG

PSA_ALG_AEAD_WITH_AT_LEAST_THIS_LENGTH_TAG(*aead_alg, min_tag_length*)

Macro to build an AEAD minimum-tag-length wildcard algorithm.

Parameters

- **aead_alg** - [in] An AEAD algorithm such that [PSA_ALG_IS_AEAD](#) is true.
- **min_tag_length** - [in] Desired minimum length of the authentication tag in bytes. This must be at least 1 and at most the largest allowed tag length of the algorithm.

Description

A key with a minimum-tag-length AEAD wildcard algorithm as permitted algorithm policy can be used with all AEAD algorithms sharing the same base algorithm, and where the tag length of the specific algorithm is equal to or larger than the minimum tag length specified by the wildcard algorithm.

Note:

When setting the minimum required tag length to less than the smallest tag length allowed by the base algorithm, this effectively becomes an 'any-tag-length-allowed' policy for that base algorithm.

The AEAD algorithm with a default length tag can be recovered using [PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG](#).

Compatible key types

The resulting wildcard AEAD algorithm is compatible with the same key types as the AEAD algorithm used to construct it.

Return

The corresponding AEAD wildcard algorithm with the specified minimum tag length.

Unspecified if `aead_alg` is not a supported AEAD algorithm or if `min_tag_length` is less than 1 or too large for the specified AEAD algorithm.

6.2.3.6 Asymmetric Encryption

6.2.3.6.1 Single Part

6.2.3.6.1.1 `psa_asymmetric_encrypt`

[psa_status_t](#) `psa_asymmetric_encrypt`([psa_key_id_t](#) key, [psa_algorithm_t](#) alg, const uint8_t *input, size_t input_length, const uint8_t *salt, size_t salt_length, uint8_t *output, size_t output_size, size_t *output_length)

Encrypt a short message with a public key.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must be a public key or an asymmetric key pair. It must allow the usage `PSA_KEY_USAGE_ENCRYPT`.
- **alg** ([psa_algorithm_t](#)) - [in] An asymmetric encryption algorithm such that the [PSA_ALG_IS_ASYMMETRIC_ENCRYPTION](#) is true.
- **input** (const uint8_t*) - [in] The message to encrypt.
- **input_length** (size_t) - [in] Size of the `input` buffer in bytes.
- **salt** (const uint8_t*) - [in] A salt or label, if supported by the encryption algorithm. If the algorithm does not support a salt, pass NULL. If the algorithm supports an optional salt, pass NULL to indicate that there is no salt.
- **salt_length** (size_t) - [in] Size of the `salt` buffer in bytes. If salt is NULL, pass 0.
- **output** (uint8_t*) - [out] Buffer where the encrypted message is to be written.
- **output_size** (size_t) - [in] Size of the `output` buffer in bytes.
- **output_length** (size_t*) - [out] On success, the number of bytes that make up the returned output.

Description

For `PSA_ALG_RSA_PKCS1V15_CRYPT`, no salt is supported.

The `output_size` parameter must be appropriate for the selected algorithm and key:

- The required output size is [PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE](#) where `key_type` and `key_bits` are the type and bit-size respectively of `key`.
- [PSA_ASYMMETRIC_ENCRYPT_OUTPUT_MAX_SIZE](#) evaluates to the maximum output size of any supported asymmetric encryption.

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE:**
`key` is not a valid key identifier.
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_ENCRYPT` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the `output` buffer is too small. [PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE](#) or [PSA_ASYMMETRIC_ENCRYPT_OUTPUT_MAX_SIZE](#) can be used to determine the required buffer size.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not an asymmetric encryption algorithm.
- `key` is not a public key or asymmetric key pair, that is compatible with `alg`.
- `input_length` is not valid for the algorithm and key type.
- `salt_length` is not valid for the algorithm and key type.

- **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported.
- `key` is not supported for use with `alg`.
- `input_length` or `salt_length` are too large.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_INSUFFICIENT_ENTROPY**
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.6.1.2 `psa_asymmetric_decrypt`

[psa_status_t](#) `psa_asymmetric_decrypt`([psa_key_id_t](#) `key`, [psa_algorithm_t](#) `alg`, `const uint8_t *input`, `size_t input_length`, `const uint8_t *salt`, `size_t salt_length`, `uint8_t *output`, `size_t output_size`, `size_t *output_length`)

Decrypt a short message with a private key.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must be an asymmetric key pair. It must allow the usage `PSA_KEY_USAGE_DECRYPT`.
- **alg** ([psa_algorithm_t](#)) - [in] An asymmetric encryption algorithm that is compatible with the type of key.
- **input** (`const uint8_t*`) - [in] The message to decrypt.
- **input_length** (`size_t`) - [in] Size of the `input` buffer in bytes.
- **salt** (`const uint8_t*`) - [in] A salt or label, if supported by the encryption algorithm. If the algorithm does not support a salt, pass `NULL`. If the algorithm supports an optional salt, pass `NULL` to indicate that there is no salt.
- **salt_length** (`size_t`) - [in] Size of the `salt` buffer in bytes. If `salt` is `NULL`, pass 0.
- **output** (`uint8_t*`) - [out] Buffer where the decrypted message is to be written.
- **output_size** (`size_t`) - [in] Size of the `output` buffer in bytes.
- **output_length** (`size_t*`) - [out] On success, the number of bytes that make up the returned output.

Description

For `PSA_ALG_RSA_PKCS1V15_CRYPT`, no salt is supported.

The `output_size` parameter must be appropriate for the selected algorithm and key:

- The required output size is [PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE](#) where `key_type` and `key_bits` are the type and bit-size respectively of `key`.
- [PSA_ASYMMETRIC_DECRYPT_OUTPUT_MAX_SIZE](#) evaluates to the maximum output size of any supported asymmetric decryption.

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_INVALID_HANDLE:**

`key` is not a valid key identifier.

- **PSA_ERROR_NOT_PERMITTED:**

The key does not have the `PSA_KEY_USAGE_DECRYPT` flag, or it does not permit the requested algorithm.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the `output` buffer is too small. [PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE](#) or [PSA_ASYMMETRIC_DECRYPT_OUTPUT_MAX_SIZE](#) can be used to determine the required buffer size.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not an asymmetric encryption algorithm.
- `key` is not an asymmetric key pair, that is compatible with `alg`.
- `input_length` is not valid for the algorithm and key type.
- `salt_length` is not valid for the algorithm and key type.

- **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported.
- `key` is not supported for use with `alg`.
- `input_length` or `salt_length` are too large.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_INSUFFICIENT_ENTROPY**
- **PSA_ERROR_INVALID_PADDING**
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.6.1.3 macro `PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE`

`PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE(key_type, key_bits, alg)`

Sufficient output buffer size for [psa_asymmetric_decrypt\(\)](#).

Parameters

- **key_type** - [in] An asymmetric key type, either a key pair or a public key.
- **key_bits** - [in] The size of the key in bits.
- **alg** - [in] The asymmetric encryption algorithm such that [PSA_ALG_IS_ASYMMETRIC_ENCRYPTION](#) is true.

Description

If the size of the output buffer is at least this large, it is guaranteed that [psa_asymmetric_decrypt\(\)](#) will not fail due to an insufficient buffer size. The actual size of the output might be smaller in any given call.

See also [PSA_ASYMMETRIC_DECRYPT_OUTPUT_MAX_SIZE](#).

Return

A sufficient output buffer size for the specified asymmetric encryption algorithm and key parameters. If algorithm or key is not recognized, return 0.

6.2.3.6.1.4 macro PSA_ASYMMETRIC_DECRYPT_OUTPUT_MAX_SIZE

PSA_ASYMMETRIC_DECRYPT_OUTPUT_MAX_SIZE()

A sufficient output buffer size for [psa_asymmetric_decrypt\(\)](#), for any supported asymmetric decryption.

Description

See also [PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE](#).

6.2.3.6.1.5 macro PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE

PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE(*key_type*, *key_bits*, *alg*)

Sufficient output buffer size for [psa_asymmetric_encrypt\(\)](#).

Parameters

- **key_type** - [in] An asymmetric key type, either a key pair or a public key.
- **key_bits** - [in] The size of the key in bits.
- **alg** - [in] The asymmetric encryption algorithm such that [PSA_ALG_IS_ASYMMETRIC_ENCRYPTION](#) is true.

Description

If the size of the output buffer is at least this large, it is guaranteed that [psa_asymmetric_encrypt\(\)](#) will not fail due to an insufficient buffer size. The actual size of the output might be smaller in any given call.

See also [PSA_ASYMMETRIC_ENCRYPT_OUTPUT_MAX_SIZE](#).

Return

A sufficient output buffer size for the specified asymmetric encryption algorithm and key parameters. If algorithm or key is not recognized, return 0.

6.2.3.6.1.6 macro PSA_ASYMMETRIC_ENCRYPT_OUTPUT_MAX_SIZE

PSA_ASYMMETRIC_ENCRYPT_OUTPUT_MAX_SIZE()

A sufficient output buffer size for [psa_asymmetric_encrypt\(\)](#), for any supported asymmetric encryption.

Description

See also [PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE](#).

6.2.3.7 Symmetric Encryption

6.2.3.7.1 Single Part

6.2.3.7.1.1 psa_cipher_encrypt

[psa_status_t](#) **psa_cipher_encrypt**([psa_key_id_t](#) key, [psa_algorithm_t](#) alg, const uint8_t *input, size_t input_length, uint8_t *output, size_t output_size, size_t *output_length)

Encrypt a message using a symmetric cipher.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must allow the usage PSA_KEY_USAGE_ENCRYPT.
- **alg** ([psa_algorithm_t](#)) - [in] The cipher algorithm to compute such that [PSA_ALG_IS_CIPHER](#) is true.
- **input** (const uint8_t*) - [in] Buffer containing the message to encrypt.
- **input_length** (size_t) - [in] Size of the `input` buffer in bytes.
- **output** (uint8_t*) - [out] Buffer where the output is to be written. The output contains the IV followed by the ciphertext proper.
- **output_size** (size_t) - [in] Size of the `output` buffer in bytes.
- **output_length** (size_t*) - [out] On success, the number of bytes that make up the output.

Description

This function encrypts a message with a random initialization vector (IV). The length of the IV is [PSA_CIPHER_IV_LENGTH](#) where `key_type` is the type of `key`. The output of [psa_cipher_encrypt\(\)](#) is the IV followed by the ciphertext.

Use the multi-part operation interface with a [typedef psa_cipher_operation_t](#) object to provide other forms of IV or to manage the IV and ciphertext independently.

The `output_size` parameter must be appropriate for the selected algorithm and key:

- A sufficient output size is [PSA_CIPHER_ENCRYPT_OUTPUT_SIZE](#) where `key_type` is the type of `key`.
- [PSA_CIPHER_ENCRYPT_OUTPUT_MAX_SIZE](#) evaluates to the maximum output size of any supported cipher encryption.

Return

- **PSA_SUCCESS:**
Success.

- **PSA_ERROR_INVALID_HANDLE:**

`key` is not a valid key identifier.

- **PSA_ERROR_NOT_PERMITTED:**

The key does not have the `PSA_KEY_USAGE_ENCRYPT` flag, or it does not permit the requested algorithm.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not a cipher algorithm.
- `key` is not compatible with `alg`.
- `input_length` is not valid for the algorithm and key type. The length must be a multiple of block size.

- **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported.
- `key` is not supported for use with `alg`.
- `input_length` is too large.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The `output_size` is too small. [PSA_CIPHER_ENCRYPT_OUTPUT_SIZE](#) or [PSA_CIPHER_ENCRYPT_OUTPUT_MAX_SIZE](#) can be used to determine the required buffer size.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.7.1.2 `psa_cipher_decrypt`

[psa_status_t](#) `psa_cipher_decrypt`([psa_key_id_t](#) `key`, [psa_algorithm_t](#) `alg`, *const uint8_t *input*, *size_t input_length*, *uint8_t *output*, *size_t output_size*, *size_t *output_length*)

Decrypt a message using a symmetric cipher.

Parameters

- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage `PSA_KEY_USAGE_DECRYPT`.
- **alg** ([psa_algorithm_t](#)) - [in] The cipher algorithm to compute such that [PSA_ALG_IS_CIPHER](#) is true.
- **input** (*const uint8_t**) - [in] Buffer containing the message to decrypt. This consists of the IV followed by the ciphertext proper.
- **input_length** (*size_t*) - [in] Size of the `input` buffer in bytes.
- **output** (*uint8_t**) - [out] Buffer where the plaintext is to be written.
- **output_size** (*size_t*) - [in] Size of the `output` buffer in bytes.
- **output_length** (*size_t**) - [out] On success, the number of bytes that make up the output.

Description

This function decrypts a message encrypted with a symmetric cipher.

The input to this function must contain the IV followed by the ciphertext, as output by [psa_cipher_encrypt\(\)](#). The IV must be [PSA_CIPHER_IV_LENGTH](#) bytes in length, where `key_type` is the type of `key`.

Use the multi-part operation interface with a [typedef psa_cipher_operation_t](#) object to decrypt data which is not in the expected input format.

The `output_size` parameter must be appropriate for the selected algorithm and key:

- A sufficient output size is [PSA_CIPHER_DECRYPT_OUTPUT_SIZE](#) where `key_type` is the type of `key`.
- [PSA_CIPHER_DECRYPT_OUTPUT_MAX_SIZE](#) evaluates to the maximum output size of any supported cipher decryption.

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE:**
`key` is not a valid key identifier.
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_DECRYPT` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
 - `alg` is not a cipher algorithm.
 - `key` is not compatible with `alg`.
 - `input_length` is not valid for the algorithm and key type. The length must be a multiple of block size.
- **PSA_ERROR_NOT_SUPPORTED:**
 - `alg` is not supported.
 - `key` is not supported for use with `alg`.
 - `input_length` is too large.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
The `output_size` is too small. [PSA_CIPHER_DECRYPT_OUTPUT_SIZE](#) or [PSA_CIPHER_DECRYPT_OUTPUT_MAX_SIZE](#) can be used to determine the required buffer size.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.7.2 Multiple Part

Symmetric encryption

The sequence of operations to encrypt a message with a symmetric cipher is as follows:

1. Allocate an operation object `psa_cipher_operation_t` which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for `psa_cipher_operation_t`, e.g. `PSA_CIPHER_OPERATION_INIT`.
3. Call `psa_cipher_encrypt_setup()` to specify the algorithm and key.
4. Call either `psa_cipher_generate_iv()` or `psa_cipher_set_iv()` to generate or set the initialization vector (IV), if the algorithm requires one. It is recommended to use `psa_cipher_generate_iv()` unless the protocol being implemented requires a specific IV value.
5. Call `psa_cipher_update()` zero, one or more times, passing a fragment of the message each time.
6. Call `psa_cipher_finish()`.

Symmetric decryption

The sequence of operations to decrypt a message with a symmetric cipher is as follows:

1. Allocate an operation object `psa_cipher_operation_t` which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for `psa_cipher_operation_t`, e.g. `PSA_CIPHER_OPERATION_INIT`.
3. Call `psa_cipher_decrypt_setup()` to specify the algorithm and key.
4. Call `psa_cipher_set_iv()` with the initialization vector (IV) for the decryption, if the algorithm requires one. This must match the IV used for the encryption.
5. Call `psa_cipher_update()` zero, one or more times, passing a fragment of the message each time.
6. Call `psa_cipher_finish()`.

6.2.3.7.2.1 typedef `psa_cipher_operation_t`

type `psa_cipher_operation_t`

The type of the state object for multi-part cipher operations.

Description

Before calling any function on a cipher operation object, the application must initialize it by any of the following means:

- Set the object to all-bits-zero, for example:

```
psa_cipher_operation_t operation;
memset(&operation, 0, sizeof(operation));
```

- Initialize the object to logical zero values by declaring the object as static or global without an explicit initializer, for example:

```
static psa_cipher_operation_t operation;
```

- Initialize the object to the initializer `PSA_CIPHER_OPERATION_INIT`, for example:

```
psa_cipher_operation_t operation = PSA_CIPHER_OPERATION_INIT;
```

- Assign the result of the function `psa_cipher_operation_init()` to the object, for example:

```
psa_cipher_operation_t operation;
```

```
operation = psa_cipher_operation_init();
```

6.2.3.7.2.2 macro PSA_CIPHER_OPERATION_INIT

PSA_CIPHER_OPERATION_INIT()

Suitable initializer for a cipher operation object of [typedef psa_cipher_operation_t](#).

6.2.3.7.2.3 psa_cipher_operation_init

[psa_cipher_operation_t](#) **psa_cipher_operation_init**(void)

Return an initial value for a cipher operation object.

Parameters

- **void** - no arguments

Return

Initialized value of cipher operation object.

6.2.3.7.2.4 psa_cipher_encrypt_setup

[psa_status_t](#) **psa_cipher_encrypt_setup**([psa_cipher_operation_t](#) *operation, [psa_key_id_t](#) key, [psa_algorithm_t](#) alg)

Set the key for a multi-part symmetric encryption operation.

Parameters

- **operation** ([psa_cipher_operation_t](#)*) - [in] The operation object to set up. It must have been initialized as per the documentation for [typedef psa_cipher_operation_t](#) and not yet in use.
- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage `PSA_KEY_USAGE_ENCRYPT`.
- **alg** ([psa_algorithm_t](#)) - [in] The cipher algorithm to compute such that [PSA_ALG_IS_CIPHER](#) is true.

Description

After a successful call to [psa_cipher_encrypt_setup\(\)](#), the application must eventually terminate the operation. The following events terminate an

operation

- A successful call to [psa_cipher_finish\(\)](#).
- A call to [psa_cipher_abort\(\)](#).

If [psa_cipher_encrypt_setup\(\)](#) returns an error, the operation object is unchanged. If a subsequent function call with an active operation returns an error, the operation enters an error state.

To abandon an active operation, or reset an operation in an error state, call [psa_cipher_abort\(\)](#).

Return

- **PSA_SUCCESS:**
Success. The operation is now active.
- **PSA_ERROR_INVALID_HANDLE:**
`key` is not a valid key identifier.
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_ENCRYPT` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
 - `alg` is not a cipher algorithm.
 - `key` is not compatible with `alg`.
- **PSA_ERROR_NOT_SUPPORTED:**
 - `alg` is not supported.
 - `key` is not supported for use with `alg`.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be inactive.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.7.2.5 psa_cipher_decrypt_setup

[psa_status_t](#) **psa_cipher_decrypt_setup**([psa_cipher_operation_t](#) *operation, [psa_key_id_t](#) key, [psa_algorithm_t](#) alg)

Set the key for a multi-part symmetric decryption operation.

Parameters

- **operation** ([psa_cipher_operation_t](#)*) - [in] The operation object to set up. It must have been initialized as per the documentation for [typedef psa_cipher_operation_t](#) and not yet in use.
- **key** ([psa_key_id_t](#)) - [in] Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage `PSA_KEY_USAGE_DECRYPT`.
- **alg** ([psa_algorithm_t](#)) - [in] The cipher algorithm to compute such that [PSA_ALG_IS_CIPHER](#) is true.

Description

After a successful call to [psa_cipher_decrypt_setup\(\)](#), the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to [psa_cipher_finish\(\)](#).
- A call to [psa_cipher_abort\(\)](#).

If [psa_cipher_decrypt_setup\(\)](#) returns an error, the operation object is unchanged. If a subsequent function call with an active operation returns an error, the operation enters an error state.

To abandon an active operation, or reset an operation in an error state, call [psa_cipher_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success. The operation is now active.

- **PSA_ERROR_INVALID_HANDLE**

- **PSA_ERROR_NOT_PERMITTED:**

The key does not have the PSA_KEY_USAGE_DECRYPT flag, or it does not permit the requested algorithm.

- **PSA_ERROR_INVALID_ARGUMENT:**

- `alg` is not a cipher algorithm.
- `key` is not compatible with `alg`.

- **PSA_ERROR_NOT_SUPPORTED:**

- `alg` is not supported.
- `key` is not supported for use with `alg`.

- **PSA_ERROR_INSUFFICIENT_MEMORY**

- **PSA_ERROR_COMMUNICATION_FAILURE**

- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_STORAGE_FAILURE**

- **PSA_ERROR_DATA_CORRUPT**

- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid: it must be inactive.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.7.2.6 psa_cipher_generate_iv

[psa_status_t](#) **psa_cipher_generate_iv**([psa_cipher_operation_t](#) *operation, [uint8_t](#) *iv, [size_t](#) iv_size, [size_t](#) *iv_length)

Generate an initialization vector (IV) for a symmetric encryption operation.

Parameters

- **operation** ([psa_cipher_operation_t](#)*) - [in] Active cipher operation.
- **iv** ([uint8_t](#)*) - [out] Buffer where the generated IV is to be written.
- **iv_size** ([size_t](#)) - [in] Size of the `iv` buffer in bytes. This must be at least [PSA_CIPHER_IV_LENGTH](#) where `key_type` and `alg` are type of key and the algorithm respectively that were used to set up the cipher operation.
- **iv_length** ([size_t](#)*) - [out] On success, the number of bytes of the generated IV.

Description

This function generates a random IV, nonce or initial counter value for the encryption operation as appropriate for the chosen algorithm, key type and key size.

The generated IV is always the default length for the key and algorithm, the [PSA_CIPHER_IV_LENGTH](#), where `key_type` is the type of `key` and `alg` is the algorithm that were used to set up the operation can be used to get the length. To generate different lengths of IV, use [psa_generate_random\(\)](#) and [psa_cipher_set_iv\(\)](#).

If the cipher algorithm does not use an IV, calling this function returns a `PSA_ERROR_BAD_STATE` error. For these algorithms, [PSA_CIPHER_IV_LENGTH](#) will be zero.

The application must call [psa_cipher_encrypt_setup\(\)](#) before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_cipher_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the `iv` buffer is too small. [PSA_CIPHER_IV_LENGTH](#) or [PSA_CIPHER_IV_MAX_SIZE](#) can be used to determine the required buffer size.

- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`

- **PSA_ERROR_DATA_INVALID**

- The cipher algorithm does not use an IV.
- The operation state is not valid: it must be active, with no IV set.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.7.2.7 psa_cipher_set_iv

[psa_status_t](#) [psa_cipher_set_iv](#)([psa_cipher_operation_t](#) *operation, const uint8_t *iv, size_t iv_length)

Set the initialization vector (IV) for a symmetric encryption or decryption operation.

Parameters

- **operation** ([psa_cipher_operation_t](#)*) - [in] Active cipher operation.
- **iv** (const uint8_t*) - [in] Buffer containing the IV to use.
- **iv_length** (size_t) - [in] Size of the IV in bytes.

Description

This function sets the IV, nonce or initial counter value for the encryption or decryption operation.

If the cipher algorithm does not use an IV, calling this function returns a `PSA_ERROR_BAD_STATE` error. For these algorithms, [PSA_CIPHER_IV_LENGTH](#) will be zero.

The application must call [psa_cipher_encrypt_setup\(\)](#) or [psa_cipher_decrypt_setup\(\)](#) before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_cipher_abort\(\)](#).

Note:

When encrypting, `psa_cipher_generate_iv()` is recommended instead of using this function, unless implementing a protocol that requires a non-random IV.

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_INVALID_ARGUMENT:**

- The chosen algorithm does not use an IV.
- `iv_length` is not valid for the chosen algorithm.

- **PSA_ERROR_NOT_SUPPORTED:**

The `iv_length` is not supported for use with the operation's algorithm and key.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

- The cipher algorithm does not use an IV.
- The operation state is not valid: it must be an active cipher encrypt operation, with no IV set.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.7.2.8 psa_cipher_update

[psa_status_t](#) **psa_cipher_update**([psa_cipher_operation_t](#) *operation, const uint8_t *input, size_t input_length, uint8_t *output, size_t output_size, size_t *output_length)

Encrypt or decrypt a message fragment in an active cipher operation.

Parameters

- **operation** ([psa_cipher_operation_t](#)*) - [in] Active cipher operation.
- **input** (const uint8_t*) - [in] Buffer containing the message fragment to encrypt or decrypt.
- **input_length** (size_t) - [in] Size of the `input` buffer in bytes.
- **output** (uint8_t*) - [out] Buffer where the output is to be written.
- **output_size** (size_t) - [in] Size of the `output` buffer in bytes.
- **output_length** (size_t*) - [out] On success, the number of bytes that make up the returned output.

Description

The `output_size` parameter must be appropriate for the selected algorithm and key:

- A sufficient output size is [PSA_CIPHER_UPDATE_OUTPUT_SIZE](#) where `key_type` is the type of `key` and `alg` is the algorithm that were used to set up the operation.
- [PSA_CIPHER_UPDATE_OUTPUT_MAX_SIZE](#) evaluates to the maximum output size of any supported cipher algorithm.

The following must occur before calling this function:

1. Call either [psa_cipher_encrypt_setup\(\)](#) or [psa_cipher_decrypt_setup\(\)](#). The choice of setup function determines whether this function encrypts or decrypts its input.
2. If the algorithm requires an IV, call [psa_cipher_generate_iv\(\)](#) or [psa_cipher_set_iv\(\)](#). [psa_cipher_generate_iv\(\)](#) is recommended when encrypting.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_cipher_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the `output` buffer is too small. [PSA_CIPHER_UPDATE_OUTPUT_SIZE](#) or [PSA_CIPHER_UPDATE_OUTPUT_MAX_SIZE](#) can be used to determine the required buffer size.

- **PSA_ERROR_INVALID_ARGUMENT:**

The total input size passed to this operation is too large for this particular algorithm.

- **PSA_ERROR_NOT_SUPPORTED:**

The total input size passed to this operation is too large.

- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid: it must be active, with an IV set if required for the algorithm.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.7.2.9 psa_cipher_finish

[psa_status_t](#) [psa_cipher_finish](#)([psa_cipher_operation_t](#) *operation, [uint8_t](#) *output, [size_t](#) output_size, [size_t](#) *output_length)

Finish encrypting or decrypting a message in a cipher operation.

Parameters

- **operation** ([psa_cipher_operation_t](#)*) - [in] Active cipher operation.
- **output** ([uint8_t](#)*) - [out] Buffer where the output is to be written.
- **output_size** ([size_t](#)) - [in] Size of the `output` buffer in bytes.
- **output_length** ([size_t](#)*) - [out] On success, the number of bytes that make up the returned output.

Description

The `output_size` parameter must be appropriate for the selected algorithm and key:

- A sufficient output size is [PSA_CIPHER_FINISH_OUTPUT_SIZE](#) where `key_type` is the type of key and `alg` is the algorithm that were used to set up the operation.

- [PSA_CIPHER_FINISH_OUTPUT_MAX_SIZE](#) evaluates to the maximum output size of any supported cipher algorithm.

The application must call [psa_cipher_encrypt_setup\(\)](#) or [psa_cipher_decrypt_setup\(\)](#) before calling this function. The choice of setup function determines whether this function encrypts or decrypts its input.

This function finishes the encryption or decryption of the message formed by concatenating the inputs passed to preceding calls to [psa_cipher_update\(\)](#).

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_cipher_abort\(\)](#).

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_ARGUMENT:**
The total input size passed to this operation is not valid for this particular algorithm. The total input size is not a multiple of the block size.
- **PSA_ERROR_INVALID_PADDING:**
This is a decryption operation for an algorithm that includes padding, and the ciphertext does not contain valid padding.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
The size of the output buffer is too small. [PSA_CIPHER_FINISH_OUTPUT_SIZE](#) or [PSA_CIPHER_FINISH_OUTPUT_MAX_SIZE](#) can be used to determine the required buffer size.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be active, with an IV set if required for the algorithm.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.7.2.10 psa_cipher_abort

[psa_status_t](#) **psa_cipher_abort**([psa_cipher_operation_t](#) *operation)

Abort a cipher operation.

Parameters

- **operation** ([psa_cipher_operation_t](#)*) - [in] Initialized cipher operation.

Description

Aborting an operation frees all associated resources except for the `operation` object itself. Once aborted, the operation object can be reused for another operation by calling [psa_cipher_encrypt_setup\(\)](#) or [psa_cipher_decrypt_setup\(\)](#) again.

This function can be called any time after the operation object has been initialized as described in [typedef `psa\_cipher\_operation\_t`](#).

In particular, calling [`psa\_cipher\_abort\(\)`](#) after the operation has been terminated by a call to [`psa\_cipher\_abort\(\)`](#) or [`psa\_cipher\_finish\(\)`](#) is safe and has no effect.

Return

- **PSA_SUCCESS:**

Success. The operation object can now be discarded or reused.

- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [`psa\_crypto\_init\(\)`](#).

6.2.3.7.3 Macros

6.2.3.7.3.1 macro `PSA_CIPHER_DECRYPT_OUTPUT_SIZE`

PSA_CIPHER_DECRYPT_OUTPUT_SIZE(*key_type*, *alg*, *input_length*)

A sufficient output buffer size for [`psa\_cipher\_decrypt\(\)`](#), in bytes.

Parameters

- **key_type** - [in] A symmetric key type that is compatible with algorithm *alg*.
- **alg** - [in] A cipher algorithm such that [`PSA\_ALG\_IS\_CIPHER`](#) is true.
- **input_length** - [in] Size of the input in bytes.

Description

If the size of the output buffer is at least this large, it is guaranteed that [`psa\_cipher\_decrypt\(\)`](#) will not fail due to an insufficient buffer size. Depending on the algorithm, the actual size of the output might be smaller.

See also [`PSA\_CIPHER\_DECRYPT\_OUTPUT\_MAX\_SIZE`](#).

Return

A sufficient output size for the specified key type and algorithm. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0.

6.2.3.7.3.2 macro `PSA_CIPHER_DECRYPT_OUTPUT_MAX_SIZE`

PSA_CIPHER_DECRYPT_OUTPUT_MAX_SIZE(*input_length*)

The maximum size of the output of [`psa\_cipher\_decrypt\(\)`](#), for any of the supported key types and cipher algorithms.

Parameters

- **input_length** - [in] Size of the input in bytes.

Description

If the size of the output buffer is at least this large, it is guaranteed that [psa_cipher_decrypt\(\)](#) will not fail due to an insufficient buffer size.

See also [PSA_CIPHER_DECRYPT_OUTPUT_SIZE](#).

6.2.3.7.3.3 macro PSA_CIPHER_ENCRYPT_OUTPUT_SIZE

PSA_CIPHER_ENCRYPT_OUTPUT_SIZE(*key_type*, *alg*, *input_length*)

A sufficient output buffer size for [psa_cipher_encrypt\(\)](#), for any of the supported key types and cipher algorithms.

Parameters

- **key_type** - [in] A symmetric key type that is compatible with algorithm *alg*.
- **alg** - [in] A cipher algorithm such that [PSA_ALG_IS_CIPHER](#) is true.
- **input_length** - [in] Size of the input in bytes.

Description

If the size of the output buffer is at least this large, it is guaranteed that [psa_cipher_encrypt\(\)](#) will not fail due to an insufficient buffer size. Depending on the algorithm, the actual size of the output might be smaller.

See also [PSA_CIPHER_ENCRYPT_OUTPUT_MAX_SIZE](#).

Return

A sufficient output size for the specified key type and algorithm. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0.

6.2.3.7.3.4 macro PSA_CIPHER_ENCRYPT_OUTPUT_MAX_SIZE

PSA_CIPHER_ENCRYPT_OUTPUT_MAX_SIZE(*input_length*)

The maximum size of the output of [psa_cipher_encrypt\(\)](#), for any of the supported key types and cipher algorithms.

Parameters

- **input_length** - [in] Size of the input in bytes.

Description

If the size of the output buffer is at least this large, it is guaranteed that [psa_cipher_encrypt\(\)](#) will not fail due to an insufficient buffer size.

See also [PSA_CIPHER_ENCRYPT_OUTPUT_SIZE](#).

6.2.3.7.3.5 macro PSA_CIPHER_UPDATE_OUTPUT_SIZE

PSA_CIPHER_UPDATE_OUTPUT_SIZE(*key_type*, *alg*, *input_length*)

A sufficient output buffer size for [psa_cipher_update\(\)](#), in bytes.

Parameters

- **key_type** - [in] A symmetric key type that is compatible with algorithm `alg`.
- **alg** - [in] A cipher algorithm such that [PSA_ALG_IS_CIPHER](#) is true.
- **input_length** - [in] Size of the input in bytes.

Description

If the size of the output buffer is at least this large, it is guaranteed that [psa_cipher_update\(\)](#) will not fail due to an insufficient buffer size. The actual size of the output might be smaller in any given call.

See also [PSA_CIPHER_UPDATE_OUTPUT_MAX_SIZE](#).

Return

A sufficient output size for the specified key type and algorithm. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0.

6.2.3.7.3.6 macro [PSA_CIPHER_UPDATE_OUTPUT_MAX_SIZE](#)

PSA_CIPHER_UPDATE_OUTPUT_MAX_SIZE(*input_length*)

The maximum output buffer size for [psa_cipher_update\(\)](#), for any of the supported key types and cipher algorithms.

Parameters

- **input_length** - [in] Size of the input in bytes.

Description

If the size of the output buffer is at least this large, it is guaranteed that [psa_cipher_update\(\)](#) will not fail due to an insufficient buffer size.

See also [PSA_CIPHER_UPDATE_OUTPUT_SIZE](#).

6.2.3.7.3.7 macro [PSA_CIPHER_FINISH_OUTPUT_SIZE](#)

PSA_CIPHER_FINISH_OUTPUT_SIZE(*key_type, alg*)

A sufficient ciphertext buffer size for [psa_cipher_finish\(\)](#), in bytes.

Parameters

- **key_type** - [in] A symmetric key type that is compatible with algorithm `alg`.
- **alg** - [in] A cipher algorithm such that [PSA_ALG_IS_CIPHER](#) is true.

Description

If the size of the output buffer is at least this large, it is guaranteed that [psa_cipher_finish\(\)](#) will not fail due to an insufficient buffer size. The actual size of the output might be smaller in any given call.

See also [PSA_CIPHER_FINISH_OUTPUT_MAX_SIZE](#).

Return

A sufficient output size for the specified key type and algorithm. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0.

6.2.3.7.3.8 macro PSA_CIPHER_FINISH_OUTPUT_MAX_SIZE

PSA_CIPHER_FINISH_OUTPUT_MAX_SIZE()

The maximum output buffer size for [psa_cipher_finish\(\)](#), for any of the supported key types and cipher algorithms.

Description

See also [PSA_CIPHER_FINISH_OUTPUT_SIZE](#).

6.2.3.7.3.9 macro PSA_CIPHER_IV_LENGTH

PSA_CIPHER_IV_LENGTH(*key_type*, *alg*)

The default IV size for a cipher algorithm, in bytes.

Parameters

- **key_type** - [in] A symmetric key type that is compatible with algorithm *alg*.
- **alg** - [in] A cipher algorithm such that [PSA_ALG_IS_CIPHER](#) is true.

Description

The IV that is generated as part of a call to [psa_cipher_encrypt\(\)](#) is always the default IV length for the algorithm.

This macro can be used to allocate a buffer of sufficient size to store the IV output from [psa_cipher_generate_iv\(\)](#) when using a multi-part cipher operation.

See also [PSA_CIPHER_IV_MAX_SIZE](#).

Return

The default IV size for the specified key type and algorithm. If the algorithm does not use an IV, return 0. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0.

6.2.3.7.3.10 macro PSA_CIPHER_IV_MAX_SIZE

PSA_CIPHER_IV_MAX_SIZE()

The maximum IV size for all supported cipher algorithms, in bytes.

Description

See also [PSA_CIPHER_IV_LENGTH](#).

6.2.3.7.3.11 macro PSA_BLOCK_CIPHER_BLOCK_LENGTH

PSA_BLOCK_CIPHER_BLOCK_LENGTH(*type*)

The block size of a block cipher.

Parameters

- **type** - [in] A cipher key type (value of [typedef psa_key_type_t](#)).

Description

Note:

It is possible to build stream cipher algorithms on top of a block cipher, for example CTR mode (PSA_ALG_CTR). This macro only takes the key type into account, so it cannot be used to determine the size of the data that `psa_cipher_update()` might buffer for future processing in general.

Note:

This macro expression is a compile-time constant if `@type` is a compile-time constant.

See also [PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE](#).

Return

The block size for a block cipher, or 1 for a stream cipher. The return value is undefined if `type` is not a supported cipher key type.

6.2.3.7.3.12 macro PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE

PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE()

The maximum size of a block cipher supported.

Description

See also [PSA_BLOCK_CIPHER_BLOCK_LENGTH](#).

6.2.3.8 Password-Authenticated Key Exchange (PAKE)

6.2.3.8.1 PAKE primitives

A PAKE algorithm specifies a sequence of interactions between the participants. Many PAKE algorithms are designed to allow different cryptographic primitives to be used for the key establishment operation, so long as all the participants are using the same underlying cryptography.

The cryptographic primitive for a PAKE operation is specified using a [psa_pake_primitive_t](#) value, which can be constructed using the [PSA_PAKE_PRIMITIVE](#) macro, or can be provided as a numerical constant value.

A PAKE primitive is required when constructing a PAKE cipher-suite object, [psa_pake_cipher_suite_t](#), which fully specifies the PAKE operation to be carried out.

6.2.3.8.1.1 typedef psa_pake_primitive_t

type **psa_pake_primitive_t**

Encoding of the primitive associated with the PAKE.

PAKE primitive values are constructed using [PSA_PAKE_PRIMITIVE](#).

Table 156. PAKE primitive encoding

Field	Bits
PAKE-TYPE	[31:24]
PAKE-FAMILY	[23:16]

Table 156. PAKE primitive encoding...continued

Field	Bits
PAKE-BITS	[15:0]

The components of a PAKE primitive value can be extracted using the [PSA_PAKE_PRIMITIVE_GET_TYPE](#), [PSA_PAKE_PRIMITIVE_GET_FAMILY](#), and [PSA_PAKE_PRIMITIVE_GET_BITS](#). These can be used to set key attributes for keys used in PAKE algorithms. SPAKE2+ registration provides an example of this usage.

6.2.3.8.1.2 typedef psa_pake_primitive_type_t

type **psa_pake_primitive_type_t**

Encoding of the primitive associated with the PAKE.

6.2.3.8.1.3 typedef psa_pake_family_t

type **psa_pake_family_t**

Encoding of the family of the primitive associated with the PAKE.

The following [PAKE primitive type and family](#) table lists all PAKE primitive type and associated family.

Table 157. PAKE primitive type and family

PAKE-TYPE		PAKE-FAMILY	Description
Name	Value		
PSA_PAKE_PRIMITIVE_TYPE_ECC	0x01	psa_ecc_family_t	Elliptic curve primitive
PSA_PAKE_PRIMITIVE_TYPE_DH	0x02	psa_dh_family_t	Diffie-Hellman primitive

6.2.3.8.1.4 macro PSA_PAKE_PRIMITIVE_TYPE_ECC

PSA_PAKE_PRIMITIVE_TYPE_ECC()

The PAKE primitive type indicating the use of elliptic curves.

Description

The values of the family and bits components of the PAKE primitive identify a specific elliptic curve, using the same mapping that is used for ECC keys. See the definition of [typedef psa_ecc_family_t](#).

Here family and bits refer to the values used to construct the PAKE primitive using [PSA_PAKE_PRIMITIVE](#).

Input and output during the operation can involve group elements and scalar values:

- The format for group elements is the same as that for public keys on the specific elliptic curve. See Key format within the definition of [PSA_KEY_TYPE_ECC_PUBLIC_KEY](#).
- The format for scalars is the same as that for private keys on the specific elliptic curve. See Key format within the definition of [PSA_KEY_TYPE_ECC_KEY_PAIR](#).

6.2.3.8.1.5 macro PSA_PAKE_PRIMITIVE_TYPE_DH

PSA_PAKE_PRIMITIVE_TYPE_DH()

The PAKE primitive type indicating the use of Diffie-Hellman groups.

Description

The values of the family and bits components of the PAKE primitive identify a specific Diffie-Hellman group, using the same mapping that is used for Diffie-Hellman keys. See the definition of [typedef psa_dh_family_t](#).

Here family and bits refer to the values used to construct the PAKE primitive using [PSA_PAKE_PRIMITIVE](#).

Input and output during the operation can involve group elements and scalar values:

- The format for group elements is the same as that for public keys in the specific Diffie-Hellman group. See Key format within the definition of [PSA_KEY_TYPE_DH_PUBLIC_KEY](#).
- The format for scalars is the same as that for private keys in the specific Diffie-Hellman group. See Key format within the definition of [PSA_KEY_TYPE_DH_PUBLIC_KEY](#).

6.2.3.8.1.6 macro PSA_PAKE_PRIMITIVE

PSA_PAKE_PRIMITIVE(*pake_type*, *pake_family*, *pake_bits*)

Construct a PAKE primitive from type, family and bit-size.

Parameters

- **pake_type** - [in] The type of the primitive: a value of [typedef psa_pake_primitive_type_t](#).
- **pake_family** - [in] The family of the primitive. The type and interpretation of this parameter depends on *pake_type*.
- **pake_bits** - [in] The bit-size of the primitive: a value of type *size_t*. The interpretation of this parameter depends on *pake_type* and *pake_family*.

Description

Warning:

Not supported.

A PAKE primitive value is used to specify a PAKE operation, as part of a PAKE cipher suite.

Return

The constructed primitive value. Return 0 if the requested primitive can't be encoded as [typedef psa_pake_primitive_t](#).

6.2.3.8.1.7 macro PSA_PAKE_PRIMITIVE_GET_TYPE

PSA_PAKE_PRIMITIVE_GET_TYPE(*pake_primitive*)

Extract the PAKE primitive type from a PAKE primitive.

Parameters

- **pake_primitive** - A PAKE primitive: a value of [typedef psa_pake_primitive_t](#).

Description

Warning:

Not supported.

Return

The PAKE primitive type, if `pake_primitive` is a supported PAKE primitive type.

Unspecified if `pake_primitive` is not a supported PAKE primitive type.

6.2.3.8.1.8 macro PSA_PAKE_PRIMITIVE_GET_FAMILY

PSA_PAKE_PRIMITIVE_GET_FAMILY(*pake_primitive*)

Extract the family from a PAKE primitive.

Parameters

- **pake_primitive** - A PAKE primitive: a value of [typedef psa_pake_primitive_t](#).

Description

Warning:

Not supported.

For more information on the family values, see [PSA_PAKE_PRIMITIVE_TYPE_ECC](#) and [PSA_PAKE_PRIMITIVE_TYPE_DH](#).

Return

The PAKE primitive family, if `pake_primitive` is a supported PAKE primitive family.

Unspecified if `pake_primitive` is not a supported PAKE primitive family.

6.2.3.8.1.9 macro PSA_PAKE_PRIMITIVE_GET_BITS

PSA_PAKE_PRIMITIVE_GET_BITS(*pake_primitive*)

Extract the bit-size from a PAKE primitive.

Parameters

- **pake_primitive** - A PAKE primitive: a value of type `psa_pake_primitive_t`.

Description

Warning:

Not supported.

For more information on the bit-size values, see [PSA_PAKE_PRIMITIVE_TYPE_ECC](#) and [PSA_PAKE_PRIMITIVE_TYPE_DH](#).

Return

The PAKE primitive bit-size, if `pake_primitive` is a supported PAKE primitive.

Unspecified if `pake_primitive` is not a supported PAKE primitive.

6.2.3.8.2 PAKE Cipher suites

Most PAKE algorithms have parameters that must be specified by the application. These parameters include the following:

- The cryptographic primitive used for key establishment, specified using a [PAKE primitives](#).
- A cryptographic hash algorithm.
- Whether the application requires the shared secret before, or after, it is confirmed.

The hash algorithm is encoded into the PAKE algorithm identifier. The [psa_pake_cipher_suite_t](#) object is used to fully specify a PAKE operation, combining the PAKE protocol with all of the above parameters.

A PAKE cipher suite is required when setting up a PAKE operation in [psa_pake_setup\(\)](#).

6.2.3.8.2.1 typedef psa_pake_cipher_suite_t

type **psa_pake_cipher_suite_t**

The type of an object describing a PAKE cipher suite.

Description

Before calling any function on a PAKE cipher suite object, the application must initialize it by any of the following means:

- Set the object to all-bits-zero, for example:

```
psa_pake_cipher_suite_t cipher_suite;  
memset(&cipher_suite, 0, sizeof(cipher_suite));
```

- Initialize the object to logical zero values by declaring the object as static or global without an explicit initializer, for example:

```
static psa_pake_cipher_suite_t cipher_suite;
```

- Initialize the object to the initializer [PSA_PAKE_CIPHER_SUITE_INIT](#), for example:

```
psa_pake_cipher_suite_t cipher_suite = PSA_PAKE_CIPHER_SUITE_INIT;
```

- Assign the result of the function [psa_pake_cipher_suite_init\(\)](#) to the object, for example:

```
psa_pake_cipher_suite_t cipher_suite;  
cipher_suite = psa_pake_cipher_suite_init();
```

Following initialization, the cipher-suite object contains the following values:

Table 158. PAKE cipher-suite object initialization

Attribute	Value
algorithm	PSA_ALG_NONE — an invalid algorithm identifier.
primitive	0 — an invalid PAKE primitive.
key confirmation	PSA_PAKE_CONFIRMED_KEY — requesting that the secret key is confirmed before it can be returned.

Valid algorithm, primitive, and key confirmation values must be set when using a PAKE cipher suite.

6.2.3.8.2.2 macro PSA_PAKE_CIPHER_SUITE_INIT

PSA_PAKE_CIPHER_SUITE_INIT()

Suitable initializer for a PAKE cipher suite object of [typedef psa_pake_cipher_suite_t](#).

6.2.3.8.2.3 psa_pake_cipher_suite_init

[psa_pake_cipher_suite_t](#) **psa_pake_cipher_suite_init**(void)

Return an initial value for a PAKE cipher suite object.

Parameters

- **void** - no arguments

Return

Initialized value of PAKE cipher suite object.

6.2.3.8.2.4 psa_pake_cs_get_algorithm

[psa_algorithm_t](#) **psa_pake_cs_get_algorithm**(const [psa_pake_cipher_suite_t](#) *cipher_suite)

Retrieve the PAKE algorithm from a PAKE cipher suite.

Parameters

- **cipher_suite** (const [psa_pake_cipher_suite_t](#) *) - [in] The cipher suite object to query.

Description

Warning:

Not supported.

Return

The PAKE algorithm stored in the cipher suite object.

6.2.3.8.2.5 psa_pake_cs_set_algorithm

void **psa_pake_cs_set_algorithm**([psa_pake_cipher_suite_t](#) *cipher_suite, [psa_algorithm_t](#) alg)

Declare the PAKE algorithm for the cipher suite.

Parameters

- **cipher_suite** ([psa_pake_cipher_suite_t](#) *) - [in/out] The cipher suite object to write to.
- **alg** ([psa_algorithm_t](#)) - [in] The PAKE algorithm to write such that [PSA_ALG_IS_PAKE](#) is true.

Description

Warning:

Not supported.

This function overwrites any PAKE algorithm previously set in `cipher_suite`.

Return

void

6.2.3.8.2.6 psa_pake_cs_get_primitive

[psa_pake_primitive_t](#) `psa_pake_cs_get_primitive(const psa\_pake\_cipher\_suite\_t *cipher_suite)`

Retrieve the primitive from a PAKE cipher suite.

Parameters

- `cipher_suite` (const [psa_pake_cipher_suite_t](#)*) - [in] The cipher suite object to query.

Description

Warning:

Not supported.

Return

The primitive stored in the cipher suite object.

6.2.3.8.2.7 psa_pake_cs_set_primitive

void `psa_pake_cs_set_primitive(psa\_pake\_cipher\_suite\_t *cipher_suite, psa\_pake\_primitive\_t primitive)`

Declare the primitive for a PAKE cipher suite.

Parameters

- `cipher_suite` ([psa_pake_cipher_suite_t](#)*) - [in/out] The cipher suite object to write to.
- `primitive` ([psa_pake_primitive_t](#)) - [in] The PAKE primitive to write: a value of [typedef psa_pake_primitive_t](#). If this is 0, the primitive type in `cipher_suite` becomes unspecified.

Description

Warning:

Not supported.

This function overwrites any primitive previously set in `cipher_suite`.

Return

void

6.2.3.8.2.8 psa_pake_cs_get_key_confirmation

[psa_pake_primitive_t](#) `psa_pake_cs_get_key_confirmation(const psa\_pake\_cipher\_suite\_t *cipher_suite)`

Retrieve the key confirmation from a PAKE cipher suite.

Parameters

- **cipher_suite** (const [psa_pake_cipher_suite_t](#)*) - [in] The cipher suite object to query.

Description

Warning:

Not supported.

Return

A key confirmation value: either [PSA_PAKE_CONFIRMED_KEY](#) or [PSA_PAKE_UNCONFIRMED_KEY](#).

6.2.3.8.2.9 psa_pake_cs_set_key_confirmation

void **psa_pake_cs_set_key_confirmation**([psa_pake_cipher_suite_t](#) *cipher_suite, uint32_t key_confirmation)

Declare the key confirmation from a PAKE cipher suite.

Parameters

- **cipher_suite** ([psa_pake_cipher_suite_t](#)*) - [in/out] The cipher suite object to write to.
- **key_confirmation** (uint32_t) - [in] The key confirmation value to write: either [PSA_PAKE_CONFIRMED_KEY](#) or [PSA_PAKE_UNCONFIRMED_KEY](#).

Description

Warning:

Not supported.

This function overwrites any key confirmation previously set in `cipher_suite`.

The documentation of individual PAKE algorithms specifies which key confirmation values are valid for the algorithm.

Return

void

6.2.3.8.2.10 macro PSA_PAKE_CONFIRMED_KEY

PSA_PAKE_CONFIRMED_KEY()

A key confirmation value that indicates an confirmed key in a PAKE cipher suite.

Description

This key confirmation value will result in the PAKE algorithm exchanging data to verify that the shared key is identical for both parties. This is the default key confirmation value in an initialized PAKE cipher suite object.

Some algorithms do not include confirmation of the shared key.

6.2.3.8.2.11 macro PSA_PAKE_UNCONFIRMED_KEY

PSA_PAKE_UNCONFIRMED_KEY()

A key confirmation value that indicates an unconfirmed key in a PAKE cipher suite.

Description

This key confirmation value will result in the PAKE algorithm terminating prior to confirming that the resulting shared key is identical for both parties.

Some algorithms do not support returning an unconfirmed shared key.

Warning:

When the shared key is not confirmed as part of the PAKE operation, the application is responsible for mitigating risks that arise from the possible mismatch in the output keys.

6.2.3.8.3 PAKE roles

Some PAKE algorithms need to know which role each participant is taking in the algorithm. For example:

- Augmented PAKE algorithms typically have a client and a server participant.
- Some symmetric PAKE algorithms assign an order to the two participants.

6.2.3.8.3.1 typedef psa_pake_role_t

type **psa_pake_role_t**

Encoding of the application role in a PAKE algorithm.

Description

This type is used to encode the application's role in the algorithm being executed. For more information see the documentation of individual PAKE role constants

6.2.3.8.3.2 macro PSA_PAKE_ROLE_NONE

PSA_PAKE_ROLE_NONE()

A value to indicate no role in a PAKE algorithm.

Description

This value can be used in a call to [psa_pake_set_role\(\)](#) for symmetric PAKE algorithms which do not assign roles.

6.2.3.8.3.3 macro PSA_PAKE_ROLE_FIRST

PSA_PAKE_ROLE_FIRST()

The first peer in a balanced PAKE.

Description

Although balanced PAKE algorithms are symmetric, some of them need the peers to be ordered for the transcript calculations. If the algorithm does not need a specific ordering, then either do not call [psa_pake_set_role\(\)](#), or use [PSA_PAKE_ROLE_NONE](#) as the role parameter.

6.2.3.8.3.4 macro PSA_PAKE_ROLE_SECOND

PSA_PAKE_ROLE_SECOND()

The second peer in a balanced PAKE.

Description

Although balanced PAKE algorithms are symmetric, some of them need the peers to be ordered for the transcript calculations. If the algorithm does not need a specific ordering, then either do not call [psa_pake_set_role\(\)](#), or use [PSA_PAKE_ROLE_NONE](#) as the role parameter.

6.2.3.8.3.5 macro PSA_PAKE_ROLE_CLIENT

PSA_PAKE_ROLE_CLIENT()

The client in an augmented PAKE.

Description

Augmented PAKE algorithms need to differentiate between client and server.

6.2.3.8.3.6 macro PSA_PAKE_ROLE_SERVER

PSA_PAKE_ROLE_SERVER()

The server in an augmented PAKE.

Description

Augmented PAKE algorithms need to differentiate between client and server.

6.2.3.8.4 PAKE step types

6.2.3.8.4.1 typedef psa_pake_step_t

type **psa_pake_step_t**

Encoding of input and output steps for a PAKE algorithm.

Description

Some PAKE algorithms need to exchange more data than a single key share. This type encodes additional input and output steps for such algorithms.

6.2.3.8.4.2 macro `PSA_PAKE_STEP_KEY_SHARE`

`PSA_PAKE_STEP_KEY_SHARE()`

The key share being sent to or received from the peer.

Description

The format for both input and output using this step is the same as the format for public keys on the group specified by the PAKE operation's primitive.

The public-key formats are defined in the documentation for [psa_export_public_key\(\)](#).

For information regarding how the group is determined, consult the documentation [PSA_PAKE_PRIMITIVE](#).

6.2.3.8.4.3 macro `PSA_PAKE_STEP_ZK_PUBLIC`

`PSA_PAKE_STEP_ZK_PUBLIC()`

A Schnorr NIZKP public key.

Description

This is the ephemeral public key in the Schnorr Non-Interactive Zero-Knowledge Proof, this is the value denoted by V in [\[RFC 8235\]](#).

The format for both input and output at this step is the same as that for public keys on the group specified by the PAKE operation's primitive.

For more information on the format, consult the documentation of [psa_export_public_key\(\)](#).

For information regarding how the group is determined, consult the documentation [PSA_PAKE_PRIMITIVE](#).

6.2.3.8.4.4 macro `PSA_PAKE_STEP_ZK_PROOF`

`PSA_PAKE_STEP_ZK_PROOF()`

A Schnorr NIZKP proof.

Description

This is the proof in the Schnorr Non-Interactive Zero-Knowledge Proof, this is the value denoted by r in [\[RFC 8235\]](#).

Both for input and output, the value at this step is an integer less than the order of the group specified by the PAKE operation's primitive. The format depends on the group as well:

- For Montgomery curves, the encoding is little endian.
- For other elliptic curves, and for Diffie-Hellman groups, the encoding is big endian. See [\[SEC1\]](#) §2.3.8.

In both cases leading zeroes are permitted as long as the length in bytes does not exceed the byte length of the group order.

For information regarding how the group is determined, consult the documentation [PSA_PAKE_PRIMITIVE](#).

6.2.3.8.4.5 macro `PSA_PAKE_STEP_CONFIRM`

`PSA_PAKE_STEP_CONFIRM()`

The key confirmation value.

Description

This value is used during the key confirmation phase of a PAKE protocol. The format of the value depends on the algorithm and cipher suite:

- For PSA_ALG_SPAKE2P, the format for both input and output at this step is the same as the output of the MAC algorithm specified in the cipher suite.

6.2.3.8.5 Multiple Part

Warning:

The PAKE multiple part sequence is not supported in the current implementation.

The sequence of operations to set up a password-authenticated key exchange operation is as follows:

1. Allocate a PAKE operation object which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for `&typedef psa_pake_operation_t`. For example, using [PSA_PAKE_OPERATION_INIT](#).
3. Call [psa_pake_setup\(\)](#) to specify the cipher suite.
4. Call `psa_pake_set_xxx()` functions on the operation to complete the setup. The exact sequence of `psa_pake_set_xxx()` functions that needs to be called depends on the algorithm in use.

A typical sequence of calls to perform a password-authenticated key exchange:

1. Call [psa_pake_output\(\)](#) with `step = PSA_PAKE_STEP_KEY_SHARE` to get the key share that needs to be sent to the peer.
2. Call [psa_pake_input\(\)](#) with `step = PSA_PAKE_STEP_KEY_SHARE` to provide the key share that was received from the peer.
3. Depending on the algorithm additional calls to [psa_pake_output\(\)](#) and [psa_pake_input\(\)](#) might be necessary.
4. Call [psa_pake_get_shared_key\(\)](#) to access the shared secret.

6.2.3.8.5.1 typedef psa_pake_operation_t

type `psa_pake_operation_t`

The type of the state object for PAKE operations.

Description

Before calling any function on a PAKE operation object, the application must initialize it by any of the following means:

- Set the object to all-bits-zero, for example:

```
psa_pake_operation_t operation;
memset(&operation, 0, sizeof(operation));
```

- Initialize the object to logical zero values by declaring the object as static or global without an explicit initializer, for example:

```
static psa_pake_operation_t operation;
```

- Initialize the object to the initializer [PSA_PAKE_OPERATION_INIT](#), for example:

```
psa_pake_operation_t operation = PSA_PAKE_OPERATION_INIT;
```

- Assign the result of the function [psa_pake_operation_init\(\)](#) to the object, for example:

```
psa_pake_operation_t operation;  
operation = psa_pake_operation_init();
```

6.2.3.8.5.2 macro PSA_PAKE_OPERATION_INIT

PSA_PAKE_OPERATION_INIT()

Suitable initializer for a PAKE operation object of [typedef psa_pake_operation_t](#).

6.2.3.8.5.3 psa_pake_operation_init

[psa_pake_operation_t](#) [psa_pake_operation_init\(void\)](#)

Return an initial value for a PAKE operation object.

Parameters

- void** - no arguments

Return

Initialized value of PAKE operation object.

6.2.3.8.5.4 psa_pake_setup

[psa_status_t](#) [psa_pake_setup](#)([psa_pake_operation_t](#) *operation, [psa_key_id_t](#) password_key, const [psa_pake_cipher_suite_t](#) *cipher_suite)

Setup a password-authenticated key exchange.

Parameters

- operation** ([psa_pake_operation_t](#)*) - [in] The operation object to set up. It must have been initialized as per the documentation for [typedef psa_pake_operation_t](#) and not yet in use.
- password_key** ([psa_key_id_t](#)) - [in] Identifier of the key holding the password or a value derived from the password. It must remain valid until the operation terminates. The key must permit the usage PSA_KEY_USAGE_DERIVE.
- cipher_suite** (const [psa_pake_cipher_suite_t](#)*) - [in] The cipher suite to use. A PAKE cipher suite fully characterizes a PAKE algorithm, including the PAKE algorithm.

Description

Warning:

Not supported.

After a successful call to [psa_pake_setup\(\)](#), the operation is active, and the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to [psa_pake_get_shared_key\(\)](#).

- A call to [psa_pake_abort\(\)](#).

If [psa_pake_setup\(\)](#) returns an error, the operation object is unchanged. If a subsequent function call with an active operation returns an error, the operation enters an error state.

To abandon an active operation, or reset an operation in an error state, call [psa_pake_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success. The operation is now active.

- **PSA_ERROR_INVALID_HANDLE:**

`password_key` is not a valid key identifier.

- **PSA_ERROR_NOT_PERMITTED:**

`password_key` does not have the `PSA_KEY_USAGE_DERIVE` flag, or it does not permit the algorithm in `cipher_suite`.

- **PSA_ERROR_INVALID_ARGUMENT:**

- The algorithm in `cipher_suite` is not a PAKE algorithm, or encodes an invalid hash algorithm.
- The PAKE primitive in `cipher_suite` is not compatible with the PAKE algorithm.
- The key confirmation value in `cipher_suite` is not compatible with the PAKE algorithm and primitive.
- The key type or key size of `password_key` is not compatible with `cipher_suite`.

- **PSA_ERROR_NOT_SUPPORTED:**

- The algorithm in `cipher_suite` is not a supported PAKE algorithm, or encodes an unsupported hash algorithm.
- The PAKE primitive in `cipher_suite` is not supported or not compatible with the PAKE algorithm.
- The key confirmation value in `cipher_suite` is not supported, or not compatible, with the PAKE algorithm and primitive.
- The key type or key size of `password_key` is not supported with `cipher suite`.

- **PSA_ERROR_COMMUNICATION_FAILURE**

- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_STORAGE_FAILURE**

- **PSA_ERROR_DATA_CORRUPT**

- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

- The operation state is not valid: it must be inactive.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.8.5.5 psa_pake_set_role

[psa_status_t](#) **psa_pake_set_role**([psa_pake_operation_t](#) *operation, [psa_pake_role_t](#) role)

Set the application role for a password-authenticated key exchange.

Parameters

- **operation** ([psa_pake_operation_t](#)*) - [in] Active PAKE operation.
- **role** ([psa_pake_role_t](#)) - [in] A value of [typedef psa_pake_role_t](#) indicating the application role in the PAKE algorithm.

Description

Warning:

Not supported.

Not all PAKE algorithms need to differentiate the communicating participants. For PAKE algorithms that do not require a role to be specified, the application can do either of the following:

- Not call [psa_pake_set_role\(\)](#) on the PAKE operation.
- Call [psa_pake_set_role\(\)](#) with the `PSA_PAKE_ROLE_NONE` role.

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_ARGUMENT:**
 - `role` is not a valid PAKE role in the operation's algorithm.
 - `role` is not compatible with the operation's key type.
- **PSA_ERROR_NOT_SUPPORTED:**
 - `role` is not a valid PAKE role.
 - `role` is not supported with the operation's key type.
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be active, and [psa_pake_set_role\(\)](#), [psa_pake_input\(\)](#), and [psa_pake_output\(\)](#) must not have been called yet.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.8.5.6 psa_pake_set_user

[psa_status_t](#) [psa_pake_set_user](#)([psa_pake_operation_t](#) *operation, const uint8_t *user_id, size_t user_id_len)

Set the user ID for a password-authenticated key exchange.

Parameters

- **operation** ([psa_pake_operation_t](#)*) - [in] Active PAKE operation.
- **user_id** (const uint8_t*) - [in] The user ID to authenticate with.
- **user_id_len** (size_t) - [in] Size of the `user_id` buffer in bytes.

Description

Warning:

Not supported.

Call this function to set the user ID. For PAKE algorithms that associate a user identifier with both participants in the session, also call [psa_pake_set_peer\(\)](#) with the peer ID. For PAKE algorithms that associate a single user identifier with the session, call [psa_pake_set_user\(\)](#) only.

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_ARGUMENT:**
`user_id` is not valid for the operation's algorithm and cipher suite.
- **PSA_ERROR_NOT_SUPPORTED:**
`user_id` is not supported.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be active, and [psa_pake_set_user\(\)](#), [psa_pake_input\(\)](#), and [psa_pake_output\(\)](#) must not have been called yet.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.8.5.7 psa_pake_set_peer

[psa_status_t](#) [psa_pake_set_peer](#)([psa_pake_operation_t](#) *operation, const uint8_t *peer_id, size_t peer_id_len)

Set the peer ID for a password-authenticated key exchange.

Parameters

- **operation** ([psa_pake_operation_t](#)*) - [in] Active PAKE operation.
- **peer_id** (const uint8_t*) - [in] The peer ID to authenticate with.
- **peer_id_len** (size_t) - [in] Size of the `peer_id` buffer in bytes.

Description

Warning:

Not supported.

Call this function in addition to [psa_pake_set_user\(\)](#) for PAKE algorithms that associate a user identifier with both participants in the session. For PAKE algorithms that associate a single user identifier with the session, call [psa_pake_set_user\(\)](#) only.

Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_ARGUMENT:**
`peer_id` is not valid for the operation's algorithm and cipher suite.
- **PSA_ERROR_NOT_SUPPORTED:**
`peer_id` is not supported for the implementation.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**

- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be active, and [psa_pake_set_peer\(\)](#), [psa_pake_input\(\)](#), and [psa_pake_output\(\)](#) must not have been called yet.
 - Calling [psa_pake_set_peer\(\)](#) is invalid with the operation's algorithm.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.8.5.8 psa_pake_set_context

[psa_status_t](#) **psa_pake_set_context**([psa_pake_operation_t](#) *operation, const uint8_t *context, size_t context_len)

Set the context data for a password-authenticated key exchange.

Parameters

- **operation** ([psa_pake_operation_t](#)*) - [in] Active PAKE operation.
- **context** (const uint8_t*) - [in] The context to authenticate with.
- **context_len** (size_t) - [in] Size of the `context` buffer in bytes.

Description

Warning:

Not supported.

Call this function for PAKE algorithms that accept additional context data as part of the protocol setup.

Return

- **PSA_SUCCESS:**

Success.
- **PSA_ERROR_INVALID_ARGUMENT:**

`context` is not valid for the operation's algorithm and cipher suite.
- **PSA_ERROR_NOT_SUPPORTED:**

The value of the `context` is not supported.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be active, and [psa_pake_set_context\(\)](#), [psa_pake_input\(\)](#), and [psa_pake_output\(\)](#) must not have been called yet.
 - Calling [psa_pake_set_context\(\)](#) is invalid with the operation's algorithm.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.8.5.9 psa_pake_output

[psa_status_t](#) **psa_pake_output**([psa_pake_operation_t](#) *operation, [psa_pake_step_t](#) step, uint8_t *output, size_t output_size, size_t *output_length)

Get output for a step of a password-authenticated key exchange.

Parameters

- **operation** ([psa_pake_operation_t*](#)) - [in] Active PAKE operation.
- **step** ([psa_pake_step_t](#)) - [in] The step of the algorithm for which the output is requested.
- **output** ([uint8_t*](#)) - [out] Buffer where the output is to be written. The format of the output depends on the step.
- **output_size** ([size_t](#)) - [in] Size of the output buffer in bytes.
- **output_length** ([size_t*](#)) - [out] On success, the number of bytes of the returned output.

Description

Warning:

Not supported.

The `output_size` parameter must be appropriate for the cipher suite and output step:

- A sufficient output size is [PSA_PAKE_OUTPUT_SIZE](#) where `alg` and `primitive` are the PAKE algorithm and primitive in the operation's cipher suite, and `step` is the output step.
- [PSA_PAKE_OUTPUT_MAX_SIZE](#) evaluates to the maximum output size of any supported PAKE algorithm, primitive and step.

Depending on the algorithm being executed, you might need to call this function several times or you might not need to call this at all.

The exact sequence of calls to perform a password-authenticated key exchange depends on the algorithm in use.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_pake_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the `output` buffer is too small. [PSA_PAKE_OUTPUT_SIZE](#) or [PSA_PAKE_OUTPUT_MAX_SIZE](#) can be used to determine a sufficient buffer size.

- **PSA_ERROR_INVALID_ARGUMENT:**

`step` is not compatible with the operation's algorithm.

- **PSA_ERROR_NOT_SUPPORTED:**

`step` is not supported with the operation's algorithm.

- **PSA_ERROR_INSUFFICIENT_ENTROPY**
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**

- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be active and fully set up, and this call must conform to the algorithm's requirements for ordering of input and output steps.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.8.5.10 psa_pake_input

[psa_status_t](#) **psa_pake_input**([psa_pake_operation_t](#) *operation, [psa_pake_step_t](#) step, const uint8_t *input, size_t input_length)

Provide input for a step of a password-authenticated key exchange.

Parameters

- **operation** ([psa_pake_operation_t](#)*) - [in] Active PAKE operation.
- **step** ([psa_pake_step_t](#)) - [in] The step for which the input is provided.
- **input** (const uint8_t*) - [in] Buffer containing the input. The format of the input depends on the step.
- **input_length** (size_t) - [in] Size of the input buffer in bytes.

Description

Warning:

Not supported.

Depending on the algorithm being executed, you might need to call this function several times or you might not need to call this at all.

The exact sequence of calls to perform a password-authenticated key exchange depends on the algorithm in use.

[PSA_PAKE_INPUT_SIZE](#) or [PSA_PAKE_INPUT_MAX_SIZE](#) can be used to allocate buffers of sufficient size to transfer inputs that are received from the peer into the operation.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_pake_abort\(\)](#).

Return

- **PSA_SUCCESS:**
 - Success.
- **PSA_ERROR_INVALID_SIGNATURE:**
 - The verification fails for a PSA_PAKE_STEP_ZK_PROOF or PSA_PAKE_STEP_CONFIRM input step.
- **PSA_ERROR_INVALID_ARGUMENT:**
 - `step` is not compatible with the operation's algorithm.
 - The input is not valid for the operation's algorithm, cipher suite or step.
- **PSA_ERROR_NOT_SUPPORTED:**
 - `step` is not supported with the operation's algorithm.
 - The input is not supported for the operation's algorithm, cipher suite or step.
- **PSA_ERROR_INSUFFICIENT_MEMORY**

- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- PSA_ERROR_STORAGE_FAILURE
- PSA_ERROR_DATA_CORRUPT
- PSA_ERROR_DATA_INVALID
- **PSA_ERROR_BAD_STATE:**
 - The operation state is not valid: it must be active and fully set up, and this call must conform to the algorithm's requirements for ordering of input and output steps.
 - The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.8.5.11 psa_pake_get_shared_key

[psa_status_t](#) [psa_pake_get_shared_key](#)([psa_pake_operation_t](#) *operation, const [psa_key_attributes_t](#) *attributes, [psa_key_id_t](#) *key)

Extract the shared secret from the PAKE as a key.

Parameters

- **operation** ([psa_pake_operation_t](#)*) - [in] Active PAKE operation.
- **attributes** (const [psa_key_attributes_t](#)*) - [in] The attributes for the new key.
- **key** ([psa_key_id_t](#)*) - [out] On success, an identifier for the newly created key. PSA_KEY_ID_NULL on failure.

Description

Warning:

Not supported.

The following new key attributes are required:

- The key type. All PAKE algorithms can output a key of type PSA_KEY_TYPE_DERIVE or PSA_KEY_TYPE_HMAC. PAKE algorithms that produce a pseudorandom shared secret, can also output block-cipher key types, for example PSA_KEY_TYPE_AES.
- The key permitted algorithm policy.
- The key usage flags.

The following new key attributes must be set in case of a persistent key:

- The key lifetime.
- The key identifier.

The following new key attributes is optional:

- The key size, if nonzero, it must be equal to the size of the PAKE secret shared.

The shared secret is retrieved as a key. Its location, policy, and type are taken from [attributes](#).

The size of the returned key is always the bit-size of the PAKE shared secret, rounded up to a whole number of bytes. The size of the shared secret is dependent on the PAKE algorithm and cipher suite.

This is the final call in a PAKE operation, which retrieves the shared secret as a key. It is recommended that this key is used as an input to a key derivation operation to produce additional cryptographic keys. For some PAKE algorithms, the shared secret is also suitable for use as a key in cryptographic operations such as encryption.

Depending on the key confirmation requested in the cipher suite, [psa_pake_get_shared_key\(\)](#) must be called either before or after the key-confirmation output and input steps for the PAKE algorithm. The key confirmation affects the guarantees that can be made about the shared key:

Unconfirmed key

If the cipher suite used to set up the operation requested an unconfirmed key, the application must call [psa_pake_get_shared_key\(\)](#) after the key-exchange output and input steps are completed. The PAKE algorithm provides a cryptographic guarantee that only a peer who used the same password, and identity inputs, is able to compute the same key. However, there is no guarantee that the peer is the participant it claims to be, and was able to compute the same key.

Since the peer is not authenticated, no action should be taken that assumes that the peer is who it claims to be. For example, do not access restricted resources on the peer's behalf until an explicit authentication has succeeded.

Note:

Some PAKE algorithms do not enable the output of the shared secret until it has been confirmed.

Confirmed key

If the cipher suite used to set up the operation requested a confirmed key, the application must call [psa_pake_get_shared_key\(\)](#) after the key-exchange and key-confirmation output and input steps are completed.

Following key confirmation, the PAKE algorithm provides a cryptographic guarantee that the peer used the same password and identity inputs, and has computed the identical shared secret key.

Since the peer is not authenticated, no action should be taken that assumes that the peer is who it claims to be. For example, do not access restricted resources on the peer's behalf until an explicit authentication has succeeded.

Note:

Some PAKE algorithms do not include any key-confirmation steps.

The exact sequence of calls to perform a password-authenticated key exchange depends on the algorithm in use.

When this function returns successfully, operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_pake_abort\(\)](#).

Return

- **PSA_SUCCESS:**

Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.

- **PSA_ERROR_NOT_PERMITTED:**

Creating a key with the specified attributes is not permitted.

- **PSA_ERROR_ALREADY_EXISTS:**

This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.

- **PSA_ERROR_INVALID_ARGUMENT:**

- The key type is not valid for output from this operation's algorithm.
- The key size is nonzero.
- The key lifetime is invalid.
- The key identifier is not valid for the key lifetime.

- The key usage flags include invalid values.
- The key's permitted-usage algorithm is invalid.
- The key attributes, as a whole, are invalid.

- **PSA_ERROR_NOT_SUPPORTED:**

The key attributes, as a whole, are not supported for creation from a PAKE secret.

- PSA_ERROR_INSUFFICIENT_MEMORY
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- PSA_ERROR_STORAGE_FAILURE
- PSA_ERROR_DATA_CORRUPT
- PSA_ERROR_DATA_INVALID

- **PSA_ERROR_BAD_STATE:**

- The state of PAKE operation operation is not valid: it must be ready to return the shared secret.
For an unconfirmed key, this will be when the key-exchange output and input steps are complete, but prior to any key-confirmation output and input steps.
For a confirmed key, this will be when all key-exchange and key-confirmation output and input steps are complete.
- The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.8.5.12 `psa_pake_abort`

[psa_status_t](#) `psa_pake_abort`([psa_pake_operation_t](#) *operation)

Abort a PAKE operation.

Parameters

- **operation** ([psa_pake_operation_t](#)*) - [in] Initialized PAKE operation.

Description

Warning:

Not supported

Aborting an operation frees all associated resources except for the `operation` object itself. Once aborted, the operation object can be reused for another operation by calling [psa_pake_setup\(\)](#) again.

This function can be called any time after the operation object has been initialized as described in [typedef `psa\_pake\_operation\_t`](#).

In particular, calling [psa_pake_abort\(\)](#) after the operation has been terminated by a call to [psa_pake_abort\(\)](#) or [psa_pake_get_shared_key\(\)](#) is safe and has no effect.

Return

- **PSA_SUCCESS:**

Success. The operation object can now be discarded or reused.

- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by [psa_crypto_init\(\)](#).

6.2.3.8.6 Macros

6.2.3.8.6.1 macro PSA_PAKE_OUTPUT_SIZE

PSA_PAKE_OUTPUT_SIZE(*alg*, *primitive*, *output_step*)

Sufficient output buffer size for [psa_pake_output\(\)](#), in bytes.

Parameters

- **alg** - [in] A PAKE algorithm such that [PSA_ALG_IS_PAKE](#) is true.
- **primitive** - [in] A primitive of [typedef psa_pake_primitive_t](#) that is compatible with algorithm *alg*.
- **output_step** - [in] A value of [typedef psa_pake_step_t](#) that is valid for the algorithm *alg*.

Description

Warning:

Not supported.

If the size of the output buffer is at least this large, it is guaranteed that [psa_pake_output\(\)](#) will not fail due to an insufficient buffer size. The actual size of the output might be smaller in any given call.

See also [PSA_PAKE_OUTPUT_MAX_SIZE](#)

Return

A sufficient output buffer size for the specified PAKE algorithm, primitive, and output step. If algorithm, primitive, and output step is not recognized, return 0.

6.2.3.8.6.2 macro PSA_PAKE_OUTPUT_MAX_SIZE

PSA_PAKE_OUTPUT_MAX_SIZE()

The maximum output buffer size for [psa_pake_output\(\)](#) for any of the supported PAKE algorithms, primitives and output steps.

Description

Warning:

Not supported.

If the size of the output buffer is at least this large, it is guaranteed that [psa_pake_output\(\)](#) will not fail due to an insufficient buffer size.

See also [PSA_PAKE_OUTPUT_SIZE](#).

6.2.3.8.6.3 macro PSA_PAKE_INPUT_SIZE

PSA_PAKE_INPUT_SIZE(*alg*, *primitive*, *input_step*)

Sufficient buffer size for inputs to [psa_pake_input\(\)](#).

Parameters

- **alg** - [in] A PAKE algorithm such that [PSA_ALG_IS_PAKE](#) is true.
- **primitive** - [in] A primitive of [typedef psa_pake_primitive_t](#) that is compatible with algorithm `alg`.
- **input_step** - [in] A value of [typedef psa_pake_step_t](#) that is valid for the algorithm `alg`.

Description**Warning:**

Not supported.

The value returned by this macro is guaranteed to be large enough for any valid input to [psa_pake_input\(\)](#) in an operation with the specified parameters.

This macro can be useful when transferring inputs from the peer into the PAKE operation.

See also [PSA_PAKE_INPUT_MAX_SIZE](#)

Return

A sufficient buffer size for the specified PAKE algorithm, primitive, and input step. If algorithm, primitive, and output step is not recognized, return 0.

6.2.3.8.6.4 macro PSA_PAKE_INPUT_MAX_SIZE**PSA_PAKE_INPUT_MAX_SIZE()**

The maximum buffer size for inputs to [psa_pake_input\(\)](#) for any of the supported PAKE algorithms, primitives and input steps.

Description**Warning:**

Not supported.

This macro can be useful when transferring inputs from the peer into the PAKE operation.

See also [PSA_PAKE_INPUT_SIZE](#).

6.2.4 Attestation APIs**6.2.4.1 Challenge sizes**

The following challenge sizes are supported.

Table 159. PSA initial attestation challenge sizes

Name	Size in bytes
PSA_INITIAL_ATTEST_CHALLENGE_SIZE_32	32
PSA_INITIAL_ATTEST_CHALLENGE_SIZE_48	48
PSA_INITIAL_ATTEST_CHALLENGE_SIZE_64	64

6.2.4.2 Token Attestation

6.2.4.2.1 Functions

6.2.4.2.1.1 psa_initial_attest_get_token

[psa_status_t](#) **psa_initial_attest_get_token**(const uint8_t *auth_challenge, size_t challenge_size, uint8_t *token_buf, size_t token_buf_size, size_t *token_size)

Retrieve the Initial Attestation Token.

Parameters

- **auth_challenge** (const uint8_t*) - [in] Buffer with a challenge object. The challenge object is data provided by the caller. For example, it may be a cryptographic nonce or a hash of data (such as an external object record).
- **challenge_size** (size_t) - [in] Size of the buffer `auth_challenge` in bytes. The size must always be a supported challenge size.
- **token_buf** (uint8_t*) - [out] Output buffer where the attestation token is to be written.
- **token_buf_size** (size_t) - [in] Size of `token_buf`. The expected size can be determined by using [psa_initial_attest_get_token_size\(\)](#).
- **token_size** (size_t*) - [out] Output variable for the actual token size.

Description

Warning:

Not supported.

Retrieves the Initial Attestation Token. A challenge can be passed as an input to mitigate replay attacks.

If the `auth_challenge` is a hash of data is then it is the caller's responsibility to ensure that the data is protected against replay attacks (for example, by including a cryptographic nonce within the data).

Return

- **PSA_SUCCESS:**
Action was performed successfully.
- **PSA_ERROR_SERVICE_FAILURE:**
Secure target failed to initialize.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
`token_buf` is too small for the attestation token.
- **PSA_ERROR_INVALID_ARGUMENT:**
The challenge size is not supported.
- **PSA_ERROR_GENERIC_ERROR:**
An unspecified internal error has occurred.

6.2.4.2.1.2 psa_initial_attest_get_token_size

[psa_status_t](#) **psa_initial_attest_get_token_size**(size_t challenge_size, size_t *token_size)

Calculate the size of an Initial Attestation Token.

Parameters

- **challenge_size** (size_t) - [in] Size of a challenge object in bytes. This must be a supported challenge size.
- **token_size** (size_t*) - [out] Output variable for the token size.

Description

Warning:

Not supported.

Retrieve the exact size of the Initial Attestation Token in bytes, given a specific challenge size.

Return

- **PSA_SUCCESS:**
Action was performed successfully.
- **PSA_ERROR_SERVICE_FAILURE:**
Secure target failed to initialize.
- **PSA_ERROR_INVALID_ARGUMENT:**
The challenge size is not supported.
- **PSA_ERROR_GENERIC_ERROR:**
An unspecified internal error has occurred.

6.2.5 Storage APIs

6.2.5.1 General definitions

6.2.5.1.1 Structure and Typedefs

6.2.5.1.1.1 struct psa_storage_info_t

struct **psa_storage_info_t**

Storage info

Definition

```
struct psa_storage_info_t {  
    size_t capacity;  
    size_t size;  
    psa_storage_create_flags_t flags;  
}
```

Members

capacity

The allocated capacity of the storage associated with a uid.

size

The size of the data associated with a uid.

flags

The flags set when the uid was created.

6.2.5.1.1.2 typedef psa_storage_create_flags_t

type **psa_storage_create_flags_t**

Storage create flags

Description

Flags used when creating a data entry.

The following [PSA data creation flags](#) table lists all data creation flags supported.

Table 160. PSA data creation flags

Flags	Value
PSA_STORAGE_FLAG_NONE	0x00
PSA_STORAGE_FLAG_WRITE_ONCE	0x01
PSA_STORAGE_FLAG_NO_CONFIDENTIALITY	0x02
PSA_STORAGE_FLAG_NO_REPLAY_PROTECTION	0x04

6.2.5.1.1.3 typedef psa_storage_uid_t

type **psa_storage_uid_t**

Storage uid

Description

A type for uid used for identifying data.

Warning:

The implementation does not support `psa_storage_uid_t` value greater than 0xFFFFFFFF

6.2.5.2 Internal Trusted Storage

Internal Trusted Storage is designed to store the most sensitive device data inside the Platform Root of Trust (PRoT).

- It lives in trusted, isolated memory (e.g., secure flash, OTP, secure RAM)
- Accessible only by trusted firmware / secure partitions

6.2.5.2.1 Functions

6.2.5.2.1.1 `psa_its_set`

`psa_status_t` `psa_its_set`(`psa_storage_uid_t` `uid`, `size_t` `data_length`, `const void *``p_data`, `psa_storage_create_flags_t` `create_flags`)

Create a new, or modify an existing, uid/value pair.

Parameters

- `uid` (`psa_storage_uid_t`) - [in] The identifier for the data.
- `data_length` (`size_t`) - [in] The size in bytes of the data in `p_data`.
- `p_data` (`const void *`) - [in] A buffer containing the data.
- `create_flags` (`psa_storage_create_flags_t`) - [in] The flags that the data will be stored with.

Description

Stores data in the internal storage.

Return

- **PSA_SUCCESS:**
The operation completed successfully.
- **PSA_ERROR_NOT_PERMITTED:**
The operation failed because the provided `uid` value was already created with `PSA_STORAGE_FLAG_WRITE_ONCE`.
- **PSA_ERROR_NOT_SUPPORTED:**
The operation failed because one or more of the flags provided in `create_flags` is not supported or is not valid.
- **PSA_ERROR_INSUFFICIENT_STORAGE:**
The operation failed because there was insufficient space on the storage medium.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).
- **PSA_ERROR_INVALID_ARGUMENT:**
 - `p_data` is NULL.
 - `uid` is not supported.

6.2.5.2.1.2 `psa_its_get`

`psa_status_t` `psa_its_get`(`psa_storage_uid_t` `uid`, `size_t` `data_offset`, `size_t` `data_size`, `void *``p_data`, `size_t *``p_data_length`)

Retrieve data associated with a provided UID.

Parameters

- **uid** ([psa_storage_uid_t](#)) - [in] The uid value.
- **data_offset** (size_t) - [in] The starting offset of the data requested.
- **data_size** (size_t) - [in] The amount of data requested.
- **p_data** (void*) - [out] On success, the buffer where the data will be placed.
- **p_data_length** (size_t*) - [out] On success, this will contain size of the data placed in p_data.

Description

Retrieves data associated with uid, starting at data_offset bytes from the beginning of the data. Fetches the lesser of data_size or uid.size - data_offset bytes, which can be zero.

If data_size is 0 or data_offset == uid.size, the contents of p_data_length will be set to zero, but the contents of p_data are unchanged. The function returns PSA_SUCCESS.

Return

- **PSA_SUCCESS:**
The operation completed successfully.
- **PSA_ERROR_DOES_NOT_EXIST:**
The operation failed because the provided uid value was not found in the storage.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).
- **PSA_ERROR_INVALID_ARGUMENT:**
 - p_data is NULL.
 - data_size is 0.
 - p_data_length is NULL.
 - uid is not supported.

6.2.5.2.1.3 psa_its_get_info

[psa_status_t](#) **psa_its_get_info**([psa_storage_uid_t](#) uid, [struct psa_storage_info_t](#) *p_info)

Retrieve the metadata about the provided uid.

Parameters

- **uid** ([psa_storage_uid_t](#)) - [in] The uid value.
- **p_info** ([struct psa_storage_info_t](#)*) - [out] A pointer to the [struct psa_storage_info_t](#) that will be populated with the metadata.

Description

Retrieves the metadata stored for a given uid as a [struct psa_storage_info_t](#).

Return

- **PSA_SUCCESS:**
The operation completed successfully.

- **PSA_ERROR_DOES_NOT_EXIST:**

The operation failed because the provided `uid` value was not found in the storage.

- **PSA_ERROR_STORAGE_FAILURE:**

The operation failed because the physical storage has failed (Fatal error).

- **PSA_ERROR_INVALID_ARGUMENT:**

- `uid` is not supported.
- `p_info` is NULL.

6.2.5.2.1.4 `psa_its_remove`

[`psa\_status\_t`](#) `psa_its_remove`([`psa\_storage\_uid\_t`](#) `uid`)

Remove the provided key and its associated data from the storage.

Parameters

- `uid` ([`psa\_storage\_uid\_t`](#)) - [in] The uid value.

Description

Deletes the data from internal storage.

Return

- **PSA_SUCCESS:**

The operation completed successfully.

- **PSA_ERROR_INVALID_ARGUMENT:**

`uid` is not supported.

- **PSA_ERROR_DOES_NOT_EXIST:**

The operation failed because the provided key value was not found in the storage.

- **PSA_ERROR_NOT_PERMITTED:**

The operation failed because the provided key value was created with `PSA_STORAGE_FLAG_WRITE_ONCE`.

- **PSA_ERROR_STORAGE_FAILURE:**

The operation failed because the physical storage has failed (Fatal error).

6.2.5.3 Protected Storage

The Protected Storage is a general-purpose secure storage layer for larger or external data.

- Typically stored in:
 - External flash
 - Non-secure memory
- Security is provided via crypto mechanisms.

Warning:

The current implementation does not support such of storage.

6.2.5.3.1 Functions

6.2.5.3.1.1 psa_ps_set

[psa_status_t](#) **psa_ps_set**([psa_storage_uid_t](#) uid, [size_t](#) data_length, const void *p_data, [psa_storage_create_flags_t](#) create_flags)

Create a new or modify an existing key/value pair.

Parameters

- **uid** ([psa_storage_uid_t](#)) - [in] The identifier for the data.
- **data_length** ([size_t](#)) - [in] The size in bytes of the data in p_data.
- **p_data** (const void*) - [in] A buffer containing the data.
- **create_flags** ([psa_storage_create_flags_t](#)) - [in] The flags indicating the properties of the data.

Description

Warning:

Not supported.

The newly created asset has a capacity and size that are equal to data_length.

Return

- **PSA_SUCCESS:**
The operation completed successfully.
- **PSA_ERROR_NOT_PERMITTED:**
The operation failed because the provided uid value was already created with PSA_STORAGE_FLAG_WRITE_ONCE.
- **PSA_ERROR_INVALID_ARGUMENT:**
The operation failed because one or more of the given arguments were invalid.
- **PSA_ERROR_NOT_SUPPORTED:**
The operation failed because one or more of the flags provided in create_flags is not supported or is not valid.
- **PSA_ERROR_INSUFFICIENT_STORAGE:**
The operation failed because there was insufficient space on the storage medium.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).
- **PSA_ERROR_GENERIC_ERROR:**
The operation failed because of an unspecified internal failure.

6.2.5.3.1.2 psa_ps_get

[psa_status_t](#) **psa_ps_get**([psa_storage_uid_t](#) uid, [size_t](#) data_offset, [size_t](#) data_size, void *p_data, [size_t](#) *p_data_length)

Retrieve data associated with a provided uid.

Parameters

- **uid** ([psa_storage_uid_t](#)) - [in] The uid value.
- **data_offset** (size_t) - [in] The starting offset of the data requested.
- **data_size** (size_t) - [in] The amount of data requested.
- **p_data** (void*) - [out] On success, the buffer where the data will be placed.
- **p_data_length** (size_t*) - [out] On success, will contain size of the data placed in p_data.

Description

Warning:

Not supported.

Retrieves data associated with uid, starting at data_offset bytes from the beginning of the data. Fetches the smaller of data_size or uid.size - data_offset bytes. This can be zero.

If data_size is 0 or data_offset == uid.size, the contents of p_data_length will be set to zero, but the contents of p_data are unchanged. The function returns PSA_SUCCESS.

Return

- **PSA_SUCCESS:**
The operation completed successfully.
- **PSA_ERROR_INVALID_ARGUMENT:**
 - The operation failed because one of the provided arguments (e.g. p_data, p_data_length) is invalid.
 - The uid is 0.
 - The caller cannot access some or all of the memory in the range [p_data, p_data + data_size - 1].
 - data_offset is larger than the size of the data associated with uid.
- **PSA_ERROR_DOES_NOT_EXIST:**
The operation failed because the provided uid value was not found in the storage.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).
- **PSA_ERROR_GENERIC_ERROR:**
The operation failed because of an unspecified internal failure.
- **PSA_ERROR_DATA_CORRUPT:**
The operation failed because of an authentication failure when attempting to get the key.
- **PSA_ERROR_INVALID_SIGNATURE:**
The operation failed because the data associated with the uid failed authentication.

6.2.5.3.1.3 psa_ps_get_info

[psa_status_t](#) **psa_ps_get_info**([psa_storage_uid_t](#) uid, struct [psa_storage_info_t](#) *p_info)

Retrieve the metadata about the provided uid.

Parameters

- **uid** ([psa_storage_uid_t](#)) - [in] The identifier for the data.
- **p_info** (struct [psa_storage_info_t](#)*) - [out] A pointer to the [struct psa_storage_info_t](#) that will be populated with the metadata.

Description

Warning:

Not supported.

Retrieves the metadata stored for a given `uid` as a [struct psa_storage_info_t](#).

Return

- **PSA_SUCCESS:**
The operation completed successfully.
- **PSA_ERROR_INVALID_ARGUMENT:**
The operation failed because one or more of the given arguments were invalid (null pointer, wrong flags and so on).
- **PSA_ERROR_DOES_NOT_EXIST:**
The operation failed because the provided `uid` value was not found in the storage.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).
- **PSA_ERROR_GENERIC_ERROR:**
The operation failed because of an unspecified internal failure.
- **PSA_ERROR_DATA_CORRUPT:**
The operation failed because of an authentication failure when attempting to get the key.
- **PSA_ERROR_INVALID_SIGNATURE:**
The operation failed because the data associated with the `uid` failed authentication.

6.2.5.3.1.4 `psa_ps_remove`

[psa_status_t](#) `psa_ps_remove`([psa_storage_uid_t](#) `uid`)

Remove the provided `uid` and its associated data from the storage.

Parameters

- **uid** ([psa_storage_uid_t](#)) - [in] The identifier for the data to be removed.

Description

Warning:

Not supported.

Removes previously stored data and any associated metadata, including rollback protection data.

Return

- **PSA_SUCCESS:**
The operation completed successfully.
- **PSA_ERROR_INVALID_ARGUMENT:**
The `uid` is not valid.
- **PSA_ERROR_DOES_NOT_EXIST:**
The operation failed because the provided `uid` value was not found in the storage.
- **PSA_ERROR_NOT_PERMITTED:**
The operation failed because the provided `uid` value was created with `PSA_STORAGE_FLAG_WRITE_ONCE`.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).
- **PSA_ERROR_GENERIC_ERROR:**
The operation failed because of an unspecified internal failure.

6.2.5.3.1.5 `psa_ps_create`

[`psa\_status\_t`](#) `psa_ps_create`([`psa\_storage\_uid\_t`](#) `uid`, `size_t` `capacity`, [`psa\_storage\_create\_flags\_t`](#) `create_flags`)

Reserves storage for the specified `uid`.

Parameters

- `uid` ([`psa\_storage\_uid\_t`](#)) - [in] A unique identifier for the asset.
- `capacity` (`size_t`) - [in] The allocated capacity, in bytes, of the `uid`.
- `create_flags` ([`psa\_storage\_create\_flags\_t`](#)) - [in] Flags indicating properties of the storage.

Description

Warning:

Not supported.

Reserves storage for the specified `uid`. Upon success, the capacity of the storage is `capacity`, and the size is 0.

It is only necessary to call this function for assets that will be written with the [`psa\_ps\_set\_extended\(\)`](#) function. If only [`psa\_ps\_set\(\)`](#) is needed, calls to this function are redundant.

This function cannot be used to replace an existing asset, and attempting to do so will return `PSA_ERROR_ALREADY_EXISTS`.

If the `PSA_STORAGE_FLAG_WRITE_ONCE` flag is passed, [`psa\_ps\_create\(\)`](#) will return `PSA_ERROR_NOT_SUPPORTED`.

This function is supported only if the [`psa\_ps\_get\_support\(\)`](#) returns `PSA_STORAGE_SUPPORT_SET_EXTENDED`.

Return

- **PSA_SUCCESS:**

The storage was successfully reserved.

- **PSA_ERROR_STORAGE_FAILURE:**

The operation failed because the physical storage has failed (Fatal error).

- **PSA_ERROR_INSUFFICIENT_STORAGE:**

`capacity` is bigger than the current available space.

- **PSA_ERROR_NOT_SUPPORTED:**

The function is not implemented or one or more `create_flags` are not supported.

- **PSA_ERROR_INVALID_ARGUMENT:**

`uid` was 0 or `create_flags` specified flags that are not defined in the API.

- **PSA_ERROR_GENERIC_ERROR:**

The operation has failed due to an unspecified error.

- **PSA_ERROR_ALREADY_EXISTS:**

Storage for the specified `uid` already exists.

6.2.5.3.1.6 `psa_ps_set_extended`

`psa_status_t` `psa_ps_set_extended`(`psa_storage_uid_t` `uid`, `size_t` `data_offset`, `size_t` `data_length`, `const void *``p_data`)

Overwrite part of the data of the specified `uid`.

Parameters

- `uid` (`psa_storage_uid_t`) - [in] The unique identifier for the asset.
- `data_offset` (`size_t`) - [in] Offset within the asset to start the write.
- `data_length` (`size_t`) - [in] The size in bytes of the data in `p_data` to write.
- `p_data` (`const void *`) - [in] Pointer to a buffer which contains the data to write.

Description

Warning:

Not supported.

Sets partial data into an asset based on the given `uid`, `data_offset`, `data_length` and `p_data`.

Before calling this function, the storage must have been reserved with a call to `psa_ps_create()`. It can also be used to overwrite data in an asset that was created with a call to `psa_ps_set()`.

Calling this function with `data_length = 0` is permitted. This makes no change to the stored data.

This function can overwrite existing data and/or extend it up to the capacity for the `uid` specified in `psa_ps_create()`, but cannot create gaps. That is, it has preconditions:

- `data_offset` $\leq$ `size`
- `data_offset` + `data_length` $\leq$ `capacity`

and postconditions:

- `size` = $\max(\text{size}, \text{data_offset} + \text{data_length})$
- `capacity` unchanged.

This function is supported only if the `psa_ps_get_support()` returns `PSA_STORAGE_SUPPORT_SET_EXTENDED`.

Return

- **PSA_SUCCESS:**

The asset exists, the input parameters are correct and the data is correctly written in the physical storage.

- **PSA_ERROR_STORAGE_FAILURE:**

The data was not written correctly in the physical storage.

- **PSA_ERROR_INVALID_ARGUMENT:**

The operation failed because one or more of the preconditions listed above regarding `data_offset`, `size`, or `data_length` was violated.

- **PSA_ERROR_DOES_NOT_EXIST:**

The specified `uid` was not found.

- **PSA_ERROR_NOT_SUPPORTED:**

The implementation of the API does not support this function.

- **PSA_ERROR_GENERIC_ERROR:**

The operation failed due to an unspecified error.

- **PSA_ERROR_DATA_CORRUPT:**

The operation failed because the existing data has been corrupted.

- **PSA_ERROR_INVALID_SIGNATURE:**

The operation failed because the existing data failed authentication (MAC check failed).

- **PSA_ERROR_NOT_PERMITTED:**

The operation failed because it was attempted on an asset which was written with the flag `PSA_STORAGE_FLAG_WRITE_ONCE`.

6.2.5.3.1.7 `psa_ps_get_support`

`uint32_t psa_ps_get_support(void)`

Get implemented optional features

Parameters

- `void` - no arguments

Description

Warning:

Not supported.

Returns a bitmask with flags set for all of the optional features supported by the implementation.

Currently defined flags are limited to:

- `PSA_STORAGE_SUPPORT_SET_EXTENDED`

Return

Bitmask of the features supported.

6.2.6 Return codes

6.2.6.1 typedef psa_status_t

type **psa_status_t**

Function return status.

6.2.6.1.1 Description

This is either `PSA_SUCCESS`, which is zero, indicating success; or a small negative value indicating that an error occurred. Errors are encoded as one of the `PSA_ERROR_XXX` values defined here.

6.2.6.1.2 Values

- **PSA_SUCCESS:**

The action was completed successfully.

- **PSA_ERROR_ALREADY_EXISTS:**

Asking for an item that already exists.

It is recommended that implementations return this error code when attempting to write to a location where a key is already present.

- **PSA_ERROR_BAD_STATE:**

The requested action cannot be performed in the current state.

Multi-part operations return this error when one of the functions is called out of sequence. Refer to the function descriptions for permitted sequencing of functions.

Implementations must not return this error code to indicate that a key identifier is invalid, but must return `PSA_ERROR_INVALID_HANDLE` instead.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

An output buffer is too small.

Applications can call the `PSA_XXX_SIZE` macro listed in the function description to determine a sufficient buffer size.

It is recommended that implementations only return this error code in cases when performing the operation with a larger output buffer would succeed. However, implementations can also return this error if a function has invalid or unsupported parameters in addition to an insufficient output buffer size.

- **PSA_ERROR_COMMUNICATION_FAILURE:**

There was a communication failure inside the implementation.

This can indicate a communication failure between the application and an external cryptoprocessor or between the cryptoprocessor and an external volatile or persistent memory. A communication failure can be transient or permanent depending on the cause.

Warning:

If a function returns this error, it is undetermined whether the requested action has completed. Returning `PSA_SUCCESS` is recommended on successful completion whenever possible, however functions can return `PSA_ERROR_COMMUNICATION_FAILURE` if the requested action was completed successfully in an external cryptoprocessor but there was a breakdown of communication before the cryptoprocessor could report the status to the application.

- **PSA_ERROR_CORRUPTION_DETECTED:**

A tampering attempt was detected.

If an application receives this error code, there is no guarantee that previously accessed or computed data was correct and remains confidential. In this situation, it is recommended that applications perform no further security functions and enter a safe failure state.

Implementations can return this error code if they detect an invalid state that cannot happen during normal operation and that indicates that the implementation's security guarantees no longer hold. Depending on the implementation architecture and on its security and safety goals, the implementation might forcibly terminate the application.

This error code is intended as a last resort when a security breach is detected and it is unsure whether the keystore data is still protected. Implementations must only return this error code to report an alarm from a tampering detector, to indicate that the confidentiality of stored data can no longer be guaranteed, or to indicate that the integrity of previously returned data is now considered compromised. Implementations must not use this error code to indicate a hardware failure that merely makes it impossible to perform the requested operation, instead use `PSA_ERROR_COMMUNICATION_FAILURE`, `PSA_ERROR_STORAGE_FAILURE`, `PSA_ERROR_HARDWARE_FAILURE`, `PSA_ERROR_INSUFFICIENT_ENTROPY` or other applicable error code.

This error indicates an attack against the application. Implementations must not return this error code as a consequence of the behavior of the application itself.

- **PSA_ERROR_DATA_CORRUPT:**

Stored data has been corrupted.

This error indicates that some persistent storage has suffered corruption. It does not indicate the following situations, which have specific error codes:

- A corruption of volatile memory - use `PSA_ERROR_CORRUPTION_DETECTED`.
- A communication error between the cryptoprocessor and its external storage - use `PSA_ERROR_COMMUNICATION_FAILURE`.
- When the storage is in a valid state but is full - use `PSA_ERROR_INSUFFICIENT_STORAGE`.
- When the storage fails for other reasons - use `PSA_ERROR_STORAGE_FAILURE`.
- When the stored data is not valid - use `PSA_ERROR_DATA_INVALID`.

Note that a storage corruption does not indicate that any data that was previously read is invalid. However this previously read data might no longer be readable from storage.

When a storage failure occurs, it is no longer possible to ensure the global integrity of the keystore. Depending on the global integrity guarantees offered by the implementation, access to other data might fail even if the data is still readable but its integrity cannot be guaranteed.

It is recommended to only use this error code to report when a storage component indicates that the stored data is corrupt, or fails an integrity check. For example, in situations that the PSA Storage API [PSA-ITS] reports `PSA_ERROR_DATA_CORRUPT` or `PSA_ERROR_INVALID_SIGNATURE`.

- **PSA_ERROR_DATA_INVALID:**

Data read from storage is not valid for the implementation.

This error indicates that some data read from storage does not have a valid format. It does not indicate the following situations, which have specific error codes:

- When the storage or stored data is corrupted - use `PSA_ERROR_DATA_CORRUPT`.
- When the storage fails for other reasons - use `PSA_ERROR_STORAGE_FAILURE`.
- An invalid argument to the API - use `PSA_ERROR_INVALID_ARGUMENT`.

This error is typically a result of an integration failure, where the implementation reading the data is not compatible with the implementation that stored the data.

It is recommended to only use this error code to report when data that is successfully read from storage is invalid.

- **PSA_ERROR_DOES_NOT_EXIST:**

Asking for an item that doesn't exist.

Implementations must not return this error code to indicate that a key identifier is invalid, but must return `PSA_ERROR_INVALID_HANDLE` instead.

- **PSA_ERROR_GENERIC_ERROR:**

An error occurred that does not correspond to any defined failure cause.

Implementations can use this error code if none of the other standard error codes are applicable.

- **PSA_ERROR_HARDWARE_FAILURE:**

A hardware failure was detected.

A hardware failure can be transient or permanent depending on the cause.

- **PSA_ERROR_INSUFFICIENT_DATA:**

Return this error when there's insufficient data when attempting to read from a resource.

- **PSA_ERROR_INSUFFICIENT_ENTROPY:**

There is not enough entropy to generate random data needed for the requested action.

This error indicates a failure of a hardware random generator. Application writers must note that this error can be returned not only by functions whose purpose is to generate random data, such as key, IV or nonce generation, but also by functions that execute an algorithm with a randomized result, as well as functions that use randomization of intermediate computations as a countermeasure to certain attacks.

It is recommended that implementations do not return this error after [psa_crypto_init\(\)](#) has succeeded.

This can be achieved if the implementation generates sufficient entropy during initialization and subsequently a cryptographically secure pseudorandom generator (PRNG) is used. However, implementations might return this error at any time, for example, if a policy requires the PRNG to be reseeded during normal operation.

- **PSA_ERROR_INSUFFICIENT_MEMORY:**

There is not enough runtime memory.

If the action is carried out across multiple security realms, this error can refer to available memory in any of the security realms.

- **PSA_ERROR_INSUFFICIENT_STORAGE:**

There is not enough persistent storage.

Functions that modify the key storage return this error code if there is insufficient storage space on the host media. In addition, many functions that do not otherwise access storage might return this error code if the implementation requires a mandatory log entry for the requested action and the log storage space is full.

- **PSA_ERROR_INVALID_ARGUMENT:**

The parameters passed to the function are invalid.

Implementations can return this error any time a parameter or combination of parameters are recognized as invalid.

Implementations must not return this error code to indicate that a key identifier is invalid, but must return `PSA_ERROR_INVALID_HANDLE` instead.

- **PSA_ERROR_INVALID_HANDLE:**

The key identifier is not valid.

- **PSA_ERROR_INVALID_PADDING:**

The decrypted padding is incorrect.

Warning:

In some protocols, when decrypting data, it is essential that the behavior of the application does not depend on whether the padding is correct, down to precise timing. Protocols that use authenticated encryption are recommended for use by applications, rather than plain encryption. If the application must perform a decryption of unauthenticated data, the application writer must take care not to reveal whether the padding is invalid.

Implementations must handle padding carefully, aiming to make it impossible for an external observer to distinguish between valid and invalid padding. In particular, it is recommended that the timing of a decryption operation does not depend on the validity of the padding.

- **PSA_ERROR_INVALID_SIGNATURE:**

The signature, MAC or hash is incorrect.

Verification functions return this error if the verification calculations completed successfully, and the value to be verified was determined to be incorrect.

If the value to verify has an invalid size, implementations can return either `PSA_ERROR_INVALID_ARGUMENT` or `PSA_ERROR_INVALID_SIGNATURE`.

- **PSA_ERROR_NOT_PERMITTED:**

The requested action is denied by a policy.

It is recommended that implementations return this error code when the parameters are recognized as valid and supported, and a policy explicitly denies the requested operation.

If a subset of the parameters of a function call identify a forbidden operation, and another subset of the parameters are not valid or not supported, it is unspecified whether the function returns `PSA_ERROR_NOT_PERMITTED`, `PSA_ERROR_NOT_SUPPORTED` or `PSA_ERROR_INVALID_ARGUMENT`.

- **PSA_ERROR_NOT_SUPPORTED:**

The requested operation or a parameter is not supported by this implementation.

It is recommended that implementations return this error code when an enumeration parameter such as a key type, algorithm, etc. is not recognized. If a combination of parameters is recognized and identified as not valid, return `PSA_ERROR_INVALID_ARGUMENT` instead.

- **PSA_ERROR_STORAGE_FAILURE:**

There was a storage failure that might have led to data loss.

This error indicates that some persistent storage could not be read or written by the implementation. It does not indicate the following situations, which have specific error codes:

- A corruption of volatile memory - use `PSA_ERROR_CORRUPTION_DETECTED`.
- A communication error between the cryptoprocessor and its external storage - use `PSA_ERROR_COMMUNICATION_FAILURE`.
- When the storage is in a valid state but is full - use `PSA_ERROR_INSUFFICIENT_STORAGE`.
- When the storage or stored data is corrupted - use `PSA_ERROR_DATA_CORRUPT`.
- When the stored data is not valid - use `PSA_ERROR_DATA_INVALID`.

A storage failure does not indicate that any data that was previously read is invalid. However this previously read data might no longer be readable from storage.

When a storage failure occurs, it is no longer possible to ensure the global integrity of the keystore. Depending on the global integrity guarantees offered by the implementation, access to other data might fail even if the data is still readable but its integrity cannot be guaranteed.

It is recommended to only use this error code to report a permanent storage corruption. However application writers must keep in mind that transient errors while reading the storage might be reported using this error code.

6.2.7 Objects Attributes definitions

Objects Attributes definitions provide bitmasks and functions to manage attributes associated to keys and data objects.

Functions are provided to set and get attribute values.

A key is defined by the following attributes:

- [Key types](#): Defines the type of the key.
- [Permitted algorithm](#): Defines which algorithm(s) can use this key.
- [Usage restriction](#): Defines how the key can be used.
- [Lifetime](#):
 - Persistence: Defines if the key is persistent or volatile (transient).
 - Location: Defines where the object is stored.

6.2.7.1 Key types

Definition of the key type.

Refer to the [Key type encoding](#), the key type is defined as a 16-bit encoding format.

6.2.7.2 Permitted algorithm

Definition of the key permitted algorithm restriction if supported by the subsystem.

6.2.7.3 Usage restriction

Definition of the key usage restriction if supported by the subsystem.

6.2.7.3.1 typedef psa_key_usage_t

type **psa_key_usage_t**

Encoding of permitted usage on a key.

Table 161. Key Usage bitmask value

Value	Define	Description
0x00000004	PSA_KEY_USAGE_CACHE	Permission to cache the key.
0x00000002	PSA_KEY_USAGE_COPY	Permission to copy the key.
0x00000001	PSA_KEY_USAGE_EXPORT	Permission to export the key.
0x00000100	PSA_KEY_USAGE_ENCRYPT	Permission to encrypt a message with the key.
0x00000200	PSA_KEY_USAGE_DECRYPT	Permission to decrypt a message with the key.
0x00000400	PSA_KEY_USAGE_SIGN_MESSAGE	Permission to sign a message with the key.
0x00000800	PSA_KEY_USAGE_VERIFY_MESSAGE	Permission to verify a message signature with the key.
0x00001000	PSA_KEY_USAGE_SIGN_HASH	Permission to sign a message hash with the key.
0x00002000	PSA_KEY_USAGE_VERIFY_HASH	Permission to verify a message hash with the key.
0x00004000	PSA_KEY_USAGE_DERIVE	Permission to derive other keys from this key.
0x00008000	PSA_KEY_USAGE_VERIFY_DERIVATION	Permission to verify the result of a key derivation, including password hashing.

6.2.7.3.2 Functions

6.2.7.3.2.1 psa_set_key_usage_flags

void **psa_set_key_usage_flags**([psa_key_attributes_t](#) *attributes, [psa_key_usage_t](#) usage_flags)

Declare usage flags for a key.

Parameters

- **attributes** ([psa_key_attributes_t](#)*) - [in/out] The attribute object to write to.
- **usage_flags** ([psa_key_usage_t](#)) - [in] The usage flags to write.

Description

Usage flags are part of a key's policy. They encode what kind of operations are permitted on the key.

This function overwrites any usage flags previously set in `attributes`.

Return

void

6.2.7.3.2.2 psa_get_key_usage_flags

[psa_key_usage_t](#) **psa_get_key_usage_flags**(const [psa_key_attributes_t](#) *attributes)

Retrieve the usage flags from key attributes.

Parameters

- **attributes** (const [psa_key_attributes_t](#)*) - [in] The key attribute object to query.

Return

The usage flags stored in the attribute object.

6.2.7.4 Lifetime

Definition of the key object attributes as persistence and location.

6.2.7.4.1 typedef psa_key_lifetime_t

type **psa_key_lifetime_t**

Encoding of key lifetimes.

Table 162. Key lifetime bit field

Bits	Description
[31:8]	Location indicator. The value indicates where the key material is stored. See psa_key_location_t for more information.
[7:0]	Object persistency. See psa_key_persistence_t for more information.

6.2.7.4.2 typedef psa_key_location_t

type **psa_key_location_t**

Encoding of key location indicators.

It's a 24 bits encoding value and only significant for the ELE Secure Subsystem. This encoding value is transported in the upper bytes of the key lifetime attribute ([psa_key_lifetime_t](#)).

Table 163. Key storage location bit field

Bits	Description
[23]	Set to 1 to use this NXP vendor encoding.

Table 163. Key storage location bit field...continued

Bits	Description
[22]	Vendor NXP Identifier.
[21]	NXP's EdgeLock 2GO object.
[20:16]	Reserved must be 0s.
[15]	Object type:
[14:8]	NXP's Secure Enclave storage identifier:
[7:0]	NXP's Secure Enclave identifier. Defined For future use

Example

In case of *EdgeLock 2GO* object, the storage location attribute should be set as follows:

where _ means any value.

In case of *EdgeLock Key Import* object, the storage location attribute should be set as follows:

For other key creation operations (generate, derivation), the storage location attribute can be any value. The ELE Subsystem is not interpreting this attribute. Attribute is stored and returned as-is to the user. It's advice to set it to 0x00000 for compatibility with future versions.

6.2.7.4.3 typedef psa_key_persistence_t

type **psa_key_persistence_t**

Encoding of key persistence levels.

The key persistence level is a property of a key that reflects the extent to which the key is protected against device management events. It's 8-bits encoding value transported in the lower byte of the key lifetime attribute ([psa_key_lifetime_t](#)).

Table 164. Key persistence bit field

Value	Define	Description
0x00	PSA_KEY_PERSISTENCE_VOLATILE	Object is transient. Does not resist a device reset.
0x01	PSA_KEY_PERSISTENCE_DEFAULT	Object is persistent. Resists device reset.
0xFF	PSA_KEY_PERSISTENCE_READ_ONLY	Object is permanent. Can't be deleted.

6.2.7.4.4 Functions

6.2.7.4.4.1 psa_set_key_lifetime

void **psa_set_key_lifetime**([psa_key_attributes_t](#) *attributes, [psa_key_lifetime_t](#) lifetime)

Set the location of a persistent key.

Parameters

- **attributes** ([psa_key_attributes_t](#)*) - [in/out] The attribute object to write to.
- **lifetime** ([psa_key_lifetime_t](#)) - [in] The lifetime for the key.

Description

If this is `PSA_KEY_LIFETIME_VOLATILE`, the key will be volatile, and the key identifier attribute is reset to `PSA_KEY_ID_NULL`.

To make a key persistent, give it a persistent key identifier by using [psa_set_key_id\(\)](#). By default, a key that has a persistent identifier is stored in the default storage area identifier by `PSA_KEY_LIFETIME_PERSISTENT`. Call this function to choose a storage area, or to explicitly declare the key as volatile.

This function does not access storage, it merely stores the given value in the attribute object. The persistent key will be written to storage when the attribute object is passed to a key creation function such as [psa_import_key\(\)](#), [psa_generate_key\(\)](#), [psa_key_derivation_output_key\(\)](#) or [psa_copy_key\(\)](#).

Return

void

6.2.7.4.4.2 psa_get_key_lifetime

[psa_key_lifetime_t](#) `psa_get_key_lifetime(const psa\_key\_attributes\_t *attributes)`

Retrieve the lifetime from key attributes.

Parameters

- **attributes** (const [psa_key_attributes_t](#)*) - [in] The key attribute object to query.

Return

The lifetime value stored in the attribute object.

6.2.7.4.5 Macros

6.2.7.4.5.1 macro PSA_KEY_LIFETIME_FROM_PERSISTENCE_AND_LOCATION

`PSA_KEY_LIFETIME_FROM_PERSISTENCE_AND_LOCATION(persistence, location)`

Construct a lifetime from a persistence level and a location.

Parameters

- **persistence** - [in] The persistence level (value of [typedef psa_key_persistence_t](#)).
- **location** - [in] The location indicator (value of [typedef psa_key_location_t](#)).

Return

The constructed lifetime value.

6.2.7.4.5.2 macro PSA_KEY_LIFETIME_GET_PERSISTENCE

`PSA_KEY_LIFETIME_GET_PERSISTENCE(lifetime)`

Extract the persistence level from a key lifetime.

Parameters

- **lifetime** - [in] The lifetime value to query (value of [typedef psa_key_lifetime_t](#)).

6.2.7.4.5.3 macro PSA_KEY_LIFETIME_IS_VOLATILE**PSA_KEY_LIFETIME_IS_VOLATILE**(lifetime)

Whether a key lifetime indicates that the key is volatile.

Parameters

- **lifetime** - [in] The lifetime value to query (value of [typedef psa_key_lifetime_t](#)).

Return

1 if the key is volatile, otherwise 0.

6.2.7.4.5.4 macro PSA_KEY_LIFETIME_GET_LOCATION**PSA_KEY_LIFETIME_GET_LOCATION**(lifetime)

Extract the location indicator from a key lifetime.

Parameters

- **lifetime** - [in] The lifetime value to query (value of [typedef psa_key_lifetime_t](#)).

6.2.8 Cryptographic Algorithm (psa_algorithm_t) encoding

This chapter intends to detail the algorithm encoding definitions used to describe cryptographic algorithms and their parameters within the PSA API.

The algorithm encoding is a 32-bit value that encapsulates all necessary information to define and restrict cryptographic operations.

The same 32-bit encoding is used throughout the PSA API to specify:

- Cryptographic algorithms (symmetric, asymmetric, hash functions).
- Algorithm modes and parameters.
- Permitted key algorithm when requested by the Security Subsystem.

6.2.8.1 Typedef**6.2.8.1.1 typedef psa_algorithm_t**type **psa_algorithm_t**

Encoding of a cryptographic algorithm.

This is a structured bitfield that identifies the category and type of algorithm. The range of algorithm identifier values is divided as follows:

For algorithms that can be applied to multiple key types, this identifier does not encode the key type. For example, for symmetric ciphers based on a block cipher, [psa_algorithm_t](#) encodes the block cipher mode and the padding mode while the block cipher itself is encoded via [psa_key_type_t](#).

Table 165. Algorithm encoding bit fields

Field	Bits	Description
V	[31]	Vendor additional algorithm identifier when set to 1.
CAT	[31:24]	See Algorithm Category .
S	[23]	For a cipher algorithm, this flag indicates a stream cipher when set to 1. For a key-derivation algorithm, this flag indicates a key-stretching or password-hashing algorithm when set to 1.
B	[22]	Flag to indicate an algorithm built on a block cipher when set to 1.
LEN/T2	[21:16]	LEN is the length of a MAC or AEAD tag, T2 is a key-agreement algorithm sub-type.
T1	[15:8]	Algorithm sub-type for most algorithm categories.
H	[7:0]	Hash algorithm sub-type, also used in any algorithm that is parameterized by a hash.

6.2.8.2 Algorithm Category

Table 166. Algorithm Category value

Value	Category	Description
0x00	None	No algorithm defined. <i>PSA_ALG_NONE</i>
0x02	Hash	See Hash algorithms .
0x03	MAC	Message Authentication Code algorithms.
0x04	Cipher	Symmetric cipher algorithms.
0x05	AEAD	Authenticated Encryption with Associated Data.
0x06	Asymmetric Signature	Asymmetric signature algorithms.
0x07	Asymmetric encryption	Asymmetric encryption algorithms.
0x08	Key derivation	Key derivation algorithms.
0x09	Key agreement	Key-agreement algorithms.
0x0C	Key encapsulation	Key-encapsulation algorithms.
0x0A	PAKE	PAKE algorithms.

6.2.8.3 Hash Algorithms

The table [Hash Algorithm identifiers](#) lists the hash algorithm identifiers supported in the context of the Security Middleware library and Secure Subsystems.

The Hash algorithm encoding follows the algorithm encoding format described in [Algorithm encoding bit fields](#) table, where the category is set to 0x02 (Hash) and the HASH-TYPE field [7:0] contains the hash algorithm sub-type identifier details in the [Hash Algorithm identifiers](#) table below.

Table 167. Hash Algorithm identifiers

Value	HASH-TYPE	Define	Description
0x02000003	0x03	PSA_ALG_MD5	Message Digest 5.
0x02000005	0x05	PSA_ALG_SHA_1	Secure Hash Algorithm 1.
0x02000008	0x08	PSA_ALG_SHA_224	Secure Hash Algorithm 2, 224 bits.

Table 167. Hash Algorithm identifiers...continued

Value	HASH-TYPE	Define	Description
0x02000009	0x09	PSA_ALG_SHA_256	Secure Hash Algorithm 2, 256 bits.
0x0200000A	0x0A	PSA_ALG_SHA_384	Secure Hash Algorithm 2, 384 bits.
0x0200000B	0x0B	PSA_ALG_SHA_512	Secure Hash Algorithm 2, 512 bits.
0x0200000C	0x0C	PSA_ALG_SHA_512_224	SHA-512 truncated to 224 bits.
0x0200000D	0x0D	PSA_ALG_SHA_512_256	SHA-512 truncated to 256 bits.
0x02000010	0x10	PSA_ALG_SHA3_224	Secure Hash Algorithm 3, 224 bits.
0x02000011	0x11	PSA_ALG_SHA3_256	Secure Hash Algorithm 3, 256 bits.
0x02000012	0x12	PSA_ALG_SHA3_384	Secure Hash Algorithm 3, 384 bits.
0x02000013	0x13	PSA_ALG_SHA3_512	Secure Hash Algorithm 3, 512 bits.
0x02000014	0x14	PSA_ALG_SM3	ShangMi 3.
0x02000015	0x15	PSA_ALG_SHAKE256_512	SHAKE256 with 512-bit output.
0x020000FF	0xFF	PSA_ALG_ANY_HASH	Any hash algorithm.

6.2.8.4 MAC Algorithms

The MAC algorithm encoding follows the algorithm encoding format described in [Algorithm encoding bit fields](#) table, as detailed in the following [MAC Algorithm encoding](#) table.

The MAC algorithm identifiers have been extended with NXP vendor-specific identifiers. The [MAC Algorithm identifiers](#) table defines the PSA standard identifiers and the extended identifiers are defined in the [MAC Vendor Algorithm identifiers \(V=1\)](#) table.

Table 168. MAC Algorithm encoding

Field	Bits	Description
V	[31]	Can be 0 or 1: =0, MAC Algorithm identifiers table. =1, MAC Vendor Algorithm identifiers (V=1) table.
CAT	[30:24]	=0x03 (MAC)
S	[23]	=1
B	[22]	See MAC Algorithm identifiers table.
LEN	[21:16]	Truncated MAC length if not 0.
W	[15]	Wildcard permitted algorithm policy: =0 indicates a specific MAC algorithm and MAC length. =1 indicates a wildcard key usage policy, which permits the MAC algorithm with a MAC length at least equal to <i>LEN</i>. <i>LEN</i> can't be 0.
MAC-TYPE	[14:8]	The MAC algorithm type. See MAC Algorithm identifiers table.
HASH-TYPE	[7:0]	Hash algorithm for HMAC (Table Hash Algorithm identifiers), 0 for CMAC.

Table 169. MAC Algorithm identifiers

Value	B	MAC-TYPE	Define	Description
0x038000hh (1)	0	0x00	PSA_ALG_HMAC(hash_alg)	Hash-based Message Authentication Code.
0x03c00100	1	0x01	PSA_ALG_CMAC	Cipher-based Message Authentication Code.
0x03c00200	1	0x02	PSA_ALG_CBC_MAC	CBC-MAC.

1. hh is the hash algorithm identifier as defined in the [Hash Algorithm identifiers](#) table.

The above [MAC Algorithm identifiers](#) table defines the default algorithm identifier, specifying a standard length tag.

PSA_ALG_TRUNCATED_MAC() generates identifiers with non-default LEN values.

PSA_ALG_AT_LEAST_THIS_LENGTH_MAC() generates permitted-algorithm policies with W = 1.

Table 170. MAC Vendor Algorithm identifiers (V=1)

Value	B	MAC-TYPE	Define	Description
0x83C00200	1	0x02	PSA_ALG_VENDOR_CMAC_ATTESTATION	NXP vendor CMAC key attestation.

6.2.8.5 Cipher Algorithms

The Cipher algorithm encoding follows the algorithm encoding format described in [Algorithm encoding bit fields](#) table, as detailed in the following [Cipher Algorithm encoding](#) table.

Table 171. Cipher Algorithm encoding

Field	Bits	Description
V	[31]	=0
CAT	[30:24]	=0x04 (Cipher)
S	[23]	See Cipher Algorithm identifiers table.
B	[22]	See Cipher Algorithm identifiers table.
LEN	[21:16]	=0
C-TYPE	[15:8]	The Cipher algorithm type. See Cipher Algorithm identifiers table.
HASH-TYPE	[7:0]	=0

Table 172. Cipher Algorithm identifiers

Value	S	B	C-TYPE	Define	Description
0x04800100	1	0	0x01	PSA_ALG_STREAM_CIPHER	Stream cipher mode.
0x04C01000	1	1	0x10	PSA_ALG_CTR	Counter Mode.
0x04C01100	1	1	0x11	PSA_ALG_CFB	Cipher Feedback Mode.
0x04C01200	1	1	0x12	PSA_ALG_OFB	Output Feedback Mode.
0x04C01300	1	1	0x13	PSA_ALG_CCM_STAR_NO_TAG	CCM* mode without authentication tag. The block cipher is determined by the key type.

Table 172. Cipher Algorithm identifiers...continued

Value	S	B	C-TYPE	Define	Description
0x04C09300	1	1	0x93	PSA_ALG_CCM_ANY_TAG	CCM* Wildcard. Permits a key to be used with any CCM* algorithm PSA_ALG_CCM_STAR_NO_TAG and AEAD algorithm PSA_ALG_CCM.
0x0440FF00	0	1	0xFF	PSA_ALG_XTS	XEX Tweakable Block Cipher.
0x04404000	0	1	0x40	PSA_ALG_CBC_NO_PADDING	Cipher Block Chaining without padding.
0x04404100	0	1	0x41	PSA_ALG_CBC_PKCS7	Cipher Block Chaining with PKCS#7 padding.
0x04404400	0	1	0x44	PSA_ALG_ECB_NO_PADDING	Electronic Block Chaining without padding.

6.2.8.6 AEAD Algorithms

The AEAD algorithm encoding follows the algorithm encoding format described in [Algorithm encoding bit fields](#) table, as detailed in the following [AEAD Algorithm encoding](#) table.

Table 173. AEAD Algorithm encoding

Field	Bits	Description
V	[31]	=0
CAT	[30:24]	=0x05 (AEAD)
S	[23]	=0
B	[22]	See AEAD Algorithm identifiers table.
LEN	[21:16]	Specifies the output tag length from 1 to 31 bytes.
W	[15]	Wildcard permitted algorithm policy: =0 indicates a specific AEAD algorithm and tag length. =1 indicates a wildcard key usage policy, which permits the AEAD algorithm with a tag length at least equal to <i>LEN</i>. <i>LEN</i> can't be 0.
AEAD-TYPE	[14:8]	The AEAD algorithm type. See AEAD Algorithm identifiers table.
HASH-TYPE	[7:0]	=0

Table 174. AEAD Algorithm identifiers

Value	B	AEAD-TYPE	Define	Description
0x05500100	1	0x01	PSA_ALG_CCM	Counter with CBC-MAC. The block cipher is determined by the key type.
0x05500200	1	0x02	PSA_ALG_GCM	Galois/Counter Mode. The block cipher is determined by the key type.
0x05100500	0	0x05	PSA_ALG_CHACHA20_POLY1305	ChaCha20-Poly1305
0x05100600	0	0x06	PSA_ALG_XCHACHA20_POLY1305	XChaCha20-Poly1305

The above [AEAD Algorithm identifiers](#) table defines the default algorithm identifier, specifying the default tag length for the algorithm.

PSA_ALG_AEAD_WITH_SHORTENED_TAG() generates identifiers with alternative LEN values.

PSA_ALG_AEAD_WITH_AT_LEAST_THIS_LENGTH_TAG() generates wildcard permitted-algorithm policies with W = 1.

6.2.8.7 Asymmetric Signature Algorithms

The Asymmetric Signature algorithm encoding follows the algorithm encoding format described in the [Algorithm encoding bit fields](#) table, as detailed in the following [Asymmetric Signature Algorithm encoding](#) table.

The asymmetric signature algorithm identifiers have been extended with NXP vendor-specific identifiers. The [Signature Algorithm identifiers](#) table defines the PSA standard identifiers and the extended identifiers are defined in the [Signature Vendor Algorithm identifiers \(V=1\)](#) table.

Table 175. Asymmetric Signature Algorithm encoding

Field	Bits	Description
V	[31]	Can be 0 or 1: =0, Signature Algorithm identifiers table. =1, Signature Vendor Algorithm identifiers (V=1) table.
CAT	[30:24]	=0x06 (Asymmetric Signature)
S	[23]	=0
B	[22]	=0
LEN	[21:16]	=0
SIGN-TYPE	[15:8]	The Asymmetric Signature algorithm type. See Signature Algorithm identifiers table.
HASH-TYPE	[7:0]	=0 or HASH-TYPE as defined in the Hash Algorithm identifiers table.

Table 176. Signature Algorithm identifiers

Value	SIGN-TYPE	Define	Description
0x060002hh (1)	0x02	PSA_ALG_RSA_PKCS1V15_SIGN(hash)	PKCS#1 v1.5 signature with hash.
0x06000200	0x02	PSA_ALG_RSA_PKCS1V15_SIGN_RAW	PKCS#1 v1.5 signature no hash.
0x060003hh (1)	0x03	PSA_ALG_RSA_PSS(hash)	RSA PSS signature with hash.
0x060013hh	0x13	PSA_ALG_RSA_PSS_ANY_SALT(hash)	RSA PSS with any salt length.
0x060006hh (1)	0x06	PSA_ALG_ECDSA(hash)	ECDSA signature with hash.
0x06000600	0x06	PSA_ALG_ECDSA_ANY	ECDSA signature without hash restriction.
0x060007hh (1)	0x07	PSA_ALG_DETERMINISTIC_ECDSA(hash)	Deterministic ECDSA (RFC 6979).
0x06000800	0x08	PSA_ALG_PURE_EDDSA	Pure EdDSA signature.

Table 176. Signature Algorithm identifiers...continued

Value	SIGN-TYPE	Define	Description
0x0600090B	0x09	PSA_ALG_ED25519PH	Ed25519ph (pre-hashed Ed25519).
0x06000915	0x09	PSA_ALG_ED448PH	Ed448ph (pre-hashed Ed448).

1. hh is the hash algorithm identifier as defined in the [Hash Algorithm identifiers](#) table.

Table 177. Signature Vendor Algorithm identifiers (V=1)

Value	SIGN-TYPE	Define	Description
0x860006hh (1)	0x06	PSA_ALG_VENDOR_ECDSA_ATTESTATION(hash)	NXP vendor ECDSA key attestation.

1. hh is the hash algorithm identifier as defined in the [Hash Algorithm identifiers](#) table.

6.2.8.8 Asymmetric Encryption Algorithms

The Asymmetric Encryption algorithm encoding follows the algorithm encoding format described in the [Algorithm encoding bit fields](#) table, as detailed in the following [Asymmetric Encryption Algorithm encoding](#) table.

Table 178. Asymmetric Encryption Algorithm encoding

Field	Bits	Description
V	[31]	=0
CAT	[30:24]	=0x07 (Asymmetric Encryption)
S	[23]	=0
B	[22]	=0
LEN	[21:16]	=0
ENC-TYPE	[15:8]	The Asymmetric Encryption algorithm type. See Asymmetric Encryption Algorithm identifiers table.
HASH-TYPE	[7:0]	=0 or HASH-TYPE as defined in the Hash Algorithm identifiers table.

Table 179. Asymmetric Encryption Algorithm identifiers

Value	ENC-TYPE	Define	Description
0x07000200	0x02	PSA_ALG_RSA_PKCS1V15_CRYPT	PKCS#1 v1.5 encryption.
0x070003hh (1)	0x03	PSA_ALG_RSA_OAEP(hash)	RSA OAEP encryption with hash.

1. hh is the hash algorithm identifier as defined in the [Hash Algorithm identifiers](#) table.

6.2.8.9 Key Derivation Algorithms

The Key Derivation algorithm encoding follows the algorithm encoding format described in the [Algorithm encoding bit fields](#) table, as detailed in the following [Key Derivation Algorithm encoding](#) table.

The key derivation algorithm identifiers have been extended with a specific identifier to manage TLS 1.3 key derivation and agreement targeting the ELE subsystem. The [Key Derivation Algorithm identifiers \(V=0\)](#)

table defines the PSA standard identifiers and the extended identifier is defined in the [Key Derivation Vendor Algorithm identifiers \(V=1\)](#) table.

Table 180. Key Derivation Algorithm encoding

Field	Bits	Description
V	[31]	Can be 0 or 1: =0, Key Derivation Algorithm identifiers (V=0) table. =1, Key Derivation Vendor Algorithm identifiers (V=1) table.
CAT	[30:24]	=0x08 (Key Derivation)
S	[23]	See Key Derivation Algorithm identifiers (V=0) table.
B	[22]	=0
LEN	[21:16]	=0
KDF-TYPE	[15:8]	The Key Derivation algorithm type. See Key Derivation Algorithm identifiers (V=0) table.
HASH-TYPE	[7:0]	=0 or HASH-TYPE as defined in the Hash Algorithm identifiers table.

Table 181. Key Derivation Algorithm identifiers (V=0)

Value	S	KDF-TYPE	Define	Description
0x080001hh (1)	0	0x01	PSA_ALG_HKDF(hash)	HMAC-based Key Derivation Function.
0x080004hh (1)	0	0x04	PSA_ALG_HKDF_EXTRACT(hash)	HKDF Extract step only.
0x080005hh (1)	0	0x05	PSA_ALG_HKDF_EXPAND(hash)	HKDF Expand step only.
0x080002hh (1)	0	0x02	PSA_ALG_TLS12_PRF(hash)	TLS 1.2 PRF.
0x080003hh (1)	0	0x03	PSA_ALG_TLS12_PSK_TO_MS(hash)	TLS 1.2 PSK to MasterSecret.
0x08000609	0	0x06	PSA_ALG_TLS12_ECJPAKE_TO_PMS	TLS 1.2 EC J-PAKE to PMS.
0x088001hh (1)	1	0x01	PSA_ALG_PBKDF2_HMAC(hash)	PBKDF2 with HMAC.
0x088002hh (1)	1	0x02	PSA_ALG_PBKDF2_AES_CMAC_PRF_128	PBKDF2 with AES-CMAC-PRF-128.

1. hh is the hash algorithm identifier as defined in the [Hash Algorithm identifiers](#) table.

Table 182. Key Derivation Vendor Algorithm identifiers (V=1)

Value	S	KDF-TYPE	Define	Description
0x88000Dhh (1)	0	0x0D	PSA_ALG_VENDOR_TLS13	TLS 1.3 key derivation and agreement.

1. hh is the hash algorithm identifier as defined in the [Hash Algorithm identifiers](#) table.

6.2.8.10 Key Agreement Algorithms

The Key Agreement algorithm encoding follows the algorithm encoding format described in the [Algorithm encoding bit fields](#) table, as detailed in the following [Key Agreement Algorithm encoding](#) table.

Table 183. Key Agreement Algorithm encoding

Field	Bits	Description
V	[31]	=0
CAT	[30:24]	=0x09 (Key Agreement)
S	[23]	=0
B	[22]	=0
LEN	[21:16]	=0
KA-TYPE	[15:8]	The Key Agreement algorithm type. See Key Agreement Algorithm identifiers table.
HASH-TYPE	[7:0]	=0

Table 184. Key Agreement Algorithm identifiers

Value	KA-TYPE	Define	Description
0x09010000	0x01	PSA_ALG_FFDH	Finite Field Diffie-Hellman.
0x09020000	0x02	PSA_ALG_ECDH	Elliptic Curve Diffie-Hellman.

6.2.8.11 Password-authenticated key exchange (PAKE) Algorithm

The PAKE algorithm encoding follows the algorithm encoding format described in the [Algorithm encoding bit fields](#) table, as detailed in the following [PAKE Algorithm encoding](#) table.

Table 185. PAKE Algorithm encoding

Field	Bits	Description
V	[31]	=0
CAT	[30:24]	=0x0A (PAKE)
S	[23]	=0
B	[22]	=0
LEN	[21:16]	=0
PAKE-TYPE	[15:8]	The PAKE algorithm type. See PAKE Algorithm identifiers table.
HASH-TYPE	[7:0]	=0

Table 186. PAKE Algorithm identifiers

Value	PAKE-TYPE	Define	Description
0x0A0001hh (1)	0x01	PSA_ALG_JPAKE(hash)	J-PAKE.
0x0A0004hh (1)	0x04	PSA_ALG_SPAKE2P_HMAC(hash)	SPAKE2+ with HMAC.
0x0A0005hh (1)	0x05	PSA_ALG_SPAKE2P_CMAC(hash)	SPAKE2+ with CMAC.
0x0A000609	0x06	PSA_ALG_SPAKE2P_Matter	SPAKE2+ for Matter.

1. hh is the hash algorithm identifier as defined in the [Hash Algorithm identifiers](#) table.

6.2.8.12 Algorithm Properties

The PSA Crypto macros provides macros to query algorithm properties:

6.2.8.12.1 Macros

6.2.8.12.1.1 macro `PSA_ALG_IS_AEAD`

`PSA_ALG_IS_AEAD(alg)`

Whether the specified algorithm is an authenticated encryption with associated data (AEAD) algorithm.

Parameters

- **alg** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if **alg** is an AEAD algorithm.
- 0 otherwise.
- Either 0 or 1 if **alg** is not a supported algorithm identifier.

6.2.8.12.1.2 macro `PSA_ALG_IS_AEAD_ON_BLOCK_CIPHER`

`PSA_ALG_IS_AEAD_ON_BLOCK_CIPHER(alg)`

Whether the specified algorithm is an AEAD mode on a block cipher.

Parameters

- **alg** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if **alg** is an AEAD algorithm which is an AEAD mode based on a block cipher.
- 0 otherwise.
- Either 0 or 1 if **alg** is not a supported algorithm identifier.

6.2.8.12.1.3 macro `PSA_ALG_IS_ASYMMETRIC_ENCRYPTION`

`PSA_ALG_IS_ASYMMETRIC_ENCRYPTION(alg)`

Whether the specified algorithm is an asymmetric encryption algorithm, also known as public-key encryption algorithm.

Parameters

- **alg** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if **alg** is an asymmetric encryption algorithm.
- 0 otherwise.
- Either 0 or 1 if **alg** is not a supported algorithm identifier.

6.2.8.12.1.4 macro `PSA_ALG_IS_BLOCK_CIPHER_MAC`

`PSA_ALG_IS_BLOCK_CIPHER_MAC`(*alg*)

Whether the specified algorithm is a MAC algorithm based on a block cipher.

Parameters

- **`alg`** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if `alg` is a MAC algorithm based on a block cipher.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.5 macro `PSA_ALG_IS_CIPHER`

`PSA_ALG_IS_CIPHER`(*alg*)

Whether the specified algorithm is a symmetric cipher algorithm.

Parameters

- **`alg`** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if `alg` is a symmetric cipher algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.6 macro `PSA_ALG_IS_DETERMINISTIC_ECDSA`

`PSA_ALG_IS_DETERMINISTIC_ECDSA`(*alg*)

Whether the specified algorithm is deterministic ECDSA.

Parameters

- **`alg`** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

See also [PSA_ALG_IS_ECDSA](#) and [PSA_ALG_IS_RANDOMIZED_ECDSA](#).

Return

- 1 if `alg` is a deterministic ECDSA algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.7 macro `PSA_ALG_IS_ECDH`

`PSA_ALG_IS_ECDH`(*alg*)

Whether the specified algorithm is an elliptic curve Diffie-Hellman algorithm.

Parameters

- **alg** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

This includes the raw elliptic curve Diffie-Hellman algorithm as well as elliptic curve Diffie-Hellman followed by any supporter key derivation algorithm.

Return

- 1 if **alg** is an elliptic curve Diffie-Hellman algorithm.
- 0 otherwise.
- Either 0 or 1 if **alg** is not a supported key agreement algorithm identifier.

6.2.8.12.1.8 macro PSA_ALG_IS_ECDSA**PSA_ALG_IS_ECDSA(*alg*)**

Whether the specified algorithm is ECDSA.

Parameters

- **alg** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if **alg** is an ECDSA algorithm.
- 0 otherwise.
- Either 0 or 1 if **alg** is not a supported algorithm identifier.

6.2.8.12.1.9 macro PSA_ALG_IS_FFDH**PSA_ALG_IS_FFDH(*alg*)**

Whether the specified algorithm is a finite field Diffie-Hellman algorithm.

Parameters

- **alg** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

This includes the raw finite field Diffie-Hellman algorithm as well as finite-field Diffie-Hellman followed by any supporter key derivation algorithm.

Return

- 1 if **alg** is a finite field Diffie-Hellman algorithm.
- 0 otherwise.
- Either 0 or 1 if **alg** is not a supported key agreement algorithm identifier.

6.2.8.12.1.10 macro PSA_ALG_IS_HASH**PSA_ALG_IS_HASH(*alg*)**

Whether the specified algorithm is a hash algorithm.

Parameters

- **alg** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

See Hash algorithms for a list of defined hash algorithms.

Return

- 1 if **alg** is a hash algorithm.
- 0 otherwise.
- Either 0 or 1 if **alg** is not a supported algorithm identifier.

6.2.8.12.1.11 macro PSA_ALG_IS_HASH_AND_SIGN**PSA_ALG_IS_HASH_AND_SIGN(*alg*)**

Whether the specified algorithm is a hash-and-sign algorithm that signs exactly the hash value.

Parameters

- **alg** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

This macro identifies algorithms that can be used with [psa_sign_hash\(\)](#) that use the exact message hash value as an input the signature operation. This excludes hash-and-sign algorithms that require a encoded or modified hash for the signature step in the algorithm, such as PSA_ALG_RSA_PKCS1V15_SIGN_RAW.

Return

- 1 if **alg** is a hash-and-sign algorithm that signs exactly the hash value.
- 1 0 otherwise.
- Either 0 or 1 if **alg** is not a supported algorithm identifier.

6.2.8.12.1.12 macro PSA_ALG_IS_HKDF**PSA_ALG_IS_HKDF(*alg*)**

Whether the specified algorithm is an HKDF algorithm.

Parameters

- **alg** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

HKDF is a family of key derivation algorithms that are based on a hash function and the HMAC construction.

Return

- 1 if **alg** is an HKDF algorithm.
- 0 otherwise.

- Either 0 or 1 if `alg` is not a supported key derivation algorithm identifier.

6.2.8.12.1.13 macro `PSA_ALG_IS_JPAKE`

`PSA_ALG_IS_JPAKE(alg)`

Whether the specified algorithm is a J-PAKE algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef `psa\_algorithm\_t`](#)).

Return

- 1 if `alg` is a J-PAKE algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported PAKE algorithm identifier.

6.2.8.12.1.14 macro `PSA_ALG_IS_VENDOR_TLS13`

`PSA_ALG_IS_VENDOR_TLS13(alg)`

Whether the specified algorithm is a TLS1.3 algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef `psa\_algorithm\_t`](#)).

Return

- 1 if `alg` is a TLS1.3 algorithm.
- 0 otherwise.

6.2.8.12.1.15 macro `PSA_ALG_IS_HKDF_EXTRACT`

`PSA_ALG_IS_HKDF_EXTRACT(alg)`

Whether the specified algorithm is an HKDF-Extract algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef `psa\_algorithm\_t`](#)).

Return

- 1 if `alg` is an HKDF-Extract algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported key derivation algorithm identifier.

6.2.8.12.1.16 macro `PSA_ALG_IS_HKDF_EXPAND`

`PSA_ALG_IS_HKDF_EXPAND(alg)`

Whether the specified algorithm is an HKDF-Expand algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef `psa\_algorithm\_t`](#)).

Return

- 1 if `alg` is an HKDF-Expand algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported key derivation algorithm identifier.

6.2.8.12.1.17 macro `PSA_ALG_IS_HMAC`

`PSA_ALG_IS_HMAC(alg)`

Whether the specified algorithm is an HMAC algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef `psa\_algorithm\_t`](#)).

Description

HMAC is a family of MAC algorithms that are based on a hash function.

Return

- 1 if `alg` is an HMAC algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.18 macro `PSA_ALG_IS_KEY_AGREEMENT`

`PSA_ALG_IS_KEY_AGREEMENT(alg)`

Whether the specified algorithm is a key agreement algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef `psa\_algorithm\_t`](#)).

Return

- 1 if `alg` is a key agreement algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.19 macro `PSA_ALG_IS_PAKE`

`PSA_ALG_IS_PAKE(alg)`

Whether the specified algorithm is a password-authenticated key exchange.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef `psa\_algorithm\_t`](#)).

Return

- 1 if `alg` is a password-authenticated key exchange (PAKE) algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.20 macro PSA_ALG_IS_KEY_ENCAPSULATION

PSA_ALG_IS_KEY_ENCAPSULATION(*alg*)

Whether the specified algorithm is a key-encapsulation algorithm.

Parameters

- **alg** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if **alg** is a key encapsulation algorithm.
- 0 otherwise.
- Either 0 or 1 if **alg** is not a supported algorithm identifier.

6.2.8.12.1.21 macro PSA_ALG_IS_KEY_DERIVATION

PSA_ALG_IS_KEY_DERIVATION(*alg*)

Whether the specified algorithm is a key derivation algorithm.

Parameters

- **alg** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if **alg** is a key derivation algorithm.
- 0 otherwise.
- Either 0 or 1 if **alg** is not a supported algorithm identifier.

6.2.8.12.1.22 macro PSA_ALG_IS_KEY_DERIVATION_STRETCHING

PSA_ALG_IS_KEY_DERIVATION_STRETCHING(*alg*)

Whether the specified algorithm is a key-stretching or password-hashing algorithm.

Parameters

- **alg** - [in] An algorithm identifier: a value of [typedef psa_algorithm_t](#).

Description

A key-stretching or password-hashing algorithm is a key derivation algorithm that is suitable for use with a low-entropy secret such as a password. Equivalently, it's a key derivation algorithm that uses a PSA_KEY_DERIVATION_INPUT_PASSWORD input step.

Return

- 1 if **alg** is a key-stretching or password-hashing algorithm.
- 0 otherwise.
- Either 0 or 1 if **alg** is not a supported key derivation algorithm identifier.

6.2.8.12.1.23 macro PSA_ALG_IS_MAC

PSA_ALG_IS_MAC(*alg*)

Whether the specified algorithm is a MAC algorithm.

Parameters

- **alg** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if **alg** is a MAC algorithm.
- 0 otherwise.

6.2.8.12.1.24 macro PSA_ALG_IS_MAC_TRUNCATED**PSA_ALG_IS_MAC_TRUNCATED(*alg*)**

Whether the specified algorithm is a MAC truncated algorithm.

Parameters

- **alg** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if **alg** is a MAC truncated algorithm.
- 0 otherwise.

6.2.8.12.1.25 macro PSA_ALG_IS_PBKDF2_HMAC**PSA_ALG_IS_PBKDF2_HMAC(*alg*)**

Whether the specified algorithm is a PBKDF2-HMAC algorithm.

Parameters

- **alg** - [in] An algorithm identifier: a value of [typedef psa_algorithm_t](#).

Return

- 1 if **alg** is a PBKDF2-HMAC algorithm.
- 0 otherwise.
- Either 0 or 1 if **alg** is not a supported key derivation algorithm identifier.

6.2.8.12.1.26 macro PSA_ALG_IS_RANDOMIZED_ECDSA**PSA_ALG_IS_RANDOMIZED_ECDSA(*alg*)**

Whether the specified algorithm is randomized ECDSA.

Parameters

- **alg** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

See also [PSA_ALG_IS_ECDSA\(\)](#) and [PSA_ALG_IS_DETERMINISTIC_ECDSA\(\)](#).

Return

- 1 if `alg` is a randomized ECDSA algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.27 macro `PSA_ALG_IS_STANDALONE_KEY_AGREEMENT`

`PSA_ALG_IS_STANDALONE_KEY_AGREEMENT(alg)`

Whether the specified algorithm is a standalone key agreement algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

A standalone key agreement algorithm is one that does not specify a key derivation function. Usually, standalone key agreement algorithms are constructed directly with a `PSA_ALG_xxx` macro while combined key agreement algorithms are constructed with `PSA_ALG_KEY_AGREEMENT()`.

The standalone key agreement algorithm can be extracted from a combined key agreement algorithm identifier using `PSA_ALG_KEY_AGREEMENT_GET_BASE()`.

Return

- 1 if `alg` is a standalone key agreement algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.28 macro `PSA_ALG_IS_RAW_KEY_AGREEMENT`

`PSA_ALG_IS_RAW_KEY_AGREEMENT(alg)`

Whether the specified algorithm is a raw key agreement algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

Warning:

Deprecated since version 1.2. Use `PSA_ALG_IS_STANDALONE_KEY_AGREEMENT()` instead.

6.2.8.12.1.29 macro `PSA_ALG_IS_RSA_OAEP`

`PSA_ALG_IS_RSA_OAEP(alg)`

Whether the specified algorithm is an RSA OAEP encryption algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if `alg` is an RSA OAEP algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.30 macro `PSA_ALG_IS_RSA_PKCS1V15_SIGN`

`PSA_ALG_IS_RSA_PKCS1V15_SIGN(alg)`

Whether the specified algorithm is an RSA PKCS#1 v1.5 signature algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if `alg` is an RSA PKCS#1 v1.5 signature algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.31 macro `PSA_ALG_IS_RSA_PSS`

`PSA_ALG_IS_RSA_PSS(alg)`

Whether the specified algorithm is an RSA PSS signature algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if `alg` is an RSA PSS signature algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.32 macro `PSA_ALG_IS_RSA_PSS_ANY_SALT`

`PSA_ALG_IS_RSA_PSS_ANY_SALT(alg)`

Whether the specified algorithm is an RSA PSS signature algorithm that permits any salt length.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

An RSA PSS signature algorithm that permits any salt length is constructed using `PSA_ALG_RSA_PSS_ANY_SALT()`. See also [PSA_ALG_IS_RSA_PSS\(\)](#) and [PSA_ALG_IS_RSA_PSS_STANDARD_SALT\(\)](#).

Return

- 1 if `alg` is an RSA PSS signature algorithm that permits any salt length.

- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.33 macro `PSA_ALG_IS_RSA_PSS_STANDARD_SALT`

`PSA_ALG_IS_RSA_PSS_STANDARD_SALT(alg)`

Whether the specified algorithm is an RSA PSS signature algorithm that requires the standard salt length.

Parameters

- `alg` - [in] An algorithm identifier: a value of [typedef `psa\_algorithm\_t`](#).

Description

An RSA PSS signature algorithm that requires the standard salt length is constructed using `PSA_ALG_RSA_PSS()`.

See also [PSA_ALG_IS_RSA_PSS\(\)](#) and [PSA_ALG_IS_RSA_PSS_ANY_SALT\(\)](#).

Return

- 1 if `alg` is an RSA PSS signature algorithm that requires the standard salt length.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.34 macro `PSA_ALG_IS_SIGN`

`PSA_ALG_IS_SIGN(alg)`

Whether the specified algorithm is an asymmetric signature algorithm, also known as public-key signature algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef `psa\_algorithm\_t`](#)).

Return

- 1 if `alg` is an asymmetric signature algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.35 macro `PSA_ALG_IS_SIGN_HASH`

`PSA_ALG_IS_SIGN_HASH(alg)`

Whether the specified algorithm is a signature algorithm that can be used with [psa_sign_hash\(\)](#) and [psa_verify_hash\(\)](#).

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef `psa\_algorithm\_t`](#)).

Return

- 1 if `alg` is a signature algorithm that can be used to sign a hash.
- 0 if `alg` is a signature that can only be used to sign a message.

- 0 if `alg` is not a signature algorithm.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.36 macro `PSA_ALG_IS_SIGN_MESSAGE`

`PSA_ALG_IS_SIGN_MESSAGE(alg)`

Whether the specified algorithm is a signature algorithm that can be used with [psa_sign_message\(\)](#) and [psa_verify_message\(\)](#).

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if `alg` is a signature algorithm that can be used to sign a message.
- 0 if `alg` is a signature algorithm that can only be used to sign an already-calculated hash.
- 0 if `alg` is not a signature algorithm.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.37 macro `PSA_ALG_IS_SP800_108_COUNTER_HMAC`

`PSA_ALG_IS_SP800_108_COUNTER_HMAC(alg)`

Whether the specified algorithm is a key derivation algorithm constructed using `PSA_ALG_SP800_108_COUNTER_HMAC(hash_alg)`.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

Warning:

Not supported.

Return

- 1 if `alg` is a key derivation algorithm constructed using `PSA_ALG_SP800_108_COUNTER_HMAC()`.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported key derivation algorithm identifier.

6.2.8.12.1.38 macro `PSA_ALG_IS_STREAM_CIPHER`

`PSA_ALG_IS_STREAM_CIPHER(alg)`

Whether the specified algorithm is a stream cipher.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

A stream cipher is a symmetric cipher that encrypts or decrypts messages by applying a bitwise-xor with a stream of bytes that is generated from a key.

Return

- 1 if `alg` is a stream cipher algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported algorithm identifier or if it is not a symmetric cipher algorithm.

6.2.8.12.1.39 macro `PSA_ALG_IS_TLS12_PRF`

`PSA_ALG_IS_TLS12_PRF(alg)`

Whether the specified algorithm is a TLS-1.2 PRF algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef `psa\_algorithm\_t`](#)).

Return

- 1 if `alg` is a TLS-1.2 PRF algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported key derivation algorithm identifier.

6.2.8.12.1.40 macro `PSA_ALG_IS_TLS12_PSK_TO_MS`

`PSA_ALG_IS_TLS12_PSK_TO_MS(alg)`

Whether the specified algorithm is a TLS-1.2 PSK to MS algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef `psa\_algorithm\_t`](#)).

Return

- 1 if `alg` is a TLS-1.2 PSK to MS algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported key derivation algorithm identifier.

6.2.8.12.1.41 macro `PSA_ALG_IS_SPAKE2P`

`PSA_ALG_IS_SPAKE2P(alg)`

Whether the specified algorithm is a SPAKE2+ algorithm.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef `psa\_algorithm\_t`](#)).

Description

Warning:

Not supported.

SPAKE2+ algorithms are constructed using `PSA_ALG_SPAKE2P_HMAC(hash_alg)`, `PSA_ALG_SPAKE2P_CMAC(hash_alg)`, or `PSA_ALG_SPAKE2P_MATTER`.

Return

- 1 if `alg` is a SPAKE2+ algorithm.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported PAKE algorithm identifier.

6.2.8.12.1.42 macro `PSA_ALG_IS_SPAKE2P_HMAC`

`PSA_ALG_IS_SPAKE2P_HMAC(alg)`

Whether the specified algorithm is a SPAKE2+ algorithm that uses a HMAC-based key confirmation.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

Warning:

Not supported.

SPAKE2+ algorithms, using HMAC-based key confirmation, are constructed using `PSA_ALG_SPAKE2P_HMAC(hash_alg)`.

Return

- 1 if `alg` is a SPAKE2+ algorithm that uses a HMAC-based key confirmation.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported PAKE algorithm identifier.

6.2.8.12.1.43 macro `PSA_ALG_IS_SPAKE2P_CMAC`

`PSA_ALG_IS_SPAKE2P_CMAC(alg)`

Whether the specified algorithm is a SPAKE2+ algorithm that uses a CMAC-based key confirmation.

Parameters

- `alg` - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

Warning:

Not supported.

SPAKE2+ algorithms, using CMAC-based key confirmation, are constructed using `PSA_ALG_SPAKE2P_CMAC(hash_alg)`.

Return

- 1 if `alg` is a SPAKE2+ algorithm that uses a CMAC-based key confirmation.
- 0 otherwise.
- Either 0 or 1 if `alg` is not a supported PAKE algorithm identifier.

6.2.8.12.1.44 macro `PSA_ALG_IS_WILDCARD`

`PSA_ALG_IS_WILDCARD`(*alg*)

Whether the specified algorithm encoding is a wildcard.

Parameters

- **`alg`** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Description

Wildcard algorithm values can only be used to set the permitted algorithm field in a key policy, wildcard values cannot be used to perform an operation.

See `PSA_ALG_ANY_HASH` for example of how a wildcard algorithm can be used in a key policy.

Return

- 1 if `alg` is a wildcard algorithm encoding.
- 0 if `alg` is a non-wildcard algorithm encoding that is suitable for an operation.
- Either 0 or 1 if `alg` is not a supported algorithm identifier.

6.2.8.12.1.45 macro `PSA_ALG_IS_VENDOR_ECDSA_ATTESTATION`

`PSA_ALG_IS_VENDOR_ECDSA_ATTESTATION`(*alg*)

Whether the specified algorithm is a vendor ECDSA attestation algorithm.

Parameters

- **`alg`** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if `alg` is a vendor ECDSA attestation algorithm.
- 0 otherwise.

6.2.8.12.1.46 macro `PSA_ALG_IS_VENDOR_CMAC_ATTESTATION`

`PSA_ALG_IS_VENDOR_CMAC_ATTESTATION`(*alg*)

Whether the specified algorithm is a vendor CMAC attestation algorithm.

Parameters

- **`alg`** - [in] An algorithm identifier (value of [typedef psa_algorithm_t](#)).

Return

- 1 if `alg` is a vendor CMAC attestation algorithm.
- 0 otherwise.

6.2.8.13 Composite Algorithm

Some algorithms can be composed with a hash algorithm. The composite algorithm is constructed by combining a base algorithm with a hash algorithm. The following [PSA_ALG_GET_HASH](#) macro is used to extract the hash algorithm embedded within the composite algorithm's encoding.

6.2.8.13.1 Macros

6.2.8.13.1.1 macro `PSA_ALG_GET_HASH`

`PSA_ALG_GET_HASH`(*alg*)

Get the hash used by a composite algorithm.

Parameters

- **`alg`** - [in] An algorithm identifier (value of [typedef `psa\_algorithm\_t`](#)).

Description

The following composite algorithms require a hash algorithm:

- `PSA_ALG_DETERMINISTIC_ECDSA()`
- `PSA_ALG_ECDSA()`
- `PSA_ALG_HKDF()`
- `PSA_ALG_HKDF_EXPAND()`
- `PSA_ALG_HKDF_EXTRACT()`
- `PSA_ALG_HMAC()`
- `PSA_ALG_JPAKE()`
- `PSA_ALG_PDKDF2_HMAC()`
- `PSA_ALG_RSA_OAEP()`
- `PSA_ALG_RSA_PKCS1V15_SIGN()`
- `PSA_ALG_RSA_PSS()`
- `PSA_ALG_RSA_PSS_ANY_SALT()`
- `PSA_ALG_SP800_108_COUNTER_HMAC()`
- `PSA_ALG_SPAKE2P_CMAC()`
- `PSA_ALG_SPAKE2P_HMAC()`
- `PSA_ALG_TLS12_PRF()`
- `PSA_ALG_TLS12_PSK_TO_MS()`
- `PSA_ALG_VENDOR_TLS13()`
- `PSA_ALG_VENDOR_ECDSA_ATTESTATION()`

Return

The underlying hash algorithm if `alg` is a composite algorithm that uses a hash algorithm. `PSA_ALG_NONE` if `alg` is not a composite algorithm that uses a hash.

6.2.9 Key type encoding

The key types are 16-bit encoded values. This chapter describes how this bit field encoded value is structured and which key types are supported.

6.2.9.1 Typedef

6.2.9.1.1 typedef `psa_key_type_t`

type **`psa_key_type_t`**

Encoding of a key type.

The key type is a 16-bit encoded value that specifies the cryptographic key type.

Table 187. Key type bit field

Field	Bits	Description
V	[15]	Flag to indicate an vendor defined key type (V=1).
A	[14]	Flag to indicate an asymmetric key type (A=1).
CAT	[13:12]	Key type category. See Key type categories .
category-specific type	[11:1]	The meaning of this field is specific to each key type category.
P	[0]	Parity bit. Valid key type values have even parity.

6.2.9.2 Key type categories

The key type is further categorized into the following categories:

Table 188. Key type categories

Category	A	CAT	Description
None	0	0	No key type. See PSA_KEY_TYPE_NONE (0x0000).
Raw data	0	1	Unstructured raw data. See Raw key encoding .
Symmetric	0	2	Symmetric keys. See Symmetric key encoding .
Asymmetric public key	1	0	Asymmetric public keys. See Asymmetric key encoding .
Asymmetric keypair	1	3	Asymmetric keypairs. See Asymmetric key encoding .

6.2.9.2.1 Raw key encoding

The raw key encoding category (CAT=1, A=0) is used for unstructured raw data keys. All PSA raw key type are not supported in this implementation.

Table 189. Raw key encoding

Field	Bits	Details
V	[15]	=0
A	[14]	=0
CAT	[13:12]	=1
RAW-TYPE	[11:8]	See the Raw key types table
SUB-TYPE	[7:1]	
P	[0]	

Table 190. Raw key types

Raw Key Type	RAW-TYPE	SUB-TYPE	P	Name	Value
Raw data	0	0	1	PSA_KEY_TYPE_RAW_DATA	0x1001
HMAC	1	0	0	PSA_KEY_TYPE_HMAC	0x1100
Derive secret	2	0	0	PSA_KEY_TYPE_DERIVE	0x1200
Password	2	1	1	PSA_KEY_TYPE_PASSWORD	0x1203
Password hash	2	2	1	PSA_KEY_TYPE_PASSWORD_HASH	0x1205

Table 190. Raw key types...continued

Raw Key Type	RAW-TYPE	SUB-TYPE	P	Name	Value
Derivation Pepper	2	3	0	PSA_KEY_TYPE_PEPPER	0x1206

6.2.9.2.2 Symmetric key encoding

The symmetric key encoding category (CAT=2, A=0) is used for symmetric keys. All PSA symmetric key type are not supported in this implementation.

Table 191. Symmetric key encoding

Field	Bits	Details
V	[15]	=0
A	[14]	=0
CAT	[13:12]	=2
Not used	[11]	=0
BLK	[10:8]	Block size for the cipher algorithm (2 ^{BLK}).
SYM-TYPE	[7:1]	See the Symmetric key types table.
P	[0]	

Table 192. Symmetric key types

Symmetric Key Type	BLK	SYM-TYPE	P	Name	Value
ARC4	0	1	0	PSA_KEY_TYPE_ARC4	0x2002
ChaCha20	0	2	0	PSA_KEY_TYPE_CHACHA20	0x2004
XChaCha20	0	3	1	PSA_KEY_TYPE_XCHACHA20	0x2007
DES	3	0	1	PSA_KEY_TYPE_DES	0x2301
AES	4	0	0	PSA_KEY_TYPE_AES	0x2400
Camellia	4	1	1	PSA_KEY_TYPE_CAMELLIA	0x2403
SM4	4	2	1	PSA_KEY_TYPE_SM4	0x2405
ARIA	4	3	0	PSA_KEY_TYPE_ARIA	0x2406

6.2.9.2.3 Asymmetric key encoding

The asymmetric key encoding category (CAT=0 or CAT=3, A=1) is used for asymmetric keys and keypairs. All PSA asymmetric key types are not supported in this implementation.

Table 193. Asymmetric key encoding

Field	Bits	Details
V	[15]	=0
A	[14]	=1
PAIR	[13:12]	=0 for public, =3 for a keypair.
ASYM-TYPE	[11:7]	See the Asymmetric key sub-type table.
FAMILY	[6:1]	Depends on the ASYM-TYPE value. More details

Table 193. Asymmetric key encoding...continued

Field	Bits	Details
P	[0]	for each asymmetric key sub-type.

Table 194. Asymmetric key sub-type

Asymmetric Key Type	ASYM-TYPE	Details
RSA	0	See the RSA key family .
ECC	2	See the ECC key family .
Diffie-Hellman	4	See the Diffie-Hellman key family .
SPAKE2+	8	See the SPAKE2+ key family .

RSA key family

The RSA key type encoding uses ASYM-TYPE=0. The FAMILY and P fields are used as follows:

Table 195. Asymmetric RSA key family

Key family	PAIR	FAMILY	P	Name	Value
RSA Public key	0	0	1	PSA_KEY_TYPE_RSA_PUBLIC_KEY	0x4001
RSA Keypair	3	0	1	PSA_KEY_TYPE_RSA_KEY_PAIR	0x7001

Elliptic Curve key family

The Elliptic Curve (ECC) key type encoding uses ASYM-TYPE=2. The FAMILY and P fields are used as follows:

Table 196. Asymmetric Elliptic curve key family

Key family	FAMILY	P	Name	Value / Key	
				Public	Keypair
SECP K1	0x0B	1	PSA_ECC_FAMILY_SECP_K1	0x4117	0x7117
SECP R1	0x09	0	PSA_ECC_FAMILY_SECP_R1	0x4112	0x7112
SECP R2	0x0D	1	PSA_ECC_FAMILY_SECP_R2	0x411B	0x711B
SECT K1	0x13	1	PSA_ECC_FAMILY_SECT_K1	0x4127	0x7127
SECT R1	0x11	0	PSA_ECC_FAMILY_SECT_R1	0x4122	0x7122
SECT R2	0x15	1	PSA_ECC_FAMILY_SECT_R2	0x412B	0x712B
BRAINPOOL-P R1	0x18	0	PSA_ECC_FAMILY_BRAINPOOL_P_R1	0x4130	0x7130
FRP	0x19	1	PSA_ECC_FAMILY_FRP	0x4133	0x7133
Montgomery	0x20	1	PSA_ECC_FAMILY_MONTGOMERY	0x4141	0x7141
Twisted Edwards	0x21	0	PSA_ECC_FAMILY_TWISTED_EDWARDS	0x4142	0x7142

The asymmetric elliptic curve key type value is constructed from the elliptic curve family using either PSA_KEY_TYPE_ECC_PUBLIC_KEY(family) or PSA_KEY_TYPE_ECC_KEY_PAIR(family).

Diffie-Hellman key family

The Diffie-Hellman key type encoding uses ASYM-TYPE=4. The FAMILY and P fields are used as follows:

Table 197. Asymmetric Diffie-Hellman key family

Key family	FAMILY	P	Name	Value / Key	
				Public	Keypair
RFC7919	0x01	1	PSA_DH_FAMILY_RFC7919	0x4203	0x7203

The asymmetric Diffie-Hellman key type value is constructed from the Diffie-Hellman family using either `PSA_KEY_TYPE_DH_PUBLIC_KEY(family)` or `PSA_KEY_TYPE_DH_KEY_PAIR(family)`.

SPAKE2+ key family

The SPAKE2+ key type encoding uses `ASYM-TYPE=8`. The FAMILY and P fields are used as follows:

Table 198. Asymmetric SPAKE2+ key family

Key family	FAMILY	P	Name	Value / Key	
				Public	Keypair
SECP R1	0x09	0	PSA_ECC_FAMILY_SECP_R1	0x4412	0x7412
Twisted Edwards	0x21	0	PSA_ECC_FAMILY_TWISTED_EDWARDS	0x4442	0x7442

The asymmetric SPAKE2+ key type value is constructed from the elliptic curve family using either `PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY(family)` or `PSA_KEY_TYPE_SPAKE2P_KEY_PAIR(family)`.

6.2.9.3 Functions and Macros

6.2.9.3.1 Functions

6.2.9.3.1.1 `psa_set_key_type`

```
void psa_set_key_type(psa\_key\_attributes\_t *attributes, psa\_key\_type\_t type)
```

Declare the type of a key.

Parameters

- **attributes** ([psa_key_attributes_t](#)*) - [in/out] The attribute object to write to.
- **type** ([psa_key_type_t](#)) - [in] The key type to write.

Description

This function overwrites any key type previously set in `attributes`.

If the `type` is `PSA_KEY_TYPE_NONE`, the key type in `attributes` becomes unspecified.

Return

void

6.2.9.3.1.2 `psa_get_key_type`

```
psa\_key\_type\_t psa_get_key_type(const psa\_key\_attributes\_t *attributes)
```

Retrieve the key type from key attributes.

Parameters

- **attributes** (const [psa_key_attributes_t](#)*) - [in] The key attribute object to query.

Return

The key type stored in the attribute object.

6.2.9.3.2 Key categories

6.2.9.3.2.1 macro PSA_KEY_TYPE_IS_ASYMMETRIC

PSA_KEY_TYPE_IS_ASYMMETRIC(*type*)

Whether a key type is asymmetric: either a key pair or a public key.

Parameters

- **type** - [in] A key type (value of [typedef psa_key_type_t](#)).

Description

See [Asymmetric Keys](#) for a list of asymmetric key types.

6.2.9.3.2.2 macro PSA_KEY_TYPE_IS_KEY_PAIR

PSA_KEY_TYPE_IS_KEY_PAIR(*type*)

Whether a key type is a key pair containing a private part and a public part.

Parameters

- **type** - [in] A key type (value of [typedef psa_key_type_t](#)).

6.2.9.3.2.3 macro PSA_KEY_TYPE_IS_PUBLIC_KEY

PSA_KEY_TYPE_IS_PUBLIC_KEY(*type*)

Whether a key type is the public part of a key pair.

Parameters

- **type** - [in] A key type (value of [typedef psa_key_type_t](#)).

6.2.9.3.2.4 macro PSA_KEY_TYPE_IS_UNSTRUCTURED

PSA_KEY_TYPE_IS_UNSTRUCTURED(*type*)

Whether a key type is an unstructured array of bytes.

Parameters

- **type** - [in] A key type (value of [typedef psa_key_type_t](#)).

Description

This encompasses both symmetric keys and non-key data.

See [Symmetric keys](#) for a list of symmetric key types.

6.2.9.3.3 Symmetric Keys

6.2.9.3.3.1 macro `PSA_KEY_TYPE_AES`

`PSA_KEY_TYPE_AES()`

Key for a cipher, AEAD or MAC algorithm based on the AES block cipher. .. [[symmetric, cipher, aead, mac]]

Description

The size of the key is related to the AES algorithm variant. For algorithms except the XTS block cipher mode, the following key sizes are used:

- AES-128 uses a 16-byte key : `key_bits = 128`
- AES-192 uses a 24-byte key : `key_bits = 192`
- AES-256 uses a 32-byte key : `key_bits = 256`

For the XTS block cipher mode (`PSA_ALG_XTS`), the following key sizes are used:

- AES-128-XTS uses two 16-byte keys : `key_bits = 256`
- AES-192-XTS uses two 24-byte keys : `key_bits = 384`
- AES-256-XTS uses two 32-byte keys : `key_bits = 512`

Compatible algorithms:

- `PSA_ALG_CBC_MAC()`
- `PSA_ALG_CMAC()`
- `PSA_ALG_CTR()`
- `PSA_ALG_CFB()`
- `PSA_ALG_OFB()`
- `PSA_ALG_XTS()`
- `PSA_ALG_CBC_NO_PADDING()`
- `PSA_ALG_CBC_PKCS7()`
- `PSA_ALG_ECB_NO_PADDING()`
- `PSA_ALG_CCM()`
- `PSA_ALG_GCM()`
- `PSA_ALG_SP800_108_COUNTER_CMAC()`

6.2.9.3.3.2 macro `PSA_KEY_TYPE_DERIVE`

`PSA_KEY_TYPE_DERIVE()`

A secret for key derivation.

Description

This key type is for high-entropy secrets only. For low-entropy secrets, `PSA_KEY_TYPE_PASSWORD` should be used instead.

These keys can be used in the `PSA_KEY_DERIVATION_INPUT_SECRET` or `PSA_KEY_DERIVATION_INPUT_PASSWORD` input step of key-derivation algorithms.

The key policy determines which key derivation algorithm the key can be used for.

The bit size of a secret for key derivation must be a non-zero multiple of 8. The maximum size of a secret for key derivation is function of the key derivation algorithm.

Compatible algorithms:

- PSA_ALG_HKDF() (input)
- PSA_ALG_HKDF_EXPAND() (input)
- PSA_ALG_HKDF_EXTRACT() (input)
- PSA_ALG_TLS12_PRF() (input)
- PSA_ALG_TLS12_PSK_TO_MS() (input)

6.2.9.3.3 macro PSA_KEY_TYPE_DES**PSA_KEY_TYPE_DES()**

Key for a cipher or MAC algorithm based on DES or 3DES (Triple-DES). .. [[symmetric, cipher, mac]]

Description

The size of the key determines which DES algorithm is used:

- Single DES uses an 8-byte key : key_bits = 64
- 2-key 3DES uses a 16-byte key : key_bits = 128
- 3-key 3DES uses a 24-byte key : key_bits = 192

Warning:

Single DES and 2-key 3DES are weak and strongly deprecated and are only recommended for decrypting legacy data.

3-key 3DES is weak and deprecated and is only recommended for use in legacy protocols.

Compatible algorithms:

- PSA_ALG_CBC_MAC()
- PSA_ALG_CMAC()
- PSA_ALG_CTR()
- PSA_ALG_CFB()
- PSA_ALG_OFB()
- PSA_ALG_XTS()
- PSA_ALG_CBC_NO_PADDING()
- PSA_ALG_CBC_PKCS7()
- PSA_ALG_ECB_NO_PADDING()

6.2.9.3.3.4 macro PSA_KEY_TYPE_HMAC**PSA_KEY_TYPE_HMAC()**

HMAC key. .. [[symmetric, mac]]

Description

The key policy determines which underlying hash algorithm the key can be used for.

The bit size of an HMAC key must be a non-zero multiple of 8. An HMAC key is typically the same size as the output of the underlying hash algorithm. An HMAC key that is longer than the block size of the underlying hash algorithm will be hashed before use.

Note:

- *PSA_HASH_LENGTH(alg)* provides the output size of hash algorithm *alg*, in bytes.
- *PSA_HASH_BLOCK_LENGTH(alg)* provides the block size of hash algorithm *alg*, in bytes.

Compatible algorithms:

- PSA_ALG_HMAC()
- PSA_ALG_SP800_108_COUNTER_HMAC()

6.2.9.3.3.5 macro PSA_KEY_TYPE_RAW_DATA

PSA_KEY_TYPE_RAW_DATA()

Raw data.

Description

A “key” of this type cannot be used for any cryptographic operation. This type of key is the result of a key derivation operation. Applications can use this type to store arbitrary data in the keystore.

The bit size of a raw key must be a non-zero multiple of 8. The maximum size of a raw key is function of the key derivation.

Compatible algorithms:

- PSA_ALG_HKDF() (output)
- PSA_ALG_HKDF_EXPAND() (output)
- PSA_ALG_HKDF_EXTRACT() (output)
- PSA_ALG_SP800_108_COUNTER_HMAC() (output)
- PSA_ALG_SP800_108_COUNTER_CMAC() (output)
- PSA_ALG_TLS12_PRF() (output)
- PSA_ALG_TLS12_PSK_TO_MS() (output)

6.2.9.3.3.6 macro PSA_KEY_TYPE_SM4

PSA_KEY_TYPE_SM4()

Key for a cipher, AEAD or MAC algorithm based on the SM4 block cipher. .. [[symmetric, cipher, aead]]

Description

For algorithms except the XTS block cipher mode, the SM4 key size is 128 bits (16 bytes).

For the XTS block cipher mode (PSA_ALG_XTS), the SM4 key size is 256 bits (two 16-byte keys).

Compatible algorithms:

- PSA_ALG_CBC_MAC()
- PSA_ALG_CMAC()
- PSA_ALG_CTR()
- PSA_ALG_CFB()
- PSA_ALG_OFB()
- PSA_ALG_XTS()
- PSA_ALG_CBC_NO_PADDING()
- PSA_ALG_CBC_PKCS7()
- PSA_ALG_ECB_NO_PADDING()

- `PSA_ALG_CCM()`
- `PSA_ALG_GCM()`
- `PSA_ALG_SP800_108_COUNTER_CMAC()`

6.2.9.3.3.7 macro `PSA_KEY_TYPE_ARC4`

`PSA_KEY_TYPE_ARC4()`

Key for the ARC4 stream cipher.

Description

Warning:

The ARC4 cipher is weak and deprecated and is only recommended for use in legacy protocols.

The ARC4 cipher supports key sizes between 40 and 2048 bits, that are multiples of 8. (5 to 256 bytes)

Use algorithm `PSA_ALG_STREAM_CIPHER` to use this key with the ARC4 cipher.

Warning:

Not supported.

Compatible algorithms:

- `PSA_ALG_STREAM_CIPHER()`

6.2.9.3.3.8 macro `PSA_KEY_TYPE_ARIA`

`PSA_KEY_TYPE_ARIA()`

Key for a cipher, AEAD or MAC algorithm based on the ARIA block cipher.

Description

The size of the key is related to the ARIA algorithm variant. For algorithms except the XTS block cipher mode, the following key sizes are used:

- ARIA-128 uses a 16-byte key : `key_bits = 128`
- ARIA-192 uses a 24-byte key : `key_bits = 192`
- ARIA-256 uses a 32-byte key : `key_bits = 256`

For the XTS block cipher mode (`PSA_ALG_XTS`), the following key sizes are used:

- ARIA-128-XTS uses two 16-byte keys : `key_bits = 256`
- ARIA-192-XTS uses two 24-byte keys : `key_bits = 384`
- ARIA-256-XTS uses two 32-byte keys : `key_bits = 512`

Warning:

Not supported

Compatible algorithms:

- `PSA_ALG_CBC_MAC()`
- `PSA_ALG_CMAC()`
- `PSA_ALG_CTR()`
- `PSA_ALG_CFB()`

- `PSA_ALG_OFB()`
- `PSA_ALG_XTS()`
- `PSA_ALG_CBC_NO_PADDING()`
- `PSA_ALG_CBC_PKCS7()`
- `PSA_ALG_ECB_NO_PADDING()`
- `PSA_ALG_CCM()`
- `PSA_ALG_GCM()`
- `PSA_ALG_SP800_108_COUNTER_CMAC()`

6.2.9.3.3.9 macro `PSA_KEY_TYPE_CAMELLIA`

`PSA_KEY_TYPE_CAMELLIA()`

Key for a cipher, AEAD or MAC algorithm based on the Camellia block cipher.

Description

The size of the key is related to the Camellia algorithm variant. For algorithms except the XTS block cipher mode, the following key sizes are used:

- Camellia-128 uses a 16-byte key : `key_bits = 128`
- Camellia-192 uses a 24-byte key : `key_bits = 192`
- Camellia-256 uses a 32-byte key : `key_bits = 256`

For the XTS block cipher mode (`PSA_ALG_XTS`), the following key sizes are used:

- Camellia-128-XTS uses two 16-byte keys : `key_bits = 256`
- Camellia-192-XTS uses two 24-byte keys : `key_bits = 384`
- Camellia-256-XTS uses two 32-byte keys : `key_bits = 512`

Warning:

Not supported

Compatible algorithms:

- `PSA_ALG_CBC_MAC()`
- `PSA_ALG_CMIC()`
- `PSA_ALG_CTR()`
- `PSA_ALG_CFB()`
- `PSA_ALG_OFB()`
- `PSA_ALG_XTS()`
- `PSA_ALG_CBC_NO_PADDING()`
- `PSA_ALG_CBC_PKCS7()`
- `PSA_ALG_ECB_NO_PADDING()`
- `PSA_ALG_CCM()`
- `PSA_ALG_GCM()`
- `PSA_ALG_SP800_108_COUNTER_CMIC()`

6.2.9.3.3.10 macro `PSA_KEY_TYPE_CHACHA20`

`PSA_KEY_TYPE_CHACHA20()`

Key for the ChaCha20 stream cipher or the ChaCha20-Poly1305 AEAD algorithm.

Description

The ChaCha20 key size is 256 bits (32 bytes).

- Use algorithm `PSA_ALG_STREAM_CIPHER` to use this key with the ChaCha20 cipher for unauthenticated encryption. See `PSA_ALG_STREAM_CIPHER` for details of this algorithm.
- Use algorithm `PSA_ALG_CHACHA20_POLY1305` to use this key with the ChaCha20 cipher and Poly1305 authenticator for AEAD. See `PSA_ALG_CHACHA20_POLY1305` for details of this algorithm.

Warning:

Not supported.

Compatible algorithms:

- `PSA_ALG_STREAM_CIPHER()`
- `PSA_ALG_CHACHA20_POLY1305()`

6.2.9.3.3.11 macro `PSA_KEY_TYPE_XCHACHA20`

`PSA_KEY_TYPE_XCHACHA20()`

Key for the XChaCha20 stream cipher or the XChaCha20-Poly1305 AEAD algorithm.

Description

The XChaCha20 key size is 256 bits (32 bytes):

- Use algorithm `PSA_ALG_STREAM_CIPHER` to use this key with the XChaCha20 cipher for unauthenticated encryption. See `PSA_ALG_STREAM_CIPHER` for details of this algorithm.
- Use algorithm `PSA_ALG_XCHACHA20_POLY1305` to use this key with the XChaCha20 cipher and Poly1305 authenticator for AEAD. See `PSA_ALG_XCHACHA20_POLY1305` for details of this algorithm.

Warning:

Not supported.

Compatible algorithms:

- `PSA_ALG_STREAM_CIPHER()`
- `PSA_ALG_XCHACHA20_POLY1305()`

6.2.9.3.3.12 macro `PSA_KEY_TYPE_PASSWORD`

`PSA_KEY_TYPE_PASSWORD()`

A low-entropy secret for password hashing or key derivation.

Description

This key type is suitable for passwords and passphrases which are typically intended to be memorable by humans, and have a low entropy relative to their size. It can be used for randomly generated or derived keys with maximum or near-maximum entropy, but `PSA_KEY_TYPE_DERIVE` is more suitable for such keys. It is not suitable for passwords with extremely low entropy, such as numerical PINs.

These keys can be used in the `PSA_KEY_DERIVATION_INPUT_PASSWORD` input step of key derivation algorithms. Algorithms that accept such an input were designed to accept low-entropy secret and are known as password hashing or key stretching algorithms.

These keys cannot be used in the PSA_KEY_DERIVATION_INPUT_SECRET input step of key derivation algorithms, as the algorithms expect such an input to have high entropy.

The key policy determines which key derivation algorithm the key can be used for, among the permissible subset defined above.

Warning:

Not supported.

Compatible algorithms:

- PSA_ALG_PBKDF2_HMAC() (password input)
- PSA_ALG_PBKDF2_AES_CMAC_PRF_128() (password input)

6.2.9.3.3.13 macro PSA_KEY_TYPE_PASSWORD_HASH

PSA_KEY_TYPE_PASSWORD_HASH()

A secret value that can be used to verify a password hash.

Description

The key policy determines which key derivation algorithm the key can be used for, among the same permissible subset as for PSA_KEY_TYPE_PASSWORD.

Warning:

Not supported.

Compatible algorithms:

- PSA_ALG_PBKDF2_HMAC() (key output and verification)
- PSA_ALG_PBKDF2_AES_CMAC_PRF_128() (key output and verification)

6.2.9.3.3.14 macro PSA_KEY_TYPE_PEPPER

PSA_KEY_TYPE_PEPPER()

A secret value that can be used when computing a password hash.

Description

The key policy determines which key derivation algorithm the key can be used for, among the subset of algorithms that can use pepper.

Warning:

Not supported.

Compatible algorithms:

- PSA_ALG_PBKDF2_HMAC() (salt input)
- PSA_ALG_PBKDF2_AES_CMAC_PRF_128() (salt input)

6.2.9.3.4 None keys

6.2.9.3.4.1 macro PSA_KEY_TYPE_NONE

PSA_KEY_TYPE_NONE()

An invalid key type value.

Description

Zero is not the encoding of any key type.

6.2.9.3.5 Asymmetric Keys

6.2.9.3.5.1 RSA Keys

macro PSA_KEY_TYPE_IS_RSA

PSA_KEY_TYPE_IS_RSA(*type*)

Whether a key type is an RSA key. This includes both key pairs and public keys.

Parameters

- **type** - [in] A key type (value of [typedef psa_key_type_t](#)).

macro PSA_KEY_TYPE_RSA_KEY_PAIR

PSA_KEY_TYPE_RSA_KEY_PAIR()

RSA key pair: both the private and public key. ... [[asymmetric, signature, encryption]]

Description

The size of an RSA key is the bit size of the modulus.

Compatible algorithms:

- **PSA_ALG_RSA_OAEP()** (encryption/decryption)
- **PSA_ALG_RSA_PKCS1V15_CRYPT()** (encryption/decryption)
- **PSA_ALG_RSA_PKCS1V15_SIGN()** (signing/verification)
- **PSA_ALG_RSA_PKCS1V15_SIGN_RAW()** (signing/verification)
- **PSA_ALG_RSA_PSS()** (signing/verification)
- **PSA_ALG_RSA_PSS_ANY_SALT()** (signing/verification)

macro PSA_KEY_TYPE_RSA_PUBLIC_KEY

PSA_KEY_TYPE_RSA_PUBLIC_KEY()

RSA public key.

Description

The size of an RSA key is the bit size of the modulus.

Compatible algorithms:

- `PSA_ALG_RSA_OAEP()` (encryption only)
- `PSA_ALG_RSA_PKCS1V15_CRYPT()` (encryption only)
- `PSA_ALG_RSA_PKCS1V15_SIGN()` (signature verification only)
- `PSA_ALG_RSA_PKCS1V15_SIGN_RAW()` (signature verification only)
- `PSA_ALG_RSA_PSS()` (signature verification only)
- `PSA_ALG_RSA_PSS_ANY_SALT()` (signature verification only)

6.2.9.3.5.2 Elliptic Curve Keys**typedef `psa_ecc_family_t`**

type `psa_ecc_family_t`

The type of PSA elliptic curve family identifiers.

Description

The curve identifier is required to create an ECC key using the [PSA_KEY_TYPE_ECC_KEY_PAIR\(\)](#) or [PSA_KEY_TYPE_ECC_PUBLIC_KEY\(\)](#) macros.

The specific ECC curve within a family is identified by the `key_bits` attribute of the key.

The range of Elliptic curve family identifier values is divided as follows:

- **0x00**
Reserved. Not allocated to an ECC family.
- **0x01 - 0x7f**
ECC family identifiers defined by this standard. Unallocated values in this range are reserved for future use.
- **0x80 - 0xff**
Implementation defines additional ECC family.

macro `PSA_KEY_TYPE_ECC_GET_FAMILY`

`PSA_KEY_TYPE_ECC_GET_FAMILY(type)`

Extract the curve family from an elliptic curve key type.

Parameters

- **`type`** - [in] An elliptic curve key type such that [PSA_KEY_TYPE_IS_ECC](#) is true.

Return

The elliptic curve family id [typedef `psa\_ecc\_family\_t`](#), if `type` is a supported elliptic curve key.

Unspecified if `type` is not a supported elliptic curve key.

macro `PSA_KEY_TYPE_ECC_KEY_PAIR`

`PSA_KEY_TYPE_ECC_KEY_PAIR(curve)`

Elliptic curve key pair: both the private and public key.

Parameters

- **curve** - [in] A value of [typedef psa_ecc_family_t](#) that identifies the ECC curve family to be used.

Description

Build a ECC key pair [typedef psa_key_type_t](#) of the given curve.

macro PSA_KEY_TYPE_ECC_PUBLIC_KEY

PSA_KEY_TYPE_ECC_PUBLIC_KEY(curve)

Elliptic curve public key.

Parameters

- **curve** - [in] A value of [typedef psa_ecc_family_t](#) that identifies the ECC curve family to be used.

Description

Build a ECC public key [typedef psa_key_type_t](#) of the given curve.

macro PSA_KEY_TYPE_IS_ECC

PSA_KEY_TYPE_IS_ECC(type)

Whether a key type is an elliptic curve key, either a key pair or a public key.

Parameters

- **type** - [in] A key type (value of [typedef psa_key_type_t](#)).

macro PSA_KEY_TYPE_IS_ECC_KEY_PAIR

PSA_KEY_TYPE_IS_ECC_KEY_PAIR(type)

Whether a key type is an elliptic curve key pair.

Parameters

- **type** - [in] A key type (value of [typedef psa_key_type_t](#)).

macro PSA_KEY_TYPE_IS_ECC_PUBLIC_KEY

PSA_KEY_TYPE_IS_ECC_PUBLIC_KEY(type)

Whether a key type is an elliptic curve public key.

Parameters

- **type** - [in] A key type (value of [typedef psa_key_type_t](#)).

macro PSA_ECC_FAMILY_BRAINPOOL_P_R1

PSA_ECC_FAMILY_BRAINPOOL_P_R1()

Brainpool P random curves. .. [[ecdsa]]

Description

This family comprises the following curves:

- brainpoolP160r1 : key_bits = 160 (Deprecated)
- brainpoolP192r1 : key_bits = 192
- brainpoolP224r1 : key_bits = 224
- brainpoolP256r1 : key_bits = 256
- brainpoolP320r1 : key_bits = 320
- brainpoolP384r1 : key_bits = 384
- brainpoolP512r1 : key_bits = 512

Warning:

The 160-bit curve brainpoolP160r1 is weak and deprecated and is only recommended for use in legacy protocols.

macro PSA_ECC_FAMILY_FRP

PSA_ECC_FAMILY_FRP()

Curve used primarily in France and elsewhere in Europe.

Description

This family comprises one 256-bit curve:

- FRP256v1 : key_bits = 256

macro PSA_ECC_FAMILY_MONTGOMERY

PSA_ECC_FAMILY_MONTGOMERY()

Montgomery curves. ... [[key_ex, size:255,448]]

Description

This family comprises the following Montgomery curves:

- Curve25519 : key_bits = 255
- Curve448 : key_bits = 448

Keys in this family can only be used with the PSA_ALG_ECDH key agreement algorithm.

macro PSA_ECC_FAMILY_SECP_K1

PSA_ECC_FAMILY_SECP_K1()

SEC Koblitz curves over prime fields.

Description

This family comprises the following curves:

- secp192k1 : key_bits = 192
- secp224k1 : key_bits = 225

- secp256k1 : key_bits = 256

macro PSA_ECC_FAMILY_SECP_R1

PSA_ECC_FAMILY_SECP_R1()

SEC random curves over prime fields. ... [[ecdsa, key_ex]]

Description

This family comprises the following curves:

- secp192r1 : key_bits = 192
- secp224r1 : key_bits = 224
- secp256r1 : key_bits = 256
- secp384r1 : key_bits = 384
- secp521r1 : key_bits = 521

macro PSA_ECC_FAMILY_SECP_R2

PSA_ECC_FAMILY_SECP_R2()

SEC random curves over prime fields.

Description

Warning:

This family of curves is weak and deprecated.

This family comprises the following curves:

- secp160r2 : key_bits = 160 (Deprecated)

macro PSA_ECC_FAMILY_SECT_K1

PSA_ECC_FAMILY_SECT_K1()

SEC Koblitz curves over binary fields.

Description

This family comprises the following curves:

- sect163k1 : key_bits = 163 (Deprecated)
- sect233k1 : key_bits = 233
- sect239k1 : key_bits = 239
- sect283k1 : key_bits = 283
- sect409k1 : key_bits = 409
- sect571k1 : key_bits = 571

Warning:

The 163-bit curve sect163k1 is weak and deprecated and is only recommended for use in legacy protocols.

macro PSA_ECC_FAMILY_SECT_R1

PSA_ECC_FAMILY_SECT_R1()

SEC random curves over binary fields.

This family comprises the following curves

- sect163r1 : key_bits = 163 (Deprecated)
- sect233r1 : key_bits = 233
- sect283r1 : key_bits = 283
- sect409r1 : key_bits = 409
- sect571r1 : key_bits = 571

Warning:

The 163-bit curve sect163r1 is weak and deprecated and is only recommended for use in legacy protocols.

macro PSA_ECC_FAMILY_SECT_R2

PSA_ECC_FAMILY_SECT_R2()

SEC additional random curves over binary fields.

This family comprises the following curves

- sect163r2 : key_bits = 163 (Deprecated)

Warning:

The 163-bit curve sect163r2 is weak and deprecated and is only recommended for use in legacy protocols.

macro PSA_ECC_FAMILY_TWISTED_EDWARDS

PSA_ECC_FAMILY_TWISTED_EDWARDS()

Twisted Edwards curves. ... [[eddsa, key_ex, size:255,448]]

Description

This family comprises the following twisted Edwards curves:

- Edwards25519: key_bits = 255. This curve is birationally equivalent to Curve25519.
- Edwards448: key_bits = 448. This curve is birationally equivalent to Curve448.

6.2.9.3.5.3 Diffie-Hellman Keys

typedef psa_dh_family_t

type psa_dh_family_t

The type of PSA finite-field Diffie-Hellman group family identifiers.

Description

The group family identifier is required to create an Diffie-Hellman key using the [PSA_KEY_TYPE_DH_KEY_PAIR\(\)](#) or [PSA_KEY_TYPE_DH_PUBLIC_KEY\(\)](#) macros.

The specific Diffie-Hellman group within a family is identified by the *key_bits* attribute of the key.

The range of Diffie-Hellman group family identifier values is divided as follows:

- **0x00**
Reserved. Not allocated to a DH group family
- **0x01 - 0x7f**
DH group family identifiers defined by this standard. Unallocated values in this range are reserved for future use.
- **0x80 - 0xff**
Implementation defines additional DH family.

macro PSA_KEY_TYPE_DH_GET_FAMILY

PSA_KEY_TYPE_DH_GET_FAMILY(*type*)

Extract the group family from a Diffie-Hellman key type.

Parameters

- **type** - [in] A Diffie-Hellman key type such that [PSA_KEY_TYPE_IS_DH](#) is true.

Return

The Diffie-Hellman group family id [typedef psa_dh_family_t](#) if *type* is a supported Diffie-Hellman key.

Unspecified if *type* is not a supported Diffie-Hellman key.

macro PSA_KEY_TYPE_DH_KEY_PAIR

PSA_KEY_TYPE_DH_KEY_PAIR(*group*)

Finite-field Diffie-Hellman key pair: both the private key and public key.

Parameters

- **group** - [in] A value of [typedef psa_dh_family_t](#) that identifies the Diffie-Hellman group family to be used.

Description

Build a DH key pair type [typedef psa_key_type_t](#) of the given *group*.

macro PSA_KEY_TYPE_DH_PUBLIC_KEY

PSA_KEY_TYPE_DH_PUBLIC_KEY(*group*)

Finite-field Diffie-Hellman public key.

Parameters

- **group** - [in] A value of [typedef psa_dh_family_t](#) that identifies the Diffie-Hellman group family to be used.

Description

Build a DH public key type [typedef psa_key_type_t](#) of the given group.

macro PSA_KEY_TYPE_IS_DH

PSA_KEY_TYPE_IS_DH(*type*)

Whether a key type is a Diffie-Hellman key, either a key pair or a public key.

Parameters

- **type** - [in] A key type (value of [typedef psa_key_type_t](#)).

macro PSA_KEY_TYPE_IS_DH_KEY_PAIR

PSA_KEY_TYPE_IS_DH_KEY_PAIR(*type*)

Whether a key type is a Diffie-Hellman key pair.

Parameters

- **type** - [in] A key type (value of [typedef psa_key_type_t](#)).

macro PSA_KEY_TYPE_IS_DH_PUBLIC_KEY

PSA_KEY_TYPE_IS_DH_PUBLIC_KEY(*type*)

Whether a key type is a Diffie-Hellman public key.

Parameters

- **type** - [in] A key type (value of [typedef psa_key_type_t](#)).

macro PSA_DH_FAMILY_RFC7919

PSA_DH_FAMILY_RFC7919()

Finite-field Diffie-Hellman groups defined for TLS in [\[RFC 7919\]](#).

Description

This family includes groups with the following key sizes (in bits): 2048, 3072, 4096, 6144, 8192. An implementation can support all of these sizes or only a subset.

Keys in this group can only be used with the PSA_ALG_FFDH key agreement algorithm.

6.2.9.3.5.4 SPAKE2+ Keys

macro PSA_KEY_TYPE_IS_SPAKE2P_KEY_PAIR

PSA_KEY_TYPE_IS_SPAKE2P_KEY_PAIR(*type*)

Whether a key type is a SPAKE2+ key pair.

Parameters

- **type** - [in] A key type (value of [typedef psa_key_type_t](#)).

macro PSA_KEY_TYPE_IS_SPAKE2P_PUBLIC_KEY**PSA_KEY_TYPE_IS_SPAKE2P_PUBLIC_KEY**(*type*)

Whether a key type is a SPAKE2+ public key.

Parameters

- **type** - [in] A key type (value of [typedef psa_key_type_t](#)).

macro PSA_KEY_TYPE_IS_SPAKE2P**PSA_KEY_TYPE_IS_SPAKE2P**(*type*)

Whether a key type is a SPAKE2+ key, either a key pair or a public key.

Parameters

- **type** - [in] A key type (value of [typedef psa_key_type_t](#)).

macro PSA_KEY_TYPE_SPAKE2P_GET_FAMILY**PSA_KEY_TYPE_SPAKE2P_GET_FAMILY**(*type*)

Extract the curve family from a SPAKE2+ key type.

Parameters

- **type** - [in] A SPAKE2+ key type: a value of [typedef psa_key_type_t](#) such that `PSA_KEY_TYPE_IS_SPAKE2P` is true.

ReturnThe elliptic curve family id [typedef psa_ecc_family_t](#), if type is a supported SPAKE2+ key.

Unspecified if type is not a supported SPAKE2+ key.

macro PSA_KEY_TYPE_SPAKE2P_KEY_PAIR**PSA_KEY_TYPE_SPAKE2P_KEY_PAIR**(*curve*)

SPAKE2+ key pair: both the prover and verifier key.

Parameters

- **curve** - [in] A value of type `psa_ecc_family_t` that identifies the elliptic curve family to be used.

DescriptionBuild a SPAKE2+ key pair type [typedef psa_key_type_t](#) of the given *curve*.**macro PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY****PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY**(*curve*)

SPAKE2+ public key: the verifier key.

Parameters

- **curve** - [in] A value of type `psa_ecc_family_t` that identifies the elliptic curve family to be used.

Description

Build a SPAKE2+ public key type `typedef psa_key_type_t` of the given `curve`.

6.2.9.3.5.5 Additional macros**macro PSA_KEY_TYPE_KEY_PAIR_OF_PUBLIC_KEY**

PSA_KEY_TYPE_KEY_PAIR_OF_PUBLIC_KEY(*type*)

The key pair type corresponding to a public key type.

Parameters

- **type** - [in] A public key type or key pair type.

Description

If `type` is a key pair type, it will be left unchanged.

Return

The corresponding key pair type. If `type` is not a public key or a key pair, the return value is undefined.

macro PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR

PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(*type*)

The public key type corresponding to a key pair type.

Parameters

- **type** - [in] A public key type or key pair type.

Description

If `type` is a public key type, it will be left unchanged.

Return

The corresponding public key type. If `type` is not a public key or a key pair, the return value is undefined.

6.3 PKCS11 APIs**6.3.1 General Purpose**

The general purpose APIs allow users of the library to:

- Initialize the library.
- Finalize the library.
- Get general information about the library.

- Get the entry points of the library.
- Get the list of available interface.

Before executing any PKCS#11 operation but [C_GetInfo\(\)](#), [C_GetFunctionList\(\)](#), [C_GetInterfaceList\(\)](#) and [C_GetInterface\(\)](#), the library must be initialized by the [C_Initialize\(\)](#). If this function is not called, any operation will return the CKR_CRYPTOKI_NOT_INITIALIZED error.

To finish with the library usage, the application calls the [C_Finalize\(\)](#) to free all resources allocated during the library execution. Before calling this function, the application must ensure that all operations are complete.

Note:

In the context of multiple applications using the library simultaneously, the [C_Initialize\(\)](#) function must be called at least once before any cryptographic operations can be performed. The function is not required to be thread-safe, but the application is responsible for ensuring proper synchronization if [C_Initialize\(\)](#) is called from multiple threads. Once initialized, the library remains active until [C_Finalize\(\)](#) is called.

6.3.1.1 Library start/stop

6.3.1.1.1 Initialization

6.3.1.1.1.1 C_Initialize

[CK_RV](#) [C_Initialize](#)([CK_VOID_PTR](#) pInitArgs)

Initialize the Cryptoki library.

Parameters

- **pInitArgs** ([CK_VOID_PTR](#)) - [in] Pointer to initialization arguments or [NULL_PTR](#).

Description

This function initializes the Cryptoki library and must be called before any other Cryptoki function except [C_GetInfo\(\)](#), [C_GetFunctionList\(\)](#), [C_GetInterfaceList\(\)](#), or [C_GetInterface\(\)](#). It sets up the library's internal state, including multi-threading support and mutex handling based on the provided initialization arguments.

In the context of multiple applications using the same Cryptoki library, the [C_Initialize\(\)](#) function must be called once.

If **pInitArgs** is not [NULL_PTR](#), the argument is a pointer to a [CK_C_INITIALIZE_ARGS](#) structure.

If the CKF_LIBRARY_CANT_CREATE_OS_THREADS flag in the flags field is set, that indicates that application threads which are executing calls to the Cryptoki library are not permitted to use the native operation system calls to spawn off new threads. In other words, the library's code may not create its own threads.

Note:

The library does not support the thread creation by itself.

A call to [C_Initialize\(\)](#) specifies one of four different ways to support multi-threaded access via the value of the CKF_OS_LOCKING_OK flag in the flags field and the values of the CreateMutex, DestroyMutex, LockMutex, and UnlockMutex function pointer fields:

1. If the flag isn't set, and the function pointer fields aren't supplied (i.e., they all have the value [NULL_PTR](#)), that means that the application won't be accessing the Cryptoki library from multiple threads simultaneously.

2. If the flag is set, and the function pointer fields aren't supplied (i.e., they all have the value [NULL_PTR](#)), that means that the application will be performing multi-threaded Cryptoki access, and the library needs to use the native operating system primitives to ensure safe multi-threaded access. If the library is unable to do this, [C_Initialize\(\)](#) returns with the value `CKR_CANT_LOCK`.
3. If the flag isn't set, and the function pointer fields are supplied (i.e., they all have non [NULL_PTR](#) values), that means that the application will be performing multi-threaded Cryptoki access, and the library needs to use the supplied function pointers for mutex-handling to ensure safe multi-threaded access. If the library is unable to do this, [C_Initialize\(\)](#) returns with the value `CKR_CANT_LOCK`.
4. If the flag is set, and the function pointer fields are supplied (i.e., they all have non [NULL_PTR](#) values), that means that the application will be performing multi-threaded Cryptoki access, and the library needs to use either the native operating system primitives or the supplied function pointers for mutex-handling to ensure safe multi-threaded access. If the library is unable to do this, [C_Initialize\(\)](#) returns with the value `CKR_CANT_LOCK`.

If some, but not all, of the supplied function pointers to [C_Initialize\(\)](#) are non [NULL_PTR](#), then [C_Initialize](#) returns with the value `CKR_ARGUMENTS_BAD`.

If `pInitArgs` is [NULL_PTR](#), the library assumes single-threaded operation. Otherwise, the flags and function pointers in `pInitArgs` determine the threading model and mutex primitives to use.

Return

- **CKR_OK:**
Success. Library is initialized.
- **CKR_ARGUMENTS_BAD:**
 - `pInitArgs->pReserved` is not [NULL_PTR](#).
 - Some, but not all, of the supplied function pointers are non [NULL_PTR](#).
- **CKR_CANT_LOCK:**
Unable to acquire necessary locks for thread-safe operation.
- **CKR_CRYPTOKI_ALREADY_INITIALIZED:**
The library has already been initialized.
- **CKR_GENERAL_ERROR**
- **CKR_NEED_TO_CREATE_THREADS:**
The library cannot create OS threads as required.

6.3.1.1.2 Deinitialization

6.3.1.1.2.1 C_Finalize

[CK_RV](#) [C_Finalize](#)([CK_VOID_PTR](#) `pReserved`)

Finalize the Cryptoki library.

Parameters

- `pReserved` ([CK_VOID_PTR](#)) - Reserved for future use; must be [NULL_PTR](#).

Description

This function cleans up the Cryptoki library and releases all resources allocated during [C_Initialize\(\)](#). After calling this function, no other Cryptoki functions (except [C_Initialize\(\)](#)) may be called until the library is re-initialized.

If several applications are using Cryptoki, each one should call [C_Finalize\(\)](#).

Return

- **CKR_OK**
- **CKR_ARGUMENTS_BAD:**
The `pReserved` is not [NULL_PTR](#).
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- **CKR_GENERAL_ERROR**

6.3.1.2 General Information

6.3.1.2.1 Get library information

6.3.1.2.1.1 C_GetInfo

[CK_RV](#) **C_GetInfo**([CK_INFO_PTR](#) *pInfo*)

Obtain general information about the library.

Parameters

- **pInfo** ([CK_INFO_PTR](#)) - [out] Pointer to a CK_INFO structure to receive the library information.

Description

This function returns general information about the library, including the version supported, manufacturer ID, library description, and library version. The information is copied into the structure pointed to by `pInfo`.

The `manufacturerID` and `libraryDescription` fields are padded with blank spaces to their full length as defined in the PKCS#11 specification.

This function can be called before [C_Initialize\(\)](#).

Return

- **CKR_OK:**
Success. Library information has been copied to `pInfo`.
- **CKR_ARGUMENTS_BAD:**
The `pInfo` is [NULL_PTR](#).

6.3.1.2.2 Get library entry points

6.3.1.2.2.1 C_GetFunctionList

CK_RV C_GetFunctionList([CK_FUNCTION_LIST_PTR_PTR](#) ppFunctionList)

Obtain a pointer to the library's function list.

Parameters

- **ppFunctionList** ([CK_FUNCTION_LIST_PTR_PTR](#)) - [out] Pointer to a location that receives a pointer to the library's CK_FUNCTION_LIST structure.

Description

This function returns a pointer to the library's list of Cryptoki function pointers. The returned function list corresponds to PKCS#11 v2.40.

This function can be called before [C_Initialize\(\)](#).

Return

- **CKR_OK:**
Success. ppFunctionList points to the function list.
- **CKR_ARGUMENTS_BAD:**
The ppFunctionList is [NULL_PTR](#).

6.3.1.2.3 Get library interfaces

6.3.1.2.3.1 C_GetInterfaceList

CK_RV C_GetInterfaceList([CK_INTERFACE_PTR](#) pInterfacesList, [CK_ULONG_PTR](#) pulCount)

Obtain the list of interfaces supported by the library.

Parameters

- **pInterfacesList** ([CK_INTERFACE_PTR](#)) - [out] Pointer to an array of CK_INTERFACE structures, or [NULL_PTR](#) to query the number of interfaces.
- **pulCount** ([CK_ULONG_PTR](#)) - [in/out] Pointer to a location that gives the number element in pInterfacesList, or receives the number of interfaces.

Description

This function returns information about all interfaces supported by the library.

If pInterfacesList is [NULL_PTR](#), the function returns the total number of supported interfaces in pulCount.

If the pInterfacesList is not [NULL_PTR](#), the pulCount contains the number of element in pInterfacesList on input, and receives the number of interfaces actually copied on output. If the buffer is too small to hold all interface structures, the function returns CKR_BUFFER_TOO_SMALL, otherwise the function copies all interface structures into the pInterfaceList.

The library supports both PKCS#11 v2.40 and v3.2 interfaces.

This function can be called before [C_Initialize\(\)](#).

Return

- **CKR_OK:**
Success. Interface list has been returned or count has been set.
- **CKR_ARGUMENTS_BAD:**
The `pulCount` is [NULL_PTR](#).
- **CKR_BUFFER_TOO_SMALL:**
The buffer provided is too small to hold all interface structures.

6.3.1.2.3.2 C_GetInterface

[CK_RV](#) **C_GetInterface**([CK_UTF8CHAR_PTR](#) *pInterfaceName*, [CK_VERSION_PTR](#) *pVersion*, [CK_INTERFACE_PTR_PTR](#) *ppInterface*, [CK_FLAGS](#) *flags*)

Obtain a specific interface.

Parameters

- **pInterfaceName** ([CK_UTF8CHAR_PTR](#)) - [in] Pointer to a UTF-8 string specifying the interface name, or [NULL_PTR](#) to match any name.
- **pVersion** ([CK_VERSION_PTR](#)) - [in] Pointer to a version structure specifying the desired interface version, or [NULL_PTR](#) to match any version.
- **ppInterface** ([CK_INTERFACE_PTR_PTR](#)) - [out] Pointer to a location that receives a pointer to the requested [typedef CK_INTERFACE](#) structure.
- **flags** ([CK_FLAGS](#)) - [in] Flags that must be present in the interface, or 0 to match any flags, refer to [typedef CK_INTERFACE](#).

Description

This function returns a pointer to a specific interface based on the provided search criteria. The interface is selected by matching the interface name, version, and flags. If no criteria are specified (all parameters are [NULL_PTR](#) or 0), the default interface (PKCS#11 v2.40) is returned.

The search criteria are applied as follows

- If `pInterfaceName` is provided, only interfaces with matching names are considered.
- If `pVersion` is provided, only interfaces with matching major and minor versions are considered.
- If `flags` is non-zero, only interfaces that have all specified flags set are considered.

This function can be called before [C_Initialize\(\)](#).

Return

- **CKR_OK:**
Success. `ppInterface` points to the requested interface.
- **CKR_ARGUMENTS_BAD:**
The `ppInterface` is [NULL_PTR](#), or no interface matches the criteria.

6.3.1.3 Example

The following example demonstrates how to retrieve library information, function entry points, and the list of available interfaces:

```
#include <stdio.h>
#include <pkcs11.h>

int main(void)
{
    CK_RV rv;
    CK_INFO info = { 0 };
    CK_FUNCTION_LIST_PTR pFunctionList = NULL;
    CK_INTERFACE_PTR pInterfacesList = NULL;
    CK_ULONG ulCount = 0;
    CK_ULONG i = 0;

    /* Get library information (can be called before C_Initialize) */
    rv = C_GetInfo(&info);
    if (rv != CKR_OK) {
        printf("C_GetInfo failed: 0x%08lX\n", rv);
        return 1;
    }

    printf("Library Information:\n");
    printf("    Cryptoki Version: %d.%d\n",
        info.cryptokiVersion.major, info.cryptokiVersion.minor);
    printf("    Manufacturer ID: %.32s\n", info.manufacturerID);
    printf("    Library Description: %.32s\n", info.libraryDescription);
    printf("    Library Version: %d.%d\n",
        info.libraryVersion.major, info.libraryVersion.minor);

    /* Get function list (can be called before C_Initialize) */
    rv = C_GetFunctionList(&pFunctionList);
    if (rv != CKR_OK) {
        printf("C_GetFunctionList failed: 0x%08lX\n", rv);
        return 1;
    }

    printf("\nFunction List Version: %d.%d\n",
        pFunctionList->version.major, pFunctionList->version.minor);

    /* Get number of interfaces */
    rv = C_GetInterfaceList(NULL, &ulCount);
    if (rv != CKR_OK) {
        printf("C_GetInterfaceList failed: 0x%08lX\n", rv);
        return 1;
    }

    printf("\nNumber of interfaces: %lu\n", ulCount);

    /* Allocate memory for interface list */
    pInterfacesList = malloc(ulCount * sizeof(CK_INTERFACE));
    if (!pInterfacesList) {
        printf("Memory allocation failed\n");
        return 1;
    }

    /* Get interface list */
}
```

```

rv = C_GetInterfaceList(pInterfacesList, &ulCount);
if (rv != CKR_OK) {
    printf("C_GetInterfaceList failed: 0x%08lX\n", rv);
    free(pInterfacesList);
    return 1;
}

/* Display available interfaces */
printf("\nAvailable Interfaces:\n");
for (i = 0; i < ulCount; i++) {
    CK_FUNCTION_LIST_PTR pFuncList =
        (CK_FUNCTION_LIST_PTR)pInterfacesList[i].pFunctionList;
    printf("    Interface %lu:\n", i);
    printf("        Name: %s\n", pInterfacesList[i].pInterfaceName);
    printf("        Version: %d.%d\n",
        pFuncList->version.major, pFuncList->version.minor);
    printf("        Flags: 0x%lX\n", pInterfacesList[i].flags);
}

free(pInterfacesList);

/* Get a specific interface (PKCS#11 v3.2) */
CK_VERSION version = { 3, 2 };
CK_INTERFACE_PTR pInterface = NULL;

rv = C_GetInterface((CK_UTF8CHAR_PTR) "PKCS 11", &version,
                    &pInterface, 0);
if (rv == CKR_OK) {
    printf("\nRequested PKCS#11 v3.2 interface found\n");
} else {
    printf("\nPKCS#11 v3.2 interface not available\n");
}

return 0;
}

```

6.3.2 Slot and Token Management

6.3.2.1 Overview

In PKCS#11, the concepts of **slots** and **tokens** are fundamental to understanding how cryptographic devices are accessed and managed.

What is a Slot?

A **slot** is a logical reader that potentially contains a token. It represents an interface to a cryptographic device, which is a connection point to the Security Middleware Library exposing interface to any secure subsystem enabled (refer to [Secure Subsystems Capabilities](#)).

What is a Token?

A **token** is a logical view of a cryptographic device. It represents the actual cryptographic resource that:

- Stores cryptographic keys and certificates
- Performs cryptographic operations (encryption, decryption, signing, verification)
- Maintains its own security state (initialized, user logged in, etc.)
- Has its own storage for objects (keys, certificates, data)

A token can be either:

- The unique SMW device abstracting the Secure Subsystem(s)
- Each Secure Subsystem exposed by the SMW device.

By default, only the SMW device token is available. However, compiling the PKCS#11 library with the cmake option `SMW_DEVICE_ONLY` set to `OFF` will expose each Secure Subsystem as a separate token, allowing independent access to each subsystem's capabilities and operations.

Typical Workflow

1. **Enumerate slots** to discover available cryptographic devices
2. **Query slot information** to determine if a token is present
3. **Access token** to retrieve its capabilities and state
4. **Initialize token** (if required) to prepare it for use
5. **Open session** with the token to perform cryptographic operations

6.3.2.2 Functions

6.3.2.2.1 Slot enumeration and information

6.3.2.2.1.1 C_GetSlotList

[CK_RV](#) **C_GetSlotList**([CK_BBOOL](#) tokenPresent, [CK_SLOT_ID_PTR](#) pSlotList, [CK_ULONG_PTR](#) pulCount)

Obtains a list of slots in the system.

Parameters

- **tokenPresent** ([CK_BBOOL](#)) - [in] Indicates whether the list returned includes only those slots with a token present (`CK_TRUE`), or all slots (`CK_FALSE`).
- **pSlotList** ([CK_SLOT_ID_PTR](#)) - [out] Pointer to the location that receives the array of slot IDs, or [NULL_PTR](#) to query the number of slots.
- **pulCount** ([CK_ULONG_PTR](#)) - [in/out] Pointer to a location that gives the number elements in pSlotList, or receives the number of slots.

Description

This function is used to obtain a list of slots in the system. A slot is a logical reader that potentially contains a token.

If pSlotList is [NULL_PTR](#), the function returns the total number of supported slots in pulCount.

If the pSlotList is not [NULL_PTR](#), the pulCount contains the number of elements in pSlotList on input, and receives the number of slots actually copied on output. If the buffer is too small to hold all slot identifiers, the function returns `CKR_BUFFER_TOO_SMALL`, otherwise the function copies all slot identifiers into the pSlotList.

Return

- **CKR_OK**
Success.
- **CKR_ARGUMENTS_BAD:**
The pulCount is NULL.
- **CKR_BUFFER_TOO_SMALL:**

The buffer provided is too small.

- **CKR_ERROR_GENERAL:**

No slot available.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

6.3.2.2.1.2 C_GetSlotInfo

[CK_RV](#) **C_GetSlotInfo**([CK_SLOT_ID](#) slotID, [CK_SLOT_INFO_PTR](#) pInfo)

Obtains information about a particular slot in the system.

Parameters

- **slotID** ([CK_SLOT_ID](#)) - [in] The ID of the slot.
- **pInfo** ([CK_SLOT_INFO_PTR](#)) - [out] Pointer to the location that receives the slot information.

Description

This function obtains information about a particular slot in the system.

Return

- **CKR_OK:**

Success.

- **CKR_ARGUMENTS_BAD:**

The pInfo is NULL.

- **CKR_SLOT_ID_INVALID:**

The slot ID is not valid.

- **CKR_ERROR_GENERAL:**

No slot available.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

6.3.2.2.2 Slot event

6.3.2.2.2.1 C_WaitForSlotEvent

[CK_RV](#) **C_WaitForSlotEvent**([CK_FLAGS](#) flags, [CK_SLOT_ID_PTR](#) pSlot, [CK_VOID_PTR](#) pReserved)

Waits for a slot event.

Parameters

- **flags** ([CK_FLAGS](#)) - [in] Determines whether or not the function blocks.
- **pSlot** ([CK_SLOT_ID_PTR](#)) - [in] Pointer to the location that receives the slot ID.
- **pReserved** ([CK_VOID_PTR](#)) - Reserved for future use, should be [NULL_PTR](#).

Description

This function waits for a slot event, such as token insertion or removal, to occur.

Only one flag is supported: `CKF_DONT_BLOCK`. If this flag is set:

- If some slot's event is pending, the event is cleared and the function returns with the ID of the slot in `pSlot`.
- If no slot's event is pending, the function returns with the value `CKR_NO_EVENT`. The content of the `pSlot` is no significant.

Note:

As the device supported is not a removal device, this functionality is not supported.

Return

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- **CKR_FUNCTION_NOT_SUPPORTED:**

Function is not supported.

6.3.2.2.3 Mechanisms

6.3.2.2.3.1 C_GetMechanismList

[CK_RV](#) **C_GetMechanismList**([CK_SLOT_ID](#) slotID, [CK_MECHANISM_TYPE_PTR](#) pMechanismList, [CK_ULONG_PTR](#) pulCount)

Obtains a list of mechanism types supported by a token.

Parameters

- **slotID** ([CK_SLOT_ID](#)) - [in] The ID of the token's slot.
- **pMechanismList** ([CK_MECHANISM_TYPE_PTR](#)) - [out] Pointer to the location that receives the array of mechanism types, or [NULL_PTR](#) to query the number of mechanisms.
- **pulCount** ([CK_ULONG_PTR](#)) - [in/out] Pointer to the location that gives the number of elements in `pMechanismList`, or receives the number of mechanisms

Description

This function is used to obtain a list of mechanism types supported by a token.

If `pMechanismList` is [NULL_PTR](#), the function returns the total number of supported mechanisms in `pulCount`.

If the `pMechanismList` is not [NULL_PTR](#), the `pulCount` contains the number of elements in `pMechanismList` on input, and receives the number of mechanisms actually copied on output. If the buffer is too small to hold all mechanisms, the function returns `CKR_BUFFER_TOO_SMALL`, otherwise the function copies all mechanisms into the `pMechanismList`.

Return

- **CKR_OK**
Success.
- **CKR_ARGUMENTS_BAD:**
The `pulCount` is NULL.
- **CKR_BUFFER_TOO_SMALL:**
The buffer provided is too small.
- **CKR_HOST_MEMORY:**
Memory allocation failed.
- **CKR_TOKEN_NOT_PRESENT:**
The token is not present.
- **CKR_SLOT_ID_INVALID:**
The slot ID is not valid.
- **CKR_ERROR_GENERAL:**
No slot available.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).

6.3.2.2.3.2 C_GetMechanismInfo

[CK_RV](#) **C_GetMechanismInfo**([CK_SLOT_ID](#) slotID, [CK_MECHANISM_TYPE](#) type, [CK_MECHANISM_INFO_PTR](#) pInfo)

Obtains information about a particular mechanism.

Parameters

- **slotID** ([CK_SLOT_ID](#)) - [in] The ID of the token's slot.
- **type** ([CK_MECHANISM_TYPE](#)) - [in] The type of mechanism.
- **pInfo** ([CK_MECHANISM_INFO_PTR](#)) - [out] Pointer to the location that receives the mechanism information.

Description

This function obtains information about a particular mechanism possibly supported by a token.

Return

- **CKR_OK**
Success.
- **CKR_ARGUMENTS_BAD:**
The `pInfo` is NULL.
- **CKR_MECHANISM_INVALID:**
The mechanism `type` is not valid.
- **CKR_TOKEN_NOT_PRESENT:**

The token is not present.

- **CKR_SLOT_ID_INVALID:**

The slot ID is not valid.

- **CKR_ERROR_GENERAL:**

No slot available.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

6.3.2.2.4 Token information and initialization

6.3.2.2.4.1 C_GetTokenInfo

[CK_RV](#) **C_GetTokenInfo**([CK_SLOT_ID](#) slotID, [CK_TOKEN_INFO_PTR](#) pInfo)

Obtains information about a particular token in the system.

Parameters

- **slotID** ([CK_SLOT_ID](#)) - [in] The ID of the token's slot.
- **pInfo** ([CK_TOKEN_INFO_PTR](#)) - [out] Pointer to the location that receives the token information.

Description

This function obtains information about a particular token in the system.

Return

- **CKR_OK:**

Success.

- **CKR_ARGUMENTS_BAD:**

The pInfo is NULL.

- **CKR_SLOT_ID_INVALID:**

The slot ID is not valid.

- **CKR_ERROR_GENERAL:**

General error.

- **CKR_FUNCTION_FAILED:**

Failure in the function.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

6.3.2.2.4.2 C_InitToken

[CK_RV](#) **C_InitToken**([CK_SLOT_ID](#) slotID, [CK_UTF8CHAR_PTR](#) pPin, [CK_ULONG](#) ulPinLen, [CK_UTF8CHAR_PTR](#) pLabel)

Initializes a token.

Parameters

- **slotID** ([CK_SLOT_ID](#)) - [in] The ID of the token's slot.
- **pPin** ([CK_UTF8CHAR_PTR](#)) - [in] The SO's initial PIN (not used in this implementation).
- **ulPinLen** ([CK_ULONG](#)) - [in] The length in bytes of the PIN (not used in this implementation).
- **pLabel** ([CK_UTF8CHAR_PTR](#)) - [out] Pointer to the 32-byte token label (blank padded).

Description

This function initializes a token. The token label is set and the token is prepared for use.

Note:

This implementation does not support PIN management.

Return

- **CKR_OK**
Success.
- **CKR_ARGUMENTS_BAD:**
The `pLabel` is NULL.
- **CKR_TOKEN_NOT_PRESENT:**
The token is not present.
- **CKR_SESSION_EXISTS:**
A session is already open with the token.
- **CKR_SLOT_ID_INVALID:**
The slot ID is not valid.
- **CKR_ERROR_GENERAL:**
No slot available.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).

6.3.2.2.5 Token PIN management

6.3.2.2.5.1 C_InitPIN

[CK_RV](#) **C_InitPIN**([CK_SESSION_HANDLE](#) *hSession*, [CK_UTF8CHAR_PTR](#) *pPin*, [CK_ULONG](#) *ulPinLen*)

Initializes the normal user's PIN.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.
- **pPin** ([CK_UTF8CHAR_PTR](#)) - [in] The normal user's PIN.
- **ulPinLen** ([CK_ULONG](#)) - [in] The length in bytes of the PIN.

Description

This function initializes the normal user's PIN.

Note:

This implementation does not support PIN management.

Return

• CKR_FUNCTION_NOT_SUPPORTED:

Function is not supported.

6.3.2.2.5.2 C_SetPIN

[CK_RV](#) **C_SetPIN**([CK_SESSION_HANDLE](#) hSession, [CK_UTF8CHAR_PTR](#) pOldPin, [CK_ULONG](#) ulOldLen, [CK_UTF8CHAR_PTR](#) pNewPin, [CK_ULONG](#) ulNewLen)

Modifies the PIN of the user that is currently logged in.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle
- **pOldPin** ([CK_UTF8CHAR_PTR](#)) - [in] The old PIN
- **ulOldLen** ([CK_ULONG](#)) - [in] The length in bytes of the old PIN
- **pNewPin** ([CK_UTF8CHAR_PTR](#)) - [in] The new PIN
- **ulNewLen** ([CK_ULONG](#)) - [in] The length in bytes of the new PIN

Description

This function modifies the PIN of the user who is currently logged in, or the CKU_USER PIN if the session is not logged in.

Note:

This implementation does not support PIN management.

Return

• CKR_FUNCTION_NOT_SUPPORTED:

Function is not supported.

6.3.2.3 Example

The following example demonstrates how to enumerate slots, query token information, and get the mechanisms list and information:

```
#include <stdio.h>
#include <string.h>
#include <pkcs11.h>

static void print_slot_info(CK_SLOT_ID slotID, CK_SLOT_INFO *pInfo)
{
    printf("\nSlot ID: %lu\n", slotID);
    printf("  Description: %.64s\n", pInfo->slotDescription);
    printf("  Manufacturer: %.32s\n", pInfo->manufacturerID);
}
```

```

printf("  Flags: 0x%08lx\n", pInfo->flags);
if (pInfo->flags & CKF_TOKEN_PRESENT)
    printf("    - Token present\n");
if (pInfo->flags & CKF_REMOVABLE_DEVICE)
    printf("    - Removable device\n");
if (pInfo->flags & CKF_HW_SLOT)
    printf("    - Hardware slot\n");
}

static void print_token_info(CK_TOKEN_INFO *pInfo)
{
    printf("\nToken Information:\n");
    printf("  Label: %.32s\n", pInfo->label);
    printf("  Manufacturer: %.32s\n", pInfo->manufacturerID);
    printf("  Model: %.16s\n", pInfo->model);
    printf("  Serial Number: %.16s\n", pInfo->serialNumber);
    printf("  Flags: 0x%08lx\n", pInfo->flags);
    if (pInfo->flags & CKF_TOKEN_INITIALIZED)
        printf("    - Token initialized\n");
    if (pInfo->flags & CKF_WRITE_PROTECTED)
        printf("    - Write protected\n");
    if (pInfo->flags & CKF_USER_PIN_INITIALIZED)
        printf("    - User PIN initialized\n");
}

static void print_mechanism_info(CK_MECHANISM_TYPE type, CK_MECHANISM_INFO
*pInfo)
{
    printf("  Min key size: %lu bits\n", pInfo->ulMinKeySize);
    printf("  Max key size: %lu bits\n", pInfo->ulMaxKeySize);
    printf("  Flags: 0x%08lx\n", pInfo->flags);
    if (pInfo->flags & CKF_HW)
        printf("    - Hardware\n");
    if (pInfo->flags & CKF_ENCRYPT)
        printf("    - Encrypt\n");
    if (pInfo->flags & CKF_DECRYPT)
        printf("    - Decrypt\n");
    if (pInfo->flags & CKF_SIGN)
        printf("    - Sign\n");
    if (pInfo->flags & CKF_VERIFY)
        printf("    - Verify\n");
    if (pInfo->flags & CKF_GENERATE)
        printf("    - Generate\n");
    if (pInfo->flags & CKF_GENERATE_KEY_PAIR)
        printf("    - Generate key pair\n");
}

static const char *get_mechanism_name(CK_MECHANISM_TYPE type)
{
    switch (type) {
        case CKM_RSA_PKCS: return "CKM_RSA_PKCS";
        case CKM_RSA_PKCS_KEY_PAIR_GEN: return "CKM_RSA_PKCS_KEY_PAIR_GEN";
        case CKM_AES_KEY_GEN: return "CKM_AES_KEY_GEN";
        case CKM_AES_CBC: return "CKM_AES_CBC";
        case CKM_AES_GCM: return "CKM_AES_GCM";
        case CKM_SHA256: return "CKM_SHA256";
        case CKM_SHA384: return "CKM_SHA384";
        case CKM_SHA512: return "CKM_SHA512";
    }
}

```

```

    case CKM_ECDSA: return "CKM_ECDSA";
    case CKM_EC_KEY_PAIR_GEN: return "CKM_EC_KEY_PAIR_GEN";
    default: return "Unknown";
}

}

int main(void)
{
    CK_RV rv;
    CK_FUNCTION_LIST_PTR pFunctionList = NULL;
    CK_SLOT_ID_PTR pSlotList = NULL;
    CK_ULONG ulSlotCount = 0;
    CK_SLOT_INFO slotInfo = { 0 };
    CK_TOKEN_INFO tokenInfo = { 0 };
    CK_MECHANISM_TYPE_PTR pMechanismList = NULL;
    CK_ULONG ulMechCount = 0;
    CK_MECHANISM_INFO mechInfo = { 0 };
    CK_ULONG i = 0;
    CK_ULONG j = 0;

    /* Get function list */
    rv = C_GetFunctionList(&pFunctionList);
    if (rv != CKR_OK) {
        printf("C_GetFunctionList failed: 0x%08lx\n", rv);
        return 1;
    }

    /* Initialize PKCS#11 library */
    rv = pFunctionList->C_Initialize(NULL);
    if (rv != CKR_OK) {
        printf("C_Initialize failed: 0x%08lx\n", rv);
        return 1;
    }

    /* Get number of slots */
    rv = pFunctionList->C_GetSlotList(CK_TRUE, NULL, &ulSlotCount);
    if (rv != CKR_OK) {
        printf("C_GetSlotList (count) failed: 0x%08lx\n", rv);
        goto cleanup;
    }

    printf("Number of slots with tokens: %lu\n", ulSlotCount);

    if (ulSlotCount == 0) {
        printf("No slots with tokens found\n");
        goto cleanup;
    }

    /* Allocate memory for slot list */
    pSlotList = (CK_SLOT_ID_PTR)malloc(ulSlotCount * sizeof(CK_SLOT_ID));
    if (!pSlotList) {
        printf("Memory allocation failed\n");
        goto cleanup;
    }

    /* Get slot list */
    rv = pFunctionList->C_GetSlotList(CK_TRUE, pSlotList, &ulSlotCount);
    if (rv != CKR_OK) {
        printf("C_GetSlotList failed: 0x%08lx\n", rv);
    }
}

```

```
        goto cleanup;
    }

    /* Iterate through each slot */
    for (i = 0; i < ulSlotCount; i++) {
        /* Get slot information */
        rv = pFunctionList->C_GetSlotInfo(pSlotList[i], &slotInfo);
        if (rv != CKR_OK) {
            printf("C_GetSlotInfo failed for slot %lu: 0x%08lx\n",
                pSlotList[i], rv);
            continue;
        }

        print_slot_info(pSlotList[i], &slotInfo);

        /* Get token information */
        rv = pFunctionList->C_GetTokenInfo(pSlotList[i], &tokenInfo);
        if (rv != CKR_OK) {
            printf("C_GetTokenInfo failed for slot %lu: 0x%08lx\n",
                pSlotList[i], rv);
            continue;
        }

        print_token_info(&tokenInfo);

        /* Get number of mechanisms */
        rv = pFunctionList->C_GetMechanismList(pSlotList[i], NULL,
        &ulMechCount);
        if (rv != CKR_OK) {
            printf("C_GetMechanismList (count) failed: 0x%08lx\n", rv);
            continue;
        }

        printf("\nNumber of mechanisms: %lu\n", ulMechCount);

        if (ulMechCount == 0)
            continue;

        /* Allocate memory for mechanism list */
        pMechanismList = (CK_MECHANISM_TYPE_PTR)malloc(ulMechCount *
        sizeof(CK_MECHANISM_TYPE));
        if (!pMechanismList) {
            printf("Memory allocation failed for mechanisms\n");
            continue;
        }

        /* Get mechanism list */
        rv = pFunctionList->C_GetMechanismList(pSlotList[i], pMechanismList,
        &ulMechCount);
        if (rv != CKR_OK) {
            printf("C_GetMechanismList failed: 0x%08lx\n", rv);
            free(pMechanismList);
            pMechanismList = NULL;
            continue;
        }

        /* Display each mechanism and its information */
        printf("\nSupported Mechanisms:\n");
```

```

    for (j = 0; j < ulMechCount; j++) {
        printf("  %s (0x%08lx)\n",
               get_mechanism_name(pMechanismList[j]),
               pMechanismList[j]);

        rv = pFunctionList->C_GetMechanismInfo(pSlotList[i],
                                                pMechanismList[j],
                                                &mechInfo);

        if (rv == CKR_OK) {
            print_mechanism_info(pMechanismList[j], &mechInfo);
        } else {
            printf("    C_GetMechanismInfo failed: 0x%08lx\n", rv);
        }
    }

    free(pMechanismList);
    pMechanismList = NULL;
}

cleanup:
    if (pMechanismList)
        free(pMechanismList);
    if (pSlotList)
        free(pSlotList);

    /* Finalize PKCS#11 library */
    pFunctionList->C_Finalize(NULL);

    return 0;
}

```

6.3.3 Session Management

Session management in PKCS#11 is fundamental for interacting with cryptographic tokens. A session represents a logical connection between an application and a token, providing the context for performing cryptographic operations.

SMW's PKCS#11 implementation supports session management functions that allow applications to:

- Open and close sessions with cryptographic tokens
- Manage session state and properties
- Control access to token resources

Sessions can be opened in read-only or read-write mode, depending on the operations that need to be performed. All cryptographic operations (key generation, signing, encryption, etc.) must be performed within the context of an active session.

6.3.3.1 Functions

6.3.3.1.1 Open and close sessions

6.3.3.1.1.1 C_OpenSession

[CK_RV C_OpenSession\(CK_SLOT_ID slotID, CK_FLAGS flags, CK_VOID_PTR pApplication, CK_NOTIFY Notify, CK_SESSION_HANDLE_PTR phSession\)](#)

Opens a session between an application and a token.

Parameters

- **slotID** ([CK_SLOT_ID](#)) - [in] The slot's ID.
- **flags** ([CK_FLAGS](#)) - [in] Flags from CK_SESSION_INFO.
- **pApplication** ([CK_VOID_PTR](#)) - [in] Pointer passed to callback.
- **Notify** (CK_NOTIFY) - [in] Callback function.
- **phSession** ([CK_SESSION_HANDLE_PTR](#)) - [out] Pointer to the location that receives the session handle.

Description

This function opens a session between an application and a token in a particular slot.

The **flags** parameter must always set the CKF_SERIAL_SESSION flag. If the CKF_RW_SESSION flag is set, the session is opened as read/write, otherwise it is read-only.

If **pApplication** and **Notify** are provided, they define a callback function for notification events. Both must be set or both must be [NULL_PTR](#).

Return

- **CKR_OK:**
Success.
- **CKR_ARGUMENTS_BAD:**
 - The **phSession** is NULL.
 - The **pApplication** and **Notify** are inconsistent.
- **CKR_SESSION_PARALLEL_NOT_SUPPORTED:**
The CKF_SERIAL_SESSION flag is not set.
- **CKR_SLOT_ID_INVALID:**
The slot ID is not valid.
- **CKR_TOKEN_NOT_PRESENT:**
The token is not present.
- **CKR_TOKEN_NOT_RECOGNIZED:**
The token is not initialized.
- **CKR_HOST_MEMORY:**
Memory allocation failed.
- **CKR_SESSION_COUNT:**
The maximum number of sessions has been reached.
- **CKR_SESSION_READ_WRITE_SO_EXISTS:**
A read/write session already exists.
- **CKR_TOKEN_WRITE_PROTECTED:**
The token is write-protected.
- **CKR_GENERAL_ERROR:**
General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

6.3.3.1.1.2 C_CloseSession

[CK_RV](#) **C_CloseSession**([CK_SESSION_HANDLE](#) hSession)

Closes a session between an application and a token.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.

Description

This function closes a session between an application and a token. When a session is closed, all session objects created by the session are destroyed automatically.

Return

- **CKR_OK:**

Success.

- **CKR_SESSION_HANDLE_INVALID:**

The session handle is not valid.

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.3.1.1.3 C_CloseAllSessions

[CK_RV](#) **C_CloseAllSessions**([CK_SLOT_ID](#) slotID)

Closes all sessions with a token.

Parameters

- **slotID** ([CK_SLOT_ID](#)) - [in] The token's slot ID.

Description

This function closes all sessions an application has with a token. All session objects created by any session are destroyed automatically.

Return

- **CKR_OK:**

Success.

- **CKR_SLOT_ID_INVALID:**

The slot ID is not valid.

- **CKR_TOKEN_NOT_PRESENT:**

The token is not present.

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.3.1.1.4 C_SessionCancel

[CK_RV](#) [C_SessionCancel](#)([CK_SESSION_HANDLE](#) hSession, [CK_FLAGS](#) flags)

Cancels operations in a session.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.
- **flags** ([CK_FLAGS](#)) - [in] Flags indicating which operations to cancel.

Description

This function cancels one or more operations in a session.

Warning:

This function is not supported.

Return

- **CKR_FUNCTION_NOT_SUPPORTED:**

Function is not supported.

6.3.3.1.2 Get session information

6.3.3.1.2.1 C_GetSessionInfo

[CK_RV](#) [C_GetSessionInfo](#)([CK_SESSION_HANDLE](#) hSession, [CK_SESSION_INFO_PTR](#) pInfo)

Obtains information about the session.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.
- **pInfo** ([CK_SESSION_INFO_PTR](#)) - [out] Pointer to the location that receives the session information.

Description

This function obtains information about a session.

Return

- **CKR_OK:**
Success.
- **CKR_ARGUMENTS_BAD:**
The `pInfo` is NULL.
- **CKR_SESSION_HANDLE_INVALID:**
The session handle is not valid.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).

6.3.3.1.3 Operation state

6.3.3.1.3.1 C_GetOperationState

[CK_RV](#) **C_GetOperationState**([CK_SESSION_HANDLE](#) *hSession*, [CK_BYTE_PTR](#) *pOperationState*, [CK_ULONG_PTR](#) *pulOperationStateLen*)

Obtains the state of the cryptographic operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.
- **pOperationState** ([CK_BYTE_PTR](#)) - [out] Pointer to the location that receives the operation state, or [NULL_PTR](#) to query the state size.
- **pulOperationStateLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to the location that gives the size of `pOperationState` buffer, or receives the operation state length.

Description

This function obtains a snapshot of the cryptographic operation state of a session, encoded in a byte array.

If `pOperationState` is [NULL_PTR](#), the function returns the required buffer size in `pulOperationStateLen`.

If the `pOperationState` is not [NULL_PTR](#), the `pulOperationStateLen` contains the size of the buffer on input, and receives the actual state length on output. If the buffer is too small, the function returns `CKR_BUFFER_TOO_SMALL`.

Return

- **CKR_OK:**
Success.
- **CKR_ARGUMENTS_BAD:**
The `pulOperationStateLen` is NULL.
- **CKR_SESSION_HANDLE_INVALID:**
The session handle is not valid.

- **CKR_OPERATION_NOT_INITIALIZED:**
No operation is active in the session.
- **CKR_BUFFER_TOO_SMALL:**
The buffer provided is too small.
- **CKR_HOST_MEMORY:**
Memory allocation failed.
- **CKR_STATE_UNSAVEABLE:**
The operation state cannot be saved.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.3.1.3.2 C_SetOperationState

[CK_RV](#) **C_SetOperationState**([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pOperationState, [CK_ULONG](#) ulOperationStateLen, [CK_OBJECT_HANDLE](#) hEncryptionKey, [CK_OBJECT_HANDLE](#) hAuthenticationKey)

Restores the state of the cryptographic operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.
- **pOperationState** ([CK_BYTE_PTR](#)) - [in] Pointer to the saved operation state.
- **ulOperationStateLen** ([CK_ULONG](#)) - [in] Length of the saved operation state.
- **hEncryptionKey** ([CK_OBJECT_HANDLE](#)) - [in] Handle to encryption/decryption key (must be CK_INVALID_HANDLE in this implementation).
- **hAuthenticationKey** ([CK_OBJECT_HANDLE](#)) - [in] Handle to signature/verification key (must be CK_INVALID_HANDLE in this implementation).

Description

This function restores the cryptographic operation state of a session from a byte array previously obtained with [C_GetOperationState\(\)](#).

Note:

This implementation does not support key handles in this function. Both `hEncryptionKey` and `hAuthenticationKey` must be CK_INVALID_HANDLE.

Return

- **CKR_OK:**
Success.
- **CKR_ARGUMENTS_BAD:**
 - The `pOperationState` is NULL
 - The `ulOperationStateLen` is 0.

- **CKR_SESSION_HANDLE_INVALID:**

The session handle is not valid.

- **CKR_KEY_NOT_NEEDED:**

A key handle was provided when not needed.

- **CKR_SAVED_STATE_INVALID:**

The saved state is invalid or corrupted.

- **CKR_OPERATION_ACTIVE:**

An operation is already active in the session.

- **CKR_HOST_MEMORY:**

Memory allocation failed.

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

6.3.3.1.4 Session login

6.3.3.1.4.1 C_Login

[CK_RV](#) **C_Login**([CK_SESSION_HANDLE](#) hSession, [CK_USER_TYPE](#) userType, [CK_UTF8CHAR_PTR](#) pPin, [CK_ULONG](#) ulPinLen)

Logs a user into a token.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.
- **userType** ([CK_USER_TYPE](#)) - [in] The user type (CKU_SO, CKU_USER, or CKU_CONTEXT_SPECIFIC).
- **pPin** ([CK_UTF8CHAR_PTR](#)) - [in] The user's PIN (not used in this implementation).
- **ulPinLen** ([CK_ULONG](#)) - [in] The length of the PIN (not used in this implementation).

Description

This function logs a user into a token.

The `userType` parameter can be:

- CKU_SO: Security Officer
- CKU_USER: Normal user
- CKU_CONTEXT_SPECIFIC: Context-specific login

Note:

This implementation does not support PIN management. The `pPin` and `ulPinLen` parameters are ignored.

Return

- **CKR_OK:**
Success.
- **CKR_ARGUMENTS_BAD:**
The `userType` is invalid.
- **CKR_SESSION_HANDLE_INVALID:**
The session handle is not valid.
- **CKR_USER_ALREADY_LOGGED_IN:**
A user is already logged in.
- **CKR_USER_ANOTHER_USER_ALREADY_LOGGED_IN:**
Another user is already logged in.
- **CKR_SESSION_READ_ONLY_EXISTS:**
A read-only session exists.
- **CKR_HOST_MEMORY:**
Memory allocation failed.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).

6.3.3.1.4.2 C_LoginUser

[CK_RV](#) **C_LoginUser**([CK_SESSION_HANDLE](#) hSession, [CK_USER_TYPE](#) userType, [CK_UTF8CHAR_PTR](#) pPin, [CK_ULONG](#) ulPinLen, [CK_UTF8CHAR_PTR](#) pUsername, [CK_ULONG](#) ulUsernameLen)

Logs a user into a token with username.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.
- **userType** ([CK_USER_TYPE](#)) - [in] The user type (CKU_SO, CKU_USER, or CKU_CONTEXT_SPECIFIC).
- **pPin** ([CK_UTF8CHAR_PTR](#)) - [in] The user's PIN (not used in this implementation).
- **ulPinLen** ([CK_ULONG](#)) - [in] The length of the PIN (not used in this implementation).
- **pUsername** ([CK_UTF8CHAR_PTR](#)) - [in] The username (not used in this implementation).
- **ulUsernameLen** ([CK_ULONG](#)) - [in] The length of the username (not used in this implementation).

Description

This function logs a user into a token, providing both PIN and username.

Note:

This implementation does not support PIN or username management. The `pPin`, `ulPinLen`, `pUsername`, and `ulUsernameLen` parameters are ignored.

Return

- **CKR_OK:**
Success.
- **CKR_ARGUMENTS_BAD:**
The `userType` is invalid.
- **CKR_SESSION_HANDLE_INVALID:**
The session handle is not valid.
- **CKR_USER_ALREADY_LOGGED_IN:**
A user is already logged in.
- **CKR_USER_ANOTHER_USER_ALREADY_LOGGED_IN:**
Another user is already logged in.
- **CKR_SESSION_READ_ONLY_EXISTS:**
A read-only session exists.
- **CKR_HOST_MEMORY:**
Memory allocation failed.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).

6.3.3.1.4.3 C_Logout

[CK_RV](#) **C_Logout**([CK_SESSION_HANDLE](#) `hSession`)

Logs a user out from a token.

Parameters

- **`hSession`** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.

Description

This function logs a user out from a token.

Return

- **CKR_OK:**
Success.
- **CKR_SESSION_HANDLE_INVALID:**
The session handle is not valid.
- **CKR_USER_NOT_LOGGED_IN:**

No user is logged in.

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

6.3.3.1.5 Session flags

6.3.3.1.5.1 C_GetSessionValidationFlags

[CK_RV](#) **C_GetSessionValidationFlags**([CK_SESSION_HANDLE](#) hSession, [CK_SESSION_VALIDATION_FLAGS_TYPE](#) type, [CK_FLAGS_PTR](#) pFlags)

Gets session validation flags.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.
- **type** ([CK_SESSION_VALIDATION_FLAGS_TYPE](#)) - [in] The type of validation flags.
- **pFlags** ([CK_FLAGS_PTR](#)) - [out] Pointer to the location that receives the flags.

Description

This function gets session validation flags.

Warning:

This function is not supported.

Return

- **CKR_FUNCTION_NOT_SUPPORTED:**

Function is not supported.

6.3.3.2 Example

The following example demonstrates how to open a session, retrieve session information, and close the session:

```
CK_RV rv;
CK_SESSION_HANDLE hSession;
CK_SESSION_INFO sessionInfo = { 0 };
CK_SLOT_ID slotID = 0;
CK_FLAGS flags = CKF_SERIAL_SESSION | CKF_RW_SESSION;

/* Open a read-write session */
rv = C_OpenSession(slotID, flags, NULL, NULL, &hSession);
if (rv != CKR_OK) {
    printf("Failed to open session: 0x%lx\n", rv);
    return rv;
}

printf("Session opened successfully: %lu\n", hSession);
```

```
/* Get session information */
rv = C_GetSessionInfo(hSession, &sessionInfo);
if (rv != CKR_OK) {
    printf("Failed to get session info: 0x%lx\n", rv);
    C_CloseSession(hSession);
    return rv;
}

printf("Session state: %lu\n", sessionInfo.state);
printf("Session flags: 0x%lx\n", sessionInfo.flags);
printf("Device error: %lu\n", sessionInfo.ulDeviceError);

/* Perform cryptographic operations here */
/* ... */

/* Close the session */
rv = C_CloseSession(hSession);
if (rv != CKR_OK) {
    printf("Failed to close session: 0x%lx\n", rv);
    return rv;
}

printf("Session closed successfully\n");

return CKR_OK;
```

Note:

SMW's PKCS#11 implementation does not require PIN authentication for session operations. The `CKF_SERIAL_SESSION` flag must always be set when opening a session as per PKCS#11 specification.

Note:

To close all sessions associated with a token at once, use [C_CloseAllSessions\(\)](#) instead of closing each session individually.

6.3.4 Key Management

Key management in PKCS#11 provides comprehensive functions for generating, deriving, wrapping, and unwrapping cryptographic keys. These operations are essential for secure key lifecycle management.

SMW's PKCS#11 implementation supports key management operations that allow applications to:

- Generate symmetric and asymmetric keys
- Generate key pairs for public-key cryptography
- Derive keys from existing keys or shared secrets
- Wrap (encrypt) keys for secure transport
- Unwrap (decrypt) keys received from external sources

All key management operations must be performed within the context of an active session. Some operations may require specific session states or user authentication.

6.3.4.1 Key Creation

6.3.4.1.1 Symmetric Key Generation

6.3.4.1.1.1 C_GenerateKey

CK_RV C_GenerateKey([CK_SESSION_HANDLE](#) hSession, [CK_MECHANISM_PTR](#) pMechanism, [CK_ATTRIBUTE_PTR](#) pTemplate, [CK_ULONG](#) ulCount, [CK_OBJECT_HANDLE_PTR](#) phKey)

Generates a symmetric key.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.
- **pMechanism** ([CK_MECHANISM_PTR](#)) - [in] Pointer to the key generation mechanism.
- **pTemplate** ([CK_ATTRIBUTE_PTR](#)) - [in] Pointer to the template of the key attributes to create.
- **ulCount** ([CK_ULONG](#)) - [in] Number of attributes in the template.
- **phKey** ([CK_OBJECT_HANDLE_PTR](#)) - [out] Pointer to the location that receives the handle of the new key.

Description

This function generates a symmetric key, creating a new key object.

The type of key generated depends on the mechanism specified, hence it is not necessary to specify key type in template. If specified, it must match the key type generated by the mechanism otherwise function returns CKR_TEMPLATE_INCONSISTENT error.

The class CKA_CLASS attribute is not needed as well, if specified and not CKO_SECRET_KEY, the function returns CKR_TEMPLATE_INCONSISTENT error.

Return

- **CKR_OK:**
Success.
- **CKR_ARGUMENTS_BAD:**
The phKey is NULL.
- **CKR_SESSION_HANDLE_INVALID:**
The session handle is not valid.
- **CKR_MECHANISM_PARAM_INVALID:**
The pMechanism is NULL.
- **CKR_TEMPLATE_INCOMPLETE:**
The pTemplate and ulCount is incomplete or inconsistent.
- **CKR_TEMPLATE_INCONSISTENT:**
One of the template's attribute type must not be defined.
- **CKR_ATTRIBUTE_READ_ONLY:**
One attribute is read only

- **CKR_ATTRIBUTE_VALUE_INVALID:**
One of the template's attribute value is not valid.
- **CKR_USER_NOT_LOGGED_IN:**
User must login to create object.
- **CKR_SLOT_ID_INVALID:**
Slot ID is not valid.
- **CKR_MECHANISM_INVALID:**
Mechanism not supported.
- **CKR_HOST_MEMORY:**
Allocation error.
- **CKR_SESSION_CLOSED:**
Session closed.
- **CKR_FUNCTION_FAILED:**
Function failure.
- **CKR_FUNCTION_CANCELED:**
Application callback canceled function.
- **CKR_TOKEN_NOT_PRESENT:**
Token not present.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.4.1.2 Asymmetric Key Pair Generation

6.3.4.1.2.1 C_GenerateKeyPair

[CK_RV](#) **C_GenerateKeyPair**([CK_SESSION_HANDLE](#) hSession, [CK_MECHANISM_PTR](#) pMechanism, [CK_ATTRIBUTE_PTR](#) pPublicKeyTemplate, [CK_ULONG](#) ulPublicKeyAttributeCount, [CK_ATTRIBUTE_PTR](#) pPrivateKeyTemplate, [CK_ULONG](#) ulPrivateKeyAttributeCount, [CK_OBJECT_HANDLE_PTR](#) phPublicKey, [CK_OBJECT_HANDLE_PTR](#) phPrivateKey)

Generates an asymmetric key pair.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.
- **pMechanism** ([CK_MECHANISM_PTR](#)) - [in] Pointer to the key generation mechanism.
- **pPublicKeyTemplate** ([CK_ATTRIBUTE_PTR](#)) - [in] Pointer to the template of the public key attributes to create.
- **ulPublicKeyAttributeCount** ([CK_ULONG](#)) - [in] Number of attributes in the public key template.
- **pPrivateKeyTemplate** ([CK_ATTRIBUTE_PTR](#)) - [in] Pointer to the template of the private key attributes to create.
- **ulPrivateKeyAttributeCount** ([CK_ULONG](#)) - [in] Number of attributes in the private key template.
- **phPublicKey** ([CK_OBJECT_HANDLE_PTR](#)) - [out] Pointer to the location that receives the handle of the public key.
- **phPrivateKey** ([CK_OBJECT_HANDLE_PTR](#)) - [out] Pointer to the location that receives the handle of the private key.

Description

This function generates an asymmetric key pair, creating new key objects.

The type of key generated depends on the mechanism specified, hence it is not necessary to specify key type in templates. If specified, it must match the key type generated by the mechanism otherwise function returns `CKR_TEMPLATE_INCONSISTENT` error.

The class `CKA_CLASS` attribute is not needed as well, if specified and not `CKO_PUBLIC_KEY` or `CKO_PRIVATE_KEY` in the respective Public and Private template, the function returns `CKR_TEMPLATE_INCONSISTENT` error.

Return

- **CKR_OK:**
Success.
- **CKR_ARGUMENTS_BAD:**
 - The `phPublic` is NULL.
 - The `phPrivateKey` is NULL.
- **CKR_SESSION_HANDLE_INVALID:**
The session handle is not valid.
- **CKR_MECHANISM_PARAM_INVALID:**
The `pMechanism` is NULL.
- **CKR_TEMPLATE_INCOMPLETE:**
The `pTemplate` and `ulCount` is incomplete or inconsistent.
- **CKR_TEMPLATE_INCONSISTENT:**
One of the template's attribute type must not be defined.
- **CKR_ATTRIBUTE_READ_ONLY:**
One attribute is read only

- **CKR_ATTRIBUTE_VALUE_INVALID:**
One of the template's attribute value is not valid.
- **CKR_USER_NOT_LOGGED_IN:**
User must login to create object.
- **CKR_SLOT_ID_INVALID:**
Slot ID is not valid.
- **CKR_MECHANISM_INVALID:**
Mechanism not supported.
- **CKR_HOST_MEMORY:**
Allocation error.
- **CKR_SESSION_CLOSED:**
Session closed.
- **CKR_FUNCTION_FAILED:**
Function failure.
- **CKR_FUNCTION_CANCELED:**
Application callback canceled function.
- **CKR_TOKEN_NOT_PRESENT:**
Token not present.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.4.2 Key Derivation

6.3.4.2.1 C_DeriveKey

[CK_RV](#) **C_DeriveKey**([CK_SESSION_HANDLE](#) hSession, [CK_MECHANISM_PTR](#) pMechanism, [CK_OBJECT_HANDLE](#) hBaseKey, [CK_ATTRIBUTE_PTR](#) pTemplate, [CK_ULONG](#) ulAttributeCount, [CK_OBJECT_HANDLE_PTR](#) phKey)

Derives a secret key from a base key.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.
- **pMechanism** ([CK_MECHANISM_PTR](#)) - [in] Pointer to the key derivation mechanism.
- **hBaseKey** ([CK_OBJECT_HANDLE](#)) - [in] Handle of the base key.
- **pTemplate** ([CK_ATTRIBUTE_PTR](#)) - [in] Pointer to the template attributes for the new key.
- **ulAttributeCount** ([CK_ULONG](#)) - [in] Number of attributes in the template.
- **phKey** ([CK_OBJECT_HANDLE_PTR](#)) - [out] Pointer to the location that receives the handle of the derived key.

6.3.4.2.1.1 Description

This function derives a secret key from a base key, creating a new key object.

The class CKA_CLASS attribute is not needed as well, if specified and not CKO_PUBLIC_KEY or CKO_PRIVATE_KEY in the respective Public and Private template, the function returns CKR_TEMPLATE_INCONSISTENT error.

If the derived key type is a supported key derivation type, the function returns CKR_TEMPLATE_INCONSISTENT error.

6.3.4.2.1.2 Return

- **CKR_OK:**
Success.
- **CKR_SESSION_HANDLE_INVALID:**
The session handle is not valid.
- **CKR_KEY_HANDLE_INVALID:**
The hBaseKey is not valid.
- **CKR_ARGUMENTS_BAD:**
 - The pMechanism is NULL.
 - If the mechanism is not CKM_TLS12_KEY_AND_MAC_DERIVE and phKey is NULL.
- **CKR_TEMPLATE_INCOMPLETE:**
The pTemplate and ulCount is incomplete or inconsistent.
- **CKR_TEMPLATE_INCONSISTENT:**
One of the template's attribute type must not be defined.
- **CKR_ATTRIBUTE_READ_ONLY:**
One attribute is read only
- **CKR_ATTRIBUTE_VALUE_INVALID:**
One of the template's attribute value is not valid.
- **CKR_USER_NOT_LOGGED_IN:**
User must login to create object.
- **CKR_SLOT_ID_INVALID:**
Slot ID is not valid.
- **CKR_MECHANISM_INVALID:**
Mechanism not supported.
- **CKR_HOST_MEMORY:**
Allocation error.
- **CKR_SESSION_CLOSED:**
Session closed.
- **CKR_FUNCTION_FAILED:**
Function failure.

- **CKR_FUNCTION_CANCELED:**

Application callback canceled function.

- **CKR_TOKEN_NOT_PRESENT:**

Token not present.

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.4.3 Key Wrapping

6.3.4.3.1 C_WrapKey

[CK_RV](#) **C_WrapKey**([CK_SESSION_HANDLE](#) hSession, [CK_MECHANISM_PTR](#) pMechanism, [CK_OBJECT_HANDLE](#) hWrappingKey, [CK_OBJECT_HANDLE](#) hKey, [CK_BYTE_PTR](#) pWrappedKey, [CK_ULONG_PTR](#) pulWrappedKeyLen)

Wraps (encrypts) a private or secret key.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.
- **pMechanism** ([CK_MECHANISM_PTR](#)) - [in] Pointer to the wrapping mechanism.
- **hWrappingKey** ([CK_OBJECT_HANDLE](#)) - [in] Handle of the wrapping key.
- **hKey** ([CK_OBJECT_HANDLE](#)) - [in] Handle of the key to be wrapped.
- **pWrappedKey** ([CK_BYTE_PTR](#)) - [out] Pointer to the location that receives the wrapped key, or [NULL_PTR](#) to query the wrapped key size.
- **pulWrappedKeyLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to the location that gives the size of pWrappedKey buffer, or receives the wrapped key length.

6.3.4.3.1.1 Description

This function wraps (encrypts) a private or secret key.

Warning:

This function is not supported.

6.3.4.3.1.2 Return

- **CKR_FUNCTION_NOT_SUPPORTED:**

Function is not supported.

6.3.4.3.2 C_UnwrapKey

[CK_RV](#) **C_UnwrapKey**([CK_SESSION_HANDLE](#) hSession, [CK_MECHANISM_PTR](#) pMechanism, [CK_OBJECT_HANDLE](#) hUnwrappingKey, [CK_BYTE_PTR](#) pWrappedKey, [CK_ULONG](#) ulWrappedKeyLen, [CK_ATTRIBUTE_PTR](#) pTemplate, [CK_ULONG](#) ulAttributeCount, [CK_OBJECT_HANDLE_PTR](#) phKey)

Unwraps (decrypts) a private or secret key.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] The session's handle.
- **pMechanism** ([CK_MECHANISM_PTR](#)) - [in] Pointer to the unwrapping mechanism.
- **hUnwrappingKey** ([CK_OBJECT_HANDLE](#)) - [in] Handle of the unwrapping key.
- **pWrappedKey** ([CK_BYTE_PTR](#)) - [in] Pointer to the wrapped key.
- **ulWrappedKeyLen** ([CK_ULONG](#)) - [in] Length of the wrapped key.
- **pTemplate** ([CK_ATTRIBUTE_PTR](#)) - [in] Pointer to the template attributes for the unwrapped key.
- **ulAttributeCount** ([CK_ULONG](#)) - [in] Number of attributes in the template.
- **phKey** ([CK_OBJECT_HANDLE_PTR](#)) - [out] Pointer to the location that receives the handle of the unwrapped key.

6.3.4.3.2.1 Description

This function unwraps (decrypts) a private or secret key and creates a new key.

Warning:

This function is not supported.

6.3.4.3.2.2 Return

- **CKR_FUNCTION_NOT_SUPPORTED:**

Function is not supported.

6.3.4.4 Key Exportation

The key exportation functionality is not directly supported through a dedicated API. The key value can be accessed through the [C_GetAttributeValue\(\)](#) function by querying the CKA_VALUE attribute, provided the key object has the CKA_EXTRACTABLE attribute set to CK_TRUE.

6.3.4.5 Key Import

A key can be imported into the token through the [C_CreateObject\(\)](#) function. This allows applications to load pre-existing keys or keys generated externally into the PKCS#11 token for use in cryptographic operations.

6.3.4.6 Key Deletion

Keys can be deleted from the token using the [C_DestroyObject\(\)](#) function. This operation permanently removes the key object and its associated attributes from the token storage.

6.3.4.7 Key Attributes

Key attributes define the properties and permissions of a key object. These attributes can be set during key creation and queried at any time using [C_GetAttributeValue\(\)](#).

The key attributes differs based on the type of the key object. For detailed information on key attributes, refer to the PKCS#11 specification:

- [PKCS#11 v2.40](#).
- [PKCS#11 v3.2](#).

6.3.4.8 Examples

6.3.4.8.1 Example 1: Generate AES Key

```
#include <stdio.h>
#include <pkcs11.h>

CK_RV generate_aes_key(CK_SESSION_HANDLE hSession,
                      CK_OBJECT_HANDLE *phKey)
{
    CK_RV rv;
    CK_MECHANISM mechanism = { CKM_AES_KEY_GEN, NULL_PTR, 0 };
    CK_OBJECT_CLASS keyClass = CKO_SECRET_KEY;
    CK_KEY_TYPE keyType = CKK_AES;
    CK_ULONG keyLen = 32; /* 256 bits */
    CK_BBOOL bTrue = CK_TRUE;
    CK_BBOOL bFalse = CK_FALSE;
    CK_UTF8CHAR label[] = "My AES Key";

    CK_ATTRIBUTE keyTemplate[] = {
        { CKA_CLASS, &keyClass, sizeof(keyClass) },
        { CKA_KEY_TYPE, &keyType, sizeof(keyType) },
        { CKA_VALUE_LEN, &keyLen, sizeof(keyLen) },
        { CKA_TOKEN, &bTrue, sizeof(bTrue) },
        { CKA_PRIVATE, &bTrue, sizeof(bTrue) },
        { CKA_SENSITIVE, &bTrue, sizeof(bTrue) },
        { CKA_ENCRYPT, &bTrue, sizeof(bTrue) },
        { CKA_DECRYPT, &bTrue, sizeof(bTrue) },
        { CKA_EXTRACTABLE, &bFalse, sizeof(bFalse) },
        { CKA_LABEL, label, sizeof(label) - 1 }
    };

    rv = C_GenerateKey(hSession, &mechanism, keyTemplate, 10, phKey);
    if (rv != CKR_OK) {
        printf("Failed to generate AES key: 0x%lx\n", rv);
        return rv;
    }

    printf("AES key generated successfully: %lu\n", *phKey);
    return CKR_OK;
}
```

6.3.4.8.2 Example 2: Generate RSA Key Pair

```
#include <stdio.h>
#include <pkcs11.h>

CK_RV generate_rsa_keypair(CK_SESSION_HANDLE hSession,
                          CK_OBJECT_HANDLE *phPublicKey,
                          CK_OBJECT_HANDLE *phPrivateKey)
{
    CK_RV rv;
    CK_MECHANISM mechanism = { CKM_RSA_PKCS_KEY_PAIR_GEN, NULL_PTR, 0 };
    CK_ULONG modulusBits = 2048;
    CK_BYTE publicExponent[] = { 0x01, 0x00, 0x01 }; /* 65537 */
    CK_BBOOL bTrue = CK_TRUE;
    CK_BBOOL bFalse = CK_FALSE;
```



```

CK_UTF8CHAR pubLabel[] = "My RSA Public Key";
CK_UTF8CHAR privLabel[] = "My RSA Private Key";
CK_BYTE id[] = { 0x01, 0x02, 0x03, 0x04 };

CK_ATTRIBUTE publicKeyTemplate[] = {
    { CKA_TOKEN, &bTrue, sizeof(bTrue) },
    { CKA_PRIVATE, &bFalse, sizeof(bFalse) },
    { CKA_ENCRYPT, &bTrue, sizeof(bTrue) },
    { CKA_VERIFY, &bTrue, sizeof(bTrue) },
    { CKA_WRAP, &bTrue, sizeof(bTrue) },
    { CKA_MODULUS_BITS, &modulusBits, sizeof(modulusBits) },
    { CKA_PUBLIC_EXPONENT, publicExponent, sizeof(publicExponent) },
    { CKA_LABEL, pubLabel, sizeof(pubLabel) - 1 },
    { CKA_ID, id, sizeof(id) }
};

CK_ATTRIBUTE privateKeyTemplate[] = {
    { CKA_TOKEN, &bTrue, sizeof(bTrue) },
    { CKA_PRIVATE, &bTrue, sizeof(bTrue) },
    { CKA_SENSITIVE, &bTrue, sizeof(bTrue) },
    { CKA_DECRYPT, &bTrue, sizeof(bTrue) },
    { CKA_SIGN, &bTrue, sizeof(bTrue) },
    { CKA_UNWRAP, &bTrue, sizeof(bTrue) },
    { CKA_EXTRACTABLE, &bFalse, sizeof(bFalse) },
    { CKA_LABEL, privLabel, sizeof(privLabel) - 1 },
    { CKA_ID, id, sizeof(id) }
};

rv = C_GenerateKeyPair(hSession, &mechanism,
                      publicKeyTemplate, 9,
                      privateKeyTemplate, 8,
                      phPublicKey, phPrivateKey);

if (rv != CKR_OK) {
    printf("Failed to generate RSA key pair: 0x%lx\n", rv);
    return rv;
}

printf("RSA key pair generated successfully\n");
printf("  Public key: %lu\n", *phPublicKey);
printf("  Private key: %lu\n", *phPrivateKey);
return CKR_OK;
}

```

6.3.4.8.3 Example 3: ECDH Key Derivation

```

#include <stdio.h>
#include <string.h>
#include <pkcs11.h>

CK_RV derive_ecdh_key(CK_SESSION_HANDLE hSession,
                     CK_OBJECT_HANDLE hPrivateKey,
                     CK_BYTE_PTR pPublicData,
                     CK_ULONG ulPublicDataLen,
                     CK_OBJECT_HANDLE *phDerivedKey)
{
    CK_RV rv;
    CK_ECDH1_DERIVE_PARAMS ecdhParams;
    CK_MECHANISM mechanism;

```

```

CK_OBJECT_CLASS keyClass = CKO_SECRET_KEY;
CK_KEY_TYPE keyType = CKK_AES;
CK_ULONG keyLen = 32; /* 256 bits */
CK_BBOOL bTrue = CK_TRUE;
CK_BBOOL bFalse = CK_FALSE;
CK_UTF8CHAR label[] = "Derived AES Key";

/* Setup ECDH parameters */
ecdhParams.kdf = CKD_NULL;
ecdhParams.ulSharedDataLen = 0;
ecdhParams.pSharedData = NULL_PTR;
ecdhParams.ulPublicDataLen = ulPublicDataLen;
ecdhParams.pPublicData = pPublicData;

mechanism.mechanism = CKM_ECDH1_DERIVE;
mechanism.pParameter = &ecdhParams;
mechanism.ulParameterLen = sizeof(ecdhParams);

CK_ATTRIBUTE derivedKeyTemplate[] = {
    { CKA_CLASS, &keyClass, sizeof(keyClass) },
    { CKA_KEY_TYPE, &keyType, sizeof(keyType) },
    { CKA_VALUE_LEN, &keyLen, sizeof(keyLen) },
    { CKA_TOKEN, &bFalse, sizeof(bFalse) },
    { CKA_SENSITIVE, &bTrue, sizeof(bTrue) },
    { CKA_ENCRYPT, &bTrue, sizeof(bTrue) },
    { CKA_DECRYPT, &bTrue, sizeof(bTrue) },
    { CKA_EXTRACTABLE, &bFalse, sizeof(bFalse) },
    { CKA_LABEL, label, sizeof(label) - 1 }
};

rv = C_DeriveKey(hSession, &mechanism, hPrivateKey,
                derivedKeyTemplate, 9, phDerivedKey);
if (rv != CKR_OK) {
    printf("Failed to derive key: 0x%lx\n", rv);
    return rv;
}

printf("Key derived successfully: %lu\n", *phDerivedKey);
return CKR_OK;
}

```

6.3.5 Object Management

Object management in PKCS#11 provides comprehensive functions for creating, searching, modifying, and destroying cryptographic objects. These operations are essential for managing keys, certificates, and data objects throughout their lifecycle.

SMW's PKCS#11 implementation supports object management operations that allow applications to:

- Create new objects with specified attributes.
- Search for objects based on attribute criteria.
- Retrieve and modify object attributes.
- Copy existing objects.
- Destroy objects when no longer needed.

All object management operations must be performed within the context of an active session. Some operations may require specific session states or user authentication depending on the object's attributes (e.g., private objects).

6.3.5.1 Object Creation and Destruction

6.3.5.1.1 Create Object

6.3.5.1.1.1 C_CreateObject

[CK_RV](#) **C_CreateObject**([CK_SESSION_HANDLE](#) hSession, [CK_ATTRIBUTE_PTR](#) pTemplate, [CK_ULONG](#) ulCount, [CK_OBJECT_HANDLE_PTR](#) phObject)

Create a new object.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pTemplate** ([CK_ATTRIBUTE_PTR](#)) - [in] Pointer to the object's attribute template.
- **ulCount** ([CK_ULONG](#)) - [in] Number of attributes in the template.
- **phObject** ([CK_OBJECT_HANDLE_PTR](#)) - [out] Pointer to location that receives the handle of the new object.

Description

The function creates a new object from a template and returns the new object's handle.

If the attribute 'CKA_TOKEN' is set to 'CK_TRUE', the object is a token object and will be stored in the token. Otherwise, it is a session object.

Return

- **CKR_OK:**
Success.
- **CKR_ARGUMENTS_BAD:**
The `phObject` is NULL.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle is not valid.
- **CKR_TEMPLATE_INCOMPLETE:**
The `pTemplate` and `ulCount` is incomplete or inconsistent.
- **CKR_TEMPLATE_INCONSISTENT:**
One of the template's attribute type must not be defined.
- **CKR_ATTRIBUTE_READ_ONLY:**
One attribute is read only
- **CKR_ATTRIBUTE_VALUE_INVALID:**
One of the template's attribute value is not valid.
- **CKR_HOST_MEMORY:**
Allocation error.
- **CKR_GENERAL_ERROR:**
General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.5.1.2 Copy Object

6.3.5.1.2.1 C_CopyObject

[CK_RV](#) **C_CopyObject**([CK_SESSION_HANDLE](#) hSession, [CK_OBJECT_HANDLE](#) hObject, [CK_ATTRIBUTE_PTR](#) pTemplate, [CK_ULONG](#) ulCount, [CK_OBJECT_HANDLE_PTR](#) phNewObject)

Copy an object.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **hObject** ([CK_OBJECT_HANDLE](#)) - [in] Handle of the object to be copied.
- **pTemplate** ([CK_ATTRIBUTE_PTR](#)) - [in] Pointer to attribute template for the new object.
- **ulCount** ([CK_ULONG](#)) - [in] Number of attributes in the template.
- **phNewObject** ([CK_OBJECT_HANDLE_PTR](#)) - [out] Pointer to location that receives the handle of the copy.

Description

The function copies an object hObject, creating a new object for the copy.

Note:

This implementation does not support object copying.

Return

- **CKR_FUNCTION_NOT_SUPPORTED:**

Function is not supported.

6.3.5.1.3 Destroy Object

6.3.5.1.3.1 C_DestroyObject

[CK_RV](#) **C_DestroyObject**([CK_SESSION_HANDLE](#) hSession, [CK_OBJECT_HANDLE](#) hObject)

Destroy an object.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **hObject** ([CK_OBJECT_HANDLE](#)) - [in] Handle of the object to be destroyed.

Description

The function destroys a token or session object.

Return

- **CKR_OK:**
Success.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle is not valid.
- **CKR_OBJECT_HANDLE_INVALID:**
The `hObject` handle is not valid.
- **CKR_ACTION_PROHIBITED:**
 - The object cannot be destroyed due to `CKA_DESTROYABLE` attribute.
 - The session is read-only.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.5.2 Object Search

6.3.5.2.1 Find Objects Initialize

6.3.5.2.1.1 C_FindObjectsInit

[CK_RV](#) **C_FindObjectsInit**([CK_SESSION_HANDLE](#) *hSession*, [CK_ATTRIBUTE_PTR](#) *pTemplate*, [CK_ULONG](#) *ulCount*)

Initialize a search for token and session objects.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pTemplate** ([CK_ATTRIBUTE_PTR](#)) - [in] Pointer to search template that specifies the attribute values to match.
- **ulCount** ([CK_ULONG](#)) - [in] Number of attributes in the search template.

Description

The function initializes a search for token and session objects that match the input `pTemplate`. After calling [C_FindObjectsInit\(\)](#), the application may call [C_FindObjects\(\)](#) one or more times to obtain handles for objects matching the template, and then eventually call [C_FindObjectsFinal\(\)](#) to finish the active search operation.

Passing the `ulCount` as 0 will match all objects.

Return

- **CKR_OK:**
Success.
- **CKR_ARGUMENTS_BAD:**

The `pTemplate` is NULL and `ulCount` is not 0.

- **CKR_SESSION_HANDLE_INVALID:**
Session handle is not valid.
- **CKR_ATTRIBUTE_TYPE_INVALID:**
Attribute type is invalid.
- **CKR_ATTRIBUTE_VALUE_INVALID:**
Attribute value is invalid.
- **CKR_OPERATION_ACTIVE:**
Find operation is already active. The [C_FindObjectsFinal\(\)](#) must be called to start a new search.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.5.2.2 Find Objects

6.3.5.2.2.1 C_FindObjects

CK_RV C_FindObjects([CK_SESSION_HANDLE](#) *hSession*, [CK_OBJECT_HANDLE_PTR](#) *phObject*, [CK_ULONG](#) *ulMaxObjectCount*, [CK_ULONG_PTR](#) *pulObjectCount*)

Continue a search for token and session objects.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **phObject** ([CK_OBJECT_HANDLE_PTR](#)) - [out] Pointer to location that receives the list of object handles.
- **ulMaxObjectCount** ([CK_ULONG](#)) - [in] Maximum number of object handles to be returned.
- **pulObjectCount** ([CK_ULONG_PTR](#)) - [out] Pointer to location that receives the actual number of object handles returned.

Description

The function returns the object handles matching the template specified during the [C_FindObjectsInit\(\)](#) initialization function.

If there is no more objects to find, the function returns CKR_OK and sets `pulObjectCount` to 0.

Return

- **CKR_OK:**
Success.
- **CKR_ARGUMENTS_BAD:**
The `phObject` is NULL. The `pulObjectCount` is NULL.

- **CKR_SESSION_HANDLE_INVALID:**

Session handle is not valid.

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.5.2.3 Find Objects Final

6.3.5.2.3.1 C_FindObjectsFinal

[CK_RV](#) **C_FindObjectsFinal**([CK_SESSION_HANDLE](#) hSession)

Finish a search for token and session objects.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.

Description

The function finishes a search for token and session objects.

Return

CKR_SESSION_HANDLE_INVALID - Session handle is not valid. CKR_OK - Success. Others - Error from `libobj_find_final()`.

6.3.5.3 Object Attributes

6.3.5.3.1 Get Attribute Value

6.3.5.3.1.1 C_GetAttributeValue

[CK_RV](#) **C_GetAttributeValue**([CK_SESSION_HANDLE](#) hSession, [CK_OBJECT_HANDLE](#) hObject, [CK_ATTRIBUTE_PTR](#) pTemplate, [CK_ULONG](#) ulCount)

Get the value of one or more attributes of an object.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **hObject** ([CK_OBJECT_HANDLE](#)) - [in] Handle of the object.
- **pTemplate** ([CK_ATTRIBUTE_PTR](#)) - [in/out] Pointer to template that specifies which attributes to get.
- **ulCount** ([CK_ULONG](#)) - [in] Number of attributes in the template.

Description

The function obtains the value of one or more attributes of an object. The function is parsing the template to get the requested attributes, while there is no fatal failure (error other than CKR_ATTRIBUTE_TYPE_INVALID,

CKR_BUFFER_TOO_SMALL, CKR_ATTRIBUTE_SENSITIVE), function continues to parse the template. At the end, the first error encountered is returned.

Setting the 'pValue' of the template to NULL will return the size of the attribute value in the 'ulValueLen' field without returning error.

Return

- **CKR_OK:**
Success.
- **CKR_ARGUMENTS_BAD:**
The `hObject` is NULL.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle is not valid.
- **CKR_TEMPLATE_INCOMPLETE:**
Template is incomplete or invalid.
- **CKR_BUFFER_TOO_SMALL:**
The buffer for an attribute is too small, attribute's value is updated with the expected value.
- **CKR_ATTRIBUTE_TYPE_INVALID:**
Attribute type is invalid.
- **CKR_ATTRIBUTE_SENSITIVE:**
Attribute is sensitive or unextractable.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.5.3.2 Set Attribute Value

6.3.5.3.2.1 C_SetAttributeValue

[CK_RV](#) **C_SetAttributeValue**([CK_SESSION_HANDLE](#) *hSession*, [CK_OBJECT_HANDLE](#) *hObject*, [CK_ATTRIBUTE_PTR](#) *pTemplate*, [CK_ULONG](#) *ulCount*)

Modify the value of one or more attributes of an object.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **hObject** ([CK_OBJECT_HANDLE](#)) - [in] Handle of the object.
- **pTemplate** ([CK_ATTRIBUTE_PTR](#)) - [in] Pointer to template that specifies which attributes to modify.
- **ulCount** ([CK_ULONG](#)) - [in] Number of attributes in the template.

Description

The function modifies the value of one or more attributes of an object.

If an object can not be modified, the function returns with `CKR_ACTION_PROHIBITED`. The application can consult the object's `CKA_MODIFIABLE` attribute to determine if modification is allowed.

Return

- **CKR_OK:**
Success.
- **CKR_ARGUMENTS_BAD:**
The `hObject` is NULL.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle is not valid.
- **CKR_TEMPLATE_INCOMPLETE:**
Template is incomplete or invalid.
- **CKR_ATTRIBUTE_TYPE_INVALID:**
Attribute type is invalid.
- **CKR_ATTRIBUTE_VALUE_INVALID:**
Attribute *pValue* is NULL.
- **CKR_ATTRIBUTE_READ_ONLY:**
Attribute is read-only.
- **CKR_ACTION_PROHIBITED:**
Object can not be modified.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.5.3.3 Get Object Size

6.3.5.3.3.1 C_GetObjectSize

[CK_RV](#) **C_GetObjectSize**([CK_SESSION_HANDLE](#) *hSession*, [CK_OBJECT_HANDLE](#) *hObject*, [CK_ULONG_PTR](#) *pulSize*)

Get the size of an object.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **hObject** ([CK_OBJECT_HANDLE](#)) - [in] Handle of the object.
- **pulSize** ([CK_ULONG_PTR](#)) - [out] Pointer to location that receives the size in bytes.

The function returns the size in bytes of an object in bytes

- For a key object: the size of the private or secret key.
- For a certificate object: the size of the certificate data.
- For a data object: the size of the data.

Return

- **CKR_OK:**
Success.
- **CKR_ARGUMENTS_BAD:**
 - The `hObject` is NULL.
 - The `pulSize` is NULL.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle is not valid.
- **CKR_OBJECT_HANDLE_INVALID:**
The `hObject` handle is not valid.
- **CKR_ACTION_PROHIBITED:**
 - The object cannot be destroyed due to `CKA_DESTROYABLE` attribute.
 - The session is read-only.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.5.4 Examples

6.3.5.4.1 Example 1: Create Data Object

```
#include <stdio.h>
#include <string.h>
#include <pkcs11.h>

CK_RV create_data_object(CK_SESSION_HANDLE hSession,
                        CK_OBJECT_HANDLE *phObject)
{
    CK_RV rv;
    CK_OBJECT_CLASS objClass = CKO_DATA;
    CK_BBOOL bTrue = CK_TRUE;
    CK_BBOOL bFalse = CK_FALSE;
    CK_UTF8CHAR label[] = "My Data Object";
    CK_UTF8CHAR application[] = "My Application";
    CK_BYTE data[] = "This is my secret data";

    CK_ATTRIBUTE template[] = {
        { CKA_CLASS, &objClass, sizeof(objClass) },
        { CKA_TOKEN, &bTrue, sizeof(bTrue) },
```

```

    { CKA_PRIVATE, &bTrue, sizeof(bTrue) },
    { CKA_MODIFIABLE, &bTrue, sizeof(bTrue) },
    { CKA_LABEL, label, sizeof(label) - 1 },
    { CKA_APPLICATION, application, sizeof(application) - 1 },
    { CKA_VALUE, data, sizeof(data) - 1 }
};

rv = C_CreateObject(hSession, template, 7, phObject);
if (rv != CKR_OK) {
    printf("Failed to create data object: 0x%lx\n", rv);
    return rv;
}

printf("Data object created successfully: %lu\n", *phObject);
return CKR_OK;
}

```

6.3.5.4.2 Example 2: Search for Objects

```

#include <stdio.h>
#include <string.h>
#include <pkcs11.h>

CK_RV find_aes_keys(CK_SESSION_HANDLE hSession)
{
    CK_RV rv;
    CK_OBJECT_CLASS keyClass = CKO_SECRET_KEY;
    CK_KEY_TYPE keyType = CKK_AES;
    CK_BBOOL bTrue = CK_TRUE;
    CK_OBJECT_HANDLE hObjects[10] = { 0 };
    CK_ULONG ulObjectCount = 0;
    CK_ULONG i = 0;

    CK_ATTRIBUTE template[] = {
        { CKA_CLASS, &keyClass, sizeof(keyClass) },
        { CKA_KEY_TYPE, &keyType, sizeof(keyType) },
        { CKA_ENCRYPT, &bTrue, sizeof(bTrue) }
    };

    /* Initialize search */
    rv = C_FindObjectsInit(hSession, template, 3);
    if (rv != CKR_OK) {
        printf("Failed to initialize object search: 0x%lx\n", rv);
        return rv;
    }

    /* Find objects */
    rv = C_FindObjects(hSession, hObjects, 10, &ulObjectCount);
    if (rv != CKR_OK) {
        printf("Failed to find objects: 0x%lx\n", rv);
        C_FindObjectsFinal(hSession);
        return rv;
    }

    printf("Found %lu AES key(s):\n", ulObjectCount);
    for (i = 0; i < ulObjectCount; i++) {
        printf("    Object handle: %lu\n", hObjects[i]);
    }
}

```

```

/* Finalize search */
rv = C_FindObjectsFinal(hSession);
if (rv != CKR_OK) {
    printf("Failed to finalize object search: 0x%lx\n", rv);
    return rv;
}

return CKR_OK;
}

```

6.3.5.4.3 Example 3: Get and Set Object Attributes

```

#include <stdio.h>
#include <string.h>
#include <pkcs11.h>

CK_RV modify_object_label(CK_SESSION_HANDLE hSession,
                          CK_OBJECT_HANDLE hObject)
{
    CK_RV rv;
    CK_UTF8CHAR oldLabel[256] = { 0 };
    CK_ULONG oldLabelLen = 0;
    CK_UTF8CHAR newLabel[] = "Updated Label";

    /* Get current label */
    CK_ATTRIBUTE getTemplate[] = {
        { CKA_LABEL, oldLabel, sizeof(oldLabel) }
    };

    rv = C_GetAttributeValue(hSession, hObject, getTemplate, 1);
    if (rv != CKR_OK) {
        printf("Failed to get object label: 0x%lx\n", rv);
        return rv;
    }

    oldLabelLen = getTemplate[0].ulValueLen;
    oldLabel[oldLabelLen] = '\0';
    printf("Current label: %s\n", oldLabel);

    /* Set new label */
    CK_ATTRIBUTE setTemplate[] = {
        { CKA_LABEL, newLabel, sizeof(newLabel) - 1 }
    };

    rv = C_SetAttributeValue(hSession, hObject, setTemplate, 1);
    if (rv != CKR_OK) {
        printf("Failed to set object label: 0x%lx\n", rv);
        return rv;
    }

    printf("Label updated to: %s\n", newLabel);
    return CKR_OK;
}

```

6.3.5.4.4 Example 4: Destroy Object

```
#include <stdio.h>
#include <pkcs11.h>

CK_RV destroy_object(CK_SESSION_HANDLE hSession,
                     CK_OBJECT_HANDLE hObject)
{
    CK_RV rv;

    rv = C_DestroyObject(hSession, hObject);
    if (rv != CKR_OK) {
        printf("Failed to destroy object: 0x%lx\n", rv);
        return rv;
    }

    printf("Object %lu destroyed successfully\n", hObject);
    return CKR_OK;
}
```

6.3.6 Cryptography APIs

The Cryptography APIs allow user of the library to perform cryptographic operations such as hashing, signing, encryption, decryption, MAC and RNG. For some cryptographic operations, both oneshot and multipart modes are supported.

6.3.6.1 Random Number Generation

6.3.6.1.1 Functions

6.3.6.1.1.1 C_SeedRandom

CK_RV C_SeedRandom(CK_SESSION_HANDLE hSession, CK_BYTE_PTR pSeed, CK_ULONG ulSeedLen)

Mix additional seed material into the random number generator.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pSeed** ([CK_BYTE_PTR](#)) - [in] Pointer to the seed material.
- **ulSeedLen** ([CK_ULONG](#)) - [in] Length of the seed material in bytes.

Description

Note:

This function is not supported by this implementation.

Return

- **CKR_RANDOM_SEED_NOT_SUPPORTED:**
Function not supported

6.3.6.1.1.2 C_GenerateRandom

[CK_RV](#) C_GenerateRandom([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pRandomData, [CK_ULONG](#) ulRandomLen)

Generate random data.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pRandomData** ([CK_BYTE_PTR](#)) - [out] Pointer to buffer to receive the random data.
- **ulRandomLen** ([CK_ULONG](#)) - [in] Length of random data to generate in bytes.

Description

This function generates random or pseudo-random data for the specified session.

Return

- **CKR_OK:**
Success.
- **CKR_SESSION_HANDLE_INVALID:**
Invalid session handle.
- **CKR_ARGUMENTS_BAD**
 - The pRandomData is NULL.
 - The ulRandomLen is 0.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.2 Message digests (Hashes)

The PKCS#11 library provides message digest (hash) operations through a set of APIs that support both single-part and multi-part hashing.

The sequence of operations to digest a message with a single part operation is as follows:

1. Initialize the operation by calling [C_DigestInit\(\)](#).
2. Digest data in a single operation with the [C_Digest\(\)](#).

The sequence of operations to digest a message with a multiple part operation is as follows:

1. Initialize the operation by calling [C_DigestInit\(\)](#).
2. Process one or more data parts (can be called multiple times) with the [C_DigestUpdate\(\)](#).
3. Finish the digest operation and obtain the digest value with the [C_DigestFinal\(\)](#).

6.3.6.2.1 Initialization

6.3.6.2.1.1 C_DigestInit

CK_RV C_DigestInit([CK_SESSION_HANDLE](#) hSession, [CK_MECHANISM_PTR](#) pMechanism)

Initialize message-digesting operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pMechanism** ([CK_MECHANISM_PTR](#)) - [in] Digesting mechanism.

Description

After calling [C_DigestInit\(\)](#), the application can either call [C_Digest\(\)](#) to digest data in a single part; or call [C_DigestUpdate\(\)](#) zero or more times, followed by [C_DigestFinal\(\)](#), to digest data in multiple parts. The digesting operation is active until the application uses a call to [C_Digest\(\)](#) or [C_DigestFinal\(\)](#) to actually obtain the final message digest. To process additional data (in single or multiple parts), the application MUST call [C_DigestInit\(\)](#) again.

[C_DigestInit\(\)](#) can be called with pMechanism set to NULL_PTR to terminate an active digesting operation. If an active operation cannot be cancelled, CKR_OPERATION_CANCEL_FAILED must be returned.

Return

- **CKR_OK:**
Success. Digesting operation initialized or cancelled if the pMechanism is NULL_PTR.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle hSession is invalid.
- **CKR_MECHANISM_INVALID**
- **CKR_GENERAL_ERROR**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.2.2 Single Part

6.3.6.2.2.1 C_Digest

CK_RV C_Digest([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pData, [CK_ULONG](#) ulDataLen, [CK_BYTE_PTR](#) pDigest, [CK_ULONG_PTR](#) pulDigestLen)

Digest data in a single part.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pData** ([CK_BYTE_PTR](#)) - [in] Data buffer to be digested.
- **ulDataLen** ([CK_ULONG](#)) - [in] Length of data to be digested.
- **pDigest** ([CK_BYTE_PTR](#)) - [out] Message digest buffer.
- **pulDigestLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to length of message digest buffer.

Description

The digesting operation MUST have been initialized with [C_DigestInit\(\)](#). A call to [C_Digest\(\)](#) always terminates the active digesting operation unless it returns `CKR_BUFFER_TOO_SMALL` or is a successful call (i.e., one which returns `CKR_OK`) to determine the length of the buffer needed to hold the message digest.

[C_Digest\(\)](#) cannot be used to terminate a multi-part operation, and MUST be called after [C_DigestInit\(\)](#) without intervening [C_DigestUpdate\(\)](#) calls.

To query the required message digest buffer length, set `pDigest` to `NULL_PTR`. The function will then set the required message digest buffer length in `pulDigestLen` and return `CKR_OK`.

On operation completion, the `pulDigestLen` is updated to the correct value when:

- `pulDigestLen` is bigger than expected. In this case, operation succeeds.
- `pulDigestLen` is shorter than expected. In this case, operation fails and returns `CKR_BUFFER_TOO_SMALL`.

[C_Digest\(\)](#) is equivalent to a sequence of [C_DigestUpdate\(\)](#) operations followed by [C_DigestFinal\(\)](#).

Return

- **CKR_OK:**
Success. Digesting operation complete.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
 - `pulDigestLen` is `NULL_PTR`.
 - Operation bad state.
- **CKR_BUFFER_TOO_SMALL:**
The output buffer is too small to hold the message digest.
- **CKR_GENERAL_ERROR**
General error.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_DigestInit\(\)](#).
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.2.3 Multiple Part

6.3.6.2.3.1 C_DigestUpdate

CK_RV C_DigestUpdate([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pPart, [CK_ULONG](#) ulPartLen)

Continue a multi-part digesting operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pPart** ([CK_BYTE_PTR](#)) - [in] Data part buffer to be digested.
- **ulPartLen** ([CK_ULONG](#)) - [in] Length of data part to be digested.

Description

The digesting operation MUST have been initialized with [C_DigestInit\(\)](#). This function may be called any number of times in succession. A call to [C_DigestUpdate\(\)](#) which results in an error terminates the current digesting operation.

Return

- **CKR_OK:**
Success. Digesting operation updated.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
Operation bad state.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_DigestInit\(\)](#).
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.2.3.2 C_DigestFinal

CK_RV C_DigestFinal([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pDigest, [CK_ULONG_PTR](#) pulDigestLen)

Finish a multi-part digesting operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pDigest** ([CK_BYTE_PTR](#)) - [out] Message digest buffer.
- **pulDigestLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to length of message digest buffer.

Description

The digesting operation MUST have been initialized with [C_DigestInit\(\)](#). A call to [C_DigestFinal\(\)](#) always terminates the active digesting operation unless it returns CKR_BUFFER_TOO_SMALL or is a successful call (i.e., one which returns CKR_OK) to determine the length of the buffer needed to hold the message digest.

To query the required message digest buffer length, set `pDigest` to `NULL_PTR`. The function will then set the required message digest buffer length in `pulDigestLen` and return CKR_OK.

On operation completion, the `pulDigestLen` is updated to the correct value when:

- `pulDigestLen` is bigger than expected. In this case, operation succeeds.
- `pulDigestLen` is shorter than expected. In this case, operation fails and returns CKR_BUFFER_TOO_SMALL.

Return

- **CKR_OK:**
Success. Digesting operation complete.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
 - `pulDigestLen` is `NULL_PTR`.
 - Operation bad state.
- **CKR_BUFFER_TOO_SMALL:**
The output buffer is too small to hold the message digest.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_DigestInit\(\)](#).
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.2.4 Example

6.3.6.2.4.1 Single-part digest

```
CK_RV rv;
CK_SESSION_HANDLE session;
CK_MECHANISM mechanism = { CKM_SHA256, NULL, 0 };
CK_BYTE data[] = "Hello, World!";
CK_BYTE digest[32];
CK_ULONG digest_len = sizeof(digest);

// Initialize the digest operation
rv = C_DigestInit(session, &mechanism);
if (rv != CKR_OK) {
```

```
    // Handle error
}

// Perform single-part digest
rv = C_Digest(session, data, sizeof(data) - 1, digest, &digest_len);
if (rv != CKR_OK) {
    // Handle error
}
```

6.3.6.2.4.2 Multi-part digest operation

```
CK_RV rv;
CK_SESSION_HANDLE session;
CK_MECHANISM mechanism = { CKM_SHA256, NULL, 0 };
CK_BYTE data1[] = "Hello, ";
CK_BYTE data2[] = "World!";
CK_BYTE digest[32];
CK_ULONG digest_len = sizeof(digest);

// Initialize the digest operation
rv = C_DigestInit(session, &mechanism);
if (rv != CKR_OK) {
    // Handle error
}

// Update with first data part
rv = C_DigestUpdate(session, data1, sizeof(data1) - 1);
if (rv != CKR_OK) {
    // Handle error
}

// Update with second data part
rv = C_DigestUpdate(session, data2, sizeof(data2) - 1);
if (rv != CKR_OK) {
    // Handle error
}

// Finalize and get the digest
rv = C_DigestFinal(session, digest, &digest_len);
if (rv != CKR_OK) {
    // Handle error
}
```

6.3.6.3 Message authentication code (MAC)

PKCS#11 provides the same set of functions for message authentication code (MAC) as for the asymmetric signature. Refer to the [Asymmetric Signature](#) section for the detailed description of the operation sequences.

The operation initialization must be setup to use the asymmetric encryption mechanisms as listed in the [Message Authentication Code \(MAC\) Mechanisms](#).

6.3.6.4 Asymmetric signature

PKCS#11 provides two sets of functions for asymmetric signature operations function of the PKCS#11 interface used. If the PKCS#11 3.x interface is used, the new message-based APIs should be preferred. However, for backward compatibility, the PKCS#11 2.x APIs are still available:

Table 199. PKCS#11 Asymmetric Signature APIs

Version	Signing Functions	Verification Functions
PKCS#11 2.40	C_SignInit() , C_Sign() , C_SignUpdate() , C_SignFinal()	C_VerifyInit() , C_Verify() , C_VerifyUpdate() , C_VerifyFinal()
PKCS#11 3.2	C_MessageSignInit() , C_SignMessage() , C_SignMessageBegin() , C_SignMessageNext() , C_MessageSignFinal()	C_MessageVerifyInit() , C_VerifyMessage() , C_VerifyMessageBegin() , C_VerifyMessageNext() , C_MessageVerifyFinal() , C_VerifySignatureInit() , C_VerifySignature() , C_VerifySignatureUpdate() , C_VerifySignatureFinal()

Regardless of the API version used, the operation must be initialized first before performing the signature or verification operation.

6.3.6.4.1 PKCS#11 2.40 APIs sequences

Asymmetric Signing

The sequence of operations to sign a message with a single part signature is as follows:

1. Initialize the operation by calling [C_SignInit\(\)](#).
2. Sign data in a single operation with [C_Sign\(\)](#).

The sequence of operations to sign a message with a multiple part signature is as follows:

1. Initialize the operation by calling [C_SignInit\(\)](#).
2. Process one or more data parts (can be called multiple times) with [C_SignUpdate\(\)](#).
3. Finish the signing operation and obtain the final signature with [C_SignFinal\(\)](#).

Asymmetric Verification

The sequence of operations to verify a signature with a single part verification is as follows:

1. Initialize the operation by calling [C_VerifyInit\(\)](#).
2. Verify signature in a single operation with [C_Verify\(\)](#).

The sequence of operations to verify a signature with a multiple part verification is as follows:

1. Initialize the operation by calling [C_VerifyInit\(\)](#).
2. Process one or more data parts (can be called multiple times) with [C_VerifyUpdate\(\)](#).
3. Finish the verification operation with [C_VerifyFinal\(\)](#).

6.3.6.4.2 PKCS#11 3.2 APIs sequences

Message Signing

The sequence of operations to sign a message with a single part signature is as follows:

1. Initialize the operation by calling [C_MessageSignInit\(\)](#).
2. Sign data in a single operation with [C_SignMessage\(\)](#).

The sequence of operations to sign a message with a multiple part signature is as follows:

1. Initialize the operation by calling [C_MessageSignInit\(\)](#).
2. Begin the message signing with [C_SignMessageBegin\(\)](#).
3. Process one or more data parts (can be called multiple times) with [C_SignMessageNext\(\)](#).
4. Finish the signing operation with [C_MessageSignFinal\(\)](#).

Message Verification

The sequence of operations to verify a signature with a single part verification is as follows:

1. Initialize the operation by calling [C_MessageVerifyInit\(\)](#).
2. Verify signature in a single operation with [C_VerifyMessage\(\)](#).

The sequence of operations to verify a signature with a multiple part verification is as follows:

1. Initialize the operation by calling [C_MessageVerifyInit\(\)](#).
2. Begin the message verification with [C_VerifyMessageBegin\(\)](#).
3. Process one or more data parts (can be called multiple times) with [C_VerifyMessageNext\(\)](#).
4. Finish the verification operation with [C_MessageVerifyFinal\(\)](#).

Signature Verification

The sequence of operations to verify a signature with signature provided at initialization is as follows:

1. Initialize the operation by calling [C_VerifySignatureInit\(\)](#), providing the signature to verify.
2. Verify data in a single operation with [C_VerifySignature\(\)](#).

The sequence of operations for multiple part verification with signature at initialization is as follows:

1. Initialize the operation by calling [C_VerifySignatureInit\(\)](#), providing the signature to verify.
2. Process one or more data parts (can be called multiple times) with [C_VerifySignatureUpdate\(\)](#).
3. Finish the verification operation with [C_VerifySignatureFinal\(\)](#).

6.3.6.4.3 Initialization

6.3.6.4.3.1 C_SignInit

[CK_RV](#) **C_SignInit**([CK_SESSION_HANDLE](#) hSession, [CK_MECHANISM_PTR](#) pMechanism, [CK_OBJECT_HANDLE](#) hKey)

Initialize signature operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pMechanism** ([CK_MECHANISM_PTR](#)) - [in] Signature mechanism.
- **hKey** ([CK_OBJECT_HANDLE](#)) - [in] Key object handle.

Description

The CKA_SIGN attribute of the signature key, which indicates whether the key supports signing, MUST be CK_TRUE.

After calling [C_SignInit\(\)](#), the application can either call [C_Sign\(\)](#) to sign data in a single part; or call [C_SignUpdate\(\)](#) zero or more times, followed by [C_SignFinal\(\)](#), to sign data in multiple parts. The signature operation is active until the application uses a call to [C_Sign\(\)](#) or [C_SignFinal\(\)](#) to actually obtain the final signature. To process additional data (in single or multiple parts), the application MUST call [C_SignInit\(\)](#) again.

[C_SignInit\(\)](#) can be called with pMechanism set to NULL_PTR to terminate an active signature operation. If an active operation cannot be cancelled, CKR_OPERATION_CANCEL_FAILED must be returned.

Return

• CKR_OK:

Success. Signature operation initialized or cancelled if the pMechanism is NULL_PTR.

- **CKR_SESSION_HANDLE_INVALID:**

Session handle `hSession` is invalid.

- **CKR_KEY_HANDLE_INVALID:**

The `hKey` is invalid.

- **CKR_MECHANISM_INVALID**

- **CKR_MECHANISM_PARAM_INVALID**

- **CKR_OPERATION_ACTIVE**

- **CKR_SLOT_ID_INVALID**

- **CKR_KEY_FUNCTION_NOT_PERMITTED**

Key is not allowed to do the operation.

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.4.3.2 C_VerifyInit

[CK_RV](#) **C_VerifyInit**([CK_SESSION_HANDLE](#) `hSession`, [CK_MECHANISM_PTR](#) `pMechanism`, [CK_OBJECT_HANDLE](#) `hKey`)

Initialize verification operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pMechanism** ([CK_MECHANISM_PTR](#)) - [in] Verification mechanism.
- **hKey** ([CK_OBJECT_HANDLE](#)) - [in] Key object handle.

Description

The CKA_VERIFY attribute of the verification key, which indicates whether the key supports verification, MUST be CK_TRUE.

After calling [C_VerifyInit\(\)](#), the application can either call [C_Verify\(\)](#) to verify a signature in a single part; or call [C_VerifyUpdate\(\)](#) zero or more times, followed by [C_VerifyFinal\(\)](#), to verify a signature in multiple parts. The verification operation is active until the application uses a call to [C_Verify\(\)](#) or [C_VerifyFinal\(\)](#) to actually perform the final verification. To process additional data (in single or multiple parts), the application MUST call [C_VerifyInit\(\)](#) again.

[C_VerifyInit\(\)](#) can be called with `pMechanism` set to NULL_PTR to terminate an active verification operation. If an active operation cannot be cancelled, CKR_OPERATION_CANCEL_FAILED must be returned.

Return

- **CKR_OK:**

Success. Verification operation initialized or cancelled if the `pMechanism` is NULL_PTR.

- **CKR_SESSION_HANDLE_INVALID:**

Session handle `hSession` is invalid.

- **CKR_KEY_HANDLE_INVALID:**

The `hKey` is invalid.

- CKR_MECHANISM_INVALID
- CKR_MECHANISM_PARAM_INVALID
- CKR_OPERATION_ACTIVE
- CKR_SLOT_ID_INVALID
- **CKR_KEY_FUNCTION_NOT_PERMITTED**

Key is not allowed to do the operation.

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.4.3.3 C_MessageSignInit

[CK_RV](#) **C_MessageSignInit**([CK_SESSION_HANDLE](#) `hSession`, [CK_MECHANISM_PTR](#) `pMechanism`, [CK_OBJECT_HANDLE](#) `hKey`)

Initialize message signature operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pMechanism** ([CK_MECHANISM_PTR](#)) - [in] Signature mechanism.
- **hKey** ([CK_OBJECT_HANDLE](#)) - [in] Key object handle.

Description

The CKA_SIGN attribute of the signature key, which indicates whether the key supports signing, MUST be CK_TRUE.

After calling [C_MessageSignInit\(\)](#), the application can either call [C_SignMessage\(\)](#) to sign a message in a single part; or call [C_SignMessageBegin\(\)](#) followed by [C_SignMessageNext\(\)](#) one or more times, and finally [C_MessageSignFinal\(\)](#) to sign a message in multiple parts. The message signature operation is active until the application uses a call to [C_SignMessage\(\)](#) or [C_MessageSignFinal\(\)](#) to terminate the operation. To process additional messages, the application MUST call [C_MessageSignInit\(\)](#) again.

[C_MessageSignInit\(\)](#) can be called with `pMechanism` set to NULL_PTR to terminate an active message signature operation. If an active operation cannot be cancelled, CKR_OPERATION_CANCEL_FAILED must be returned.

Return

- **CKR_OK:**

Success. Message signature operation initialized or cancelled if the `pMechanism` is NULL_PTR.

- **CKR_SESSION_HANDLE_INVALID:**

Session handle `hSession` is invalid.

- **CKR_KEY_HANDLE_INVALID:**

The `hKey` is invalid.

- `CKR_MECHANISM_INVALID`
- `CKR_MECHANISM_PARAM_INVALID`
- `CKR_OPERATION_ACTIVE`
- `CKR_SLOT_ID_INVALID`
- **`CKR_KEY_FUNCTION_NOT_PERMITTED`**

Key is not allowed to do the operation.

- **`CKR_GENERAL_ERROR`:**

General error.

- **`CKR_CRYPTOKI_NOT_INITIALIZED`:**

The library has not been previously initialized by `C_Initialize()`.

- Other errors from the underlying implementation.

6.3.6.4.3.4 C_MessageVerifyInit

`CK_RV C_MessageVerifyInit(CK_SESSION_HANDLE hSession, CK_MECHANISM_PTR pMechanism, CK_OBJECT_HANDLE hKey)`

Initialize message verification operation.

Parameters

- **`hSession`** (`CK_SESSION_HANDLE`) - [in] Session handle.
- **`pMechanism`** (`CK_MECHANISM_PTR`) - [in] Verification mechanism.
- **`hKey`** (`CK_OBJECT_HANDLE`) - [in] Key object handle.

Description

The `CKA_VERIFY` attribute of the verification key, which indicates whether the key supports verification, MUST be `CK_TRUE`.

After calling `C_MessageVerifyInit()`, the application can either call `C_VerifyMessage()` to verify a message in a single part; or call `C_VerifyMessageBegin()` followed by `C_VerifyMessageNext()` one or more times, and finally `C_MessageVerifyFinal()` to verify a message in multiple parts. The message verification operation is active until the application uses a call to `C_VerifyMessage()` or `C_MessageVerifyFinal()` to terminate the operation. To process additional messages, the application MUST call `C_MessageVerifyInit()` again.

`C_MessageVerifyInit()` can be called with `pMechanism` set to `NULL_PTR` to terminate an active message verification operation. If an active operation cannot be cancelled, `CKR_OPERATION_CANCEL_FAILED` must be returned.

Return

- **`CKR_OK`:**
Success. Message verification operation initialized or cancelled if the `pMechanism` is `NULL_PTR`.
- **`CKR_SESSION_HANDLE_INVALID`:**
Session handle `hSession` is invalid.
- **`CKR_KEY_HANDLE_INVALID`:**

The `hKey` is invalid.

- `CKR_MECHANISM_INVALID`
- `CKR_MECHANISM_PARAM_INVALID`
- `CKR_OPERATION_ACTIVE`
- `CKR_SLOT_ID_INVALID`
- **`CKR_KEY_FUNCTION_NOT_PERMITTED`**

Key is not allowed to do the operation.

- **`CKR_GENERAL_ERROR`:**

General error.

- **`CKR_CRYPTOKI_NOT_INITIALIZED`:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.4.3.5 C_VerifySignatureInit

[CK_RV](#) **C_VerifySignatureInit**([CK_SESSION_HANDLE](#) *hSession*, [CK_MECHANISM_PTR](#) *pMechanism*, [CK_OBJECT_HANDLE](#) *hKey*, [CK_BYTE_PTR](#) *pSignature*, [CK_ULONG](#) *ulSignatureLen*)

Initialize a signature verification operation.

Parameters

- **`hSession`** ([CK_SESSION_HANDLE](#)) - [in] Session's handle.
- **`pMechanism`** ([CK_MECHANISM_PTR](#)) - [in] Verification mechanism.
- **`hKey`** ([CK_OBJECT_HANDLE](#)) - [in] Verification key handle.
- **`pSignature`** ([CK_BYTE_PTR](#)) - [in] Signature to verify.
- **`ulSignatureLen`** ([CK_ULONG](#)) - [in] Length of signature in bytes.

Description

This function initializes a signature verification operation where the signature is provided at initialization time rather than at finalization.

Note:

This function is not supported.

Return

- **`CKR_FUNCTION_NOT_SUPPORTED`:**

Function not supported.

6.3.6.4.4 Single Part

6.3.6.4.4.1 C_Sign

[CK_RV](#) **C_Sign**([CK_SESSION_HANDLE](#) *hSession*, [CK_BYTE_PTR](#) *pData*, [CK_ULONG](#) *ulDataLen*, [CK_BYTE_PTR](#) *pSignature*, [CK_ULONG_PTR](#) *pulSignatureLen*)

Sign data in a single part.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pData** ([CK_BYTE_PTR](#)) - [in] Data to sign buffer.
- **ulDataLen** ([CK_ULONG](#)) - [in] Length of data to sign.
- **pSignature** ([CK_BYTE_PTR](#)) - [out] Signature buffer.
- **pulSignatureLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to length of signature buffer.

Description

The signature operation MUST have been initialized with [C_SignInit\(\)](#). A call to [C_Sign\(\)](#) always terminates the active signature operation unless it returns [CKR_BUFFER_TOO_SMALL](#) or is a successful call (i.e., one which returns [CKR_OK](#)) to determine the length of the buffer needed to hold the signature.

[C_Sign\(\)](#) cannot be used to terminate a multi-part operation, and MUST be called after [C_SignInit\(\)](#) without intervening [C_SignUpdate\(\)](#) calls.

To query the required signature buffer length, set **pSignature** to [NULL_PTR](#). The function will then set the required signature buffer length in **pulSignatureLen** and return [CKR_OK](#).

On operation completion, the **pulSignatureLen** is updated to the correct value when:

- **pulSignatureLen** is bigger than expected. In this case, operation succeeds.
- **pulSignatureLen** is shorter than expected. In this case, operation fails and returns [CKR_BUFFER_TOO_SMALL](#).

For most mechanisms, [C_Sign\(\)](#) is equivalent to a sequence of [C_SignUpdate\(\)](#) operations followed by [C_SignFinal\(\)](#).

Return

- **CKR_OK:**
Success. Signature operation complete.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle **hSession** is invalid.
- **CKR_ARGUMENTS_BAD:**
 - **pulSignatureLen** is [NULL_PTR](#).
 - Operation bad state.
- **CKR_BUFFER_TOO_SMALL:**
The output buffer is too small to hold the signature.
- **CKR_DATA_INVALID:**
Data length is invalid.
- **CKR_DATA_LEN_RANGE:**
Data length is invalid.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_SignInit\(\)](#).

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.4.4.2 C_Verify

[CK_RV](#) **C_Verify**([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pData, [CK_ULONG](#) ulDataLen, [CK_BYTE_PTR](#) pSignature, [CK_ULONG](#) ulSignatureLen)

Verify signature in a single part.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pData** ([CK_BYTE_PTR](#)) - [in] Data buffer.
- **ulDataLen** ([CK_ULONG](#)) - [in] Length of data.
- **pSignature** ([CK_BYTE_PTR](#)) - [in] Signature buffer.
- **ulSignatureLen** ([CK_ULONG](#)) - [in] Length of signature.

Description

The verification operation MUST have been initialized with [C_VerifyInit\(\)](#). A call to [C_Verify\(\)](#) always terminates the active verification operation.

[C_Verify\(\)](#) cannot be used to terminate a multi-part operation, and MUST be called after [C_VerifyInit\(\)](#) without intervening [C_VerifyUpdate\(\)](#) calls.

For most mechanisms, [C_Verify\(\)](#) is equivalent to a sequence of [C_VerifyUpdate\(\)](#) operations followed by [C_VerifyFinal\(\)](#).

Return

- **CKR_OK:**

Success. Signature is valid.

- **CKR_SESSION_HANDLE_INVALID:**

Session handle hSession is invalid.

- **CKR_ARGUMENTS_BAD:**

Operation bad state.

- **CKR_DATA_INVALID:**

Data length is invalid.

- **CKR_DATA_LEN_RANGE:**

Data length is invalid.

- **CKR_SIGNATURE_INVALID:**

The signature is invalid.

- **CKR_SIGNATURE_LEN_RANGE:**

The signature length is invalid.

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_OPERATION_NOT_INITIALIZED:**

The operation must be initialized with [C_VerifyInit\(\)](#).

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.4.3 C_SignMessage

[CK_RV C_SignMessage\(CK_SESSION_HANDLE hSession, CK_VOID_PTR pParameter, CK_ULONG ulParameterLen, CK_BYTE_PTR pData, CK_ULONG ulDataLen, CK_BYTE_PTR pSignature, CK_ULONG_PTR pulSignatureLen\)](#)

Sign a message in a single part.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pParameter** ([CK_VOID_PTR](#)) - [in/out] Message-specific parameter buffer.
- **ulParameterLen** ([CK_ULONG](#)) - [in] Length of message-specific parameter.
- **pData** ([CK_BYTE_PTR](#)) - [in] Message data to sign buffer.
- **ulDataLen** ([CK_ULONG](#)) - [in] Length of message data to sign.
- **pSignature** ([CK_BYTE_PTR](#)) - [out] Signature buffer.
- **pulSignatureLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to length of signature buffer.

Description

The message signature operation MUST have been initialized with [C_MessageSignInit\(\)](#). A call to [C_SignMessage\(\)](#) always terminates the active message signature operation unless it returns CKR_BUFFER_TOO_SMALL or is a successful call (i.e., one which returns CKR_OK) to determine the length of the buffer needed to hold the signature.

[C_SignMessage\(\)](#) cannot be used to terminate a multi-part operation, and MUST be called after [C_MessageSignInit\(\)](#) without intervening [C_SignMessageBegin\(\)](#) or [C_SignMessageNext\(\)](#) calls.

To query the required signature buffer length, set `pSignature` to `NULL_PTR`. The function will then set the required signature buffer length in `pulSignatureLen` and return CKR_OK.

On operation completion, the `pulSignatureLen` is updated to the correct value when:

- `pulSignatureLen` is bigger than expected. In this case, operation succeeds.
- `pulSignatureLen` is shorter than expected. In this case, operation fails and returns CKR_BUFFER_TOO_SMALL.

Return

- **CKR_OK:**

Success. Message signature operation complete.

- **CKR_SESSION_HANDLE_INVALID:**

Session handle `hSession` is invalid.

- **CKR_ARGUMENTS_BAD:**

- `pulSignatureLen` is `NULL_PTR`.
- Operation bad state.

- **CKR_BUFFER_TOO_SMALL:**

The output buffer is too small to hold the signature.

- **CKR_DATA_INVALID:**

Data length is invalid.

- **CKR_DATA_LEN_RANGE:**

Data length is invalid.

- **CKR_MECHANISM_PARAM_INVALID:**

Mechanism parameter is invalid.

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_OPERATION_NOT_INITIALIZED:**

The operation must be initialized with [C_MessageSignInit\(\)](#).

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.4.4.4 C_VerifyMessage

[CK_RV](#) **C_VerifyMessage**([CK_SESSION_HANDLE](#) hSession, [CK_VOID_PTR](#) pParameter, [CK_ULONG](#) ulParameterLen, [CK_BYTE_PTR](#) pData, [CK_ULONG](#) ulDataLen, [CK_BYTE_PTR](#) pSignature, [CK_ULONG](#) ulSignatureLen)

Verify a message in a single part.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pParameter** ([CK_VOID_PTR](#)) - [in] Message-specific parameter buffer.
- **ulParameterLen** ([CK_ULONG](#)) - [in] Length of message-specific parameter.
- **pData** ([CK_BYTE_PTR](#)) - [in] Message data to verify buffer.
- **ulDataLen** ([CK_ULONG](#)) - [in] Length of message data to verify.
- **pSignature** ([CK_BYTE_PTR](#)) - [in] Signature buffer.
- **ulSignatureLen** ([CK_ULONG](#)) - [in] Length of signature.

Description

The message verification operation MUST have been initialized with [C_MessageVerifyInit\(\)](#). A call to [C_VerifyMessage\(\)](#) always terminates the active message verification operation unless it returns an error that indicates the operation should continue.

[C_VerifyMessage\(\)](#) cannot be used to terminate a multi-part operation, and MUST be called after [C_MessageVerifyInit\(\)](#) without intervening [C_VerifyMessageBegin\(\)](#) or [C_VerifyMessageNext\(\)](#) calls.

Return

- **CKR_OK:**

Success. Message verification operation complete and signature is valid.

- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
 - Operation bad state.
- **CKR_DATA_INVALID:**
Data length is invalid.
- **CKR_DATA_LEN_RANGE:**
Data length is invalid.
- **CKR_MECHANISM_PARAM_INVALID:**
Mechanism parameter is invalid.
- **CKR_SIGNATURE_INVALID:**
The signature is invalid.
- **CKR_SIGNATURE_LEN_RANGE:**
The signature length is invalid.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_MessageVerifyInit\(\)](#).
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.4.4.5 C_VerifySignature

[CK_RV](#) **C_VerifySignature**([CK_SESSION_HANDLE](#) *hSession*, [CK_BYTE_PTR](#) *pData*, [CK_ULONG](#) *ulDataLen*)

Verify a signature in a single-part operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session's handle.
- **pData** ([CK_BYTE_PTR](#)) - [in] Data to verify.
- **ulDataLen** ([CK_ULONG](#)) - [in] Length of data in bytes.

Description

This function verifies a signature on data in a single operation. The signature must have been provided during initialization with [C_VerifySignatureInit\(\)](#).

Note:

This function is not supported.

Return

- **CKR_FUNCTION_NOT_SUPPORTED:**

Function not supported.

6.3.6.4.5 Multiple Part

6.3.6.4.5.1 C_SignUpdate

[CK_RV](#) **C_SignUpdate**([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pPart, [CK_ULONG](#) ulPartLen)

Continue a multi-part signature operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pPart** ([CK_BYTE_PTR](#)) - [in] Data part buffer.
- **ulPartLen** ([CK_ULONG](#)) - [in] Length of data part.

Description

The signature operation MUST have been initialized with [C_SignInit\(\)](#). This function may be called any number of times in succession. A call to [C_SignUpdate\(\)](#) which results in an error terminates the current signature operation.

Return

- **CKR_OK:**
Success. Signature operation updated.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
Operation bad state.
- **CKR_DATA_INVALID:**
Data length is invalid.
- **CKR_DATA_LEN_RANGE:**
Data length is invalid.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_SignInit\(\)](#).
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.4.5.2 C_SignFinal

[CK_RV](#) **C_SignFinal**([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pSignature, [CK_ULONG_PTR](#) pulSignatureLen)

Finish a multi-part signature operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pSignature** ([CK_BYTE_PTR](#)) - [out] Signature buffer.
- **pulSignatureLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to length of signature buffer.

Description

The signature operation MUST have been initialized with [C_SignInit\(\)](#). A call to [C_SignFinal\(\)](#) always terminates the active signature operation unless it returns `CKR_BUFFER_TOO_SMALL` or is a successful call (i.e., one which returns `CKR_OK`) to determine the length of the buffer needed to hold the signature.

To query the required signature buffer length, set `pSignature` to `NULL_PTR`. The function will then set the required signature buffer length in `pulSignatureLen` and return `CKR_OK`.

On operation completion, the `pulSignatureLen` is updated to the correct value when:

- `pulSignatureLen` is bigger than expected. In this case, operation succeeds.
- `pulSignatureLen` is shorter than expected. In this case, operation fails and returns `CKR_BUFFER_TOO_SMALL`.

Return

- **CKR_OK:**
Success. Signature operation complete.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
 - `pulSignatureLen` is `NULL_PTR`.
 - Operation bad state.
- **CKR_BUFFER_TOO_SMALL:**
The output buffer is too small to hold the signature.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_SignInit\(\)](#).
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.4.5.3 C_VerifyUpdate

[CK_RV](#) [C_VerifyUpdate](#)([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pPart, [CK_ULONG](#) ulPartLen)

Continue a multi-part verification operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pPart** ([CK_BYTE_PTR](#)) - [in] Data part buffer.
- **ulPartLen** ([CK_ULONG](#)) - [in] Length of data part.

Description

The verification operation MUST have been initialized with [C_VerifyInit\(\)](#). This function may be called any number of times in succession. A call to [C_VerifyUpdate\(\)](#) which results in an error terminates the current verification operation.

Return

- **CKR_OK:**
Success. Verification operation updated.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
Operation bad state.
- **CKR_DATA_INVALID:**
Data length is invalid.
- **CKR_DATA_LEN_RANGE:**
Data length is invalid.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_VerifyInit\(\)](#).
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.4.5.4 C_VerifyFinal

[CK_RV](#) **C_VerifyFinal**([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pSignature, [CK_ULONG](#) ulSignatureLen)

Finish a multi-part verification operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pSignature** ([CK_BYTE_PTR](#)) - [in] Signature buffer.
- **ulSignatureLen** ([CK_ULONG](#)) - [in] Length of signature.

Description

The verification operation MUST have been initialized with [C_VerifyInit\(\)](#). A call to [C_VerifyFinal\(\)](#) always terminates the active verification operation.

Return

- **CKR_OK:**
Success. Signature is valid.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
Operation bad state.
- **CKR_SIGNATURE_INVALID:**
The signature is invalid.
- **CKR_SIGNATURE_LEN_RANGE:**
The signature length is invalid.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_VerifyInit\(\)](#).
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.4.5.5 C_SignMessageBegin

[CK_RV](#) **C_SignMessageBegin**([CK_SESSION_HANDLE](#) hSession, [CK_VOID_PTR](#) pParameter, [CK_ULONG](#) ulParameterLen)

Begin a multi-part message signature operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pParameter** ([CK_VOID_PTR](#)) - [in/out] Message-specific parameter buffer.
- **ulParameterLen** ([CK_ULONG](#)) - [in] Length of message-specific parameter.

Description

The message signature operation MUST have been initialized with [C_MessageSignInit\(\)](#). This function begins the processing of a new message in a multi-part message signature operation. A call to [C_SignMessageBegin\(\)](#) which results in an error terminates the current message signature operation.

After calling [C_SignMessageBegin\(\)](#), the application should call [C_SignMessageNext\(\)](#) one or more times to process message data parts.

Return

- **CKR_OK:**
Success. Message signature operation reset for new message.
- **CKR_SESSION_HANDLE_INVALID:**

Session handle `hSession` is invalid.

- **CKR_ARGUMENTS_BAD:**

Operation bad state.

- **CKR_MECHANISM_PARAM_INVALID:**

Mechanism parameter is invalid.

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_OPERATION_NOT_INITIALIZED:**

The operation must be initialized with [C_MessageSignInit\(\)](#).

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.4.5.6 C_SignMessageNext

[CK_RV](#) **C_SignMessageNext**([CK_SESSION_HANDLE](#) `hSession`, [CK_VOID_PTR](#) `pParameter`, [CK_ULONG](#) `ulParameterLen`, [CK_BYTE_PTR](#) `pData`, [CK_ULONG](#) `ulDataLen`, [CK_BYTE_PTR](#) `pSignature`, [CK_ULONG_PTR](#) `pulSignatureLen`)

Continue a multi-part message signature operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pParameter** ([CK_VOID_PTR](#)) - [in] Message-specific parameter buffer.
- **ulParameterLen** ([CK_ULONG](#)) - [in] Length of message-specific parameter.
- **pData** ([CK_BYTE_PTR](#)) - [in] Message data part buffer.
- **ulDataLen** ([CK_ULONG](#)) - [in] Length of message data part.
- **pSignature** ([CK_BYTE_PTR](#)) - [out] Signature buffer (only for final call).
- **pulSignatureLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to length of signature buffer (only for final call).

Description

The message signature operation MUST have been initialized with [C_MessageSignInit\(\)](#) and a message started with [C_SignMessageBegin\(\)](#). This function may be called any number of times in succession to process message data parts.

When `pulSignatureLen` is `NULL_PTR`, this function processes a message data part without generating the signature (intermediate call).

When `pulSignatureLen` is not `NULL_PTR`, this function processes the final message data part and generates the signature (final call). In this case, the operation for the current message is terminated, but the message signature operation remains active for processing additional messages.

To query the required signature buffer length on the final call, set `pSignature` to `NULL_PTR`. The function will then set the required signature buffer length in `pulSignatureLen` and return `CKR_OK`.

On final call completion, the `pulSignatureLen` is updated to the correct value when:

- `pulSignatureLen` is bigger than expected. In this case, operation succeeds.
- `pulSignatureLen` is shorter than expected. In this case, operation fails and returns `CKR_BUFFER_TOO_SMALL`.

A call to [C_SignMessageNext\(\)](#) which results in an error terminates the current message signature operation.

Return

- **CKR_OK:**
Success. Message data part processed or signature generated.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
Operation bad state.
- **CKR_BUFFER_TOO_SMALL:**
The output buffer is too small to hold the signature (final call only).
- **CKR_DATA_INVALID:**
Data length is invalid.
- **CKR_DATA_LEN_RANGE:**
Data length is invalid.
- **CKR_MECHANISM_PARAM_INVALID:**
Mechanism parameter is invalid.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_MessageSignInit\(\)](#).
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.4.5.7 C_MessageSignFinal

[CK_RV](#) **C_MessageSignFinal**([CK_SESSION_HANDLE](#) `hSession`)

Finish a multi-part message signature operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.

Description

The message signature operation MUST have been initialized with [C_MessageSignInit\(\)](#). A call to [C_MessageSignFinal\(\)](#) always terminates the active message signature operation.

This function is used to terminate the message signature operation without processing any additional messages. It cancels the operation context and releases associated resources.

Return

- **CKR_OK:**
Success. Message signature operation terminated.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_MessageSignInit\(\)](#).
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.4.5.8 C_VerifyMessageBegin

[CK_RV](#) **C_VerifyMessageBegin**([CK_SESSION_HANDLE](#) hSession, [CK_VOID_PTR](#) pParameter, [CK_ULONG](#) ulParameterLen)

Begin a multi-part message verification operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pParameter** ([CK_VOID_PTR](#)) - [in] Message-specific parameter buffer.
- **ulParameterLen** ([CK_ULONG](#)) - [in] Length of message-specific parameter.

Description

The message verification operation MUST have been initialized with [C_MessageVerifyInit\(\)](#). This function begins the processing of a new message in a multi-part message verification operation. A call to [C_VerifyMessageBegin\(\)](#) which results in an error terminates the current message verification operation.

After calling [C_VerifyMessageBegin\(\)](#), the application should call [C_VerifyMessageNext\(\)](#) one or more times to process message data parts.

Return

- **CKR_OK:**
Success. Message verification operation reset for new message.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
Operation bad state.
- **CKR_MECHANISM_PARAM_INVALID:**
Mechanism parameter is invalid.
- **CKR_GENERAL_ERROR:**

General error.

- **CKR_OPERATION_NOT_INITIALIZED:**

The operation must be initialized with [C_MessageVerifyInit\(\)](#).

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.4.5.9 C_VerifyMessageNext

CK_RV C_VerifyMessageNext([CK_SESSION_HANDLE](#) hSession, [CK_VOID_PTR](#) pParameter, [CK_ULONG](#) ulParameterLen, [CK_BYTE_PTR](#) pData, [CK_ULONG](#) ulDataLen, [CK_BYTE_PTR](#) pSignature, [CK_ULONG](#) ulSignatureLen)

Continue a multi-part message verification operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pParameter** ([CK_VOID_PTR](#)) - [in] Message-specific parameter buffer.
- **ulParameterLen** ([CK_ULONG](#)) - [in] Length of message-specific parameter.
- **pData** ([CK_BYTE_PTR](#)) - [in] Message data part buffer.
- **ulDataLen** ([CK_ULONG](#)) - [in] Length of message data part.
- **pSignature** ([CK_BYTE_PTR](#)) - [in] Signature buffer (only for final call).
- **ulSignatureLen** ([CK_ULONG](#)) - [in] Length of signature (only for final call).

Description

The message verification operation MUST have been initialized with [C_MessageVerifyInit\(\)](#) and a message started with [C_VerifyMessageBegin\(\)](#). This function may be called any number of times in succession to process message data parts.

When **pSignature** is **NULL_PTR**, this function processes a message data part without verifying the signature (intermediate call).

When **pSignature** is not **NULL_PTR**, this function processes the final message data part and verifies the signature (final call). In this case, the operation for the current message is terminated, but the message verification operation remains active for processing additional messages.

A call to [C_VerifyMessageNext\(\)](#) which results in an error terminates the current message verification operation.

Return

- **CKR_OK:**

Success. Message data part processed or signature verified.

- **CKR_SESSION_HANDLE_INVALID:**

Session handle **hSession** is invalid.

- **CKR_ARGUMENTS_BAD:**

Operation bad state.

- **CKR_DATA_INVALID:**

Data length is invalid.

- **CKR_DATA_LEN_RANGE:**
Data length is invalid.
- **CKR_MECHANISM_PARAM_INVALID:**
Mechanism parameter is invalid.
- **CKR_SIGNATURE_INVALID:**
The signature is invalid (final call only).
- **CKR_SIGNATURE_LEN_RANGE:**
The signature length is invalid (final call only).
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_MessageVerifyInit\(\)](#).
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.4.5.10 C_MessageVerifyFinal

[CK_RV](#) **C_MessageVerifyFinal**([CK_SESSION_HANDLE](#) hSession)

Finish a multi-part message verification operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.

Description

The message verification operation MUST have been initialized with [C_MessageVerifyInit\(\)](#). A call to [C_MessageVerifyFinal\(\)](#) always terminates the active message verification operation.

This function is used to terminate the message verification operation without processing any additional messages. It cancels the operation context and releases associated resources.

Return

- **CKR_OK:**
Success. Message verification operation terminated.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_MessageVerifyInit\(\)](#).
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.4.5.11 C_VerifySignatureUpdate

[CK_RV](#) C_VerifySignatureUpdate([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pPart, [CK_ULONG](#) ulPartLen)

Continue a multi-part signature verification.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session's handle.
- **pPart** ([CK_BYTE_PTR](#)) - [in] Data part to verify.
- **ulPartLen** ([CK_ULONG](#)) - [in] Length of data part in bytes.

Description

This function continues a multi-part signature verification operation, processing another data part. The signature must have been provided during initialization with [C_VerifySignatureInit\(\)](#).

Note:

This function is not supported.

Return

- **CKR_FUNCTION_NOT_SUPPORTED**

Function not supported.

6.3.6.4.5.12 C_VerifySignatureFinal

[CK_RV](#) C_VerifySignatureFinal([CK_SESSION_HANDLE](#) hSession)

Finish a multi-part signature verification.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session's handle.

Description

This function finishes a multi-part signature verification operation. The signature must have been provided during initialization with [C_VerifySignatureInit\(\)](#).

Note:

This function is not supported.

Return

- **CKR_FUNCTION_NOT_SUPPORTED**

Function not supported.

6.3.6.4.6 Examples

6.3.6.4.6.1 PKCS#11 2.40 - Single Part Signing

```
CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_MECHANISM mechanism = { CKM_RSA_PKCS, NULL_PTR, 0 };
CK_BYTE data[] = "Data to sign";
CK_BYTE signature[256];
CK_ULONG signature_len = sizeof(signature);
CK_RV rv;

// Initialize signing operation
rv = C_SignInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    // Handle error
}

// Perform single-part signing
rv = C_Sign(hSession, data, sizeof(data), signature, &signature_len);
if (rv != CKR_OK) {
    // Handle error
}
```

6.3.6.4.6.2 PKCS#11 2.40 - Multiple Part Signing

```
CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_MECHANISM mechanism = { CKM_ECDSA, NULL_PTR, 0 };
CK_BYTE data_part1[] = "First part ";
CK_BYTE data_part2[] = "Second part";
CK_BYTE signature[256];
CK_ULONG signature_len = sizeof(signature);
CK_RV rv;

// Initialize signing operation
rv = C_SignInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    // Handle error
}

// Process first part
rv = C_SignUpdate(hSession, data_part1, sizeof(data_part1));
if (rv != CKR_OK) {
    // Handle error
}

// Process second part
rv = C_SignUpdate(hSession, data_part2, sizeof(data_part2));
if (rv != CKR_OK) {
    // Handle error
}

// Finalize signing
rv = C_SignFinal(hSession, signature, &signature_len);
if (rv != CKR_OK) {
    // Handle error
}
```

```
}

```

6.3.6.4.6.3 PKCS#11 2.40 - Single Part Verification

```
CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_MECHANISM mechanism = { CKM_RSA_PKCS, NULL_PTR, 0 };
CK_BYTE data[] = "Data to verify";
CK_BYTE signature[256];
CK_ULONG signature_len = 256;
CK_RV rv;

// Initialize verification operation
rv = C_VerifyInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    // Handle error
}

// Perform single-part verification
rv = C_Verify(hSession, data, sizeof(data), signature, signature_len);
if (rv != CKR_OK) {
    // Handle error - signature invalid or other error
}
```

6.3.6.4.6.4 PKCS#11 2.40 - Multiple Part Verification

```
CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_MECHANISM mechanism = { CKM_ECDSA, NULL_PTR, 0 };
CK_BYTE data_part1[] = "First part ";
CK_BYTE data_part2[] = "Second part";
CK_BYTE signature[256];
CK_ULONG signature_len = 256;
CK_RV rv;

// Initialize verification operation
rv = C_VerifyInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    // Handle error
}

// Process first part
rv = C_VerifyUpdate(hSession, data_part1, sizeof(data_part1));
if (rv != CKR_OK) {
    // Handle error
}

// Process second part
rv = C_VerifyUpdate(hSession, data_part2, sizeof(data_part2));
if (rv != CKR_OK) {
    // Handle error
}

// Finalize verification
rv = C_VerifyFinal(hSession, signature, signature_len);
if (rv != CKR_OK) {
    // Handle error - signature invalid or other error
}
```

```
}

```

6.3.6.4.6.5 PKCS#11 3.2 - Single Part Message Signing

```
CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_MECHANISM mechanism = { CKM_RSA_PKCS, NULL_PTR, 0 };
CK_BYTE data[] = "Message to sign";
CK_BYTE signature[256];
CK_ULONG signature_len = sizeof(signature);
CK_RV rv;

// Initialize message signing operation
rv = C_MessageSignInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    // Handle error
}

// Perform single-part message signing
rv = C_SignMessage(hSession, NULL_PTR, 0, data, sizeof(data),
                  signature, &signature_len);
if (rv != CKR_OK) {
    // Handle error
}
```

6.3.6.4.6.6 PKCS#11 3.2 - Multiple Part Message Signing

```
CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_MECHANISM mechanism = { CKM_ECDSA, NULL_PTR, 0 };
CK_BYTE data_part1[] = "First part ";
CK_BYTE data_part2[] = "Second part";
CK_BYTE signature[256];
CK_ULONG signature_len = sizeof(signature);
CK_RV rv;

// Initialize message signing operation
rv = C_MessageSignInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    // Handle error
}

// Begin message signing
rv = C_SignMessageBegin(hSession, NULL_PTR, 0);
if (rv != CKR_OK) {
    // Handle error
}

// Process first part
rv = C_SignMessageNext(hSession, NULL_PTR, 0, data_part1,
                      sizeof(data_part1), NULL_PTR, NULL_PTR);
if (rv != CKR_OK) {
    // Handle error
}

// Process second part and get signature
rv = C_SignMessageNext(hSession, NULL_PTR, 0, data_part2,
```

```
                                sizeof(data_part2), signature, &signature_len);  
if (rv != CKR_OK) {  
    // Handle error  
}  
  
// Finalize message signing  
rv = C_MessageSignFinal(hSession);  
if (rv != CKR_OK) {  
    // Handle error  
}
```

6.3.6.4.6.7 PKCS#11 3.2 - Single Part Message Verification

```
CK_SESSION_HANDLE hSession;  
CK_OBJECT_HANDLE hKey;  
CK_MECHANISM mechanism = { CKM_RSA_PKCS, NULL_PTR, 0 };  
CK_BYTE data[] = "Message to verify";  
CK_BYTE signature[256];  
CK_ULONG signature_len = 256;  
CK_RV rv;  
  
// Initialize message verification operation  
rv = C_MessageVerifyInit(hSession, &mechanism, hKey);  
if (rv != CKR_OK) {  
    // Handle error  
}  
  
// Perform single-part message verification  
rv = C_VerifyMessage(hSession, NULL_PTR, 0, data, sizeof(data),  
                    signature, signature_len);  
if (rv != CKR_OK) {  
    // Handle error - signature invalid or other error  
}
```

6.3.6.4.6.8 PKCS#11 3.2 - Multiple Part Message Verification

```
CK_SESSION_HANDLE hSession;  
CK_OBJECT_HANDLE hKey;  
CK_MECHANISM mechanism = { CKM_ECDSA, NULL_PTR, 0 };  
CK_BYTE data_part1[] = "First part ";  
CK_BYTE data_part2[] = "Second part";  
CK_BYTE signature[256];  
CK_ULONG signature_len = 256;  
CK_RV rv;  
  
// Initialize message verification operation  
rv = C_MessageVerifyInit(hSession, &mechanism, hKey);  
if (rv != CKR_OK) {  
    // Handle error  
}  
  
// Begin message verification  
rv = C_VerifyMessageBegin(hSession, NULL_PTR, 0);  
if (rv != CKR_OK) {  
    // Handle error  
}
```

```

// Process first part
rv = C_VerifyMessageNext(hSession, NULL_PTR, 0, data_part1,
                        sizeof(data_part1), NULL_PTR, 0);
if (rv != CKR_OK) {
    // Handle error
}

// Process second part with signature
rv = C_VerifyMessageNext(hSession, NULL_PTR, 0, data_part2,
                        sizeof(data_part2), signature, signature_len);
if (rv != CKR_OK) {
    // Handle error - signature invalid or other error
}

// Finalize message verification
rv = C_MessageVerifyFinal(hSession);
if (rv != CKR_OK) {
    // Handle error
}

```

6.3.6.5 Asymmetric Encryption/Decryption

PKCS#11 provides the same set of functions for asymmetric encryption and decryption as for symmetric operations. Refer to the [Symmetric Encryption](#) section for the detailed description of the operation sequences.

However, asymmetric encryption and decryption operations differ in the key types used. Asymmetric encryption requires a public key for encryption and a private key for decryption.

The operation initialization must be setup to use the asymmetric encryption mechanisms as listed in the [Symmetric Encryption Mechanisms](#).

6.3.6.6 Symmetric Encryption

PKCS#11 provides two sets of functions for symmetric encryption and decryption operations function of the PKCS#11 interface used. If the PKCS#11 3.x interface is used, the new message-based APIs should be preferred. However, for backward compatibility, the PKCS#11 2.x APIs are still available:

Table 200. PKCS#11 Symmetric Encryption APIs

Version	Encryption Functions	Decryption Functions
PKCS#11 2.x	C_EncryptInit() , C_Encrypt() , C_EncryptUpdate() , C_EncryptFinal()	C_DecryptInit() , C_Decrypt() , C_DecryptUpdate() , C_DecryptFinal()
PKCS#11 3.x	C_MessageEncryptInit() , C_EncryptMessage() , C_EncryptMessageBegin() , C_EncryptMessageNext() , C_MessageEncryptFinal()	C_MessageDecryptInit() , C_DecryptMessage() , C_DecryptMessageBegin() , C_DecryptMessageNext() , C_MessageDecryptFinal()

Regardless of the API version used, the operation must be initialized first before performing the encryption or decryption operation.

6.3.6.6.1 PKCS#11 2.40 APIs sequences

Symmetric Encryption

The sequence of operations to encrypt a message with a single part encryption is as follows:

1. Initialize the operation by calling [C_EncryptInit\(\)](#).

2. Encrypt data in a single operation with [C_Encrypt\(\)](#).

The sequence of operations to encrypt a message with a multiple part encryption is as follows:

1. Initialize the operation by calling [C_EncryptInit\(\)](#).
2. Process one or more data parts (can be called multiple times) with [C_EncryptUpdate\(\)](#).
3. Finish the encryption operation and obtain the final ciphertext with [C_EncryptFinal\(\)](#).

Symmetric Decryption

The sequence of operations to decrypt a message with a single part decryption is as follows:

1. Initialize the operation by calling [C_DecryptInit\(\)](#).
2. Decrypt data in a single operation with [C_Decrypt\(\)](#).

The sequence of operations to decrypt a message with a multiple part decryption is as follows:

1. Initialize the operation by calling [C_DecryptInit\(\)](#).
2. Process one or more data parts (can be called multiple times) with [C_DecryptUpdate\(\)](#).
3. Finish the decryption operation and obtain the final plaintext with [C_DecryptFinal\(\)](#).

6.3.6.6.2 PKCS#11 3.2 APIs sequences

Message Encryption

The sequence of operations to encrypt a message with a single part encryption is as follows:

1. Initialize the operation by calling [C_MessageEncryptInit\(\)](#).
2. Encrypt data in a single operation with [C_EncryptMessage\(\)](#).

The sequence of operations to encrypt a message with a multiple part encryption is as follows:

1. Initialize the operation by calling [C_MessageEncryptInit\(\)](#).
2. Begin the message encryption with [C_EncryptMessageBegin\(\)](#).
3. Process one or more data parts (can be called multiple times) with [C_EncryptMessageNext\(\)](#).
4. Finish the encryption operation with [C_MessageEncryptFinal\(\)](#).

Message Decryption

The sequence of operations to decrypt a message with a single part decryption is as follows:

1. Initialize the operation by calling [C_MessageDecryptInit\(\)](#).
2. Decrypt data in a single operation with [C_DecryptMessage\(\)](#).

The sequence of operations to decrypt a message with a multiple part decryption is as follows:

1. Initialize the operation by calling [C_MessageDecryptInit\(\)](#).
2. Begin the message decryption with [C_DecryptMessageBegin\(\)](#).
3. Process one or more data parts (can be called multiple times) with [C_DecryptMessageNext\(\)](#).
4. Finish the decryption operation with [C_MessageDecryptFinal\(\)](#).

6.3.6.6.3 Initialization

6.3.6.6.3.1 C_EncryptInit

[CK_RV](#) [C_EncryptInit](#)([CK_SESSION_HANDLE](#) hSession, [CK_MECHANISM_PTR](#) pMechanism, [CK_OBJECT_HANDLE](#) hKey)

Initialize encryption operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pMechanism** ([CK_MECHANISM_PTR](#)) - [in] Encryption mechanism.
- **hKey** ([CK_OBJECT_HANDLE](#)) - [in] Key object handle.

Description

The CKA_ENCRYPT attribute of the encryption key, which indicates whether the key supports encryption, MUST be CK_TRUE.

After calling [C_EncryptInit\(\)](#), the application can either call [C_Encrypt\(\)](#) to encrypt data in a single part; or call [C_EncryptUpdate\(\)](#) zero or more times, followed by [C_EncryptFinal\(\)](#), to encrypt data in multiple parts. The encryption operation is active until the application uses a call to [C_Encrypt\(\)](#) or [C_EncryptFinal\(\)](#) to actually obtain the final piece of ciphertext. To process additional data (in single or multiple parts), the application MUST call [C_EncryptInit\(\)](#) again.

[C_EncryptInit\(\)](#) can be called with `pMechanism` set to `NULL_PTR` to terminate an active encryption operation. If an active operation cannot be cancelled, `CKR_OPERATION_CANCEL_FAILED` must be returned.

Return

- **CKR_OK:**
Success. Encryption operation initialized or cancelled if the `pMechanism` is `NULL_PTR`.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_KEY_HANDLE_INVALID:**
Key object handle `hKey` is invalid.
- **CKR_MECHANISM_INVALID**
- **CKR_MECHANISM_PARAM_INVALID**
- **CKR_OPERATION_ACTIVE**
- **CKR_SLOT_ID_INVALID**
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.6.3.2 C_DecryptInit

CK_RV C_DecryptInit([CK_SESSION_HANDLE](#) hSession, [CK_MECHANISM_PTR](#) pMechanism, [CK_OBJECT_HANDLE](#) hKey)

Initialize decryption operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pMechanism** ([CK_MECHANISM_PTR](#)) - [in] Decryption mechanism.
- **hKey** ([CK_OBJECT_HANDLE](#)) - [in] Key object handle.

Description

The CKA_DECRYPT attribute of the decryption key, which indicates whether the key supports decryption, MUST be CK_TRUE.

After calling [C_DecryptInit\(\)](#), the application can either call [C_Decrypt\(\)](#) to decrypt data in a single part; or call [C_DecryptUpdate\(\)](#) zero or more times, followed by [C_DecryptFinal\(\)](#), to decrypt data in multiple parts. The decryption operation is active until the application uses a call to [C_Decrypt\(\)](#) or [C_DecryptFinal\(\)](#) to actually obtain the final piece of plaintext. To process additional data (in single or multiple parts), the application MUST call [C_DecryptInit\(\)](#) again.

[C_DecryptInit\(\)](#) can be called with `pMechanism` set to NULL_PTR to terminate an active decryption operation. If an active operation cannot be cancelled, CKR_OPERATION_CANCEL_FAILED must be returned.

Return

- **CKR_OK:**
Success. Decryption operation initialized or cancelled if the `pMechanism` is NULL_PTR.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_KEY_HANDLE_INVALID:**
Key object handle `hKey` is invalid.
- **CKR_GENERAL_ERROR**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.6.3.3 C_MessageEncryptInit

CK_RV C_MessageEncryptInit([CK_SESSION_HANDLE](#) `hSession`, [CK_MECHANISM_PTR](#) `pMechanism`, [CK_OBJECT_HANDLE](#) `hKey`)

Initialize a message encryption operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pMechanism** ([CK_MECHANISM_PTR](#)) - [in] Pointer to the encryption mechanism.
- **hKey** ([CK_OBJECT_HANDLE](#)) - [in] Handle of the encryption key.

Description

The CKA_ENCRYPT attribute of the encryption key, which indicates whether the key supports encryption, MUST be CK_TRUE.

After calling [C_MessageEncryptInit\(\)](#), the application can either call [C_EncryptMessage\(\)](#) to encrypt data in a single part; or call [C_EncryptMessageBegin\(\)](#) followed by [C_EncryptMessageNext\(\)](#) zero or more times, followed by [C_MessageEncryptFinal\(\)](#), to encrypt data in multiple parts. The encryption operation is active until the application uses a call to [C_EncryptMessage\(\)](#) or [C_MessageEncryptFinal\(\)](#) to actually obtain the final piece of ciphertext. To process additional data (in single or multiple parts), the application MUST call [C_MessageEncryptInit\(\)](#) again.

[C_MessageEncryptInit\(\)](#) can be called with `pMechanism` set to `NULL_PTR` to terminate an active encryption operation. If an active operation cannot be cancelled, `CKR_OPERATION_CANCEL_FAILED` must be returned.

Return

- **CKR_OK:**
Success. Encryption operation initialized or cancelled if the `pMechanism` is `NULL_PTR`.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle is invalid.
- **CKR_KEY_HANDLE_INVALID:**
The `hKey` is invalid.
- **CKR_MECHANISM_INVALID**
- **CKR_MECHANISM_PARAM_INVALID**
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.6.3.4 C_MessageDecryptInit

CK_RV C_MessageDecryptInit([CK_SESSION_HANDLE](#) `hSession`, [CK_MECHANISM_PTR](#) `pMechanism`, [CK_OBJECT_HANDLE](#) `hKey`)

Initialize a message decryption operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pMechanism** ([CK_MECHANISM_PTR](#)) - [in] Pointer to the decryption mechanism.
- **hKey** ([CK_OBJECT_HANDLE](#)) - [in] Handle of the decryption key.

Description

The `CKA_DECRYPT` attribute of the decryption key, which indicates whether the key supports decryption, MUST be `CK_TRUE`.

After calling [C_MessageDecryptInit\(\)](#), the application can either call [C_DecryptMessage\(\)](#) to decrypt data in a single part; or call [C_DecryptMessageBegin\(\)](#) followed by [C_DecryptMessageNext\(\)](#) zero or more times, followed by [C_MessageDecryptFinal\(\)](#), to decrypt data in multiple parts. The decryption operation is active until the application uses a call to [C_DecryptMessage\(\)](#) or [C_MessageDecryptFinal\(\)](#) to actually obtain the final piece of plaintext. To process additional data (in single or multiple parts), the application MUST call [C_MessageDecryptInit\(\)](#) again.

[C_MessageDecryptInit\(\)](#) can be called with `pMechanism` set to `NULL_PTR` to terminate an active decryption operation. If an active operation cannot be cancelled, `CKR_OPERATION_CANCEL_FAILED` must be returned.

Return

- **CKR_OK:**
Success. Decryption operation initialized or cancelled if the `pMechanism` is `NULL_PTR`.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle is invalid.
- **CKR_KEY_HANDLE_INVALID:**
The `hKey` is invalid.
- **CKR_MECHANISM_INVALID**
- **CKR_MECHANISM_PARAM_INVALID**
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by `C_Initialize()`.
- Other errors from the underlying implementation.

6.3.6.6.4 Single Part

6.3.6.6.4.1 C_Encrypt

CK_RV C_Encrypt([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pData, [CK_ULONG](#) ulDataLen, [CK_BYTE_PTR](#) pEncryptedData, [CK_ULONG_PTR](#) pulEncryptedDataLen)

Encrypt data in a single part.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pData** ([CK_BYTE_PTR](#)) - [in] Plaintext data buffer.
- **ulDataLen** ([CK_ULONG](#)) - [in] Length of plaintext data.
- **pEncryptedData** ([CK_BYTE_PTR](#)) - [out] Encrypted data buffer.
- **pulEncryptedDataLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to length of encrypted data buffer.

Description

The encryption operation MUST have been initialized with `C_EncryptInit()`. A call to `C_Encrypt()` always terminates the active encryption operation unless it returns `CKR_BUFFER_TOO_SMALL` or is a successful call (i.e., one which returns `CKR_OK`) to determine the length of the buffer needed to hold the ciphertext.

`C_Encrypt()` cannot be used to terminate a multi-part operation, and MUST be called after `C_EncryptInit()` without intervening `C_EncryptUpdate()` calls.

To query the required ciphertext buffer length, set `pEncryptedData` to `NULL_PTR`. The function will then set the required ciphertext buffer length in `pulEncryptedDataLen` and return `CKR_OK`.

On operation completion, the `pulEncryptedDataLen` is updated to the correct value when:

- `pulEncryptedDataLen` is bigger than expected. In this case, operation succeeds.
- `pulEncryptedDataLen` is shorter than expected. In this case, operation fails and returns `CKR_BUFFER_TOO_SMALL`.

For most mechanisms, [C_Encrypt\(\)](#) is equivalent to a sequence of [C_EncryptUpdate\(\)](#) operations followed by [C_EncryptFinal\(\)](#).

Return

- **CKR_OK:**
Success. Encryption operation complete.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
 - `pulEncryptedDataLen` is `NULL_PTR`.
 - Operation bad state.
- **CKR_BUFFER_TOO_SMALL:**
The output buffer is too small to hold the encrypted data.
- **CKR_DATA_INVALID:**
If it's not an Asymmetric Encryption or AEAD encryption, plaintext length is invalid.
- **CKR_DATA_LEN_RANGE:**
If Asymmetric Encryption or AEAD encryption, plaintext length is invalid.
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_EncryptInit\(\)](#).
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.6.4.2 C_Decrypt

CK_RV C_Decrypt([CK_SESSION_HANDLE](#) *hSession*, [CK_BYTE_PTR](#) *pEncryptedData*, [CK_ULONG](#) *ulEncryptedDataLen*, [CK_BYTE_PTR](#) *pData*, [CK_ULONG_PTR](#) *pulDataLen*)

Decrypt data in a single part.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pEncryptedData** ([CK_BYTE_PTR](#)) - [in] Encrypted data buffer.
- **ulEncryptedDataLen** ([CK_ULONG](#)) - [in] Length of encrypted data.
- **pData** ([CK_BYTE_PTR](#)) - [out] Decrypted data buffer.
- **pulDataLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to length of decrypted data buffer.

Description

The decryption operation MUST have been initialized with [C_DecryptInit\(\)](#). A call to [C_Decrypt\(\)](#) always terminates the active decryption operation unless it returns `CKR_BUFFER_TOO_SMALL` or is a successful call (i.e., one which returns `CKR_OK`) to determine the length of the buffer needed to hold the plaintext.

[C_Decrypt\(\)](#) cannot be used to terminate a multi-part operation, and MUST be called after [C_DecryptInit\(\)](#) without intervening [C_DecryptUpdate\(\)](#) calls.

To query the required plaintext buffer length, set `pData` to `NULL_PTR`. The function will then set the required plaintext buffer length in `pulDataLen` and return `CKR_OK`.

On operation completion, the `pulDataLen` is updated to the correct value when:

- `pulDataLen` is bigger than expected. In this case, operation succeeds.
- `pulDataLen` is shorter than expected. In this case, operation fails and returns `CKR_BUFFER_TOO_SMALL`.

For most mechanisms, [C_Decrypt\(\)](#) is equivalent to a sequence of [C_DecryptUpdate\(\)](#) operations followed by [C_DecryptFinal\(\)](#).

Return

- **CKR_OK:**
Success. Decryption operation complete.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
 - `pulDataLen` is `NULL_PTR`.
 - Operation bad state.
- **CKR_BUFFER_TOO_SMALL:**
The output buffer is too small to hold the decrypted data.
- **CKR_ENCRYPTED_DATA_INVALID:**
If it's not an Asymmetric Encryption or AEAD decryption, ciphertext length is invalid.
- **CKR_ENCRYPTED_DATA_LEN_RANGE:**
If Asymmetric Encryption or AEAD decryption, ciphertext length is invalid.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_DecryptInit\(\)](#).
- **CKR_GENERAL_ERROR**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.6.4.3 C_EncryptMessage

CK_RV C_EncryptMessage([CK_SESSION_HANDLE](#) `hSession`, [CK_VOID_PTR](#) `pParameter`, [CK_ULONG](#) `ulParameterLen`, [CK_BYTE_PTR](#) `pAssociatedData`, [CK_ULONG](#) `ulAssociatedDataLen`, [CK_BYTE_PTR](#) `pPlaintext`, [CK_ULONG](#) `ulPlaintextLen`, [CK_BYTE_PTR](#) `pCiphertext`, [CK_ULONG_PTR](#) `pulCiphertextLen`)

Encrypt a message in a single operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pParameter** ([CK_VOID_PTR](#)) - [in] Pointer to mechanism-specific parameters.
- **ulParameterLen** ([CK_ULONG](#)) - [in] Length of the parameter data.
- **pAssociatedData** ([CK_BYTE_PTR](#)) - [in] Pointer to associated data (for AEAD).
- **ulAssociatedDataLen** ([CK_ULONG](#)) - [in] Length of associated data.
- **pPlaintext** ([CK_BYTE_PTR](#)) - [in] Pointer to plaintext data.
- **ulPlaintextLen** ([CK_ULONG](#)) - [in] Length of plaintext data.
- **pCiphertext** ([CK_BYTE_PTR](#)) - [out] Pointer to buffer for ciphertext output.
- **pulCiphertextLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to ciphertext length.

Description

This function encrypts a complete message in a single call. For AEAD mechanisms, associated data can be provided. The plaintext and ciphertext can be in the same place, i.e., it is OK if `pPlaintext` and `pCiphertext` point to the same location.

To query the required ciphertext buffer length, set `pCiphertext` to `NULL_PTR`. The function will then set the required ciphertext buffer length in `pulCiphertextLen` and return `CKR_OK`.

On operation completion, the `pulCiphertextLen` is updated to the correct value when:

- `pulCiphertextLen` is bigger than expected. In this case, operation succeeds.
- `pulCiphertextLen` is shorter than expected. In this case, operation fails and returns `CKR_BUFFER_TOO_SMALL`.

Return

- **CKR_OK:**
Success. Message encryption operation complete.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
 - `pulCiphertextLen` is `NULL_PTR`.
 - Operation bad state.
- **CKR_BUFFER_TOO_SMALL:**
The output buffer is too small to hold the encrypted data.
- **CKR_DATA_INVALID:**
If it's not an Asymmetric Encryption or AEAD encryption, plaintext length is invalid.
- **CKR_DATA_LEN_RANGE:**
If Asymmetric Encryption or AEAD encryption, plaintext length is invalid.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_MessageEncryptInit\(\)](#).
- **CKR_MECHANISM_PARAM_INVALID**
- **CKR_GENERAL_ERROR:**
General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.6.4.4 C_DecryptMessage

[CK_RV](#) **C_DecryptMessage**([CK_SESSION_HANDLE](#) hSession, [CK_VOID_PTR](#) pParameter, [CK_ULONG](#) ulParameterLen, [CK_BYTE_PTR](#) pAssociatedData, [CK_ULONG](#) ulAssociatedDataLen, [CK_BYTE_PTR](#) pCiphertext, [CK_ULONG](#) ulCiphertextLen, [CK_BYTE_PTR](#) pPlaintext, [CK_ULONG_PTR](#) pulPlaintextLen)

Decrypt a message in a single operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pParameter** ([CK_VOID_PTR](#)) - [in] Pointer to mechanism-specific parameters.
- **ulParameterLen** ([CK_ULONG](#)) - [in] Length of the parameter data.
- **pAssociatedData** ([CK_BYTE_PTR](#)) - [in] Pointer to associated data (for AEAD).
- **ulAssociatedDataLen** ([CK_ULONG](#)) - [in] Length of associated data.
- **pCiphertext** ([CK_BYTE_PTR](#)) - [in] Pointer to ciphertext data.
- **ulCiphertextLen** ([CK_ULONG](#)) - [in] Length of ciphertext data.
- **pPlaintext** ([CK_BYTE_PTR](#)) - [out] Pointer to buffer for plaintext output.
- **pulPlaintextLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to plaintext length.

Description

This function decrypts a complete message in a single call. For AEAD mechanisms, associated data can be provided. The ciphertext and plaintext can be in the same place, i.e., it is OK if `pCiphertext` and `pPlaintext` point to the same location.

To query the required plaintext buffer length, set `pPlaintext` to `NULL_PTR`. The function will then set the required plaintext buffer length in `pulPlaintextLen` and return `CKR_OK`.

On operation completion, the `pulPlaintextLen` is updated to the correct value when:

- `pulPlaintextLen` is bigger than expected. In this case, operation succeeds.
- `pulPlaintextLen` is shorter than expected. In this case, operation fails and returns `CKR_BUFFER_TOO_SMALL`.

Return

- **CKR_OK:**

Success. Message decryption operation complete.

- **CKR_SESSION_HANDLE_INVALID:**

Session handle `hSession` is invalid.

- **CKR_ARGUMENTS_BAD:**

- `pulPlaintextLen` is `NULL_PTR`.
- Operation bad state.

- **CKR_BUFFER_TOO_SMALL:**

The output buffer is too small to hold the decrypted data.

- **CKR_DATA_INVALID:**

If it's not an Asymmetric Decryption or AEAD decryption, ciphertext length is invalid.

- **CKR_DATA_LEN_RANGE:**

If Asymmetric Decryption or AEAD decryption, ciphertext length is invalid.

- **CKR_OPERATION_NOT_INITIALIZED:**

The operation must be initialized with [C_MessageDecryptInit\(\)](#).

- **CKR_MECHANISM_PARAM_INVALID**

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.6.5 Multiple Part

6.3.6.6.5.1 C_EncryptUpdate

[CK_RV](#) **C_EncryptUpdate**([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pPart, [CK_ULONG](#) ulPartLen, [CK_BYTE_PTR](#) pEncryptedPart, [CK_ULONG_PTR](#) pulEncryptedPartLen)

Continue a multi-part encryption operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pPart** ([CK_BYTE_PTR](#)) - [in] Plaintext data part buffer.
- **ulPartLen** ([CK_ULONG](#)) - [in] Length of plaintext data part.
- **pEncryptedPart** ([CK_BYTE_PTR](#)) - [out] Encrypted data part buffer.
- **pulEncryptedPartLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to length of encrypted data part buffer.

Description

The encryption operation MUST have been initialized with [C_EncryptInit\(\)](#). This function may be called any number of times in succession. A call to [C_EncryptUpdate\(\)](#) which results in an error other than CKR_BUFFER_TOO_SMALL terminates the current encryption operation.

To query the required ciphertext buffer length, set pEncryptedPart to NULL_PTR. The function will then set the required ciphertext buffer length in pulEncryptedPartLen and return CKR_OK.

On operation completion, the pulEncryptedPartLen is updated to the correct value when:

- pulEncryptedPartLen is bigger than expected. In this case, operation succeeds.
- pulEncryptedPartLen is shorter than expected. In this case, operation fails and returns CKR_BUFFER_TOO_SMALL.

Return

- **CKR_OK:**

Success. Encryption operation updated.

- **CKR_SESSION_HANDLE_INVALID:**

Session handle `hSession` is invalid.

- **CKR_ARGUMENTS_BAD:**

- `pulEncryptedPartLen` is `NULL_PTR`.
- Operation bad state.

- **CKR_BUFFER_TOO_SMALL:**

The output buffer is too small to hold the encrypted data part.

- **CKR_DATA_INVALID:**

If it's not an Asymmetric Encryption or AEAD encryption, plaintext length is invalid.

- **CKR_DATA_LEN_RANGE:**

If Asymmetric Encryption or AEAD encryption, plaintext length is invalid.

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_OPERATION_NOT_INITIALIZED:**

The operation must be initialized with [C_EncryptInit\(\)](#).

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.6.5.2 C_EncryptFinal

CK_RV C_EncryptFinal([CK_SESSION_HANDLE](#) `hSession`, [CK_BYTE_PTR](#) `pLastEncryptedPart`, [CK_ULONG_PTR](#) `pulLastEncryptedPartLen`)

Finish a multi-part encryption operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pLastEncryptedPart** ([CK_BYTE_PTR](#)) - [out] Last encrypted data part buffer.
- **pulLastEncryptedPartLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to length of last encrypted data part.

Description

The encryption operation MUST have been initialized with [C_EncryptInit\(\)](#). A call to [C_EncryptFinal\(\)](#) always terminates the active encryption operation unless it returns `CKR_BUFFER_TOO_SMALL` or is a successful call (i.e., one which returns `CKR_OK`) to determine the length of the buffer needed to hold the ciphertext.

To query the required ciphertext buffer length, set `pLastEncryptedPart` to `NULL_PTR`. The function will then set the required ciphertext buffer length in `pulLastEncryptedPartLen` and return `CKR_OK`.

On operation completion, the `pulLastEncryptedPartLen` is updated to the correct value when:

- `pulLastEncryptedPartLen` is bigger than expected. In this case, operation succeeds.
- `pulLastEncryptedPartLen` is shorter than expected. In this case, operation fails and returns `CKR_BUFFER_TOO_SMALL`.

Return

- **CKR_OK:**
Success. Encryption operation complete.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
 - `pulLastEncryptedPartLen` is `NULL_PTR`.
 - Operation bad state.
- **CKR_BUFFER_TOO_SMALL:**
The output buffer is too small to hold the last encrypted data part.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_EncryptInit\(\)](#).
- **CKR_GENERAL_ERROR:**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.6.5.3 C_DecryptUpdate

[CK_RV](#) **C_DecryptUpdate**([CK_SESSION_HANDLE](#) `hSession`, [CK_BYTE_PTR](#) `pEncryptedPart`, [CK_ULONG](#) `ulEncryptedPartLen`, [CK_BYTE_PTR](#) `pPart`, [CK_ULONG_PTR](#) `pulPartLen`)

Continue a multi-part decryption operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pEncryptedPart** ([CK_BYTE_PTR](#)) - [in] Encrypted data part buffer.
- **ulEncryptedPartLen** ([CK_ULONG](#)) - [in] Length of encrypted data part.
- **pPart** ([CK_BYTE_PTR](#)) - [out] Decrypted data part buffer.
- **pulPartLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to length of decrypted data part buffer.

Description

The decryption operation MUST have been initialized with [C_DecryptInit\(\)](#). This function may be called any number of times in succession. A call to [C_DecryptUpdate\(\)](#) which results in an error other than `CKR_BUFFER_TOO_SMALL` terminates the current decryption operation.

To query the required plaintext buffer length, set `pData` to `NULL_PTR`. The function will then set the required plaintext buffer length in `pulDataLen` and return `CKR_OK`.

On operation completion, the `pulDataLen` is updated to the correct value when:

- `pulDataLen` is bigger than expected. In this case, operation succeeds.
- `pulDataLen` is shorter than expected. In this case, operation fails and returns `CKR_BUFFER_TOO_SMALL`.

Return

- **CKR_OK:**
Success. Decryption operation updated.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
 - `pulPartLen` is `NULL_PTR`.
 - Operation bad state.
- **CKR_BUFFER_TOO_SMALL:**
The output buffer is too small to hold the decrypted data part.
- **CKR_ENCRYPTED_DATA_INVALID:**
If it's not an Asymmetric Encryption or AEAD decryption, cipher text length is invalid.
- **CKR_ENCRYPTED_DATA_LEN_RANGE:**
If Asymmetric Encryption or AEAD decryption, cipher text length is invalid.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_DecryptInit\(\)](#).
- **CKR_GENERAL_ERROR**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.6.5.4 C_DecryptFinal

[CK_RV](#) **C_DecryptFinal**([CK_SESSION_HANDLE](#) `hSession`, [CK_BYTE_PTR](#) `pLastPart`, [CK_ULONG_PTR](#) `pulLastPartLen`)

Finish a multi-part decryption operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pLastPart** ([CK_BYTE_PTR](#)) - [out] Last decrypted data part buffer.
- **pulLastPartLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to length of last decrypted data part.

Description

The decryption operation MUST have been initialized with [C_DecryptInit\(\)](#). A call to [C_DecryptFinal\(\)](#) always terminates the active decryption operation unless it returns `CKR_BUFFER_TOO_SMALL` or is a successful call (i.e., one which returns `CKR_OK`) to determine the length of the buffer needed to hold the plaintext.

To query the required plaintext buffer length, set `pLastPart` to `NULL_PTR`. The function will then set the required plaintext buffer length in `pulLastPartLen` and return `CKR_OK`.

On operation completion, the `pulLastPartLen` is updated to the correct value when:

- `pullLastPartLen` is bigger than expected. In this case, operation succeeds.
- `pullLastPartLen` is shorter than expected. In this case, operation fails and returns `CKR_BUFFER_TOO_SMALL`.

Return

- **CKR_OK:**
Success. Decryption operation complete.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
 - `pullLastPartLen` is `NULL_PTR`.
 - Operation bad state.
- **CKR_BUFFER_TOO_SMALL:**
The output buffer is too small to hold the last decrypted data part.
- **CKR_GENERAL_ERROR**
General error.
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with `C_DecryptInit()`.
- Other errors from the underlying implementation.

6.3.6.6.5.5 C_EncryptMessageBegin

CK_RV `C_EncryptMessageBegin`([CK_SESSION_HANDLE](#) `hSession`, [CK_VOID_PTR](#) `pParameter`, [CK_ULONG](#) `ulParameterLen`, [CK_BYTE_PTR](#) `pAssociatedData`, [CK_ULONG](#) `ulAssociatedDataLen`)

Begin a multi-part message encryption operation.

Parameters

- `hSession` ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- `pParameter` ([CK_VOID_PTR](#)) - [in] Pointer to mechanism-specific parameters.
- `ulParameterLen` ([CK_ULONG](#)) - [in] Length of the parameter data.
- `pAssociatedData` ([CK_BYTE_PTR](#)) - [in] Pointer to associated data (for AEAD).
- `ulAssociatedDataLen` ([CK_ULONG](#)) - [in] Length of associated data.

Description

The message encryption operation MUST have been initialized with `C_MessageEncryptInit()`. This function begins a multi-part message encryption operation. For AEAD mechanisms, all associated data must be provided in this call before encrypting plaintext parts with `C_EncryptMessageNext()`.

Return

- **CKR_OK:**
Success. Multi-part message encryption operation begun.
- **CKR_SESSION_HANDLE_INVALID:**

Session handle `hSession` is invalid.

- **CKR_ARGUMENTS_BAD:**

Operation bad state.

- **CKR_MECHANISM_PARAM_INVALID**

- **CKR_OPERATION_NOT_INITIALIZED:**

The operation must be initialized with [C_MessageEncryptInit\(\)](#).

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.6.5.6 C_EncryptMessageNext

CK_RV C_EncryptMessageNext([CK_SESSION_HANDLE](#) hSession, [CK_VOID_PTR](#) pParameter, [CK_ULONG](#) ulParameterLen, [CK_BYTE_PTR](#) pPlaintextPart, [CK_ULONG](#) ulPlaintextPartLen, [CK_BYTE_PTR](#) pCiphertextPart, [CK_ULONG_PTR](#) pulCiphertextPartLen, [CK_FLAGS](#) flags)

Continue a multi-part message encryption operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pParameter** ([CK_VOID_PTR](#)) - [in] Pointer to mechanism-specific parameters.
- **ulParameterLen** ([CK_ULONG](#)) - [in] Length of the parameter data.
- **pPlaintextPart** ([CK_BYTE_PTR](#)) - [in] Pointer to plaintext data part.
- **ulPlaintextPartLen** ([CK_ULONG](#)) - [in] Length of plaintext part.
- **pCiphertextPart** ([CK_BYTE_PTR](#)) - [out] Pointer to buffer for ciphertext output.
- **pulCiphertextPartLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to ciphertext part length.
- **flags** ([CK_FLAGS](#)) - [in] Operation flags (CKF_END_OF_MESSAGE to indicate final part).

Description

The message encryption operation MUST have been initialized with [C_MessageEncryptInit\(\)](#) and begun with [C_EncryptMessageBegin\(\)](#). This function may be called any number of times in succession. A call to [C_EncryptMessageNext\(\)](#) which results in an error other than CKR_BUFFER_TOO_SMALL terminates the current encryption operation.

The `flags` parameter should include CKF_END_OF_MESSAGE when encrypting the final part of the message.

To query the required ciphertext buffer length, set `pCiphertextPart` to `NULL_PTR`. The function will then set the required ciphertext buffer length in `pulCiphertextPartLen` and return CKR_OK.

On operation completion, the `pulCiphertextPartLen` is updated to the correct value when:

- `pulCiphertextPartLen` is bigger than expected. In this case, operation succeeds.
- `pulCiphertextPartLen` is shorter than expected. In this case, operation fails and returns CKR_BUFFER_TOO_SMALL.

Return

- **CKR_OK:**
Success. Message encryption operation updated.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle `hSession` is invalid.
- **CKR_ARGUMENTS_BAD:**
 - `pulCiphertextPartLen` is `NULL_PTR`.
 - Operation bad state.
- **CKR_BUFFER_TOO_SMALL:**
The output buffer is too small to hold the encrypted data part.
- **CKR_DATA_INVALID:**
If it's not an Asymmetric Encryption or AEAD encryption, plaintext length is invalid.
- **CKR_DATA_LEN_RANGE:**
If Asymmetric Encryption or AEAD encryption, plaintext length is invalid.
- **CKR_MECHANISM_PARAM_INVALID**
- **CKR_OPERATION_NOT_INITIALIZED:**
The operation must be initialized with [C_MessageEncryptInit\(\)](#).
- **CKR_GENERAL_ERROR**
General error.
- **CKR_CRYPTOKI_NOT_INITIALIZED:**
The library has not been previously initialized by [C_Initialize\(\)](#).
- Other errors from the underlying implementation.

6.3.6.6.5.7 C_MessageEncryptFinal

[CK_RV](#) **C_MessageEncryptFinal**([CK_SESSION_HANDLE](#) `hSession`)

Finalize and cancel a message encryption operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.

Description

This function finalizes and cancels an active message encryption operation, releasing any resources associated with the operation context. A call to [C_MessageEncryptFinal\(\)](#) always terminates the active message encryption operation.

Return

- **CKR_OK:**
Success. Message encryption operation finalized and cancelled.
- **CKR_SESSION_HANDLE_INVALID:**

Session handle `hSession` is invalid.

- **CKR_OPERATION_NOT_INITIALIZED:**

The operation must be initialized with [C_MessageEncryptInit\(\)](#).

- **CKR_GENERAL_ERROR**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.6.5.8 C_DecryptMessageBegin

CK_RV C_DecryptMessageBegin([CK_SESSION_HANDLE](#) `hSession`, [CK_VOID_PTR](#) `pParameter`, [CK_ULONG](#) `ulParameterLen`, [CK_BYTE_PTR](#) `pAssociatedData`, [CK_ULONG](#) `ulAssociatedDataLen`)

Begin a multi-part message decryption operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pParameter** ([CK_VOID_PTR](#)) - [in] Pointer to mechanism-specific parameters.
- **ulParameterLen** ([CK_ULONG](#)) - [in] Length of the parameter data.
- **pAssociatedData** ([CK_BYTE_PTR](#)) - [in] Pointer to associated data (for AEAD).
- **ulAssociatedDataLen** ([CK_ULONG](#)) - [in] Length of associated data.

Description

The message decryption operation MUST have been initialized with [C_MessageDecryptInit\(\)](#). This function begins a multi-part message decryption operation. For AEAD mechanisms, all associated data must be provided in this call before decrypting ciphertext parts with [C_DecryptMessageNext\(\)](#).

Return

- **CKR_OK:**

Success. Multi-part message decryption operation begun.

- **CKR_SESSION_HANDLE_INVALID:**

Session handle `hSession` is invalid.

- **CKR_ARGUMENTS_BAD:**

Operation bad state.

- **CKR_MECHANISM_PARAM_INVALID**

- **CKR_OPERATION_NOT_INITIALIZED:**

The operation must be initialized with [C_MessageDecryptInit\(\)](#).

- **CKR_GENERAL_ERROR:**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.6.5.9 C_DecryptMessageNext

CK_RV C_DecryptMessageNext([CK_SESSION_HANDLE](#) hSession, [CK_VOID_PTR](#) pParameter, [CK_ULONG](#) ulParameterLen, [CK_BYTE_PTR](#) pCiphertext, [CK_ULONG](#) ulCiphertextLen, [CK_BYTE_PTR](#) pPlaintext, [CK_ULONG_PTR](#) pulPlaintextLen, [CK_FLAGS](#) flags)

Continue a multi-part message decryption operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.
- **pParameter** ([CK_VOID_PTR](#)) - [in] Pointer to mechanism-specific parameters.
- **ulParameterLen** ([CK_ULONG](#)) - [in] Length of the parameter data.
- **pCiphertext** ([CK_BYTE_PTR](#)) - [in] Pointer to ciphertext data part.
- **ulCiphertextLen** ([CK_ULONG](#)) - [in] Length of ciphertext part.
- **pPlaintext** ([CK_BYTE_PTR](#)) - [out] Pointer to buffer for plaintext output.
- **pulPlaintextLen** ([CK_ULONG_PTR](#)) - [in/out] Pointer to plaintext part length.
- **flags** ([CK_FLAGS](#)) - [in] Operation flags (CKF_END_OF_MESSAGE to indicate final part).

Description

The message decryption operation MUST have been initialized with [C_MessageDecryptInit\(\)](#) and begun with [C_DecryptMessageBegin\(\)](#). This function may be called any number of times in succession. A call to [C_DecryptMessageNext\(\)](#) which results in an error other than CKR_BUFFER_TOO_SMALL terminates the current decryption operation.

The **flags** parameter should include CKF_END_OF_MESSAGE when decrypting the final part of the message.

To query the required plaintext buffer length, set **pPlaintext** to NULL_PTR. The function will then set the required plaintext buffer length in **pulPlaintextLen** and return CKR_OK.

On operation completion, the **pulPlaintextLen** is updated to the correct value when:

- **pulPlaintextLen** is bigger than expected. In this case, operation succeeds.
- **pulPlaintextLen** is shorter than expected. In this case, operation fails and returns CKR_BUFFER_TOO_SMALL.

Return

- **CKR_OK:**
Success. Message decryption operation updated.
- **CKR_SESSION_HANDLE_INVALID:**
Session handle **hSession** is invalid.
- **CKR_ARGUMENTS_BAD:**
 - **pulPlaintextLen** is NULL_PTR.
 - Operation bad state.
- **CKR_BUFFER_TOO_SMALL:**
The output buffer is too small to hold the decrypted data part.
- **CKR_DATA_INVALID:**
If it's not an Asymmetric Decryption or AEAD decryption, ciphertext length is invalid.

- **CKR_DATA_LEN_RANGE:**

If Asymmetric Decryption or AEAD decryption, ciphertext length is invalid.

- **CKR_MECHANISM_PARAM_INVALID**

- **CKR_OPERATION_NOT_INITIALIZED:**

The operation must be initialized with [C_MessageDecryptInit\(\)](#).

- **CKR_GENERAL_ERROR**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.6.5.10 C_MessageDecryptFinal

[CK_RV](#) **C_MessageDecryptFinal**([CK_SESSION_HANDLE](#) hSession)

Finalize and cancel a message decryption operation.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session handle.

Description

This function finalizes and cancels an active message decryption operation, releasing any resources associated with the operation context. A call to [C_MessageDecryptFinal\(\)](#) always terminates the active message decryption operation.

Return

- **CKR_OK:**

Success. Message decryption operation finalized and cancelled.

- **CKR_SESSION_HANDLE_INVALID:**

Session handle hSession is invalid.

- **CKR_OPERATION_NOT_INITIALIZED:**

The operation must be initialized with [C_MessageDecryptInit\(\)](#).

- **CKR_GENERAL_ERROR**

General error.

- **CKR_CRYPTOKI_NOT_INITIALIZED:**

The library has not been previously initialized by [C_Initialize\(\)](#).

- Other errors from the underlying implementation.

6.3.6.6.6 Examples

6.3.6.6.6.1 PKCS#11 2.40 - Single Part Encryption

```
CK_SESSION_HANDLE hSession;
```



```
CK_OBJECT_HANDLE hKey;
CK_MECHANISM mechanism = { CKM_AES_CBC, iv, sizeof(iv) };
CK_BYTE plaintext[] = "Hello World";
CK_BYTE ciphertext[256];
CK_ULONG ciphertext_len = sizeof(ciphertext);
CK_RV rv;

// Initialize encryption operation
rv = C_EncryptInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    // Handle error
}

// Perform single-part encryption
rv = C_Encrypt(hSession, plaintext, sizeof(plaintext),
               ciphertext, &ciphertext_len);
if (rv != CKR_OK) {
    // Handle error
}
```

6.3.6.6.2 PKCS#11 2.40 - Multiple Part Encryption

```
CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_MECHANISM mechanism = { CKM_AES_CBC, iv, sizeof(iv) };
CK_BYTE plaintext_part1[] = "Hello ";
CK_BYTE plaintext_part2[] = "World";
CK_BYTE ciphertext[256];
CK_ULONG ciphertext_len = sizeof(ciphertext);
CK_RV rv;

// Initialize encryption operation
rv = C_EncryptInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    // Handle error
}

// Process first part
rv = C_EncryptUpdate(hSession, plaintext_part1, sizeof(plaintext_part1),
                    ciphertext, &ciphertext_len);
if (rv != CKR_OK) {
    // Handle error
}

// Process second part
CK_ULONG remaining_len = sizeof(ciphertext) - ciphertext_len;
rv = C_EncryptUpdate(hSession, plaintext_part2, sizeof(plaintext_part2),
                    ciphertext + ciphertext_len, &remaining_len);
if (rv != CKR_OK) {
    // Handle error
}
ciphertext_len += remaining_len;

// Finalize encryption
remaining_len = sizeof(ciphertext) - ciphertext_len;
rv = C_EncryptFinal(hSession, ciphertext + ciphertext_len, &remaining_len);
if (rv != CKR_OK) {
    // Handle error
}
```

```
}  
ciphertext_len += remaining_len;
```

6.3.6.6.3 PKCS#11 2.40 - Single Part Decryption

```
CK_SESSION_HANDLE hSession;  
CK_OBJECT_HANDLE hKey;  
CK_MECHANISM mechanism = { CKM_AES_CBC, iv, sizeof(iv) };  
CK_BYTE ciphertext[256];  
CK_ULONG ciphertext_len = 256;  
CK_BYTE plaintext[256];  
CK_ULONG plaintext_len = sizeof(plaintext);  
CK_RV rv;  
  
// Initialize decryption operation  
rv = C_DecryptInit(hSession, &mechanism, hKey);  
if (rv != CKR_OK) {  
    // Handle error  
}  
  
// Perform single-part decryption  
rv = C_Decrypt(hSession, ciphertext, ciphertext_len,  
               plaintext, &plaintext_len);  
if (rv != CKR_OK) {  
    // Handle error  
}
```

6.3.6.6.4 PKCS#11 2.40 - Multiple Part Decryption

```
CK_SESSION_HANDLE hSession;  
CK_OBJECT_HANDLE hKey;  
CK_MECHANISM mechanism = { CKM_AES_CBC, iv, sizeof(iv) };  
CK_BYTE ciphertext[256];  
CK_ULONG ciphertext_len = 256;  
CK_BYTE plaintext[256];  
CK_ULONG plaintext_len = sizeof(plaintext);  
CK_RV rv;  
  
// Initialize decryption operation  
rv = C_DecryptInit(hSession, &mechanism, hKey);  
if (rv != CKR_OK) {  
    // Handle error  
}  
  
// Process ciphertext in parts  
rv = C_DecryptUpdate(hSession, ciphertext, ciphertext_len / 2,  
                    plaintext, &plaintext_len);  
if (rv != CKR_OK) {  
    // Handle error  
}  
  
CK_ULONG remaining_len = sizeof(plaintext) - plaintext_len;  
rv = C_DecryptUpdate(hSession, ciphertext + ciphertext_len / 2,  
                    ciphertext_len - ciphertext_len / 2,  
                    plaintext + plaintext_len, &remaining_len);  
if (rv != CKR_OK) {  
    // Handle error  
}
```

```

}
plaintext_len += remaining_len;

// Finalize decryption
remaining_len = sizeof(plaintext) - plaintext_len;
rv = C_DecryptFinal(hSession, plaintext + plaintext_len, &remaining_len);
if (rv != CKR_OK) {
    // Handle error
}
plaintext_len += remaining_len;

```

6.3.6.6.5 PKCS#11 3.2 - Single Part Message Encryption

```

CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_MECHANISM mechanism = { CKM_AES_GCM, &gcm_params, sizeof(gcm_params) };
CK_BYTE plaintext[] = "Message to encrypt";
CK_BYTE aad[] = "Additional authenticated data";
CK_BYTE ciphertext[256];
CK_ULONG ciphertext_len = sizeof(ciphertext);
CK_RV rv;

// Initialize message encryption operation
rv = C_MessageEncryptInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    // Handle error
}

// Perform single-part message encryption
rv = C_EncryptMessage(hSession, NULL_PTR, 0, aad, sizeof(aad),
                     plaintext, sizeof(plaintext),
                     ciphertext, &ciphertext_len);

if (rv != CKR_OK) {
    // Handle error
}

```

6.3.6.6.6 PKCS#11 3.2 - Multiple Part Message Encryption

```

CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_MECHANISM mechanism = { CKM_AES_GCM, &gcm_params, sizeof(gcm_params) };
CK_BYTE plaintext_part1[] = "First part ";
CK_BYTE plaintext_part2[] = "Second part";
CK_BYTE aad[] = "Additional authenticated data";
CK_BYTE ciphertext[256];
CK_ULONG ciphertext_len = sizeof(ciphertext);
CK_RV rv;

// Initialize message encryption operation
rv = C_MessageEncryptInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    // Handle error
}

// Begin message encryption with AAD
rv = C_EncryptMessageBegin(hSession, NULL_PTR, 0, aad, sizeof(aad));

```

```
if (rv != CKR_OK) {
    // Handle error
}

// Process first part
rv = C_EncryptMessageNext(hSession, NULL_PTR, 0, plaintext_part1,
                          sizeof(plaintext_part1), NULL_PTR, NULL_PTR, 0);
if (rv != CKR_OK) {
    // Handle error
}

// Process second part and get ciphertext
rv = C_EncryptMessageNext(hSession, NULL_PTR, 0, plaintext_part2,
                          sizeof(plaintext_part2), ciphertext,
                          &ciphertext_len, CKF_END_OF_MESSAGE);
if (rv != CKR_OK) {
    // Handle error
}

// Finalize message encryption
rv = C_MessageEncryptFinal(hSession);
if (rv != CKR_OK) {
    // Handle error
}
```

6.3.6.6.7 PKCS#11 3.2 - Single Part Message Decryption

```
CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_MECHANISM mechanism = { CKM_AES_GCM, &gcm_params, sizeof(gcm_params) };
CK_BYTE ciphertext[256];
CK_ULONG ciphertext_len = 256;
CK_BYTE aad[] = "Additional authenticated data";
CK_BYTE plaintext[256];
CK_ULONG plaintext_len = sizeof(plaintext);
CK_RV rv;

// Initialize message decryption operation
rv = C_MessageDecryptInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    // Handle error
}

// Perform single-part message decryption
rv = C_DecryptMessage(hSession, NULL_PTR, 0, aad, sizeof(aad),
                      ciphertext, ciphertext_len,
                      plaintext, &plaintext_len);
if (rv != CKR_OK) {
    // Handle error
}
```

6.3.6.6.8 PKCS#11 3.2 - Multiple Part Message Decryption

```
CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_MECHANISM mechanism = { CKM_AES_GCM, &gcm_params, sizeof(gcm_params) };
CK_BYTE ciphertext_part1[128];
```

```

CK_ULONG ciphertext_part1_len = 128;
CK_BYTE ciphertext_part2[128];
CK_ULONG ciphertext_part2_len = 128;
CK_BYTE aad[] = "Additional authenticated data";
CK_BYTE plaintext[256];
CK_ULONG plaintext_len = sizeof(plaintext);
CK_RV rv;

// Initialize message decryption operation
rv = C_MessageDecryptInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    // Handle error
}

// Begin message decryption with AAD
rv = C_DecryptMessageBegin(hSession, NULL_PTR, 0, aad, sizeof(aad));
if (rv != CKR_OK) {
    // Handle error
}

// Process first part
rv = C_DecryptMessageNext(hSession, NULL_PTR, 0, ciphertext_part1,
                           ciphertext_part1_len, NULL_PTR, NULL_PTR, 0);
if (rv != CKR_OK) {
    // Handle error
}

// Process second part and get plaintext
rv = C_DecryptMessageNext(hSession, NULL_PTR, 0, ciphertext_part2,
                           ciphertext_part2_len, plaintext,
                           &plaintext_len, CKF_END_OF_MESSAGE);
if (rv != CKR_OK) {
    // Handle error
}

// Finalize message decryption
rv = C_MessageDecryptFinal(hSession);
if (rv != CKR_OK) {
    // Handle error
}

```

6.3.6.7 Authentication Encryption/Decryption with associated data (AEAD)

PKCS#11 provides two families of functions for authenticated encryption and decryption with associated data (AEAD). Refer to [Symmetric Encryption](#) section for the detailed description of the operation sequences:

- The **single-operation** family (`C_EncryptInit / C_Encrypt` and `C_DecryptInit / C_Decrypt`) embeds the IV/nonce and AAD inside the mechanism parameters and processes the entire message in one call.
- The **message** family (`C_MessageEncryptInit / C_EncryptMessage` and `C_MessageDecryptInit / C_DecryptMessage`) allows per-message IV/nonce and AAD to be supplied at encrypt/decrypt time, and supports both single-part and multi-part message processing.

The mechanisms are listed in the [Authentication Encryption Mechanisms \(AEAD\)](#) section.

6.3.6.7.1 Examples

6.3.6.7.1.1 Single-Operation AEAD Encryption (C_EncryptInit / C_Encrypt)

The IV/nonce, AAD and tag length are all supplied through the mechanism parameters at initialization time.

```
CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_BYTE iv[] = { /* 12-byte IV */ };
CK_BYTE aad[] = { /* additional authenticated data */ };
CK_BYTE plaintext[] = { /* data to encrypt */ };
CK_BYTE ciphertext[sizeof(plaintext) + 16]; /* plaintext + 128-bit tag */
CK_ULONG ciphertext_len = sizeof(ciphertext);
CK_RV rv;

CK_GCM_PARAMS gcm_params = {
    .pIv = iv,
    .ulIvLen = sizeof(iv),
    .pAAD = aad,
    .ulAADLen = sizeof(aad),
    .ulTagBits = 128
};

CK_MECHANISM mechanism = {
    .mechanism = CKM_AES_GCM,
    .pParameter = &gcm_params,
    .ulParameterLen = sizeof(gcm_params)
};

rv = C_EncryptInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    /* Handle error */
}

rv = C_Encrypt(hSession, plaintext, sizeof(plaintext),
               ciphertext, &ciphertext_len);
if (rv != CKR_OK) {
    /* Handle error */
}
/* ciphertext_len bytes of authenticated ciphertext (data + tag) are ready */
```

6.3.6.7.1.2 Single-Operation AEAD Decryption (C_DecryptInit / C_Decrypt)

The ciphertext buffer must include the authentication tag appended after the encrypted data (i.e. ulTagBits / 8 bytes appended).

```
CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_BYTE iv[] = { /* 12-byte IV used during encryption */ };
CK_BYTE aad[] = { /* same additional authenticated data used during encryption */ };
CK_BYTE ciphertext[data_len + 16]; /* encrypted data + 128-bit tag */
CK_BYTE plaintext[data_len];
CK_ULONG plaintext_len = sizeof(plaintext);
CK_RV rv;

CK_GCM_PARAMS gcm_params = {
    .pIv = iv,
    .ulIvLen = sizeof(iv),

```

```

        .pAAD = aad,
        .ulAADLen = sizeof(aad),
        .ulTagBits = 128
    };
    CK_MECHANISM mechanism = {
        .mechanism = CKM_AES_GCM,
        .pParameter = &gcm_params,
        .ulParameterLen = sizeof(gcm_params)
    };

    rv = C_DecryptInit(hSession, &mechanism, hKey);
    if (rv != CKR_OK) {
        /* Handle error */
    }

    rv = C_Decrypt(hSession, ciphertext, sizeof(ciphertext),
        plaintext, &plaintext_len);
    if (rv != CKR_OK) {
        /* Handle error - CKR_ENCRYPTED_DATA_INVALID if authentication fails */
    }
    /* plaintext_len bytes of verified plaintext are ready */

```

6.3.6.7.1.3 Single-Part Message AEAD Encryption (C_MessageEncryptInit / C_EncryptMessage)

The mechanism is initialized once; per-message IV/nonce and AAD are supplied at each C_EncryptMessage call via pParameter (a CK_GCM_MESSAGE_PARAMS for AES-GCM) and the pAssociatedData / ulAssociatedDataLen arguments.

```

CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_BYTE iv[12];    /* IV buffer - filled by the library when using CKG_GENERATE */
CK_BYTE tag[16];   /* tag output buffer */
CK_BYTE aad[] = { /* additional authenticated data */ };
CK_BYTE plaintext[] = { /* data to encrypt */ };
CK_BYTE ciphertext[sizeof(plaintext)];
CK_ULONG ciphertext_len = sizeof(ciphertext);
CK_RV rv;

/* Mechanism has no per-init parameters for the message API */
CK_MECHANISM mechanism = {
    .mechanism = CKM_AES_GCM,
    .pParameter = NULL,
    .ulParameterLen = 0
};

rv = C_MessageEncryptInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    /* Handle error */
}

/* Per-message parameters: supply IV or let the library generate it */
CK_GCM_MESSAGE_PARAMS msg_params = {
    .pIv = iv,
    .ulIvLen = sizeof(iv),
    .ulIvFixedBits = 0,
    .ivGenerator = CKG_GENERATE, /* library generates the IV */

```

```

        .pTag = tag,
        .ulTagBits = 128
    };

    rv = C_EncryptMessage(hSession,
                          &msg_params, sizeof(msg_params),
                          aad, sizeof(aad),
                          plaintext, sizeof(plaintext),
                          ciphertext, &ciphertext_len);
    if (rv != CKR_OK) {
        /* Handle error */
    }
    /* iv[] now holds the generated IV; tag[] holds the authentication tag;
       ciphertext[] holds the encrypted data */

    /* Terminate the message encryption session */
    C_MessageEncryptFinal(hSession);

```

6.3.6.7.1.4 Single-Part Message AEAD Decryption (C_MessageDecryptInit / C_DecryptMessage)

```

CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_BYTE iv[12];    /* IV used during encryption */
CK_BYTE tag[16];   /* tag produced during encryption */
CK_BYTE aad[] = { /* same AAD used during encryption */ };
CK_BYTE ciphertext[] = { /* encrypted data (without tag) */ };
CK_BYTE plaintext[sizeof(ciphertext)];
CK_ULONG plaintext_len = sizeof(plaintext);
CK_RV rv;

CK_MECHANISM mechanism = {
    .mechanism = CKM_AES_GCM,
    .pParameter = NULL,
    .ulParameterLen = 0
};

rv = C_MessageDecryptInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    /* Handle error */
}

CK_GCM_MESSAGE_PARAMS msg_params = {
    .pIv = iv,
    .ulIvLen = sizeof(iv),
    .ulIvFixedBits = 0,
    .ivGenerator = CKG_NO_GENERATE, /* caller supplies the IV */
    .pTag = tag,
    .ulTagBits = 128
};

rv = C_DecryptMessage(hSession,
                      &msg_params, sizeof(msg_params),
                      aad, sizeof(aad),
                      ciphertext, sizeof(ciphertext),
                      plaintext, &plaintext_len);
if (rv != CKR_OK) {
    /* Handle error - CKR_ENCRYPTED_DATA_INVALID if authentication fails */
}

```



```

}

/* Terminate the message decryption session */
C_MessageDecryptFinal(hSession);

```

6.3.6.7.1.5 Multi-Part Message AEAD Encryption

C_EncryptMessageBegin supplies the per-message parameters and the complete AAD.

C_EncryptMessageNext is then called for each plaintext chunk, with CKF_END_OF_MESSAGE set on the last chunk. C_MessageEncryptFinal terminates the session.

```

CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_BYTE iv[12];
CK_BYTE tag[16];
CK_BYTE aad[] = { /* additional authenticated data */ };
CK_BYTE plaintext_part1[128];
CK_BYTE plaintext_part2[128];
CK_BYTE ciphertext[sizeof(plaintext_part1) + sizeof(plaintext_part2)];
CK_ULONG part_len;
CK_RV rv;

CK_MECHANISM mechanism = {
    .mechanism = CKM_AES_GCM,
    .pParameter = NULL,
    .ulParameterLen = 0
};

rv = C_MessageEncryptInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    /* Handle error */
}

CK_GCM_MESSAGE_PARAMS msg_params = {
    .pIv = iv,
    .ulIvLen = sizeof(iv),
    .ulIvFixedBits = 0,
    .ivGenerator = CKG_GENERATE,
    .pTag = tag,
    .ulTagBits = 128
};

/* Begin: provide per-message params and the full AAD */
rv = C_EncryptMessageBegin(hSession,
                           &msg_params, sizeof(msg_params),
                           aad, sizeof(aad));

if (rv != CKR_OK) {
    /* Handle error */
}

/* Encrypt first part */
part_len = sizeof(plaintext_part1);
rv = C_EncryptMessageNext(hSession,
                           &msg_params, sizeof(msg_params),
                           plaintext_part1, sizeof(plaintext_part1),
                           ciphertext, &part_len,
                           0); /* not the last part */

if (rv != CKR_OK) {

```

```

    /* Handle error */
}

/* Encrypt last part - signal end of message */
CK_ULONG last_len = sizeof(ciphertext) - part_len;
rv = C_EncryptMessageNext(hSession,
                           &msg_params, sizeof(msg_params),
                           plaintext_part2, sizeof(plaintext_part2),
                           ciphertext + part_len, &last_len,
                           CKF_END_OF_MESSAGE);

if (rv != CKR_OK) {
    /* Handle error */
}
/* iv[] holds the generated IV; tag[] holds the authentication tag */

/* Terminate the message encryption session */
C_MessageEncryptFinal(hSession);

```

6.3.6.7.1.6 Multi-Part Message AEAD Decryption

```

CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_BYTE iv[12];
CK_BYTE tag[16];
CK_BYTE aad[] = { /* same AAD used during encryption */ };
CK_BYTE ciphertext_part1[128];
CK_BYTE ciphertext_part2[128];
CK_BYTE plaintext[sizeof(ciphertext_part1) + sizeof(ciphertext_part2)];
CK_ULONG part_len;
CK_RV rv;

CK_MECHANISM mechanism = {
    .mechanism = CKM_AES_GCM,
    .pParameter = NULL,
    .ulParameterLen = 0
};

rv = C_MessageDecryptInit(hSession, &mechanism, hKey);
if (rv != CKR_OK) {
    /* Handle error */
}

CK_GCM_MESSAGE_PARAMS msg_params = {
    .pIv = iv,
    .ulIvLen = sizeof(iv),
    .ulIvFixedBits = 0,
    .ivGenerator = CKG_NO_GENERATE,
    .pTag = tag,
    .ulTagBits = 128
};

/* Begin: provide per-message params and the full AAD */
rv = C_DecryptMessageBegin(hSession,
                           &msg_params, sizeof(msg_params),
                           aad, sizeof(aad));

if (rv != CKR_OK) {
    /* Handle error */
}

```

```

/* Decrypt first part */
part_len = sizeof(ciphertext_part1);
rv = C_DecryptMessageNext(hSession,
                           &msg_params, sizeof(msg_params),
                           ciphertext_part1, sizeof(ciphertext_part1),
                           plaintext, &part_len,
                           0); /* not the last part */

if (rv != CKR_OK) {
    /* Handle error */
}

/* Decrypt last part - signal end of message */
CK_ULONG last_len = sizeof(plaintext) - part_len;
rv = C_DecryptMessageNext(hSession,
                           &msg_params, sizeof(msg_params),
                           ciphertext_part2, sizeof(ciphertext_part2),
                           plaintext + part_len, &last_len,
                           CKF_END_OF_MESSAGE);

if (rv != CKR_OK) {
    /* Handle error - CKR_ENCRYPTED_DATA_INVALID if authentication fails */
}

/* Terminate the message decryption session */
C_MessageDecryptFinal(hSession);

```

6.3.6.8 Dual functions

PKCS#11 provides dual-function cryptographic operations that combine two operations in a single call. These functions allow for efficient processing by combining encryption/decryption with signing/verification or digest operations.

The dual functions available are:

Table 201. PKCS#11 Dual Function APIs

Function	Description
C_DigestEncryptUpdate()	Continues a multiple-part digest and encryption operation
C_DecryptDigestUpdate()	Continues a multiple-part decryption and digest operation
C_SignEncryptUpdate()	Continues a multiple-part signing and encryption operation
C_DecryptVerifyUpdate()	Continues a multiple-part decryption and verification operation

These functions must be used in conjunction with their respective initialization and finalization functions. Each dual operation requires both operations to be initialized separately before the dual function can be called.

6.3.6.8.1 Operation Sequences

Digest and Encrypt

The sequence of operations to digest and encrypt data is as follows:

1. Initialize the digest operation by calling [C_DigestInit\(\)](#).
2. Initialize the encryption operation by calling [C_EncryptInit\(\)](#).
3. Process one or more data parts (can be called multiple times) with [C_DigestEncryptUpdate\(\)](#).
4. Finalize the encryption operation with [C_EncryptFinal\(\)](#).
5. Finalize the digest operation with [C_DigestFinal\(\)](#).

Decrypt and Digest

The sequence of operations to decrypt and digest data is as follows:

1. Initialize the decryption operation by calling [C_DecryptInit\(\)](#).
2. Initialize the digest operation by calling [C_DigestInit\(\)](#).
3. Process one or more data parts (can be called multiple times) with [C_DecryptDigestUpdate\(\)](#).
4. Finalize the decryption operation with [C_DecryptFinal\(\)](#).
5. Finalize the digest operation with [C_DigestFinal\(\)](#).

Sign and Encrypt

The sequence of operations to sign and encrypt data is as follows:

1. Initialize the signing operation by calling [C_SignInit\(\)](#).
2. Initialize the encryption operation by calling [C_EncryptInit\(\)](#).
3. Process one or more data parts (can be called multiple times) with [C_SignEncryptUpdate\(\)](#).
4. Finalize the encryption operation with [C_EncryptFinal\(\)](#).
5. Finalize the signing operation with [C_SignFinal\(\)](#).

Decrypt and Verify

The sequence of operations to decrypt and verify data is as follows:

1. Initialize the decryption operation by calling [C_DecryptInit\(\)](#).
2. Initialize the verification operation by calling [C_VerifyInit\(\)](#).
3. Process one or more data parts (can be called multiple times) with [C_DecryptVerifyUpdate\(\)](#).
4. Finalize the decryption operation with [C_DecryptFinal\(\)](#).
5. Finalize the verification operation with [C_VerifyFinal\(\)](#).

6.3.6.8.2 API Reference

6.3.6.8.2.1 C_DigestEncryptUpdate

[CK_RV](#) **C_DigestEncryptUpdate**([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pPart, [CK_ULONG](#) ulPartLen, [CK_BYTE_PTR](#) pEncryptedPart, [CK_ULONG_PTR](#) pulEncryptedPartLen)

Continue multi-part digest and encryption.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session's handle.
- **pPart** ([CK_BYTE_PTR](#)) - [in] Data part to digest and encrypt.
- **ulPartLen** ([CK_ULONG](#)) - [in] Length of data part in bytes.
- **pEncryptedPart** ([CK_BYTE_PTR](#)) - [out] Encrypted data part.
- **pulEncryptedPartLen** ([CK_ULONG_PTR](#)) - [in/out] Length of encrypted data part in bytes.

Description

This function continues a multi-part dual operation, processing another data part for both digesting and encryption operations.

Note:

This function is not supported.

Return

- **CKR_FUNCTION_NOT_SUPPORTED:**

Function not supported.

6.3.6.8.2.2 C_DecryptDigestUpdate

[CK_RV](#) **C_DecryptDigestUpdate**([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pEncryptedPart, [CK_ULONG](#) ulEncryptedPartLen, [CK_BYTE_PTR](#) pPart, [CK_ULONG_PTR](#) pulPartLen)

Continue multi-part decryption and digest.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session's handle.
- **pEncryptedPart** ([CK_BYTE_PTR](#)) - [in] Encrypted data part to decrypt and digest.
- **ulEncryptedPartLen** ([CK_ULONG](#)) - [in] Length of encrypted data part in bytes.
- **pPart** ([CK_BYTE_PTR](#)) - [out] Decrypted data part.
- **pulPartLen** ([CK_ULONG_PTR](#)) - [in/out] Length of decrypted data part in bytes.

Description

This function continues a multi-part dual operation, processing another encrypted data part for both decryption and digesting operations.

Note:

This function is not supported.

Return

- **CKR_FUNCTION_NOT_SUPPORTED:**

Function not supported.

6.3.6.8.2.3 C_SignEncryptUpdate

[CK_RV](#) **C_SignEncryptUpdate**([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pPart, [CK_ULONG](#) ulPartLen, [CK_BYTE_PTR](#) pEncryptedPart, [CK_ULONG_PTR](#) pulEncryptedPartLen)

Continue multi-part signing and encryption.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session's handle.
- **pPart** ([CK_BYTE_PTR](#)) - [in] Data part to sign and encrypt.
- **ulPartLen** ([CK_ULONG](#)) - [in] Length of data part in bytes.
- **pEncryptedPart** ([CK_BYTE_PTR](#)) - [out] Encrypted data part.
- **pulEncryptedPartLen** ([CK_ULONG_PTR](#)) - [in/out] Length of encrypted data part in bytes.

Description

This function continues a multi-part dual operation, processing another data part for both signing and encryption operations.

Note:

This function is not supported.

Return

- **CKR_FUNCTION_NOT_SUPPORTED:**

Function not supported.

6.3.6.8.2.4 C_DecryptVerifyUpdate

[CK_RV](#) **C_DecryptVerifyUpdate**([CK_SESSION_HANDLE](#) hSession, [CK_BYTE_PTR](#) pEncryptedPart, [CK_ULONG](#) ulEncryptedPartLen, [CK_BYTE_PTR](#) pPart, [CK_ULONG_PTR](#) pulPartLen)

Continue multi-part decryption and verification.

Parameters

- **hSession** ([CK_SESSION_HANDLE](#)) - [in] Session's handle.
- **pEncryptedPart** ([CK_BYTE_PTR](#)) - [in] Encrypted data part to decrypt and verify.
- **ulEncryptedPartLen** ([CK_ULONG](#)) - [in] Length of encrypted data part in bytes.
- **pPart** ([CK_BYTE_PTR](#)) - [out] Decrypted data part.
- **pulPartLen** ([CK_ULONG_PTR](#)) - [in/out] Length of decrypted data part in bytes.

Description

This function continues a multi-part dual operation, processing another encrypted data part for both decryption and signature verification operations.

Note:

This function is not supported.

Return

- **CKR_FUNCTION_NOT_SUPPORTED:**

Function not supported.

6.3.6.8.3 Examples

6.3.6.8.3.1 Digest and Encrypt

```
CK_SESSION_HANDLE hSession;  
CK_OBJECT_HANDLE hKey;  
CK_MECHANISM digest_mech = { CKM_SHA256, NULL_PTR, 0 };  
CK_MECHANISM encrypt_mech = { CKM_AES_CBC, iv, sizeof(iv) };  
CK_BYTE data[] = "Data to digest and encrypt";  
CK_BYTE encrypted[256];
```

```
CK_ULONG encrypted_len = sizeof(encrypted);
CK_BYTE digest[32];
CK_ULONG digest_len = sizeof(digest);
CK_RV rv;

// Initialize digest operation
rv = C_DigestInit(hSession, &digest_mech);
if (rv != CKR_OK) {
    // Handle error
}

// Initialize encryption operation
rv = C_EncryptInit(hSession, &encrypt_mech, hKey);
if (rv != CKR_OK) {
    // Handle error
}

// Process data with dual function
rv = C_DigestEncryptUpdate(hSession, data, sizeof(data),
                           encrypted, &encrypted_len);
if (rv != CKR_OK) {
    // Handle error
}

// Finalize encryption
rv = C_EncryptFinal(hSession, encrypted + encrypted_len, &encrypted_len);
if (rv != CKR_OK) {
    // Handle error
}

// Finalize digest
rv = C_DigestFinal(hSession, digest, &digest_len);
if (rv != CKR_OK) {
    // Handle error
}
```

6.3.6.8.3.2 Decrypt and Verify

```
CK_SESSION_HANDLE hSession;
CK_OBJECT_HANDLE hKey;
CK_MECHANISM decrypt_mech = { CKM_AES_CBC, iv, sizeof(iv) };
CK_MECHANISM verify_mech = { CKM_RSA_PKCS, NULL_PTR, 0 };
CK_BYTE encrypted_data[] = "Encrypted data";
CK_BYTE decrypted[256];
CK_ULONG decrypted_len = sizeof(decrypted);
CK_BYTE signature[256];
CK_ULONG signature_len = 256;
CK_RV rv;

// Initialize decryption operation
rv = C_DecryptInit(hSession, &decrypt_mech, hKey);
if (rv != CKR_OK) {
    // Handle error
}

// Initialize verification operation
rv = C_VerifyInit(hSession, &verify_mech, hKey);
if (rv != CKR_OK) {
```

```
    // Handle error
}

// Process data with dual function
rv = C_DecryptVerifyUpdate(hSession, encrypted_data, sizeof(encrypted_data),
                           decrypted, &decrypted_len);
if (rv != CKR_OK) {
    // Handle error
}

// Finalize decryption
rv = C_DecryptFinal(hSession, decrypted + decrypted_len, &decrypted_len);
if (rv != CKR_OK) {
    // Handle error
}

// Finalize verification
rv = C_VerifyFinal(hSession, signature, signature_len);
if (rv != CKR_OK) {
    // Handle error - signature invalid or other error
}
```

6.3.7 Key Types

SMW's PKCS#11 implementation supports the key types describes in this chapter.

6.3.7.1 Symmetric Keys

Table 202. Symmetric Key Types

Key Type	Description
CKK_AES	AES (Advanced Encryption Standard) keys
CKK_DES	DES (Data Encryption Standard) keys
CKK_DES3	Triple-length DES keys (3-key 3DES)
CKK_GENERIC_SECRET	Generic secret keys for HMAC and other purposes

6.3.7.2 Asymmetric Keys

Table 203. Asymmetric Key Types

Key Type	Description
CKK_RSA	RSA public/private key pairs
CKK_EC	Elliptic Curve public/private key pairs
CKK_EC_EDWARDS	Edwards Curve keys (Ed25519, Ed448)
CKK_EC_MONTGOMERY	Montgomery Curve keys (X25519, X448)
CKK_DH	Diffie-Hellman key pairs

6.3.8 Mechanisms

The PKCS#11 API uses mechanisms to specify cryptographic operations. A mechanism defines the algorithm and parameters for operations such as encryption, decryption, signing, verification, key generation, and key derivation.

The following sections describe the mechanisms supported by SMW's PKCS#11.

6.3.8.1 Symmetric Key Generation Mechanisms

Table 204. Symmetric Key Generation Mechanisms

Mechanism	Description
CKM_AES_KEY_GEN	Generate AES keys (128, 192, or 256 bits).
CKM_DES_KEY_GEN	Generate single DES keys.
CKM_DES3_KEY_GEN	Generate triple-length DES keys.
CKM_SM4_KEY_GEN	Generate SM4 keys (NXP vendor define).
CKM_GENERIC_SECRET_KEY_GEN	Generate generic secret keys.

6.3.8.2 Asymmetric Key Generation Mechanisms

Table 205. Asymmetric Key Generation Mechanisms

Mechanism	Description
CKM_RSA_PKCS_KEY_PAIR_GEN	Generate RSA key pairs.
CKM_EC_KEY_PAIR_GEN	Generate EC key pairs.
CKM_EC_EDWARDS_KEY_PAIR_GEN	Generate Edwards curve key pairs.
CKM_EC_MONTGOMERY_KEY_PAIR_GEN	Generate Montgomery curve key pairs.
CKM_RSA_X9_31_KEY_PAIR_GEN	Generate RSA key pairs.

6.3.8.3 Key Derivation Mechanisms

Table 206. Key Derivation Mechanisms

Mechanism	Description
CKM_ECDH1_DERIVE	ECDH key derivation (Elliptic Curve Diffie-Hellman)
CKM_HKDF_DERIVE	HMAC-based Key Derivation Function
CKM_TLS12_MASTER_KEY_DERIVE	TLS 1.2 master secret derivation
CKM_TLS12_KEY_AND_MAC_DERIVE	TLS 1.2 key and MAC derivation
CKM_TLS12_EXTENDED_MASTER_KEY_DERIVE	TLS 1.2 extended master secret derivation (RFC 7627)
CKM_TLS12_EXTENDED_MASTER_KEY_DERIVE_DH	TLS 1.2 extended master secret derivation using Diffie-Hellman (RFC 7627)

6.3.8.3.1 ECDH Parameters

For ECDH key derivation (CKM_ECDH1_DERIVE):

```
typedef struct CK_ECDH1_DERIVE_PARAMS {
    CK_EC_KDF_TYPE kdf;           /* Key derivation function */
    CK_ULONG ulSharedDataLen;     /* Length of shared data in bytes */
    CK_BYTE_PTR pSharedData;      /* Shared data (e.g. additional input) */
    CK_ULONG ulPublicDataLen;     /* Length of public key data in bytes */
    CK_BYTE_PTR pPublicData;      /* Public key of other party */
} CK_ECDH1_DERIVE_PARAMS;
```

6.3.8.3.2 HKDF Parameters

For HMAC-based Key Derivation (CKM_HKDF_DERIVE):

```
typedef struct CK_HKDF_PARAMS {
    CK_BBOOL bExtract;           /* Perform extraction step */
    CK_BBOOL bExpand;           /* Perform expansion step */
    CK_MECHANISM_TYPE prfHashMechanism; /* PRF hash mechanism */
    CK_ULONG ulSaltType;        /* Salt type */
    CK_BYTE_PTR pSalt;          /* Salt data (if ulSaltType is
    CKF_HKDF_SALT_DATA) */
    CK_ULONG ulSaltLen;         /* Length of salt in bytes */
    CK_OBJECT_HANDLE hSaltKey;  /* Salt key handle (if ulSaltType is
    CKF_HKDF_SALT_KEY) */
    CK_BYTE_PTR pInfo;          /* Context and application specific
    information */
    CK_ULONG ulInfoLen;         /* Length of info in bytes */
} CK_HKDF_PARAMS;
```

Where *ulSaltType* can be:

- CKF_HKDF_SALT_NULL: no salt supplied.
- CKF_HKDF_SALT_DATA: salt supplied as data in pSalt with length ulSaltLen.
- CKF_HKDF_SALT_KEY: salt supplied as key in the hSaltKey.

6.3.8.3.3 TLS 1.2 Master Key Derive Parameters

For TLS 1.2 master secret derivation (CKM_TLS12_MASTER_KEY_DERIVE):

```
typedef struct CK_SSL3_RANDOM_DATA {
    CK_BYTE_PTR pClientRandom; /* Client random data */
    CK_ULONG ulClientRandomLen; /* Length of client random data in bytes */
    CK_BYTE_PTR pServerRandom; /* Server random data */
    CK_ULONG ulServerRandomLen; /* Length of server random data in bytes */
} CK_SSL3_RANDOM_DATA;

typedef struct CK_TLS12_MASTER_KEY_DERIVE_PARAMS {
    CK_SSL3_RANDOM_DATA RandomInfo; /* Client and server random data */
    CK_VERSION_PTR pVersion;        /* TLS protocol version (output) */
    CK_MECHANISM_TYPE prfHashMechanism; /* PRF hash mechanism (e.g. CKM_SHA256) */
} CK_TLS12_MASTER_KEY_DERIVE_PARAMS;
```

6.3.8.3.4 TLS 1.2 Key and MAC Derive Parameters

For TLS 1.2 key and MAC derivation (CKM_TLS12_KEY_AND_MAC_DERIVE):

```
typedef struct CK_SSL3_KEY_MAT_OUT {
    CK_OBJECT_HANDLE hClientMacSecret; /* Client MAC secret key handle */
    CK_OBJECT_HANDLE hServerMacSecret; /* Server MAC secret key handle */
    CK_OBJECT_HANDLE hClientKey;       /* Client encryption key handle */
    CK_OBJECT_HANDLE hServerKey;       /* Server encryption key handle */
    CK_BYTE_PTR pIVClient;             /* Client IV */
    CK_BYTE_PTR pIVServer;            /* Server IV */
} CK_SSL3_KEY_MAT_OUT;

typedef struct CK_TLS12_KEY_MAT_PARAMS {
```

```

    CK_ULONG ulMacSizeInBits;           /* Length of MAC key in bits */
    /*
    CK_ULONG ulKeySizeInBits;           /* Length of encryption key in
    bits */
    CK_ULONG ulIVSizeInBits;           /* Length of IV in bits */
    CK_BBOOL bIsExport;                /* Must be FALSE (export not
    supported) */
    CK_SSL3_RANDOM_DATA RandomInfo;     /* Client and server random
    data */
    CK_SSL3_KEY_MAT_OUT_PTR pReturnedKeyMaterial; /* Output key material */
    CK_MECHANISM_TYPE prfHashMechanism; /* PRF hash mechanism (e.g.
    CKM_SHA256) */
} CK_TLS12_KEY_MAT_PARAMS;

```

6.3.8.3.5 TLS 1.2 Extended Master Key Derive Parameters

For TLS 1.2 extended master secret derivation (CKM_TLS12_EXTENDED_MASTER_KEY_DERIVE and CKM_TLS12_EXTENDED_MASTER_KEY_DERIVE_DH) as defined in [RFC 7627](#):

```

typedef struct CK_TLS12_EXTENDED_MASTER_KEY_DERIVE_PARAMS {
    CK_MECHANISM_TYPE prfHashMechanism; /* PRF hash mechanism (e.g. CKM_SHA256)
    */
    CK_BYTE_PTR pSessionHash;           /* Session hash as defined in RFC 7627
    */
    CK_ULONG ulSessionHashLen;         /* Length of session hash in bytes */
    CK_VERSION_PTR pVersion;           /* TLS protocol version (output) */
} CK_TLS12_EXTENDED_MASTER_KEY_DERIVE_PARAMS;

```

6.3.8.4 Symmetric Encryption Mechanisms

Table 207. Symmetric Encryption Mechanisms

Mechanism	Description
CKM_AES_ECB	AES in Electronic Codebook mode
CKM_AES_CBC	AES in Cipher Block Chaining mode
CKM_AES_CTR	AES in Counter mode
CKM_AES_CTS	AES in Cipher Text Stealing mode
CKM_AES_XTS	AES in XEX-based Tweaked CodeBook mode with ciphertext stealing
CKM_DES_ECB	DES in Electronic Codebook mode
CKM_DES_CBC	DES in Cipher Block Chaining mode
CKM_DES_CBC_PAD	DES in CBC mode with PKCS#7 padding
CKM_DES3_ECB	Triple DES in Electronic Codebook mode
CKM_DES3_CBC	Triple DES in Cipher Block Chaining mode
CKM_SM4_ECB	SM4 in Electronic Codebook mode
CKM_SM4_CBC	SM4 in Cipher Block Chaining mode
CKM_SM4_CTR	SM4 in Counter mode
CKM_AES_CFB128	Cipher Feedback mode
CKM_AES_OFB	Output Feedback mode

6.3.8.4.1 AES-CTR Parameters

For AES-CTR mode:

```
typedef struct CK_AES_CTR_PARAMS {
    CK_ULONG ulCounterBits;    /* Number of bits in counter block */
    CK_BYTE cb[16];           /* Counter block */
} CK_AES_CTR_PARAMS;
```

6.3.8.5 Asymmetric Encryption

Table 208. Asymmetric Encryption Mechanisms

Mechanism	Description
CKM_RSA_PKCS	RSA PKCS#1 v1.5 encryption
CKM_RSA_PKCS_OAEP	RSA PKCS#1 OAEP encryption
CKM_RSA_X_509	Raw RSA encryption (no padding)

6.3.8.5.1 RSA-OAEP Parameters

For RSA OAEP encryption:

```
typedef struct CK_RSA_PKCS_OAEP_PARAMS {
    CK_MECHANISM_TYPE hashAlg;    /* Hash algorithm */
    CK_RSA_PKCS_MGF_TYPE mgf;    /* Mask generation function */
    CK_RSA_PKCS_OAEP_SOURCE_TYPE source; /* Source of encoding parameter */
    CK_VOID_PTR pSourceData;    /* Encoding parameter */
    CK_ULONG ulSourceDataLen;    /* Length of encoding parameter */
} CK_RSA_PKCS_OAEP_PARAMS;
```

6.3.8.6 Authentication Encryption Mechanisms (AEAD)

Table 209. Authentication Encryption Mechanisms

Mechanism	Description
CKM_AES_GCM	AES in Galois/Counter Mode
CKM_AES_CCM	AES in Counter with CBC-MAC mode
CKM_CHACHA20_POLY1305	ChaCha20-Poly1305 AEAD encryption

6.3.8.6.1 AES-GCM Parameters

For AES-GCM authenticated encryption non-message encrypt and decrypt (CKM_AES_GCM):

```
typedef struct CK_GCM_PARAMS {
    CK_BYTE_PTR pIv;    /* Initialization vector */
    CK_ULONG ulIvLen;    /* Length of IV in bytes */
    CK_ULONG ulIvBits;    /* Length of IV in bits (if not byte-aligned) */
    CK_BYTE_PTR pAAD;    /* Additional authenticated data */
    CK_ULONG ulAADLen;    /* Length of AAD in bytes */
    CK_ULONG ulTagBits;    /* Length of authentication tag in bits */
} CK_GCM_PARAMS;
```

6.3.8.6.2 AES-CCM Parameters

For AES-CCM authenticated encryption non-message encrypt and decrypt (CKM_AES_CCM):

```
typedef struct CK_CCM_PARAMS {
    CK_ULONG ulDataLen;           /* Length of data to be encrypted */
    CK_BYTE_PTR pNonce;           /* Nonce */
    CK_ULONG ulNonceLen;          /* Length of nonce in bytes */
    CK_BYTE_PTR pAAD;             /* Additional authenticated data */
    CK_ULONG ulAADLen;            /* Length of AAD in bytes */
    CK_ULONG ulMACLen;            /* Length of MAC in bytes */
} CK_CCM_PARAMS;
```

6.3.8.6.3 AES-GCM Message Parameters

For message-based AES-GCM authenticated encryption (C_EncryptMessage, C_DecryptMessage, C_EncryptMessageBegin, C_DecryptMessageBegin, C_EncryptMessageNext, C_DecryptMessageNext):

```
typedef struct CK_GCM_MESSAGE_PARAMS {
    CK_BYTE_PTR pIv;              /* Initialization vector */
    CK_ULONG ulIvLen;             /* Length of IV in bytes */
    CK_ULONG ulIvFixedBits;       /* Number of fixed bits in the IV */
    CK_GENERATOR_FUNCTION ivGenerator; /* IV generator function */
    CK_BYTE_PTR pTag;             /* Authentication tag (output on
encrypt, input on decrypt) */
    CK_ULONG ulTagBits;           /* Length of authentication tag in
bits */
} CK_GCM_MESSAGE_PARAMS;
```

Where *ivGenerator* can be:

- CKG_NO_GENERATE: the caller supplies the full IV in pIv.
- CKG_GENERATE: the library generates the entire IV.
- CKG_GENERATE_COUNTER: the library generates the counter portion; ulIvFixedBits specifies how many high-order bits the caller provides in pIv.
- CKG_GENERATE_RANDOM: the library generates a random IV.

6.3.8.6.4 AES-CCM Message Parameters

For message-based AES-CCM authenticated encryption (C_EncryptMessage, C_DecryptMessage, C_EncryptMessageBegin, C_DecryptMessageBegin, C_EncryptMessageNext, C_DecryptMessageNext):

```
typedef struct CK_CCM_MESSAGE_PARAMS {
    CK_ULONG ulDataLen;           /* Length of plaintext/ciphertext
data in bytes */
    CK_BYTE_PTR pNonce;           /* Nonce */
    CK_ULONG ulNonceLen;          /* Length of nonce in bytes */
    CK_ULONG ulNonceFixedBits;    /* Number of fixed bits in the
nonce */
    CK_GENERATOR_FUNCTION nonceGenerator; /* Nonce generator function */
    CK_BYTE_PTR pMAC;             /* Authentication MAC (output on
encrypt, input on decrypt) */
    CK_ULONG ulMACLen;            /* Length of MAC in bytes */
} CK_CCM_MESSAGE_PARAMS;
```

Where *nonceGenerator* accepts the same values as *ivGenerator* in CK_GCM_MESSAGE_PARAMS (CKG_NO_GENERATE, CKG_GENERATE, CKG_GENERATE_COUNTER, CKG_GENERATE_RANDOM).

6.3.8.6.5 ChaCha20-Poly1305 Parameters

For single-part ChaCha20-Poly1305 authenticated encryption (CKM_CHACHA20_POLY1305):

```
typedef struct CK_SALSA20_CHACHA20_POLY1305_PARAMS {
    CK_BYTE_PTR pNonce;      /* Nonce/IV */
    CK_ULONG    ulNonceLen;  /* Length of nonce in bytes (8 or 12) */
    CK_BYTE_PTR pAAD;        /* Additional authenticated data */
    CK_ULONG    ulAADLen;    /* Length of AAD in bytes */
} CK_SALSA20_CHACHA20_POLY1305_PARAMS;
```

6.3.8.6.6 ChaCha20-Poly1305 Message Parameters

For message-based ChaCha20-Poly1305 authenticated encryption (C_EncryptMessage, C_DecryptMessage, C_EncryptMessageBegin, C_DecryptMessageBegin, C_EncryptMessageNext, C_DecryptMessageNext):

```
typedef struct CK_SALSA20_CHACHA20_POLY1305_MSG_PARAMS {
    CK_BYTE_PTR    pNonce;      /* Nonce/IV */
    CK_ULONG       ulNonceLen;  /* Length of nonce in bytes (8 or 12) */
    CK_GENERATOR_FUNCTION nonceGenerator; /* Nonce generator function */
    CK_BYTE_PTR    pTag;        /* Authentication tag (output on
    encrypt, input on decrypt) */
    CK_ULONG       ulTagBits;    /* Length of authentication tag in
    bits (must be 128) */
} CK_SALSA20_CHACHA20_POLY1305_MSG_PARAMS;
```

6.3.8.7 Asymmetric Signature Mechanisms

Table 210. Asymmetric Signature Mechanisms

Mechanism	Description
CKM_RSA_PKCS_PSS	RSA PSS signature of message already hashed. Limited to single part
CKM_SHA1_RSA_PKCS_PSS	RSA PSS signature hashing message with SHA-1
CKM_SHA224_RSA_PKCS_PSS	RSA PSS signature hashing message with SHA-224
CKM_SHA256_RSA_PKCS_PSS	RSA PSS signature hashing message with SHA-256
CKM_SHA384_RSA_PKCS_PSS	RSA PSS signature hashing message with SHA-384
CKM_SHA512_RSA_PKCS_PSS	RSA PSS signature hashing message with SHA-512
CKM_MD5_RSA_PKCS	RSA PKCS#1 v1.5 signature hashing message with MD5
CKM_SHA1_RSA_PKCS	RSA PKCS#1 v1.5 signature hashing message with SHA-1
CKM_SHA224_RSA_PKCS	RSA PKCS#1 v1.5 signature hashing message with SHA-224
CKM_SHA256_RSA_PKCS	RSA PKCS#1 v1.5 signature hashing message with SHA-256

Table 210. Asymmetric Signature Mechanisms...continued

Mechanism	Description
CKM_SHA384_RSA_PKCS	RSA PKCS#1 v1.5 signature hashing message with SHA-384
CKM_SHA512_RSA_PKCS	RSA PKCS#1 v1.5 signature hashing message with SHA-512
CKM_SHA3_224_RSA_PKCS	RSA PKCS#1 v1.5 signature hashing message with SHA3-224
CKM_SHA3_256_RSA_PKCS	RSA PKCS#1 v1.5 signature hashing message with SHA3-256
CKM_SHA3_384_RSA_PKCS	RSA PKCS#1 v1.5 signature hashing message with SHA3-384
CKM_SHA3_512_RSA_PKCS	RSA PKCS#1 v1.5 signature hashing message with SHA3-512
CKM_EDDSA	Edwards Curve signature. Supports optional parameters
CKM_ECDSA	ECDSA signature of message already hashed. Limited to single part
CKM_ECDSA_SHA1	ECDSA signature hashing message with SHA-1
CKM_ECDSA_SHA224	ECDSA signature hashing message with SHA-224
CKM_ECDSA_SHA256	ECDSA signature hashing message with SHA-256
CKM_ECDSA_SHA384	ECDSA signature hashing message with SHA-384
CKM_ECDSA_SHA512	ECDSA signature hashing message with SHA-512
CKM_ECDSA_SHA3_224	ECDSA signature hashing message with SHA3-224
CKM_ECDSA_SHA3_256	ECDSA signature hashing message with SHA3-256
CKM_ECDSA_SHA3_384	ECDSA signature hashing message with SHA3-384
CKM_ECDSA_SHA3_512	ECDSA signature hashing message with SHA3-512

6.3.8.7.1 RSA-PSS Parameters

For RSA PSS Signature:

```
typedef struct CK_RSA_PKCS_PSS_PARAMS {
    CK_MECHANISM_TYPE hashAlg; /* hash algorithm used in the PSS encoding */
    CK_RSA_PKCS_MGF_TYPE mgf; /* Mask generation function to use on the
    encoded block */
    CK_ULONG sLen; /* Length, in bytes, of the salt value used in
    the PSS encoding */
}
```

6.3.8.7.2 EdDSA Parameters

For Edwards Curve signatures with optional context:

```
typedef struct CK_EDDSA_PARAMS {
    CK_BYTE_PTR pPhflag; /* Indicates if Pre-hashed message */
    CK_ULONG ulContextDataLen; /* Length of context in bytes between 0 and
    255 */
}
```

```

    CK_BYTE_PTR pContext;          /* Context data for signature */
} CK_EdDSA_PARAMS;

```

6.3.8.8 Message Authentication Code (MAC)

Table 211. Message Authentication Code Mechanisms

Mechanism	Description
CKM_AES_CMAC_GENERAL	AES CMAC with variable output length
CKM_DES3_CMAC_GENERAL	Triple DES CMAC with variable output length
CKM_MD5_HMAC_GENERAL	HMAC with MD5 and variable output length
CKM_SHA_1_HMAC_GENERAL	HMAC with SHA-1 and variable output length
CKM_SHA224_HMAC_GENERAL	HMAC with SHA-224 and variable output length
CKM_SHA256_HMAC_GENERAL	HMAC with SHA-256 and variable output length
CKM_SHA384_HMAC_GENERAL	HMAC with SHA-384 and variable output length
CKM_SHA512_HMAC_GENERAL	HMAC with SHA-512 and variable output length
CKM_SHA3_224_HMAC_GENERAL	HMAC with SHA3-224 and variable output length
CKM_SHA3_256_HMAC_GENERAL	HMAC with SHA3-256 and variable output length
CKM_SHA3_384_HMAC_GENERAL	HMAC with SHA3-384 and variable output length
CKM_SHA3_512_HMAC_GENERAL	HMAC with SHA3-512 and variable output length
CKM_AES_CMAC	Full block size AES CMAC
CKM_DES3_CMAC	Full block size DES3 CMAC
CKM_MD5_HMAC	Full MD5 block HMAC
CKM_SHA_1_HMAC	Full SHA-1 block HMAC
CKM_SHA224_HMAC	Full SHA-224 block HMAC
CKM_SHA256_HMAC	Full SHA-256 block HMAC
CKM_SHA384_HMAC	Full SHA-384 block HMAC
CKM_SHA512_HMAC	Full SHA-512 block HMAC
CKM_SHA3_224_HMAC	Full SHA3-224 block HMAC
CKM_SHA3_256_HMAC	Full SHA3-256 block HMAC
CKM_SHA3_384_HMAC	Full SHA3-384 block HMAC
CKM_SHA3_512_HMAC	Full SHA3-512 block HMAC
CKM_TLS_MAC	TLS “finished” message MAC

6.3.9 Data types definitions

The purpose of this section is to document the data types used within the PKCS#11 API.

6.3.9.1 Basic Types

The basic types are fundamental data types based on the ANSI C standard types.

6.3.9.1.1 CK_BYTE

type **CK_BYTE**

Single unsigned byte value.

6.3.9.1.1.1 Definition

```
typedef unsigned char CK_BYTE;
```

6.3.9.1.1.2 Description

CK_BYTE is a single byte value used to represent an 8-bit unsigned integer.

6.3.9.1.2 CK_BYTE_PTR

type **CK_BYTE_PTR**

Pointer to a [CK_BYTE](#) value.

6.3.9.1.2.1 Definition

```
typedef CK_BYTE *CK_BYTE_PTR;
```

6.3.9.1.2.2 Description

CK_BYTE_PTR is a pointer type to a CK_BYTE value, used to reference byte data in function calls and data structures within the PKCS#11 API.

6.3.9.1.3 CK_CHAR

type **CK_CHAR**

Single unsigned character value.

6.3.9.1.3.1 Definition

```
typedef CK_BYTE CK_CHAR;
```

6.3.9.1.3.2 Description

CK_CHAR is a single character value used to represent an 8-bit unsigned character. It's a ANSI C character as listed in the following [Character Set](#) table.

Table 212. Character Set

Category	Characters
Letters	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z a b c d e f g h i j k l m n o p q r s t u v w x y z
Numbers	0 1 2 3 4 5 6 7 8 9
Graphic characters	! " # % & ' () * + , - . / : ; < = > ? [\] ^ _ { } ~

Table 212. Character Set...continued

Category	Characters
Blank character	‘ ‘

6.3.9.1.4 CK_CHAR_PTR

type **CK_CHAR_PTR**

Pointer to a [CK_CHAR](#) value.

6.3.9.1.4.1 Definition

```
typedef CK_CHAR *CK_CHAR_PTR;
```

6.3.9.1.4.2 Description

CK_CHAR_PTR is a pointer type to a CK_CHAR value, used to reference character data in function calls and data structures within the PKCS#11 API.

6.3.9.1.5 CK_UTF8CHAR

type **CK_UTF8CHAR**

Single UTF-8 character value.

6.3.9.1.5.1 Definition

```
typedef CK_BYTE CK_UTF8CHAR;
```

6.3.9.1.5.2 Description

CK_UTF8CHAR is a single UTF-8 character value used to represent an 8-bit unsigned character in UTF-8 encoding Unicode character as specified in the [RFC 2279](#).

6.3.9.1.6 CK_UTF8CHAR_PTR

type **CK_UTF8CHAR_PTR**

Pointer to a [CK_UTF8CHAR](#) value.

6.3.9.1.6.1 Definition

```
typedef CK_UTF8CHAR *CK_UTF8CHAR_PTR;
```

6.3.9.1.6.2 Description

CK_UTF8CHAR_PTR is a pointer type to a CK_UTF8CHAR value, used to reference UTF-8 character data in function calls and data structures within the PKCS#11 API.

6.3.9.1.7 CK_BBOOL

type **CK_BBOOL**

Boolean value.

6.3.9.1.7.1 Definition

```
typedef unsigned char CK_BBOOL;
```

6.3.9.1.7.2 Description

CK_BBOOL is a boolean value used to represent a true **CK_TRUE** or false **CK_FALSE** state, where a non-zero value is considered true and zero is considered false.

6.3.9.1.8 CK_ULONG

type **CK_ULONG**

Unsigned long integer value.

6.3.9.1.8.1 Definition

```
typedef unsigned long CK_ULONG;
```

6.3.9.1.8.2 Description

CK_ULONG is an unsigned long integer value used to represent non-negative integer values, commonly used for sizes, counts, and identifiers within the PKCS#11 API.

6.3.9.1.9 CK_ULONG_PTR

type **CK_ULONG_PTR**

Pointer to a [CK_ULONG](#) value.

6.3.9.1.9.1 Definition

```
typedef CK_ULONG *CK_ULONG_PTR;
```

6.3.9.1.9.2 Description

CK_ULONG_PTR is a pointer type to a **CK_ULONG** value, used to reference unsigned long integer data in function calls and data structures within the PKCS#11 API.

6.3.9.1.10 CK_LONG

type **CK_LONG**

Signed long integer value.

6.3.9.1.10.1 Definition

```
typedef long CK_LONG;
```

6.3.9.1.10.2 Description

CK_LONG is a signed long integer value used to represent both positive and negative integer values within the PKCS#11 API.

6.3.9.1.11 CK_FLAGS

type **CK_FLAGS**

Bit flags for various purposes.

6.3.9.1.11.1 Definition

```
typedef CK_ULONG CK_FLAGS;
```

6.3.9.1.11.2 Description

CK_FLAGS is a value used to provide bit flags indicating various capabilities or states. The meaning of specific flags depends on the context in which they are used.

6.3.9.1.12 CK_FLAGS_PTR

type **CK_FLAGS_PTR**

Pointer to a [CK_FLAGS](#)

6.3.9.1.12.1 Definition

```
typedef CK_FLAGS *CK_FLAGS_PTR;
```

6.3.9.1.12.2 Description

CK_FLAGS_PTR is a pointer type to a CK_FLAGS value, used to reference flags in function calls and data structures within the PKCS#11 API.

6.3.9.1.13 CK_VOID_PTR

type **CK_VOID_PTR**

Generic pointer type.

6.3.9.1.13.1 Definition

```
typedef void *CK_VOID_PTR;
```

6.3.9.1.13.2 Description

`CK_VOID_PTR` is a generic pointer type used to reference any type of data in function calls and data structures within the PKCS#11 API. It provides a way to pass pointers to different data types without explicit type casting.

6.3.9.1.14 CK_VOID_PTR_PTR

type `CK_VOID_PTR_PTR`

Pointer to a [CK_VOID_PTR](#) value.

6.3.9.1.14.1 Definition

```
typedef CK_VOID_PTR *CK_VOID_PTR_PTR;
```

6.3.9.1.14.2 Description

`CK_VOID_PTR_PTR` is a pointer type to a `CK_VOID_PTR` value, used to reference generic pointers in function calls and data structures within the PKCS#11 API.

6.3.9.1.15 NULL_PTR

`NULL_PTR`

Null pointer constant. Value is `NULL`.

6.3.9.1.15.1 Definition

```
#define NULL_PTR NULL
```

6.3.9.1.15.2 Description

`NULL_PTR` is a null pointer constant used to represent a null pointer value in function calls and data structures within the PKCS#11 API.

6.3.9.2 Constants

6.3.9.2.1 CK_TRUE

`CK_TRUE`

Boolean true constant. Value is 1.

6.3.9.2.2 CK_FALSE

`CK_FALSE`

Boolean false constant. Value is 0.

6.3.9.3 Locking-related types

Cryptoki provides some special types for use in a multi-threaded environment. These types are used to enable an application to use its own threading and locking primitives with Cryptoki.

6.3.9.3.1 CK_CREATEMUTEX

type **CK_CREATEMUTEX**

Function pointer type for creating a mutex object.

6.3.9.3.1.1 Definition

```
CK_CALLBACK_FUNCTION(CK_RV, CK_CREATEMUTEX) (  
    CK_VOID_PTR_PTR ppMutex  
) ;
```

6.3.9.3.1.2 Parameters

ppMutex

Pointer to location to receive pointer to new mutex.

6.3.9.3.1.3 Description

CK_CREATEMUTEX is a pointer to a function that creates and returns a new mutex object, storing a pointer to it in the location pointed to by **ppMutex**. If the function is unable to create a new mutex, it MUST return an error code such as **CKR_GENERAL_ERROR** or **CKR_HOST_MEMORY**.

6.3.9.3.2 CK_DESTROYMUTEX

type **CK_DESTROYMUTEX**

Function pointer type for destroying a mutex object.

6.3.9.3.2.1 Definition

```
CK_CALLBACK_FUNCTION(CK_RV, CK_DESTROYMUTEX) (  
    CK_VOID_PTR pMutex  
) ;
```

6.3.9.3.2.2 Parameters

pMutex

Pointer to mutex to be destroyed.

6.3.9.3.2.3 Description

CK_DESTROYMUTEX is a pointer to a function that destroys a mutex object. The mutex object is specified by the **pMutex** parameter. If the function is unable to destroy the mutex, it MUST return an error code such as **CKR_GENERAL_ERROR** or **CKR_MUTEX_BAD**.

6.3.9.3.3 CK_LOCKMUTEX

type **CK_LOCKMUTEX**

Function pointer type for locking a mutex.

6.3.9.3.3.1 Definition

```
CK_CALLBACK_FUNCTION(CK_RV, CK_LOCKMUTEX) (  
    CK_VOID_PTR pMutex  
) ;
```

6.3.9.3.3.2 Parameters

pMutex

Pointer to mutex to be locked.

6.3.9.3.3.3 Description

CK_LOCKMUTEX is a pointer to a function that locks a mutex. The mutex object is specified by the **pMutex** parameter. If the function is unable to lock the mutex, it MUST return an error code such as **CKR_GENERAL_ERROR** or **CKR_MUTEX_BAD**.

6.3.9.3.4 CK_UNLOCKMUTEX

type **CK_UNLOCKMUTEX**

Function pointer type for unlocking a mutex.

6.3.9.3.4.1 Definition

```
CK_CALLBACK_FUNCTION(CK_RV, CK_UNLOCKMUTEX) (  
    CK_VOID_PTR pMutex  
) ;
```

6.3.9.3.4.2 Parameters

pMutex

Pointer to mutex to be unlocked.

6.3.9.3.4.3 Description

CK_UNLOCKMUTEX is a pointer to a function that unlocks a mutex. The mutex object is specified by the **pMutex** parameter. If the function is unable to unlock the mutex, it MUST return an error code such as **CKR_GENERAL_ERROR** or **CKR_MUTEX_BAD**.

6.3.9.3.5 CK_C_INITIALIZE_ARGS

type **CK_C_INITIALIZE_ARGS**

Structure for C_Initialize function arguments.

6.3.9.3.5.1 Definition

```
typedef struct CK_C_INITIALIZE_ARGS {
    CK_CREATEMUTEX CreateMutex;
    CK_DESTROYMUTEX DestroyMutex;
    CK_LOCKMUTEX LockMutex;
    CK_UNLOCKMUTEX UnlockMutex;
    CK_FLAGS flags;
    CK_VOID_PTR pReserved;
} CK_C_INITIALIZE_ARGS;
```

6.3.9.3.5.2 Members

CreateMutex

Pointer to a function for creating a mutex.

DestroyMutex

Pointer to a function for destroying a mutex.

LockMutex

Pointer to a function for locking a mutex.

UnLockMutex

Pointer to a function for unlocking a mutex.

flags

bit flags specifying options for the [C_Initialize\(\)](#) function. See [C_Initialize parameter flags](#) table for the list of flags.

pReserved

Reserved for future use. Must be NULL_PTR.

6.3.9.3.5.3 Description

CK_C_INITIALIZE_ARGS is a structure used to pass initialization arguments to the [C_Initialize\(\)](#) function. It allows an application to provide custom mutex handling functions for multi-threaded environments. All function pointers may be set to NULL_PTR if the application does not require custom mutex handling.

Table 213. C_Initialize parameter flags

Name	Bit Mask	Description
CKF_LIBRARY_CANT_CREATE_OS_THREADS	0x00000001	If this bit is set, the library cannot create OS threads. The application must create threads on behalf of the library.
CKF_OS_LOCKING_OK	0x00000002	If this bit is set, the library can use OS-provided locking primitives. The application does not need to provide custom mutex handling functions.

6.3.9.3.6 CK_C_INITIALIZE_ARGS_PTR

type CK_C_INITIALIZE_ARGS_PTR

Pointer to a [CK_C_INITIALIZE_ARGS](#) structure.

6.3.9.3.6.1 Definition

```
typedef CK_C_INITIALIZE_ARGS CK_PTR CK_C_INITIALIZE_ARGS_PTR;
```

6.3.9.3.6.2 Description

`CK_C_INITIALIZE_ARGS_PTR` is a pointer type to a [CK_C_INITIALIZE_ARGS](#) structure. It is used to pass initialization arguments to the [C_Initialize\(\)](#) function.

6.3.9.4 General information

6.3.9.4.1 CK_VERSION

struct **CK_VERSION**

Structure that describes the version of Cryptoki.

6.3.9.4.1.1 Definition

```
typedef struct CK_VERSION {  
    CK_BYTE major;  
    CK_BYTE minor;  
} CK_VERSION;
```

6.3.9.4.1.2 Members

major

Major version number (the integer portion of the version).

minor

Minor version number (the hundredths portion of the version).

6.3.9.4.1.3 Description

`CK_VERSION` is a structure that describes the version of Cryptoki. For version 3.2, `major` would be 3 and `minor` would be 2. The version number is represented in binary-coded decimal format.

6.3.9.4.2 CK_VERSION_PTR

type **CK_VERSION_PTR**

Pointer to a [CK_VERSION](#) structure.

6.3.9.4.2.1 Definition

```
typedef CK_VERSION *CK_VERSION_PTR;
```

6.3.9.4.2.2 Description

`CK_VERSION_PTR` is a pointer type to a `CK_VERSION` structure, used to reference version information in function calls and data structures within the PKCS#11 API.

6.3.9.4.3 CK_INFO

type **CK_INFO**

Provides general information about Cryptoki.

6.3.9.4.3.1 Definition

```
typedef struct CK_INFO {  
    CK_VERSION cryptokiVersion;  
    CK_UTF8CHAR manufacturerID[32];  
    CK_FLAGS flags;  
    CK_UTF8CHAR libraryDescription[32];  
    CK_VERSION libraryVersion;  
} CK_INFO;
```

6.3.9.4.3.2 Members

cryptokiVersion

Cryptoki interface version number, for compatibility with future revisions.

manufacturerID

ID of the Cryptoki library manufacturer. Padded with blank characters (' '), and is not null-terminated.

flags

Bit flags reserved for future versions. Zero for this version.

libraryDescription

Character-string description of the library. Padded with blank characters (' '), and is not null-terminated.

libraryVersion

Cryptoki library version number.

6.3.9.4.3.3 Description

`CK_INFO` provides general information about the Cryptoki library. The `manufacturerID` and `libraryDescription` fields are fixed-length character arrays that are added with space characters (0x20) without null-terminated character. An application can use [C_GetInfo\(\)](#) to obtain this information.

6.3.9.4.4 CK_INFO_PTR

type **CK_INFO_PTR**

Pointer to a [CK_INFO](#) structure.

6.3.9.4.4.1 Definition

```
typedef CK_INFO *CK_INFO_PTR;
```

6.3.9.4.4.2 Description

`CK_INFO_PTR` is a pointer type to a [CK_INFO](#) structure. It is used to pass references to `CK_INFO` structures to Cryptoki functions.

6.3.9.4.5 CK_NOTIFICATION

type **CK_NOTIFICATION**

Notification event type for Cryptoki callbacks.

6.3.9.4.5.1 Definition

```
typedef CK_ULONG CK_NOTIFICATION;
```

6.3.9.4.5.2 Description

`CK_NOTIFICATION` is a typedef for notification event types used in Cryptoki callbacks. It is used to indicate various events that may occur during cryptographic operations.

Current version of Cryptoki defines the following notification type:

- `CKN_SURRENDER`; Cryptoki is surrendering the execution of a function executing in a session so that the application may perform other operations. After performing any desired operations, the application should indicate to Cryptoki whether to continue or cancel the function.

6.3.9.5 Slot and token types

6.3.9.5.1 CK_SLOT_ID

type **CK_SLOT_ID**

Identifier for a slot.

6.3.9.5.1.1 Definition

```
typedef CK_ULONG CK_SLOT_ID;
```

6.3.9.5.1.2 Description

`CK_SLOT_ID` is a unique identifier for a slot within a Cryptoki library. The list of available slots can be obtained by calling [C_GetSlotList\(\)](#). Each slot may contain a token, and slot IDs are used to identify which slot to perform operations on in various Cryptoki function calls.

6.3.9.5.2 CK_SLOT_ID_PTR

type **CK_SLOT_ID_PTR**

Pointer to a slot identifier [CK_SLOT_ID](#).

6.3.9.5.2.1 Definition

```
typedef CK_SLOT_ID *CK_SLOT_ID_PTR;
```

6.3.9.5.2.2 Description

CK_SLOT_ID_PTR is a pointer type to a CK_SLOT_ID identifier, used to reference slot identifiers in function calls and data structures within the PKCS#11 API.

6.3.9.5.3 CK_SLOT_INFO

type **CK_SLOT_INFO**
Provides information about a slot.

6.3.9.5.3.1 Definition

```
typedef struct CK_SLOT_INFO {
    CK_UTF8CHAR slotDescription[64];
    CK_UTF8CHAR manufacturerID[32];
    CK_FLAGS flags;
    CK_VERSION hardwareVersion;
    CK_VERSION firmwareVersion;
} CK_SLOT_INFO;
```

6.3.9.5.3.2 Members

- slotDescription**
Character-string description of the slot. Padded with blank characters (' '), and is not null-terminated.
- manufacturerID**
ID of the slot manufacturer. Padded with blank characters (' '), and is not null-terminated.
- flags**
Bit flags that provide information about the slot. The flags are listed in the [Slot Information Flags](#) table.
- hardwareVersion**
Hardware version number of the slot.
- firmwareVersion**
Firmware version number of the slot.

6.3.9.5.3.3 Description

CK_SLOT_INFO provides information about a specific slot in the Cryptoki library. The slotDescription and manufacturerID fields are fixed-length character arrays that are padded with space characters (0x20) without null-terminated character. An application can use C_GetSlotInfo() to obtain this information.

Table 214. Slot Information Flags

Flag	Value	Description
CKF_TOKEN_PRESENT	0x00000001	A token is present in the slot.
CKF_REMOVABLE_DEVICE	0x00000002	The slot is for a removable device.

Table 214. Slot Information Flags...continued

Flag	Value	Description
CKF_HW_SLOT	0x00000004	The slot is a hardware slot.

6.3.9.5.4 CK_SLOT_INFO_PTR

type **CK_SLOT_INFO_PTR**
Pointer to a [CK_SLOT_INFO](#) structure.

6.3.9.5.4.1 Definition

```
typedef CK_SLOT_INFO *CK_SLOT_INFO_PTR;
```

6.3.9.5.4.2 Description

CK_SLOT_INFO_PTR is a pointer type to a [CK_SLOT_INFO](#) structure. It is used to pass references to CK_SLOT_INFO structures to Cryptoki functions.

6.3.9.5.5 CK_TOKEN_INFO

type **CK_TOKEN_INFO**
Provides information about a token.

6.3.9.5.5.1 Definition

```
typedef struct CK_TOKEN_INFO {
    CK_UTF8CHAR label[32];
    CK_UTF8CHAR manufacturerID[32];
    CK_UTF8CHAR model[16];
    CK_CHAR serialNumber[16];
    CK_FLAGS flags;
    CK_ULONG ulMaxSessionCount;
    CK_ULONG ulSessionCount;
    CK_ULONG ulMaxRwSessionCount;
    CK_ULONG ulRwSessionCount;
    CK_ULONG ulMaxPinLen;
    CK_ULONG ulMinPinLen;
    CK_ULONG ulTotalPublicMemory;
    CK_ULONG ulFreePublicMemory;
    CK_ULONG ulTotalPrivateMemory;
    CK_ULONG ulFreePrivateMemory;
    CK_VERSION hardwareVersion;
    CK_VERSION firmwareVersion;
    CK_CHAR utcTime[16];
} CK_TOKEN_INFO;
```

6.3.9.5.5.2 Members

label
Application-defined label for the token. Padded with blank characters (' '), and is not null-terminated.

manufacturerID

ID of the device manufacturer. Padded with blank characters (' '), and is not null-terminated.

model

Character-string description of the device model. Padded with blank characters (' '), and is not null-terminated.

serialNumber

Character-string serial number of the device. Padded with blank characters (' '), and is not null-terminated.

flags

Bit flags that provide capabilities and status of the device as defined in the [Token Information Flags](#) table.

ulMaxSessionCount

Maximum number of sessions that can be opened with the token at one time by a single application.

ulSessionCount

Number of sessions that application currently has open with the token.

ulMaxRwSessionCount

Maximum number of read/write sessions that can be opened with the token at one time by a single application.

ulRwSessionCount

Number of read/write sessions that application currently has open with the token.

ulMaxPinLen

Maximum length in bytes for the PIN.

ulMinPinLen

Minimum length in bytes for the PIN.

ulTotalPublicMemory

Total amount of public memory on the token in bytes.

ulFreePublicMemory

Amount of free public memory on the token in bytes.

ulTotalPrivateMemory

Total amount of private memory on the token in bytes.

ulFreePrivateMemory

Amount of free private memory on the token in bytes.

hardwareVersion

Hardware version number.

firmwareVersion

Firmware version number.

utcTime

The current date and time in UTC. Formatted as a character string in the format YYYYMMDDhhmmss00 (year, month, day, hour, minute, second, and two trailing zeros). Not null-terminated.

6.3.9.5.5.3 Description

CK_TOKEN_INFO provides detailed information about a token in the Cryptoki library. An application can use [C_GetTokenInfo\(\)](#) to obtain this information.

Table 215. Token Information Flags

Flag	Value	Description
CKF_RNG	0x00000001	The token has a random number generator.
CKF_WRITE_PROTECTED	0x00000002	The token is write-protected.
CKF_LOGIN_REQUIRED	0x00000004	User must be logged to perform some cryptographic functions.
CKF_USER_PIN_INITIALIZED	0x00000008	The user PIN has been initialized.
CKF_RESTORE_KEY_NOT_NEEDED	0x00000020	Backup of session's cryptographic keys is not required.
CKF_CLOCK_ON_TOKEN	0x00000040	The token has its own hardware clock.
CKF_PROTECTED_AUTHENTICATION_PATH	0x00000100	The token has a protected authentication path. User does not need to login with a PIN to perform cryptographic functions.
CKF_DUAL_CRYPTO_OPERATIONS	0x00000200	The token can perform dual cryptographic operations.
CKF_TOKEN_INITIALIZED	0x00000400	The token has been initialized using the C_InitToken() function. Calling the C_InitToken() function with a token that is already initialized will cause the token to be re-initialized, clearing all stored objects.
CKF_USER_PIN_COUNT_LOW	0x00010000	Incorrect user login PIN has been entered at least once since the last successful authentication.
CK_USER_PIN_FINAL_TRY	0x00020000	Supplying an incorrect user PIN will cause it to become locked.
CKF_USER_PIN_LOCKED	0x00040000	The user PIN is locked.
CKF_USER_PIN_TO_BE_CHANGED	0x00080000	The user PIN must be changed.
CKF_SO_PIN_COUNT_LOW	0x00100000	Incorrect SO login PIN has been entered at least once since the last successful authentication.
CKF_SO_PIN_FINAL_TRY	0x00200000	Supplying an incorrect SO PIN will cause it to become locked.
CKF_SO_PIN_LOCKED	0x00400000	The SO PIN is locked.
CKF_SO_PIN_TO_BE_CHANGED	0x00800000	The SO PIN must be changed.
CKF_ERROR_STATE	0x01000000	The token failed a FIPS 140-2 self-test and entered an error state.
CKF_SEED_RANDOM_REQUIRED	0x02000000	The token random number generator requires seeding or re-seeding using C_SeedRandom() .
CKF_ASYNC_SESSION_SUPPORTED	0x04000000	The token supports asynchronous session operations.

6.3.9.5.6 CK_TOKEN_INFO_PTR

type **CK_TOKEN_INFO_PTR**

Pointer to [CK_TOKEN_INFO](#) structure.

6.3.9.5.6.1 Definition

```
typedef CK_TOKEN_INFO *CK_TOKEN_INFO_PTR;
```

6.3.9.5.6.2 Description

CK_TOKEN_INFO_PTR is a pointer type to the CK_TOKEN_INFO structure.

6.3.9.6 Session types

6.3.9.6.1 CK_SESSION_HANDLE

type **CK_SESSION_HANDLE**

Handle to a session object.

6.3.9.6.1.1 Definition

```
typedef CK_ULONG CK_SESSION_HANDLE;
```

6.3.9.6.1.2 Description

CK_SESSION_HANDLE is a handle to a session object in the Cryptoki library.

6.3.9.6.2 CK_SESSION_HANDLE_PTR

type **CK_SESSION_HANDLE_PTR**

Pointer to [CK_SESSION_HANDLE](#).

6.3.9.6.2.1 Definition

```
typedef CK_SESSION_HANDLE *CK_SESSION_HANDLE_PTR;
```

6.3.9.6.2.2 Description

CK_SESSION_HANDLE_PTR is a pointer type to the CK_SESSION_HANDLE type.

6.3.9.6.3 CK_USER_TYPE

type **CK_USER_TYPE**

Enumeration for user types.

6.3.9.6.3.1 Description

`CK_USER_TYPE` is an enumeration type for user types in the Cryptoki library.

It is used to specify the type of user logging into a token. The following user types are defined:

Table 216. User Types

User Type	Value	Description
CKU_SO	0x00000000	Security Officer user type.
CKU_USER	0x00000001	Normal user type.
CKU_CONTEXT_SPECIFIC	0x00000002	Context-specific user type.

6.3.9.6.4 CK_STATE

type **CK_STATE**

Enumeration for session states.

6.3.9.6.4.1 Definition

```
typedef CK_ULONG CK_STATE;
```

6.3.9.6.4.2 Description

`CK_STATE` is an enumeration type for session states in the Cryptoki library. It is used to specify the state of a session.

Table 217. Session States

Session State	Value	Description
CKS_RO_PUBLIC_SESSION	0x00000000	Read-only public session state.
CKS_RO_USER_FUNCTIONS	0x00000001	Read-only user functions session state.
CKS_RW_PUBLIC_SESSION	0x00000002	Read-write public session state.
CKS_RW_USER_FUNCTIONS	0x00000003	Read-write user functions session state.
CKS_RW_SO_FUNCTIONS	0x00000004	Read-write security officer functions session state.

6.3.9.6.5 CK_SESSION_INFO

type **CK_SESSION_INFO**

Structure containing information about a session.

6.3.9.6.5.1 Definition

```
typedef struct CK_SESSION_INFO {
    CK_SLOT_ID slotID;
    CK_STATE state;
    CK_FLAGS flags;
    CK_ULONG ulDeviceError;
```

```
} CK_SESSION_INFO;
```

6.3.9.6.5.2 Members

slotID

The ID of the slot that the session is open on.

state

The state of the session (see [Session States](#)).

flags

The flags set for the session (see [Session Flags](#)).

ulDeviceError

The error code define by the device. Used for errors not covered by Cryptoki.

6.3.9.6.5.3 Description

Provides information about a session.

`ulDeviceError` is a member of the `CK_SESSION_INFO` structure that contains device-specific error codes. This field is used to report errors that are not covered by the standard Cryptoki error codes.

Table 218. Session Flags

Session Flag	Value	Description
CKF_RW_SESSION	0x00000002	The session is read/write.
CKF_SERIAL_SESSION	0x00000004	This flag is provided for backward compatibility and should always be set.
CKF_ASYNC_SESSION	0x00000008	The session supports asynchronous operations.

6.3.9.6.6 CK_SESSION_INFO_PTR

type `CK_SESSION_INFO_PTR`

Pointer to [CK_SESSION_INFO](#) structure.

6.3.9.6.6.1 Definition

```
typedef CK_SESSION_INFO *CK_SESSION_INFO_PTR;
```

6.3.9.6.6.2 Description

`CK_SESSION_INFO_PTR` is a pointer type to the `CK_SESSION_INFO` structure.

6.3.9.7 Object types

6.3.9.7.1 CK_OBJECT_HANDLE

type `CK_OBJECT_HANDLE`

Handle to an object.

6.3.9.7.1.1 Definition

```
typedef CK_ULONG CK_OBJECT_HANDLE;
```

6.3.9.7.1.2 Description

CK_OBJECT_HANDLE is a handle to an object in the Cryptoki library.

6.3.9.7.2 CK_OBJECT_HANDLE_PTR

type **CK_OBJECT_HANDLE_PTR**

Pointer to [CK_OBJECT_HANDLE](#).

6.3.9.7.2.1 Definition

```
typedef CK_OBJECT_HANDLE *CK_OBJECT_HANDLE_PTR;
```

6.3.9.7.2.2 Description

CK_OBJECT_HANDLE_PTR is a pointer type to the CK_OBJECT_HANDLE type.

6.3.9.7.3 CK_OBJECT_CLASS

type **CK_OBJECT_CLASS**

Enumeration for object classes.

6.3.9.7.3.1 Definition

```
typedef CK_ULONG CK_OBJECT_CLASS;
```

6.3.9.7.3.2 Description

CK_OBJECT_CLASS is an enumeration type for object classes in the Cryptoki library. It is used to specify the class or type of an object stored on a token. The following object classes are defined:

Table 219. Object Classes

Object Class	Bit Mask	Description
CKO_DATA	0x00000000	Data object class
CKO_CERTIFICATE	0x00000001	Certificate object class.
CKO_PUBLIC_KEY	0x00000002	Public key object class.
CKO_PRIVATE_KEY	0x00000003	Private key object class.
CKO_SECRET_KEY	0x00000004	Secret key object class.
CKO_HW_FEATURE	0x00000005	Hardware feature object class.

Table 219. Object Classes...continued

Object Class	Bit Mask	Description
CKO_DOMAIN_PARAMETERS	0x00000006	Domain parameters object class.
CKO_MECHANISM	0x00000007	Mechanism object class.
CKO_OTP_KEY	0x00000008	OTP key object class.
CKO_PROFILE	0x00000009	Profile object class describes which PKCS#11 profiles the token implements.
CKO_VALIDATION	0x0000000A	Validation object class describes which third party validations the module conforms to.
CKO_TRUST	0x0000000B	Trust object class binds trusted usages to individual certificates.
CKO_VENDOR_DEFINED	0x80000000	Vendor-defined object class.

6.3.9.7.4 CK_OBJECT_CLASS_PTR

type **CK_OBJECT_CLASS_PTR**

Pointer to [CK_OBJECT_CLASS](#).

6.3.9.7.4.1 Definition

```
typedef CK_OBJECT_CLASS *CK_OBJECT_CLASS_PTR;
```

6.3.9.7.4.2 Description

CK_OBJECT_CLASS_PTR is a pointer type to the CK_OBJECT_CLASS type.

6.3.9.7.5 CK_HW_FEATURE_TYPE

type **CK_HW_FEATURE_TYPE**

Enumeration for hardware feature types.

6.3.9.7.5.1 Definition

```
typedef CK_ULONG CK_HW_FEATURE_TYPE;
```

6.3.9.7.5.2 Description

CK_HW_FEATURE_TYPE is an enumeration type for hardware feature types in the Cryptoki library. It is used to specify the type of a hardware feature object stored on a token. The following hardware feature types are defined:

Table 220. Hardware Feature Types

Hardware Feature Type	Bit Mask	Description
CKH_MONOTONIC_COUNTER	0x00000001	Monotonic counter hardware feature.

Table 220. Hardware Feature Types...continued

Hardware Feature Type	Bit Mask	Description
CKH_CLOCK	0x00000002	Clock hardware feature.
CKH_USER_INTERFACE	0x00000003	User interface hardware feature.
CKH_VENDOR_DEFINED	0x80000000	Vendor-defined hardware feature type.

6.3.9.7.6 CK_KEY_TYPE

type **CK_KEY_TYPE**

Identifies a key types.

6.3.9.7.6.1 Definition

```
typedef CK_ULONG CK_KEY_TYPE;
```

6.3.9.7.6.2 Description

CK_KEY_TYPE is an enumeration type for key types in the Cryptoki library. Key type constants are defined in the Cryptoki specification and can be extended by vendors by setting the high bit (CKK_VENDOR_DEFINED = 0x80000000).

6.3.9.7.7 CK_CERTIFICATE_TYPE

type **CK_CERTIFICATE_TYPE**

Identifies a certificate types.

6.3.9.7.7.1 Definition

```
typedef CK_ULONG CK_CERTIFICATE_TYPE;
```

6.3.9.7.7.2 Description

CK_CERTIFICATE_TYPE is an enumeration type for certificate types in the Cryptoki library. Certificate type constants are defined in the Cryptoki specification and can be extended by vendors by setting the high bit (CKC_VENDOR_DEFINED = 0x80000000).

6.3.9.7.8 CK_CERTIFICATE_CATEGORY

type **CK_CERTIFICATE_CATEGORY**

Identifies a certificate category.

6.3.9.7.8.1 Definition

```
typedef CK_ULONG CK_CERTIFICATE_CATEGORY;
```

6.3.9.7.8.2 Description

`CK_CERTIFICATE_CATEGORY` is an enumeration type for certificate categories in the Cryptoki library. The following categories are defined:

Table 221. Certificate Categories

Certificate Category	Value	Description
<code>CK_CERTIFICATE_CATEGORY_UNSPECIFIED</code>	0x00000000	Unspecified certificate category.
<code>CK_CERTIFICATE_CATEGORY_TOKEN_USER</code>	0x00000001	Token user certificate category.
<code>CK_CERTIFICATE_CATEGORY_AUTHORITY</code>	0x00000002	Certificate authority certificate category.
<code>CK_CERTIFICATE_CATEGORY_OTHER_ENTITY</code>	0x00000003	Other entity certificate category.

6.3.9.7.9 CK_ATTRIBUTE_TYPE

type **CK_ATTRIBUTE_TYPE**

Identifies an attribute type.

6.3.9.7.9.1 Definition

```
typedef CK_ULONG CK_ATTRIBUTE_TYPE;
```

6.3.9.7.9.2 Description

Attributes are defined with the objects and mechanisms that use them. Attributes are specified on an object as a list of type, length value items. These are often specified as an attribute template. Vendors can extend attribute types by setting the high bit (`CKA_VENDOR_DEFINED` = 0x80000000).

6.3.9.7.10 CK_ATTRIBUTE

type **CK_ATTRIBUTE**

Structure for specifying an attribute.

6.3.9.7.10.1 Definition

```
typedef struct CK_ATTRIBUTE {
    CK_ATTRIBUTE_TYPE type;
    CK_VOID_PTR pValue;
    CK_ULONG ulValueLen;
} CK_ATTRIBUTE;
```

6.3.9.7.10.2 Members

type

Attribute type.

pValue

Pointer to the attribute value.

ulValueLen

Length of the attribute value in bytes.

6.3.9.7.10.3 Description

CK_ATTRIBUTE is a structure used to specify an attribute of an object in the Cryptoki library. It contains the attribute type, a pointer to the attribute value, and the length of the attribute value. Attributes are used to define properties of objects such as keys, certificates, and other cryptographic objects.

If an attribute has no value, then ulValueLen = 0, and the value of pValue is irrelevant. An array of CK_ATTRIBUTE is called a “template” and is used for creating, manipulating and querying objects.

The constant CK_UNAVAILABLE_INFORMATION is used in the ulValueLen field to indicate that the attribute value is not available or not applicable for the object. See [C_GetAttributeValue\(\)](#).

6.3.9.7.11 CK_ATTRIBUTE_PTR

type CK_ATTRIBUTE_PTR

Pointer to [CK_ATTRIBUTE](#).

6.3.9.7.11.1 Definition

```
typedef CK_ATTRIBUTE *CK_ATTRIBUTE_PTR;
```

6.3.9.7.11.2 Description

CK_ATTRIBUTE_PTR is a pointer type to the CK_ATTRIBUTE structure.

6.3.9.7.12 CK_DATE

type CK_DATE

Structure for specifying a date.

6.3.9.7.12.1 Definition

```
typedef struct CK_DATE {  
    CK_CHAR year[4];  
    CK_CHAR month[2];  
    CK_CHAR day[2];  
} CK_DATE;
```

6.3.9.7.12.2 Members**year**

Year in YYYY format.

month

Month in MM format (01-12).

day

Day in DD format (01-31).

6.3.9.7.12.3 Description

`CK_DATE` is a structure used to represent a date in the Cryptoki library. The date is represented as three separate fields for year, month, and day, each stored as character arrays in ASCII format. This structure is commonly used in certificate validity periods and other time-based attributes.

6.3.9.7.13 CK_PROFILE_ID

type **CK_PROFILE_ID**

Identifies a profile type.

6.3.9.7.13.1 Definition

```
typedef CK_ULONG CK_PROFILE_ID;
```

6.3.9.7.13.2 Description

`CK_PROFILE_ID` is an enumeration type for profile identifiers in Cryptoki library. Profile IDs are used to identify specific profiles or configurations supported by a cryptographic device or token. Vendors can extend profile IDs by setting the high bit (`CKP_VENDOR_DEFINED = 0x80000000`).

6.3.9.7.14 CK_PROFILE_ID_PTR

type **CK_PROFILE_ID_PTR**

Pointer to [CK_PROFILE_ID](#).

6.3.9.7.14.1 Definition

```
typedef CK_PROFILE_ID *CK_PROFILE_ID_PTR;
```

6.3.9.7.14.2 Description

`CK_PROFILE_ID_PTR` is a pointer type to the `CK_PROFILE_ID` structure.

6.3.9.8 Data types for mechanisms

6.3.9.8.1 CK_MECHANISM_TYPE

type **CK_MECHANISM_TYPE**

Identifies a mechanism type.

6.3.9.8.1.1 Definition

```
typedef CK_ULONG CK_MECHANISM_TYPE;
```

6.3.9.8.1.2 Description

`CK_MECHANISM_TYPE` is an enumeration type for the mechanism identifier in the Cryptoki library. Mechanism types are used to identify the mechanism(s) supported by an object. Vendors can extend mechanism types by setting the high bit (`CKM_VENDOR_DEFINED = 0x80000000`).

6.3.9.8.2 CK_MECHANISM_TYPE_PTR

type **CK_MECHANISM_TYPE_PTR**

Pointer to [CK_MECHANISM_TYPE](#).

6.3.9.8.2.1 Definition

```
typedef CK_MECHANISM_TYPE *CK_MECHANISM_TYPE_PTR;
```

6.3.9.8.2.2 Description

`CK_MECHANISM_TYPE_PTR` is a pointer type to the `CK_MECHANISM_TYPE` structure.

6.3.9.8.3 CK_MECHANISM

type **CK_MECHANISM**

Structure for specifying a mechanism parameters.

6.3.9.8.3.1 Definition

```
typedef struct CK_MECHANISM {  
    CK_MECHANISM_TYPE mechanism;  
    CK_VOID_PTR pParameter;  
    CK_ULONG ulParameterLen;  
} CK_MECHANISM;
```

6.3.9.8.3.2 Members

mechanism

Mechanism type.

pParameter

Pointer to the mechanism parameters.

ulParameterLen

Length of the mechanism parameters in bytes.

6.3.9.8.3.3 Description

`CK_MECHANISM` is a structure used to specify a mechanism and its parameters in the Cryptoki library. It contains the mechanism type, a pointer to the mechanism parameters, and the length of the parameters. Mechanisms are used to specify cryptographic operations such as encryption, decryption, signing, and verification.

If a mechanism has no parameters, then `ulParameterLen` is set to 0.

6.3.9.8.4 CK_MECHANISM_PTR

type `CK_MECHANISM_PTR`

Pointer to [CK_MECHANISM](#).

6.3.9.8.4.1 Definition

```
typedef CK_MECHANISM *CK_MECHANISM_PTR;
```

6.3.9.8.4.2 Description

`CK_MECHANISM_PTR` is a pointer type to the `CK_MECHANISM` structure.

6.3.9.8.5 CK_MECHANIM_INFO

type `CK_MECHANIM_INFO`

Structure for specifying mechanism information.

6.3.9.8.5.1 Definition

```
typedef struct CK_MECHANISM_INFO {  
    CK_ULONG ulMinKeySize;  
    CK_ULONG ulMaxKeySize;  
    CK_FLAGS flags;  
} CK_MECHANISM_INFO;
```

6.3.9.8.5.2 Members

ulMinKeySize

Minimum key size in bits.

ulMaxKeySize

Maximum key size in bits.

flags

Mechanism capability flags.

6.3.9.8.5.3 Description

`CK_MECHANISM_INFO` is a structure used to specify information about a mechanism supported by a cryptographic device or token in the Cryptoki library. It contains the minimum and maximum key sizes supported

by the mechanism, as well as flags indicating the mechanism's capabilities such as encryption, decryption, signing, and verification.

Table 222. Mechanism information flags

Flag	Bit Mask	Description
CKF_HW	0x00000001	Mechanism is hardware-based.
CKF_MESSAGE_ENCRYPT	0x00000002	Mechanism can be used for message encryption.
CKF_MESSAGE_DECRYPT	0x00000004	Mechanism can be used for message decryption.
CKF_MESSAGE_SIGN	0x00000008	Mechanism can be used for signing.
CKF_MESSAGE_VERIFY	0x00000010	Mechanism can be used for verification.
CKF_MULTI_MESSAGE	0x00000020	Mechanism can be used in multipart operations.
CKF_FIND_OBJECTS	0x00000040	This flag can be passed in as a parameter to C_SessionCancel() to cancel an active object search operation. Any other use of this flag is outside the scope of this standard.
CKF_ENCRYPT	0x00000100	Mechanism can be used for encryption.
CKF_DECRYPT	0x00000200	Mechanism can be used for decryption.
CKF_DIGEST	0x00000400	Mechanism can be used for digesting.
CKF_SIGN	0x00000800	Mechanism can be used for signing.
CKF_SIGN_RECOVER	0x00001000	Mechanism can be used for signing with message recovery.
CKF_VERIFY	0x00002000	Mechanism can be used for verification.
CKF_VERIFY_RECOVER	0x00004000	Mechanism can be used for verification with message recovery.
CKF_GENERATE	0x00008000	Mechanism can be used for key generation.
CKF_GENERATE_KEY_PAIR	0x00010000	Mechanism can be used for key pair generation.
CKF_WRAP	0x00020000	Mechanism can be used for key wrapping.
CKF_UNWRAP	0x00040000	Mechanism can be used for key unwrapping.
CKF_DERIVE	0x00080000	Mechanism can be used for key derivation.
CKF_ENCAPSULATE	0x10000000	Mechanism can be used for key encapsulation.
CKF_DECAPSULATE	0x20000000	Mechanism can be used for key decapsulation.
CKF_EXTENSION	0x80000000	There is an extension to the flags. No used in this version, must not be set.

6.3.9.8.6 CK_MECHANISM_INFO_PTR

type **CK_MECHANISM_INFO_PTR**

Pointer to CK_MECHANISM_INFO.

6.3.9.8.6.1 Definition

```
typedef CK_MECHANISM_INFO *CK_MECHANISM_INFO_PTR;
```

6.3.9.8.6.2 Description

CK_MECHANISM_INFO_PTR is a pointer type to the CK_MECHANISM_INFO structure.

6.3.9.9 Function types

6.3.9.9.1 CK_RV

type **CK_RV**

Return value type for Cryptoki functions.

6.3.9.9.1.1 Definition

```
typedef CK_ULONG CK_RV;
```

6.3.9.9.1.2 Description

CK_RV is the return value type for Cryptoki functions. It is used to indicate the success or failure of a Cryptoki function call. A return value of CKR_OK indicates successful completion. Other return values indicate various error conditions.

6.3.9.9.2 CK_ASYNC_DATA

type **CK_ASYNC_DATA**

Structure for asynchronous operation data.

6.3.9.9.2.1 Definition

```
typedef struct CK_ASYNC_DATA {  
    CK_ULONG ulVersion;  
    CK_BYTE_PTR pValue;  
    CK_ULONG ulValue;  
    CK_OBJECT_HANDLE hObject;  
    CK_OBJECT_HANDLE hAdditionalObject;  
} CK_ASYNC_DATA;
```

6.3.9.9.2.2 Members

ulVersion

Version of the asynchronous data structure.

pValue

On completion contains a pointer to the original input buffer, caller is responsible for this memory.

ulValue

Size of the result.

hObject

Handle to the object being operated on.

hAdditionalObject

Handle to an additional object being operated on.

6.3.9.9.2.3 Description

CK_ASYNC_DATA is a structure used to hold asynchronous operation data in the Cryptoki library. It contains version information, a pointer to the result buffer, the size of the result, and handles to the objects being operated on. This structure is used when performing asynchronous cryptographic operations, allowing the caller to track the status and retrieve results of non-blocking function calls.

6.3.9.9.3 CK_ASYNC_DATA_PTR

type CK_ASYNC_DATA_PTR

Pointer to [CK_ASYNC_DATA](#).

6.3.9.9.3.1 Definition

```
typedef CK_ASYNC_DATA *CK_ASYNC_DATA_PTR;
```

6.3.9.9.3.2 Description

CK_ASYNC_DATA_PTR is a pointer type to the CK_ASYNC_DATA.

6.3.9.10 Library function list

6.3.9.10.1 CK_FUNCTION_LIST

type CK_FUNCTION_LIST

Structure containing function pointers to all Cryptoki API functions for PKCS#11 version 2.40. In this structure, version is the cryptoki specification version number. It is the default function list returned by [C_GetFunctionList\(\)](#). This function list may be returned via [C_GetInterfaceList\(\)](#) or [C_GetInterface\(\)](#).

6.3.9.10.1.1 Definition

```
typedef struct CK_FUNCTION_LIST {  
    CK_VERSION version;  
    CK_C_Initialize C_Initialize;  
    CK_C_Finalize C_Finalize;  
    CK_C_GetInfo C_GetInfo;  
    CK_C_GetFunctionList C_GetFunctionList;
```

```
CK_C_GetSlotList C_GetSlotList;
CK_C_GetSlotInfo C_GetSlotInfo;
CK_C_GetTokenInfo C_GetTokenInfo;
CK_C_GetMechanismList C_GetMechanismList;
CK_C_GetMechanismInfo C_GetMechanismInfo;
CK_C_InitToken C_InitToken;
CK_C_InitPIN C_InitPIN;
CK_C_SetPIN C_SetPIN;
CK_C_OpenSession C_OpenSession;
CK_C_CloseSession C_CloseSession;
CK_C_CloseAllSessions C_CloseAllSessions;
CK_C_GetSessionInfo C_GetSessionInfo;
CK_C_GetOperationState C_GetOperationState;
CK_C_SetOperationState C_SetOperationState;
CK_C_Login C_Login;
CK_C_Logout C_Logout;
CK_C_CreateObject C_CreateObject;
CK_C_CopyObject C_CopyObject;
CK_C_DestroyObject C_DestroyObject;
CK_C_GetObjectSize C_GetObjectSize;
CK_C_GetAttributeValue C_GetAttributeValue;
CK_C_SetAttributeValue C_SetAttributeValue;
CK_C_FindObjectsInit C_FindObjectsInit;
CK_C_FindObjects C_FindObjects;
CK_C_FindObjectsFinal C_FindObjectsFinal;
CK_C_EncryptInit C_EncryptInit;
CK_C_Encrypt C_Encrypt;
CK_C_EncryptUpdate C_EncryptUpdate;
CK_C_EncryptFinal C_EncryptFinal;
CK_C_DecryptInit C_DecryptInit;
CK_C_Decrypt C_Decrypt;
CK_C_DecryptUpdate C_DecryptUpdate;
CK_C_DecryptFinal C_DecryptFinal;
CK_C_DigestInit C_DigestInit;
CK_C_Digest C_Digest;
CK_C_DigestUpdate C_DigestUpdate;
CK_C_DigestKey C_DigestKey;
CK_C_DigestFinal C_DigestFinal;
CK_C_SignInit C_SignInit;
CK_C_Sign C_Sign;
CK_C_SignUpdate C_SignUpdate;
CK_C_SignFinal C_SignFinal;
CK_C_SignRecoverInit C_SignRecoverInit;
CK_C_SignRecover C_SignRecover;
CK_C_VerifyInit C_VerifyInit;
CK_C_Verify C_Verify;
CK_C_VerifyUpdate C_VerifyUpdate;
CK_C_VerifyFinal C_VerifyFinal;
CK_C_VerifyRecoverInit C_VerifyRecoverInit;
CK_C_VerifyRecover C_VerifyRecover;
CK_C_DigestEncryptUpdate C_DigestEncryptUpdate;
CK_C_DecryptDigestUpdate C_DecryptDigestUpdate;
CK_C_SignEncryptUpdate C_SignEncryptUpdate;
CK_C_DecryptVerifyUpdate C_DecryptVerifyUpdate;
CK_C_GenerateKey C_GenerateKey;
CK_C_GenerateKeyPair C_GenerateKeyPair;
CK_C_WrapKey C_WrapKey;
CK_C_UnwrapKey C_UnwrapKey;
CK_C_DeriveKey C_DeriveKey;
CK_C_SeedRandom C_SeedRandom;
```

```

    CK_C_GenerateRandom C_GenerateRandom;
    CK_C_GetFunctionStatus C_GetFunctionStatus;
    CK_C_CancelFunction C_CancelFunction;
    CK_C_WaitForSlotEvent C_WaitForSlotEvent;
} CK_FUNCTION_LIST;

```

6.3.9.10.2 CK_FUNCTION_LIST_PTR

type **CK_FUNCTION_LIST_PTR**

Pointer to a [CK_FUNCTION_LIST](#) structure.

6.3.9.10.2.1 Definition

```
typedef CK_FUNCTION_LIST *CK_FUNCTION_LIST_PTR;
```

6.3.9.10.2.2 Description

CK_FUNCTION_LIST_PTR is a pointer to a [CK_FUNCTION_LIST](#) structure, which contains function pointers to all the Cryptoki API functions. An application can obtain this pointer by calling [C_GetFunctionList\(\)](#).

6.3.9.10.3 CK_FUNCTION_LIST_PTR_PTR

type **CK_FUNCTION_LIST_PTR_PTR**

Pointer to a pointer to a [CK_FUNCTION_LIST](#) structure.

6.3.9.10.3.1 Definition

```
typedef CK_FUNCTION_LIST_PTR *CK_FUNCTION_LIST_PTR_PTR;
```

6.3.9.10.3.2 Description

CK_FUNCTION_LIST_PTR_PTR is a pointer to a pointer to a [CK_FUNCTION_LIST](#) structure. It is used to pass references to [CK_FUNCTION_LIST](#) pointers to Cryptoki functions, particularly in the [C_GetFunctionList\(\)](#) function which returns the function list through a pointer parameter.

6.3.9.10.4 CK_FUNCTION_LIST_3_2

type **CK_FUNCTION_LIST_3_2**

Structure containing function pointers to all Cryptoki API functions for PKCS#11 version 3.2. In this structure, version is the cryptoki specification version number. It should match the value of `cryptokiVersion` returned in the [CK_INFO](#) structure, but must be 3.2 at minimum.

This function list may be returned via [C_GetInterfaceList\(\)](#) or [C_GetInterface\(\)](#).

6.3.9.10.4.1 Definition

```

typedef struct CK_FUNCTION_LIST_3_2 {
    CK_VERSION version;
    CK_C_Initialize C_Initialize;
    CK_C_Finalize C_Finalize;

```

```
CK_C_GetInfo C_GetInfo;
CK_C_GetFunctionList C_GetFunctionList;
CK_C_GetSlotList C_GetSlotList;
CK_C_GetSlotInfo C_GetSlotInfo;
CK_C_GetTokenInfo C_GetTokenInfo;
CK_C_GetMechanismList C_GetMechanismList;
CK_C_GetMechanismInfo C_GetMechanismInfo;
CK_C_InitToken C_InitToken;
CK_C_InitPIN C_InitPIN;
CK_C_SetPIN C_SetPIN;
CK_C_OpenSession C_OpenSession;
CK_C_CloseSession C_CloseSession;
CK_C_CloseAllSessions C_CloseAllSessions;
CK_C_GetSessionInfo C_GetSessionInfo;
CK_C_GetOperationState C_GetOperationState;
CK_C_SetOperationState C_SetOperationState;
CK_C_Login C_Login;
CK_C_Logout C_Logout;
CK_C_CreateObject C_CreateObject;
CK_C_CopyObject C_CopyObject;
CK_C_DestroyObject C_DestroyObject;
CK_C_GetObjectSize C_GetObjectSize;
CK_C_GetAttributeValue C_GetAttributeValue;
CK_C_SetAttributeValue C_SetAttributeValue;
CK_C_FindObjectsInit C_FindObjectsInit;
CK_C_FindObjects C_FindObjects;
CK_C_FindObjectsFinal C_FindObjectsFinal;
CK_C_EncryptInit C_EncryptInit;
CK_C_Encrypt C_Encrypt;
CK_C_EncryptUpdate C_EncryptUpdate;
CK_C_EncryptFinal C_EncryptFinal;
CK_C_DecryptInit C_DecryptInit;
CK_C_Decrypt C_Decrypt;
CK_C_DecryptUpdate C_DecryptUpdate;
CK_C_DecryptFinal C_DecryptFinal;
CK_C_DigestInit C_DigestInit;
CK_C_Digest C_Digest;
CK_C_DigestUpdate C_DigestUpdate;
CK_C_DigestKey C_DigestKey;
CK_C_DigestFinal C_DigestFinal;
CK_C_SignInit C_SignInit;
CK_C_Sign C_Sign;
CK_C_SignUpdate C_SignUpdate;
CK_C_SignFinal C_SignFinal;
CK_C_SignRecoverInit C_SignRecoverInit;
CK_C_SignRecover C_SignRecover;
CK_C_VerifyInit C_VerifyInit;
CK_C_Verify C_Verify;
CK_C_VerifyUpdate C_VerifyUpdate;
CK_C_VerifyFinal C_VerifyFinal;
CK_C_VerifyRecoverInit C_VerifyRecoverInit;
CK_C_VerifyRecover C_VerifyRecover;
CK_C_DigestEncryptUpdate C_DigestEncryptUpdate;
CK_C_DecryptDigestUpdate C_DecryptDigestUpdate;
CK_C_SignEncryptUpdate C_SignEncryptUpdate;
CK_C_DecryptVerifyUpdate C_DecryptVerifyUpdate;
CK_C_GenerateKey C_GenerateKey;
CK_C_GenerateKeyPair C_GenerateKeyPair;
CK_C_WrapKey C_WrapKey;
CK_C_UnwrapKey C_UnwrapKey;
```



```

CK_C_DeriveKey C_DeriveKey;
CK_C_SeedRandom C_SeedRandom;
CK_C_GenerateRandom C_GenerateRandom;
CK_C_GetFunctionStatus C_GetFunctionStatus;
CK_C_CancelFunction C_CancelFunction;
CK_C_WaitForSlotEvent C_WaitForSlotEvent;
CK_C_GetInterfaceList C_GetInterfaceList;
CK_C_GetInterface C_GetInterface;
CK_C_LoginUser C_LoginUser;
CK_C_SessionCancel C_SessionCancel;
CK_C_MessageEncryptInit C_MessageEncryptInit;
CK_C_EncryptMessage C_EncryptMessage;
CK_C_EncryptMessageBegin C_EncryptMessageBegin;
CK_C_EncryptMessageNext C_EncryptMessageNext;
CK_C_MessageEncryptFinal C_MessageEncryptFinal;
CK_C_MessageDecryptInit C_MessageDecryptInit;
CK_C_DecryptMessage C_DecryptMessage;
CK_C_DecryptMessageBegin C_DecryptMessageBegin;
CK_C_DecryptMessageNext C_DecryptMessageNext;
CK_C_MessageDecryptFinal C_MessageDecryptFinal;
CK_C_MessageSignInit C_MessageSignInit;
CK_C_SignMessage C_SignMessage;
CK_C_SignMessageBegin C_SignMessageBegin;
CK_C_SignMessageNext C_SignMessageNext;
CK_C_MessageSignFinal C_MessageSignFinal;
CK_C_MessageVerifyInit C_MessageVerifyInit;
CK_C_VerifyMessage C_VerifyMessage;
CK_C_VerifyMessageBegin C_VerifyMessageBegin;
CK_C_VerifyMessageNext C_VerifyMessageNext;
CK_C_MessageVerifyFinal C_MessageVerifyFinal;
CK_C_EncapsulateKey C_EncapsulateKey;
CK_C_DecapsulateKey C_DecapsulateKey;
CK_C_VerifySignatureInit C_VerifySignatureInit;
CK_C_VerifySignature C_VerifySignature;
CK_C_VerifySignatureUpdate C_VerifySignatureUpdate;
CK_C_VerifySignatureFinal C_VerifySignatureFinal;
CK_C_GetSessionValidationFlags C_GetSessionValidationFlags;
CK_C_AsyncComplete C_AsyncComplete;
CK_C_AsyncGetID C_AsyncGetID;
CK_C_AsyncJoin C_AsyncJoin;
CK_C_WrapKeyAuthenticated C_WrapKeyAuthenticated;
CK_C_UnwrapKeyAuthenticated C_UnwrapKeyAuthenticated;
} CK_FUNCTION_LIST_3_2;

```

6.3.9.10.5 CK_FUNCTION_LIST_3_2_PTR

type **CK_FUNCTION_LIST_3_2_PTR**

Pointer to a [CK_FUNCTION_LIST_3_2](#) structure.

6.3.9.10.5.1 Definition

```
typedef CK_FUNCTION_LIST_3_0 *CK_FUNCTION_LIST_3_2_PTR;
```

6.3.9.10.5.2 Description

Pointer type for accessing a [CK_FUNCTION_LIST_3_2](#) structure. This pointer type is used throughout the PKCS #11 API to reference the function list containing all available cryptographic operations and token management functions provided by a PKCS #11 implementation.

6.3.9.10.6 CK_FUNCTION_LIST_3_2_PTR_PTR

type **CK_FUNCTION_LIST_3_2_PTR_PTR**

Pointer to a pointer to a [CK_FUNCTION_LIST_3_2](#) structure.

6.3.9.10.6.1 Definition

```
typedef CK_FUNCTION_LIST_3_0 **CK_FUNCTION_LIST_3_2_PTR_PTR;
```

6.3.9.10.6.2 Description

Pointer to a pointer to a [CK_FUNCTION_LIST_3_2](#) structure. This double pointer type is used to pass references to function list pointers through the PKCS #11 API, enabling dynamic allocation and manipulation of function list structures at the application level.

6.3.9.11 Library interface

6.3.9.11.1 CK_INTERFACE

type **CK_INTERFACE**

Describes a Cryptoki interface.

6.3.9.11.1.1 Definition

```
typedef struct CK_INTERFACE {  
    CK_CHAR *pInterfaceName;  
    CK_VOID_PTR pFunctionList;  
    CK_FLAGS flags;  
} CK_INTERFACE;
```

6.3.9.11.1.2 Members

pInterfaceName

Pointer to a null-terminated string identifying the interface name. For the default PKCS#11 interface, this is "PKCS 11".

pFunctionList

Pointer to the function list for this interface. For the default interface, this points to a [CK_FUNCTION_LIST](#) structure.

flags

Flags describing capabilities or properties of the interface. Must be zero for this version. See [CK_INTERFACE Flags Descriptions](#) table for details.

6.3.9.11.1.3 Description

`CK_INTERFACE` describes a Cryptoki interface. The structure is used by `C_GetInterface()` to return information about available interfaces. The default PKCS#11 interface has the name “PKCS 11” and provides a `CK_FUNCTION_LIST` structure containing function pointers to all standard Cryptoki functions.

The `pInterfaceName` field points to a null-terminated string, while the `pFunctionList` field points to the appropriate function list structure for the interface. Applications can use `C_GetInterfaceList()` to discover available interfaces and `C_GetInterface()` to obtain a specific interface.

Table 223. `CK_INTERFACE` Flags Descriptions

Bit Flag	Value	Description
<code>CK_INTERFACE_FORK_SAFE</code>	0x00000001	The returned interface will have fork tolerant semantics. When the application forks, each process will get its own copy of all session objects, session states, login states, and encryption states. Each process will also maintain access to token objects with their previously supplied handles.

6.3.9.11.2 `CK_INTERFACE_PTR`

type `CK_INTERFACE_PTR`

Pointer to a `CK_INTERFACE` structure.

6.3.9.11.2.1 Definition

```
typedef CK_INTERFACE *CK_INTERFACE_PTR;
```

6.3.9.11.2.2 Description

Pointer type for accessing a `CK_INTERFACE` structure.

6.3.9.11.3 `CK_INTERFACE_PTR_PTR`

type `CK_INTERFACE_PTR_PTR`

Pointer to a pointer to a `CK_INTERFACE` structure.

6.3.9.11.3.1 Definition

```
typedef CK_INTERFACE **CK_INTERFACE_PTR_PTR;
```

6.3.9.11.3.2 Description

Pointer to a pointer to a `CK_INTERFACE` structure.

6.3.9.12 Session validation flags

6.3.9.12.1 CK_SESSION_VALIDATION_FLAGS_TYPE

type **CK_SESSION_VALIDATION_FLAGS_TYPE**
Flags for session validation operations.

6.3.9.12.1.1 Definition

```
typedef CK_ULONG CK_SESSION_VALIDATION_FLAGS_TYPE;
```

6.3.9.12.1.2 Description

CK_SESSION_VALIDATION_FLAGS_TYPE is a type alias for **CK_ULONG** used to represent flags that control session validation behavior in PKCS#11 operations.

These flags are used with functions like [C_GetSessionValidationFlags\(\)](#) to query and manage session validation properties.

Table 224. Session validation flags

Bit Flag	Value	Description
CKS_LAST_VALIDATION_OK	0x00000001	Last operation that completed met all the requirements of a validated mechanism. This allows access to the state of operations that don't return a key object.

Legal information

Definitions

Draft — A draft status on a document indicates that the content is still under internal review and subject to formal approval, which may result in modifications or additions. NXP Semiconductors does not give any representations or warranties as to the accuracy or completeness of information included in a draft version of a document and shall have no liability for the consequences of use of such information.

Disclaimers

Limited warranty and liability — Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes — NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use — NXP Semiconductors products are not designed, authorized or warranted to be suitable for use in life support, life-critical or safety-critical systems or equipment, nor in applications where failure or malfunction of an NXP Semiconductors product can reasonably be expected to result in personal injury, death or severe property or environmental damage. NXP Semiconductors and its suppliers accept no liability for inclusion and/or use of NXP Semiconductors products in such equipment or applications and therefore such inclusion and/or use is at the customer's own risk.

Applications — Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale — NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <https://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control — This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Suitability for use in non-automotive qualified products — Unless this document expressly states that this specific NXP Semiconductors product is automotive qualified, the product is not suitable for automotive use. It is neither qualified nor tested in accordance with automotive testing or application requirements. NXP Semiconductors accepts no liability for inclusion and/or use of non-automotive qualified products in automotive equipment or applications.

In the event that customer uses the product for design-in and use in automotive applications to automotive specifications and standards, customer (a) shall use the product without NXP Semiconductors' warranty of the product for such automotive applications, use and specifications, and (b) whenever customer uses the product for automotive applications beyond NXP Semiconductors' specifications such use shall be solely at customer's own risk, and (c) customer fully indemnifies NXP Semiconductors for any liability, damages or failed product claims resulting from customer design and use of the product for automotive applications beyond NXP Semiconductors' standard warranty and NXP Semiconductors' product specifications.

HTML publications — An HTML version, if available, of this document is provided as a courtesy. Definitive information is contained in the applicable document in PDF format. If there is a discrepancy between the HTML document and the PDF document, the PDF document has priority.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately. Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at PSIRT@nxp.com) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. — NXP B.V. is not an operating company and it does not distribute or sell products.

Trademarks

Notice: All referenced brands, product names, service names, and trademarks are the property of their respective owners.

NXP — wordmark and logo are trademarks of NXP B.V.

Contents

1	Introduction	2	5.4.2	API Algorithm Identifiers	106
2	Library Configuration	3	5.5	Message Authentication Code (MAC)	107
2.1	Library Configuration	3	5.5.1	Supported Operations versus Subsystems	108
2.2	Subsystems Configuration	3	5.5.2	HMAC	108
2.2.1	Syntactic rules	3	5.5.3	CMAC	111
2.2.2	Global configuration	4	5.6	Symmetric Encryption	113
2.2.3	Secure Subsystems definition	4	5.6.1	Supported Operations versus Subsystems	113
2.2.4	Security Operations definition	5	5.6.2	AES Symmetric Encryption	115
2.2.5	Naming Convention	5	5.6.3	DES Symmetric Encryption	117
2.2.6	Example	12	5.6.4	Triple-DES (DES3) Symmetric Encryption	118
3	OSAL	16	5.6.5	SM4 Symmetric Encryption	120
3.1	Core library interface	16	5.7	Asymmetric Encryption	121
3.1.1	Object database	16	5.7.1	Supported Operations versus Subsystems	121
3.1.2	OSAL APIs	17	5.7.2	RSA PKCS#1 v1.5 Asymmetric Encryption	122
3.1.3	OSAL operations	18	5.7.3	RSA OAEP Asymmetric Encryption	123
3.2	OS-specific implementation	33	5.7.4	RSA No Padding Asymmetric Encryption	124
3.2.1	Library initialization methods	33	5.8	Asymmetric Signature	125
3.2.2	OSAL APIs	35	5.8.1	Supported Operations versus Subsystems	125
3.2.3	Linux object database implementation	39	5.8.2	ECDSA Signatures	126
3.2.4	Example	39	5.8.3	EdDSA Signatures	128
4	Subsystems Overview	41	5.8.4	RSA PKCS#1 v1.5 Signatures	129
4.1	TEE Subsystem	41	5.8.5	RSA PSS Signatures	131
4.2	ELE Subsystem	41	5.8.6	SM2 Signatures	132
4.2.1	EdgeLock Accelerator (ELA)	42	5.9	Authentication Encryption with Associated Data (AEAD)	133
4.3	SECO Subsystem	42	5.9.1	Supported Operations versus Subsystems	133
5	Secure Subsystems Features and Capabilities	43	5.9.2	AES AEAD	135
5.1	Key and Data Storage	43	5.10	Device Management	137
5.1.1	Overview	43	5.10.1	Supported Operations versus Subsystems	137
5.1.2	Storage Types	43	5.10.2	Device Attestation	137
5.1.3	Subsystem Storage Support	43	5.10.3	Get Device UUID	140
5.1.4	Configuration	43	5.10.4	Device Lifecycle	141
5.1.5	Bad Practice	46	5.10.5	Device information	141
5.2	Key Management	47	6	APIs reference	142
5.2.1	Supported Operations versus Subsystems	47	6.1	SMW APIs	142
5.2.2	Asymmetric Keys	47	6.1.1	Library information APIs	142
5.2.3	Symmetric Keys	49	6.1.2	Configuration APIs	143
5.2.4	Generate Key	50	6.1.3	Key Manager APIs	156
5.2.5	Delete Key	52	6.1.4	Cryptography APIs	190
5.2.6	Import Key	52	6.1.5	Device management APIs	241
5.2.7	Export Public Key	64	6.1.6	Data Storage APIs	250
5.2.8	Key Derivation	65	6.1.7	Object Database APIs	258
5.2.9	Key Agreement	65	6.1.8	Status APIs return codes	264
5.2.10	TLS 1.2 (PRF)	67	6.1.9	Cryptographic Algorithm (smw_attr_algo_t) encoding	272
5.2.11	TLS 1.3 (TLS13-KDF)	80	6.1.10	Objects Attributes definitions	296
5.2.12	Get key attributes	99	6.1.11	Names type definitions	312
5.2.13	Public Key Attestation	100	6.2	PSA APIs	320
5.3	Data Management	101	6.2.1	Initialization	320
5.3.1	Supported Operations versus Subsystems	102	6.2.2	Key Manager APIs	320
5.3.2	Data Size Limits	102	6.2.3	Cryptography APIs	364
5.3.3	Store Data	102	6.2.4	Attestation APIs	457
5.3.4	Retrieve Data	104	6.2.5	Storage APIs	459
5.3.5	Delete Data	104	6.2.6	Return codes	470
5.3.6	Get Information	104	6.2.7	Objects Attributes definitions	473
5.4	Digest (Hash)	105			
5.4.1	Supported Operations versus Subsystems	105			

6.2.8	Cryptographic Algorithm (psa_algorithm_t)	
	encoding	478
6.2.9	Key type encoding	502
6.3	PKCS11 APIs	524
6.3.1	General Purpose	524
6.3.2	Slot and Token Management	531
6.3.3	Session Management	542
6.3.4	Key Management	552
6.3.5	Object Management	562
6.3.6	Cryptography APIs	573
6.3.7	Key Types	640
6.3.8	Mechanisms	640
6.3.9	Data types definitions	648
	Legal information	685

Please be aware that important notices concerning this document and the product(s) described herein, have been included in section 'Legal information'.
